ASIC & FPGA Chip Design:

Introduction

Mahdi Shabany

Department of Electrical Engineering

Sharif University of technology

Course Code: 25776

Course Outline

- Course Outline
- Introduction to IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

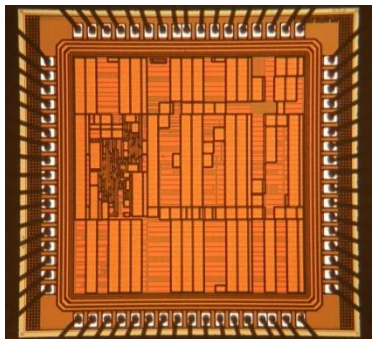
Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Course Description

- How to go from Idea/Algorithm to the actual hardware

```
module RippleCarryAdderII (Cin, X, Y, S, Cout);
   parameter n = 4;  input  Cin;  input [n-1:0] X, Y;  output  [n-1:0] S;
   wire [n-1:0] C;
    Full_Adder stage0  (Cin, X[0], Y[0], S[0], C[1]);
   Full_Adder stage1  (C[1], X[1], Y[1], S[1], C[2]);
   Full_Adder stage2  (C[2], X[2], Y[2], S[2], C[3]);
   Full_Adder stage3  (.Cout(Cout), .Cin(C[3]), .x(X[3]), .y(Y[3]), .S(S[3]));
endmodule
```



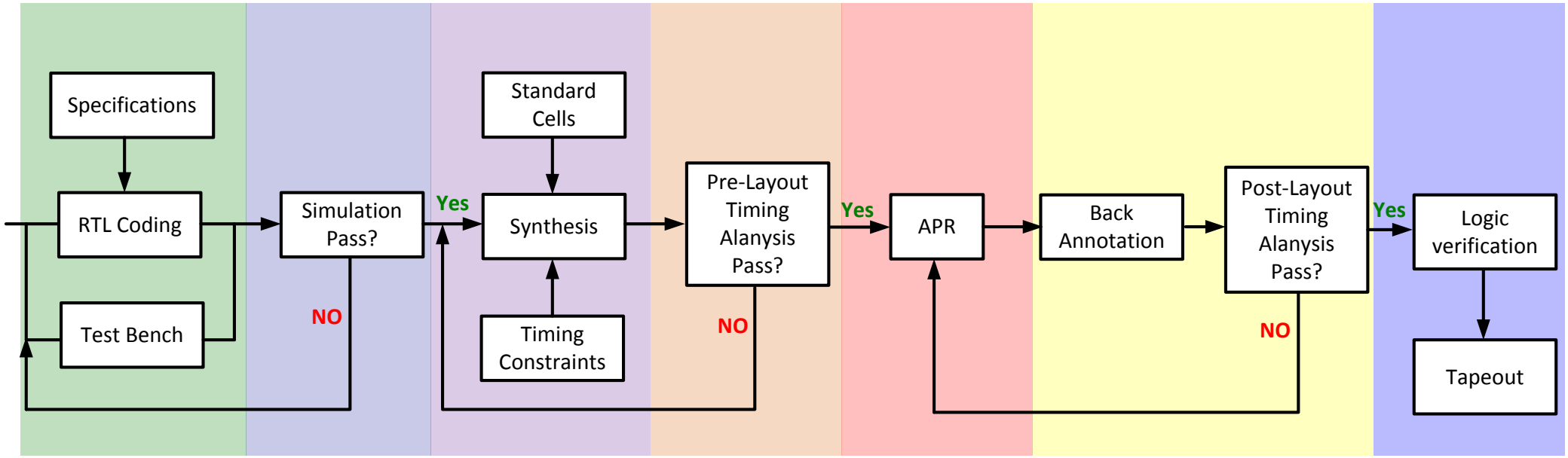
ASIC

FPGA

Course Description

- Hardware Description Language (HDL) :Verilog
- Professional Verilog Coding for Synthesis
- Verification Techniques
- FPGA Architectures
- Digital System Design with Xilinx FPGAs **(Workshop)**
- ASIC Digital Design Flow (from Verilog to the actual Chip!)
- Synthesis Algorithms
- Power Dissipation
- Power Grid and Clock Design
- Fixed-point Simulation Methodology
- Xilinx Core Generator **(Workshop)**
- Xilinx System Generator **(Workshop)**

Course Description:



1. HDL Coding **2. Simulation** **3. Synthesis** — **Front-End**

4. Placement & routing **5. Timing Analysis & Verification** — **Back-End**

- In this course we learn all the above steps in detail for
  - ASIC Platform
  - FPGA Platform

Course Description:

❑ Hardware Description Language

➢ Verilog Fundamentals

- Language Fundamentals
- Modeling Combinational/Sequential Logic Circuits
- Modeling Finite State Machines

➢ Verilog for Verification

- Verification/Simulation techniques with test-benches

➢ Verilog for synthesis

- Verilog Styles for Synthesis
- Architectural techniques for high-speed designs
  - Parallel proc., pipelining, retiming, …
- Implementations of common operations
  - Complex multiplication, division, complex norm, CORDIC
- Fundamentals of fixed-point realization

Course Description:

❑ PLDs & FPGA Architectures

- **FPGA Technologies**
  - SPLDs (PAL and PLA architectures)
  - Commercial CPLD Architectures
  - SRAM/LUT Based FPGAs
  - Anti-fuse/MUX Based FPGAs
  - Flash Based FPGAs
- **FPGA Architectures**
  - Heterogeneous/Homogeneous FPGAs
  - Fine-grained, coarse-grained and platform FPGAs
- **FPGA Elements & Design Trade-offs**
  - Logic Cells Common Architectures
  - Programmable Routing Channels Design
  - I/O & Pad architectures
- **Commercial FPGAs**
  - Altera (FLEX 10K, Stratix III) , Xilinx (XC4000, Virtex II,4,5), Actel (Act3, Axcelerator)

Course Description:

❑ Advanced Digital System Design with Xilinx FPGAs

- Design Creation
- Synthesize
- Simulation
- Constraints Entry
- Implementation
- Implementation Results Analysis – Timing Analysis
- Implementation Results Analysis – Power Analysis
- Implementation Results Improvement
- Device Configuration and Programming
- Design Debugging

Course Description:

❑ Core Generator

- CORE Generator Tool
- Intellectual Property (IP) Cores
- CORE Generator Tool files
- Design Flows
- Defining Memory Contents for RAM and ROM
- Defining Coefficient Values in a COE File

Course Description:

❑ System Generator

- ➢ All Steps from Simulink to FPGA Implementation Design Process

Course Description:

❑ ASIC Design Flow:

- **HDL Coding & Verification**
- **Synthesis & Timing Optimization**
  - Complete Synopsys Design Complier Design Flow
- **Physical Design**
  - Cadence First Encounter
  - Floorplan (Initial floorplan and power planning)
  - Placement (Full-scale floorplan and clock tree insertion)
  - Routing (power routing & Nanoroute)
  - Timing Closure (Analysis & Optimization of setup and hold time violations)
  - Fill (Filler Cells, Metal Fill, and Verify Geometry)

Course Description:

❑ CAD Tool Algorithms:

- **Synthesis Algorithms**
  - Two-level Optimization
  - Multi-level Logic Optimization
  - Technology Mapping

Course Description:

❑ Power Dissipation

- ➢ Power Dissipation concept
- ➢ Dynamic Power
- ➢ Static Power
- ➢ Challenges

Course Description:

❑ Power Grid and Clock Design

- ➢ Power Distribution Design
  - Introduction
  - IR Drop
  - Ldi/dt Drop
- ➢ Decoupling Capacitances
- ➢ Clock Considerations
- ➢ PLL/DLL Architecture

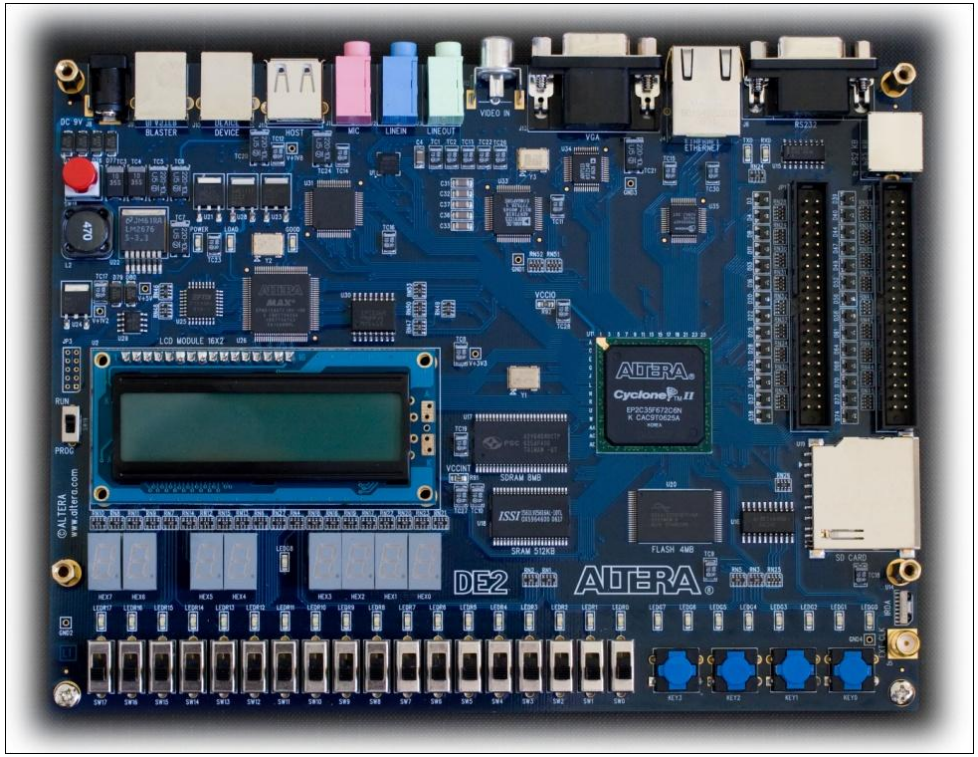
Course Description:

- **Prerequisites:**
  - Only Digital Logic!
  - All the skills you need will be taught in the course

- **Softwares you will learn:**
  - Altera Quartus
  - Xilinx ISE
  - Mentor Graphics Modelsim
  - TCL Scripting
  - Synopsys Design Compiler
  - Cadence SOC Encounter

Course Description:

- **Implementation Platform:**
  - Altera DE2 Board
  - Xilinx Boards



- You will do several practical assignments including Verilog Coding, FPGA implementation and testing based on the DE2 board and Xilinx Boards

Course Description:

- **Assignments (8)**
  - 6 implementation assignments on DE2 Board (3 Marks)
  - 1 synthesis algorithms (3 marks)
  - 1 comprehensive project from A to Z (5 marks)

- **Workshop Labs on Xilinx Boards**

Soft deadlines: 20% daily penalty for the late submissions

Course Evaluation:

Section	Quantity	Value each	Value
Assignments	8	3	26
Workshop Labs	2	2.5	5
Midterm	1	26	29
Final Exam	1	40	40
		Total:	**100**

If both the midterm and final marks are less than 50%, the total mark is calculated only based on the exams and assignments will not be taken into account.

Course Outline : References

- **HDL:**
  - Thomas & Moorby, The Verilog Hardware Description Language, 3rd ed, Kluwer Academic.
  - Introduction to Logic Synthesis using Verilog HDL, Robert Reese, Mitchell Thornton, 2006.
  - Advanced FPGA Design, Architecture, Implementation, and Optimization, Steve Kilts, 2007.

- **IC Design Flow:**
  - Course Lecture notes, 2014
  - Digital IC Design Flow, provided by the instructor, 2014

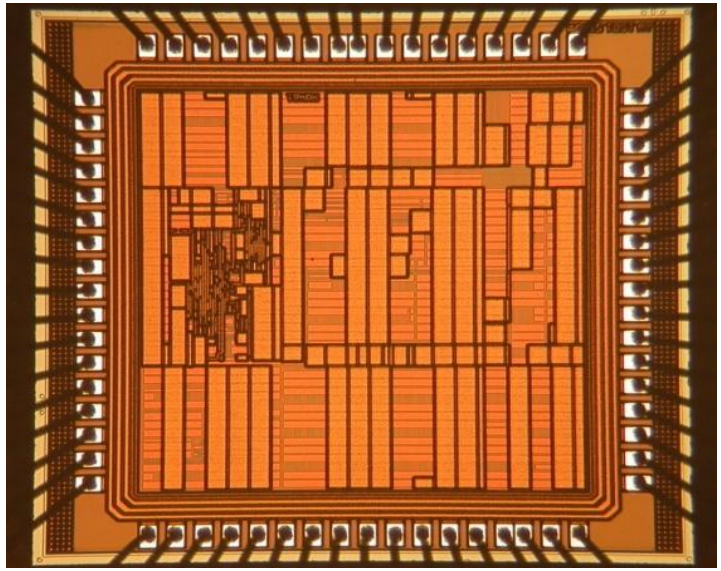
Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Integrated Circuit (IC) History

Discrete Circuits

- Limited applications
- Power hungry
- Large area
- Moderate speed



Integrated Circuits

- Many applications
- Low power
- Small area
- High speed

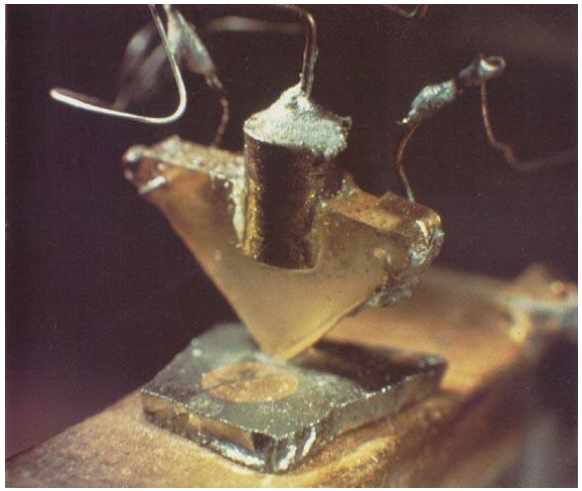
Integrated Circuits (IC) History

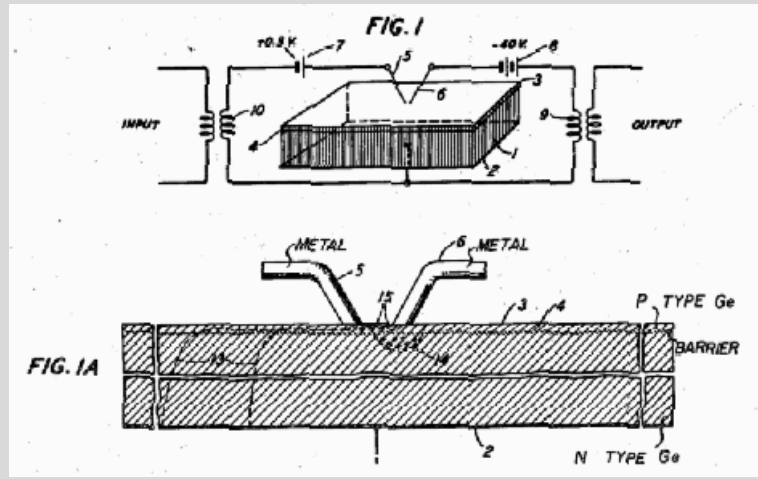
1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

Ohl built the PN Junction: 1940

Shockley Lab was established: 1945

Brattain and Bardeen invented the first transistor (US Patent 2524035): 1947



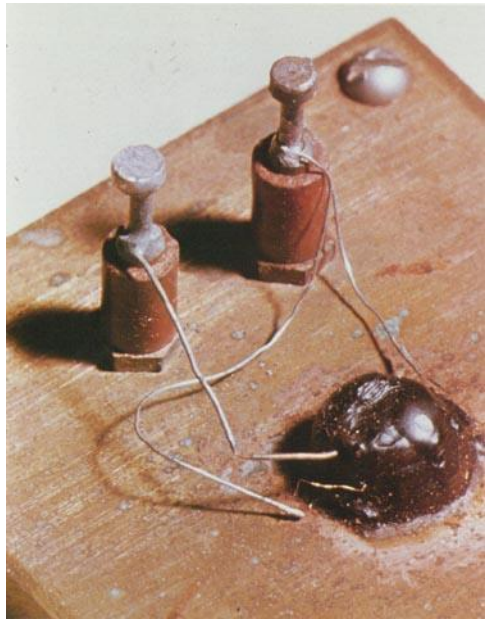


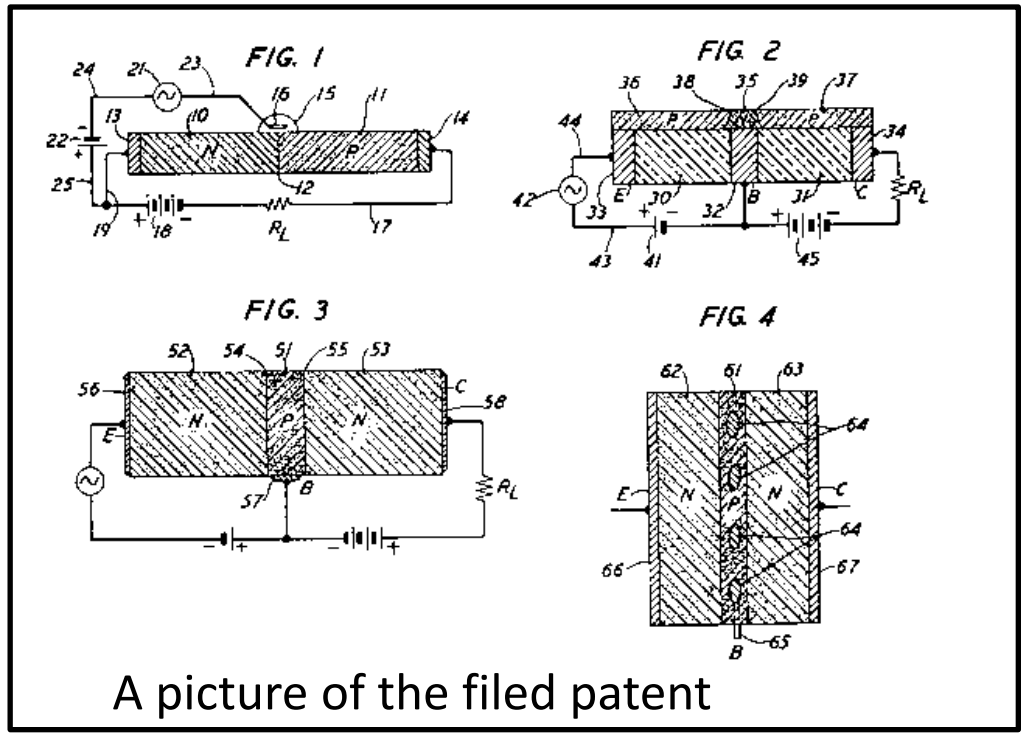
A picture of the filed patent

Integrated Circuits (IC) History

1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

- 1951: Shockley invented the first junction transistor for mass production
  - (US Patent 2623105)





A picture of the filed patent

Integrated Circuits (IC) History

1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

۱۹۵۵: Noyce به آزمایشگاه Shockley پیوست

۱۹۵۷: Noyce آزمایشگاه Shockley را به هدف تاسیس Fairchild با کمک Hoerni و Gorden Moore ترک کرد

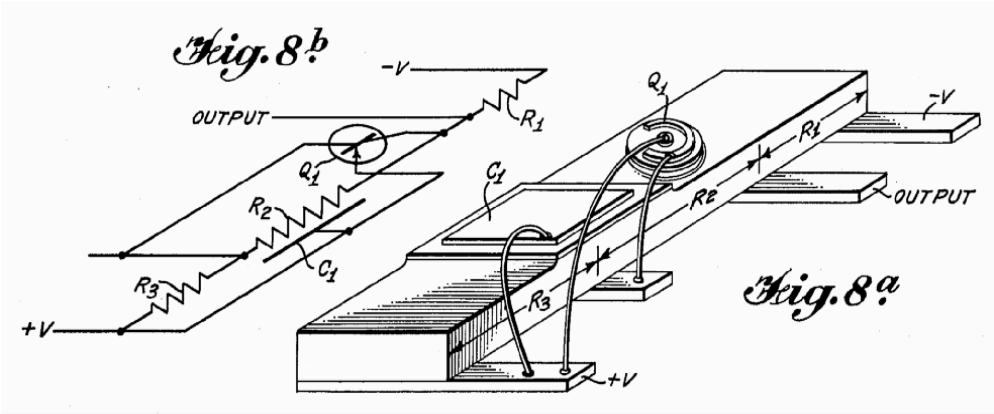
۱۹۵۸: Hoerni تکنیکی را برای افشاندن ناخالصی ها بر روی Si ابداع کرد و به این ترتیب توانست یک ترانزیستور مسطح با کمک عایق SiO_2 بسازد

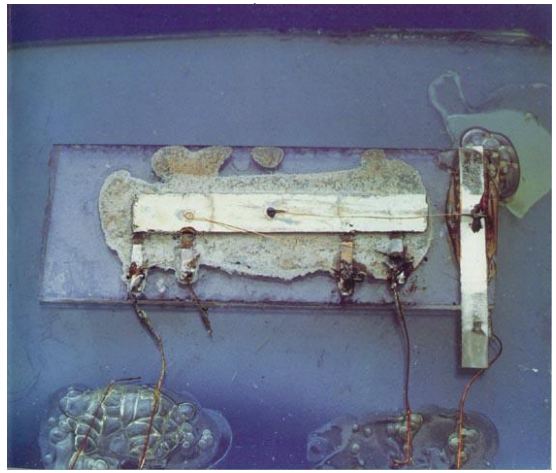
۱۹۵۹: Noyce اولین مدار مجتمع را با استفاده از ترانزیستورهای مسطح ساخت

Integrated Circuits (IC) History

1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

۱۹۵۹: Jack Kilby که در Texas Instrument (TI) کار می کرد ایده مدار مجتمع یک پارچه را مطرح کرد. در این طرح قطعات به وسیله سیم های لحیم شده به یکدیگر متصل می شدند و از دیودهای PN به عنوان مقاومت استفاده می شد (U.S Patent)





A picture of the filed patent

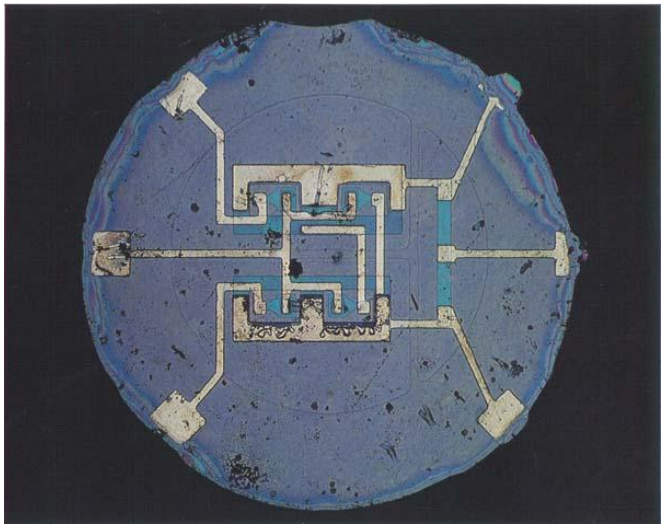
Integrated Circuits (IC) History

1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

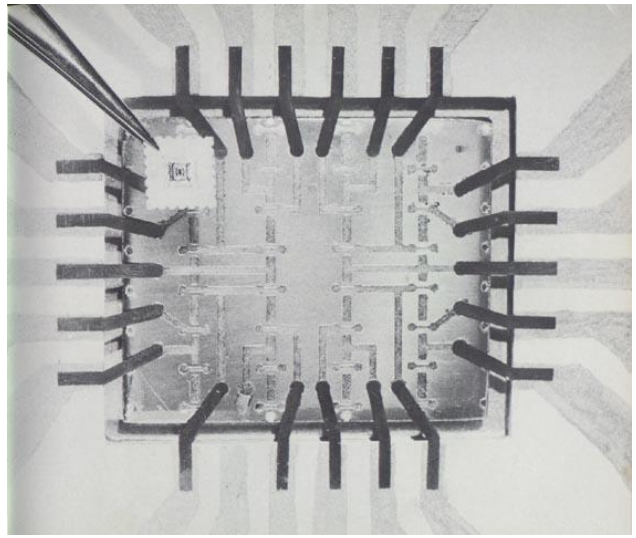
۱۹۶۱: TI و Fairchild اولین IC منطقی را تولید انبوه کردند (هر کدام 50$)

۱۹۶۲: Ripple Carry Adder شانزده بیتی ساخته شد

Fairchild bipolar RTL Flip-Flop



RCA 16-transistor MOSFET IC



Integrated Circuits (IC) History

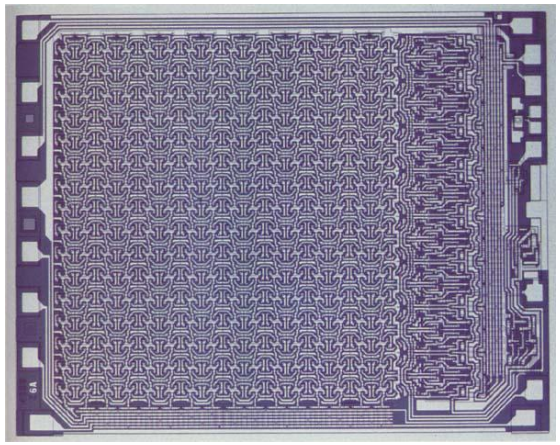
1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

۱۹۶۸: Noyce و Moore موسسه Fairchild را رها کرده و Intel را پایه گذاری کردند

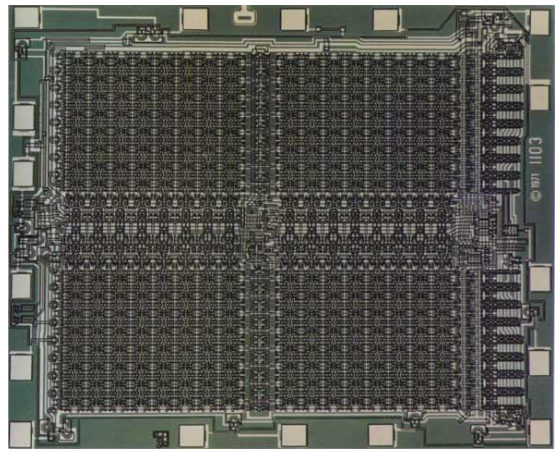
۱۹۷۰: Static RAMs 256-bit , Fairchild را تولید کرد

۱۹۷۰: Dynamic RAMs 1-K bit , Intel را فروخت

Fairchild 4100 256-bit SRAM



Intel 1103 1K-bit DRAM

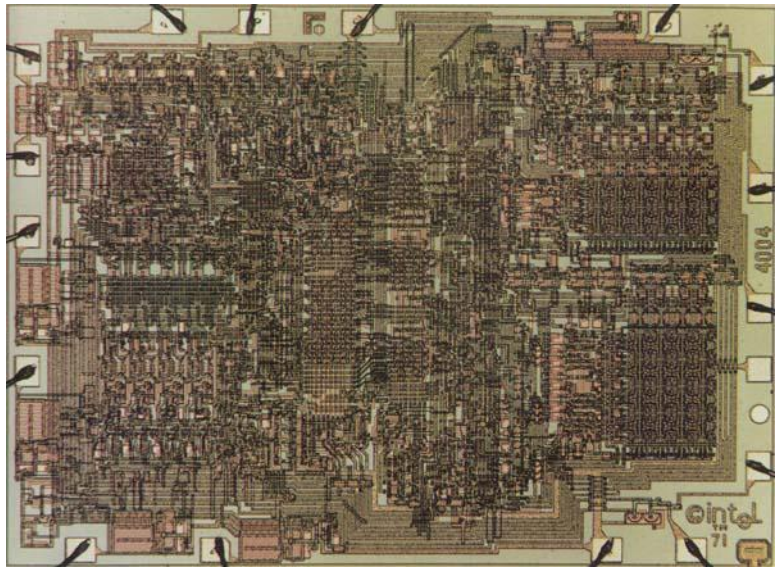


Integrated Circuits (IC) History

1940 1945 1947 1951 1955 1957 1958 1959 1961 1962 1968 1970 1971

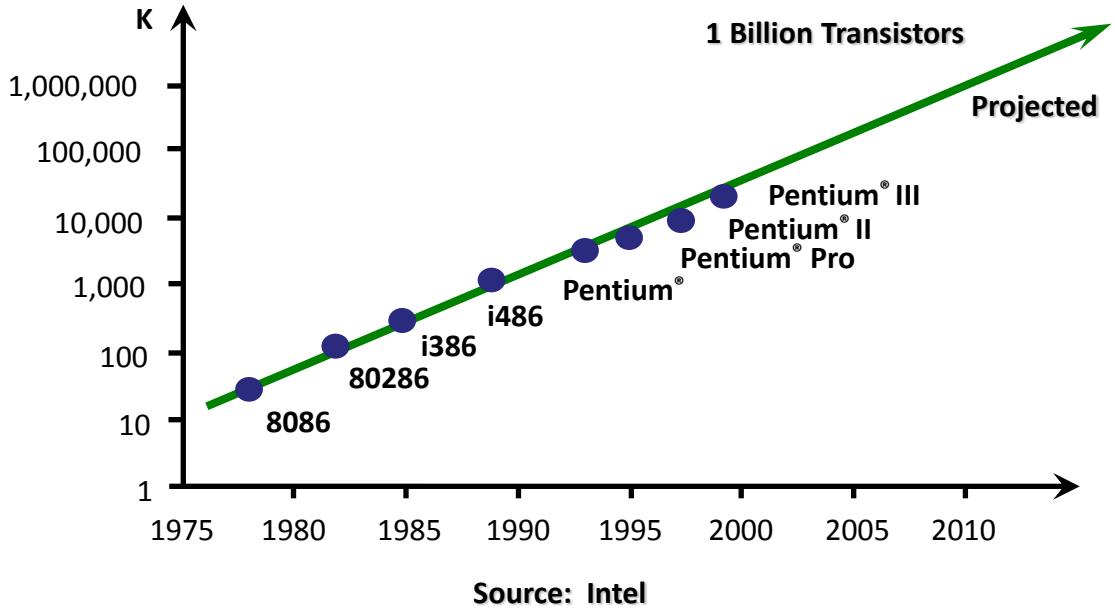
۱۹۷۱: Intel , ۴۰۰۴ را تولید کرد

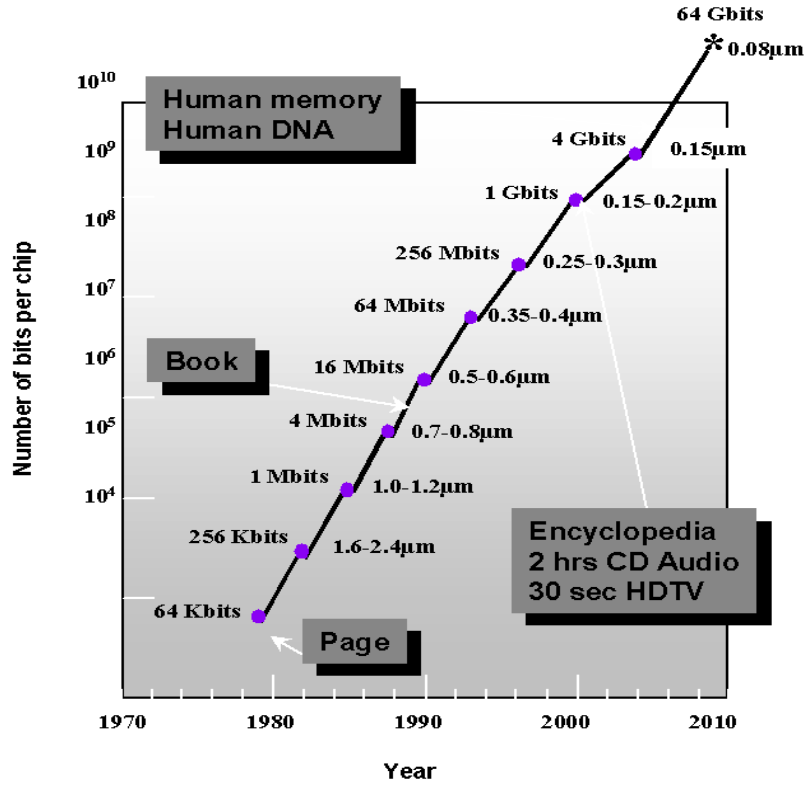
اولین کامپیوتر قابل برنامه ریزی چند منظوره برای شرکت ماشین حساب ژاپنی



IC History

1971 2010





Number of Transistors doubled every 18 months!

(Moore's Law)

Scaling still continues…

IC History

1971 2010

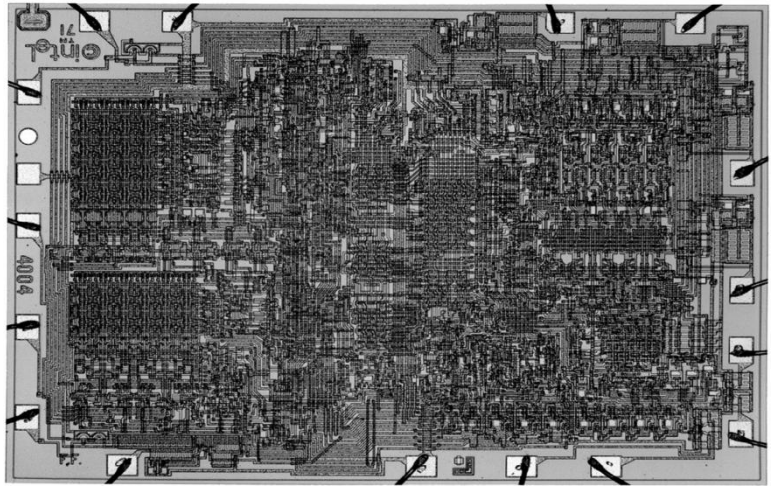
- Amazingly visionary – million transistors/chip barrier was crossed in the 1980's.
  - 2300 transistors, 1 MHz clock (Intel 4004) - 1971
  - 16 Million transistors (Ultra Sparc III)
  - 42 Million, 2 GHz clock (Intel P4) - 2001
  - 140 Million transistor (HP PA-8500)

Integration Levels

SSI:	10 gates
MSI:	1000 gates
LSI:	10,000 gates
VLSI:	> 10k gates

IC History

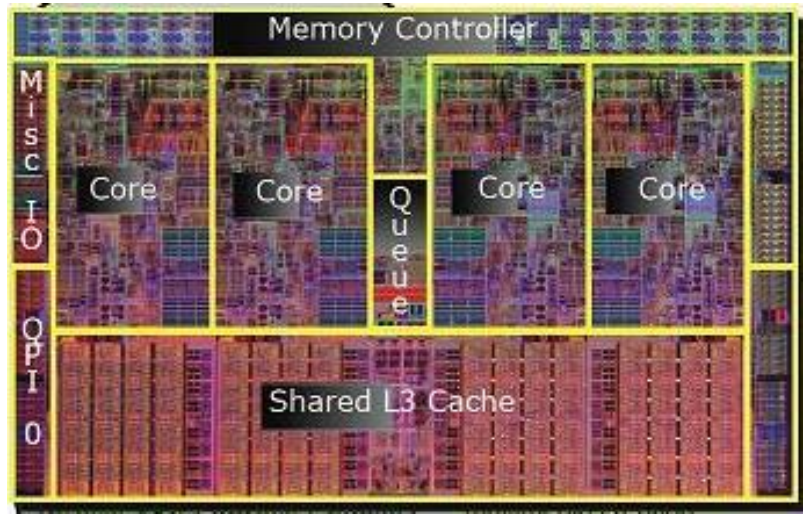
Early Designs



(1971)

- 1000 Transistors (1 MHz)
- Process: 10um
- Fully Handcrafted
- Manual Layout
- Individually Optimized

Advanced Designs

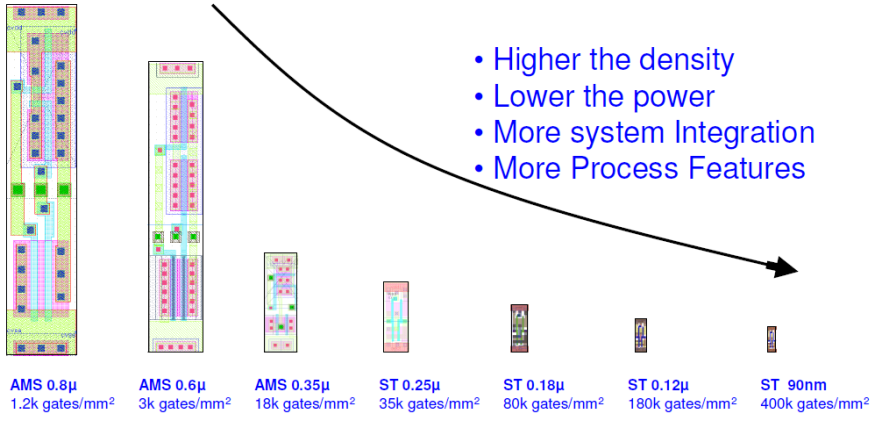


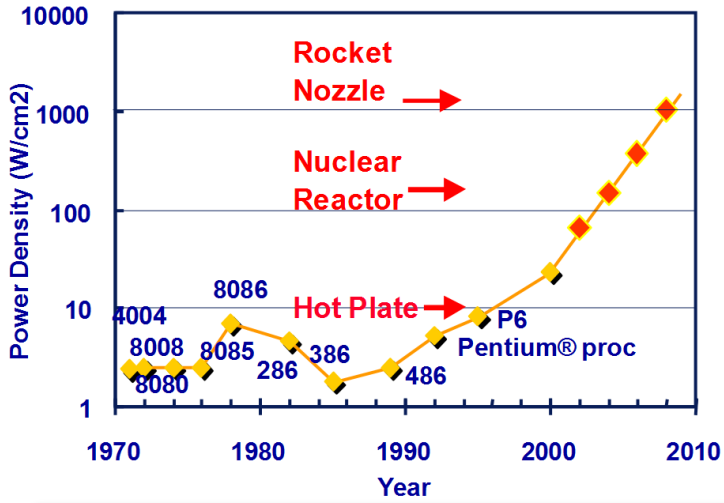
(2010)

- Intel Corei7 (~ 1.6 GHz)
- Process: 32nm
- Fully Automated
- Automated Layout
- Hierarchical Design

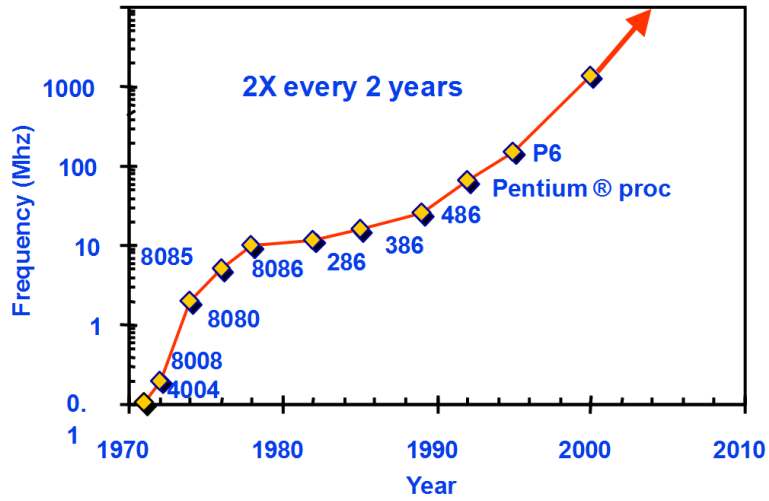
Digital Very Large Scale Integration (VLSI) (Not Analog!)

Process Improvement





Power density too high to keep junctions at low temp



up frequency doubles every 2 years

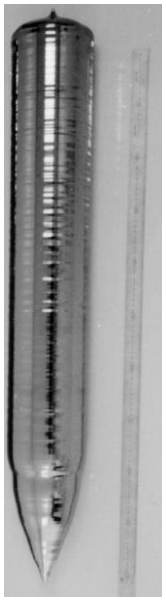
Technology Direction

Year	1999	2002	2005	2008	2011	2014
Feature size (nm)	180	130	100	70	50	35
M trans/cm^2	7	14-26	47	115	284	701
Chip size (mm^2)	170	170-214	235	269	308	354
Signal pins/chip	768	1024	1024	1280	1408	1472
Clock rate (MHz)	600	800	1100	1400	1800	2200
Wiring levels	6-7	7-8	8-9	9	9-10	10
Power supply (V)	1.8	1.5	1.2	0.9	0.6	0.6

Why Scaling?

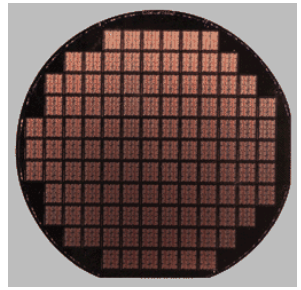
- Technology shrinks by ~0.7 per generation
- With every generation can integrate 2x more functions on a chip; chip cost does not increase significantly
- Cost of a function decreases by 2x
- But …
  - How to design chips with more and more functions?
  - Design engineering population does not double every two years…
- Hence, a need for more efficient design methods
  - Exploit different levels of abstraction

IC Manufacturing Process

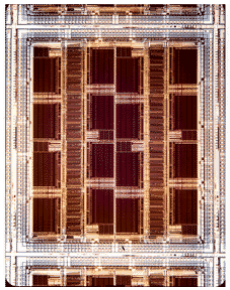


Silicon Cylinder

Silicon Wafer

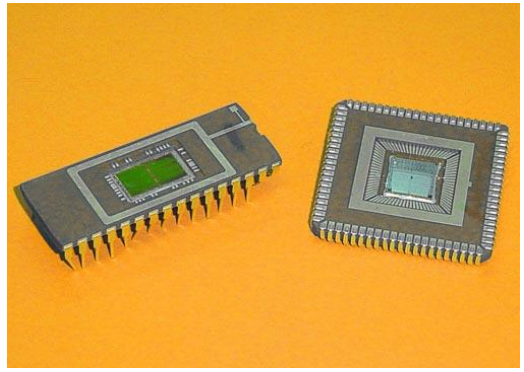


Patterned Silicon Wafer



Unpackaged Die

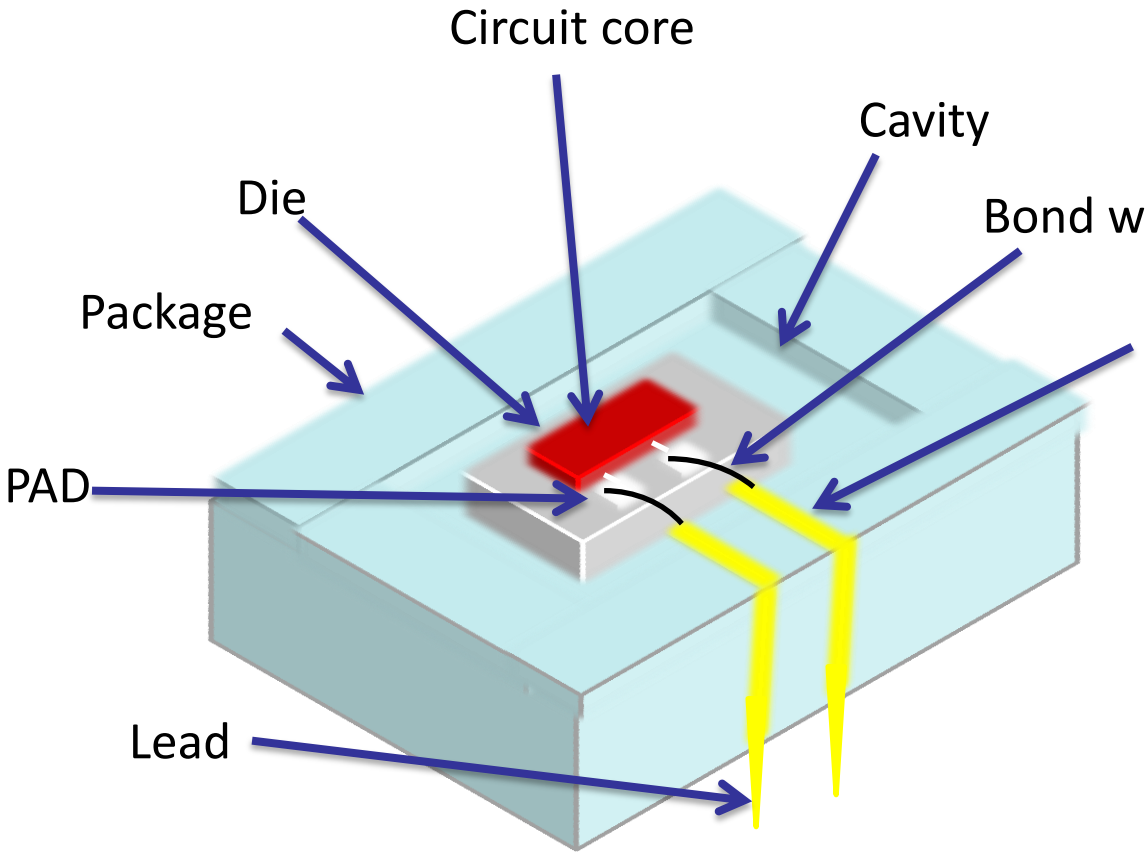
Circuit Core

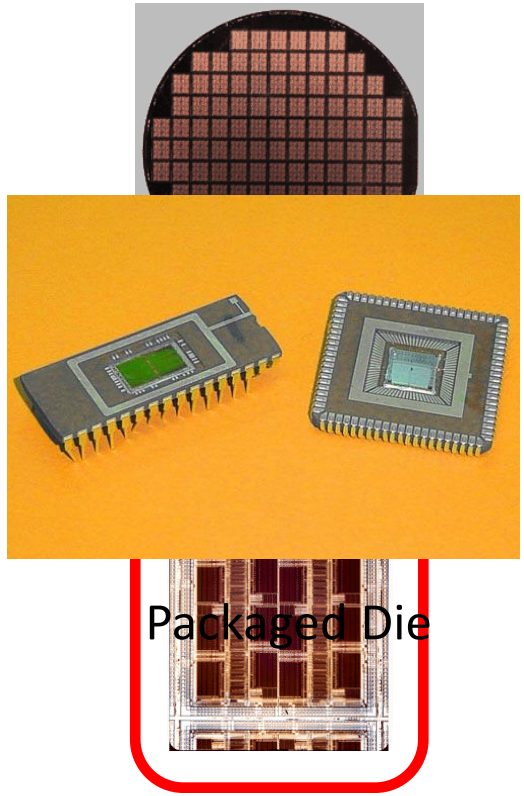


Packaged Die

Final Test

IC Manufacturing Process



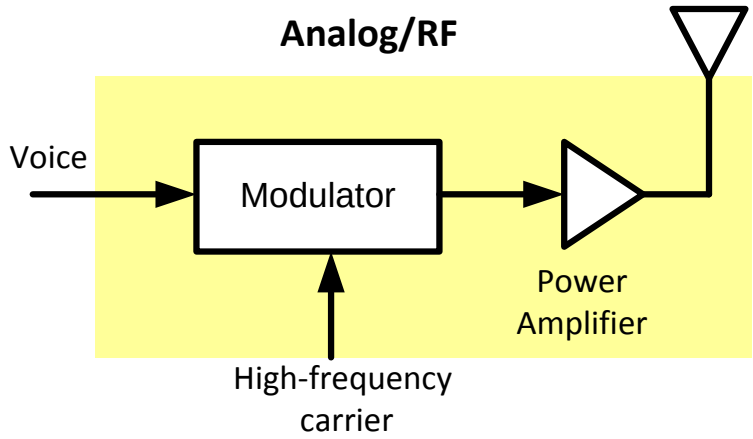


Course Outline

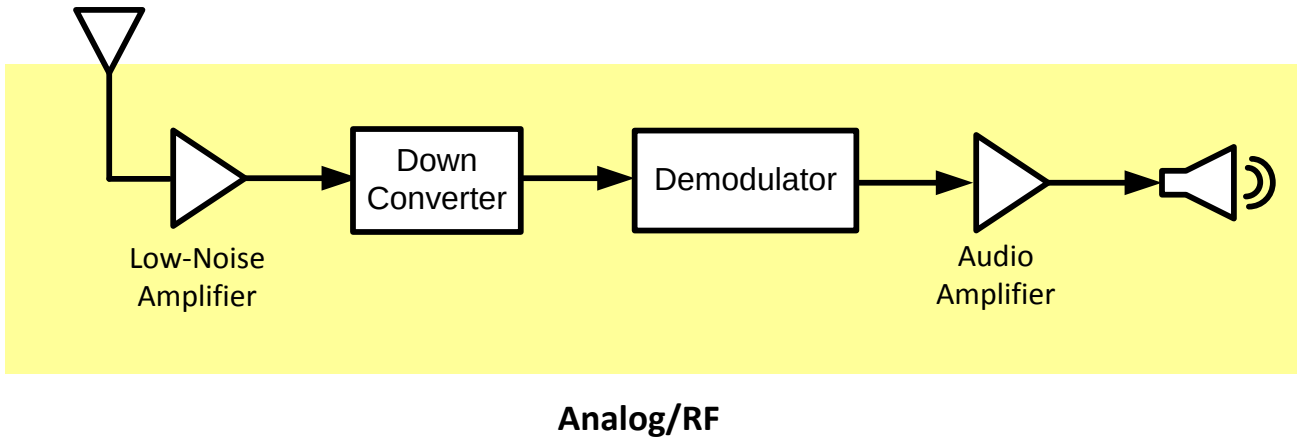
- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Digital vs. Analog Design : Analog Circuits

❑ Transmitter:

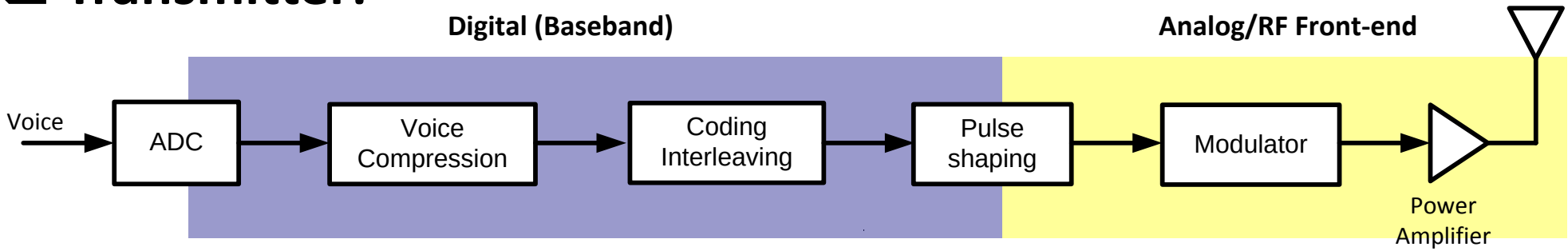


❑ Receiver:



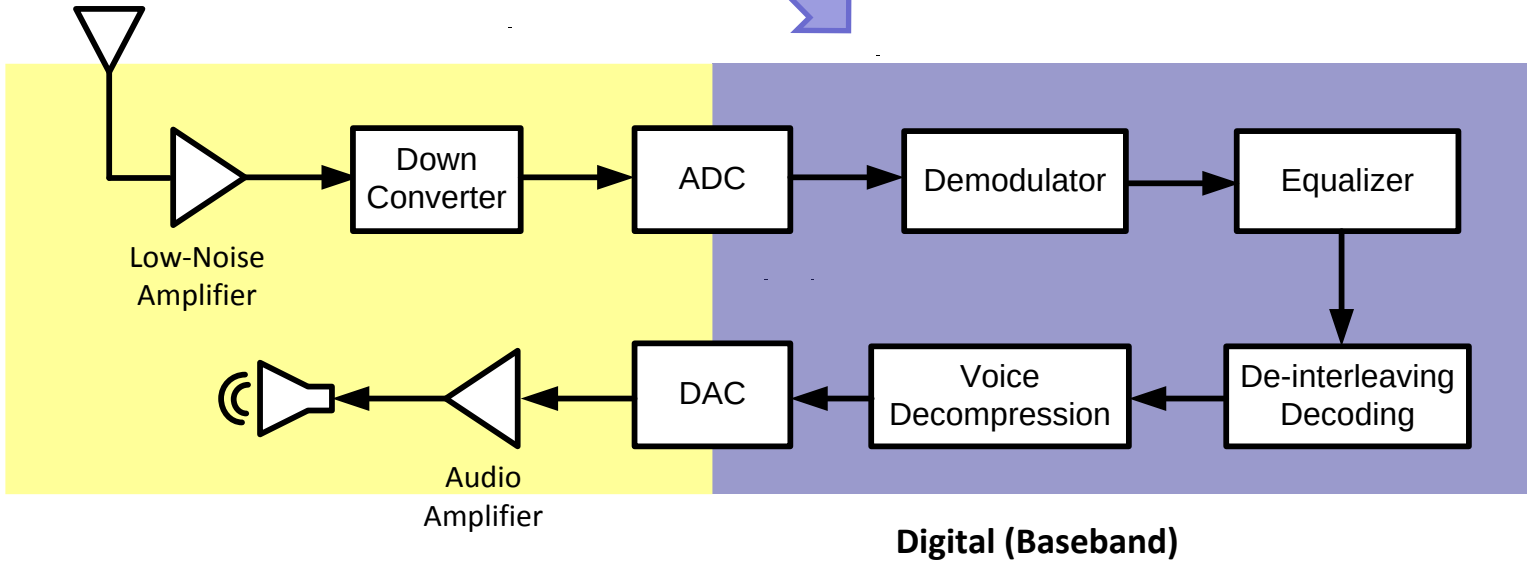
Digital vs. Analog Design : Analog-Digital Circuit

❑ Transmitter:



❑ Receiver:

This Course

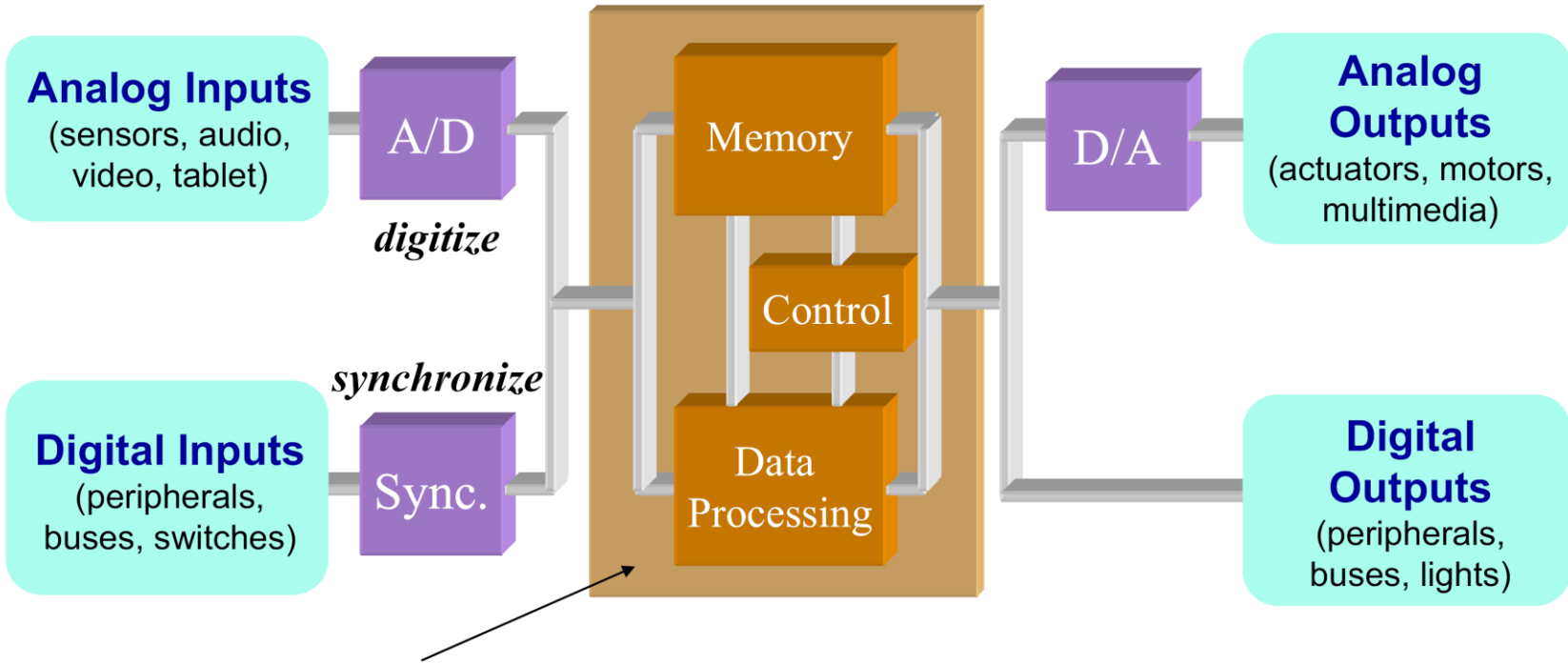


Digital vs. Analog Design

❑ Digital Design Advantages:

- ➢ More noise reliability
- ➢ Allows signal decoding and amplification
- ➢ Allows coding to achieve higher performance
- ➢ Allows encryption for higher security
- ➢ Provides a perfect vehicle for digital signal processing
- ➢ Allows modular chip design
- ➢ Enjoys the benefit of advanced sophisticated CAD tools
- ➢ Can be verified on programmable devices before tape-out
- ➢ Provides a platform to merge multiple networks such as telephone, terrestrial TV and computer networks.

Digital vs. Analog Design



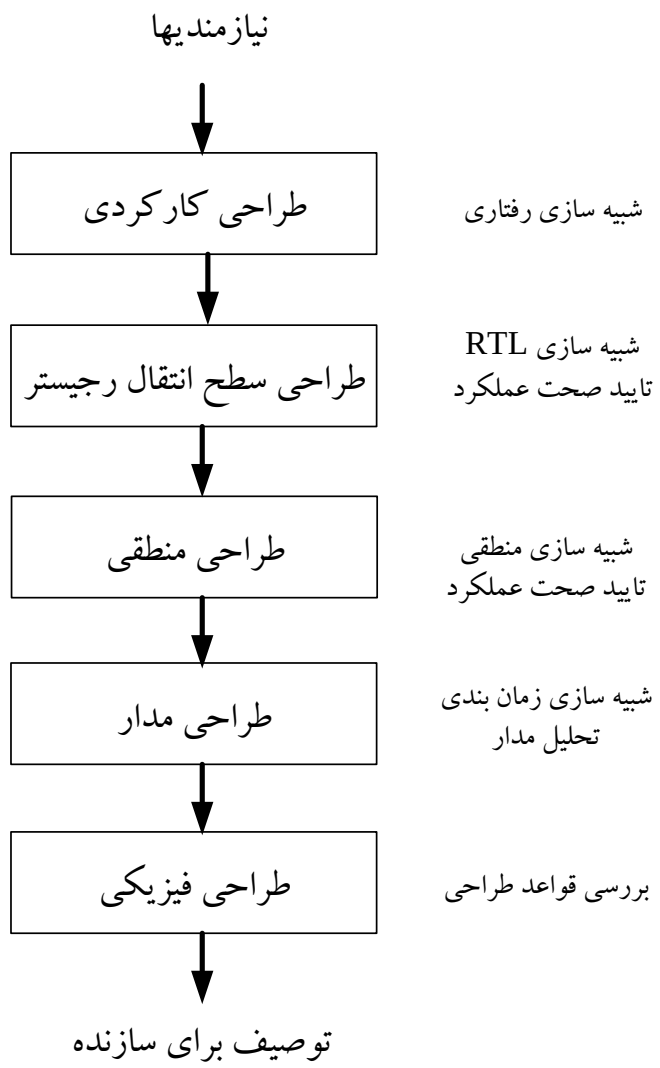
- **Digital processing systems consist of a datapath, memory, and control. Early machines for arithmetic had insufficient memory, and often depended on users for control**
- **Today's digital systems are increasingly embedded into everyday places and things**
- **Richer interaction with the user and environment**

Digital System Design Steps

- Specific application (e.g., Digital Image Processor)
  - As opposed to a general microprocessor
- Design requirements
  - Images to be processed per second
  - Required processing
  - Interfacing
  - Dimension, power, price,…
- General design and simulation
- RTL level design to verify and simulate the design

- Higher levels: Few complicated blocks
- Lower levels: Many simple blocks

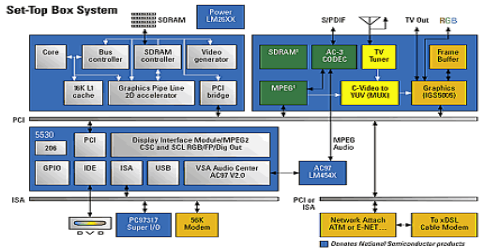
- To avoid any problem in lower levels, many simulations in higher levels



Design Flow

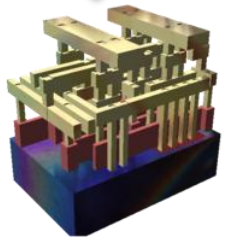
Idea

Architecture Design



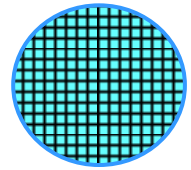
Block diagram

Circuit & Layout Design



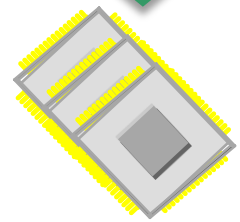
Layout

IC Fabrication



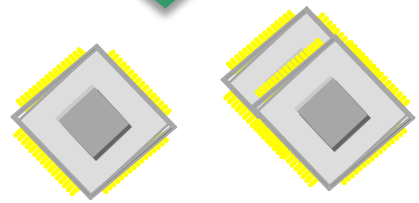
Wafer (hundreds of dies)

Sawing & Packaging



Final chips

Testing



customers

Bad chips

Good chips

Digital Systems Components

- Printed Circuit Board (PCB)
- Embedded Software
  - Microprocessor (general Purpose)
  - Microcontroller
  - Digital Signal Processor (DSP)
- Programmable Logic Devices
  - Simple Programmable Logic Device (SPLD)
  - Complex Programmable Logic Device (CPLD)
  - Field Programmable Gate Arrays (FPGAs)
- Application Specific Integrated Circuits (ASICs)

Digital Systems Components

DSPs

- ❑ Easy to program (usually standard C)
- ❑ Very efficient for complex sequential math-intensive tasks

- ❑ Fixed datapath-width. Ex: 24-bit adder, is not efficient for 5-bit addition
- ❑ Limited resources

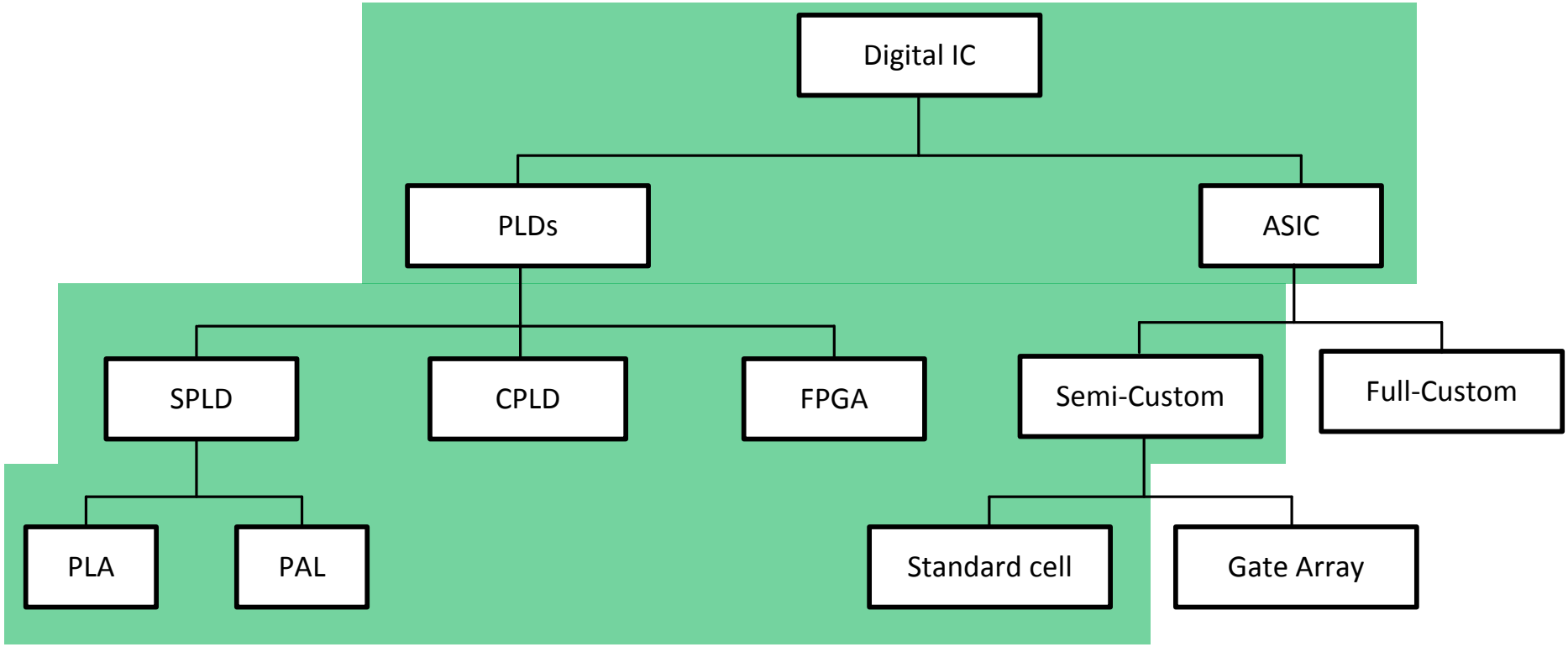
Digital Systems Components

Microprocessors/Microcontrollers

- ❑ Lessens the risk of system development by reducing design complexity
- ❑ Fixed hardware so more effort on developing a good code

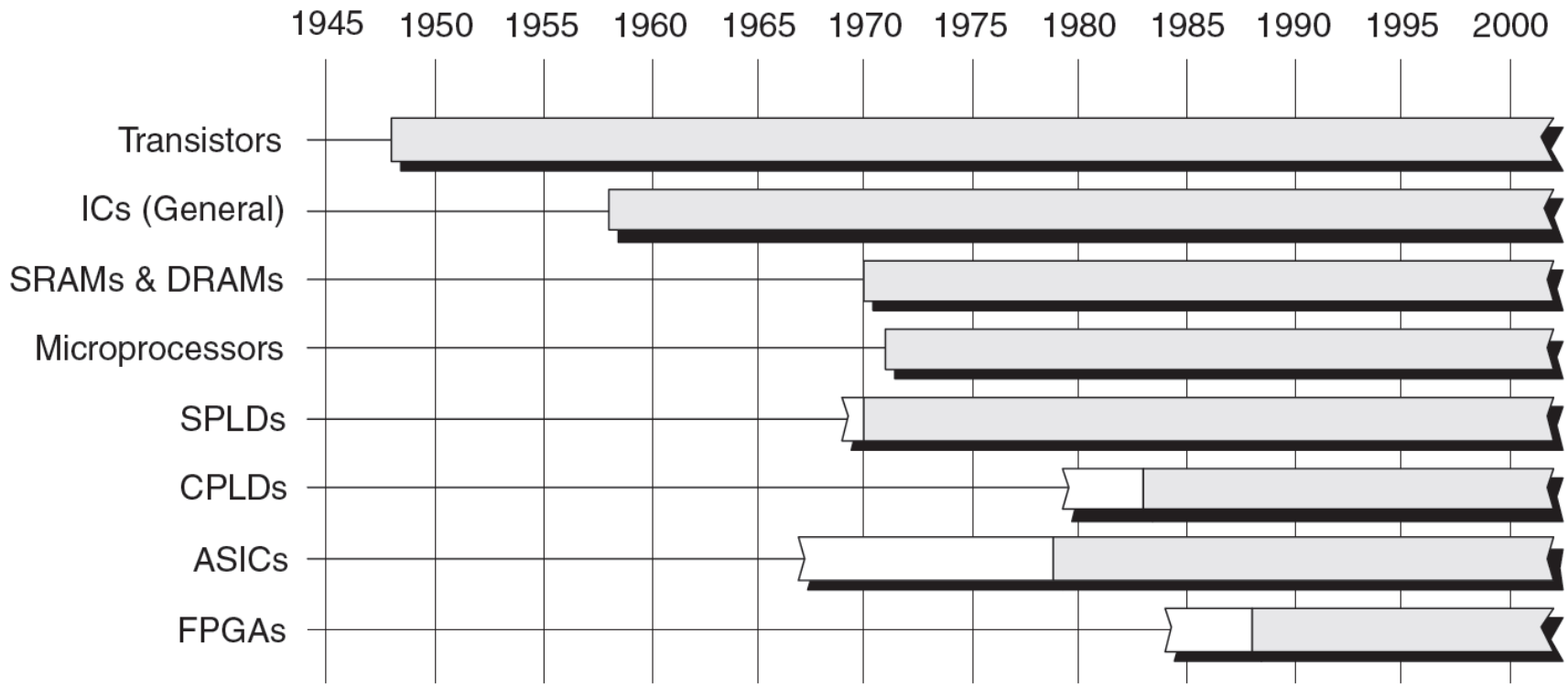
- ❑ Fixed HW not suitable for high level of parallelism/computations
- ❑ Many sequential nature
- ❑ One order of magnitude less performance than to-date FPGA/ASICs

Digital Systems Implementation Platforms



This course

Technology Timeline



The white portions of the timeline bars indicate that although early incarnations of these technologies may have been available, they weren't enthusiastically received by the engineers working in the trenches during this period. For example, although Xilinx introduced the world's first FPGA as early as 1984, design engineers didn't really start using it until the early 1990s.

Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

FPGA vs. ASIC

❑ Field Programmable Gate Array (FPGA) Advantages:

- ➢ Fast programming and testing time by the end user (instant turn-around)
- ➢ Excellent for prototyping
  - ▪ Easy to migrate from prototype to the final design
- ➢ Can be re-used for other designs
- ➢ Cheaper (in small volumes) ⟹ lower start-up costs
- ➢ Re-programmable
- ➢ Lower financial risk
- ➢ Ease of design changes/modifications
- ➢ Cheaper design tools

FPGA vs. ASIC

- **FPGA Drawbacks:**
  - Slower than ASIC (2-3 times slower)
  - Power hungry (up to 10 times more dynamic power)
  - Use more transistors per logic function
  - More area (20 to 35 times more area than a standard cell ASIC)

FPGA vs. ASIC

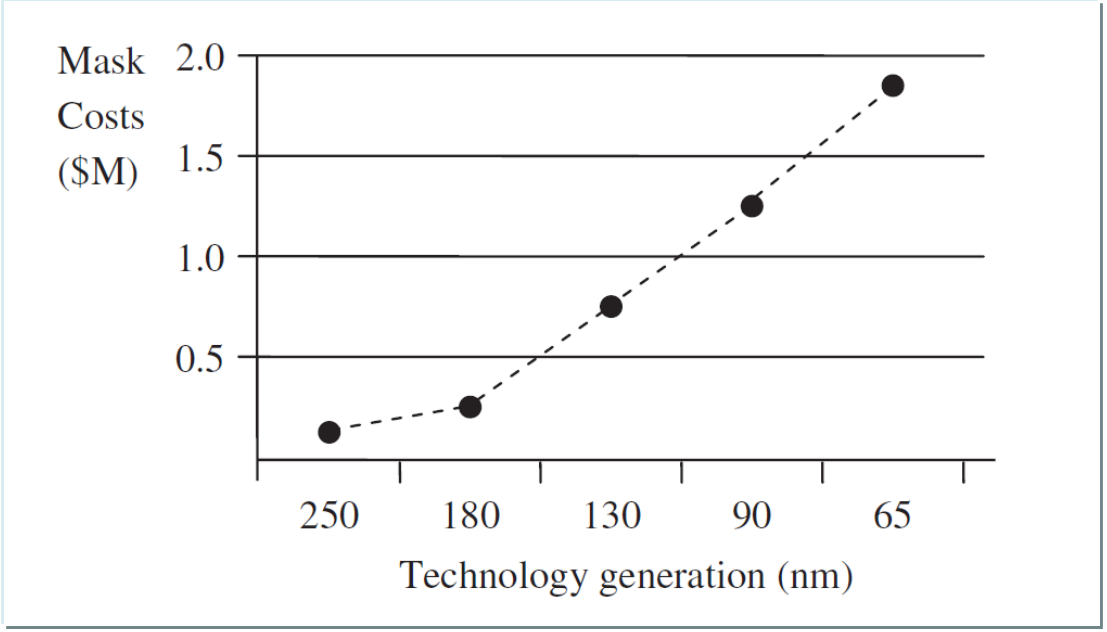
❑ Application Specific Integrated Circuit (ASIC) Advantages:

- ➢ Faster
- ➢ Lower power
- ➢ Cheaper (if manufactured in large volumes)
- ➢ Use less transistors per logic function

❑ ASIC Drawbacks:

- ➢ Implements a particular design (not programmable)
- ➢ Takes several months to fabricate (long turn-around)
- ➢ More expensive design tools
- ➢ Very expensive engineering/mask cost for the first successful design

ASIC Mask Generation Cost



- The ASIC is accompanied by increasing nonrecurrent engineering (NRE) costs which meant that there was an increased emphasis on “right first time” design.

- These NRE costs is largely due to the cost of generating masks as it is becoming more expensive to generate the masks for finer geometries needed by shrinking silicon technology dimensions.

Implementation Approaches (ASIC vs. FPGA)

ASIC

Application **S**pecific
Integrated **C**ircuit

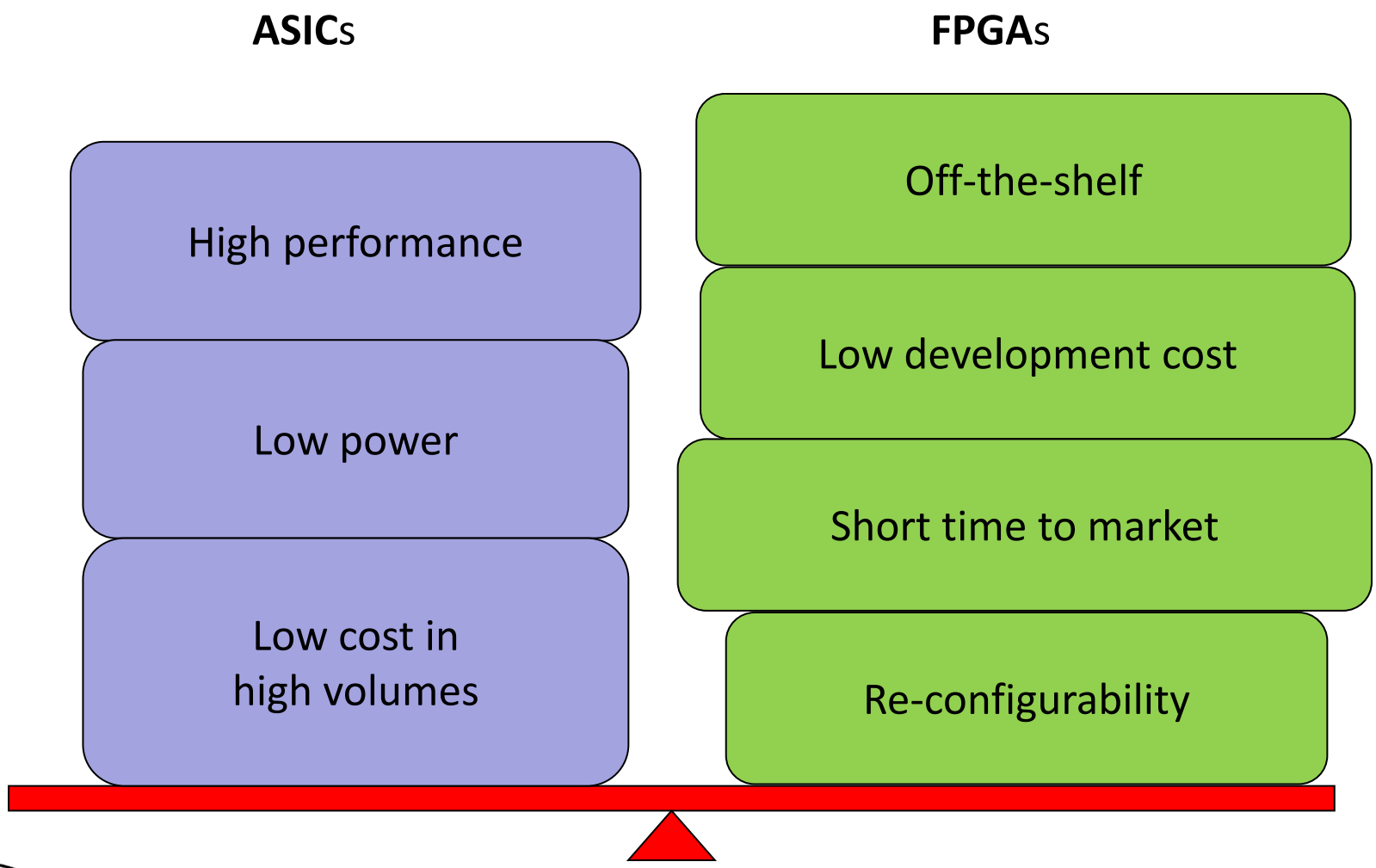
- ❑ Expensive & time consuming fabrication in semiconductor foundry
- ❑ Designed all the way from behavioral description to physical layout

FPGA

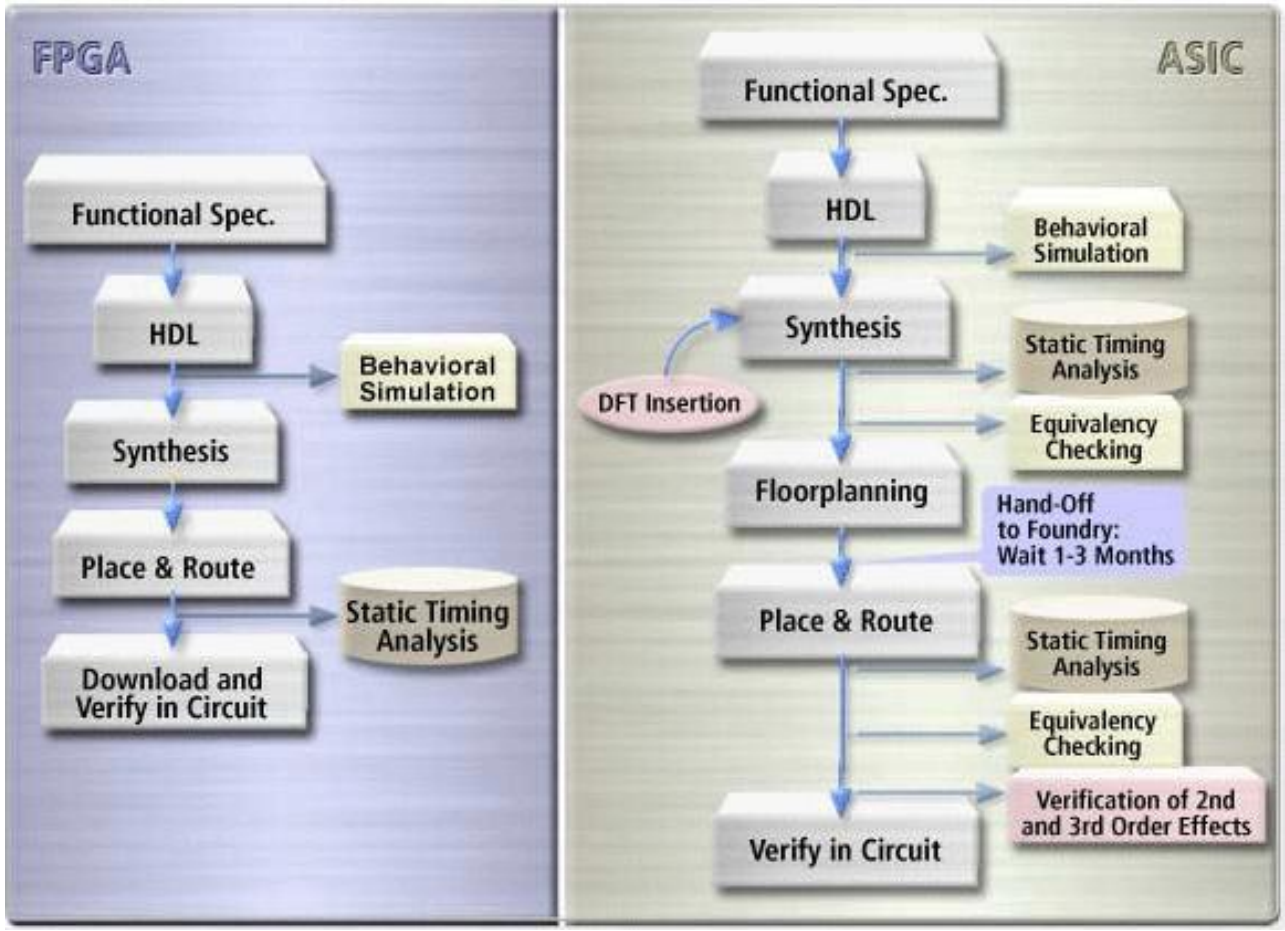
Field **P**rogrammable
Gate **A**rray

- ❑ Bought off the shelf & reconfigured by the end designers
- ❑ No physical layout design
- ❑ Design ends with a bitstream used to configure a device

Implementation Approaches (ASIC vs. FPGA)



ASIC vs. FPGA


FPGA
Functional Spec.
HDL
Behavioral Simulation
Synthesis
Place & Route
Static Timing Analysis
Download and Verify in Circuit
ASIC
Functional Spec.
HDL
Behavioral Simulation
Synthesis
Static Timing Analysis
DFT Insertion
Equivalency Checking
Floorplanning
Hand-Off to Foundry: Wait 1-3 Months
Place & Route
Static Timing Analysis
Equivalency Checking
Verification of 2nd and 3rd Order Effects
Verify in Circuit

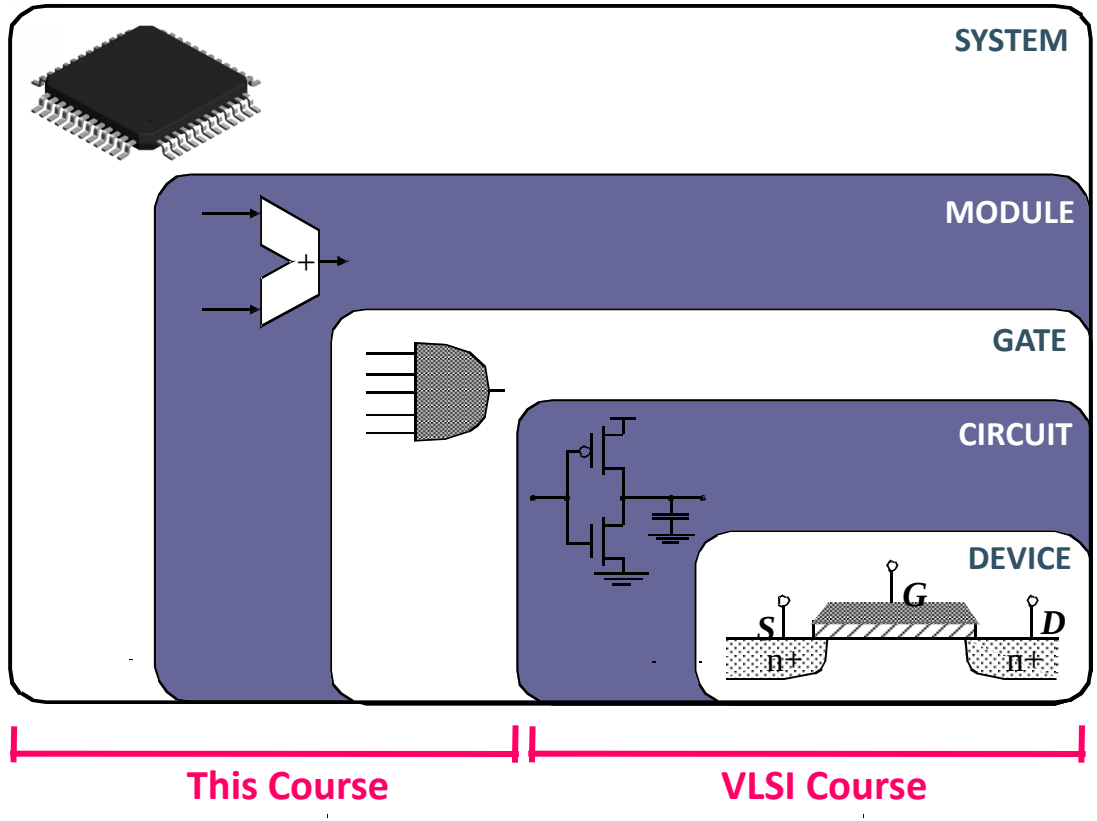
Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Design Abstraction Levels

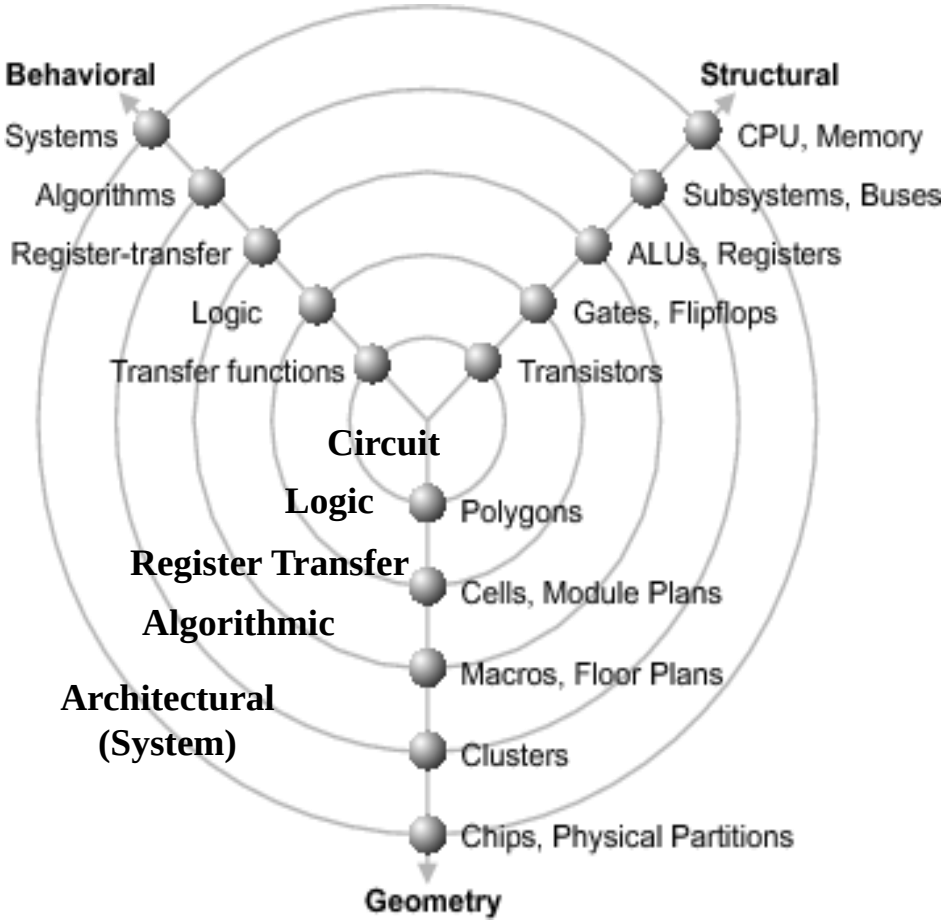
- **Divide-and-Conquer**
  - Design modules once
  - Instantiate them thereafter
  - Standard Cells
    - Already laid out
  - Avoid re-design
  - Same as programming

- **Designer cares about module's:**
  - Functionality
  - Delay characteristics
  - Area
- **NOT:**
  - How the module was designed
  - Detailed solid-state behavior

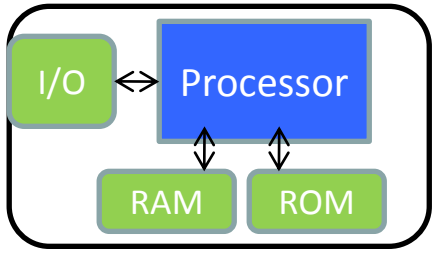


Design Abstraction Levels

Y-Chart



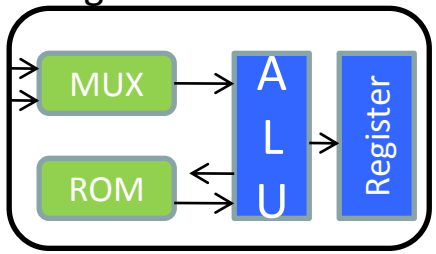
System Level



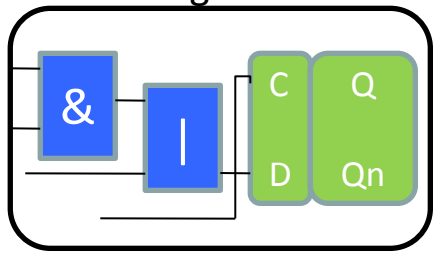
Algorithmic Level

```
A:=A*B+F
IF(C=TRUE) THEN
A:=A+2*F
ELSE A:=A-1
ENDIF
```

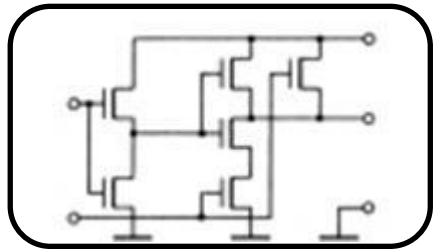
Register Transfer Level



Logic Level



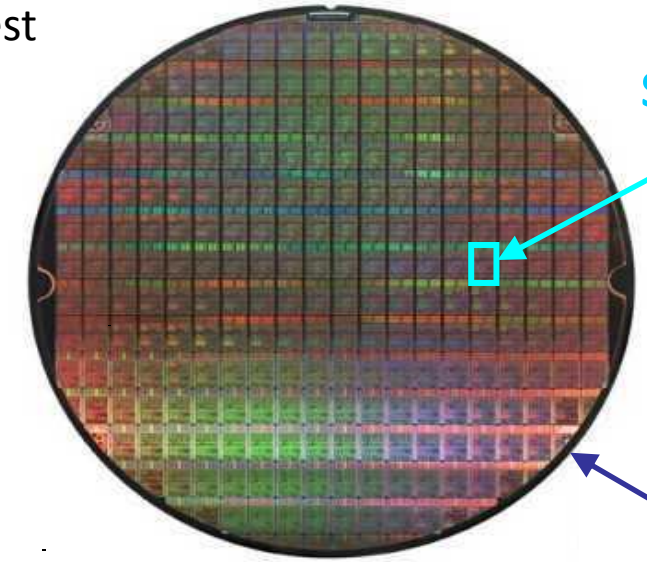
Circuit Level



Device Metrics

Performance Metrics of a Digital Chip:

- **Cost**
  - NRE (fixed) costs - design effort
  - RE (variable) costs - cost of parts, assembly, test
- **Speed**
  - Delay (ns) → Operating Frequency (MHz)
- **Power Dissipation**
- **Energy to Perform a Function**
  - Energy per bit (nJ/b)
- **Reliability**
  - Noise immunity
  - Noise margin
- **Scalability**
  - Larger Designs
- **Time-to-Market**

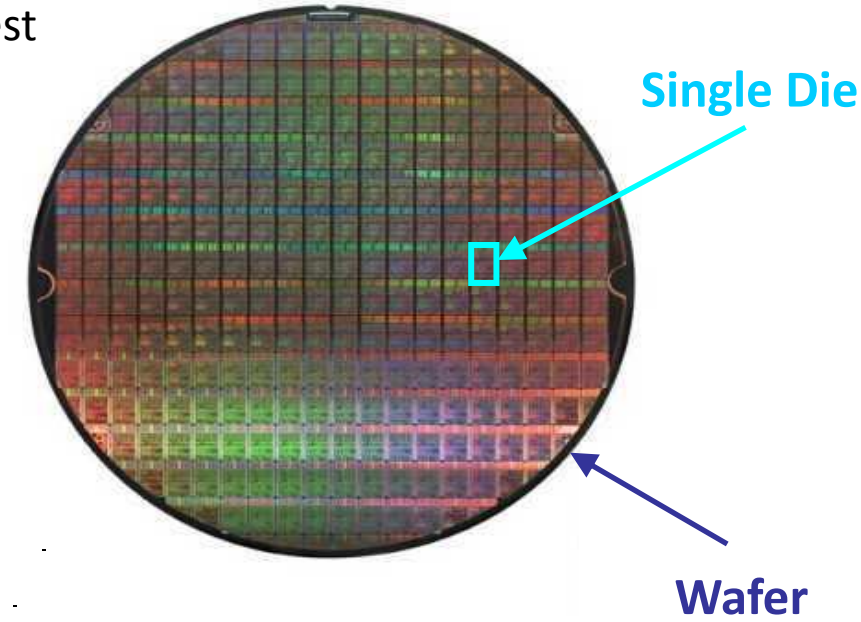


Diameter: 10-30 cm
Thickness: 1 mm

Device Metrics

Performance Metrics of a Digital Chip:

- ❑ **Cost**
  - ➢ NRE (fixed) costs - design effort
  - ➢ RE (variable) costs - cost of parts, assembly, test
- ❑ **Speed**
  - ➢ Delay (ns) → Operating Frequency (MHz)
- ❑ **Power Dissipation**
- ❑ **Energy to Perform a Function**
  - ➢ Energy per bit (nJ/b)
- ❑ **Reliability**
  - ➢ Noise immunity
  - ➢ Noise margin
- ❑ **Scalability**
  - ➢ Larger Designs
- ❑ **Time-to-Market**



Cost of Integrated Circuits

- **NRE (non-recurring engineering) costs**
  - Fixed cost to produce the design
    - Design effort
    - Design verification effort
    - Mask generation
  - Influenced by the design complexity and designer productivity
  - More pronounced for small volume products
- **RE (Recurring costs) – proportional to the product volume (i.e., Variable)**
  - Silicon processing
    - also proportional to chip area
  - Assembly (packaging)
  - Test

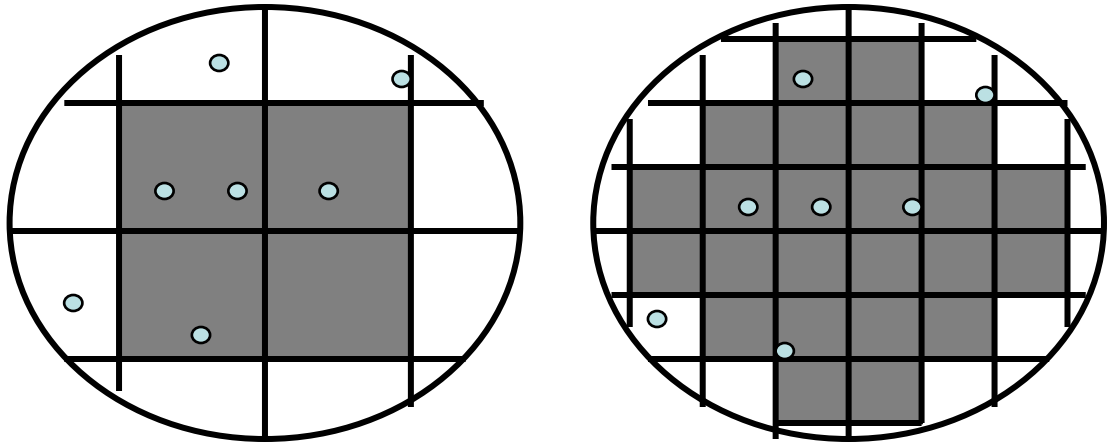
$$\text{Cost per IC} = \text{variable cost per IC} + \frac{\text{fixed cost}}{\text{volume}}$$

Recurring Costs

$$\text{variable cost} = \frac{\text{cost of die + cost of die test + cost of packaging}}{\text{final test yield}}$$

$$\text{cost of die} = \frac{\text{cost of wafer}}{\text{dies per wafer} \times \text{die yield}}$$

$$\text{Dies per wafer} = \frac{\pi \times (\text{wafer diameter}/2)^2}{\text{die area}} - \frac{\pi \times \text{wafer diameter}}{\sqrt{2} \times \text{die area}}$$



α depends on the complexity of the manufacturing process (roughly proportional to the number of masks)

$$\text{die yield} = (1 + (\text{defects per unit area} \times \text{die area})/\alpha)^{-\alpha}$$

Yield Example

- Example
  - Wafer size of 12 inches, die size of 2.5 cm^2, 1 defects/cm^2,
    - α = 3 (measure of manufacturing process complexity)
  - 252 dies/wafer (remember, wafers round & dies square)
  - Die yield of 16%
  - 252 x 16% = only 40 dies/wafer die yield !

- Die cost is a strong function of the die area
  - Proportional to the third or fourth power of the die area

$$\text{Cost of die} = f(\text{die area})^4$$

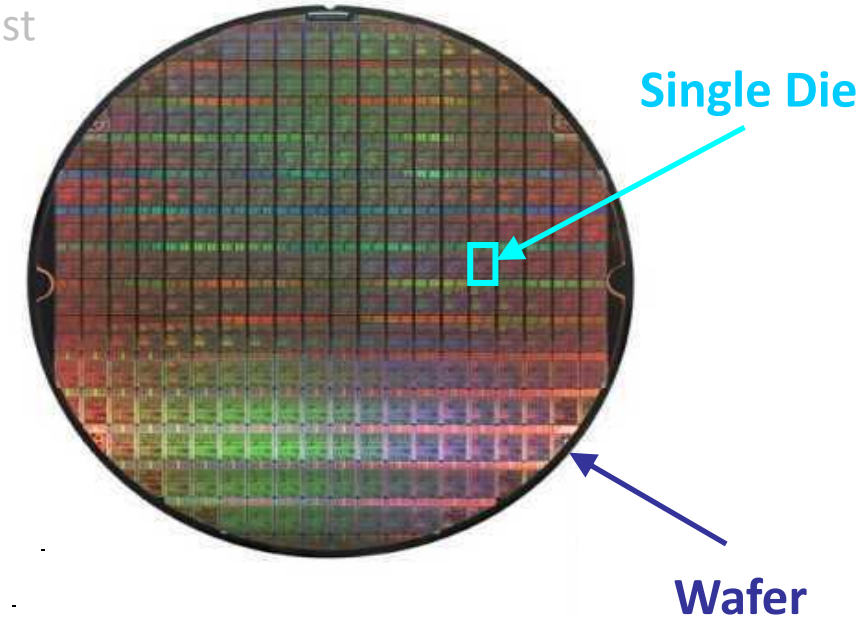
Examples of Cost Metrics (1994)

Chip	Metal layers	Line width	Wafer cost	Defects /cm^2	Area (mm^2)	Dies /wafer	Yield	Die cost
386DX	2	0.90	$900	1.0	43	360	71%	$4
486DX2	3	0.80	$1200	1.0	81	181	54%	$12
PowerPC 601	4	0.80	$1700	1.3	121	115	28%	$53
HP PA 7100	3	0.80	$1300	1.0	196	66	27%	$73
DEC Alpha	3	0.70	$1500	1.2	234	53	19%	$149
Super SPARC	3	0.70	$1700	1.6	256	48	13%	$272
Pentium	3	0.80	$1500	1.5	296	40	9%	$417

Device Metrics

Performance Metrics of a Digital Chip:

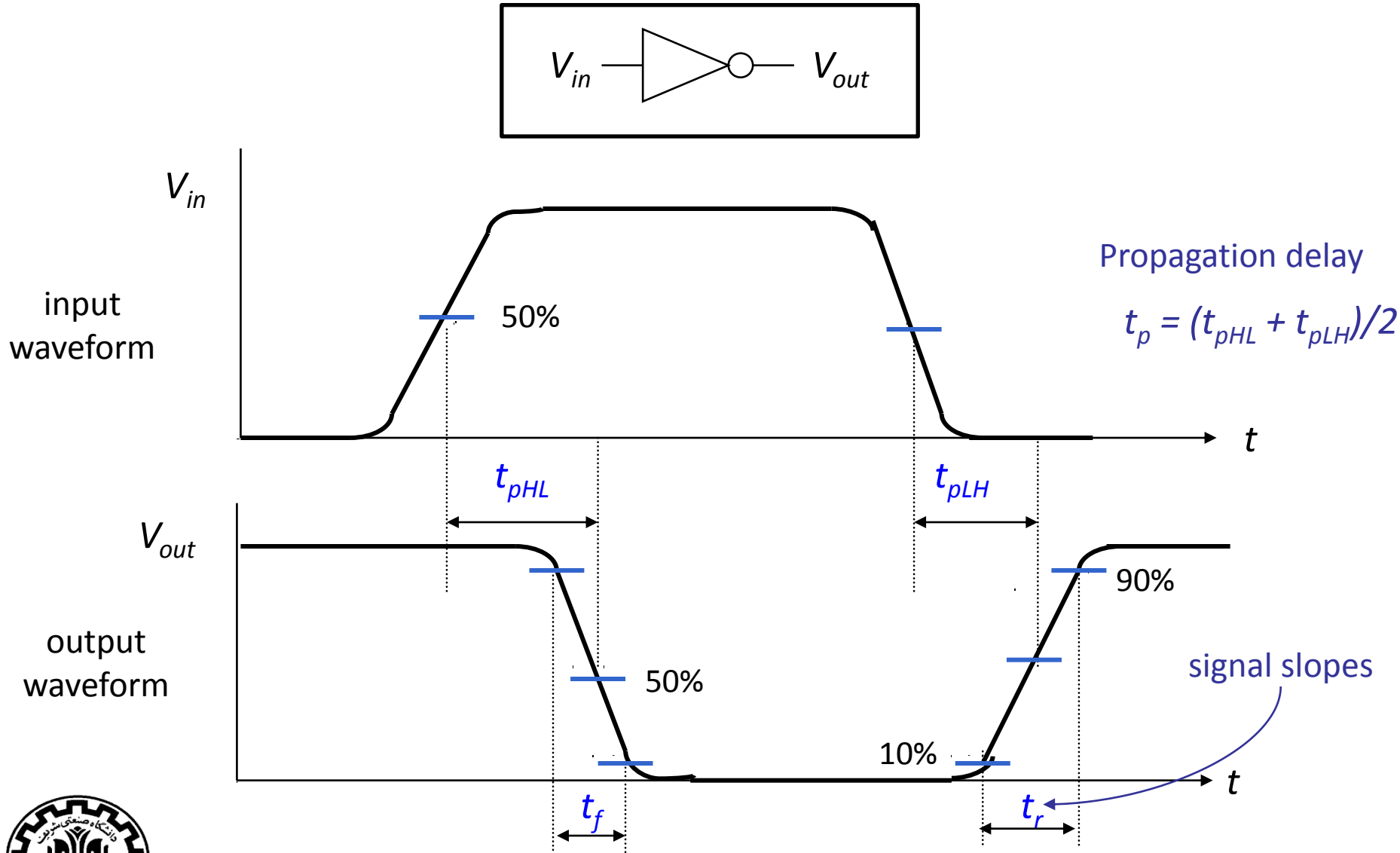
- ❑ Cost
  - ➢ NRE (fixed) costs - design effort
  - ➢ RE (variable) costs - cost of parts, assembly, test
- ❑ **Speed**
  - ➢ Delay (ns) → Operating Frequency (MHz)
- ❑ Power Dissipation
- ❑ Energy to Perform a Function
  - ➢ Energy per bit (nJ/b)
- ❑ Reliability
  - ➢ Noise immunity
  - ➢ Noise margin
- ❑ Scalability
  - ➢ Larger Designs
- ❑ Time-to-Market



Performance

- Frequency of operation: 1/T
  - Dependent on the propagation delay of signal through the logic
  - Time to get the data out/in of the registers
  - Clock uncertainty
- t_p is a typical measure (not an accurate one)
  - the delay experienced by a signal when passing through a gate
  - 50% transition points of the input and output waveforms
    - Two types LH and HL
    - Good for comparison
- Rise time/fall time affects delay
  - 10% - 90% definitions

Delay Definitions

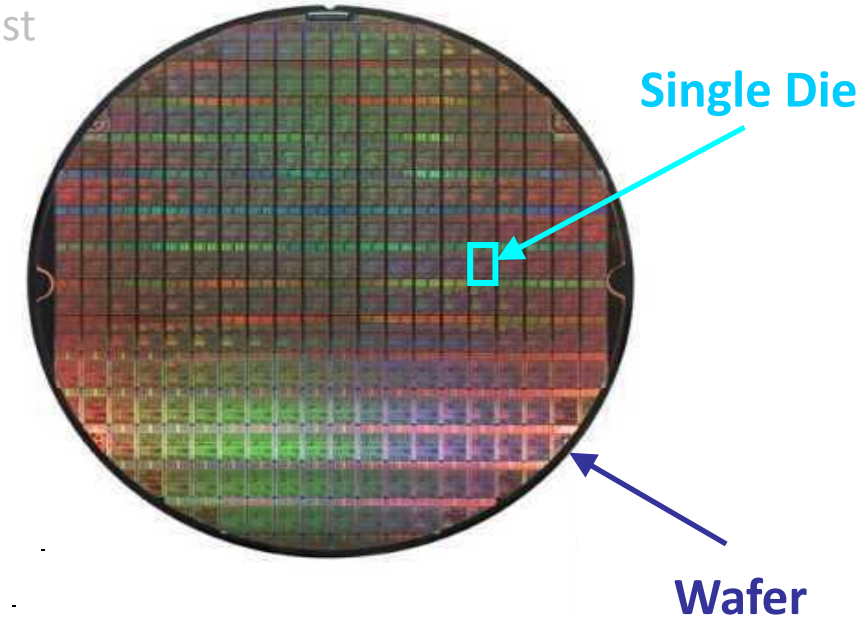


© M. Shabany, ASIC & FPGA Chip Design

Device Metrics

Performance Metrics of a Digital Chip:

- ❑ Cost
  - ➢ NRE (fixed) costs - design effort
  - ➢ RE (variable) costs - cost of parts, assembly, test
- ❑ Speed
  - ➢ Delay (ns) → Operating Frequency (MHz)
- ❑ **Power Dissipation**
- ❑ **Energy to Perform a Function**
  - ➢ Energy per bit (nJ/b)
- ❑ Reliability
  - ➢ Noise immunity
  - ➢ Noise margin
- ❑ Scalability
  - ➢ Larger Designs
- ❑ Time-to-Market



Power Consumption

- **Peak transient power**
  - Power line sizing, decoupling, etc.

$$P_{peak} = i_{peak} V_{supply} = \max[p(t)]$$

- **Average power**
  - Battery current delivery, cooling system
- **Static power vs. Dynamic power**
  - Static current ➔ no computation, etc
  - Dynamic current ➔ Switching on/off the gates
    - The higher the number of switching events, the higher the dynamic power consumption
- **Dynamic Energy** ➔ Amount of energy that is needed to be spent to do a job
  - Time is no matter

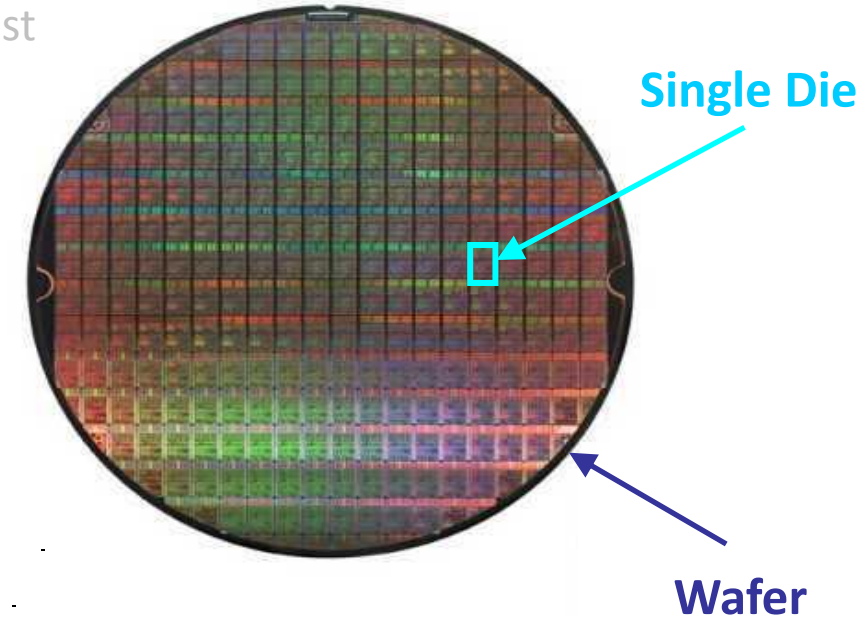
Power and Energy Dissipation

- Propagation delay and the power consumption of a gate are related
- Propagation delay is (mostly) determined by the speed at which a given amount of energy can be stored on the gate capacitors
  - the faster the energy transfer (higher power dissipation) the faster the gate
- For a given technology and gate topology, the product of the power consumption and the propagation delay is a constant
  - Power-delay product (PDP) – energy consumed by the gate per switching event
- An ideal gate is the one that is fast and consumes little energy, so the ultimate quality metric is
  - Energy-delay product (EDP) = power-delay

Device Metrics

Performance Metrics of a Digital Chip:

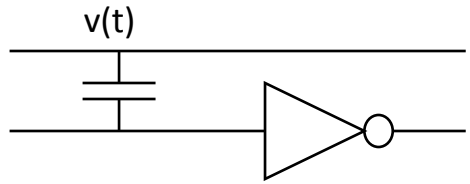
- ❑ Cost
  - ➢ NRE (fixed) costs - design effort
  - ➢ RE (variable) costs - cost of parts, assembly, test
- ❑ Speed
  - ➢ Delay (ns) → Operating Frequency (MHz)
- ❑ Power Dissipation
- ❑ Energy to Perform a Function
  - ➢ Energy per bit (nJ/b)
- ❑ **Reliability**
  - ➢ Noise immunity
  - ➢ Noise margin
- ❑ Scalability
  - ➢ Larger Designs
- ❑ Time-to-Market

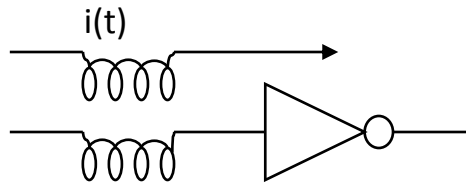


Reliability: Noise in Digital Integrated Circuits

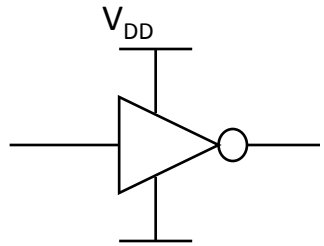
❖ **Noise** : unwanted variations of voltages and currents at the logic nodes

- ❑ from two wires placed side by side
  - ➢ Capacitive coupling
    - ▪ Voltage change on one wire can influence signal on the neighboring wire
    - ▪ Cross talk
  - ➢ Inductive coupling
    - ▪ Current change on one wire can influence signal on the neighboring wire



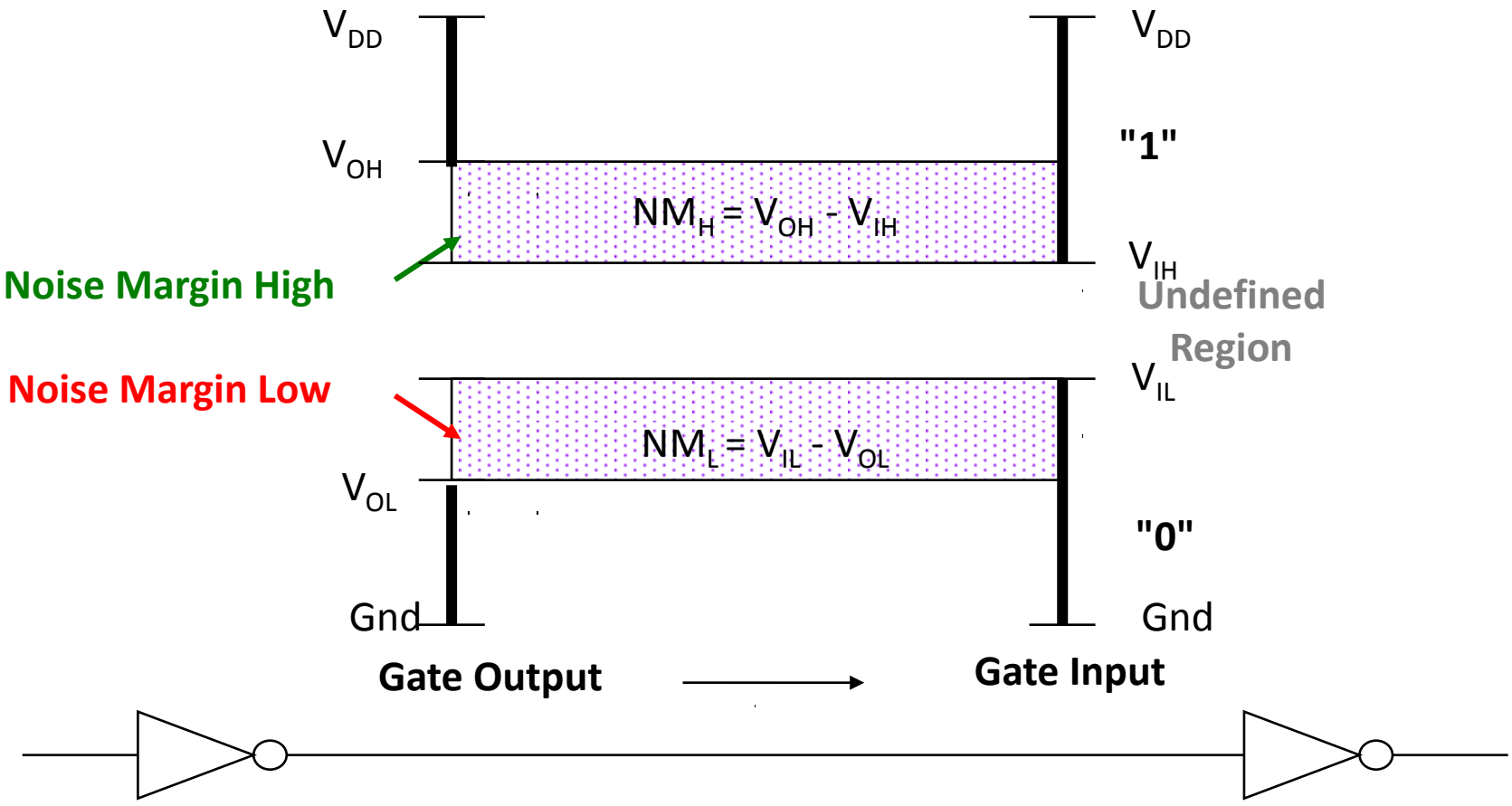


- ❑ from noise on the power and ground supply rails
  - ➢ May influence signal levels in the gate



Noise Margins

- For robust circuits, want the “0” and “1” intervals to be as large as possible



Noise margin represents the levels of noise that can be sustained when gates are cascaded

© M. Shabany, ASIC & FPGA Chip Design

Noise Immunity

- **Noise margin** expresses the ability of a circuit to overpower a noise source
  - Noise sources: supply noise, cross talk, interference, offset
- Absolute noise margin values are deceptive
  - Floating node is more easily disturbed than a node driven by a low impedance (in terms of voltage)
- **Noise immunity** expresses the ability of the system to process and transmit information correctly in the presence of noise (noise rejection)
- For good noise immunity, the signal swing (i.e., the difference between V_{OH} and V_{OL}) and the noise margin have to be large enough to overpower the impact of fixed sources of noise

Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

MOS Device Theory

❑ Bipolar Junction Transistor (BJT)

Mostly Analog

- ❑ **Small current in Base drives a larger current b/w Emitter & Collector.**
  - ➢ Quiescent power dissipation due to the Base current
  - ➢ High power dissipation limits the number of transistors on a single chip
  - ➢ Not suitable for Very Large Scale Integration (VLSI)

❑ Metal Oxide Semiconductor Field Effect Transistors (MOSFET)

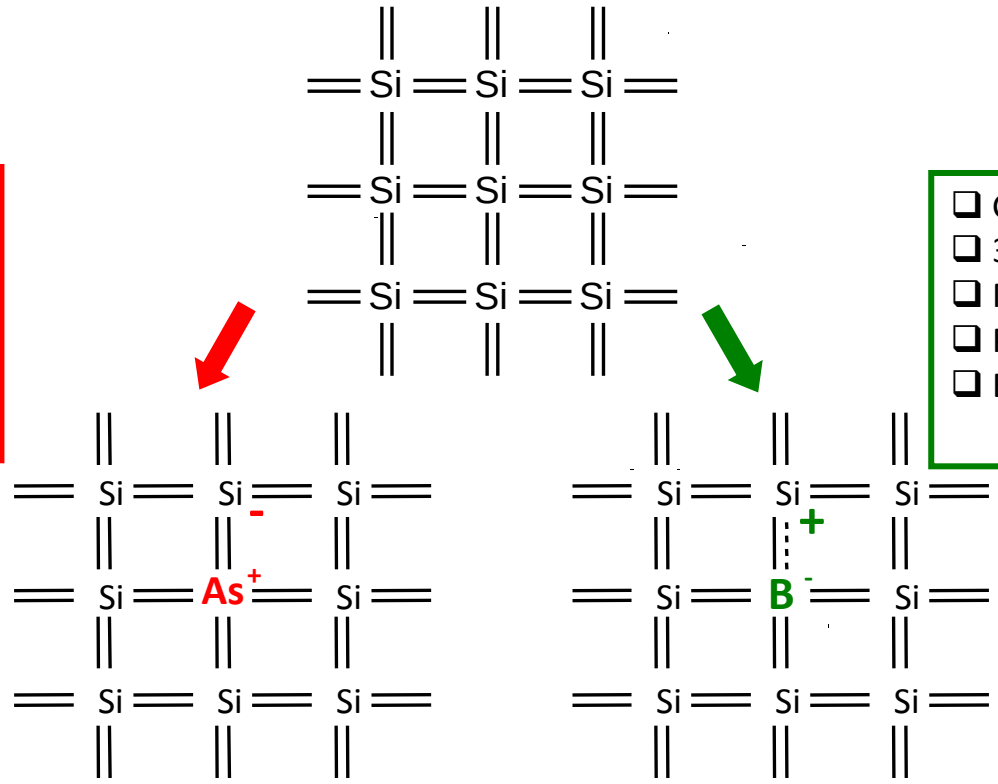
Mostly Digital

- ❑ **Come with almost zero control current (Gate voltage controls the drain current)**
  - ➢ Higher integration
  - ➢ Much lower power consumption than BJT
  - ➢ Come in two flavors : n-MOS (n-type dopants), p-MOS (p-type dopants)

- ❑ **Complementary Metal Oxide Semiconductor (CMOS)**
  - ➢ Utilizing both n-MOS and p-MOS transistors

MOS Device Theory

- Transistors are built on a silicon substrate
- Silicon is a Group IV material → covalent bonds with 4 adjacent atoms
- Silicon is a poor conductor → can be raised by adding dopants



n-type

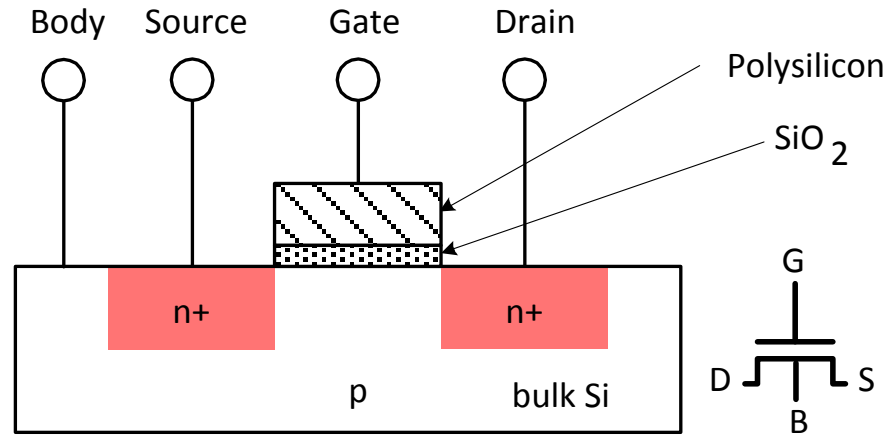
- Group V dopants
- Five valence electrons
- Extra electron free to move
- Negative carrier
- Example:
  - Arsenic, Phosphorus

p-type

- Group III dopants
- 3 valence electrons
- Missing electron(hole) free to move
- Positive carrier
- Example:
  - Boron

MOS Device Theory

- Four terminals:
  1. Gate
  2. Source
  3. Drain
  4. Body

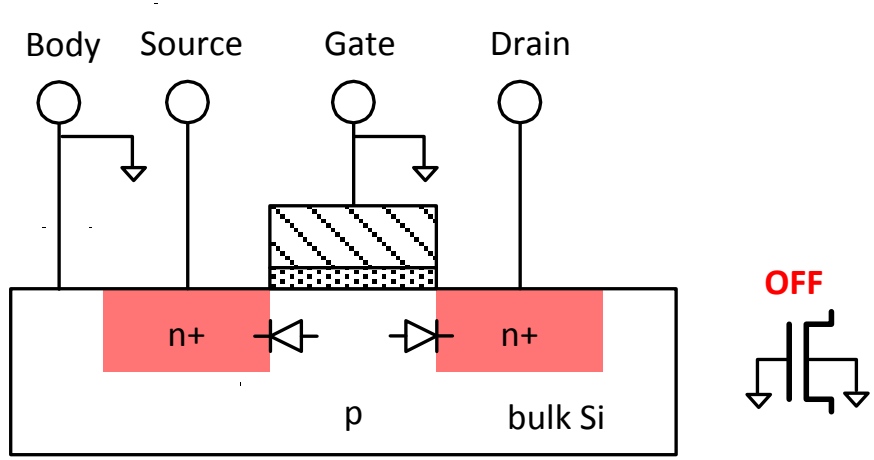


nMOS

- Consists of :
  - Gate (Metal (old), Polysilicon (now))
  - Insulating layer (SiO_2 (oxide- glass))
  - Source (n+ in nMOS, p+ in pMOS)
  - Drain (n+ in nMOS, p+ in pMOS)
  - Body (conductor)
  - n+: Heavily doped n-type
  - P+: Heavily doped p-type

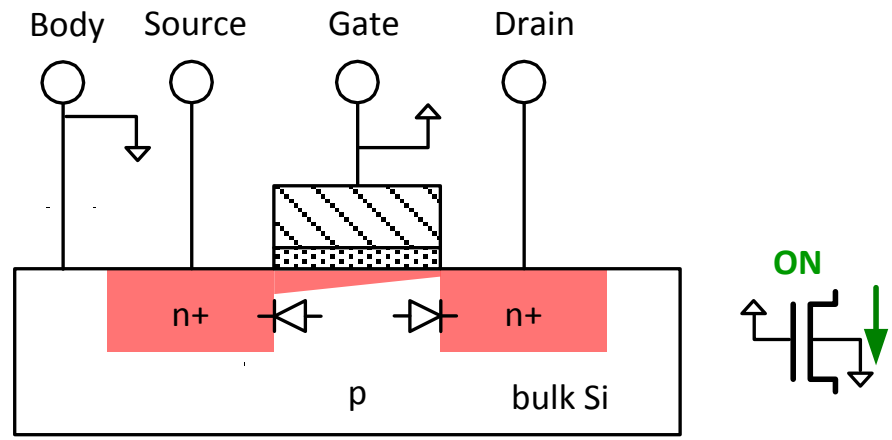
nMOS Transistors

- Gate–oxide–body stack looks like a capacitor
- **Body** is commonly tied to ground (0 V)
- **Gate** at low voltage:
  - Body is at low voltage
  - Source-body diode is OFF
  - Drain-body diode is OFF
  - No current flows
  - Transistor is **OFF**



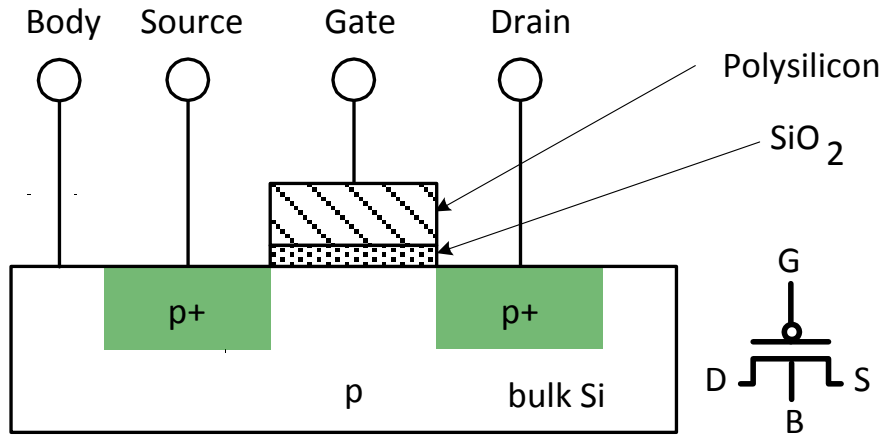
nMOS Transistors

- **Gate** at high voltage:
  - Positive charge on gate
  - Negative charge attracted to body
  - Inverts a channel under gate to n-type
  - Current flow in this channel b/w source and drain when drain voltage is nonzero
  - Transistor is **ON**



pMOS Transistors

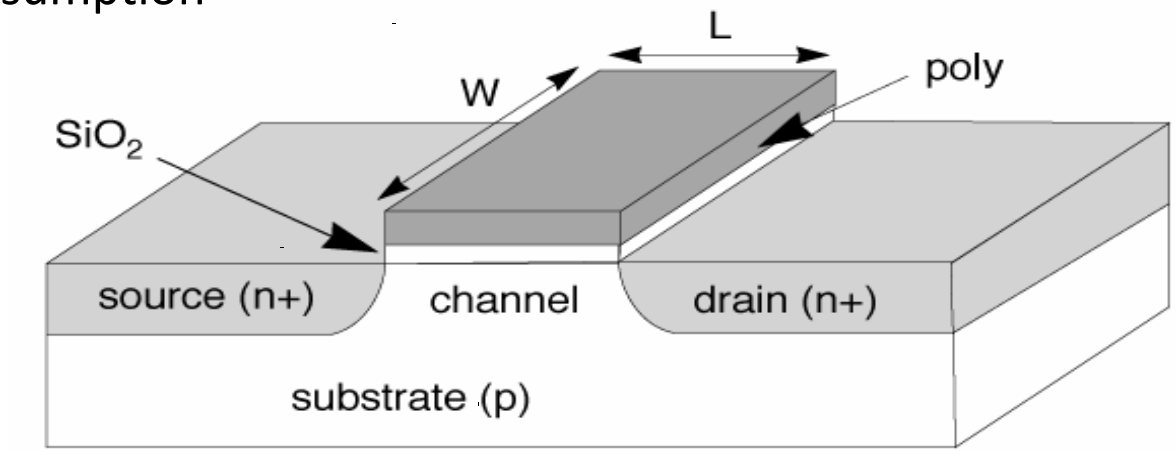
- Similar to nMOS with reversed doping and voltages
- Body is commonly tied to high voltage (VDD)
- Gate low: transistor **ON**
- Gate high: transistor **OFF**
- Bubble indicates inverted behavior



pMOS

MOS Transistors

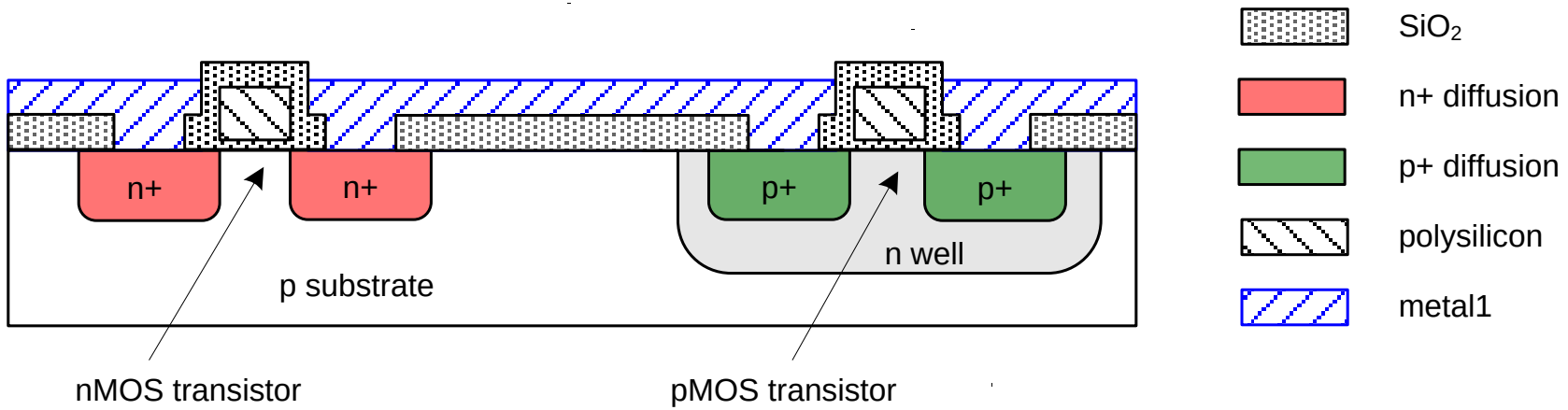
- L is the channel length
- L : Process parameter, technology
- Smaller L → Faster transistors → higher speed circuits
- Typical process values: 0.35µm, 0.18µm, 0.13µm, 90nm, 60nm, …
- VDD decreases by technology
  - 1.5 V for 0.18 µm
  - 1.2 V for 0.13 µm
- Lower VDD saves power consumption
- GND = 0 V



CMOS

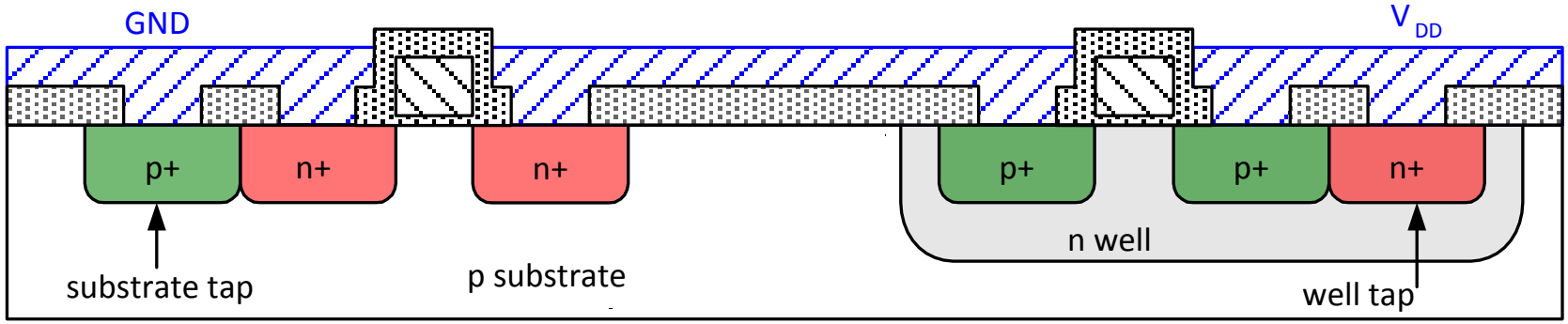
- Silicon wafer is the base material
  - Diameter: 10-30 cm
  - Thickness: 1 mm

- **CMOS** : both nMOS and pMOS transistors fabricated on a single wafer
- **Wells** : special regions to separate bulks of nMOS and pMOS

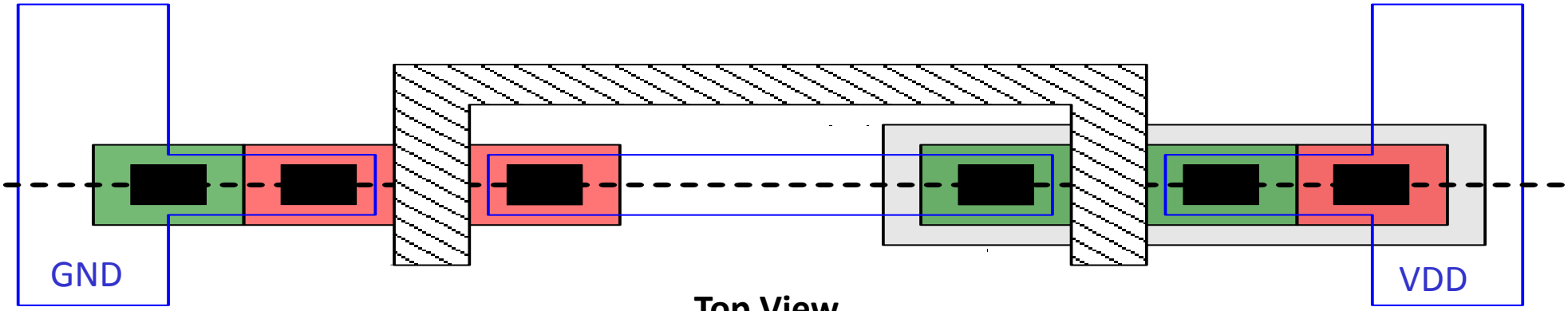


CMOS

- Substrate must be tied to GND and n-well to VDD
- Poor connection of metal to lightly-doped semiconductor (Shottky Diode)
- Use heavily doped well and substrate contacts

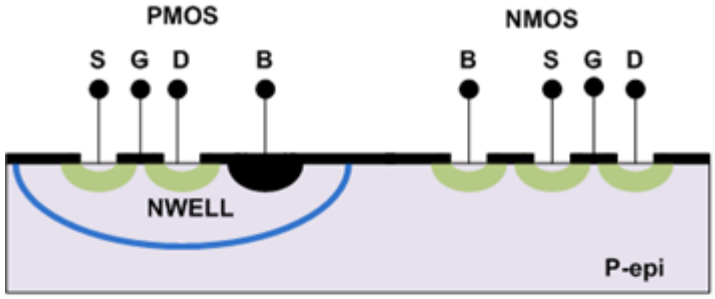


Cross Section along Dashed Line



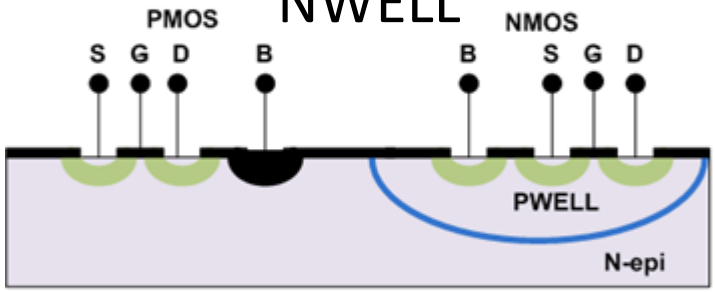
Top View

CMOS



NWELL

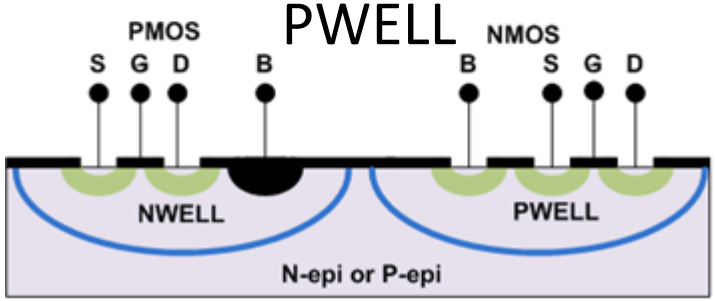
- ارزان
- اتصال subبه زمین مشترک



PWELL

اتصال subبه بزرگترین منبع

ارزان



Twin-WELL

گران

- دو درجه آزادی
- ایزوله از sub

Course Outline

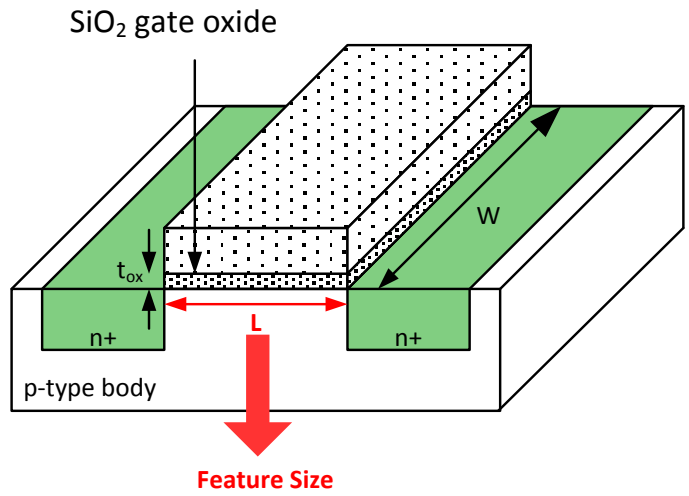
- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Layout

- Chips are specified with set of masks
- Minimum dimensions of masks determine transistor size (and hence speed, cost, and power)
- "Feature size"

  L_{min}= distance between source and drain (channel)

  (minimum width of Polysilicon)

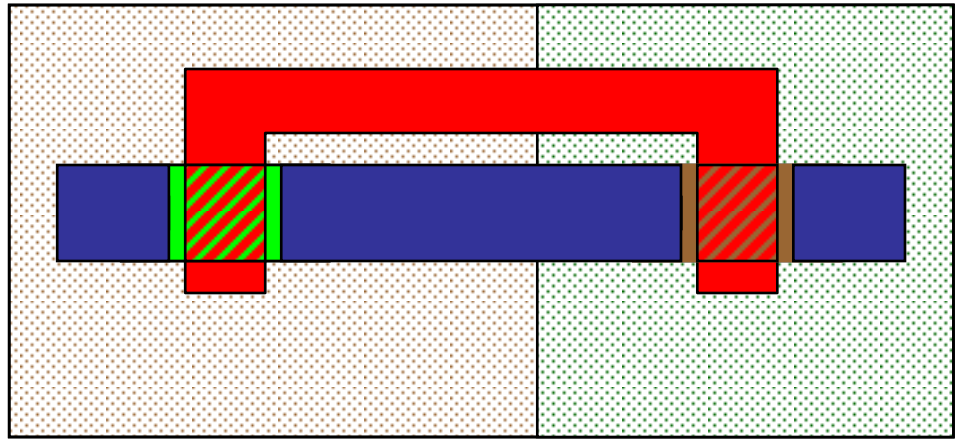


- Feature size improves 30% every 3 years or so
- Can integrate 2× more functions per chip → ~2× less cost per function
- Normalize for feature size when describing design rules ($\lambda = L_{\min} / 2$)
  - E.g., λ = 90nm in 0.18 µm process

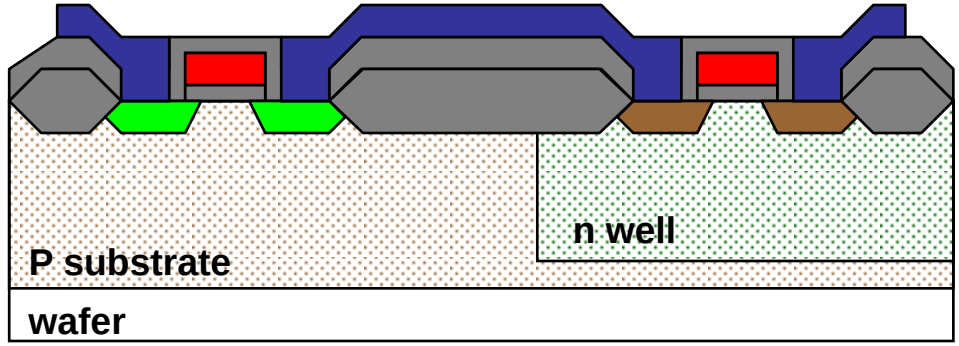
Layout Layers

- Each layout consists of various levels described by different colors.

Top View



Cross Section View

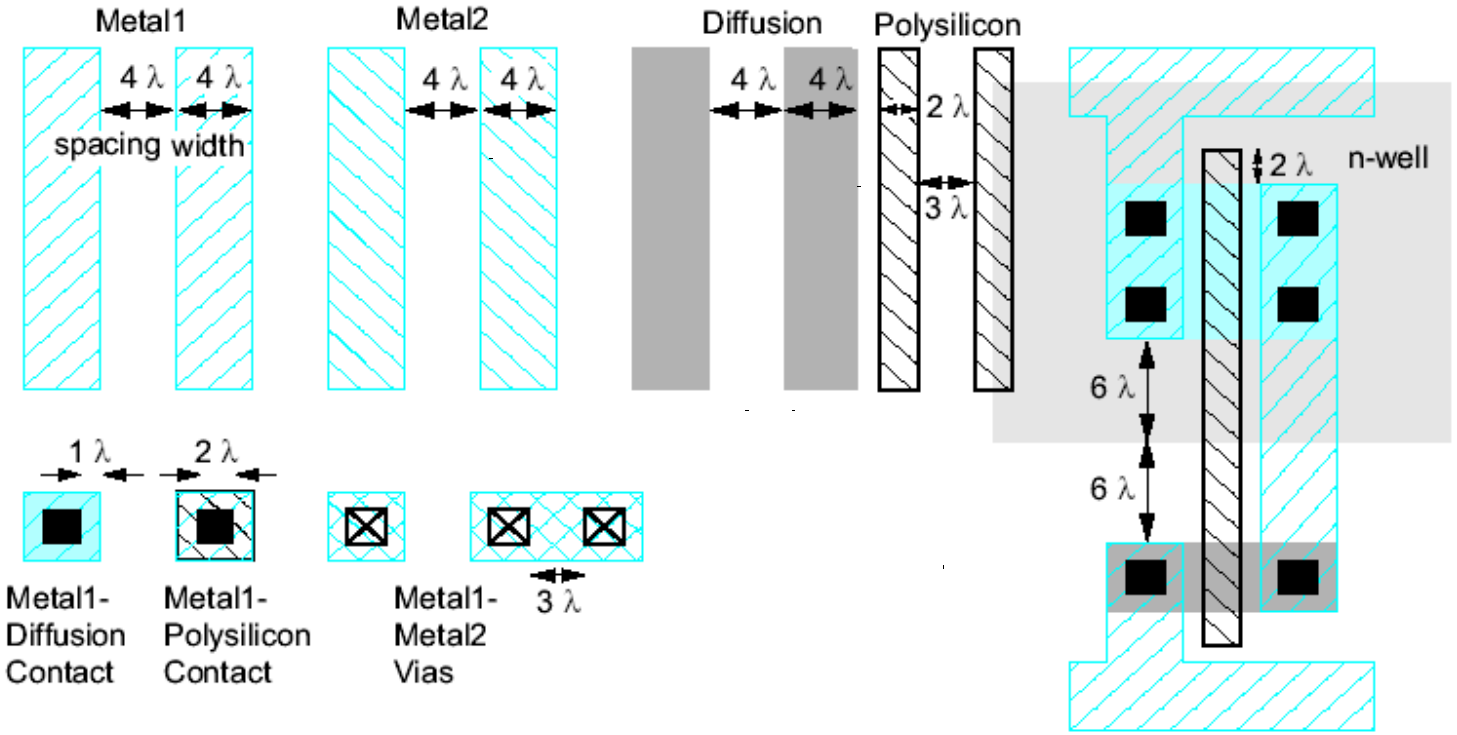


Design Rules

- Circuit engineer designs a circuit
- Process engineer fabricates the design

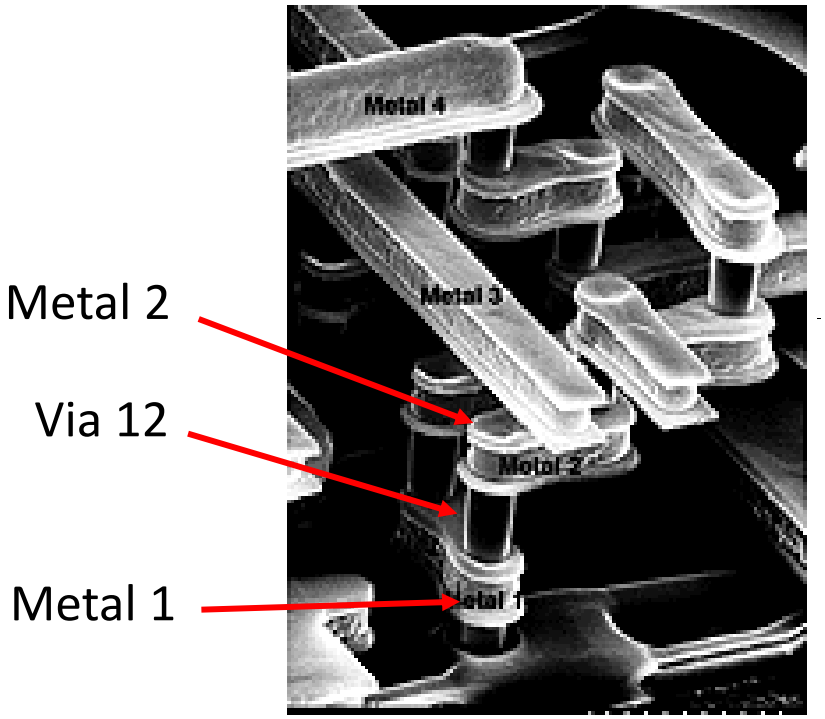
Deign Rule Check (DRC)
is the interface between them

- DRC : constraints on patterns in terms of minimum width and separation
- DRC: guarantees that the circuit to be manufacturable

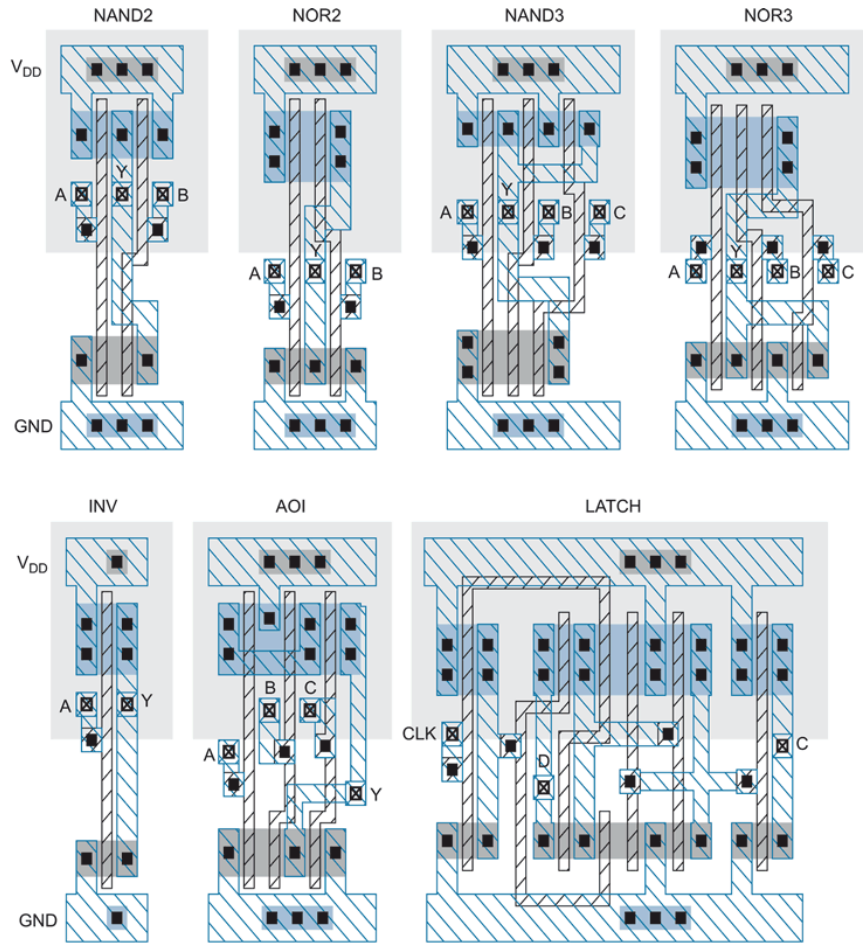


Digital VLSI Layout

- Designed and laid-out standard cells
- They are placed & abutted in a chip

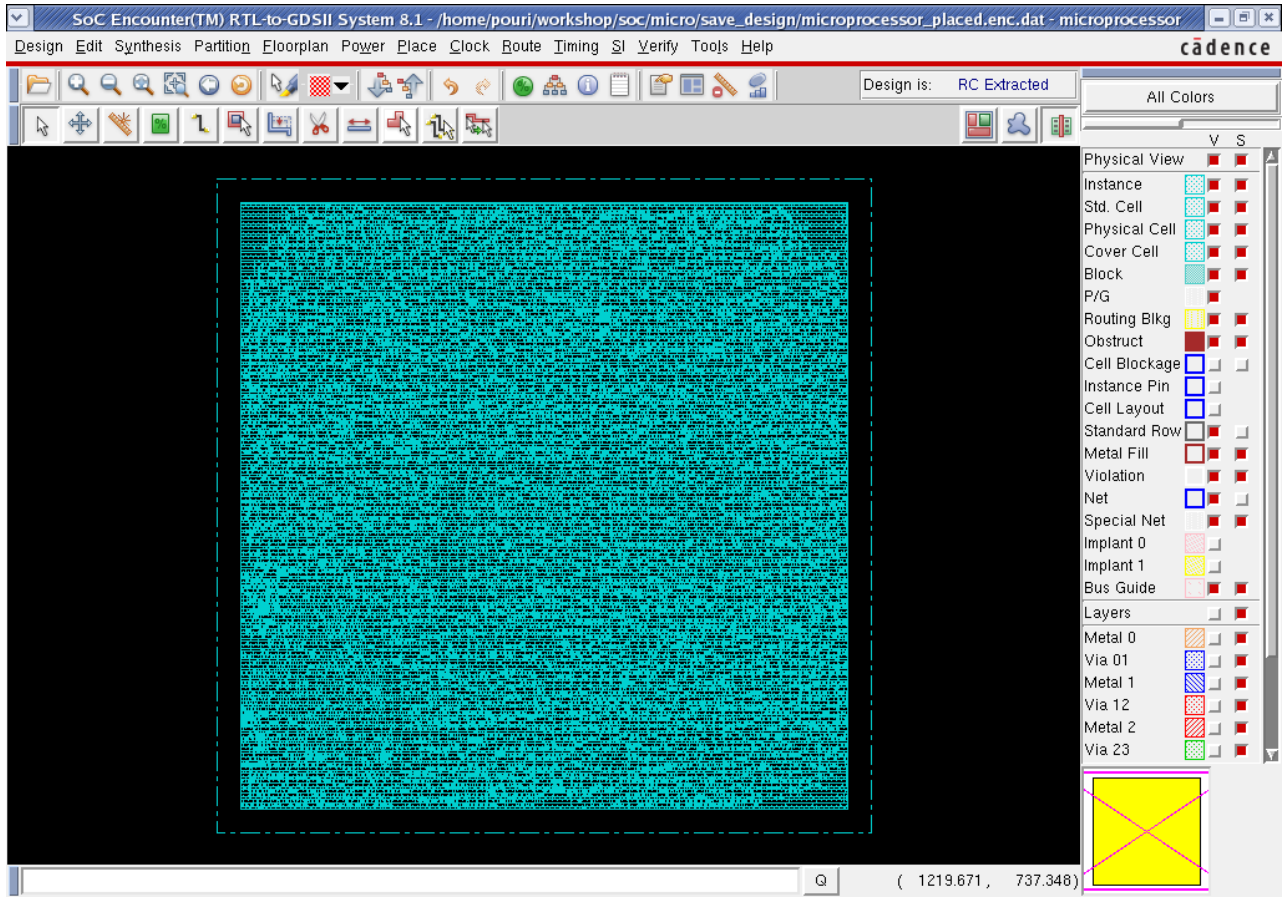


Several metal layers used for routing



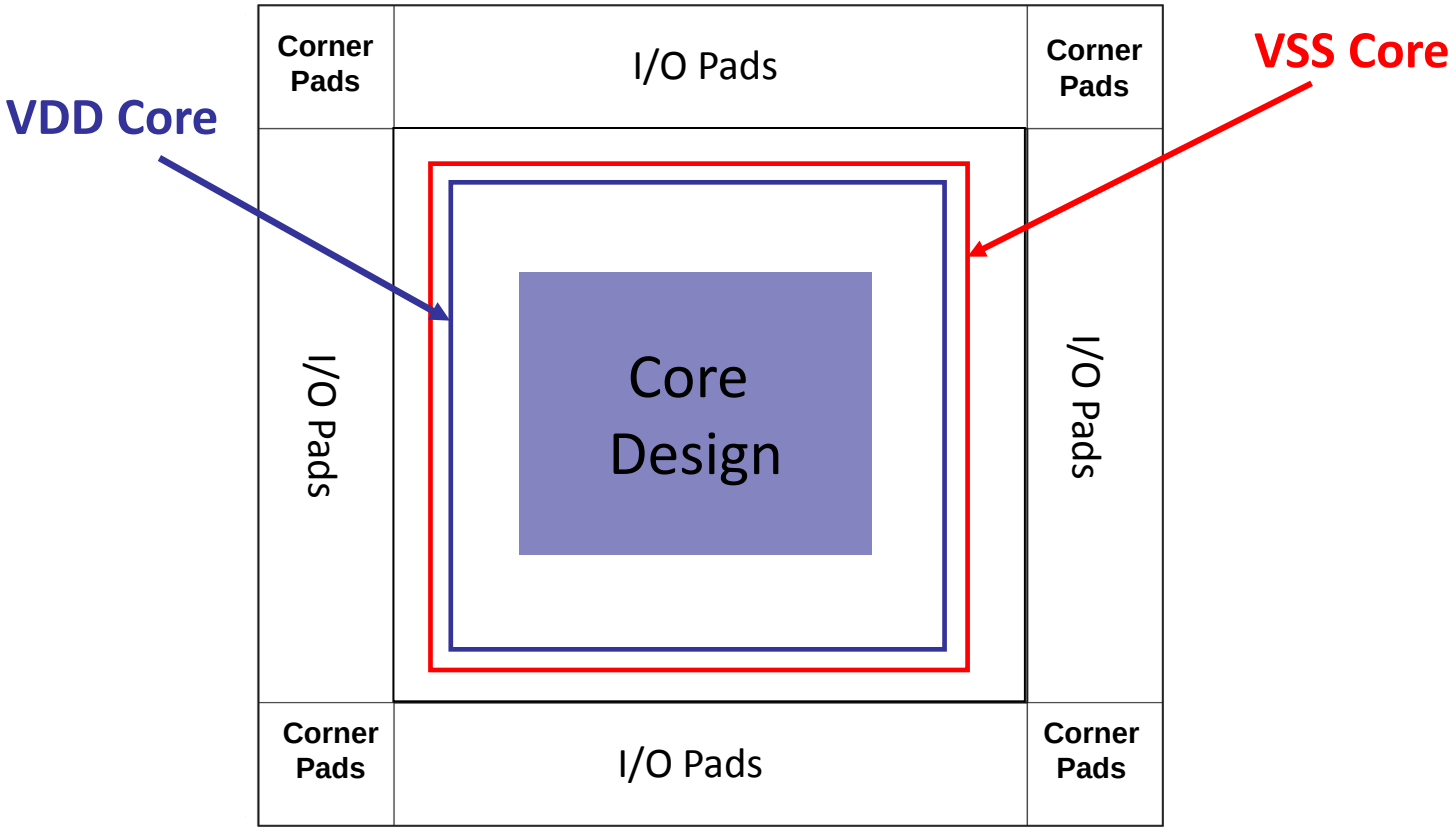
Digital VLSI Layout

- Standard cells are automatically placed and routed using different metal layers through the corresponding CAD tools

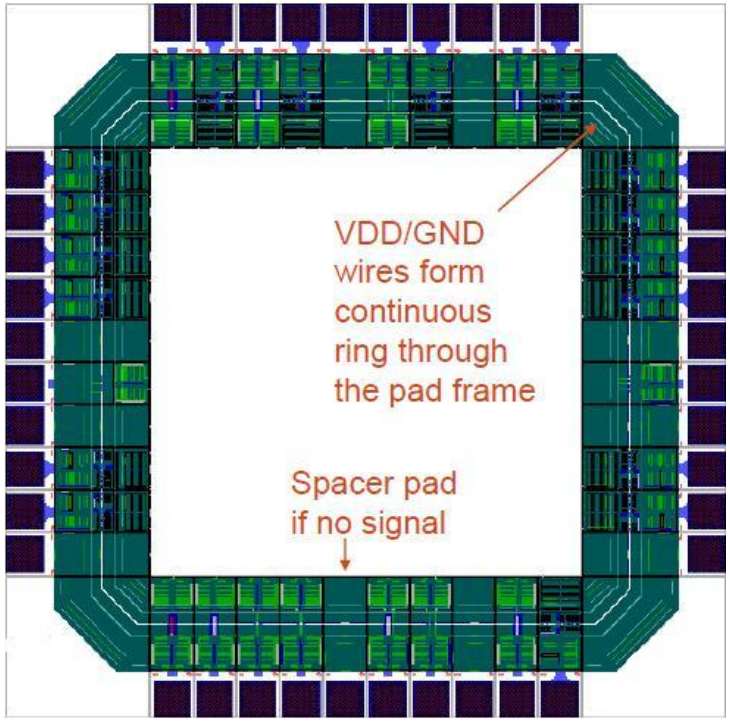


Chip Floorplan

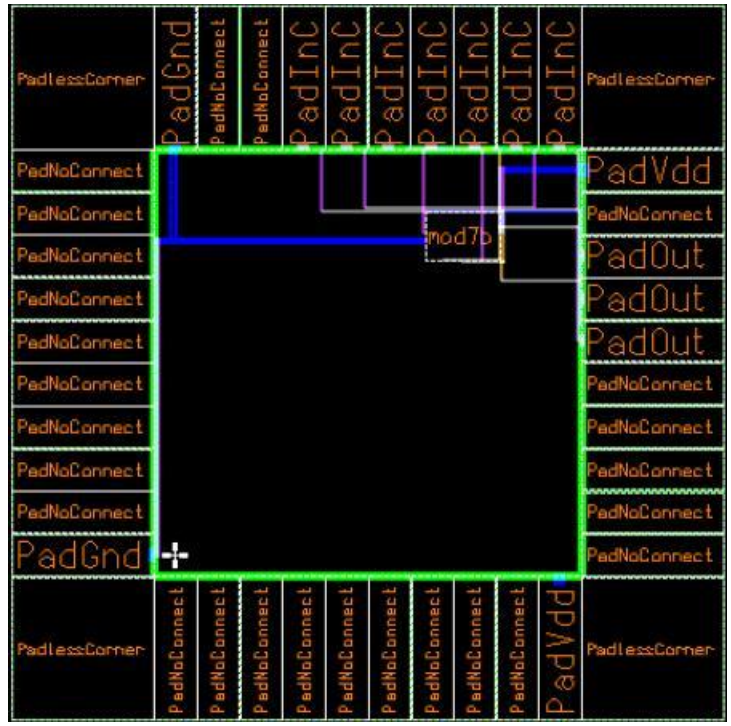
- Start with the pin count (# of I/O and VDD, VSS pads)
- Pads are already designed (provided in the design kit)



Chip Floorplan : Pads



Layout View



Design View

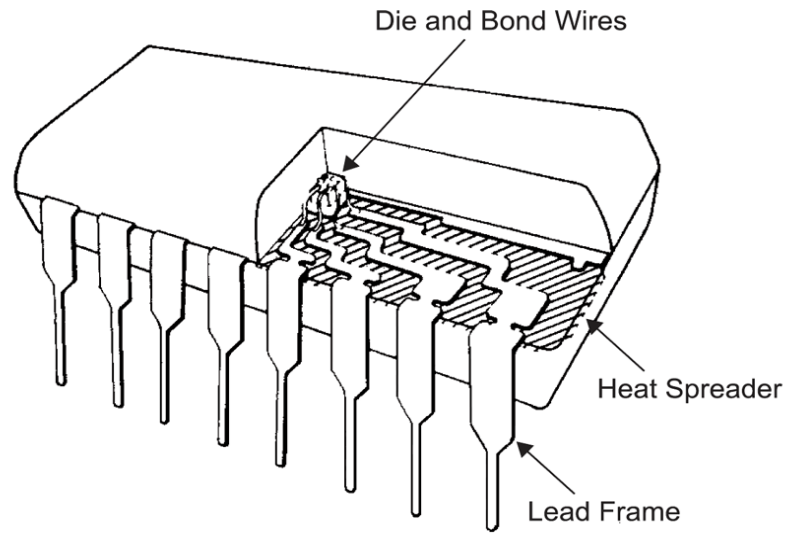
Course Outline

- Course Outline
- Introduction to ASIC/FPGA IC Design
  - Integrated Circuits (IC) History
  - Digital Design vs. Analog Design
  - ASIC vs. FPGA
  - Design Abstraction and Metrics
  - CMOS as the building block of Digital ASICs
  - Layout
  - Packaging

Packaging

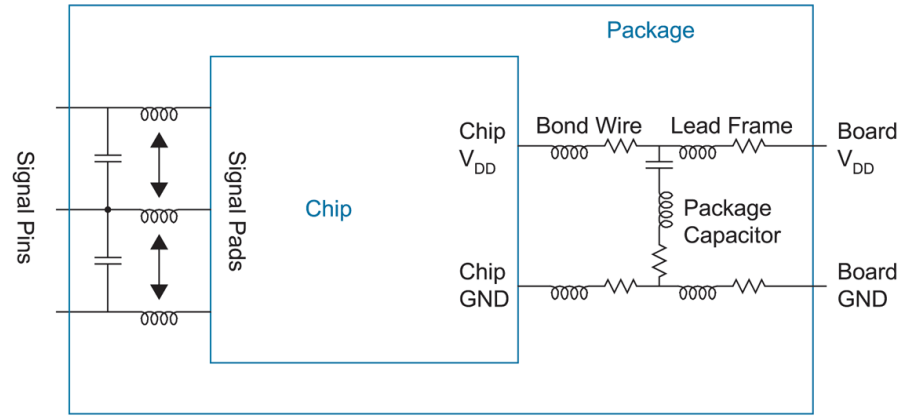
❑ Package:

- ➢ Die interface to outside world
- ✓ Removes heat from chip
- ✓ Mechanical support
- ✓ Protects die against humidity
- ✗ Introduces delay/parasitics to the chip

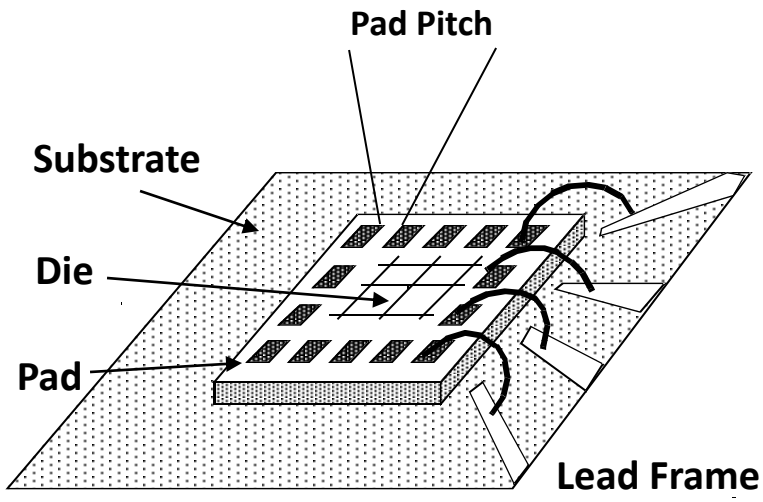


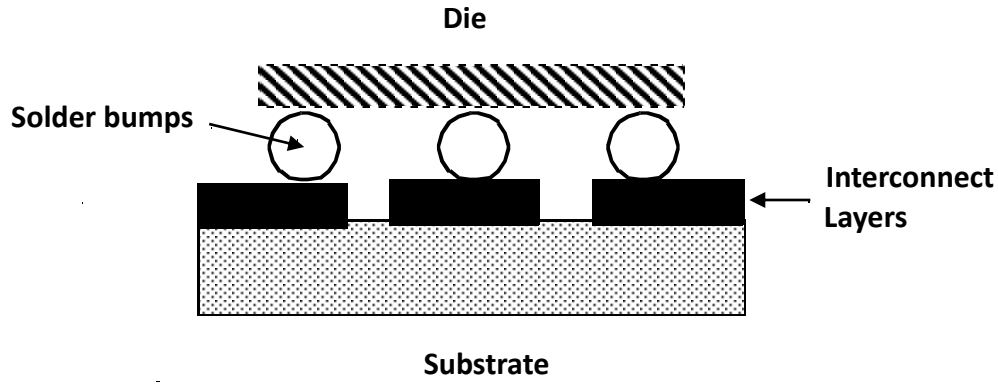
❑ Advanced Package Requirements:

- ➢ **Electrical**: Low Parasitics
- ➢ **Mechanical**: Reliable and Robust
- ➢ **Thermal**: Efficient Heat Removal
- ➢ **Economical**: Cheap



Die to Package Substrate





Wire Bonding

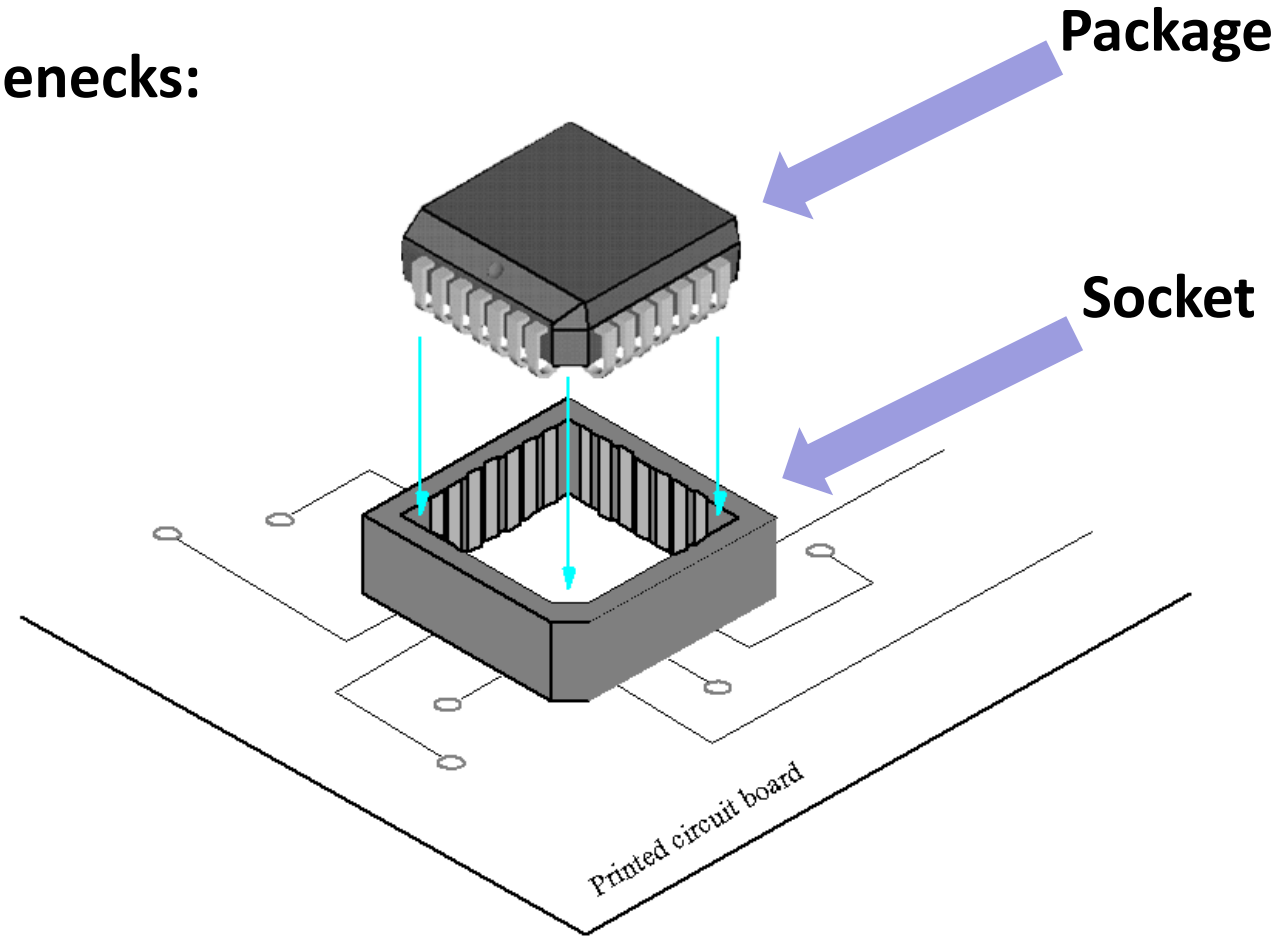
- Small pin count
- Must be attached serially
- Large inductance
- Cross talk b/w wires
- Pads on four sides of the chip

Flip-Chip Bonding

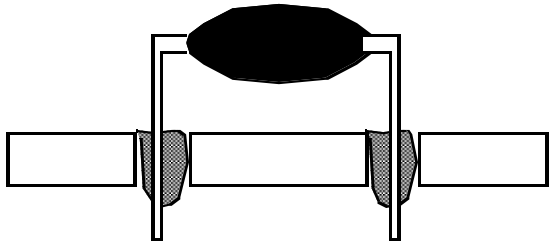
- Large pin count
- Connections made simultaneously
- Superior electrical performance
- Small cross talk
- Pads at any position on the chip

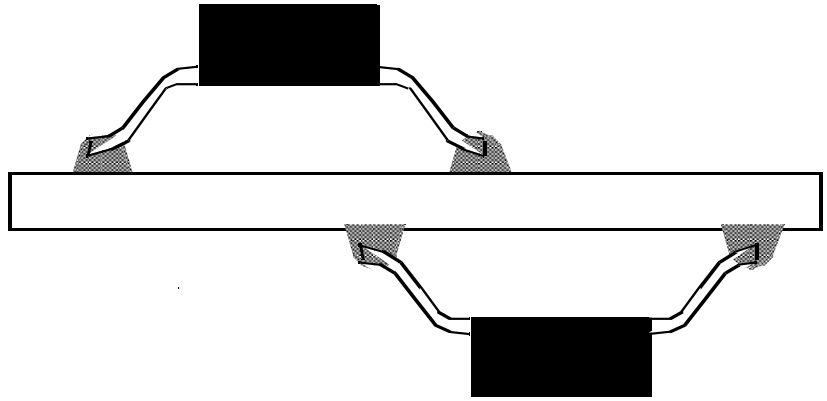
Packaging

- **Package-Socket-PCB interconnection:**
- **Performance bottlenecks:**
  - Chip itself
  - Package
  - Socket
  - Board



Package/Socket to Board Interconnection





Through-Hole Mounting

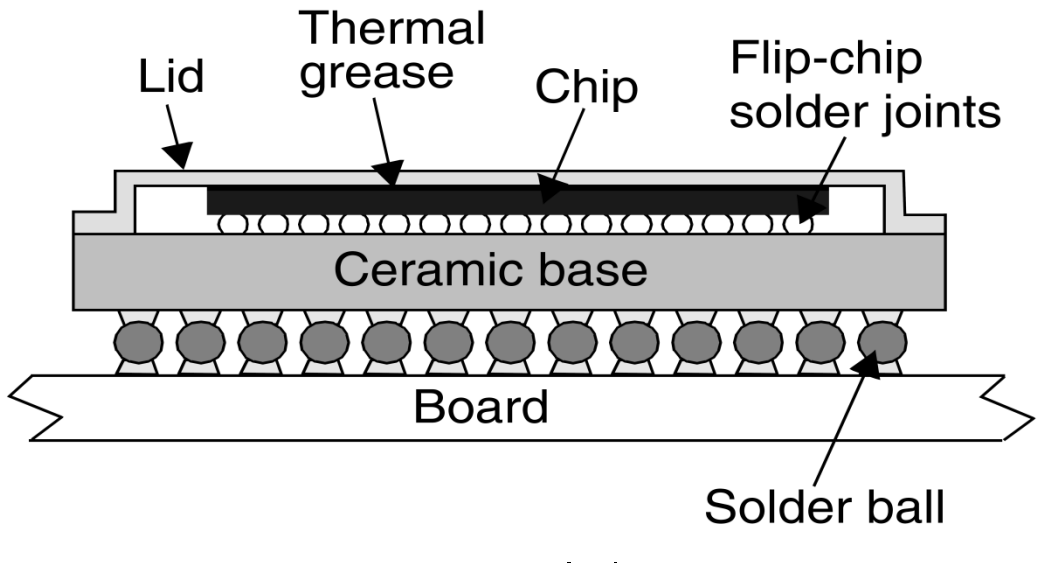
- ✓ Mechanically reliable
- ✗ Low package density
- ✗ Limits routing on the board
- Dualin-Line (DIP) (up to 64 pins)
- PGA (up to 400 pins)

Surface Mount

- ✓ More wiring space
- ✓ Higher package density
- ✓ Chips at both sides of the board
- ✗ Weak chip-board connection
- ✗ Non-accessible pins for testing

Package/Socket to Board Interconnection

- For very large pin-counts, even surface-mount packaging is not enough!
  - Ball Grid Array (BGA)



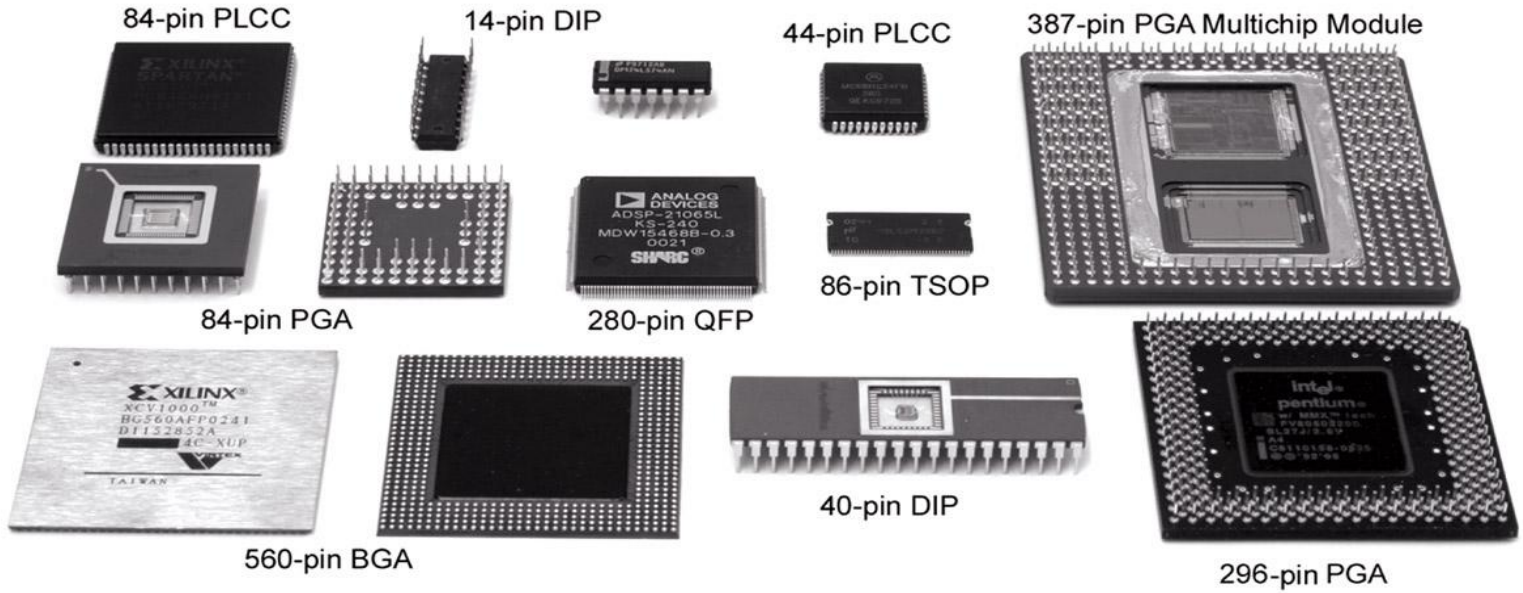
Solder bumps are used to connect both the die to the package substrate and the package to the board

Packaging Material/Type

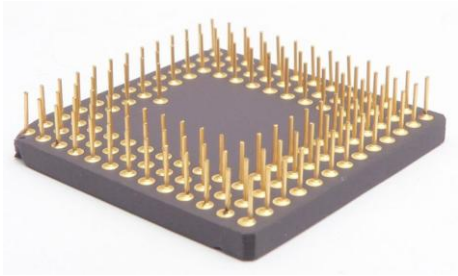
❑ Package Material:

- ➢ Plastics (cheaper)
- ➢ Ceramic (Better heat removal)

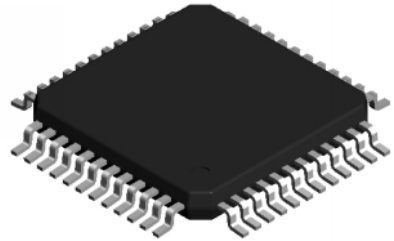
❑ Package Types:



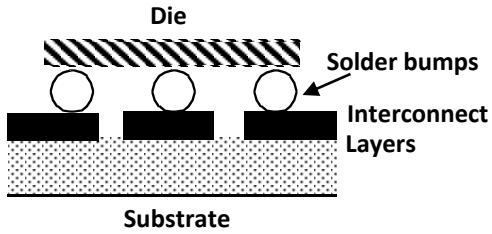
Packaging Types



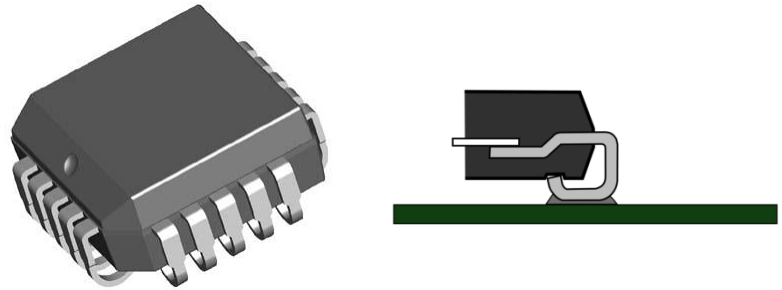
Pin Grid Array (PGA)



Quad Flat Pack (QFP)



Ball Grid Array (BGA)



Plastic Leader Chip Carrier (PLCC)

ASIC/FPGA Chip Design

HDL Coding (Verilog)

Mahdi Shabany

Department of Electrical Engineering
Sharif University of technology

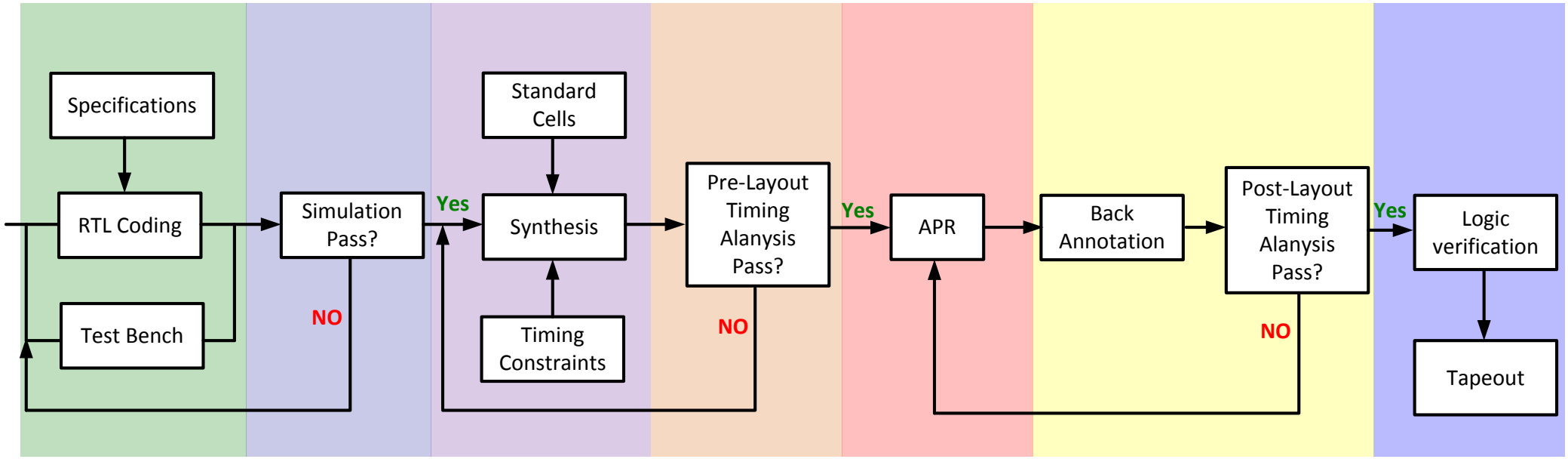
Outline

- **ASIC/FPGA Design Flow**
- **Hardware Description Language (HDL)**
  - **Verilog**
    - o Introduction
    - o Language Fundamentals
    - o Modeling Combinational & Sequential Logic Circuits
    - o Modeling Finite State Machines
    - o Verilog Operations

Outline

- **ASIC/FPGA Design Flow**
- Hardware Description Language (HDL)
  - Verilog
    - Introduction
    - Language Fundamentals
    - Modeling Combinational & Sequential Logic Circuits
    - Modeling Finite State Machines
    - Verilog Operations

ASIC/FPGA Design Flow

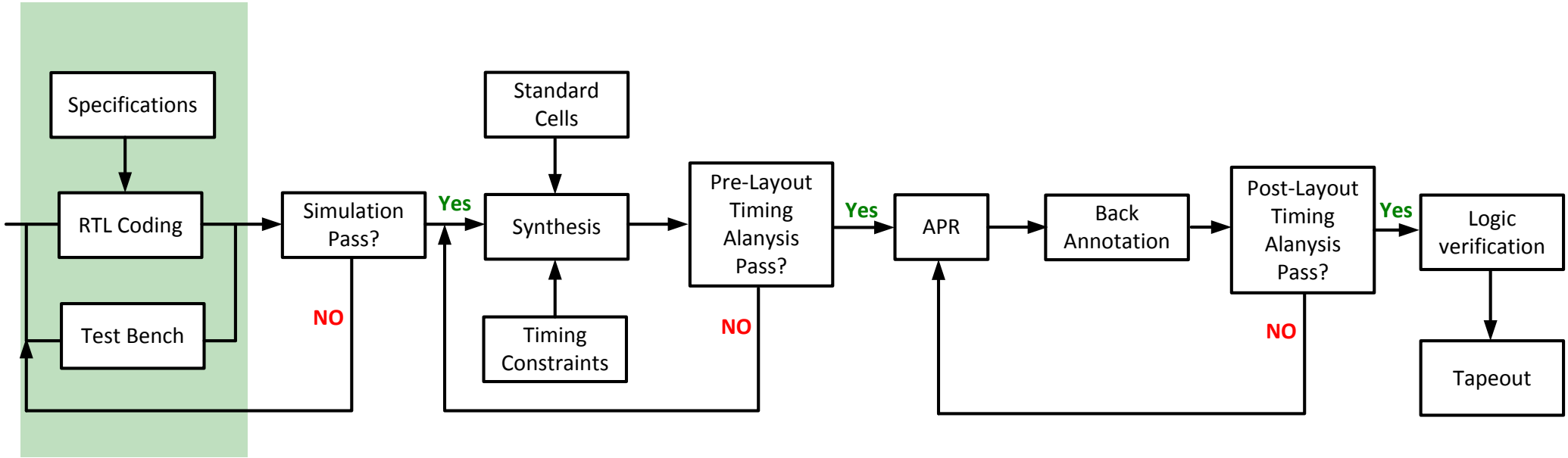


1. HDL Coding **2. Simulation** **3. Synthesis** — **Front-End**

4. Placement & routing **5. Timing Analysis & Verification** — **Back-End (Physical Design)**

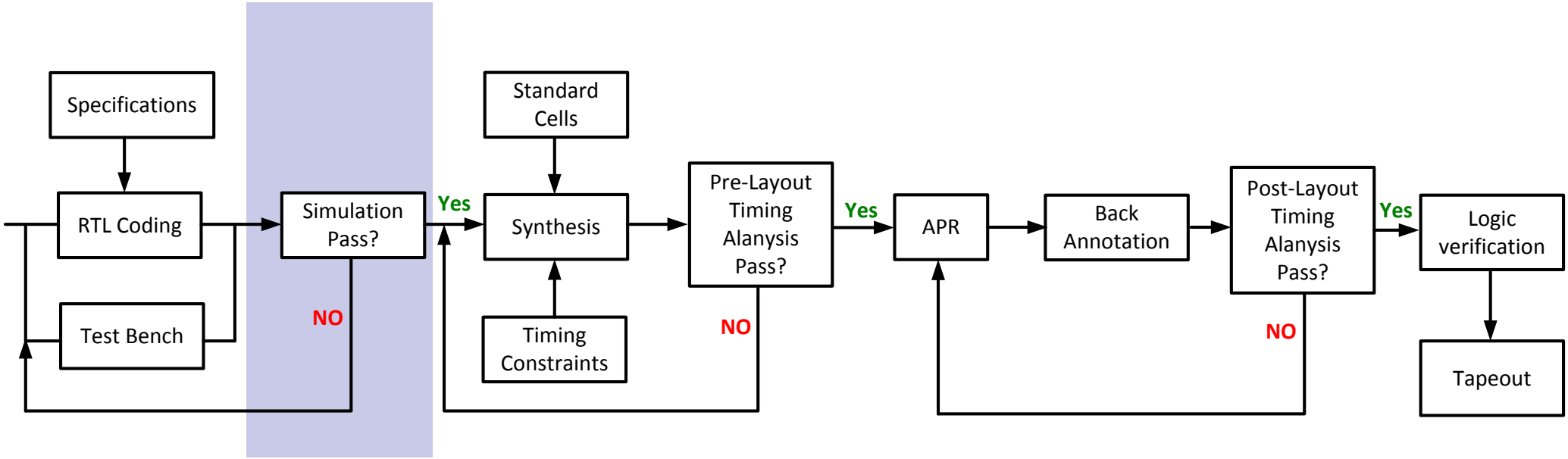
- In this course we learn all the above steps in detail for ASIC

1. HDL Coding



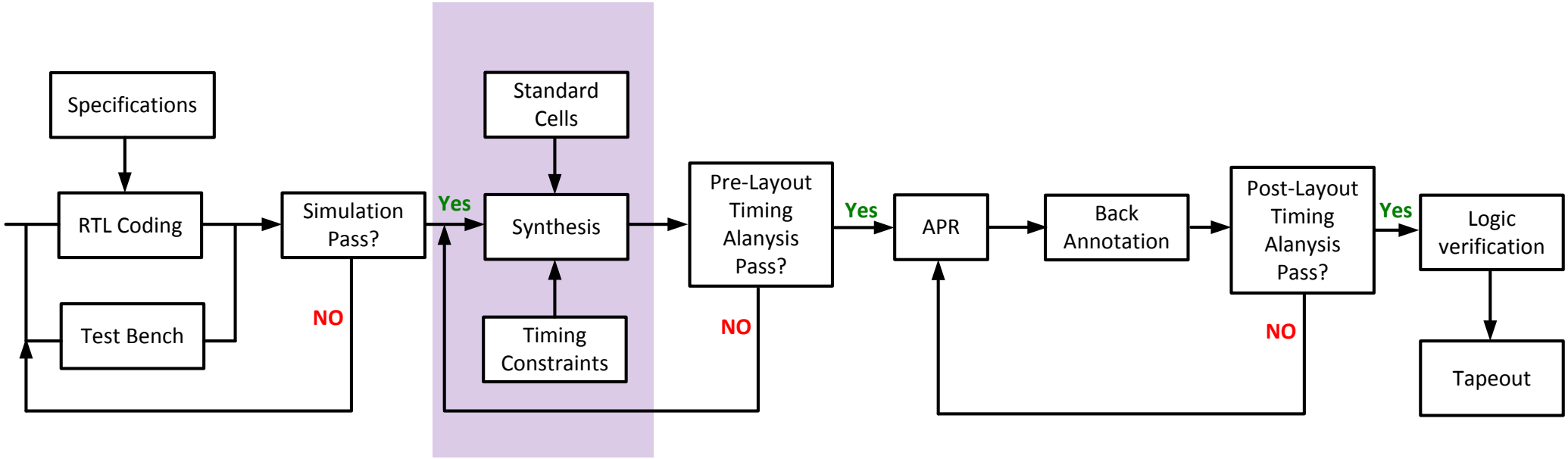
- HDL allows us to describe the functionality of a logic circuit in a language that is:
  - Easy to understand
  - Easy to share
  - Hides complicated implementation details
- Designer more concerned about the design functionality than the detailed circuit design

2. Simulation by Testbenches



- After HDL coding, the code has to be tested using “testbenches” (Verification).
- Simulation tools:
  - Synopsys VCS (Synopsys)
  - Modelsim (Mentor Graphics)
  - NCVerilog (Cadence)

3. Synthesis



- Synthesis tool:
  - Analyzes a piece of Verilog code and converts it into optimized logic gates
  - This conversion is done according to the **"language semantics"**
  - ➡ We have to learn these language semantics, i.e., Verilog code.

3. Synthesis

- Why using synthesis tools?

  - It is an important tool to improve designers' productivity to meet today's design complexity.

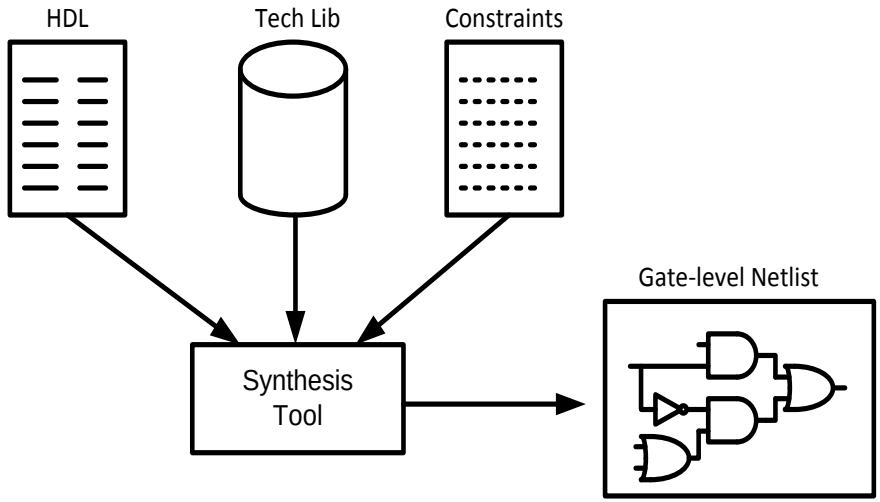
  - If a designer can design 150 gates a day, it will take 6666 man's day to design a 10-million gate design, or almost 20 years for 10 designers! This is assuming a linear grow of complexity when design gets bigger.

3. Synthesis

- Synthesis tool:
  - **Input:**
    - HDL Code
    - "Technology library" file → Standard cells (known by transistor size, 90nm)
      - Basic gates (AND, OR, NOR, …)
      - Macro cells (Adders, Muxes, Memory, Flip-flops, …)
    - Constraint file (Timing, area, power, loading requirement, optimization Alg.)
  - **Output:**
    - A gate-level "Netlist" of the design
    - Timing files (.sdf)

3. Synthesis Tools

❑ Synthesis tool:

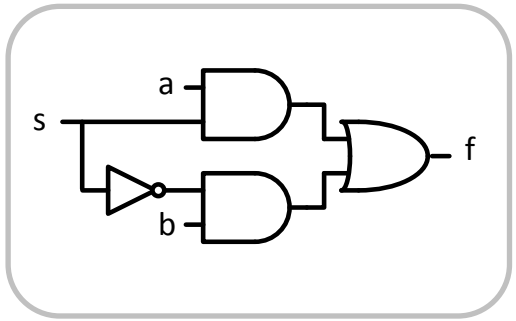


❖ **Example:** A 2-to-1 Multiplexer (2x1-MUX)

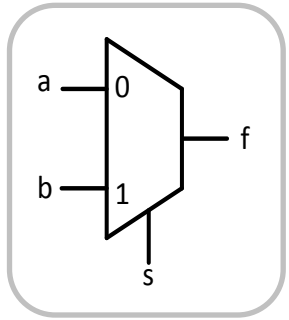
```
If (s==0)
   f = a;
else
   f = b;
```

Verilog code
(has to comply with certain structures)

Synthesis Tool



Synthesized gate-level



Schematic

3. Synthesis

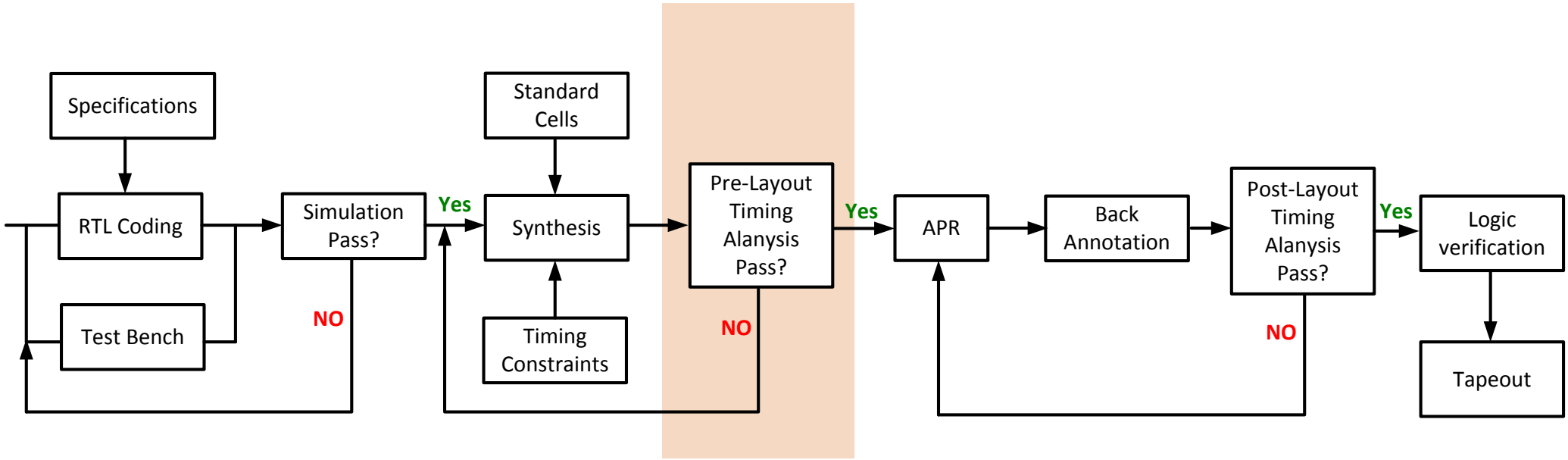
- Synthesis tools:
  - Infer logic and state elements
  - Perform technology-independent optimizations
    - e.g., logic simplification, state assignment
  - Map elements to the target technology
  - Perform technology-dependent optimizations
    - Multi-level logic optimization
    - Choose gate strengths to achieve speed goals

Synthesis Tools

- Commercial Synthesis Tools:

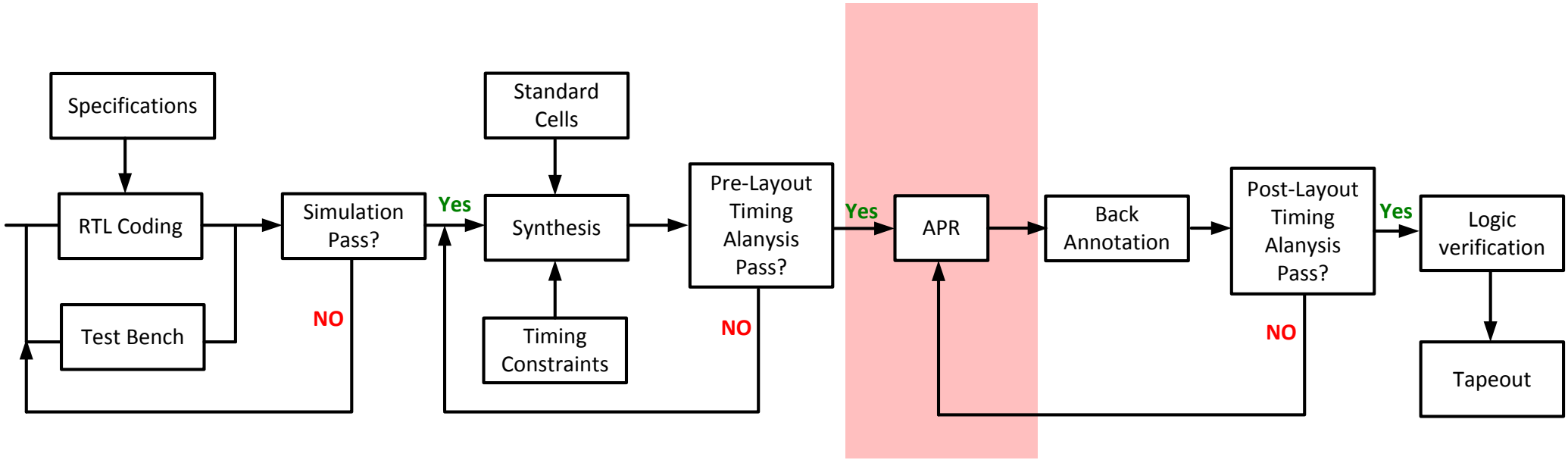
Vendor Name	Product Name	Platform
Altera	Quartus II	FPGA
Xilinx	ISE	FPGA
Mentor Graphics	Modelsim, Precision	FPGA/ASIC
Synopsys	Design Compiler, Galaxy	ASIC
Synplicity	Synplify	ASIC
Cadence	Ambit, BG, RC	ASIC

4. Pre-Layout Timing Analysis



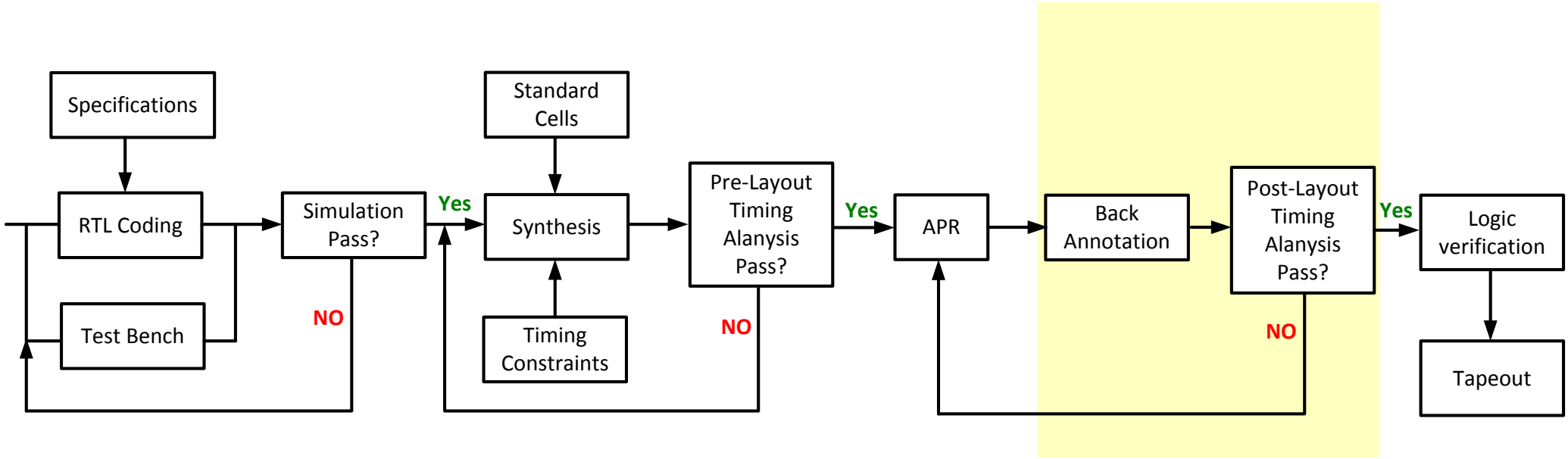
- Timing analysis across all design corners:
  - Different voltages and temperatures
  - Check for setup-time and hold-time violation
- Rough estimation as wire delays and RC models are not considered

5. APR



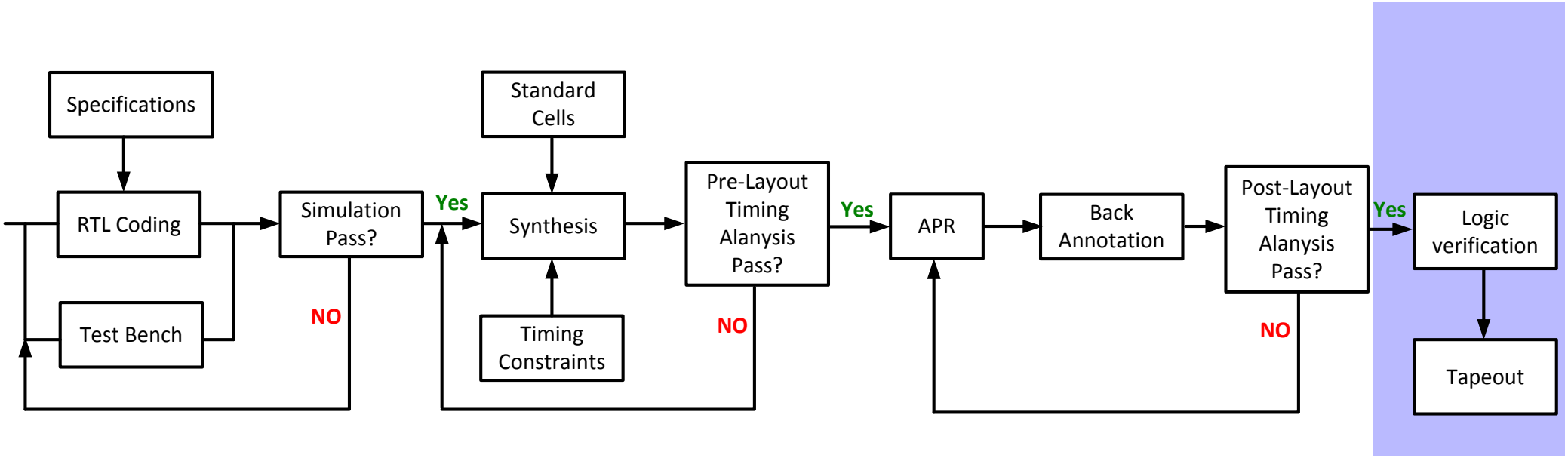
- Automatic Placement and Routing (APR)
  - Floorplan (Die size, Pad configuration, Die-to-pad space)
  - Placement (where each submodule sits in the chip)
  - Routing (metal wiring to connect all instances together according to the netlist)

6. Back Annotation & Timing Analysis



- Back Annotation (Timing Closure)
  - To estimate the delay after tapeout
  - Extraction of RC parasitics in the layout netlist interconnect delay
  - Some paths might now violate (setup-time and hold-time)
  - Causes increase in the path delay (specially in deep submicron)

7. Logic Verification & Tapeout



- Logic Verification
  - Simulate and test the very final netlist after APR
  - Timing analysis using testbenches
  - Send the final design (GDS file) for fabrication

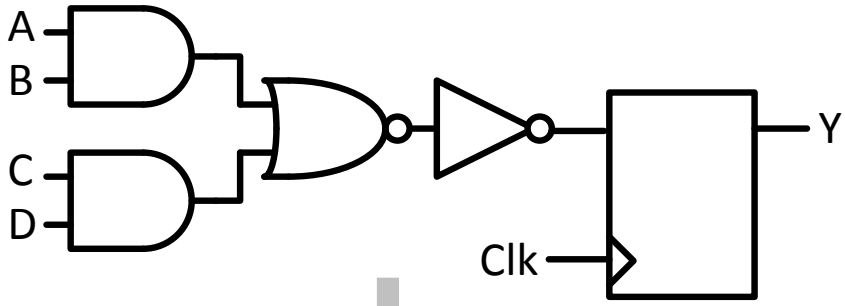
Outline

- ASIC/FPGA Design Flow
- **Hardware Description Language (HDL)**
  - **Verilog**
    - Introduction
    - Language Fundamentals
    - Modeling Combinational & Sequential Logic Circuits
    - Modeling Finite State Machines
    - Verilog Operations

Introduction: Digital Logic Design

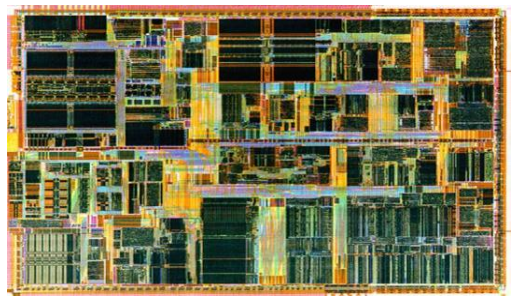
- Conventional Approach:
  - Schematic Entry → good for fairly small designs

    (Draw K-maps, optimize the Boolean logic, draw the schematic)



(10 gates)

  - Possible for large designs?
    - **NO!**



(10,000,000 gates)

Introduction: Why HDL?

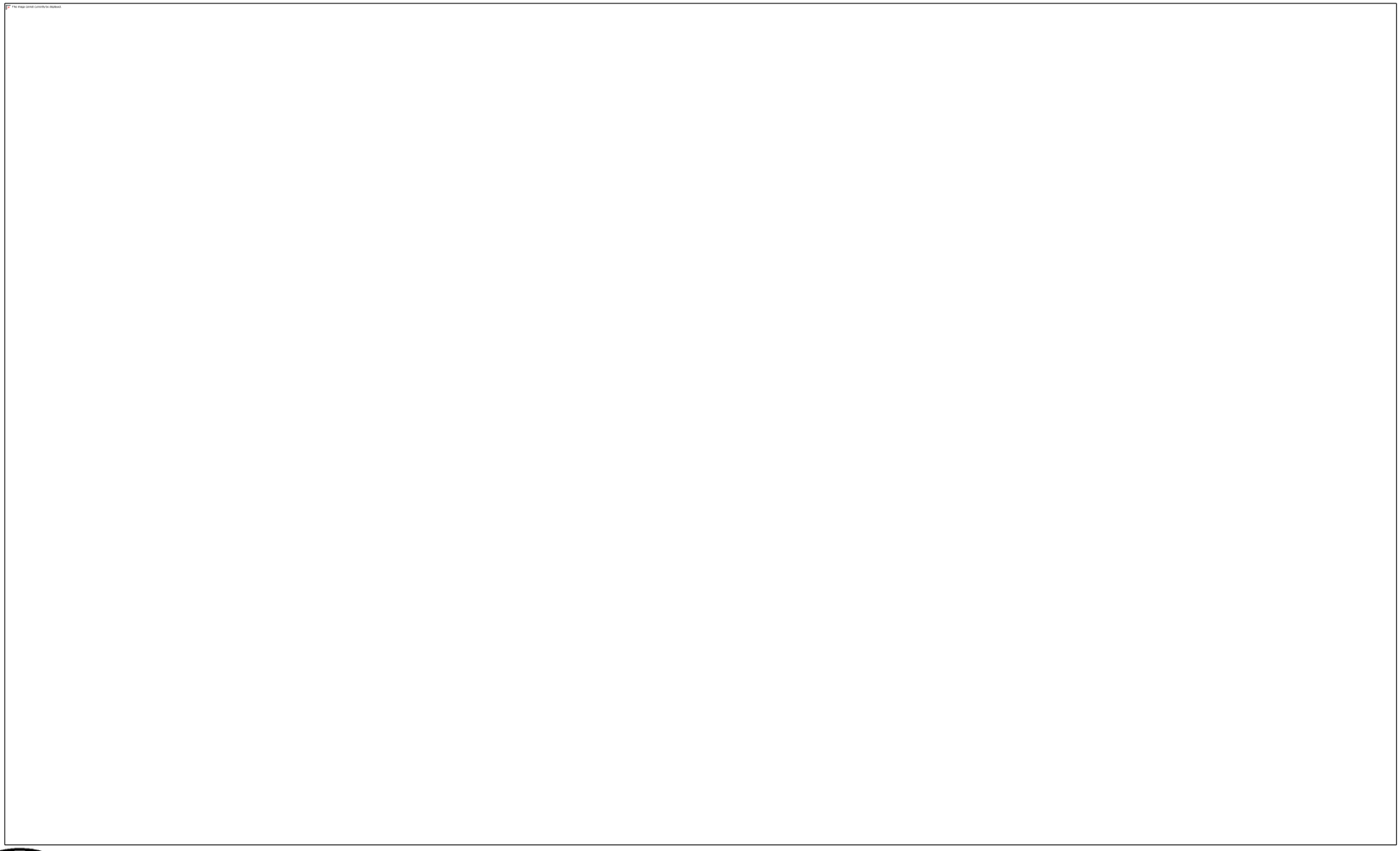
- Schematic entry not feasible for large designs:
  - Time consuming to draw the schematic for millions of gates
  - Prone to mistakes
  - Difficult design entry and sharing
  - Different design entry tools to learn
  - Tools not compatible (hard to convert the design from one to another)
  - Not easy to modify

- **Solution:**
  - Describe the design in text → Hardware Description Language (HDL)
  - Just describe the design **"behavior"** not the detailed gate-level logic
  - Gate-level logic is generated automatically by a **"synthesis"** tool

Introduction: Why HDL?

- Complicated designs can be easily described by HDL
- Can be used as the input to the synthesis tool
- Supports behavioral and structural descriptions
- Supports bit-level descriptions
- Detailed design cycle-by-cycle timing is supported
- Concurrent cores can be implemented and simultaneously simulated, which is vital to describe the hardware systems
  - Software programming languages typically have no concept of time. In hardware, there are delays associated with going from an input to an output.

HDL Coding



Advantages of HDL Coding

- Designer describes what the hardware should do without actually designing the hardware itself
- HDL Coding allows designers to separate behavior from implementation
- Designers develop an executable functional specification that documents the exact behavior of all the components and their interfaces
- Designers can make decisions about cost, performance, power, and area earlier in the design process

Advantages of HDL Coding

There are several benefits to using an HDL to describe your design:

- An HDL facilitates a top-down design methodology using synthesis
  - You can design at a high implementation-independent level
  - You can delay decisions on implementation details
  - You can easily explore design alternatives
  - You can solve architectural problems before implementation
  - You can automate mapping of your high-level description to a technology-specific implementation

- An HDL provides greater flexibility
  - You can re-use earlier design components
  - You can move your design between multiple vendors and tools

HDL Coding Goals

1. To simulate digital designs

2. To synthesize digital designs

- Some tools can automatically manipulate the design for verification, synthesis, optimization, etc.
  - Computer Aided Design (CAD) tools

HDL is NOT a Software Programming Languae

- **Software Programming Language**
  - Language which can be translated into machine instructions and then executed on a computer

- **Hardware Description Language**
  - Language with syntactic and semantic support for modeling the temporal behavior and spatial structure of hardware

HDL Coding

- A Hardware Description Language is a high-level programming language that offers special constructs, used to model microelectronic circuits
- Two standard HDLs:
  - VHDL (**V**ery high-speed integrated circuit **HDL**)
  - Verilog
- Verilog:
  - Developed by Philip Moorby in 1985 as a proprietary language
  - Open to public by Cadence Design Systems in 1990
  - IEEE standard in 1995 and revised in 2001

Verilog is used in this course!

Verilog or VHDL?

VHDL	Verilog
Commissioned in 1981 by Department of Defense	Created by Gateway Design Automation in 1985
An IEEE standard	An IEEE standard
Initially created for ASIC Synthesis	Initially an interpreted language for gate-level simulation
Strong support for package management and large designs	No special extensions for large designs
ADA-like verbose syntax, lots of redundancy	C-like concise syntax
Design is composed of **entities** each of which can have multiple architectures	Design is composed of **modules** which have just one implementation
Gate-level, dataflow, and behavioral modeling. Synthesizable subset.	Gate-level, dataflow, and behavioral modeling. Synthesizable subset.
Harder to learn and use	**Easy to learn and use**

Verilog in Three Flavors

- There are three types of Verilog Coding:
  - **Behavioral:**
    - Describes a system **by the flow of data** between its functional Blocks
    - Defines signal values when they change

    Most Descriptive
  - **Structural:**
    - Shows detailed design components, nets, and interconnects
    - Uses technology-specific, low-level components
    - Used to pass netlist information b/w design tools (e.g., from DC to APR)

    Least Descriptive
  - **RTL (Register Transfer Level):**
    - Describe how data transfers b/w registers and input/outputs
    - Describes a system by **the flow of data and control signals** between and **within its functional blocks**
    - Defines signal values **with respect to a clock**

    Somehow Descriptive

Focus of this course

Verilog Coding Styles

RTL

```
module RTL ( A,  B, C,  D, Out);
 input   A,  B, C, D;
 output Out;
 reg Out;
 always @ (A or B or C or D)
  begin
   if (A & B & ~D)
     Out = C;
   else if (A & D & ~C)
     Out = B;
   else
     Out = 0;
  end
endmodule
```

Our Focus

Synthesizable

Behavioral

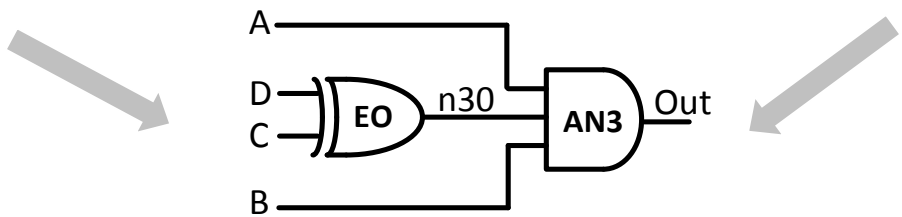
```
module behavior (A,B, C, D, Out);
 input A, B, C, D;
 output Out;
 reg Out;
 always @ (A or B or C or D)
  begin
   if (A & B & ~D)
     Out = #5 C;
   else if (A & D & ~C)
     Out = #3 B;
   else if ((A ==1'bx) | (B ==1'bx) |
            (C ==1'bx) |(D ==1'bz))
     Out = #7 1'bx;
   else if ((A ==1'bz) | (B ==1'bZ))
     Out = #7 1'bZ;
   else
     Out = #3 0;
  end
endmodule
```

Not synthesizable!

Structural

```
module structural (A,B, C, D, Out);
  input A, B, C, D;
  output Out;
  wire n30;
  EO U9 ( .A(D), .B(C), .Z(n30) );
  AN3 U8 ( .A(A), .B(n30), .C(B), .Z(Out) );
endmodule
```

Synthesizable

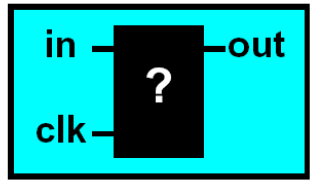


Verilog in Three Flavors : Behavioral

The behavioral level describes the behavior of a design without implying any specific internal architecture:

- You use high level constructs, such as **@**, **case**, **if**, **repeat**, **wait**, **while**
- You can use any behavioral construct of the HDL in your testbench
- Synthesis tools accept only a limited subset of these behavioral constructs

This behavioral model defines the behavior of the design as seen at its ports:

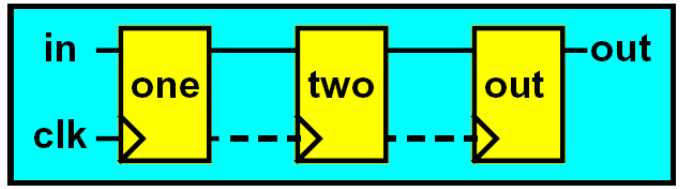



```
module pipe ( out, in, clk );
output out; reg out;
input  in, clk;
  always @(in)
    @(posedge clk)
      out <= repeat(2) @(posedge clk) in;
endmodule
```

Verilog in Three Flavors : RTL

The RTL (functional) level describes the design architecture in sufficient detail that a synthesis tool can construct the circuit.

This functional model defines three storage elements and their assignments:




```
module pipe ( out, in, clk );
output out; reg out;
input  in, clk;
  reg one, two;
  always @(posedge clk) begin
      out <= two;
      two <= one;
      one <= in;
    end
endmodule
```

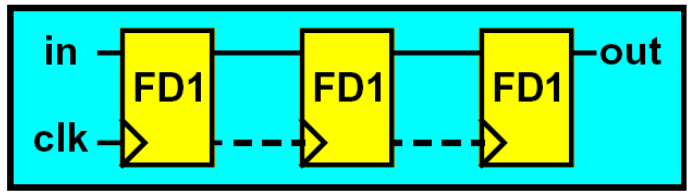
Verilog in Three Flavors : Structural

Synthesis tools produce a purely structural design description.

The structural level is also appropriate for small library components:

- You can use built-in Verilog primitives, such as the **and** gate
- You can describe your own User Defined Primitives (UDPs)

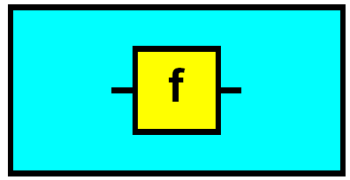
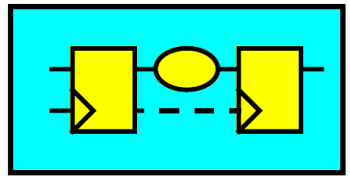
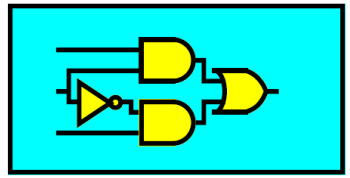
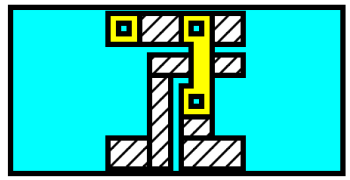
This structural model instantiates predefined library components:




```
module pipe ( out, in, clk );
output out;
input in, clk;
  FD1 one_reg(.Q(one), .D(in ), .CP(clk));
  FD1 two_reg(.Q(two), .D(one), .CP(clk));
  FD1 out_reg(.Q(out), .D(two), .CP(clk));
endmodule
```

Verilog Coding Styles: Levels of Abstraction

At each level of abstraction, you can describe a system as a group of hierarchical models in varying amount of detail. EDA tools facilitate this process.

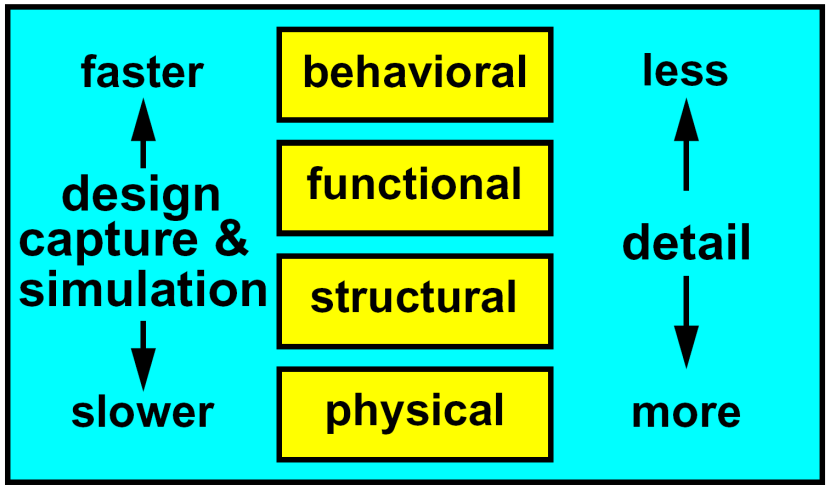
Behavioral representation		Verilog simulation, behavioral synthesis
Functional representation		Verilog simulation, logic synthesis
Structural representation		Verilog simulation static functional analysis static timing analysis place & route
Physical representation		Spice simulation design rule checking parasitic analysis

Verilog Coding Styles: Levels of Abstraction

❑ Trade-offs:

Each level of abstraction permits modeling at a higher or lower level of detail.

More detail means more work for you and the simulator.

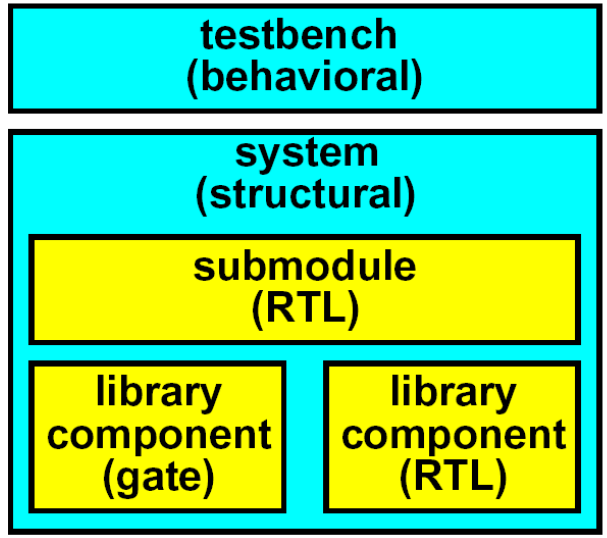


Verilog Coding Styles: Levels of Abstraction

❑ One language for all levels:

Designers usually mix levels of abstraction within a simulation:

- RTL and gate-level library components
- RTL functional submodule descriptions
- Structural system netlist
- Behavioral system testbench



Verilog Coding Styles: Design Style

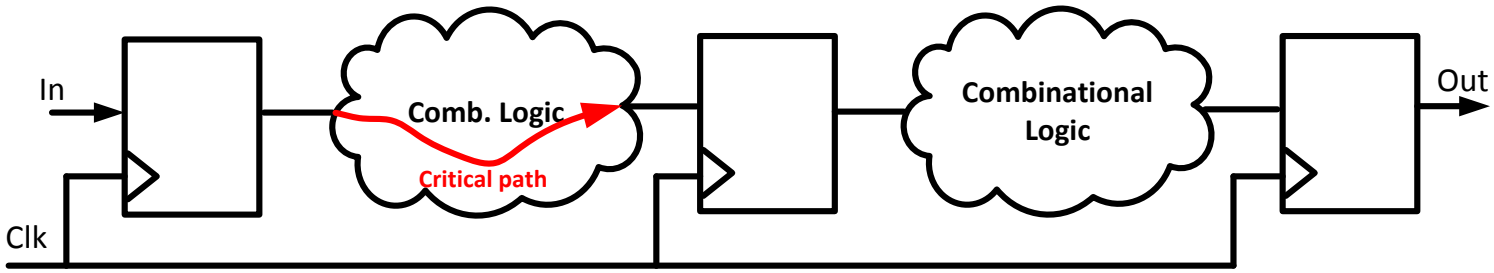
- Verilog, like any other hardware description language, permits a design in either Bottom-up or Top-down methodology.

  - **Bottom-Up Design**
    - The traditional method of electronic design is bottom-up. Each design is performed at the gate-level using the standard gates. With the increasing complexity of new designs this approach is nearly impossible to maintain. New systems consist of ASIC or microprocessors with a complexity of thousands of transistors. These traditional bottom-up designs have to give way to new structural, hierarchical design methods.

  - **Top-Down Design**
    - A real top-down design allows early testing, easy change of different technologies, a structured system design and offers many other advantages. But it is very difficult to follow a pure top-down design. Due to this fact most designs are a mix of both methods, implementing some key elements of both design styles.

Verilog for Synthesis (RTL)

- In this course we focus on RTL coding
- RTL coding is the closest one to the actual hardware implementation
- RTL code includes a subset of all Verilog syntax
  - Not all Verilog syntax are synthesizable
- We cover most Verilog coding parts that are needed for logic synthesis
- Simulation of the RTL code is also covered
- We learn how to write a “good” Verilog code for synthesis
- Lots of examples on the synthesized RTL!



Verilog Applications

The Verilog HDL is used by:

- **System Architects:** doing high level system simulations
- **Verification Engineers:** writing advanced tests for all levels of simulation
- **ASIC and FPGA Designers:** writing RTL code for synthesis
- **Library Developers:** describing ASIC or FPGA cells, or higher level components

Outline

- ASIC/FPGA Design Flow
- **Hardware Description Language (HDL)**
  - **Verilog**
    - o Introduction
    - o Language Fundamentals
    - o Modeling Combinational & Sequential Logic Circuits
    - o Modeling Finite State Machines
    - o Verilog Styles for Synthesis
    - o Testbench and Simulation

Verilog Fundamentals : Comment

- Comments are used for documentation

- Comments are in two types:
  - Short comments (single line)

```
// This is a comment
```

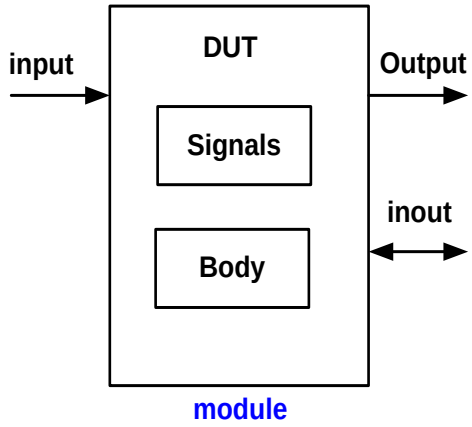
  - Long comments (Multiple lines)

```
/* This a multiple
   line comment
   in Verilog */
```

- Space, tab and blank lines are ignored by the compiler

Verilog Fundamentals : Module

- Any circuit or subcircuit is declared as a "module" in Verilog.

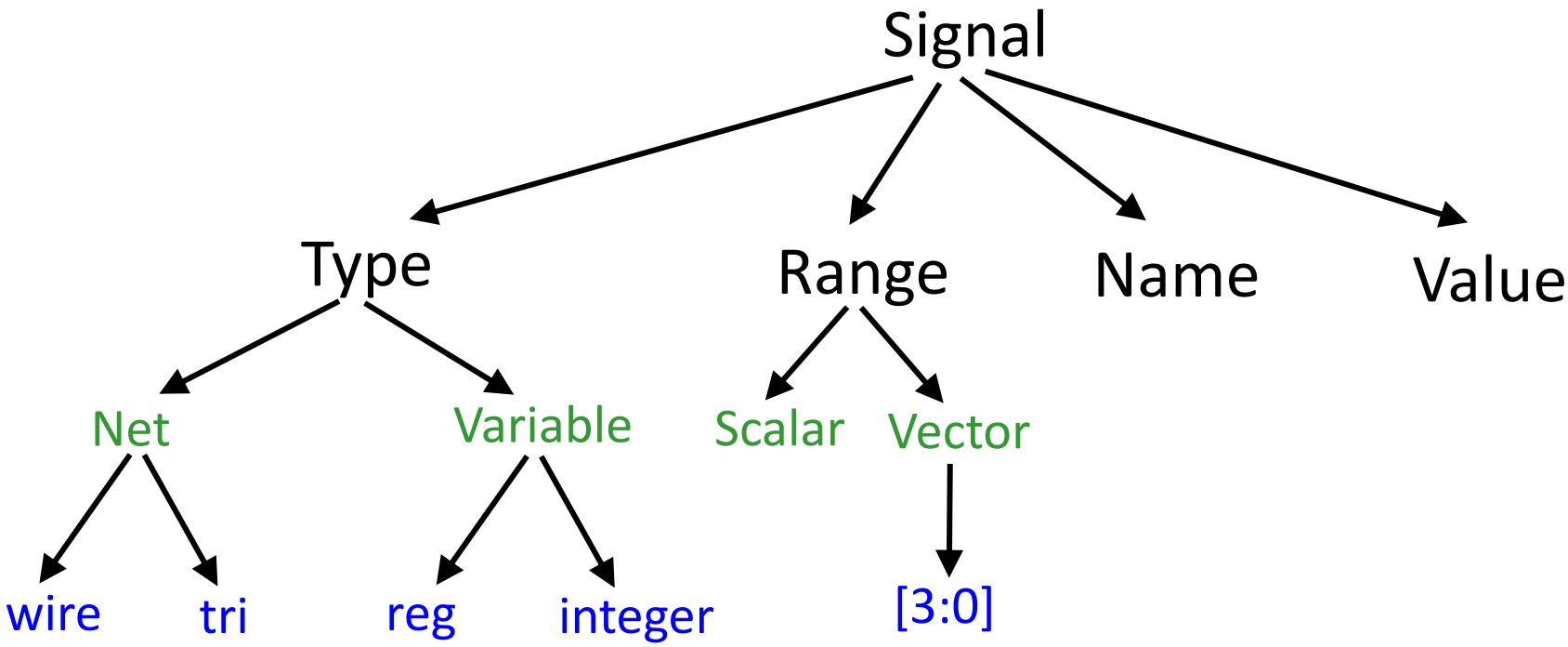


- Each module may have:
  - **Ports** (Three possibilities),
    - input
    - output
    - inout
  - **Signals** (main or intermediate)
  - **Body-code**

    (statements for module description)

```
module  DUT (A, B, C);
   input A;
   output B;
   inout C;
   Signals
   Body-code
endmodule
```

Verilog Fundamentals : Signals

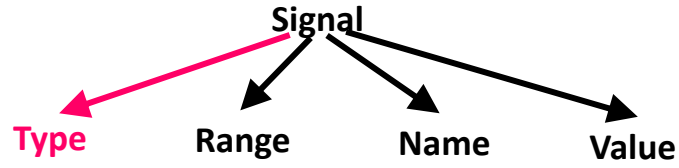


❖ **Example:** wire [2:0] tmp ;
tmp = 3'b001;

tmp[0]=1
tmp[1]=0
tmp[2]=0

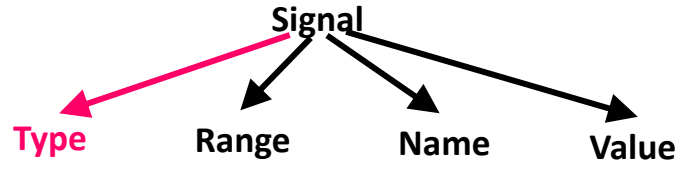
Each element of a vector can be accessed

Verilog Fundamentals : Signal Type



- **Net**
  - **wire:**
    - For interconnecting logic elements (LEs)
    - To connect an output of a logic element to the input of another LE
  - **tri**
    - Circuit nodes that are connected in a tri-state fashion
- **Variable**
  - **reg** (unsigned in general)
    - Corresponds to a circuit node (not necessarily a register!)
    - Allow a circuit to be described in terms of its behavior
    - Retains its value until it is overwritten by a subsequent assignment
  - **integer** (signed in general)
    - Used for loop counters

Verilog Fundamentals : Signal Type



- The “wire” declarations are not necessary as Verilog assumes that signals are nets by default .
- The “reg” declaration is required!

❖ Example:

```
module  DUT (A, B, C) ;
   input [1:0] A;
   output B;
   inout [2:0] C;

   wire [1:0] A;
   reg B, w;

   Body-code

endmodule
```

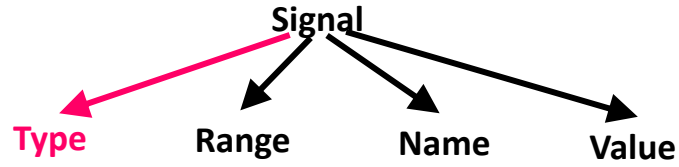
Don’t forget semicolon

Not necessary

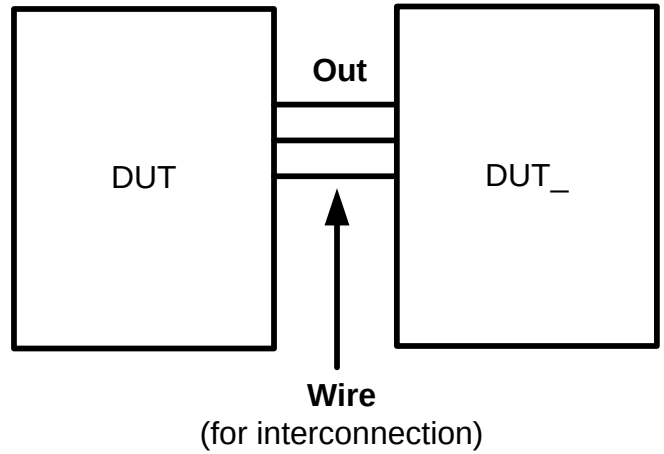
Required

Two signal declarations in one line

Verilog Fundamentals : Signal Type



❖ Example:



Ports

Signals

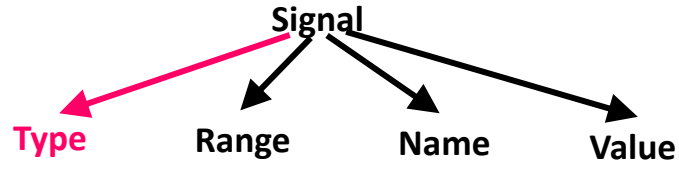
Code Body

```
module  DUT (s, Out);
   input [3:0] s;
   output [2:0] Out;

   wire [2:0] Out;
   reg [2:0] Count;
   integer k;

   Count = 0;
   for (k=0; k<4; k=k+1)
      if (s[k])
         Count = Count + 1;
   assign Out = Count;

endmodule
```

← Loop counter

← ";" at the end of each line

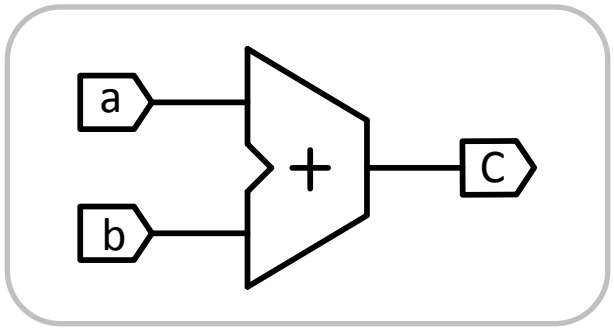
Reg Type



- The keyword “reg” does NOT necessarily denote a storage element or register.
- “reg” only models the behavior of a circuit.
- May or may not be synthesized as a register.

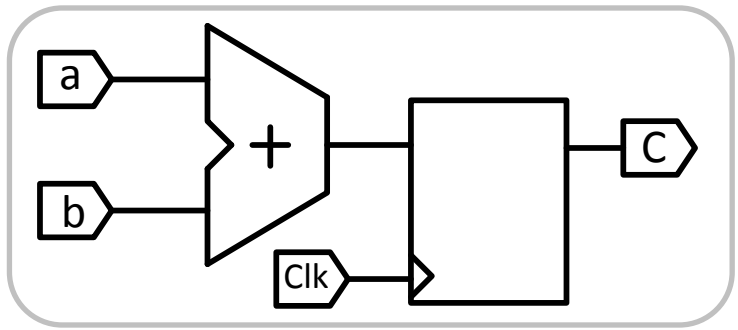
Not Register

```
reg  C;
   always @ (a,b)
      C = a+b;
```



Register

```
reg  C;
   always @ (posedge Clk)
      C <= a+b;
```



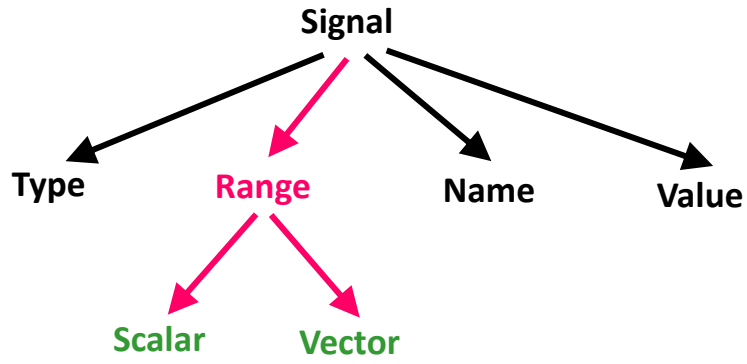
Verilog Fundamentals : Signal Range

- Signals in Verilog can be:
  - **Scalar**: representing a node

```
reg  C;
wire B;
```

  - **Vector**: representing a bus

```
reg  [10:0] Data;
reg  [0:6] S;
wire [7:4] B;
```

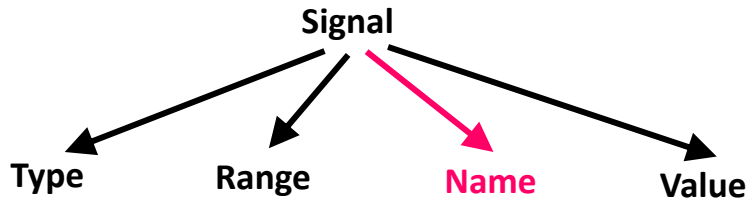


- Each element of a bus can be accessed.

```
assign a = Data[8];
```

Verilog Fundamentals : Signal Name

- Signal name may consists of:
  - Any letter
  - Any digit
  - Underscore (_) and $ sign
- **DON'Ts:**
  - Should not start with a digit
  - Should not be a Verilog keyword

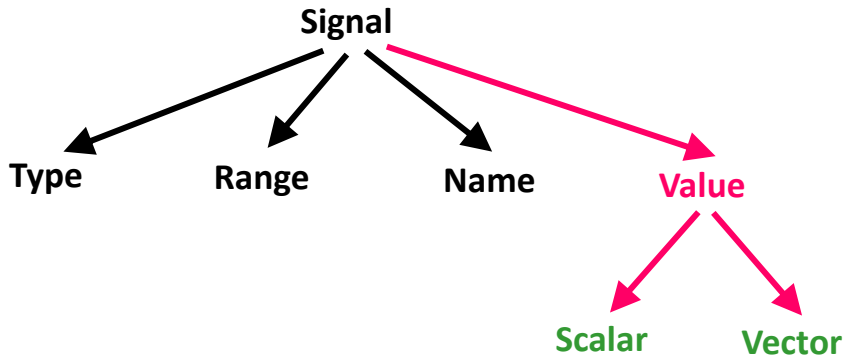


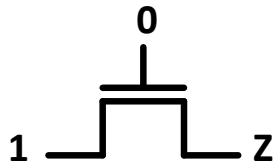
Legal	Illegal
A_m	1xb
B1_signal	wire
My$	R&z

Note: Verilog is case sensitive!

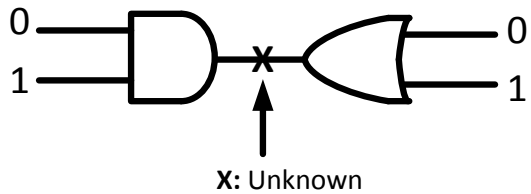
Verilog Fundamentals : Signal Value

- **Scalar**: each scalar signal can have four possible values:
  - **0:** Logic value “0”
  - **1:** Logic value “1”
  - **Z(z):** Tri-state (high impedance)





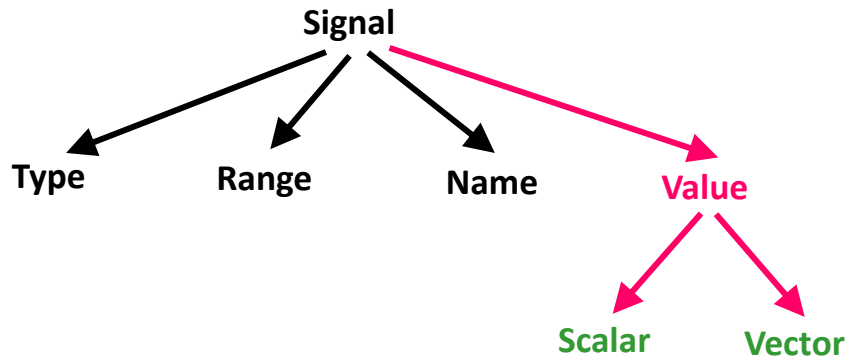
  - **X(x):** Unknown value



Verilog Fundamentals : Signal Strength

- سطوح قدرت به منظور حل اختلافات زمانیکه یک گره به درایورهایی با مقادیر یا قدرتهای متفاوت متصل شده باشد استفاده می شود .بطور مثال حاصل اتصال مقدار strong1 و weak0 مقدار strong1 خواهد بود. اگر دو سیگنال با مقدار مختلف ولی با سطح قدرت یکسان به یکدیگر متصل شوند حاصل نامعلوم (x) خواهد بود.

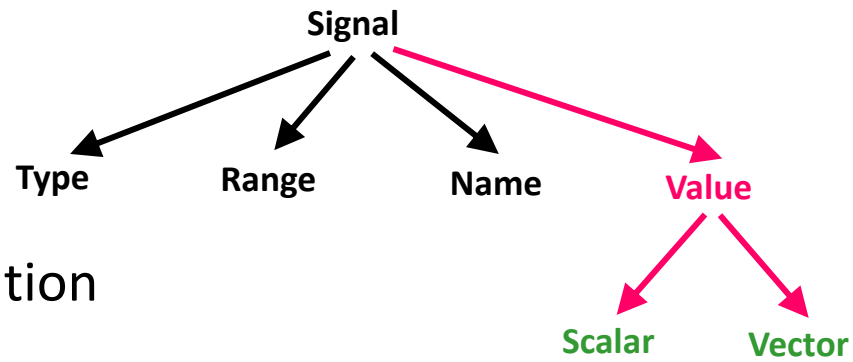
Strength Level	Type	Degree
supply	Driving	Strogest
strong	Driving	
pull	Driving	
large	Storage	
weak	Driving	
medium	Storage	
small	Storage	
highz	High Impedance	weakest



Verilog Fundamentals : Signal Value

❑ Vector: <# of bits> <base> <number>

4 ‘b 0101



- **<# of bits> :** number of bits for representation
- **<base> :** (default decimal)
  - **“d” : Decimal**
  - **“b” : Binary**
  - **“h” : Hexadecimal**
  - **“o” : Octal**
- **<number> :** signal value in base

Used for clarity

❖ **Example:**

```
K = 8’ha9;      // K=1010_1001
C= 4’d3;        // C=0011
D= 4’b100;      // D=0100
F= ‘b10x;       // F=10X
L = -6’b3       // L = 111101
```

Verilog Fundamentals : Parameters

❑ A parameter is used as a “constant” to facilitate coding.

❖ Example:

```
module  DUT (s, Out)

  parameter n = 3;
  parameter S0 = 4'b1010;

  input [n-1:0] s;
  output [n:0] Out;

  wire [n:0] Out;
  assign Out = S0;

endmodule
```

Verilog Fundamentals : Memories

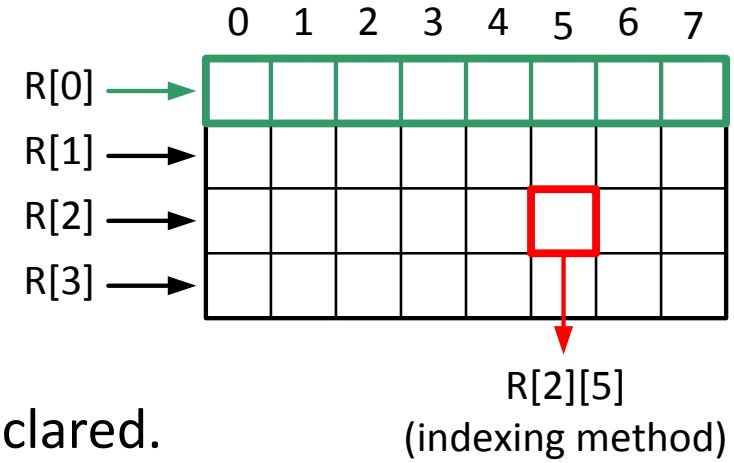
❑ **Memory**:

- ➢ A two-dimensional array of bits
- ➢ Declared in Verilog as a two-dimensional variable (reg)

❖ Example: A 4-byte memory:

```
reg [7:0] R [3:0];
```

8-bit 4 rows (cell)



❑ A three-dimensional array may also be declared.

❖ Example:

```
reg [7:0] M [3:0][1:0];
```

❑ If an 8-bit A is declared then the legal assignment is:

```
reg [7:0] A;   →   A = M[3][0];
```

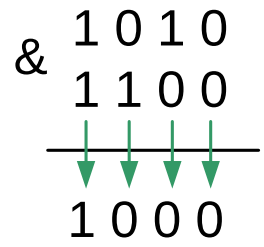
© M. Shabany, ASIC/FPGA Chip Design

Verilog Fundamentals : Operators

❖ Example:

➢ **Bitwise:**

Operation	Result
1010 **&** 1100	1000
1010 **|** 1100	1110
~1010	0101
1101 **Λ** 0100	1001



➢ **Logical:**

X || 1 = 1
X && 0 = 0

Operation	Result
1010 **&&** 1100	1
2'b11 **||** 2'b00	1
!0010	0
2'b1X **&&** 2'b11	X

Non-zero operand=logical "1"

Any operand X/Z, result is X

Verilog Fundamentals : Operators

❖ Example:

➢ **Reduction:**

Operation	Result
& 1100	0
& 111	1
^ 0100	1

➢ **Relational:**

A=2'b10

Operation	Result
B=(A == 2'b10)	B=1
B=(A == 2'b11)	B=0
B=(A === 2'b1x)	B=0
B=(A <= 2'b11)	B=1

== Used only with 0 and 1

=== Used with x and z

Verilog Fundamentals : Operators

❖ Example:

➢ **Logical Shift:**

A=6'b001100

Operation	Result
C = A >> 1	C = 000110
D = A << 2	D = 110000
F = A >> 3	F = 000001

➢ **Concatenation:**

A=2'b11
B=3'b010

Operation	Result
{A, B}	5'b11010
{3{A}}	6'b111111
{B, B}	6'b010010
{{3{A}}, {2{B}}}	12'b111111010010

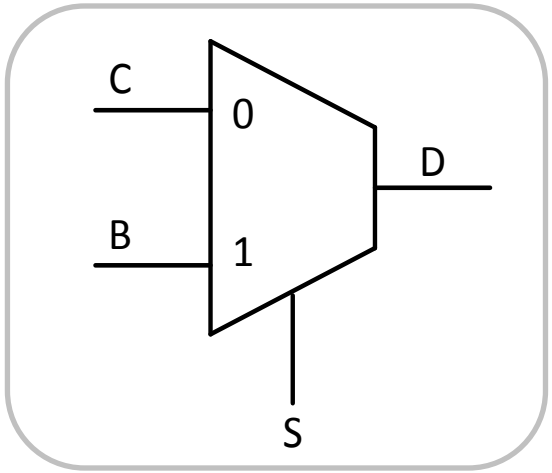
Be generous in {}

Verilog Fundamentals : Operators

❑ Conditional: (? , :)

- D = S ? B:C;

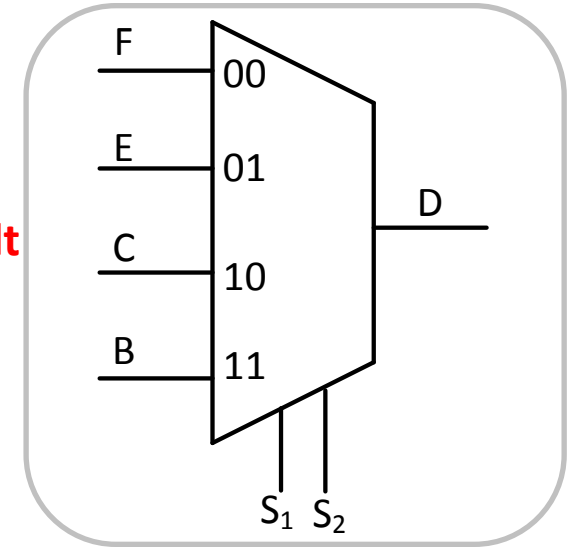
$$D = \begin{cases} B & \text{if } S=1; \\ C & \text{if } S=0; \end{cases}$$



- D = ({S_1,S_2}==2'b00)? F:
  ({S_1,S_2}==2'b01)? E:
  ({S_1,S_2}==2'b10)? C:B;

- D = ({S_1,S_2}==2'b00)? F:
  ({S_1,S_2}==2'b01)? E:
  ({S_1,S_2}==2'b10)? C:
  ({S_1,S_2}==2'b11)? B:B;

Default



4-input Multiplexer (MUX)

Verilog Fundamentals : Operators (All in One)

Type of Operators	Symbols
Concatenate & replicate	{ } {{ }}
Unary	! ~ & ^ ^~ |
Arithmetic	* / %
	+ -
Logical shift	<< >>
Relational	< <= > >=
Equality	== != === !==
Binary bit-wise	& ^ ^~ |
Binary logical	&& ||
Conditional	? :

Verilog Fundamentals : Module-Revisited

- Any circuit or subcircuit is declared as a "module" in Verilog.
- There are three types of ports:
  - input ⟹ type "wire"
  - output ⟹ type "wire" or "reg"
  - inout ⟹ type "wire"
- Note:

```
output [3:0] B;
reg [3:0] B;
```

Combined ⟹

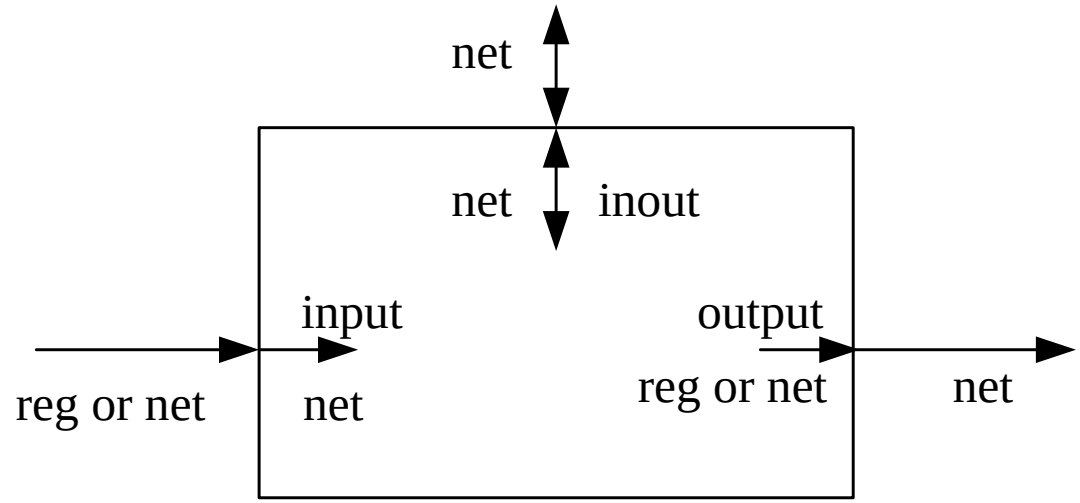
```
output reg [3:0] B;
```

```
module  DUT (A, B, C)
   input A;
   output [3:0] B;
   inout C;

   wire A;          → Optional
   wire C;          → Optional
   reg [3:0] B;     → Mandatory

   Signals

   Body-code

endmodule
```

Verilog Fundamentals : Module Ports

- Inside view of the module
  - input port: wire
  - output port : wire or reg
  - inout: wire

- Outside view of the module
  - input port: wire or reg
  - output port : wire
  - inout: wire



Verilog Fundamentals : Module-Revisited

- In Verilog-2001 the port list can directly follow the module declaration

```
module  DUT (A, B, C)
   input A;
   output [3:0] B;
   inout C;

   wire A;
   wire C;
   reg [3:0] B;

   Signals

   Body-code
endmodule
```

```
module  DUT ( input A,
              output [3:0] B,
              inout C);

   wire A;
   wire C;
   reg [3:0] B;

   Signals

   Body-code
endmodule
```

- Body-code consists of some "statements"
- Statements describe the circuit/module functionality

Verilog Fundamentals : Statements

- Programming languages:
  - **High-Level Language (HLL):** C, Pascal, Matlab
  - **Hardware Description Language (HDL):** Verilog, VHDL
- In HLL programming all statements are sequential (procedural)
  - Statements evaluated in the order and one-bye-one

- Verilog Statements
  - **Procedural** : evaluated sequentially (Order **IS** important)

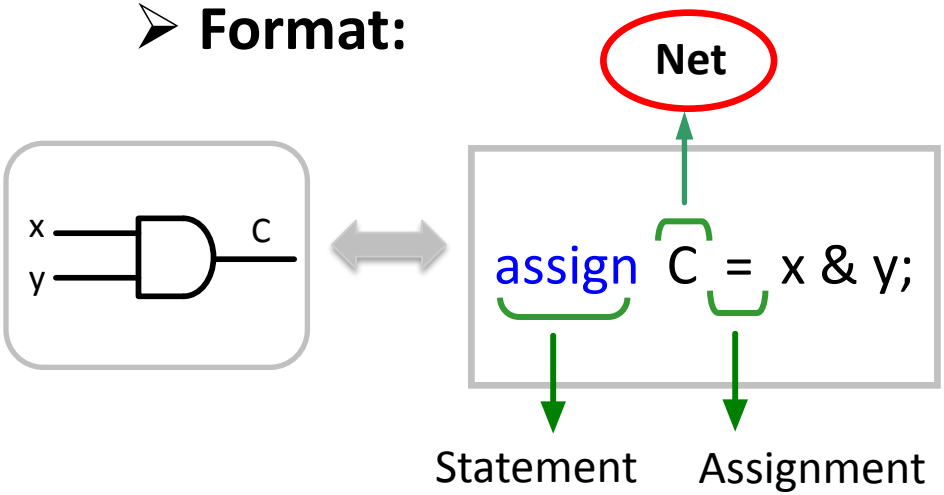
```
always @ (x, y)
  begin
    s = x^y;
    c = x&y;
  end
```

  - **Concurrent** : evaluated in parallel (Order **NOT** important)

```
assign s=x^y;
assign c=x&y;
assign out=x|y;
```

Verilog Fundamentals : Concurrent Statements

- Evaluated in parallel
- Each statement describes part of the circuit, thus concurrent
- Most popular:
  - **Continuous statements**: realized as connection or wire in the design
  - **Format:**



❖ Example:

```
wire   [1:3] A, B, C;
assign   C = A&B;
```

Equivalent

```
assign   C[1] = A[1]&B[1];
assign   C[2] = A[2]&B[2];
assign   C[3] = A[3]&B[3];
```

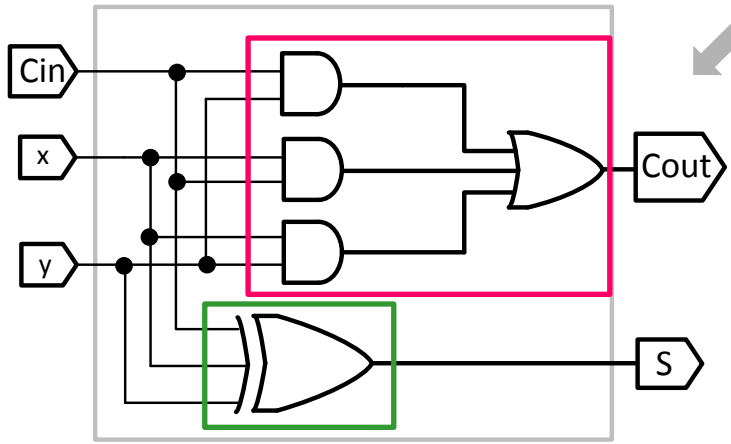
- assign used only for nets (to be synthesizable)

Concurrent Statements

❖Example: Full Adder, same circuit, two descriptions:

```
module Adder (Cin, x, y, S, Cout)
   input  x, y, Cin;
   output  S, Cout;
   wire S, Cout;

   assign S = x ^ y ^ Cin;

   assign Cout = (x & y)|(x & Cin)|(y & Cin);

endmodule
```

```
module Adder (Cin, x, y, S, Cout)
   input  x, y, Cin;
   output  S, Cout;
   wire S, Cout;


   assign {Cout, S} = x + y + Cin;

endmodule
```



x	y	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

Concurrent Statements

❖Example: Signed vs. unsigned addition:

❑ In Verilog "+" declares unsigned addition

❑ Signed addition has to be explicitly specified using the sign extension

```
module Adder_sign (X, Y, S_unsigned, S_signed);
   input  [3:0] X, Y;
   output  [4:0] S_unsigned, S_signed;

   assign S_unsigned = X + Y;

   assign S_signed = {{X[3]},X} + {{Y[3]},Y};

endmodule
```

Sign extension

```
module Adder_sign (X, Y, S_unsigned, S_signed);
   parameter n = 4;
   input  [n-1:0] X, Y;
   output  [n:0] S_unsigned, S_signed;

   assign S_unsigned = X + Y;
   assign S_signed = {{X[n-1]},X} + {{Y[n-1]},Y};

endmodule
```

X = 0011 (unsigned 3) or (signed +3)
Y = 1101 (unsigned 13) or (signed -3)

S_unsigned = 10000 (unsigned 16)
S_signed = 00000 (0 signed)

Concurrent Statements: Sign Extension

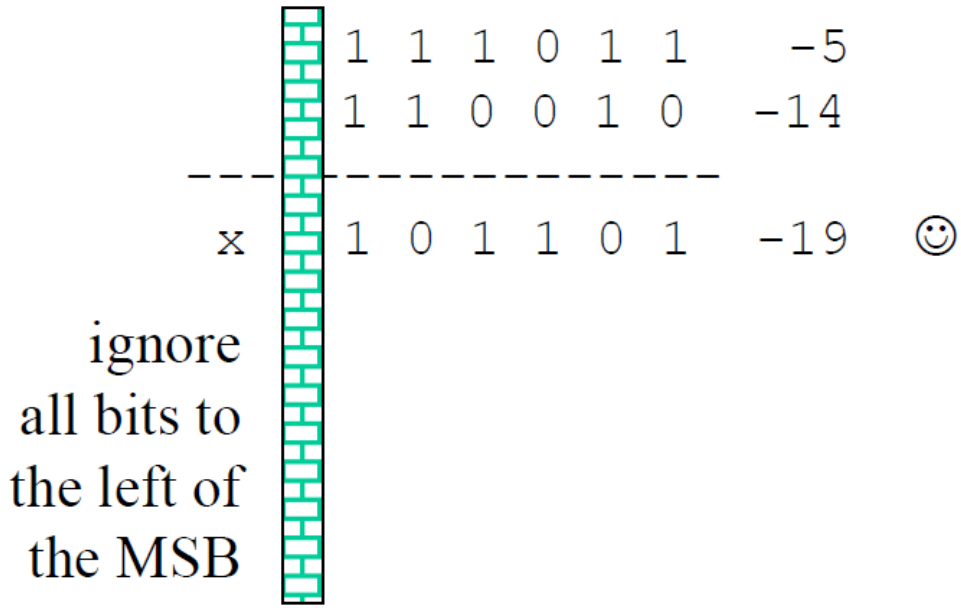
```
  0 0 1 0 1 1   -5
  0 1 0 0 1 0  +18
---------------
  0 1 1 1 0 1  +29!
```

```
    1 0 1 1   -5 = -8 + 2 + 1
  1 1 0 1 1   -5 = -16 + 8 + 2 + 1
1 1 1 0 1 1   -5 = -32 + 16 + 8 + 2 + 1
```

```
    | 1 1 1 0 1 1   -5
    | 0 1 0 0 1 0  +18
----+---------------
  x | 0 0 1 1 0 1  +13  ☺
```

Concurrent Statements: Sign Extension

- **Ignore carry bits:** Do not spend any hardware calculating any bits to the left of the answer's MSB



Verilog Fundamentals : Delay

- Delay can be used with continuous assignments by using the "#" sign

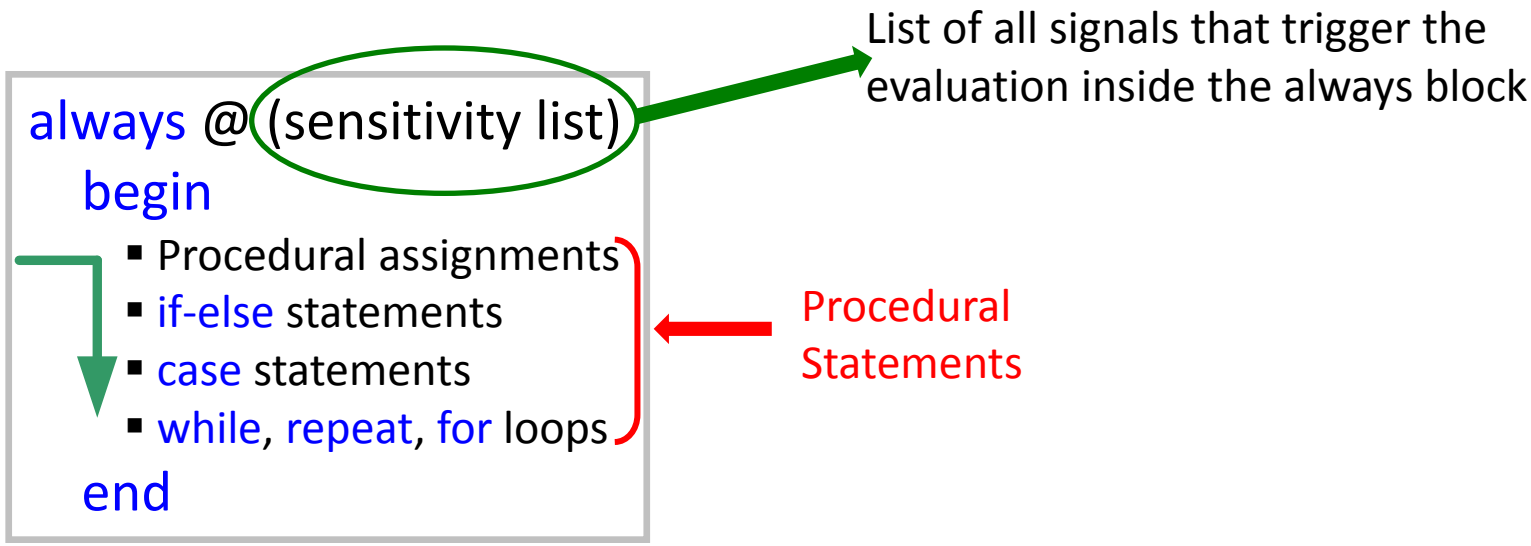
```
wire  #2  S;
assign  #5  S = x&y;
```

  - 2 time unit of delay on wire S
  - 5 time units of delay for AND gate
  - Any change in x or y reflects on S after 7 time unit delay

- Used only for simulation purposes
  - No meaning for synthesis
  - **Not** synthesizable

Procedural Statements

- Evaluated in the order in which they appear in the code (sequential)
- Should be inside an "**always**" block
- An "**always**" block contains one or more procedural statements



Procedural Statements: Half-Adder

❖Example:

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Type: "reg"

```
module Adder (x, y, S, C)
   input x,y;
   output S,C;
   reg S, C;
   always @ (x, y)
      begin
         S = x Λ y;
         C = x & y;
      end
endmodule
```

If either x or y changes, the statements inside the always block are evaluated.

```
module Adder (x, y, S, C)
   input x,y;
   output S,C;
   wire S, C;
   assign S = x Λ y;
   assign C = x & y;
endmodule
```

Type: "wire"

- Anything on the RHS should be on the sensitivity list
  - always @(*) ➡ Automatically considers all signals on the RHS in the sensitivity list
- Any signal assigned inside an always block has to be a variable of type
  - **reg**
  - **integer**

always block: Quick Notes

- The always construct requires begin-end only if there are multiple statements in the block.

❖ Example:

```
always @ (x, y, z)
    begin
        z = x;
        if (x == 1)
            z = y;
    end
```

Correct

```
always @ (x, y, z)
    z = x;
    if (x == 1)
        z = y;
```

← Not part of the always block

Incorrect

- A given variable should never be assigned a value in more than one always block.
  - Because always blocks are concurrent with respect to one another.

Incorrect →

```
always @ (x,y)
    a <= x;
always @ (x,y)
    a <= y;
```

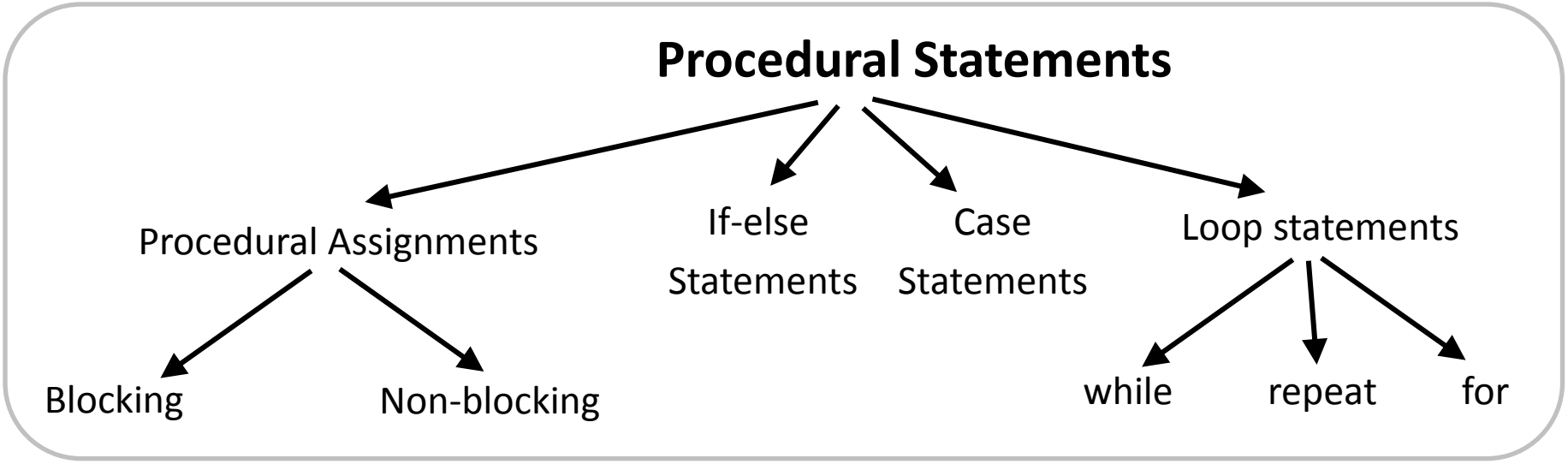
```
always @ (x,y)
begin
    a <= x;
    a <= y;
end
```

⟷

```
always @ (x,y)
    a <= y;
```

Correct

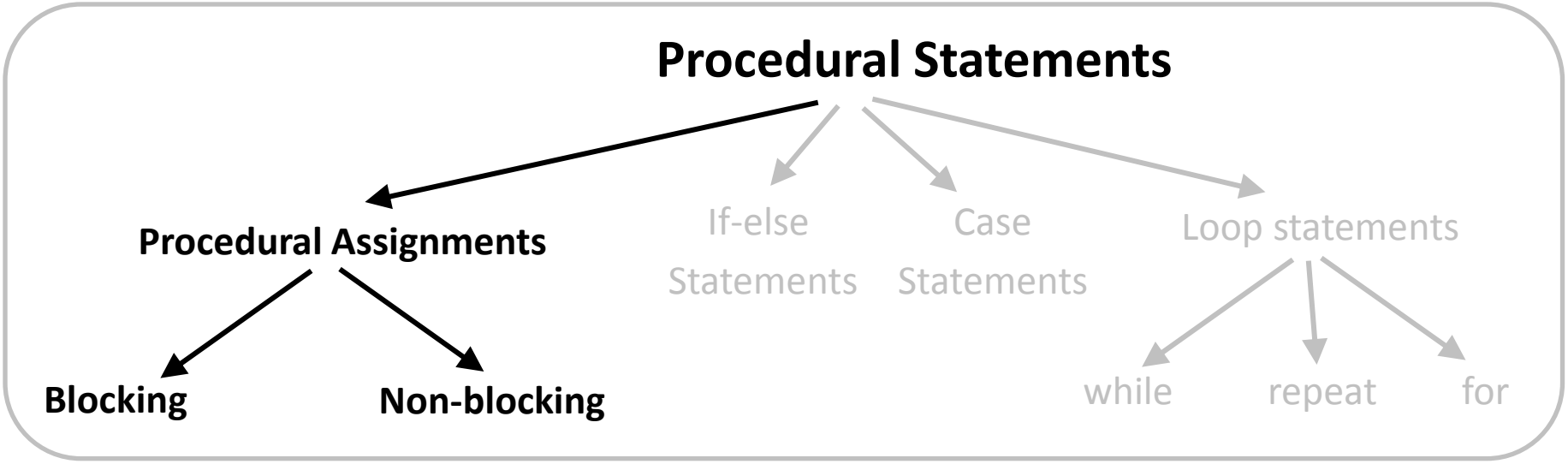
Procedural Statements



Procedural Statements

```
always @ (sensitivity list)
  begin
    ▪ Procedural assignments
    ▪ if-else statements
    ▪ case statements
    ▪ while, repeat, for loops
  end
```

Procedural Statements



- Used inside an always block and are of two types:
  - **Blocking**: denoted by "**=**" token
    - Evaluation within the always block is "blocked" until this assignment is completed
  - **Non-blocking**: denoted by "**<=**" token
    - Nothing is hold or blocked (parallel evaluation)

Blocking vs. Non-Blocking Assignments

❖ Example: assume S=2 then

Blocking

```
always @ (*)
    begin
        S = 4;
        a = S;
    end
```

S=4 & a=4
(sequential)

Non-Blocking

```
always @ (*)
    begin
        S <= 4;
        a <= S;
    end
```

S=4 & a=2
(Parallel)

- ❑ Evaluated and assigned in a single step
- ❑ Sequential nature
- ❑ Assignment ordering **IS** important
- ❑ S=4 "blocks" a=S to be evaluated
  - ➢ a=S has to wait for S=4 to be evaluated first

- ❑ Evaluated and assigned in two steps
  1. All RHSs are evaluated in parallel
  2. Assignments to LHSs are performed together
- ❑ They all are evaluated all at once
- ❑ Assignment ordering is **NOT** important
- ❑ S<=4 and a<=S evaluated in parallel

Blocking vs. Non-Blocking Assignments

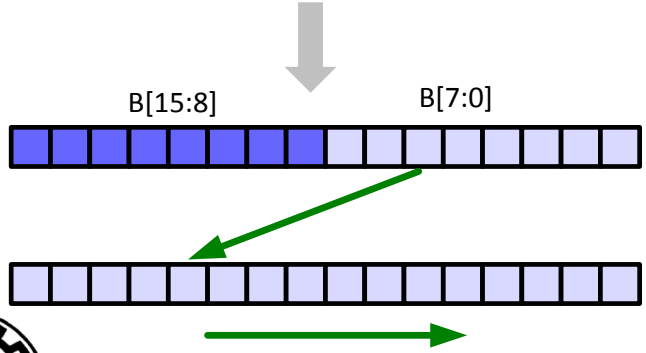
❖ Example: Swap bytes in words

❑ Which one is correct?

Blocking

Incorrect

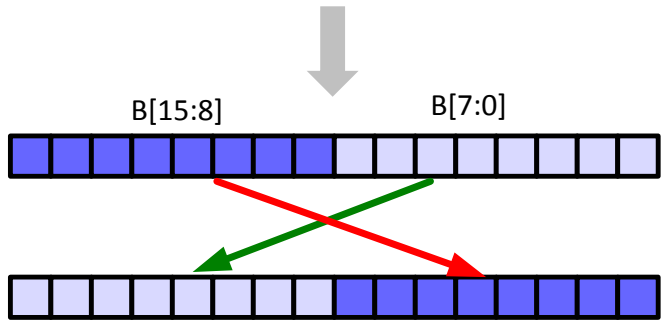
```
always @ (*)
    begin
        B[15:8] = B[7:0];
        B[7:0]  = B[15:8] ;
    end
```



Non-Blocking

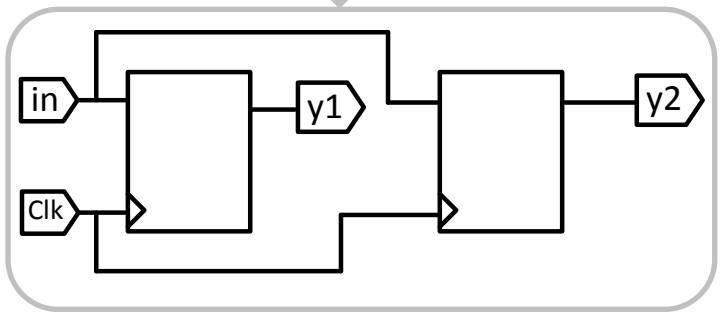
Correct

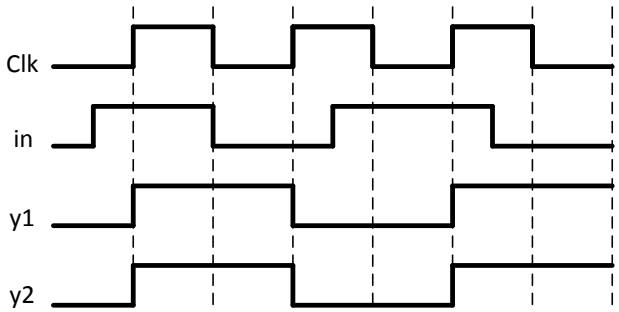
```
always @ (*)
    begin
        B[15:8] <= B[7:0];
        B[7:0]  <= B[15:8] ;
    end
```



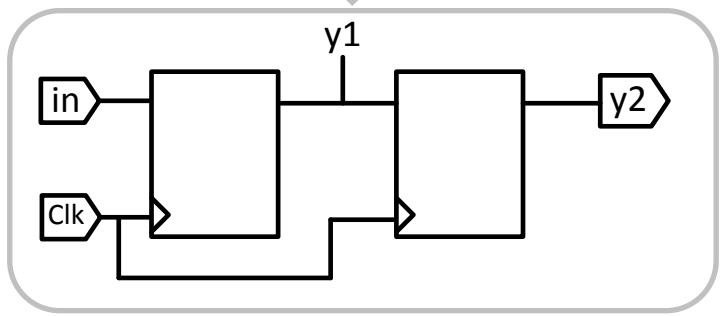
Blocking vs. Non-Blocking after Synthesis:

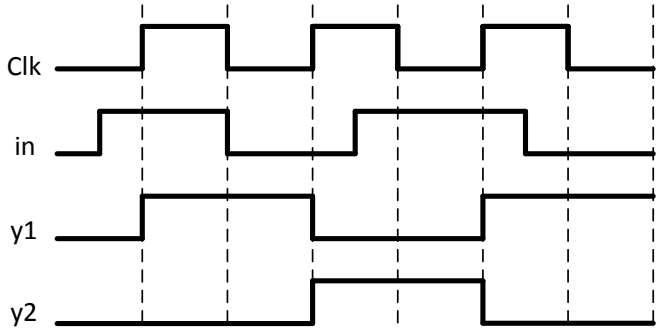
```
always @ (posedge Clk)
    begin
        y1 = in;
        y2 = y1 ;
    end
```



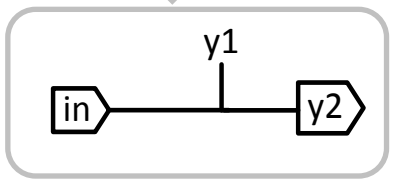



```
always @ (posedge Clk)
    begin
        y1 <= in;
        y2 <= y1 ;
    end
```



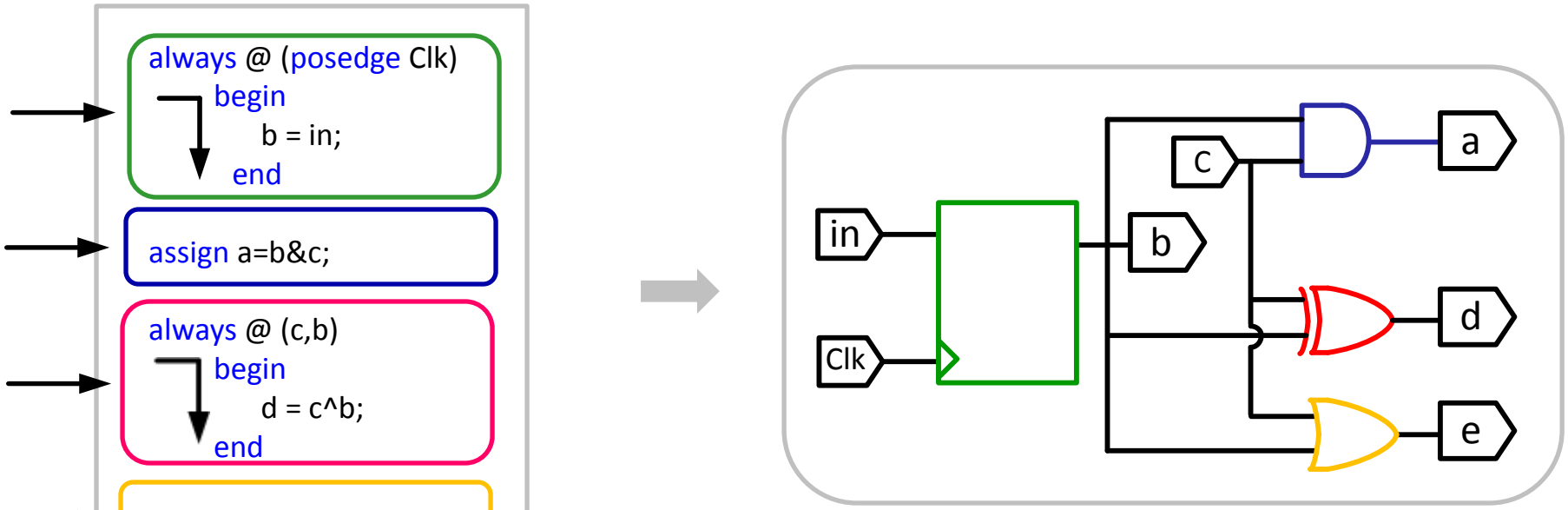



```
always @ (*)
    begin
        y1 = in;
        y2 = y1 ;
    end
```

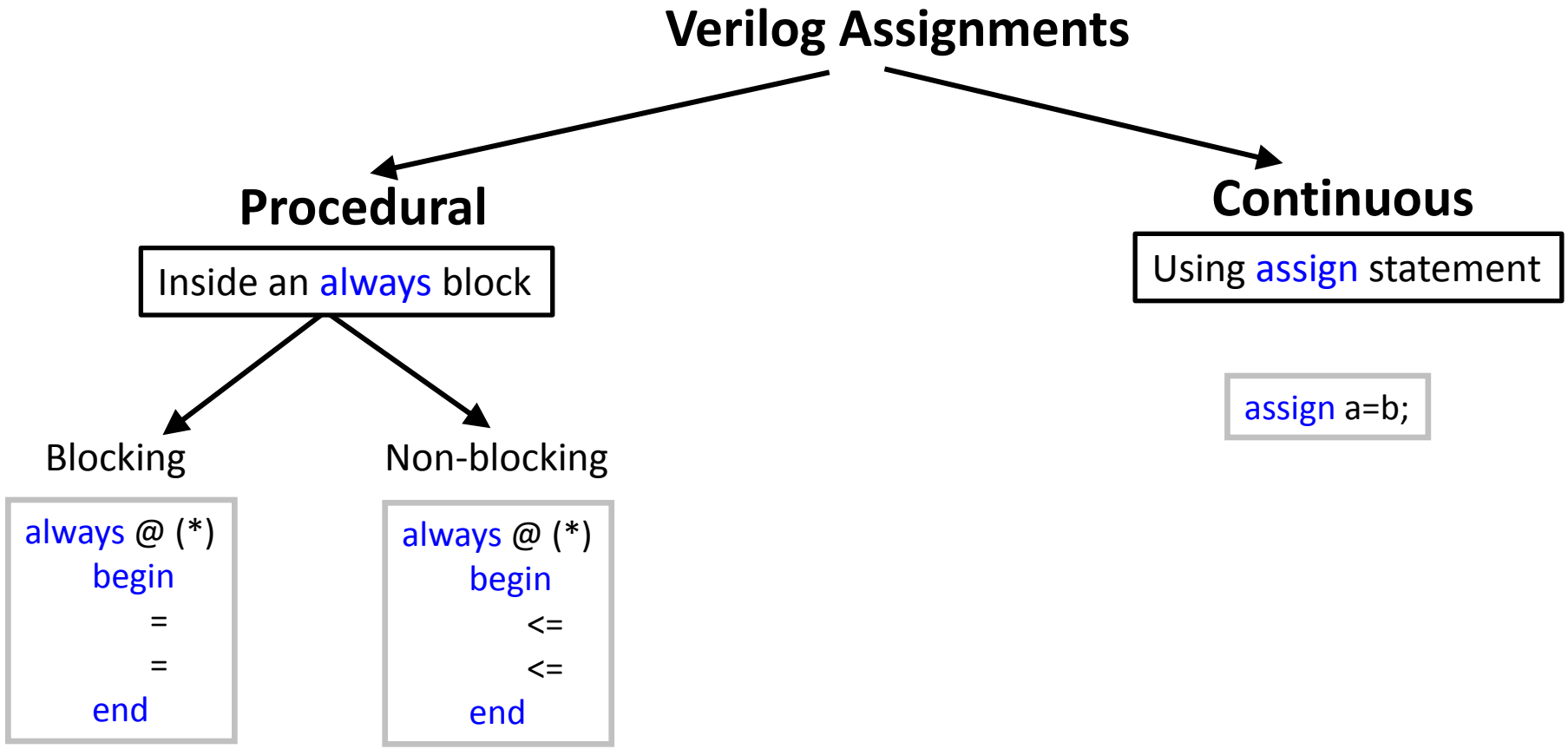


Overall Code Parallelism

- Statements inside an always block are evaluated sequentially
- However, all always blocks are evaluated concurrently
- All continuous assignments are evaluated concurrently too



Verilog Assignments in a Glance



- assign can not be used inside an always block b/c assign is used for nets.
  - Nets can not be assigned inside an always blocks (only reg or integer).

Outline

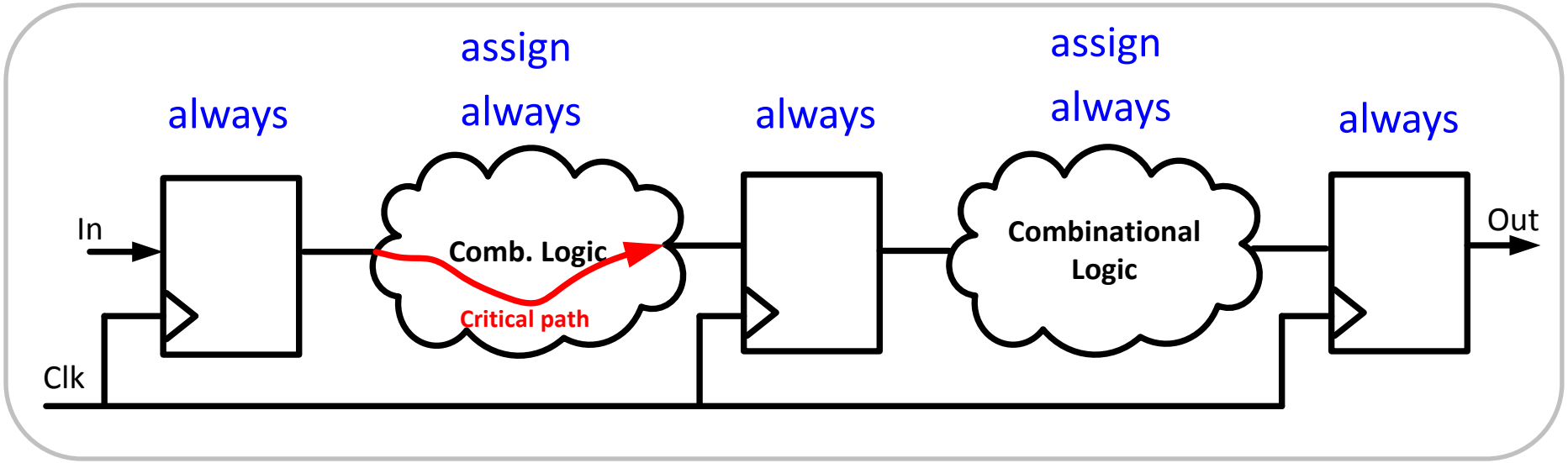
- ASIC/FPGA Design Flow
- Hardware Description Language (HDL)
  - Verilog
    - Introduction
    - Language Fundamentals
    - Modeling Combinational & Sequential Logic Circuits
    - Modeling Finite State Machines
    - Verilog Operations

Logic Circuits Category

- **Logic Circuits:**
  - **Combinational logic: (realized by assign and always)**
    - Output depends on inputs
    - Inputs propagates to the output through some gates with delay
    - e.g., adders, Mux, multiplier, all logic gates
  - **Sequential Logic: (realized only by always)**
    - Output depends on inputs and circuit history
    - Circuit history is kept using flip-flops, registers or latches
    - e.g., Finite State Machines (FSM), shift registers, Flip Flops (FF)
- Sequential logic has two flavors:
  - **Synchronous:** all registers controlled by a global clock
  - **Asynchronous:** based on the handshaking process

Logic Circuits Category

- A general system consists of both combinational and sequential circuits



- Critical path of the Comb. Logic determines the max operating frequency
- Combinational logic can be realized using assign and always constructs
- Sequential logic can only be realized using always blocks.

Combinational Logic

- Combinational logic can be realized using assign and always constructs

❖Example: Full Adder:

```
module Adder (x, y, S, C)
   input x,y;
   output S,C;
   reg S, C;
   always @ (x, y)
      begin
         S = x Λ y;
         C = x & y;
      end
endmodule
```

```
module Adder (x, y, S, C)
   input x,y;
   output S,C;
   wire S, C;
   assign S = x Λ y;
   assign C = x & y;
endmodule
```

- When using always block for Com. Logic, "blocking" assignments are used
- When using an always block, time instant changes when one of the sensitivity list variables changes

Blocking Assignment for Combinational Logic

- Use only blocking assignments for combinational logic. Why?

❖Example: Accumulator: (Assume Count == 0)

```
always @ (*)
    begin
     for (k=0; k<4; k=k+1)
            Count = Count + k;
    end
```

```
Count = Count + 0;
Count = 0 + 1;
Count = 0 + 1 + 2;
Count = 0 + 1 + 2 + 3;
Result: Count = 6
```

Correct

```
always @ (*)
    begin
     for (k=0; k<4; k=k+1)
            Count <= Count + k;
    end
```

```
Count <= Count + 0;
Count <= Count + 1;
Count <= Count + 2;
Count <= Count + 3;
Result: Count =3
```

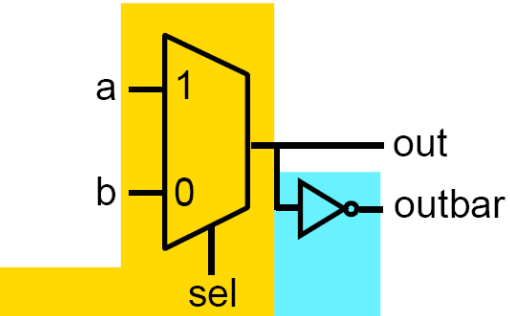
Incorrect

In multiple concurrent non-blocking assignments to a variable, the last one executes

Combinational Logic

- Procedural and continuous assignments can (and often do) co-exist within a module
- Procedural assignments update the value of `reg`. The value will remain unchanged till another procedural assignment updates the variable. This is the main difference with continuous assignments in which the right hand expression is constantly placed on the left-side

```
module mux_2_to_1(a, b, out,
                  outbar, sel);
  input a, b, sel;
  output out, outbar;
  reg out;

  always @ (a or b or sel)
  begin
    if (sel) out = a;
    else out = b;
  end

  assign outbar = ~out;

endmodule
```



procedural description

continuous description

always block vs. assign for Combinational Logic

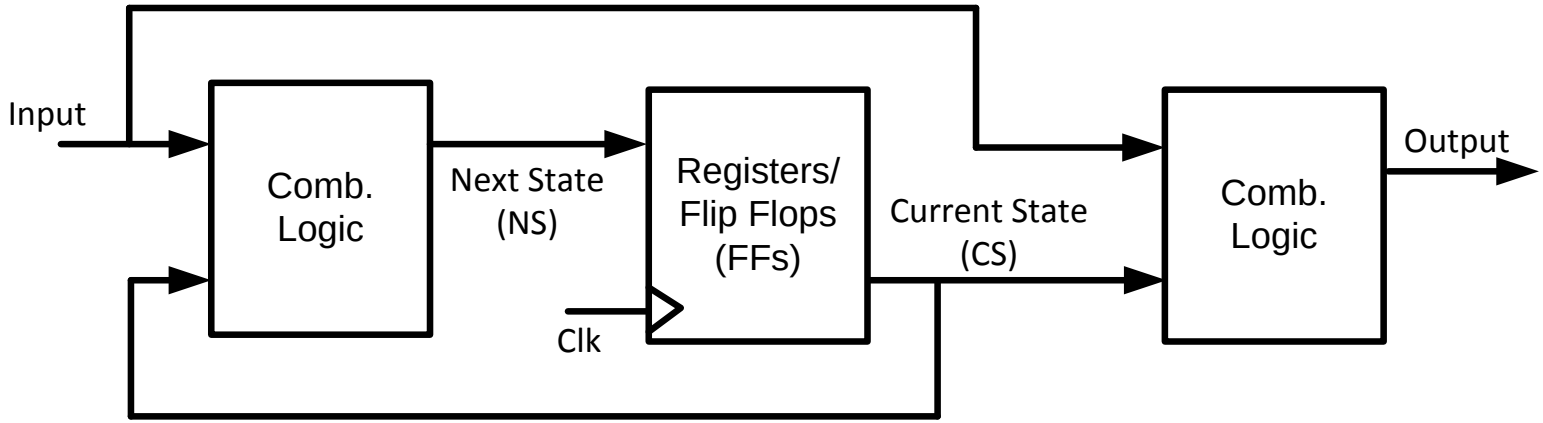
- When do we use the always block to describe a combinational logic?
  1. Normally for high-complexity Comb. Logic
  2. When output depends on several conditions, which requires if-else or case constructs to be fully described

- Why?

  1. Because powerful statements like if-else and loop constructs can only be used inside an always block
     - Comes with more clarity and more concise description than assign
  2. Multiple outputs can be assigned within a single always block

Sequential Logic

- Sequential circuits have memory (i.e., remembers the past)
- The current state is held in memory and the next state is computed through the combinational logic
- In a synchronous system, a global clock signal orchestrates the flow of the data and the sequence of events



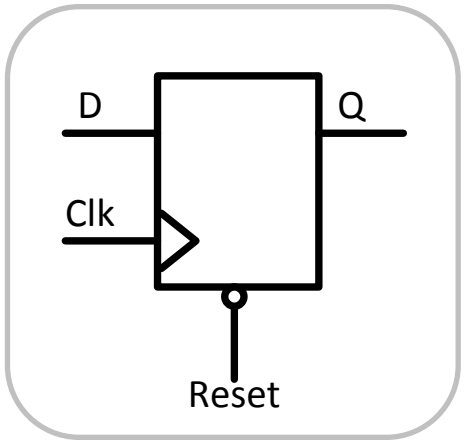
Sequential Logic

- Sequential logic can only be realized using an always block
- Consists of :
  - **Flip flops** that are normally controlled by:
    - Positive edge of the clock (posedge) → always @ (posedge Clk)
    - Negative edge of the clock (negedge) → always @ (negedge Clk)
    - Have posedge or negedge in the sensitivity list
    - Any variable assigned a value is the output of a flip-flop
  - **Latches**
    - Transfers input to output when clock is "1" and stores the value O.W.
  - **Finite State Machine (FSM)**
- When using the always block for the sequential Logic, "**Non-blocking**" assignments are used

Sequential Logic: Flip-Flop

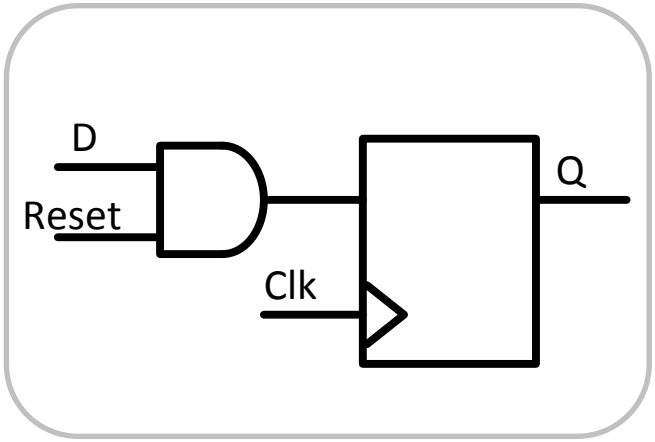
❖Example: Flip-flop with asynchronous Reset:

```
always @ (posedge Clk, negedge Reset)
      if (Reset == 0)
            Q<=0;
      else
            Q<=D;
```



❖Example: Flip-flop with synchronous Reset:

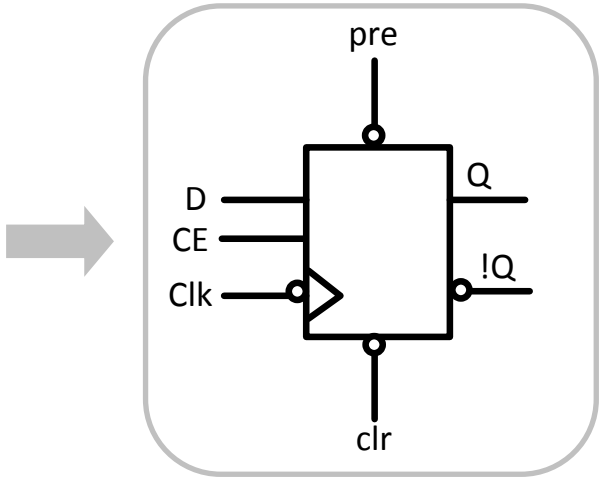
```
always @ (posedge Clk)
      if (Reset == 0)
            Q<=0;
      else
            Q<=D;
```



Sequential Logic: Flip-Flop

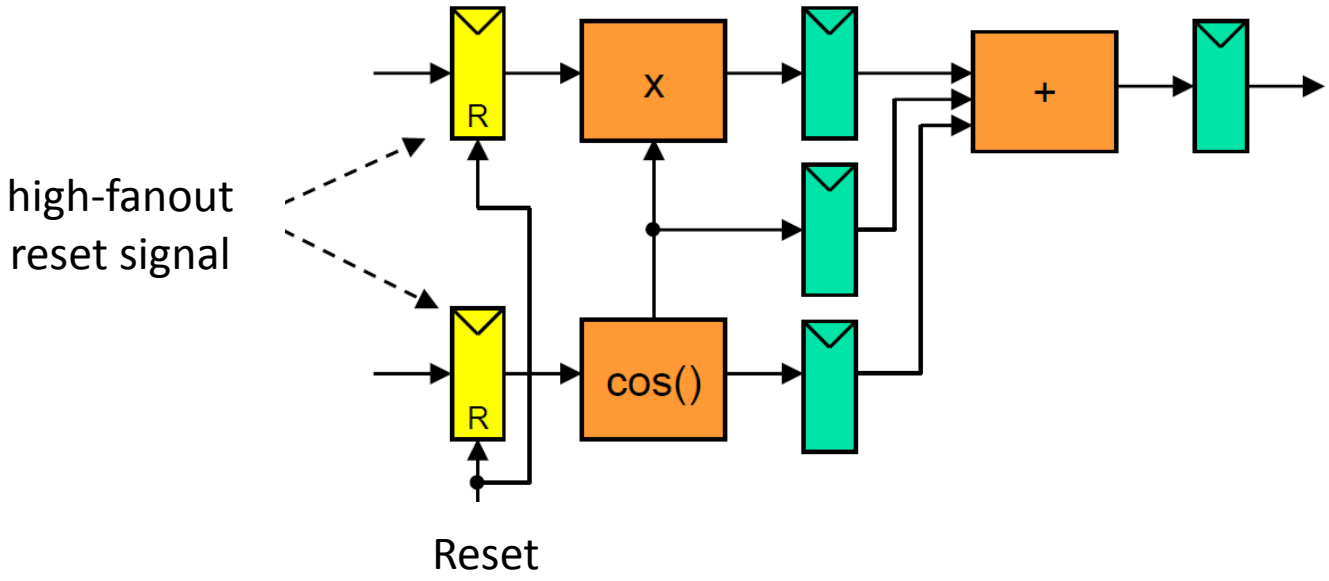
❖Example: Flip-flop with complete features:

```
module flip_flop_n ( output reg Q , output Q_n ,input pre_n, clr_n,
D, input clk_n, CE );

always@ (negedge clk_n or negedge pre_n or negedge clr_n)
        begin
                if (!pre_n) Q <= 1'b1;
                elseif (!clr_n) Q <= 1'b0;
                elseif (CE) Q <= D;
        end
assign Q_n = ~Q;
endmodule
```



Sequential Logic: Flip-Flop

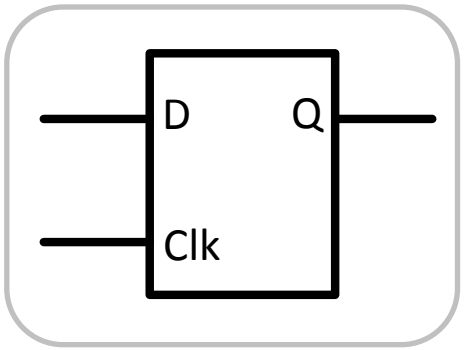
- Use reset-able FFs only where needed
  - FFs are a little larger and higher power
  - Requires the global routing of the high-fanout reset signal



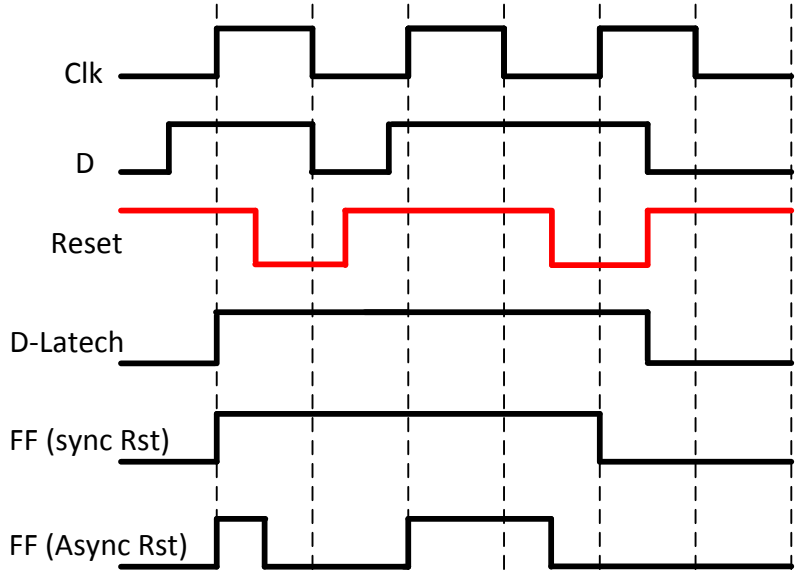
Sequential Logic

❖Example: D-Latch:

```
module Latch(D, Clk, Q);
  input D, Clk;
  output reg Q;
  always @ (D, Clk)
     if (Clk)
            Q<=D;
endmodule
```



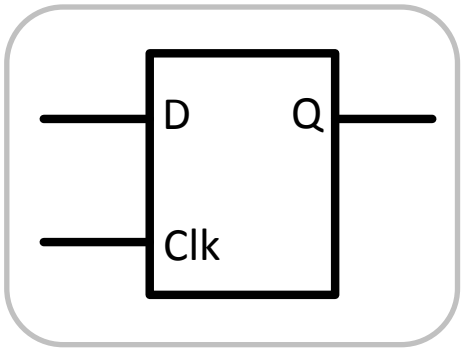
❖Example:



Sequential Logic

❖Example: D-Latch:

```
module Latch(D, Clk, Q);
  input D, Clk;
  output reg Q;
  always @ (D, Clk)
     if (Clk)
            Q<=D;
endmodule
```



Both results in a latch

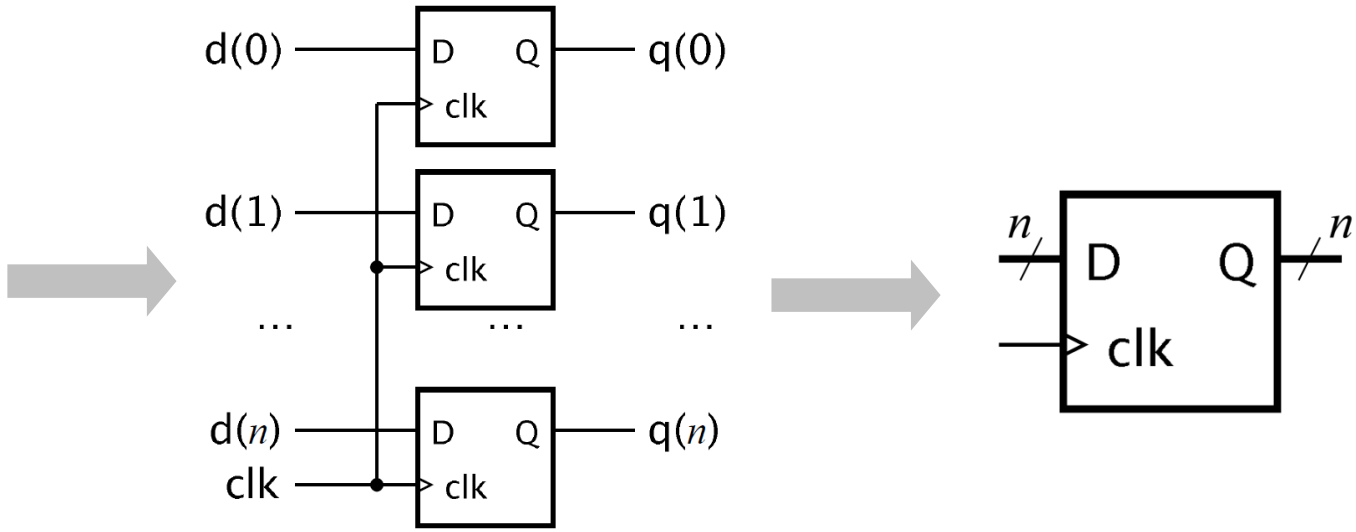
```
module Latch(D, Clk, Q);
  input D, Clk;
  output reg Q;
  always @ (Clk)
     if (Clk)
            Q<=D;
endmodule
```

This results in a warning saying D is not in the sensitivity list

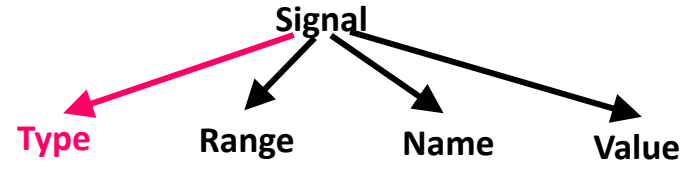
Sequential Logic: Registers

- Store a multi-bit encoded value
  - One D-FF per bit
  - Stores a new value on each clock cycle

```
wire [n:0] d;
reg [n:0] q;
...
always @ (posedge Clk)
                q<=d;
```



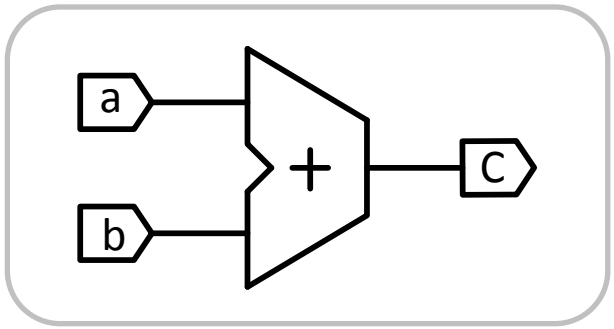
Reg Type (Revisited)



- The keyword “reg” does NOT necessarily denote a storage element or register.
- “reg” simply means a variable that can hold a value
- May or may not be synthesized as a register.

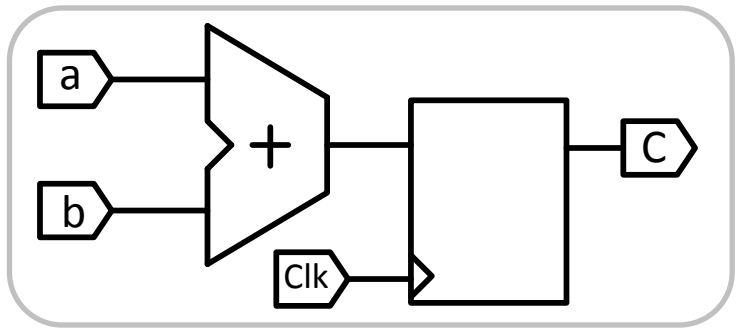
Not Register

```
reg  C;
   always @ (a,b)
      C = a+b;
```



Register

```
reg  C;
   always @ (posedge Clk)
      C <= a+b;
```

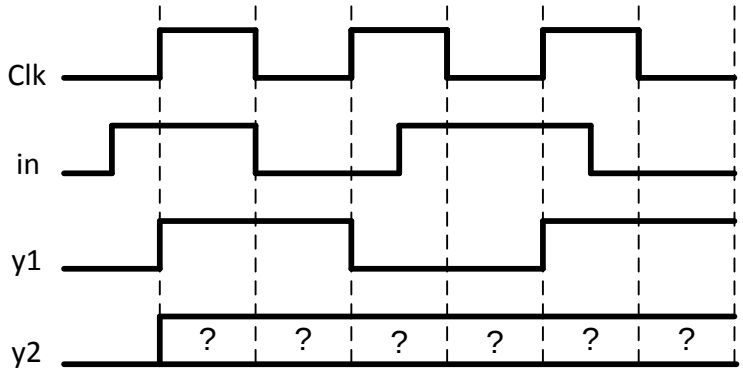


Sequential Logic

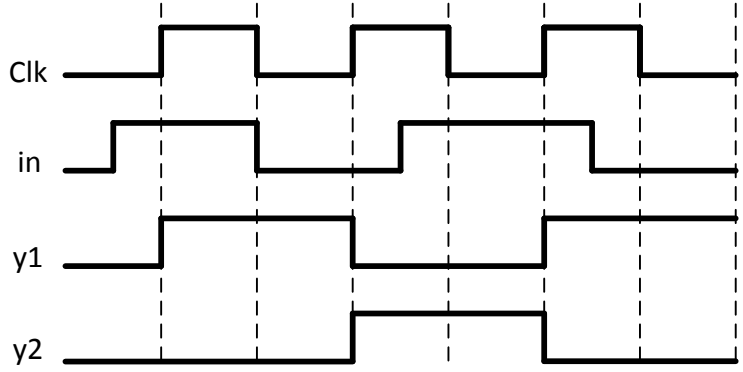
- When using always block for sequential Logic, "Non-blocking" assignments are used. Why?

```
always @ (posedge Clk)
        y1=in;

always @ (posedge Clk)
        y2=y1;
```

Race Condition




```
always @ (posedge Clk)
        y1<=in;

always @ (posedge Clk)
        y2<=y1;
```



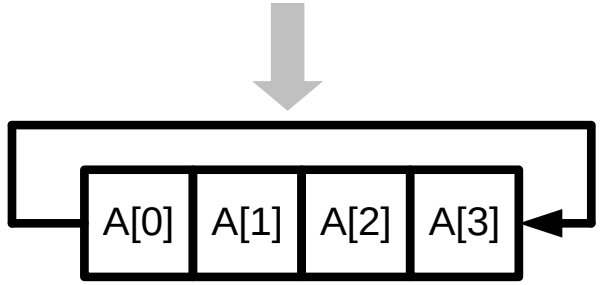
Sequential Logic

- When using always blocks for sequential Logic, "Non-blocking" assignments are used. Why?
  - Example: Shift register

```
always @ (A)
    begin
      for (k=0; k<4;k=k+1)
          A[k]=A[k+1];
      A[3] = A[0];
    end
```

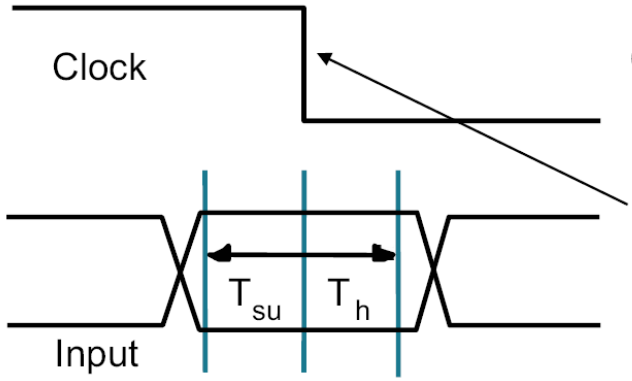
Incorrect!

```
always @ (A)
    begin
      for (k=0; k<4;k=k+1)
          A[k]<=A[k+1];
      A[3] <= A[0];
    end
```



- Do **NOT** use blocking assignments for sequential logic

Important Timing Parameters



Clock:
Periodic Event, causes state of memory element to change

memory element can be updated on the: rising *edge*, falling *edge*, high *level*, low *level*

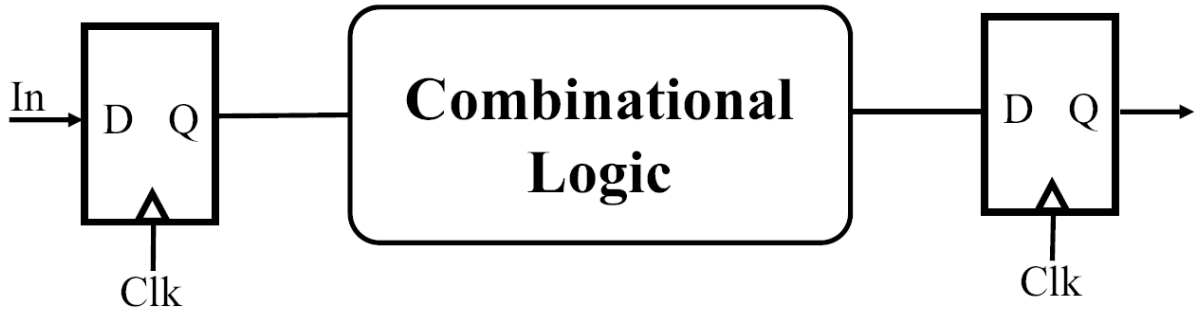
Setup Time (T_{su})
Minimum time before the clocking event by which the input must be stable

Hold Time (T_h)
Minimum time after the clocking event during which the input must remain stable

Propagation Delay (T_{cq} for an edge-triggered register and T_{dq} for a latch)
Delay overhead of the memory element

There is a timing "window" around the clocking event during which the input must remain stable and unchanged in order to be recognized

System Timing Parameters



Register Timing Parameters

T_{cq} : worst case rising edge clock to q delay

$T_{cq,\,cd}$: contamination or minimum delay from clock to q

T_{su}: setup time

T_{h}: hold time

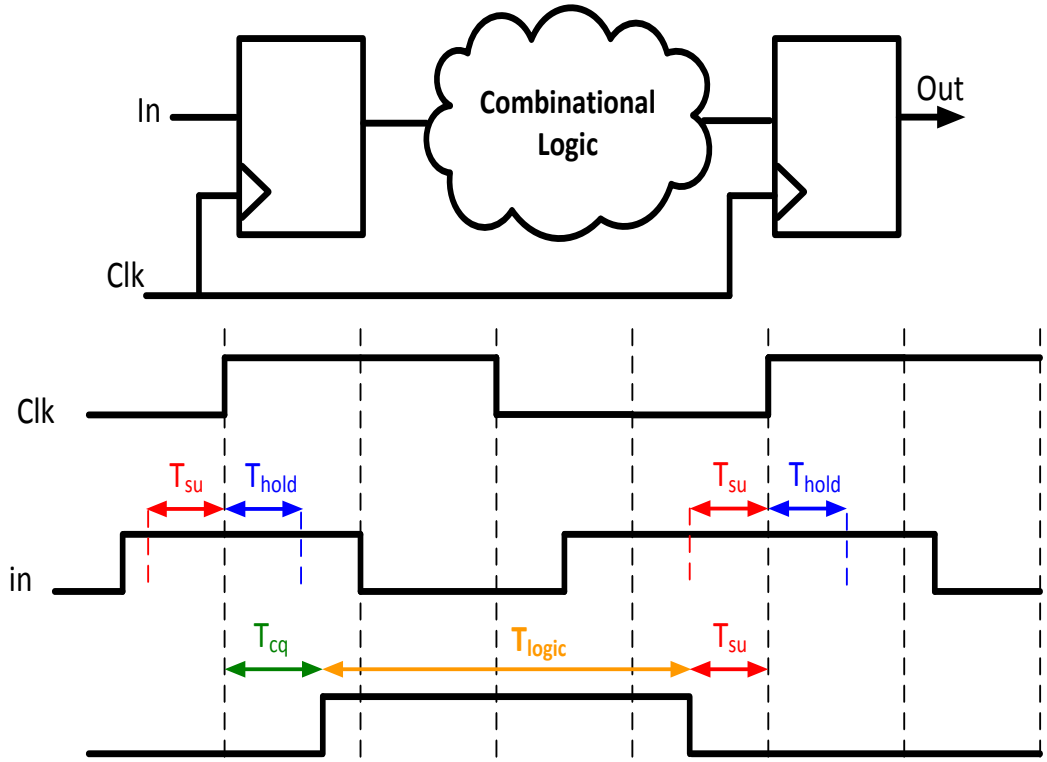
Logic Timing Parameters

T_{logic} : worst case delay through the combinational logic network

$T_{logic,cd}$: contamination or minimum delay through logic network

System Timing Parameters : Minimum Period

- **Setup-time Condition:**
  - If violates circuit works at lower frequency (why?)



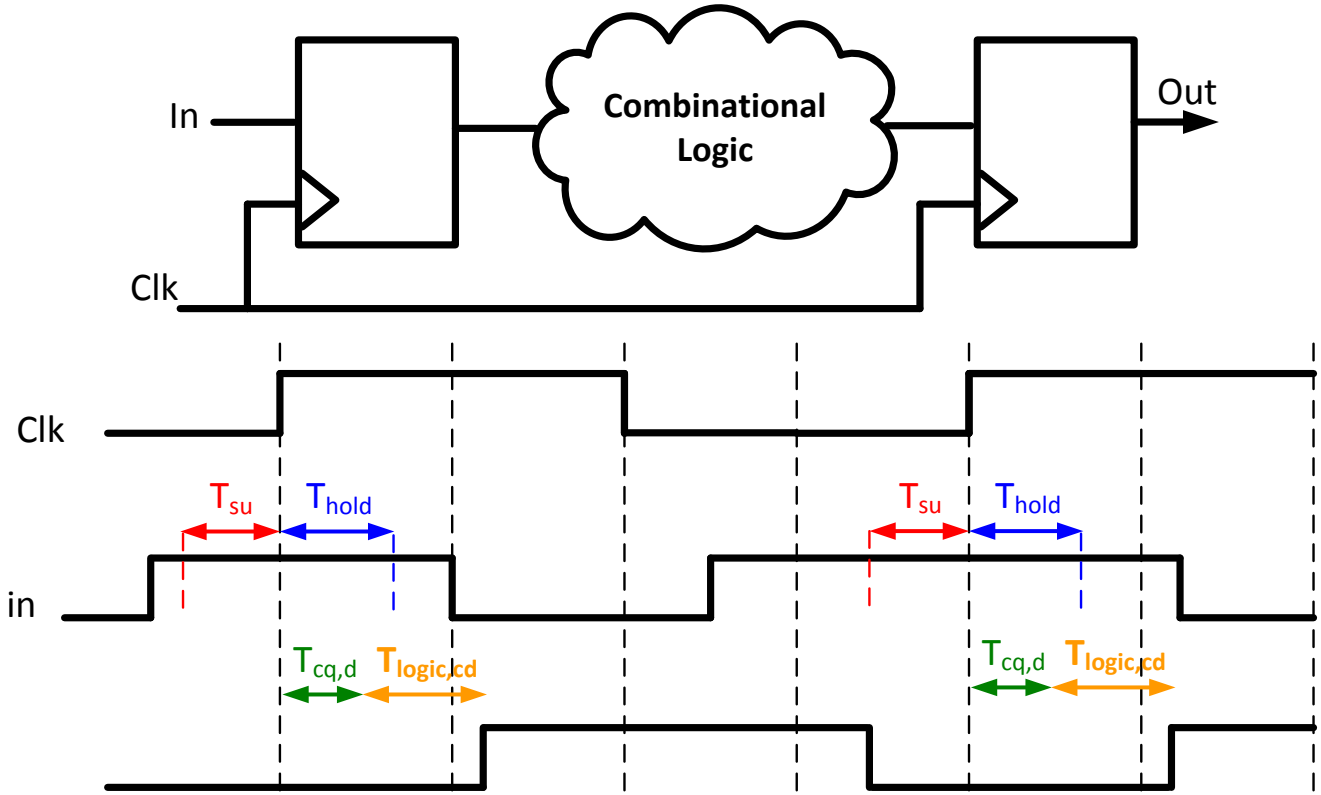
$$T_{Clk} > T_{cq} + T_{logic} + T_{su}$$

$$T_{logic} < T_{Clk} - T_{su} - T_{cq}$$

System Timing Parameters : Minimum Delay

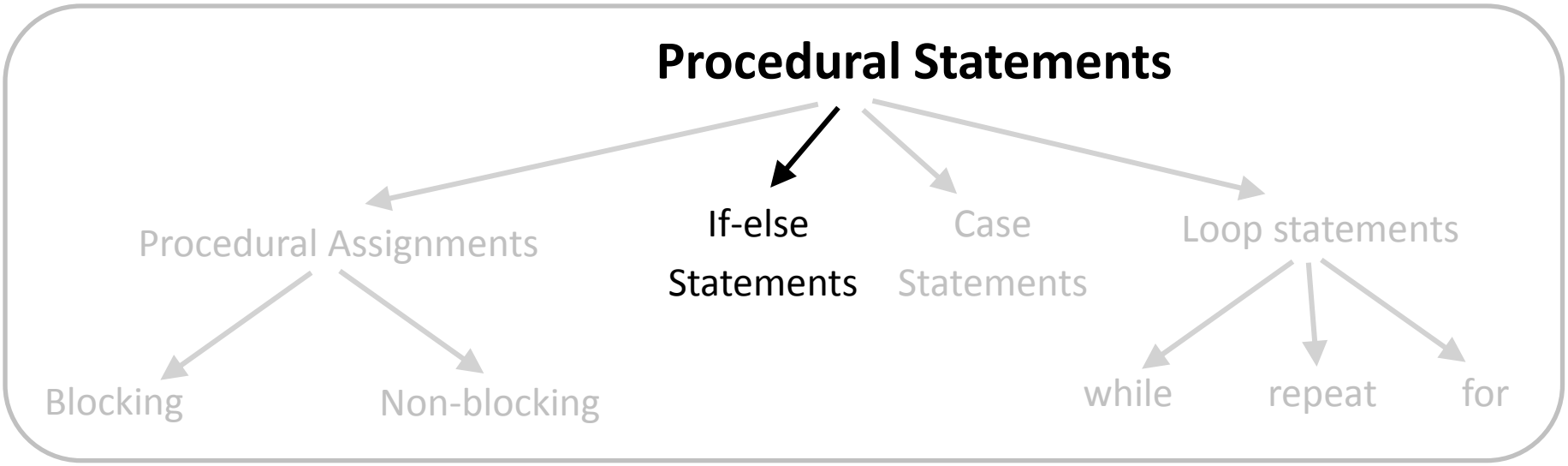
❑ Hold-time Condition:

- ➢ If violates circuit does not work (even at lower frequencies) (why?)



$$T_{cq,cd}+T_{logic,cd}>T_{hold}$$

Procedural Statements



Procedural Statements

```
always @ (sensitivity list)
  begin
    ▪ Procedural assignments
    ▪ if-else statements
    ▪ case statements
    ▪ while, repeat, for loops
  end
```

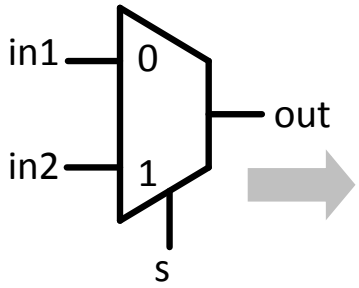
If-else statements

- Used only inside an always block
- Format:

```
If (expression1)
   statement1;
else if (expression2)
   statement2;
else
   statement3;
```

Single statement no need for begin-end
Multiple statements, begin-end is needed

- Example:



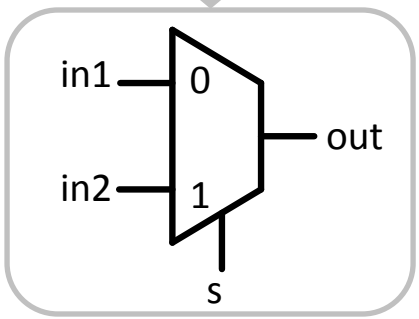

```
module Mux21 (in1, in2, s, out)
   input in1, in2, s;
   output reg out;

   always @ (in1, in2, s)
      if (s==0)
         out = in1;
      else
         out = in2;
endmodule
```

```
module Mux21 (in1, in2, s, out)
   input in1, in2, s;
   output reg out;
   always @ (in1, in2, s)
      begin
         out = in1;
         if (s==1)
            out = in2;
      end
endmodule
```

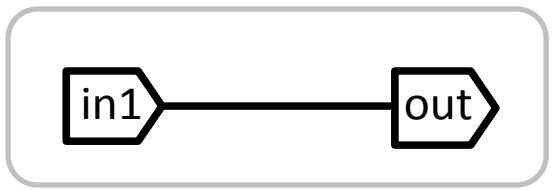
If-else statements

- If-else construct inside an always block have a sequential nature when used by blocking assignments. Sequential means direct effect on synthesis not necessarily sequential in actual hardware implementation
- This means ordering is important
- Example:

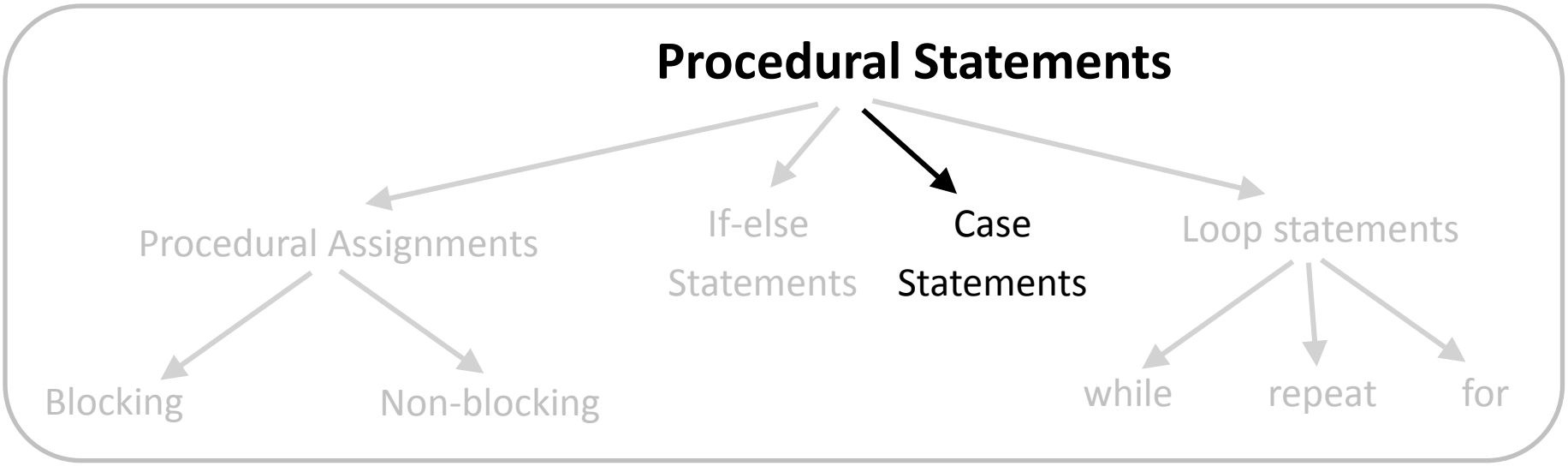
```
always @ (*)
    begin
        out = in1;
        if (s==1)
            out = in2;
    end
```




```
always @ (*)
    begin
        if (s==1)
            out = in2;
        out = in1;
    end
```



Procedural Statements



Procedural Statements

```
always @ (sensitivity list)
  begin
    ▪ Procedural assignments
    ▪ if-else statements
    ▪ case statements
    ▪ while, repeat, for loops
  end
```

Case statements

- Used only inside an always block
- Format:

```
case (expression)
   alternative1: statement1;
   alternative2: begin
                    statement2;
                 end
   default: statementn;
endcase
```

Single statement no need for begin-end

Multiple statements, begin-end is needed

- Example:

```
module Mux21 (in1, in2, s, out)
   input in1, in2, s;
   output reg out;

   always @ (in1, in2, s)
      case (s)
         1'b0: out = in1;
         1'b1: out = in2;
      endcase
endmodule
```

⟺

```
module Mux21 (in1, in2, s, out)
   input in1, in2, s;
   output reg out;

   always @ (in1, in2, s)
      case (s)
         1'b0: out = in1;
         default: out = in2;
      endcase
endmodule
```

Case statements

❖ Example: Combinational logic using both assign and always block

```
module FullAdder (Cin, x, y, S, Cout)
   input  x, y, Cin;
   output  S, Cout;
   wire S, Cout;

   assign S = x ^ y ^ Cin;

   assign Cout = (x & y)|(x & Cin)|(y & Cin);

endmodule
```

x	y	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

```
module FullAdder (Cin, x, y, S, Cout)
   input  x, y, Cin;
   output reg  S, Cout;
   always @ (Cin, x, y)
      begin
         case ({Cin, x, y})
            3'b000: {Cout, S} = 'b00;
            3'b001: {Cout, S} = 'b01;
            3'b010: {Cout, S} = 'b01;
            3'b011: {Cout, S} = 'b10;
            3'b100: {Cout, S} = 'b01;
            3'b101: {Cout, S} = 'b10;
            3'b110: {Cout, S} = 'b10;
            3'b111: {Cout, S} = 'b11;
         endcase
      end
endmodule
```

concatenation

Case statements

- In case statements, each alternative is compared for an exact match
- Synthesis tools are only concerned about matching of “0” and “1” while “Z” and “X” are not important
- If “X” or “Z” are needed to be added, casex is used (casex is synthesizable).
- In fact casex treats them as don’t care.
- Example: 4-to-2 priority encoder

w3	w2	w1	w0	y1	y0	f
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

```
module Priority (W, Y, f)
  input [3:0] W;
  output reg [1:0] Y;
  output f;
  assign f = (W!=0)
  always @ (W)
    begin
      casex (W)
          'b1xxx:  Y = 3;
          'b01xx:  Y = 2;
          'b001x:  Y = 1;
          default: Y = 0;
       endcase
    end
endmodule
```

Case statements

- **casez** allows use of wildcard "?" character for don't

w3	w2	w1	w0	y1	y0	f
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

```
module Priority (W, Y, f)
   input [3:0] W;
   output reg [1:0] Y;
   output f;
   assign f = (W!=0)
   always @ (W)
     begin
       casez (W)
           'b1???:  Y = 3;
           'b01??:  Y = 2;
           'b001?:  Y = 1;
           default: Y = 0;
       endcase
     end
endmodule
```

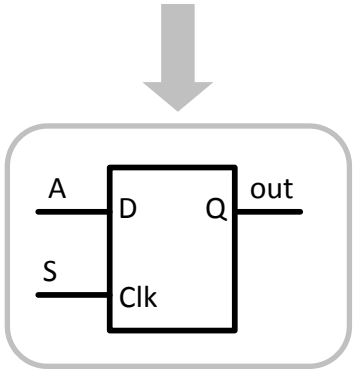
Latch Inference in Combinational Logic

- When realizing **combinational logic** with always block using if-else or case constructs care has to be taken to avoid latch inference after synthesis
- The latch is inferred when "incomplete" if-else or case statements are declared
- This latch is "unwanted" as the logic is combinational not sequential
- If there is some logic path through the always block that does not assign a value to the output, a latch is inferred

Latch Inference in If-else or Case Statements

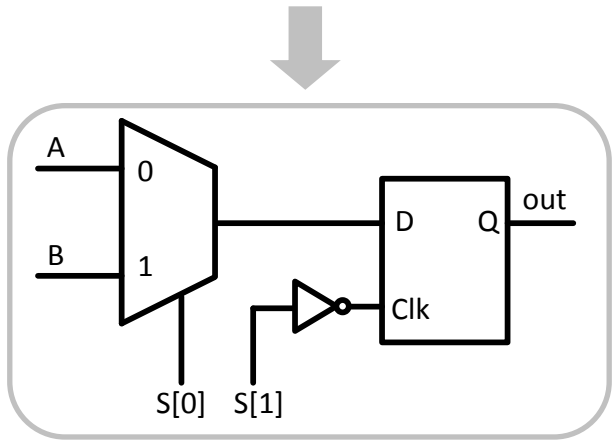
❖Example:

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
end
endmodule
```



Latch Inference

```
module DUT (A, B, S, out);
input A, B;
Input [1:0] S;
output reg out;
always @(A, B, S)
begin
   case (S)
       2'b00: out = A;
       2'b01: out = B;
   endcase
end
endmodule
```



Latch Inference

Latch Inference in Combinational Logic

- To avoid latch inference make sure to specify all possible cases "explicitly"

- Two practical approaches to avoid latch inference:

  - **For if-else construct:**
    1. Initialize the variable before the if-else construct
    2. Use else to explicitly list all possible cases
  - **For case constructs:**
    1. Use default to make sure no case is missed!

- Do NOT let it up to the synthesis tool to act in unspecified cases and do specify all cases explicitly.

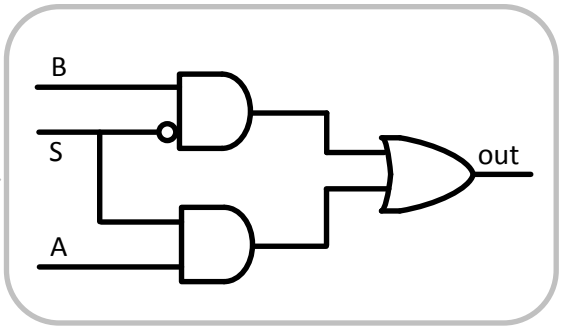
Avoid Latch Inference in If-else Statements

❖Example:

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
end
endmodule
```

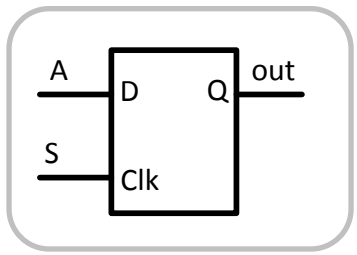
1

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
Begin
   out = B;
   if (S==1)
       out = A;
end
endmodule
```



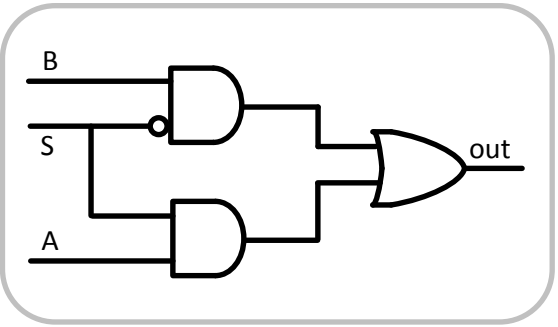
No Latch

2



Latch Inference

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
   else
       out =B;
end
endmodule
```



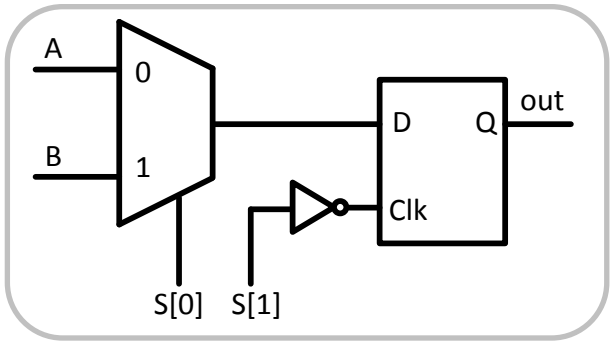
No Latch

Avoid Latch Inference in Case Statements

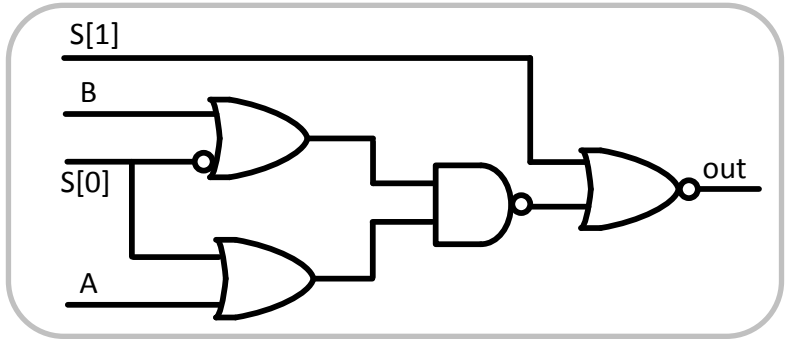
❖Example:

```
module DUT (A, B, S, out);
input A, B;
Input [1:0] S;
output reg out;

always @(A, B, S)
begin
   case (S)
      2'b00: out = A;
      2'b01: out = B;
   endcase
end
endmodule
```

```
module DUT (A, B, S, out);
input A, B;
Input [1:0] S;
output reg out;
always @(A, B, S)
begin
   case (S)
      2'b00: out = A;
      2'b01: out = B;
      default: out = 1'b0;
   endcase
end
endmodule
```

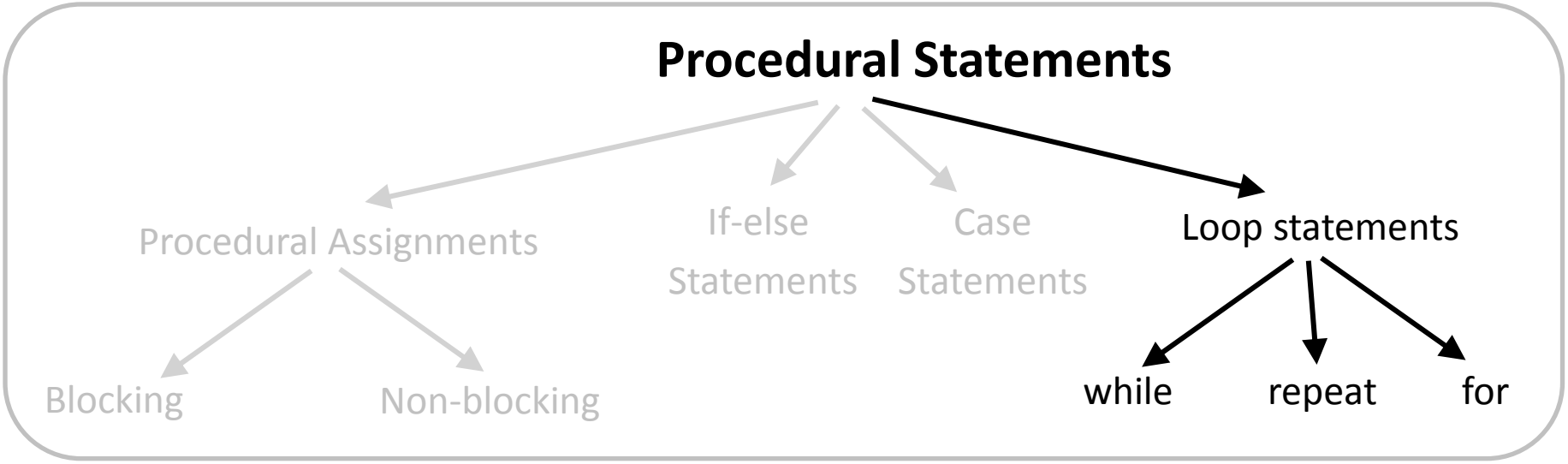


Latch Inference



No Latch

Procedural Statements



Procedural Statements

```
always @ (sensitivity list)
  begin
    ▪ Procedural assignments
    ▪ if-else statements
    ▪ case statements
    ▪ while, repeat, for loops
  end
```

Loop Statements

- To be used for efficient coding style
- **All being used inside an always block**
- Make sure to use blocking statements for combinational logic

```
for (k=0;  k<n-1; k=k+1)
    begin
        statement;
    end
```

```
while (condition)
    begin
        statement;
    end
```

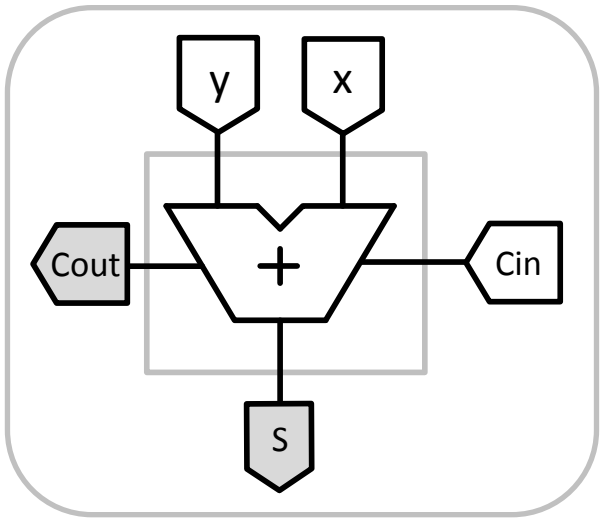
```
repeat (constant_value)
    begin
        statement;
    end
```

Single statements no need for begin-end construct
Multiple statements, begin-end construct is needed

Loop Statements

❖Example: Remember: 1-bit full Adder

```
module Full_Adder (Cin, x, y, S, Cout)
   input  x, y, Cin;
   output  S, Cout;
   wire S, Cout;

   assign S = x ^ y ^ Cin;

   assign Cout = (x & y)|(x & Cin)|(y & Cin);

endmodule
```

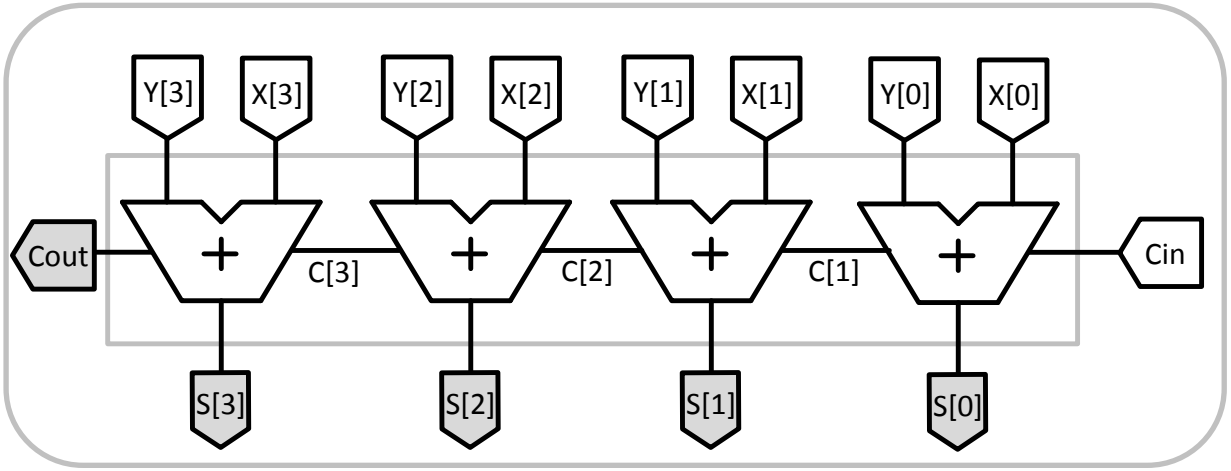


Loop Statements

❖Example: n-Bit ripple carry adder

```
module RippleCarryAdderI (Cin, X, Y, S, Cout)
   parameter n = 4;
   input  Cin;
   input [n-1:0] X, Y;
   output  reg [n-1:0] S;
   output reg  Cout;
   reg [n:0] C;
   integer k;
   always  @(X, Y, Cin)
      begin
         C[0] = Cin;
         for (k=0;k<=n-1;k=k+1)
            begin
               S[k] = X[k] ^ Y[k] ^ C[k];
               C[k+1] = (X[k] & Y[k])
                        |(X[k] & C[k])|(Y[k] & C[k]);
            end
         Cout = C[n];
      end
endmodule
```

Sequential Structure



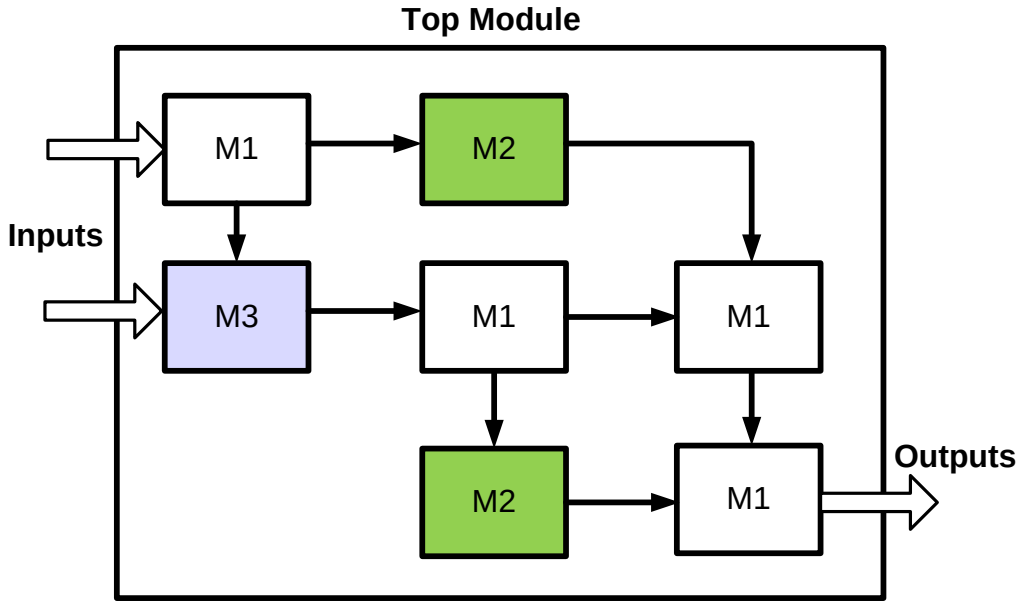
Breaking one statement in two lines is allowed!

Supported

```
module Adder (Cin, X, Y, S, Cout)
   input  Cin;
   input [3:0] X, Y;
   output [3:0] S;
   output Cout;
   assign {Cout, S} = {1'b0, X} + {1'b0, Y} + {4'b0, Cin}
endmodule
```

Using Sub-Circuits (Sub-modules)

- A design can use multiple submodules or a module multiple times
- Using a module in another is called "instantiation"
- Top-level module: the module that has not been instantiated
- To use a module inside another, it should be explicitly instantiated



Using Sub-modules

- There are some built-in **primitive** logic gates in Verilog that can be instantiated
  - Built-in primitives means there is no need to define a module for these gates
  - and, or, nor, ….

❖ Example:

1. Gate instantiation
2. assign
3. always block

```
module Myand(In1, In2, out)
   input  In1, In2;
   output out;

   and myand (out, In1, In2);

   assign out = In1 & In2;

   reg out;
   always  @(In1, In2)
      out = (In1 & In2);

endmodule
```

Using Sub-modules : Gate-level Primitives

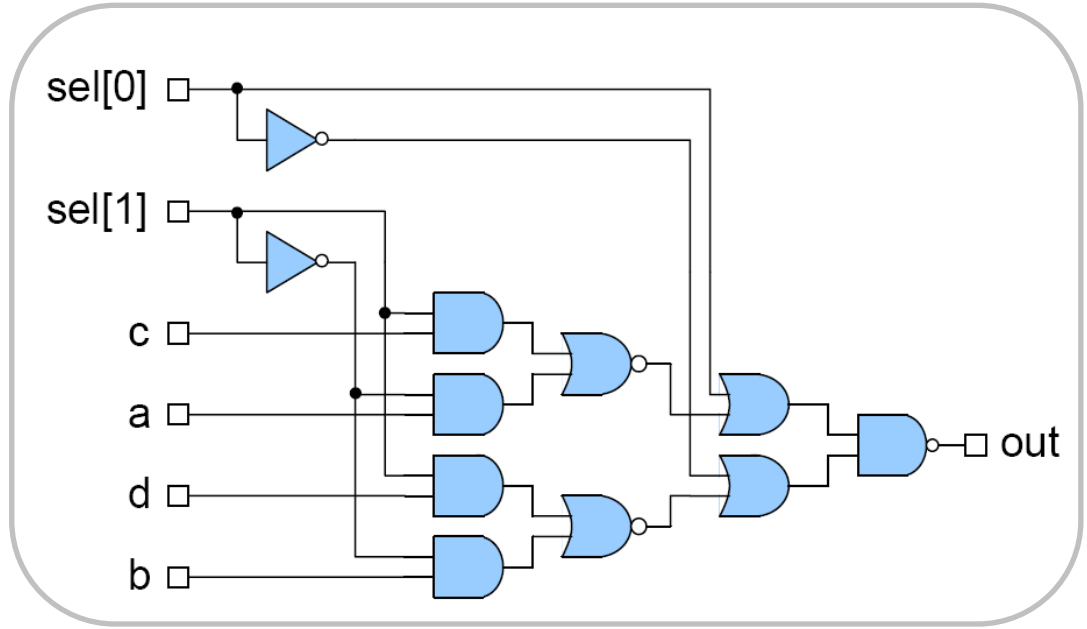
❑ **Gate-Level primitives:**

and	buf	nmos	tran
nand	not	pmos	tranif0
nor	bufif0	cmos	tranif1
or	bufif1	rnmos	rtran
xor	notif0	rpmos	rtranif0
xnor	notif1	rcmos	rtranif1

Using Sub-modules

❖ Example: (4-input MUX using primitives)

```
module mux4( input  a, b, c, d
             input [1:0] sel,
             output out );

  wire [1:0] sel_b;
  not not0( sel_b[0], sel[0] );
  not not1( sel_b[1], sel[1] );

  wire n0, n1, n2, n3;
  and and0( n0, c, sel[1]   );
  and and1( n1, a, sel_b[1] );
  and and2( n2, d, sel[1]   );
  and and3( n3, b, sel_b[1] );

  wire x0, x1;
  nor nor0( x0, n0, n1 );
  nor nor1( x1, n2, n3 );

  wire y0, y1;
  or or0( y0, x0, sel[0]   );
  or or1( y1, x1, sel_b[0] );

  nand nand0( out, y0, y1 );

endmodule
```



Sub-modules Instantiation

- To instantiate a module, two things have to be clearly specified
  - module's ports
  - module's parameters (considered as default if not specified)
- Format:

```
Module_name  #(parameter_value)  instance_name (.port_name(port-connection), .port_name(port-connection),….)
```

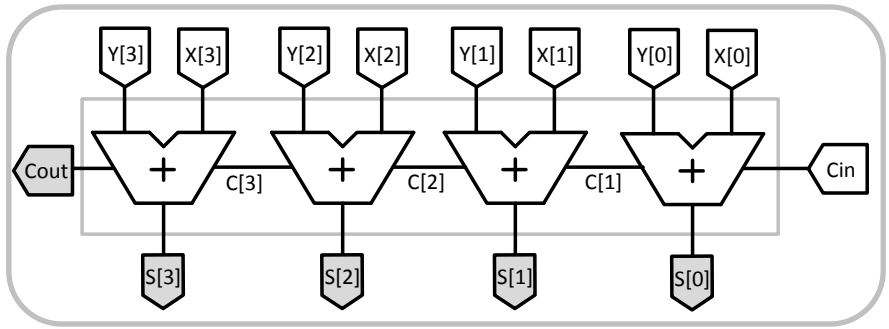
```
Module_name   instance_name (.port_name(port-connection), .port_name(port-connection),….)
defparam  instance_name.parameter_name = parameter_value
```

- If port connections are in the same order as the original module "**.port_name**" is not needed in the port list.

Sub-modules Instantiation

❖ Example:

```
module Full_Adder (Cin, x, y, S, Cout);
   input  x, y, Cin;
   output  S, Cout;
   wire S, C;
   assign S = x ^ y ^ Cin;
   assign Cout = (x & y)|(x & Cin)|(y & Cin);
endmodule
```



4-bit Ripple Carry Adder

```
module RippleCarryAdderII (Cin, X, Y, S, Cout);
   parameter n = 4;
   input  Cin;
   input [n-1:0] X, Y;
   output  [n-1:0] S;
   output  Cout;
   wire [n-1:0] C;          Cin  x   y   S   Cout

   Full_Adder stage0  (Cin, X[0], Y[0], S[0], C[1]);
   Full_Adder stage1  (C[1], X[1], Y[1], S[1], C[2]);
   Full_Adder stage2  (C[2], X[2], Y[2], S[2], C[3]);
   Full_Adder stage3  (.Cout(Cout), .Cin(C[3]), .x(X[3]), .y(Y[3]), .S(S[3]));

endmodule
```

Can **NOT** be of type "reg" (output of a submodule)

Implicit list (Order **IS** important)

Explicit list (Order **NOT** important)

Sub-modules Instantiation

❖ Example: 5-bit Ripple Carry Adder:

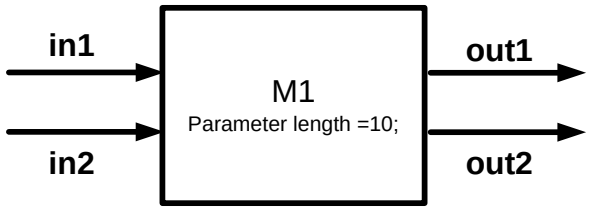
```
module 5-BitRippleCarryAdder (Cin, X, Y, S, Cout)
   parameter n = 5;
   input  Cin;
   input [n-1:0] X, Y;
   output  [n-1:0] S;
   output  Cout;
   wire  C;


   RippleCarryAdderI #(3) stage0   (.Cin(Cin), .X(X[2:0]), .Y(Y[2:0]), .S(S[2:0]), .Cout(C));

   defparam  stage1.n = 2;
   RippleCarryAdderI stage1  (.Cin(C), .X(X[4:3]), .Y(Y[4:3]), .S(S[4:3]), .Cout(Cout));

endmodule
```

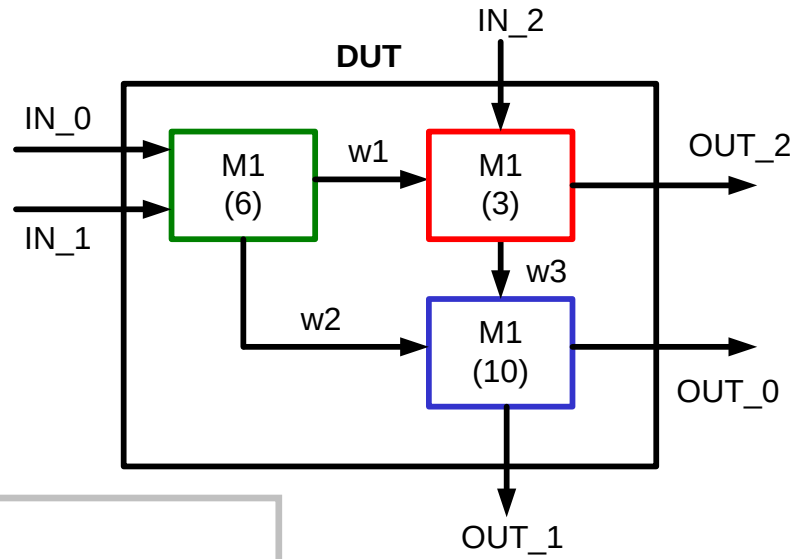
defparam stage0.n = 3;

If two parameters : # (3,8)

Sub-modules Instantiation

❖ Example:






```
module DUT (IN, OUT)
   input [2:0] IN;
   output [2:0] OUT;
   wire  w1, w2, w3;

   defparam  stage0.length = 6;
   M1 stage0  (IN[0], IN[1], w1, w2);

   defparam  stage1.length = 3;
   M1 stage1  (.in1(w1), .in2(IN[2]),  .out2(w3), .out1(OUT[2]));

   M1 stage2  (.in1(w2), .in2(w3), .out1(OUT[0]), .out2(OUT[1]));

endmodule
```

Function Construct

- function may be used to have a modular code without defining separate modules
- A function is defined inside a module
- Not crucial for Verilog but might facilitate modular coding
- A function can be called both in continuous and procedural assignments
- A function can have multiple inputs but does not have any output
- Function name serves as the output

Function Construct

❖ Example: 16-to-1 multiplexer:

```
module my16-to-1MUX (W, S, Out);
   input  [0:15] W;
   input [3:0] S;
   output  reg Out;
   reg  [0:3] M;

   function my4-to-1MUX;
       input [0:3] W;
       input [1:0] s;
       if (s==0) my4-to-1MUX = W[0];
       else if (s==1) my4-to-1MUX = W[1];
       else if (s==2) my4-to-1MUX = W[2];
       else if (s==3) my4-to-1MUX = W[3];
   endfunction

   always@ (W, S)
      begin
         M[0] = my4-to-1MUX(W[0:3],S[1:0]);
         M[1] = my4-to-1MUX(W[4:7],S[1:0]);
         M[2] = my4-to-1MUX(W[8:11],S[1:0]);
         M[3] = my4-to-1MUX(W[12:15],S[1:0]);

         Out   = my4-to-1MUX(M[0:3], S[3:2]);

      end
endmodule
```

```
if      (S[3:2]==0) Out= M[0];
else if (S[3:2]==1) Out= M[1];
else if (S[3:2]==2) Out= M[2];
else if (S[3:2]==3) Out= M[3];
```

Function Construct with multiple-bit output

❖ Example:

```
module test_fcn (a, b, c, Out);
   input  a, b, c;
   output reg [2:0] Out;

function [2:0] myfcn;
   input a, b, c;
   begin
      myfcn[0] = a^b;
      myfcn[1] = b^c;
      myfcn[2] = c^a;
   end
endfunction

always @(*)
    Out = myfcn(a,b,c);

endmodule
```

With always

```
module test_fcn (a, b, c, Out);
   input  a, b, c;
   output [2:0] Out;

function [2:0] myfcn;
   input a, b, c;
   begin
      myfcn[0] = a^b;
      myfcn[1] = b^c;
      myfcn[2] = c^a;
   end
endfunction

assign   Out = myfcn(a,b,c);

endmodule
```

With assign

Task Construct

- task may be used to have a modular code without defining separate modules
- A task is defined inside a module
- A task can only be called from inside and always (or initial) block
- A task can have multiple inputs and outputs

Task Construct

❖ Example: 16-to-1 multiplexer:

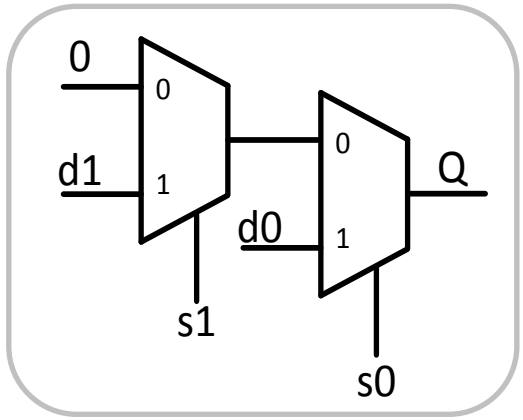
```
module 16-to-1MUX (W, S, Out)
   input  [0:15] W;
   input [3:0] S;
   output  reg Out;
   reg  [0:3] M;

    task 4-to-1MUX;
      input [0:3] W;
      input [1:0] s;
      output Result;
      begin
          if (s==0) Result= W[0];
          elseif (s==1) Result = W[1];
          elseif (s==2) Result = W[2];
          elseif (s==3) Result = W[3];
       end
   endtask
   always@ (W, S)
     begin
        4-to-1MUX(W[0:3],S[1:0], M[0]);
        4-to-1MUX(W[4:7],S[1:0] , M[1]);
        4-to-1MUX(W[8:11],S[1:0] , M[2]);
        4-to-1MUX(W[12:15],S[1:0] , M[3]);
        4-to-1MUX(M[0:3],S[3:2] , Out);
     end
endmodule
```

HDL for Synthesis (Priority logic)

- The order in which assignments are written in an always block may affect the logic that is synthesized. **(both conditions in if and else if can be true!)**

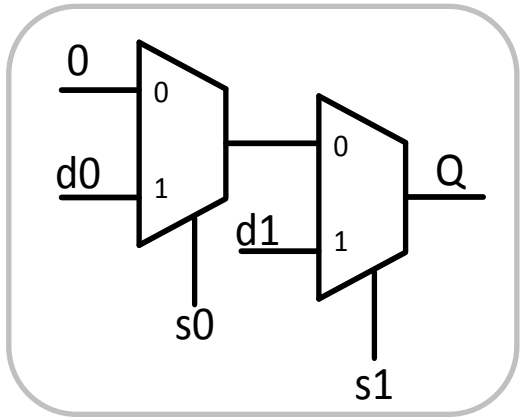
❖ Example:

```
always @ (s0,s1, d0, d1)
   begin
      Q = 0;
      if (s0)   Q = d0;
      else if (s1) Q = d1;
   end
```



Different

```
always @ (s0,s1, d0, d1)
   begin
      Q = 0;
      if (s1)   Q = d1;
      else if (s0) Q = d0;
   end
```



Non of the above infer latch, why?

Example: Up & Down Counters

4-Bit unsigned down-counter with synchronous set

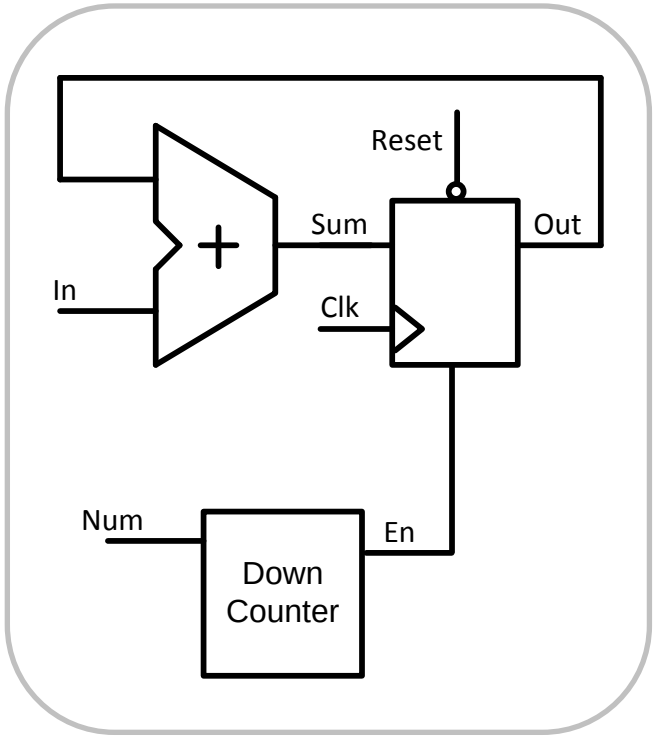
```
module D_counter (C, S, Q);

    input C, S;
    output [3:0] Q;
    reg [3:0] tmp;
    always @(posedge C)
        begin
          if (S)
            tmp <= 4'b1111;
          else
            tmp <= tmp - 1'b1;
        end
    assign Q = tmp;

endmodule
```

4-Bit up-counter with asynchronous reset and modulo maximum

```
module U_counter (C, CLR, Q);

    parameter
      MAX_SQRT = 4,
      MAX = (MAX_SQRT*MAX_SQRT);
    input C, CLR;
    output [MAX_SQRT-1:0] Q;
    reg [MAX_SQRT-1:0] cnt;
    always @ (posedge C or posedge CLR)
        begin
          if (CLR)
             cnt <= 0;
          else
             cnt <= (cnt + 1) %MAX;
          end
     assign Q = cnt;

endmodule
```

Accumulator

- Accumulates multiple successive k-bit values and stores them into a k-bit register
- The number of successive numbers (Num) as an input




```
module Accumulator (In, Num, Clk, Rst, Out);
   parameter k = 8;
   parameter m = 4;
   input  [k-1:0] In;
   input [m-1:0] Num;
   input Clk, Rst;
   output  reg  [k-1:0] Out;
   wire [k-1:0] Sum;
   reg  [m-1:0] C;
   wire En, Cout;
   defparam stage0.n = k;
   RippleCarryAdderI  stage0   (.Cin(0), .X(In), .Y(Out), .S(Sum), .Cout(Cout));
   always@ (posedge Clk, negedge Rst)
         if (Rst == 0)
           begin
             C <= Num;
             Out <= {k{1'b0}};
           end
          else if (En)
           begin
             C  <= C-1;
             Out <= Sum;
           end
assign En = |C;
endmodule
```

Outline

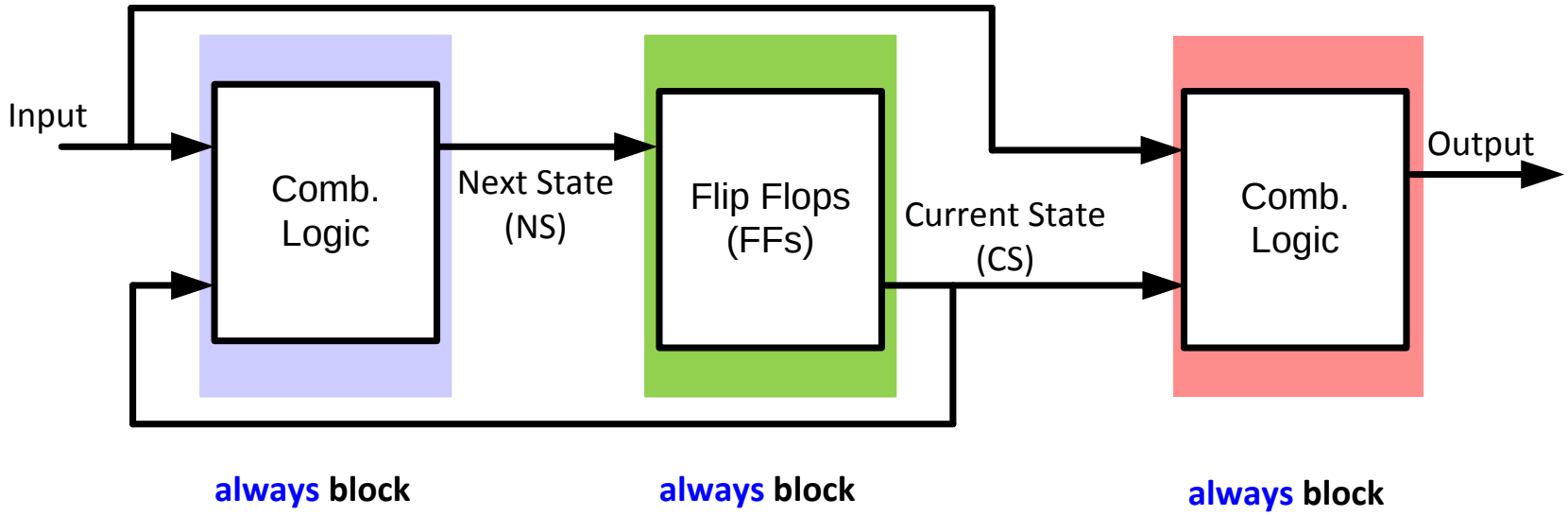
- ASIC/FPGA Design Flow
- Hardware Description Language (HDL)
  - Verilog
    - Introduction
    - Language Fundamentals
    - Modeling Combinational & Sequential Logic Circuits
    - Modeling Finite State Machines
    - Verilog Operations

Finite State Machine (FSM)

- Used to implement control sequencing
- An FSM is defined by
  - set of inputs
  - set of outputs
  - set of states
  - initial state
  - transition function
  - output function
- States are steps in a sequence of transitions
- There are “Finite” number of states.

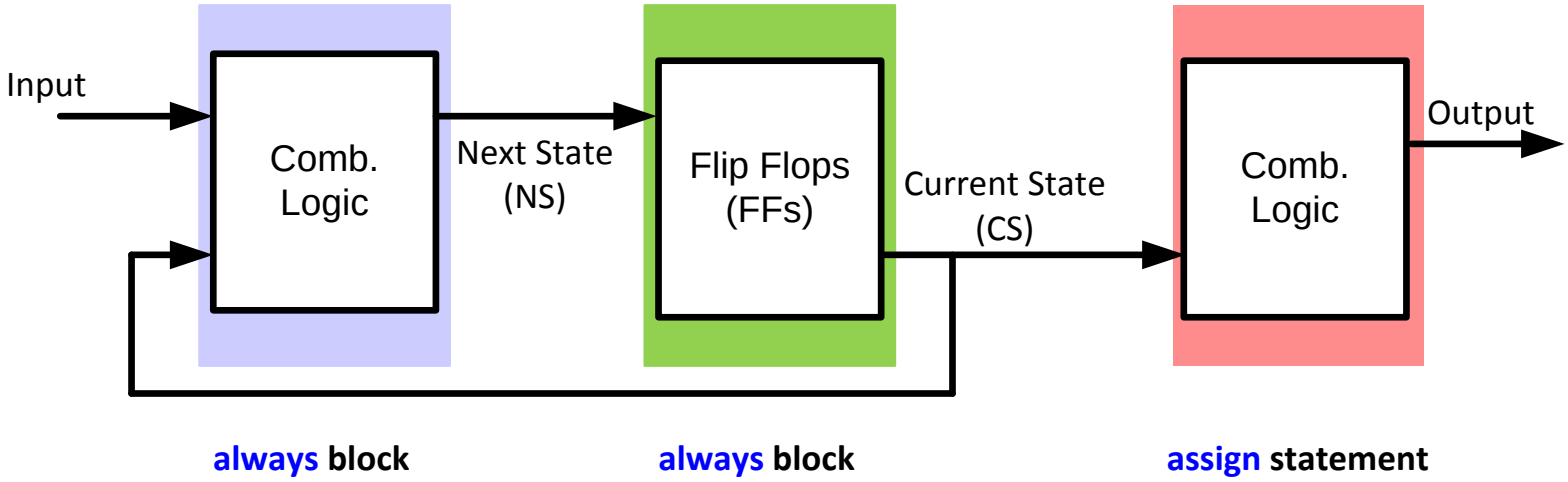
Finite State Machine (FSM)

- The behavior of the circuit can be represented using a finite number of states
- Two types:
  - **Mealy:**
    - Output depends on the "current state" and the "input"



Finite State Machine (FSM)

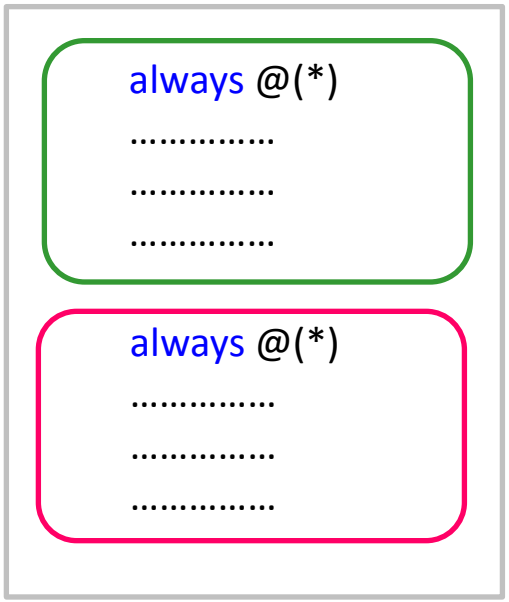
- **Moore:**
  - Output depends only on the "current state"



- Therefore, to describe an FSM in Verilog we have to show how to derive:
  - Next State (NC)
  - Current State (CS)
  - Output

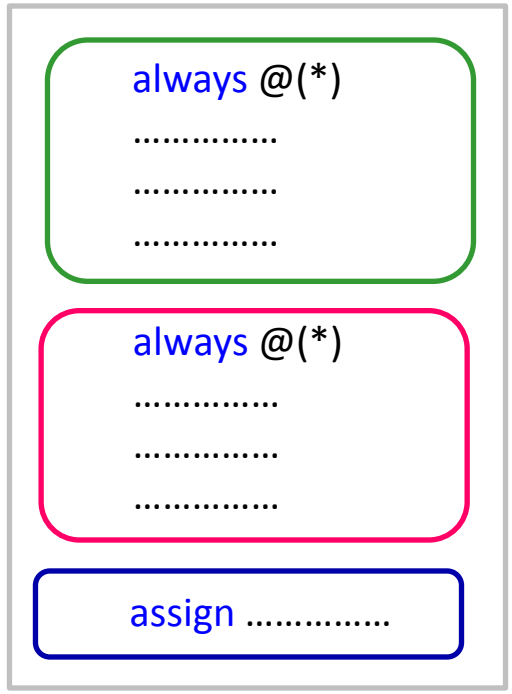
FSM Code Structure

Mealy



- Output depends on input
- Output declared as reg

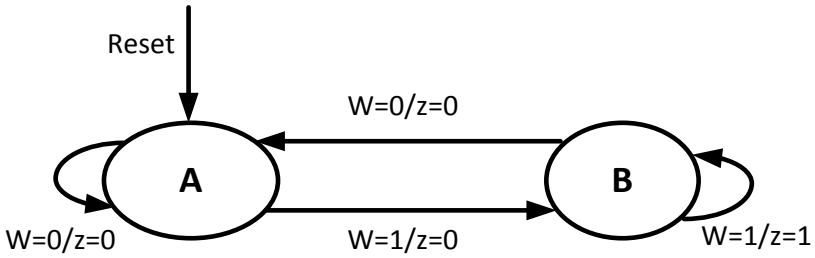
Moore



- Output does not depend on input
- Output declared as wire

FSM

❖ Example: Mealy Machine



Output: reg

```
module mealy (Clock, w, Resetn, z);
 input Clock, w, Resetn ;
 output reg z ;
 reg CS, NS;
 parameter A = 1'b0, B = 1'b1;
 always @(w, CS)
   case (CS)
     A:  if (w == 0)
           begin
             NS = A;   z = 0;
           end
         else
           begin
             NS = B;    z = 0;
           end
     B:  if (w == 0)
           begin
             NS = A;    z = 0;
           end
         else
           begin
             NS = B;    z = 1;
           end
   endcase
 always @(posedge Clock, negedge Resetn)
   if (Resetn == 0)
     CS <= A;
   else
     CS <= NS;
endmodule
```

NS & Output Calculation

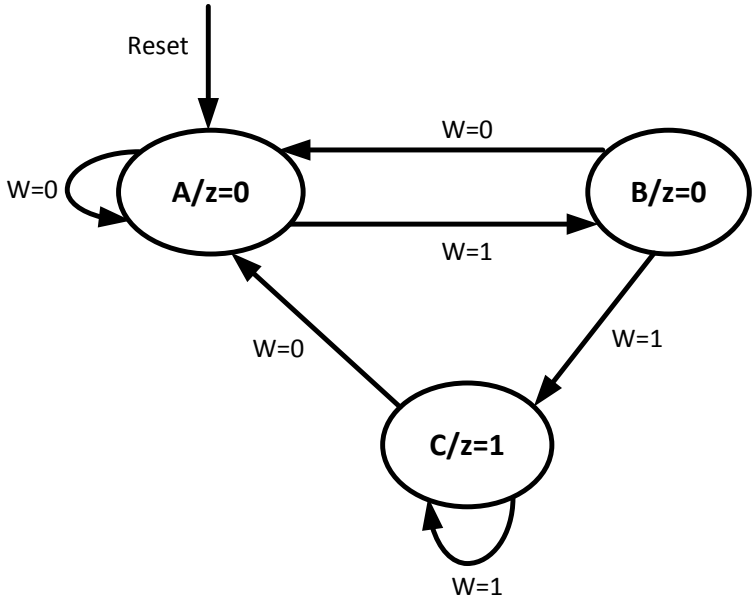
Combinational (Blocking)

CS Calculation

Sequential (Non-Blocking)

FSM

❖ Example: Moore Machine



Output: wire

```
module moore (Clk, w, Resetn, z);
 input Clk, w, Resetn;
 output z;
 reg [1:0] CS, NS;
 parameter A = 2'b00, B = 2'b01, C = 2'b10;

 always @(w, CS)
 begin
  case (CS)
   A:  if (w == 0)  NS = A;
     else  NS = B;
   B:  if (w == 0)  NS = A;
     else  NS = C;
   C:  if (w == 0)  NS = A;
     else  NS = C;
   default:   NS = 2'bxx;
  endcase
 end
 always @(posedge Clk, negedge Resetn)
 begin
  if (Resetn == 0)
   CS <= A;
  else
   CS <= NS;
 end

 assign z = (CS == C);
endmodule
```

NS Calculation

CS Calculation

Output Calculation

Combinational (Blocking)

Sequential (Non-Blocking)

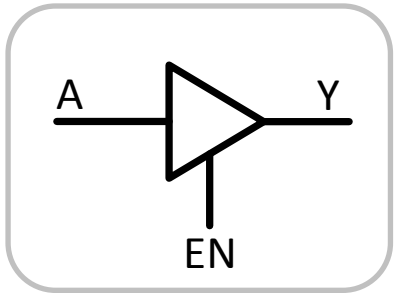
Outline

- ASIC/FPGA Design Flow
- Hardware Description Language (HDL)
  - Verilog
    - o Introduction
    - o Language Fundamentals
    - o Modeling Combinational & Sequential Logic Circuits
    - o Modeling Finite State Machines
    - o Verilog Operations

Tri-State Logic in Verilog

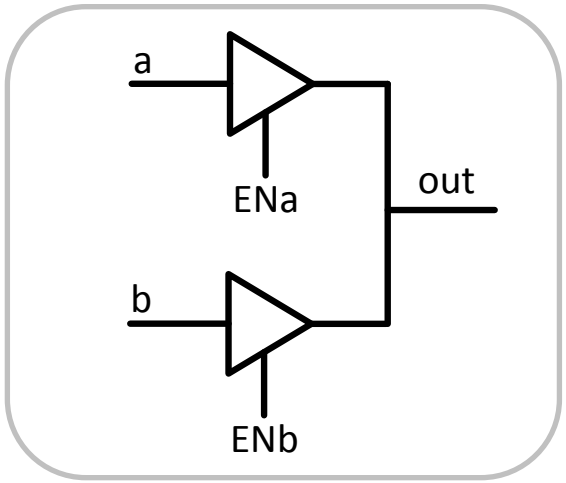
- Tri-state buffer:

$$Y = \begin{cases} A & EN = 1 \\ Z & EN = 0 \end{cases}$$




```
module tri-buffer (A, y, EN)
   input A, EN;
   output Y;

   assign Y = (EN) ? A : 1'bZ;

endmodule
```

- Tri-state driver inference:

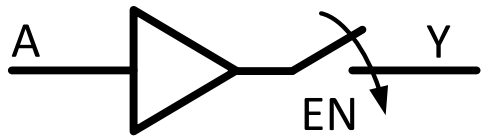



```
always @ (ENa, a)
   begin
    if (ENa)
      out = a;
    else
      out = 1'bz;
   end
always @ (ENb, b)
   begin
    if (ENb)
      out = b;
    else
      out = 1'bz;
   end
```

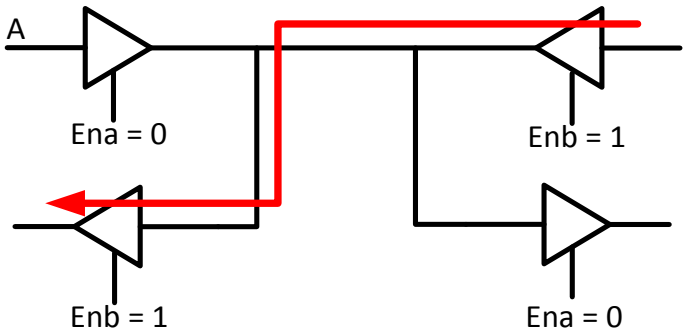
```
assign out = (ENa) ? a : 1'bz;
assign out = (ENb) ? b : 1'bz;
```

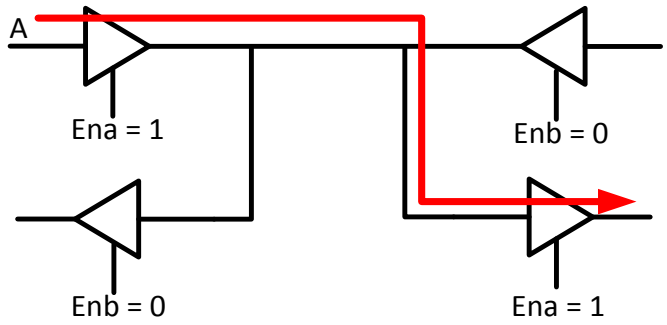
Tri-State Applications

1. **Buffering:**

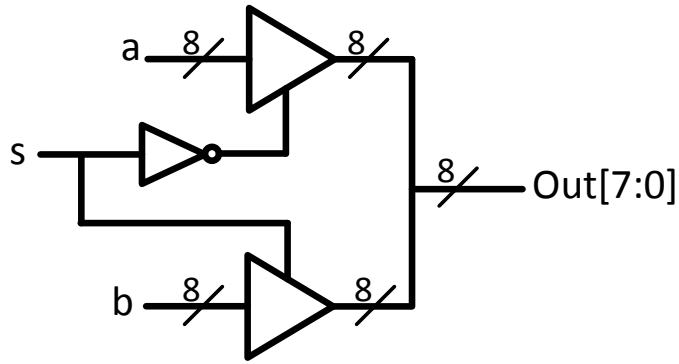


2. **Half-duplex communication:**



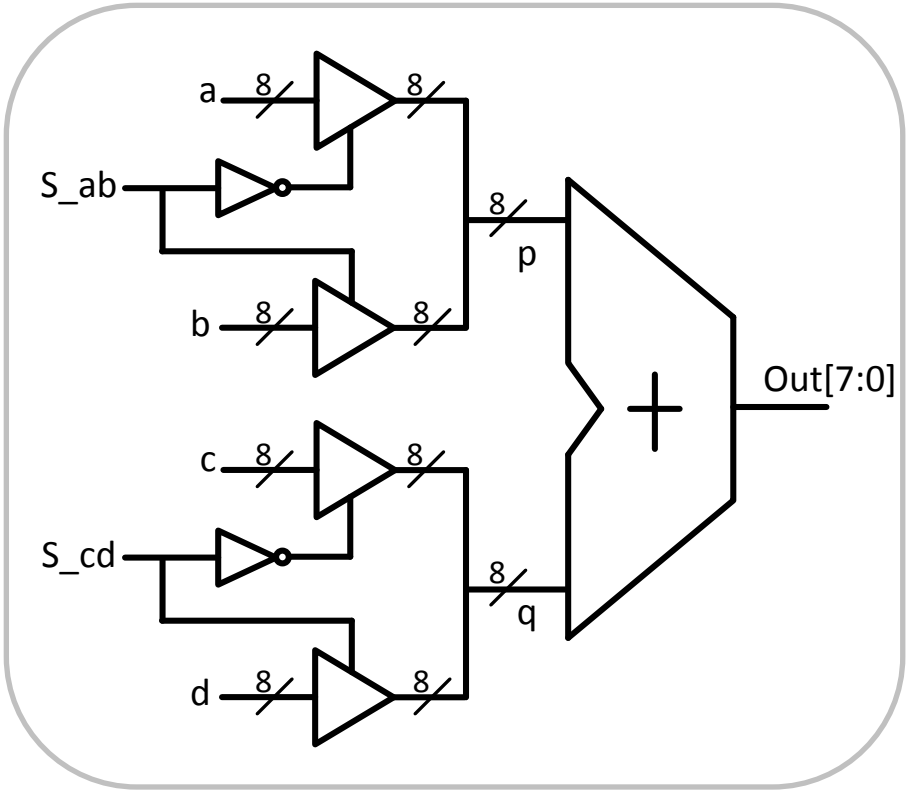


3. **Bus multiplexing:**



Tri-State Applications

❖ Example: Adder with four options




```
module tri-adder (a, b, c, d, S_ab, S_cd, Out);

  input S_ab, S_cd;
  input [7:0] a, b, c, d;
  output [8:0] Out;
  wire [7:0] p, q;

  assign p = ~S_ab ? a : 8'bzzzzzzzz;
  assign p =  S_ab ? b : 8'bzzzzzzzz;
  assign q = ~S_cd ? c : 8'bzzzzzzzz;
  assign q =  S_cd ? d : 8'bzzzzzzzz;

  assign Out = p + q;

endmodule
```

Z is an allowed logic value and implies a tri-state driver for synthesis

Verilog Operations: Right/Left Shift

- Verilog supports << for left and >> for right shift. (Only one position)
- Both of these operators use a zero for the shift input bit.
- We can also control the shift input

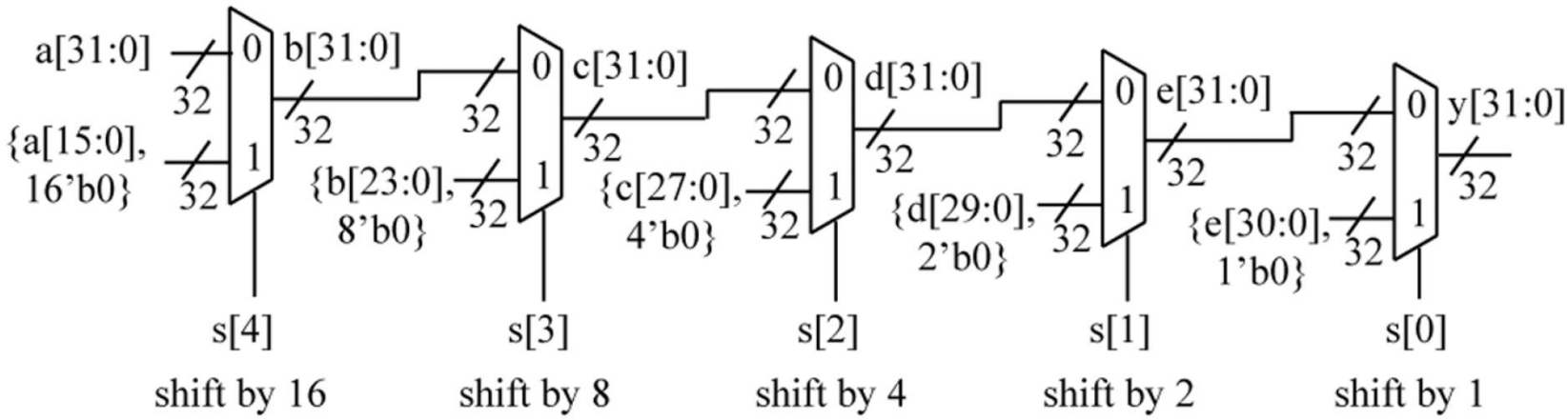
```
module LRShift (Si, L, R, In, Out);

 input Si, L, R;
 input [7:0] In;
 output [7:0] Out;

 always @ (L, R, In, Si)
       begin
             case({R,L})
                2'b01  :  Out = {In[6:0], Si};   // Left shift
                2'b10  :  Out = {Si, In[7:1]};   // Right shift
                default:  Out = In;
             endcase
      end
endmodule
```

Verilog Operations: Barrel Shifter

- **Barrel shifter** shifts a signal by multiple positions

```
module BarrelShifter (s, a, y);
 input [4:0] s;
 input [31:0] a;
 output [31:0] y;
 assign y = a<<s;
endmodule
```

- **Example:**
  - 32-bit left shift barrel shifter
  - Left shifts by 0 to 31 positions based on the 5-bit s input
  - Each of its stages corresponds to a fixed shift by a power of 2 (16, 8, 4, 2, 1)
  - Simple HDL implementation, which illustrates the power of HDL to hide implementation details from a designer

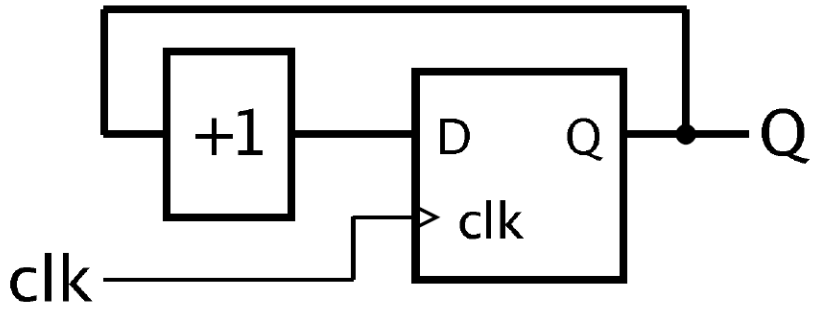


Counters

- Stores an unsigned integer value
  - Increments or decrements the value
- Used to count occurrences of
  - Events
  - Repetitions of a processing step
- Used as timers
  - Count elapsed time intervals by incrementing periodically

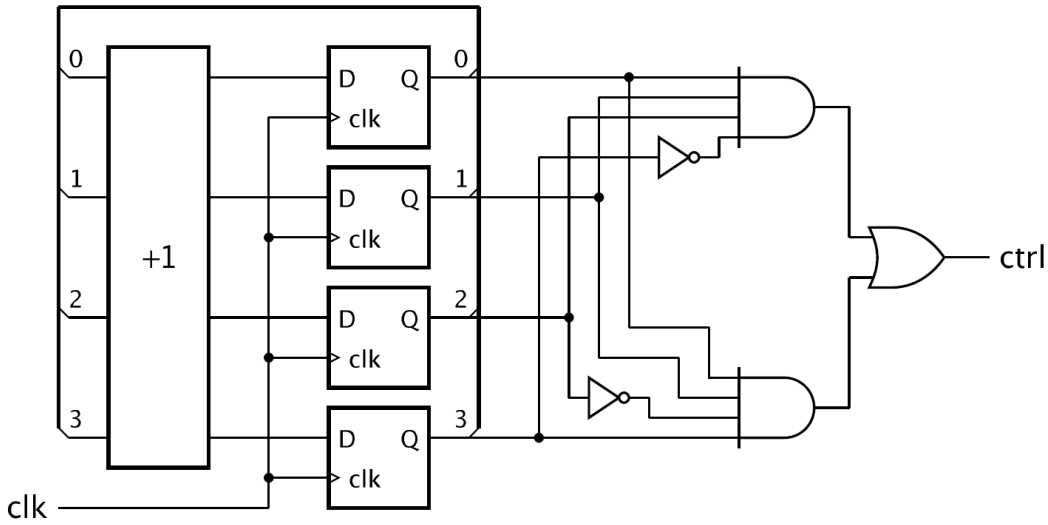
Free-running Counter:

- Increments every rising edge of clock
  - Up to 2^n–1, then wraps back to 0
  - Counts modulo 2^n
- This counter is synchronous
- All outputs governed by clock edge



Example: Periodic Control Signal

- Count modulo 16 clock cycles
- Control output = 1 every 8th and 12th cycle
- Decode count values 0111 and 1011




```
module decoded_counter ( output ctrl,
input clk );
reg [3:0] count_value;
always@(posedge clk)
        count_value <= count_value + 1;
        assign ctrl = count_value == 4'b0111
        || count_value == 4'b1011;
endmodule
```

Fixed-point vs. Floating-point

- Fixed-point means allocating a fixed number of bits with a fixed pointer position to represent numbers.
  - Simpler for implementation
  - Less accuracy

- Floating-point representation is provide a much more extensive means for providing real number representations and tend to be used extensively in scientific computation applications.
  - More flexible/accuracy
  - More complexity on implementation side (some times 10 times larger hardware than fixed-point counterpart!)

Verilog Operations: Fixed-Point Simulation

- For realization of DSP algorithms all variables should be converted to the fixed-point representation

- Normally 2's complement representation is used to represent signed numbers

- A fixed-point 2's complement representation of a number has two parts:
  - Integer part (W_I bits)
  - Fractional part (W_F bits)

- The length of W_I and W_F are calculated based on the dynamic range of variables
  - Total length: $W_I + W_F$

Verilog Operations: Fixed-Point Simulation

- Typical word lengths:

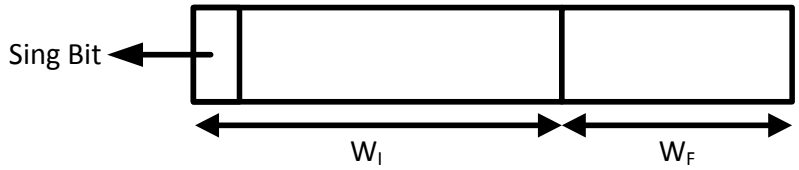
Application	Word sizes (bits)
Control systems	4–10
Speech	8–13
Audio	16–24
Video	8–10

- Fixed word-length dynamic range:

Wordlength (bits)	Wordlength range	Dynamic range dB
8	−127 to +127	$20 \log 2^8 \simeq 48$
16	−32768 to +32767	$20 \log 2^{16} \simeq 96$
24	−8388608 to +8388607	$20 \log 2^{24} \simeq 154$

Verilog Operations: Fixed-Point Simulation

- 2's complement Representation: (W_I , W_F) format



$$\text{Sign Bit} = \begin{cases} 0: & \text{positive} \\ 1: & \text{negative} \end{cases}$$

- Good to represent quantized numbers in the range: $\left[2^{-W_I}, \left(2^{W_I-1} - \left(\frac{1}{2}\right)^{W_F}\right)\right]$
  - Resolution : $\left(\frac{1}{2}\right)^{W_F}$

❖ Example:

- in (3,3) 011101 represents 3.625 (smallest number: 0.125)
- in (3,5) 10111000 represents -2.25 (smallest number: 0.03125)

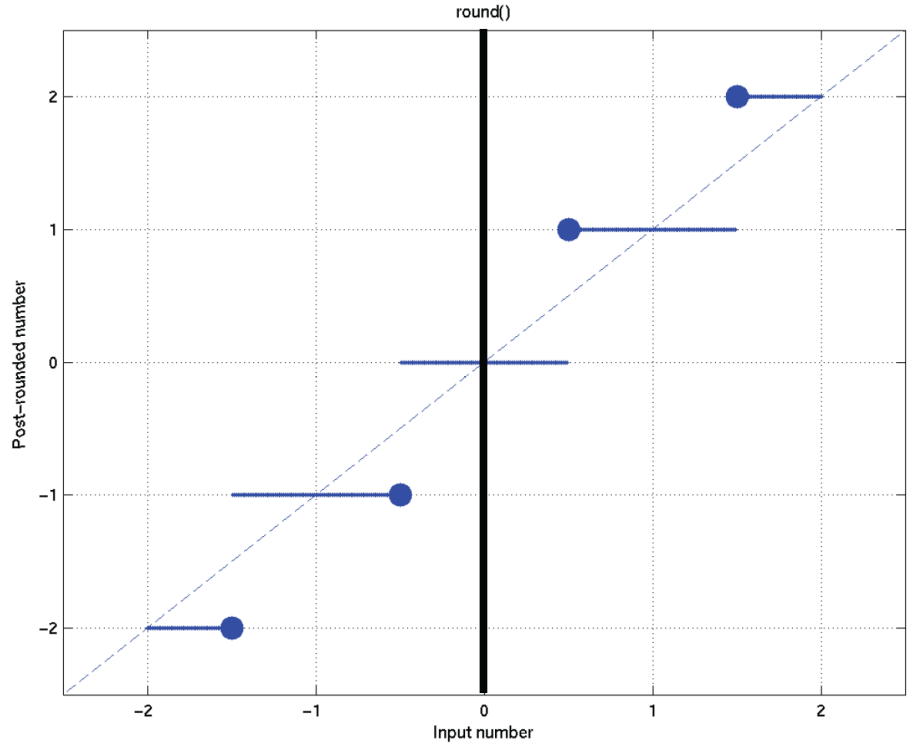
Fixed-Point Simulation: Rounding

- Eliminates LSB bits
- Need to reduce the number of bits due to word growth
  - For example, if we multiply two 5-bit words, the product will have 10 bits, i.e., xxxxx × yyyyy = zzzzzzzzzz and we likely don't want or need all that precision

- Matlab rounding:
  - **round**(·): towards nearest integer
    - Pos. and neg. numbers are rounded symmetrically about zero
    - Generally the best possible rounding algorithm
  - **fix**(·): truncates towards zero
    - Pos. and neg. numbers are rounded symmetrically about zero
  - **floor**(·): rounds towards negative infinity
  - **ceil**(·): rounds towards positive infinity

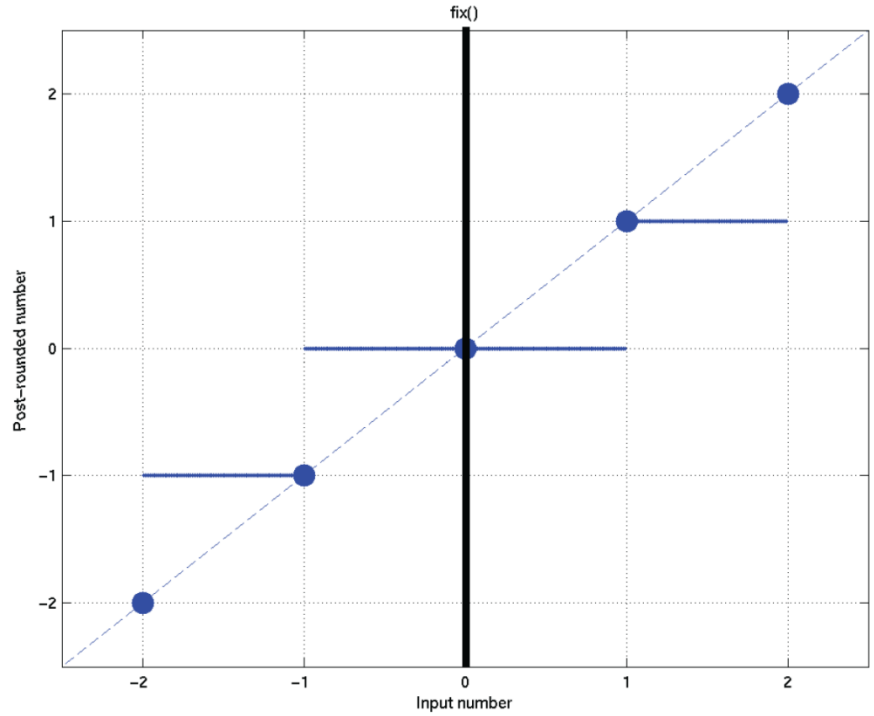
Fixed-Point Simulation: Matlab round(.)

- One of the best rounding modes
- "Unbiased" rounding
- Symmetric rounding for positive and negative numbers
- Max error ½ LSB



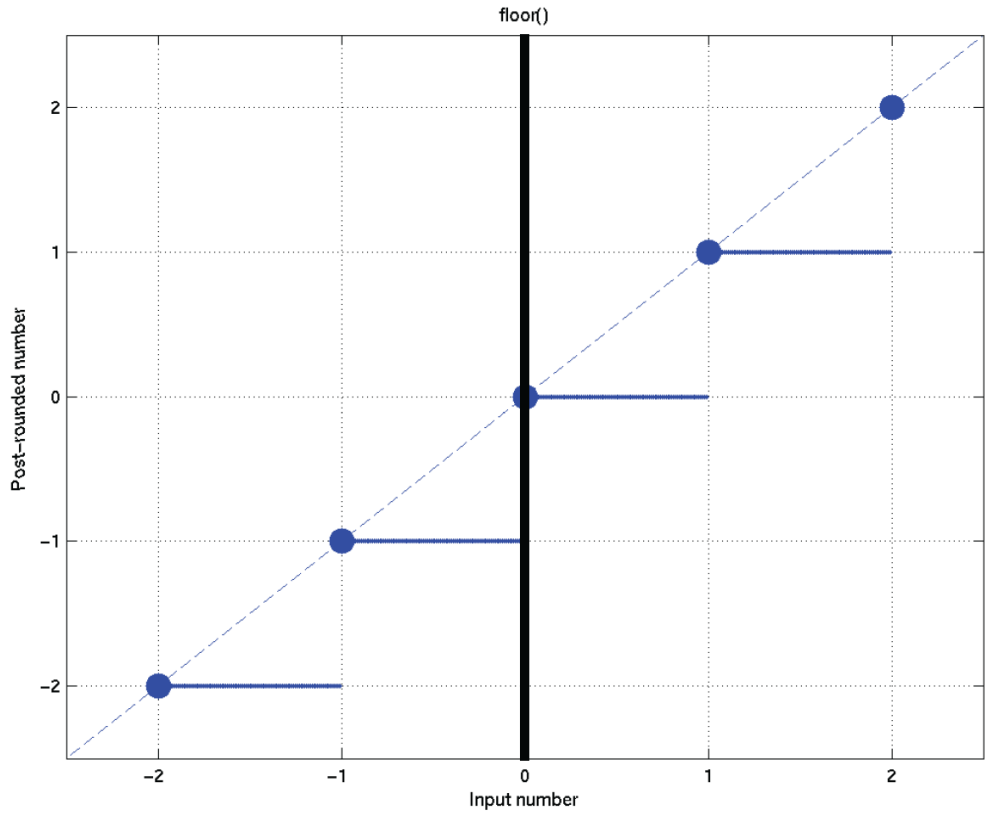
Fixed-Point Simulation: Matlab fix(.)

- Truncates toward zero
- Numerical performance poor
- Symmetric rounding for positive and negative numbers
- Max error 1 LSB



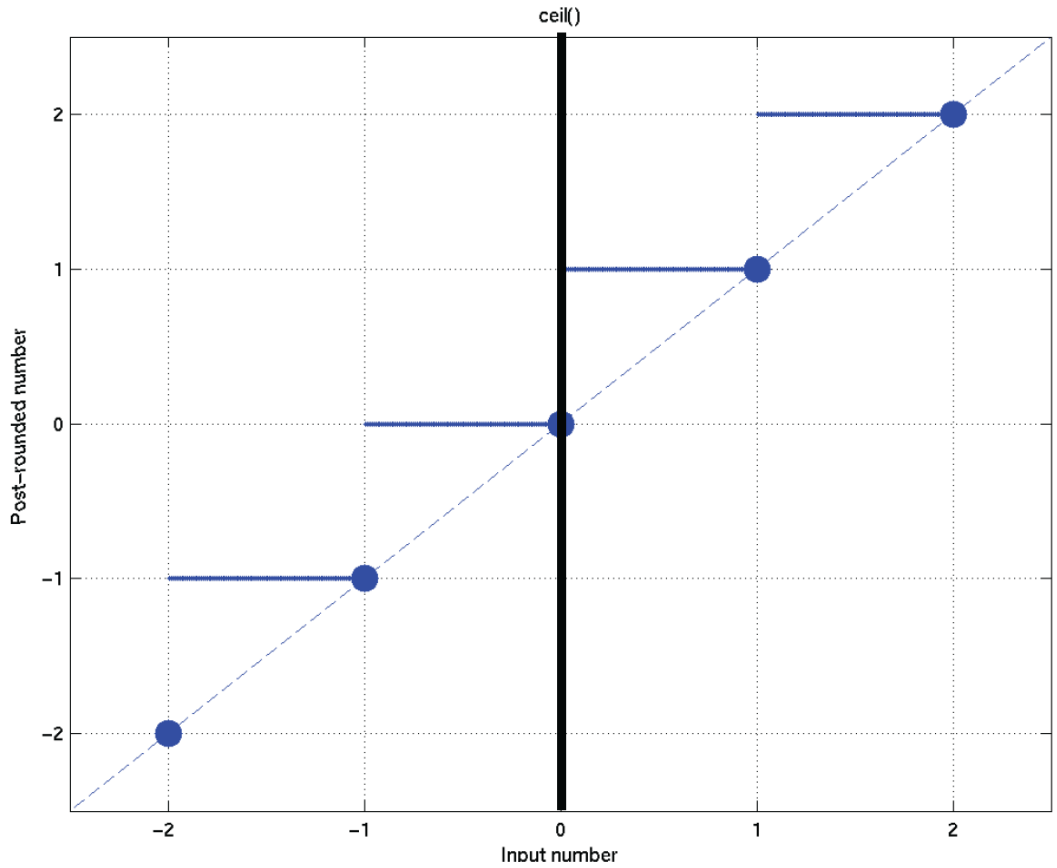
Fixed-Point Simulation: Matlab floor(.) or truncation

- Numbers rounded down towards –∞ (-infinity)
- Numerical performance poor
- Very simple hardware
- In:xxxxxx -> Out: xxxx--
- Max error 1 LSB



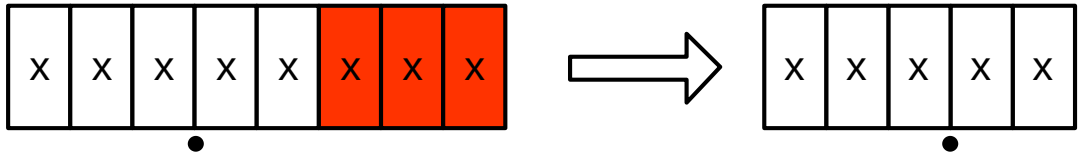
Fixed-Point Simulation: Matlab ceil(.)

- Numbers rounded up towards + ∞ (+infinity)
- Numerical performance poor
- Max error 1 LSB



Hardware Rounding

- Easiest is truncation



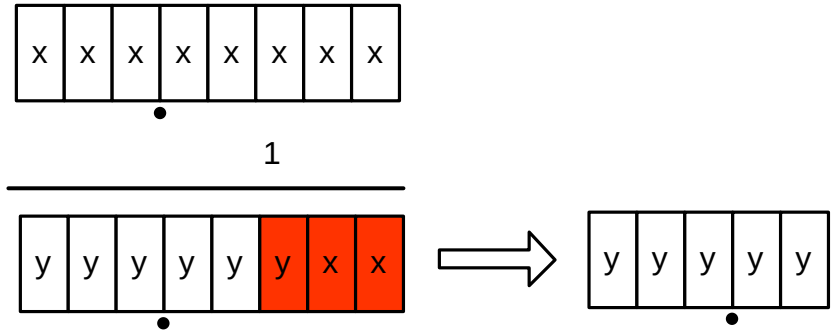
- Maximum rounding error ~1 post-rounded LSB

- Signed magnitude
  - Positive and negative numbers both truncate towards zero
  - Matlab fix(·)

- 2's complement and unsigned
  - All numbers truncate towards negative infinity
  - Matlab floor(·)

Hardware Rounding

- Better rounding numerically is to add ½ lSB and then truncate



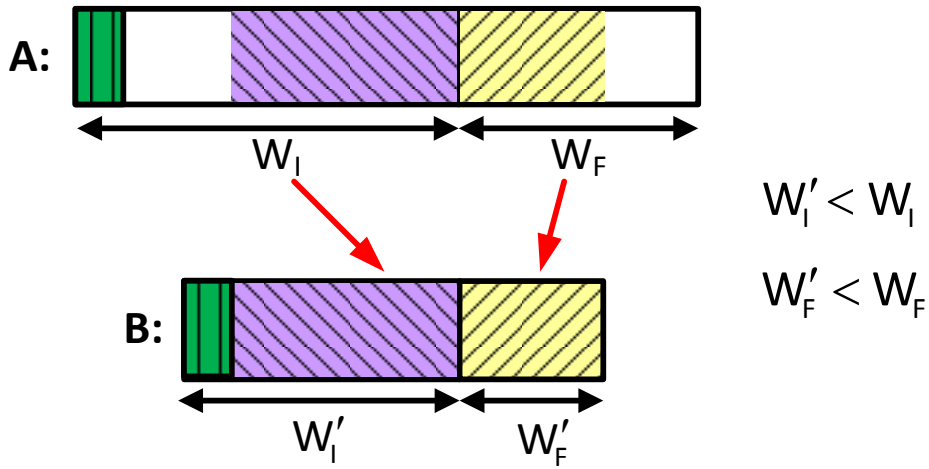
- Maximum rounding error ½ post-rounded LSB
- Two cases:

  a. When the input is xxxx.5000 (base 10) (or xxx.xx100 (base 2) in the example above)
  - Rounding is towards +∞ (for both positive and negative numbers)
  - matlab ceil(·)

  b. Otherwise
  - Performs best rounding: matlab round(·)

Fixed-Point Modeling: Casting

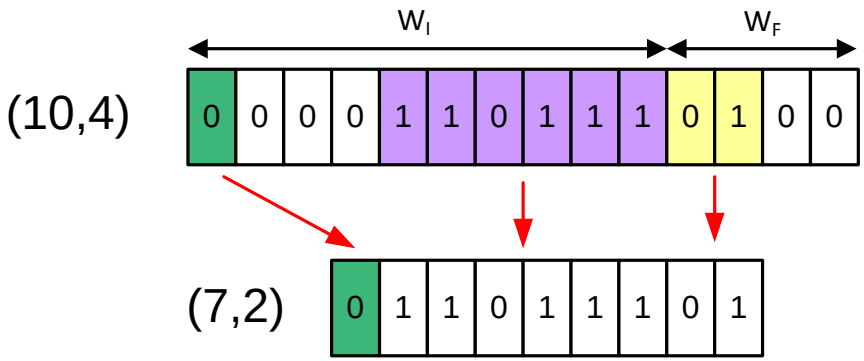
- Care must be taken when dealing with fixed-point numbers
- **Casting:** To convert a number with a larger bit length to a smaller one



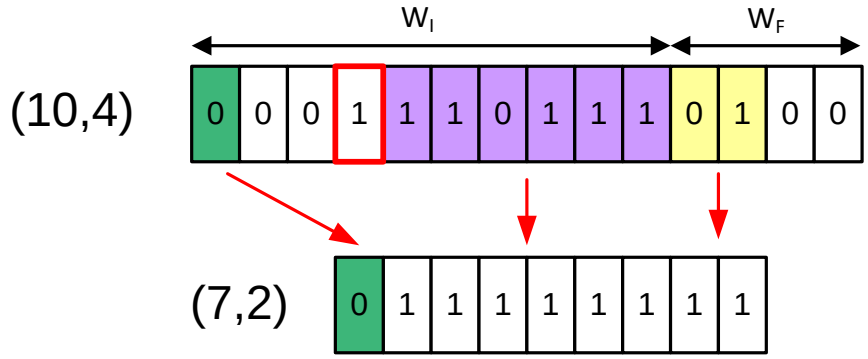
- $B = \{A[W_I + W_F - 1], A[W_I' + W_F - 2 : W_F - W_F']\}$
- Saturation happens if:
  - "A" is positive and $A[W_I + W_F - 2 : W_I' + W_F - 1] \neq 0$
  - "A" is negative and $A[W_I + W_F - 2 : W_I' + W_F - 1] \neq 111\ldots111 (\text{all one})$

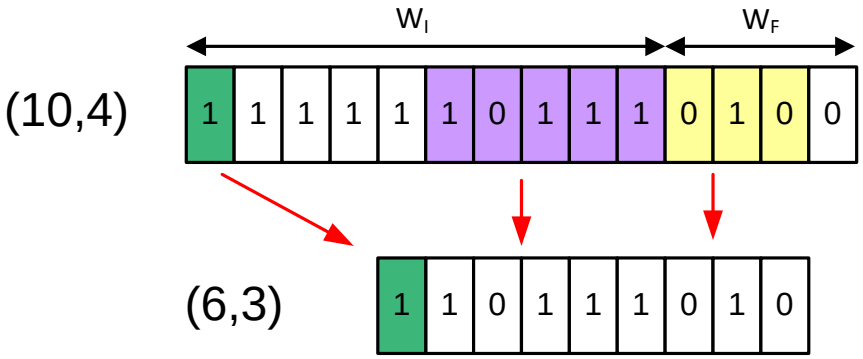
Fixed-Point Modeling: Casting

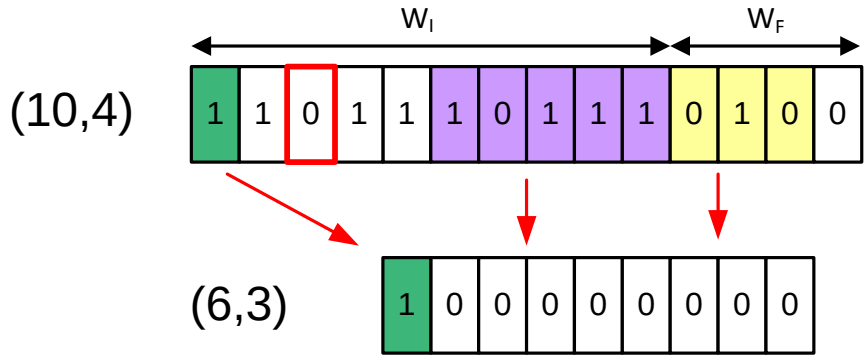
❖ Example:



Saturation

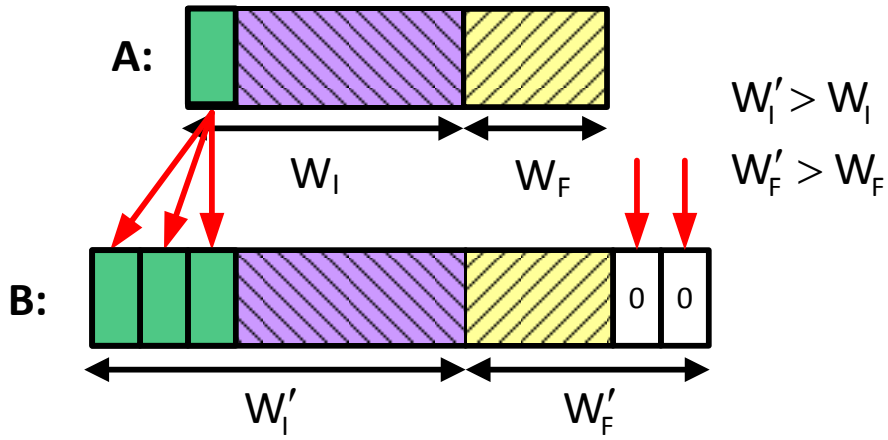






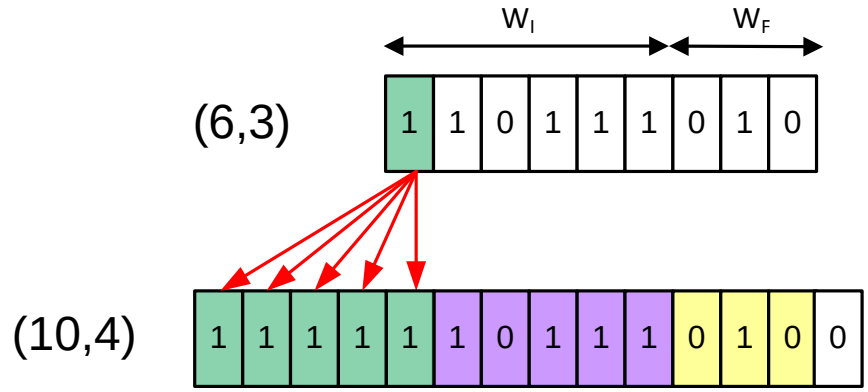
Fixed-Point Modeling: Sign Extension

- To convert a number with a smaller bit length to a larger one sign extension is required.



$W_I' > W_I$

$W_F' > W_F$



- assign $B = \{\{n\{A[W_I + W_F - 1]\}\}, A, 2'b0\}$
- Examples: Adding two numbers with different lengths:

```
wire [2:0] A;
wire [5:0] B;
wire [6:0] C;
assign C = {B[5],B} + {{4{A[2]}},A};
```

Verilog Operations: Addition with Same Length

- Adding two signed n-bit numbers and save it in a signed n-bit number:
  - Might not be safe if two number are large
  - Overflow condition should be checked

$$\begin{array}{rr} & A[n-1:0] \\ + & B[n-1:0] \\ \hline & C[n-1:0] \end{array}$$

- Overflow may happen if:
  - A[n-1]==1 and B[n-1]==1 and C[n-1]==0
  - A[n-1]==0 and B[n-1]==0 and C[n-1]==1

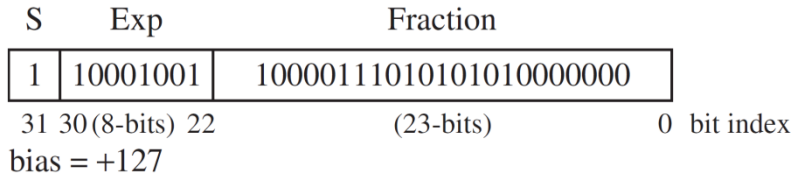
$$\begin{array}{rr} & 0110 \\ + & 0111 \\ \hline & 1101 \end{array} \qquad \begin{array}{rr} & 1010 \\ + & 1001 \\ \hline & 10011 \end{array}$$

```
assign SUM = B + A;
assign OV = (A[n-1]==1 && B[n-1]==1 && C[n-1]==0)||
            (A[n-1]==0 && B[n-1]==0 && C[n-1]==1);

assign C = (OV && A[n-1] == 1) ? MIN_NEG_n : SUM;
assign C = (OV && A[n-1] == 0) ? MAX_POS_n : SUM;
```

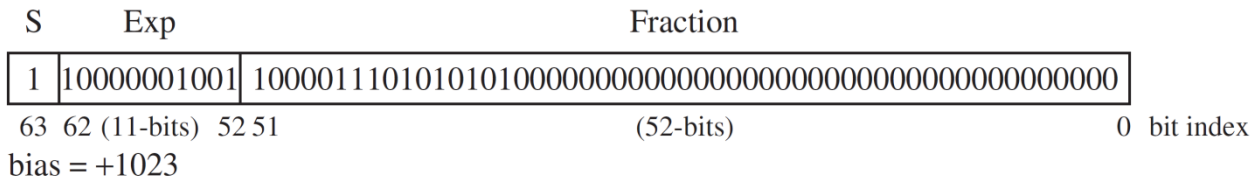
Floating-point

- In floating-point, the aim is to represent the real number using a *sign (S)*, *exponent (Exp)* and *mantissa* (or fraction).

$$N = 2^{Exp-127} \times M$$



(a) Single precision



(b) Double precision

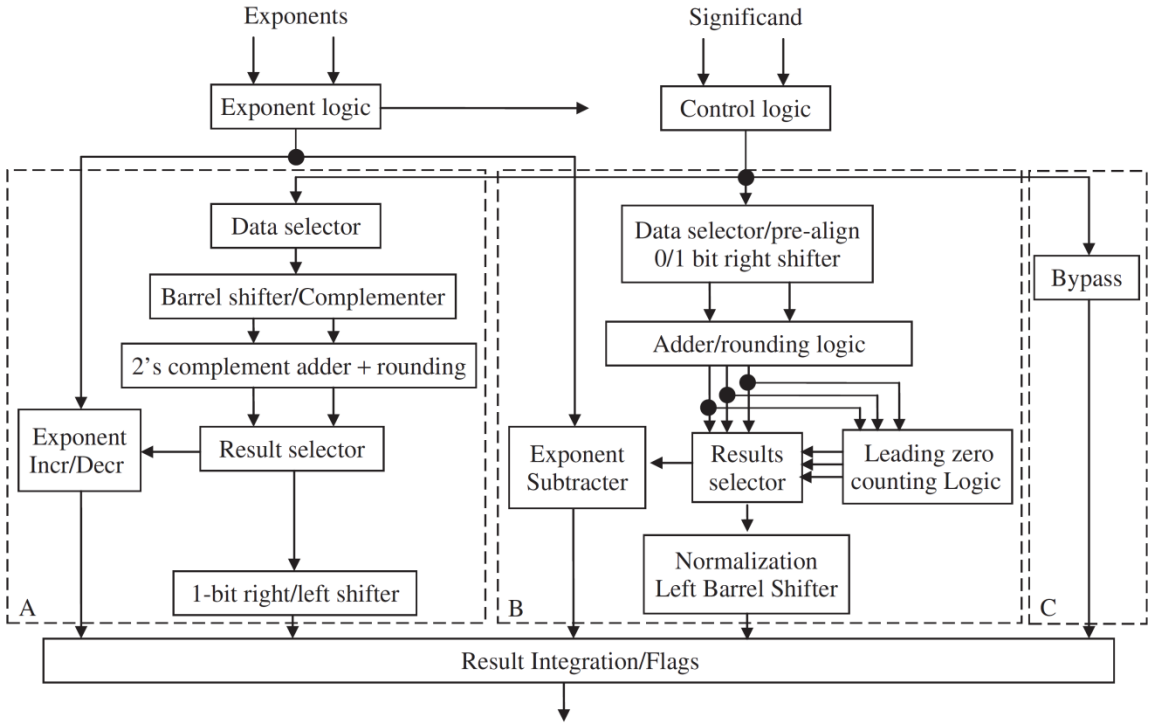
- The most widely used form of floating-point is IEEE Standard for Binary Floating-Point Arithmetic (IEEE 754) with two major formats:
  - Single-precision (32-bit)
  - Double-precision (64-bit)

Floating-point: Example

- Converting a real number, −1082.5674 IEEE 754 floating-point representation:
  - It can be determined that $S = 1$ as the number is negative.
  - The number (1082) is converted to binary by successive division, 10000111010.
  - The fractional part (0.65625) is computed in the same way as above, giving 10101.
  - The parts are combined to give the value 10000111010.10101.
  - The radix point is moved left, to leave a single 1 on the left, $1.00001110101010 1 \times 2^{10}$.
  - Filling with 0s to get the 23-bit mantissa gives the value 00000111010101010000000.
  - The exponent is 10 and with the 32-bit IEEE 754 format bias of 127, giving 137 which is given as 10001001 in binary.

Floating-point Implementation

- The floating-point implementation concurs a complicated hardware compared to the fixed-point counterpart.
- Take into account as an example a floating-point adder!
  - This additional logic is needed to perform the various normalization steps for the adder



Fixed-point vs. Floating-point

- The area comparison for floating-point is additionally complicated as the relationship between multiplier and adder area is now changed.
- In fixed-point, multipliers are generally viewed to be *N* times bigger than adders where *N* is the word length.
- However, in floating-point, the area of floating-point adders is comparable to that of floating-point multipliers which corrupts the assumption at the algorithmic stages to reduce number of multiplications in favor of additions.
- Table below gives some figures on area and speed figures for floating-point addition and multiplication implemented in a Xilinx Virtex 4 FPGA technology.

Function	DSP48	LUT	Flip-flops	Speed (MHz)
Multiplier	4	799	347	141.4
Adder	×	620	343	208.2
Reciprocal	4	745	266	116.5

Verilog Operations: $signed and $unsigned

- A = $signed(B)
  - Sign extends B and assigns it to A
  - bit width(B) < bit width (A)
  - Example

```
wire [5:0] A;
assign A = $signed (3b'110);
```

→ **A = 111110**

- A = $unsigned (B)
  - Zero fill B and assign it to A
  - bit width(B) < bit width (A)
  - Example

```
wire [5:0] A;
assign A = $unsigned (3b'110);
```

→ **A = 000110**

Verilog Operations: Signed Addition

- There are two ways to perform signed addition:
  1. Sing Extension:

```
wire [2:0] A, B;
wire [3:0] SUM;
assign SUM = {B[2],B} + {A[2],A};
```

Same result

```
 1110  (-2)
 0010  (+2)
-----
10000  (0)
```

Discard Overflow

  2. Using signed signals

```
wire signed [2:0] A, B;
wire signed [3:0] SUM;
assign SUM = B + A;
```

- Wrong otherwise:

```
wire [2:0] A, B;
wire [3:0] SUM;
assign SUM = B + A;
```

```
 110  (-2)
 010  (+2)
-----
1000  (-8)
```

(Wrong)

Verilog Operations: Signed Addition with Carry-in

- There are two correct ways to perform signed addition with carry-in:

1. Sing Extension:

```
wire [2:0] A, B;
wire Cin;
wire [3:0] SUM;
assign SUM = {B[2],B} + {A[2],A} + Cin;
```

Same result

```
 1110  (-2)
 0010  (+2)
 0001  Cin
-----
10001  (1)
```

Discard Overflow

2. Using signed signals

```
wire signed [2:0] A, B;
wire Cin;
wire signed [3:0] SUM;
assign SUM = B + A + $signed({1'b0},Cin);
```

Verilog Operations: Signed Addition with Carry-in

❑ **Incorrect** Codes:

```
wire signed [2:0] A, B;
wire Cin;
wire signed [3:0] SUM;
assign SUM = B + A + Cin;
```

```
  110  (-2)
  010  (+2)
    1  Cin
-------
 1001  (9)
```

If any operand of an operation is unsigned, the entire operation is performed unsigned

```
wire signed [2:0] A, B;
wire signed Cin;
wire signed [3:0] SUM;
assign SUM = B + A + Cin;
```

```
 1110  (-2)
 0010  (+2)
 1111  Cin
-------
 1111  (-1)
```

When Cin=1, it sign extends it, to match the size of A and B, which is incorrect!

```
wire signed [2:0] A, B;
wire Cin;
wire signed [3:0] SUM;
assign SUM = B + A + $signed(Cin);
```

```
 1110  (-2)
 0010  (+2)
 1111  Cin
-------
 1111  (-1)
```

When Cin=1, it sign extends it, to match the size of A and B, which is incorrect!

Verilog Operations: Signed Multiplication

- Use signed construct as we used for signed addition:
  1. Use Verilog constructs:

```
wire signed [16:0] A, B;
wire signed [31:0] MULT;
assign MULT = A*B;
```

  2. Write it manually as a module

Complicated!

Verilog Operations: Signed Multiplication

- Multiplication of a signed number and an unsigned number:
  - **Correct:**

```
wire signed [2:0] A;
wire [2:0] B;
wire signed [5:0] PROD;
assign PROD = A*$signed({1'b0,B});
```

```
  110   (-2)
  111   (7)
------
110010  (-14)
```

  - **Incorrect:**

```
wire signed [2:0] A;
wire [2:0] B;
wire signed [5:0] PROD;
assign PROD = A*$signed(B);
```

When B[2]==1, treats it as a negative number!

```
  110   (-2)
  111   (7) treat it as (-1)
------
000010  (+2)
```

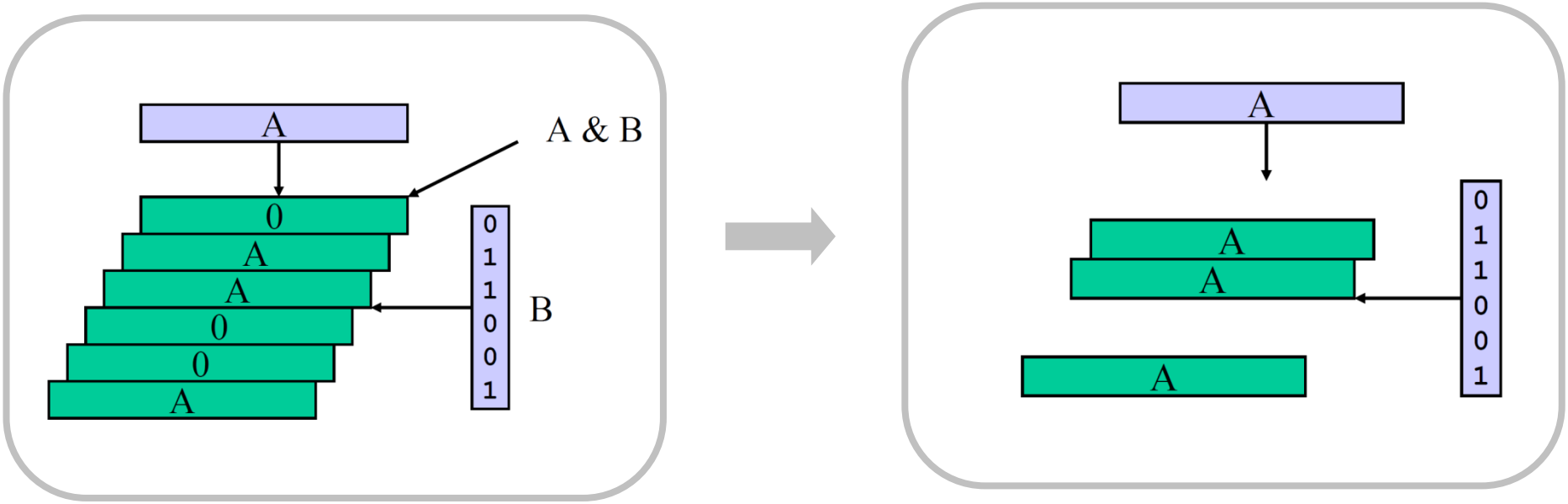
```
wire signed [2:0] A;
wire [2:0] B;
wire signed [5:0] PROD;
assign PROD = A*B;
```

Entire operation is performed unsigned

```
  110   (-2)
  111   (7)
------
101010  (42)
```

Verilog Operations: Fixed Multiplication

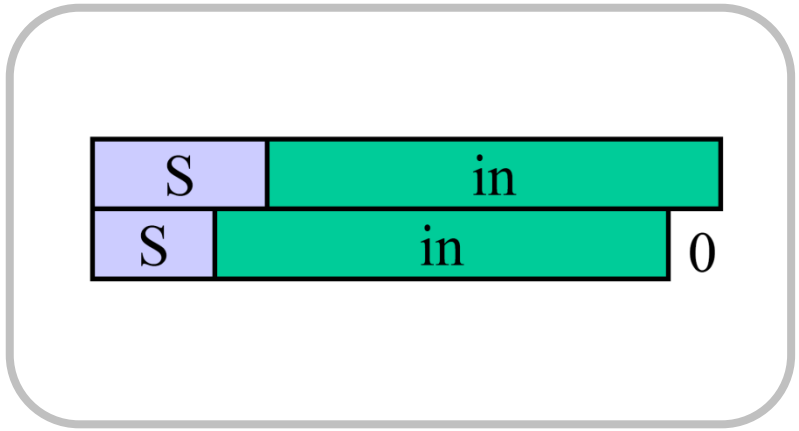
- Sometimes one input is fixed so remove partial products that are always zero
- We have to try to find the minimum number of power-of-2 numbers to add together to equal the fixed multiplier input



Verilog Operations: Fixed Multiplication

- Example: Multiply by 3:

```
input [7:0] in;
wire [9:0] product;
assign product = {in[7], in, 1'b0}
+ {in[7], in[7], in};
```



- Example: Multiply by 56:

56=32+16+8

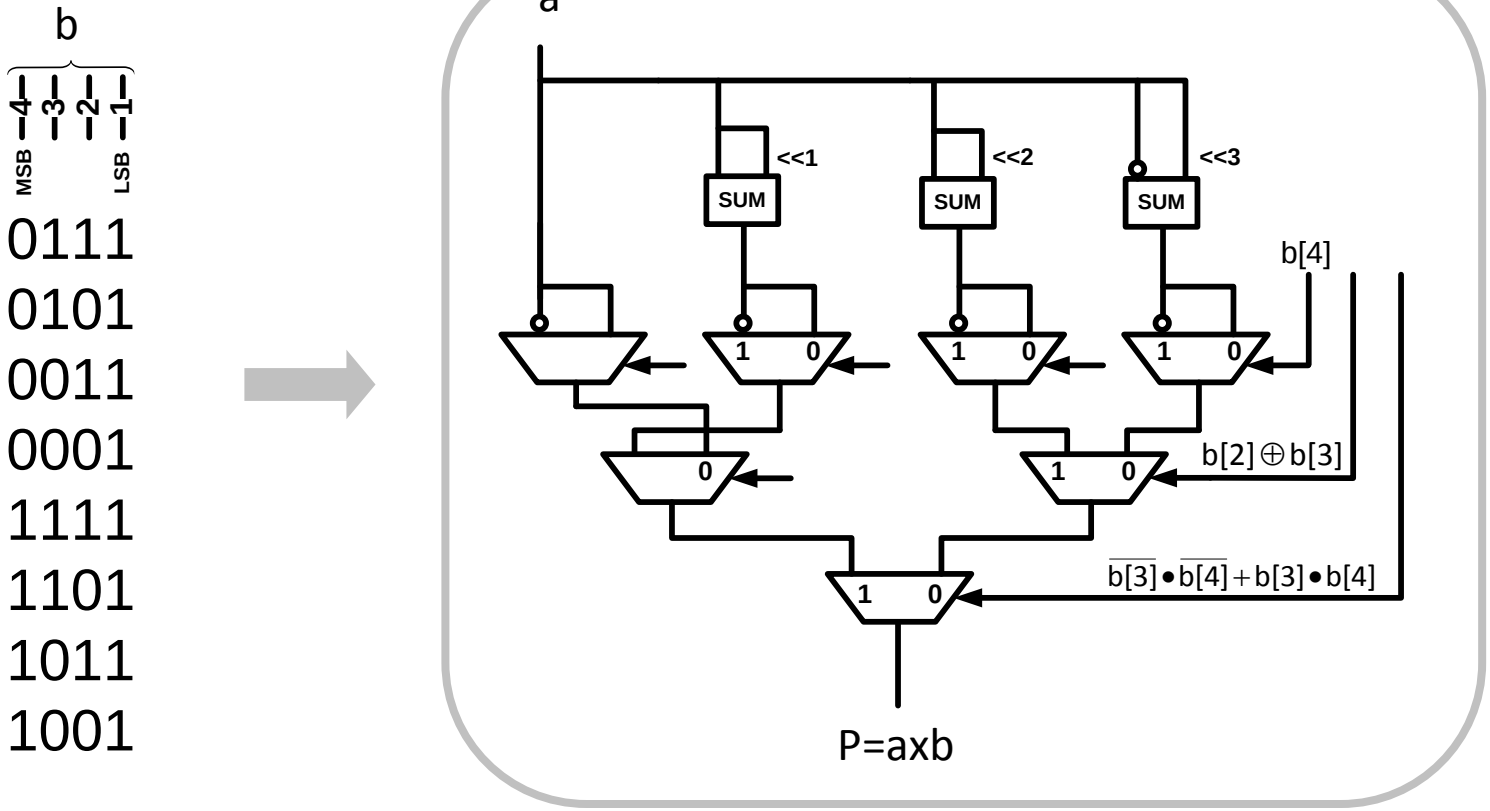
```
input [7:0] in;
wire [13:0] product;
assign product =
{in[7], in, 5'b00000}
+ {in[7], in[7], in, 4'b0000}
+ {in[7], in[7], in[7], in, 3'b000};
```

56=64-8

```
input [7:0] in;
wire [13:0] product;
assign product =
{in, 6'b00000}
- {in[7], in[7], in[7], in, 3'b000};
```

Verilog Operations: Constant Multiplication

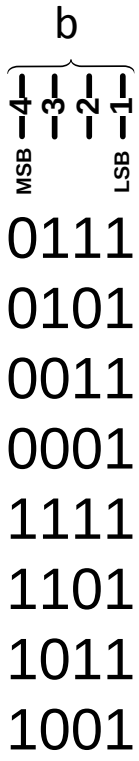
- Multiplication with a set of constant numbers may be implemented more efficiently: $P = a \times b \qquad b \in \{-7,-5,-3,-1,1,3,5,7\}$

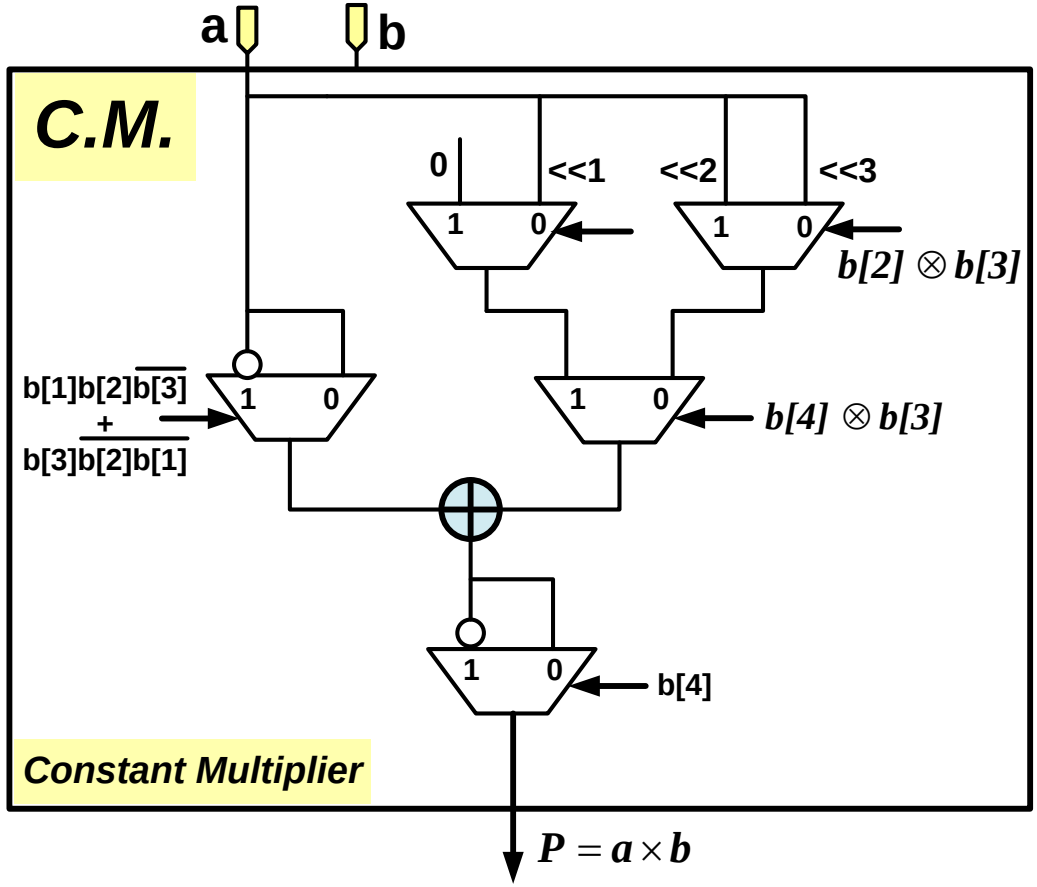


Verilog Operations: Constant Multiplication

- Simpler way for implementation: $P = a \times b$ $\quad b \in \{-7,-5,-3,-1,1,3,5,7\}$

Area (um^2)	Critical Path	Multiplier
1800	3.5	Constant MUL
12000	5.1	Normal MUL





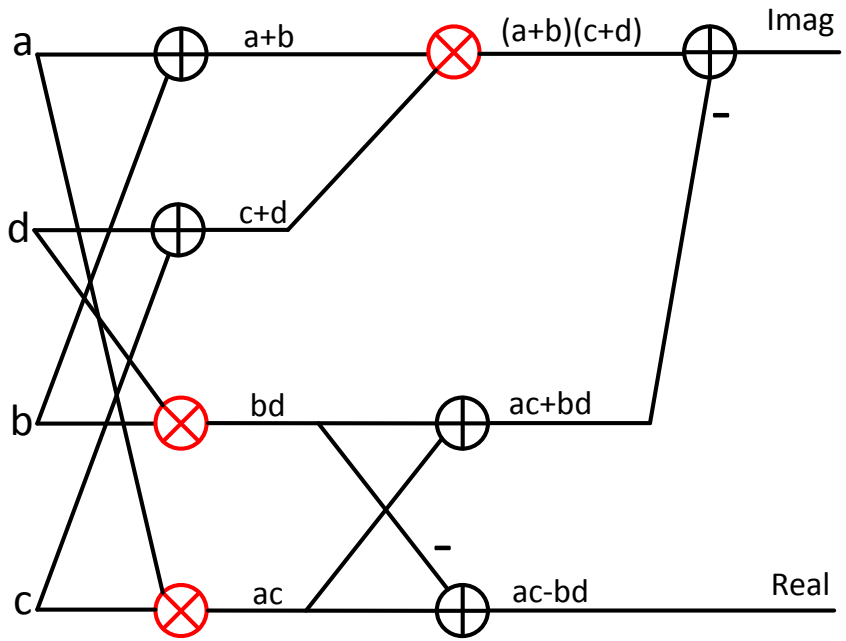
Verilog Operations: Complex Multiplication

- A complex multiplication is equivalent to four real multiplications

$$(a + jb)(c + jd) = (ac - bd) + j(ad + bc)$$

- However, it can be efficiently realized using only three real multiplications:

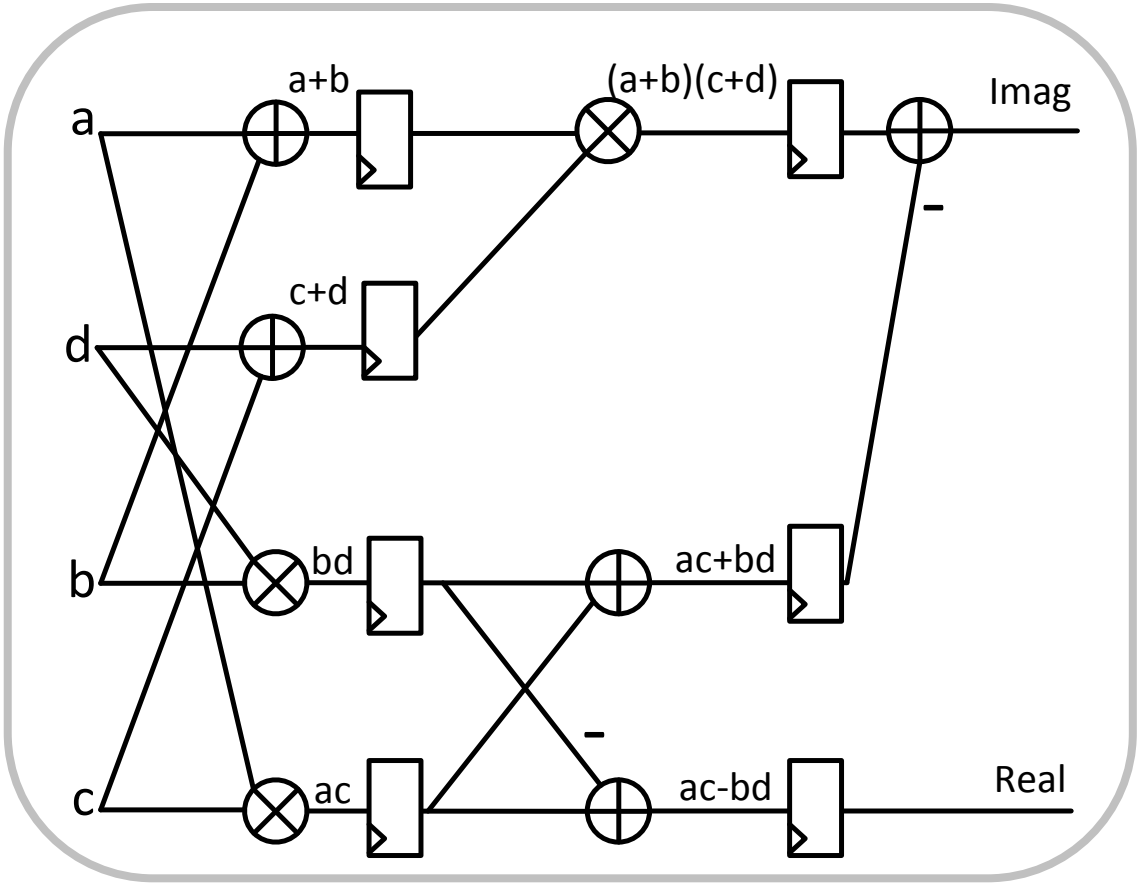
$$(a + jb)(c + jd) = (ac - bd) + j[(a + b)(c + d) - (ac + bd)]$$



Pipelined Complex Multiplication

- Pipelined Implementation:

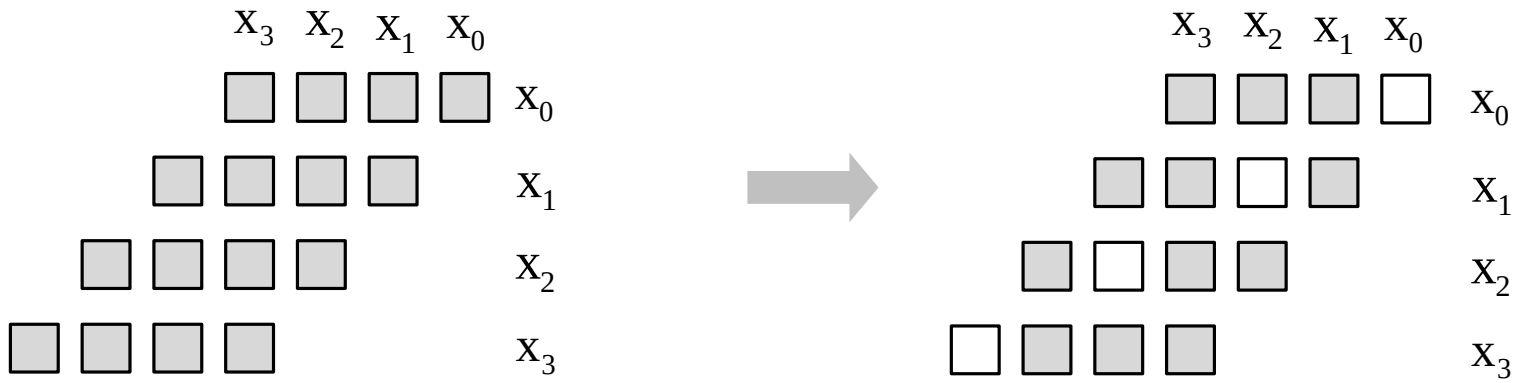
$$(a+jb)(c+jd) = (ac-bd) + j[(a+b)(c+d) - (ac+bd)]$$



© M. Shabany, ASIC/FPGA Chip Design

Squaring

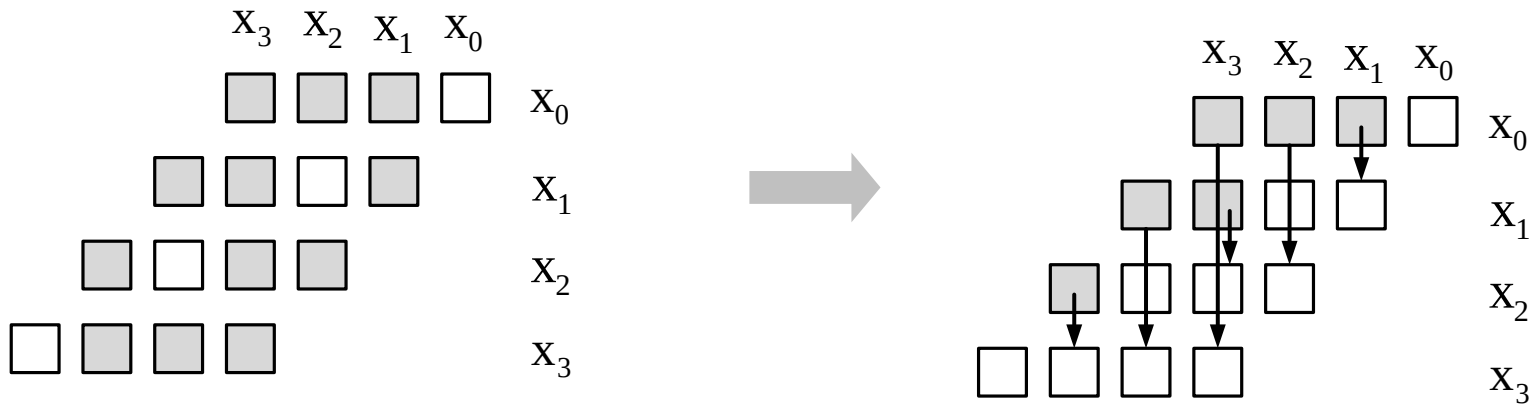
- x^2 can be done with about half the hardware of a full multiply (for a *dedicated* squaring block, of course)



- Diagonals ($x_0\ x_0$, $x_1\ x_1$, …) can be replaced by the single input bit with no computation for that bit b/c we have x_0 AND $x_{0=}\ x_0$

Squaring

- Pairs of equivalent bit products ($x_1\ x_0$ and $x_0\ x_1$, …) can be replaced by one bit product shifted over one column



Resource-Shared Complex Multiplication

- Operands: 4 integer, 12 fraction bits
- Result: 8 pre-, 24 post-binary-point bits
- Subject to tight area constraints

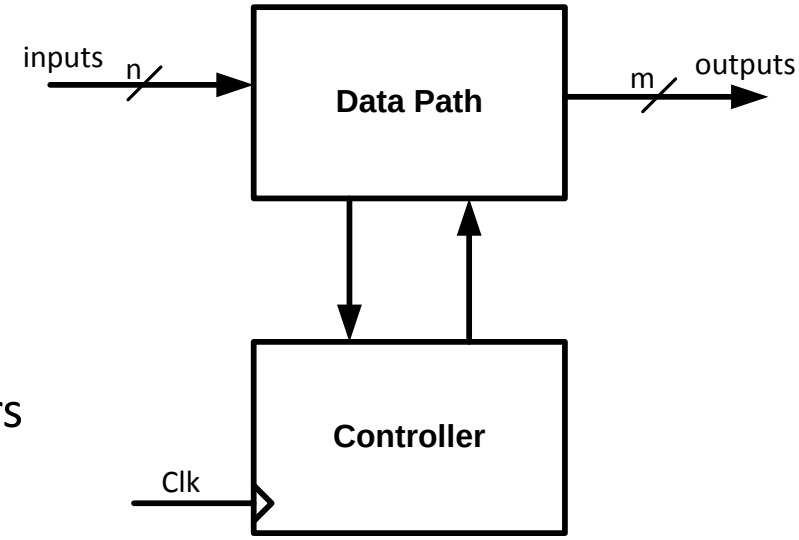
$$a = a_r + ja_i \qquad b = b_r + jb_i$$

$$p = ab = p_r + jp_i = (a_r b_r - a_i b_i) + j(a_r b_i + a_i b_r)$$

- 4 multiplies, 1 add, 1 subtract
- Perform sequentially using 1 multiplier, 1 adder/subtracter

Digital Systems

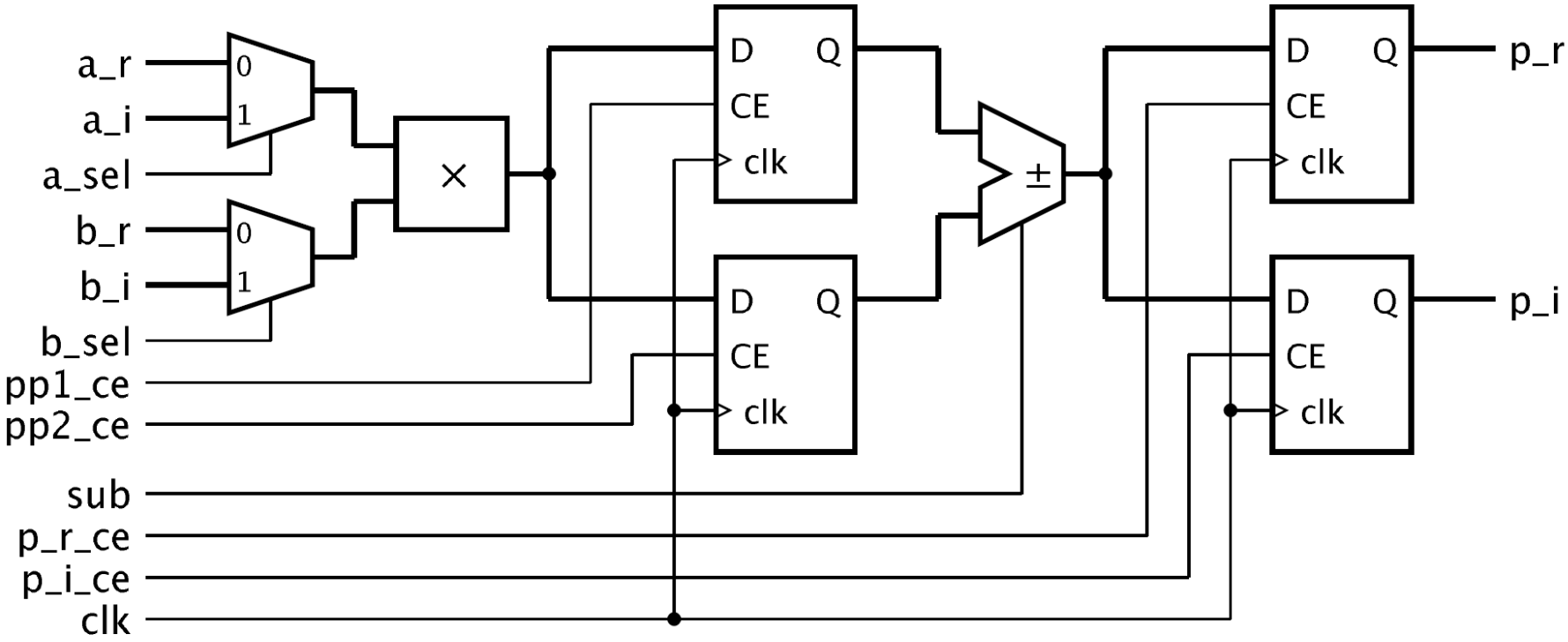
General Architecture:



- **Data Path:**
  - Transfer input data signals into outputs
  - Normally combinational logic or counters
- **Controller:**
  - Provides any control signal to determine the direction of data flow
  - Examples: Reset, set, MUX select signals, …
  - Sequential logic

Resource-Shared Complex Multiplication

- Data Path VLSI Architecture:



Resource-Shared Complex Multiplication

- HDL Code:

- Control Path:
  1. a_r * b_r →pp1_reg
  2. a_i * b_i →pp2_reg
  3. pp1 –pp2 →p_r_reg
     a_r * b_i →pp1_reg
  4. a_i * b_r →pp2_reg
  5. pp1 + pp2 →p_i_reg

- Takes 5 clock cycles

```
module multiplier
  ( output reg signed [7:-24] p_r, p_i,
    input      signed [3:-12] a_r, a_i, b_r, b_i,
    input                     clk, reset, input_rdy );
  reg a_sel, b_sel, pp1_ce, pp2_ce, sub, p_r_ce, p_i_ce;
  wire signed [3:-12] a_operand, b_operand;
  wire signed [7:-24] pp, sum
  reg  signed [7:-24] pp1, pp2;
  assign a_operand = ~a_sel ? a_r : a_i;
  assign b_operand = ~b_sel ? b_r : b_i;
  assign pp = {{4{a_operand[3]}}, a_operand, 12'b0} *
              {{4{b_operand[3]}}, b_operand, 12'b0};
  always @(posedge clk)  // Partial product 1 register
    if (pp1_ce) pp1 <= pp;
  always @(posedge clk)  // Partial product 2 register
    if (pp2_ce) pp2 <= pp;
  assign sum = ~sub ? pp1 + pp2 : pp1 - pp2;
  always @(posedge clk)  // Product real-part register
    if (p_r_ce) p_r <= sum;
  always @(posedge clk)  // Product imaginary-part register
    if (p_i_ce) p_i <= sum;
  ...
endmodule
```

Resource-Shared Complex Multiplication

- Control Logic (Timing Schedule):

Step	a_sel	b_sel	pp1_ce	pp2_ce	sub	p_r_ce	p_i_ce
1	0	0	1	0	–	0	0
2	1	1	0	1	–	0	0
3	0	1	1	0	1	1	0
4	1	0	0	1	–	0	0
5	–	–	0	0	0	0	1

Resource-Shared Complex Multiplication

- Control Logic Design:
- One state per step
- Separate idle state?
  - Wait for input_rdy = 1
  - Then proceed to steps 1, 2, ...
  - But this wastes a cycle!
- Use step 1 as idle state
  - Repeat step 1 if input_rdy ≠ 1
  - Proceed to step 2 otherwise

current_ state	input_ rdy	next_ state
step1	0	step1
step1	1	step2
step2	–	step3
step3	–	step4
step4	–	step5
step5	–	step1

Resource-Shared Complex Multiplication

- Control Logic Design:

```
parameter [2:0] step1 = 3'b000, step2 = 3'b001,
                step3 = 3'b010, step4 = 3'b011,
                step5 = 3'b100;
reg [2:0] current_state, next_state ;

 always @(posedge clk or posedge reset)  // State register
   if (reset) current_state <= step1;
   else       current_state <= next_state;
 always @*  // Next-state logic
   case (current_state)
     step1: if (!input_rdy) next_state = step1;
            else            next_state = step2;
     step2:                 next_state = step3;
     step3:                 next_state = step4;
     step4:                 next_state = step5;
     step5:                 next_state = step1;
   endcase
```

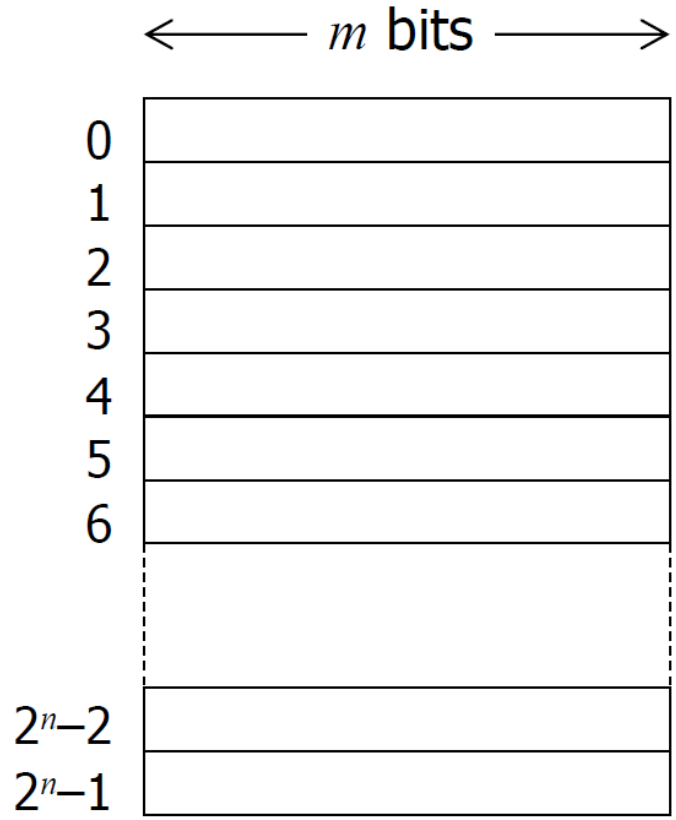
Resource-Shared Complex Multiplication

- Control Logic Design:

```
always @* begin  // Output_logic
  a_sel = 1'b0; b_sel = 1'b0;  pp1_ce = 1'b0; pp2_ce = 1'b0;
  sub = 1'b0;   p_r_ce = 1'b0; p_i_ce = 1'b0;
  case (current_state)
    step1: begin
             pp1_ce = 1'b1;
           end
    step2: begin
             a_sel = 1'b1; b_sel = 1'b1; pp2_ce = 1'b1;
           end
    step3: begin
             b_sel = 1'b1; pp1_ce = 1'b1;
             sub = 1'b1;   p_r_ce = 1'b1;
           end
    step4: begin
             a_sel = 1'b1; pp2_ce = 1'b1;
           end
    step5: begin
             p_i_ce = 1'b1;
           end
  endcase
end
```

Memories

- A memory is an array of storage locations
  - Each with a unique address
  - Like a register bank, but with optimized implementation
- Address is unsigned-binary encoded
- n address bits ⇒ 2^n locations
- All locations the same size
- $2^n \times m$ bit memory

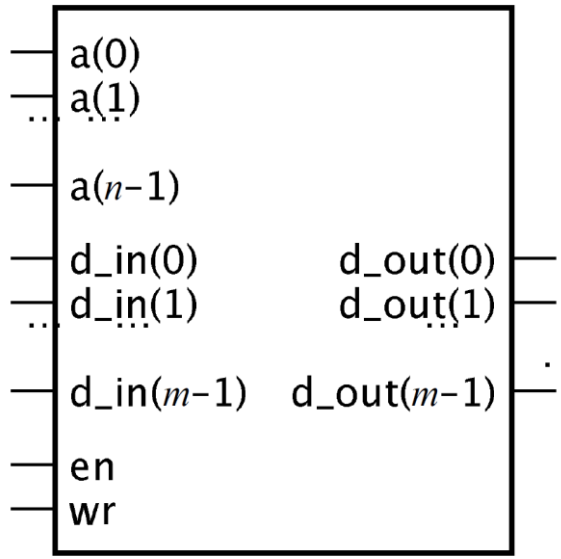


Memory Sizes

- Use power-of-2 multipliers
- Kilo (K): $2^{10} = 1{,}024 \approx 10^3$
- Mega (M): $2^{20} = 1{,}048{,}576 \approx 10^6$
- Giga (G): $2^{30} = 1{,}073{,}741{,}824 \approx 10^9$
- Example:
  - 32K ×32-bit memory
  - Capacity = 1,024K = 1Mbit
  - Requires 15 address bits
- Size is determined by application requirements

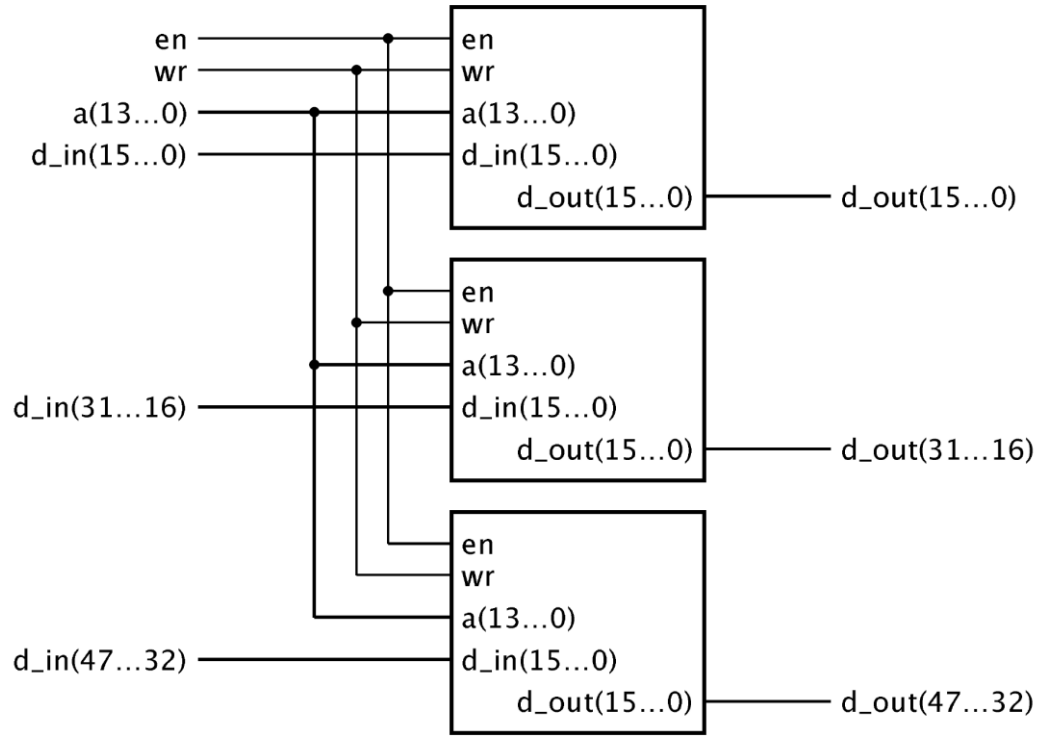
Basic Memory Operations

- a inputs: unsigned address
- d_in and d_out
- Type depends on application
- Write operation
  - en = 1, wr = 1
  - d_in value stored in location given by address inputs
- Read operation
  - en = 1, wr = 0
  - d_out driven with value of location given by address inputs
- Idle: en = 0



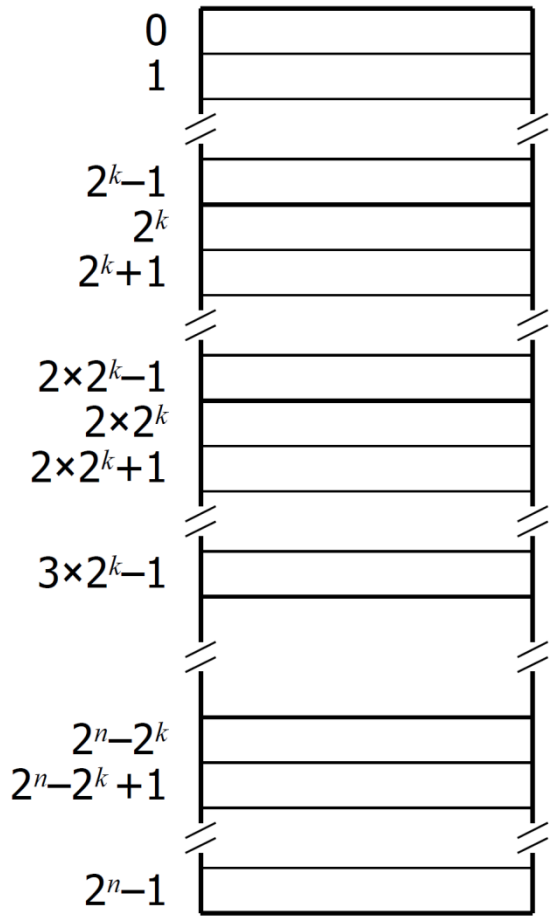
Wider Memories

- Memory components have a fixed width
- E.g., ×1, ×4, ×8, ×16, ...
- Use memory components in parallel to make a wider memory
- E.g, three 16K×16 components for a 16K×48 memory



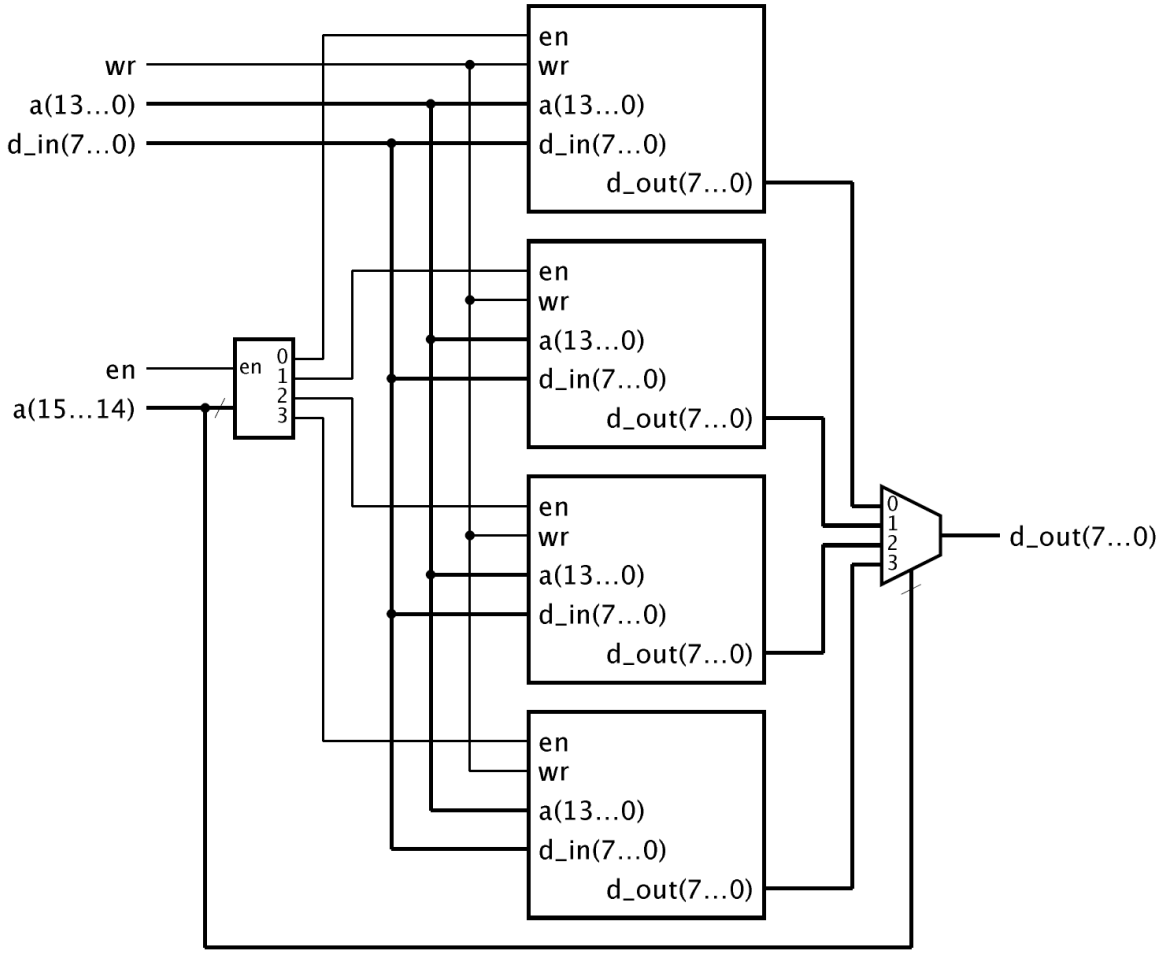
Larger Memories

- To provide 2^n locations with 2^k-location components
- Use $2^n/2^k$ components



Larger Memories

- Example: 64K×8 memory composed of 16K×8 components

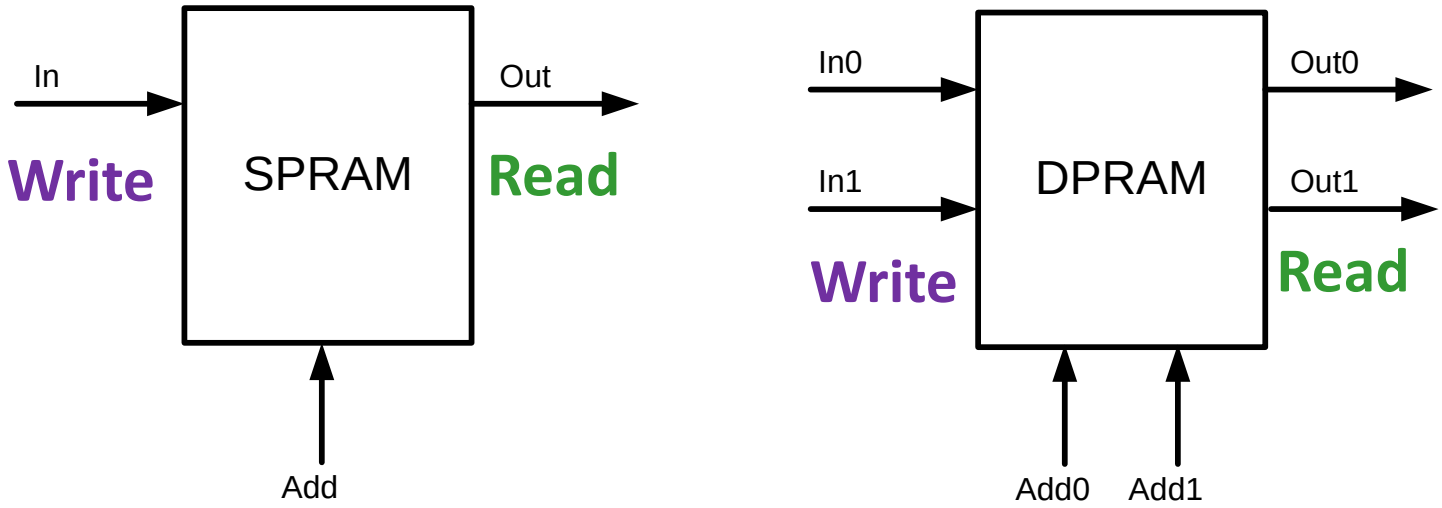


Memory Types

- Random-Access Memory (RAM)
  - Can read and write
  - Static RAM (SRAM)
    - Stores data so long as power is supplied
    - Asynchronous SRAM: not clocked
    - Synchronous SRAM (SSRAM): clocked
  - Dynamic RAM (DRAM)
    - Needs to be periodically refreshed
- Read-Only Memory (ROM)
  - Combinational
  - Programmable and Flash rewritable
- Volatile and non-volatile

Verilog Memories: Single-Port vs. Dual-Port RAM

- **Single-port RAM (SPRAM):**
  - Can only be accessed at one address at one time
  - Read or Write (not both) one memory cell at a time in each clock cycle
- **Dual-port RAM (DPRAM):**
  - Can be accessed at two addresses at one time
  - Read & Write different memory cells at different addresses simultaneously



Verilog Memories: Single-Port RAM

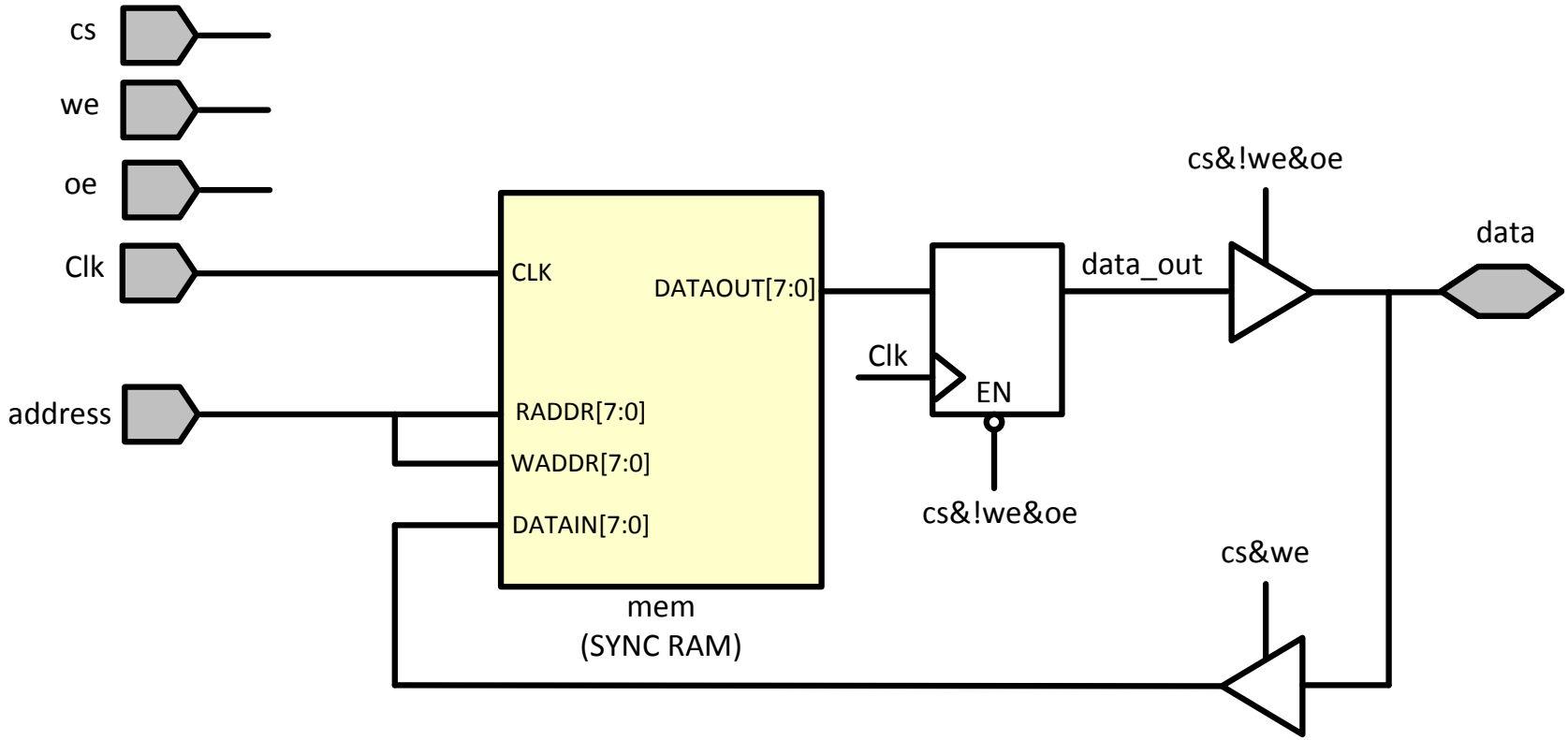
- 256-Byte SPRAM:
  - With chip select and read/write enable

```
module SPRAM ( clk , address , data, cs, we , oe);
// cs:chip select, we:Write/Read Enable, oe: Output Enable
parameter DATA_WIDTH = 8 ;
parameter ADDR_WIDTH = 8 ;
parameter RAM_DEPTH = 1 << ADDR_WIDTH;
input [ADDR_WIDTH-1:0] address ;
input clk, cs, we, oe ;
inout [DATA_WIDTH-1:0] data ;
reg [DATA_WIDTH-1:0] data_out ;
reg [DATA_WIDTH-1:0] mem [0:RAM_DEPTH-1];
//Tri-state buffer, Output: When we = 0 (read), oe = 1, cs = 1
assign data = (cs && oe && !we) ? data_out : 8'bz;
```

```
// CONTINUED
// Memory Write : when we = 1, cs = 1
always @ (posedge clk)
  begin  : MEM_WRITE_READ
   if ( cs && we )
      mem[address] <= data;
// Memory Read: when we = 0, oe = 1, cs = 1
   else if (cs && !we && oe)
      data_out <= mem[address];
  end
endmodule
```

Verilog Memories: Single-Port RAM

- 256-Byte SPRAM:
  - With chip select and read/write enable



Verilog Memories: Dual-Port RAM

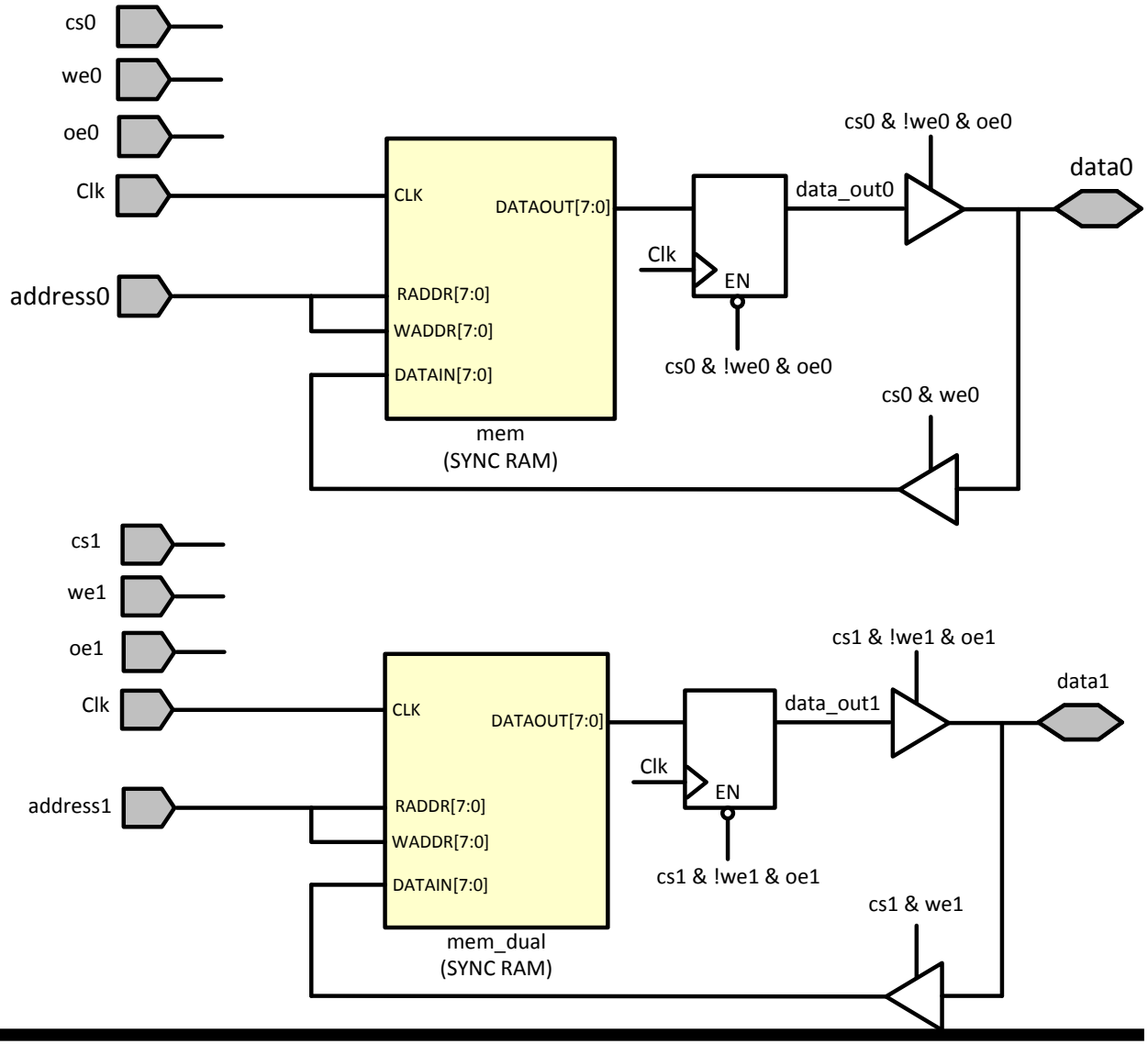
- 256-Byte DPRAM:
  - Two separate read/write operations

```
module SPRAM ( clk , address_0 , data_0, cs_0, we_0 , oe_0,
 address_1 , data_1, cs_1, we_1 , oe_1);
// cs:chip select, we:Write/Read Enable, oe: Output Enable
parameter DATA_WIDTH = 8 ;
parameter ADDR_WIDTH = 8 ;
parameter RAM_DEPTH = 1 << ADDR_WIDTH;
input [ADDR_WIDTH-1:0] address_0, address_1 ;
input clk, cs_0, we_0 , oe_0, cs_1, we_1 , oe_1 ;
inout [DATA_WIDTH-1:0] data_1, data_2 ;
reg [DATA_WIDTH-1:0] data_out _0, data_out _1;
reg [DATA_WIDTH-1:0] mem [0:RAM_DEPTH-1];
//Tri-state buffer, Output: When we = 0, oe = 1, cs = 1
assign data _0= (cs_0 && oe_0 && !we_0) ? data_out _0: 8'bz;
assign data _1= (cs_1 && oe_1&& !we_1) ? data_out _1: 8'bz;
```

```
// CONTINUED
// Memory Write : when we = 1, cs = 1
always @ (posedge clk)
  begin  : MEM_WRITE_READ
    if ( cs_0 && we_0 )
       mem[address_0] <= data_0;
  else if (cs_0 && !we_0 && oe_0)
       data_out_0 <= mem[address_0];
   end
  if ( cs_1 && we_1 )
       mem[address_1] <= data_1;
    else if (cs_1 && !we_1 && oe_1)
       data_out_1 <= mem[address_1];
  end
endmodule
```

Verilog Memories: Dual-Port RAM

- 256-Byte DPRAM:



Verilog Memories: Dual-Port RAM

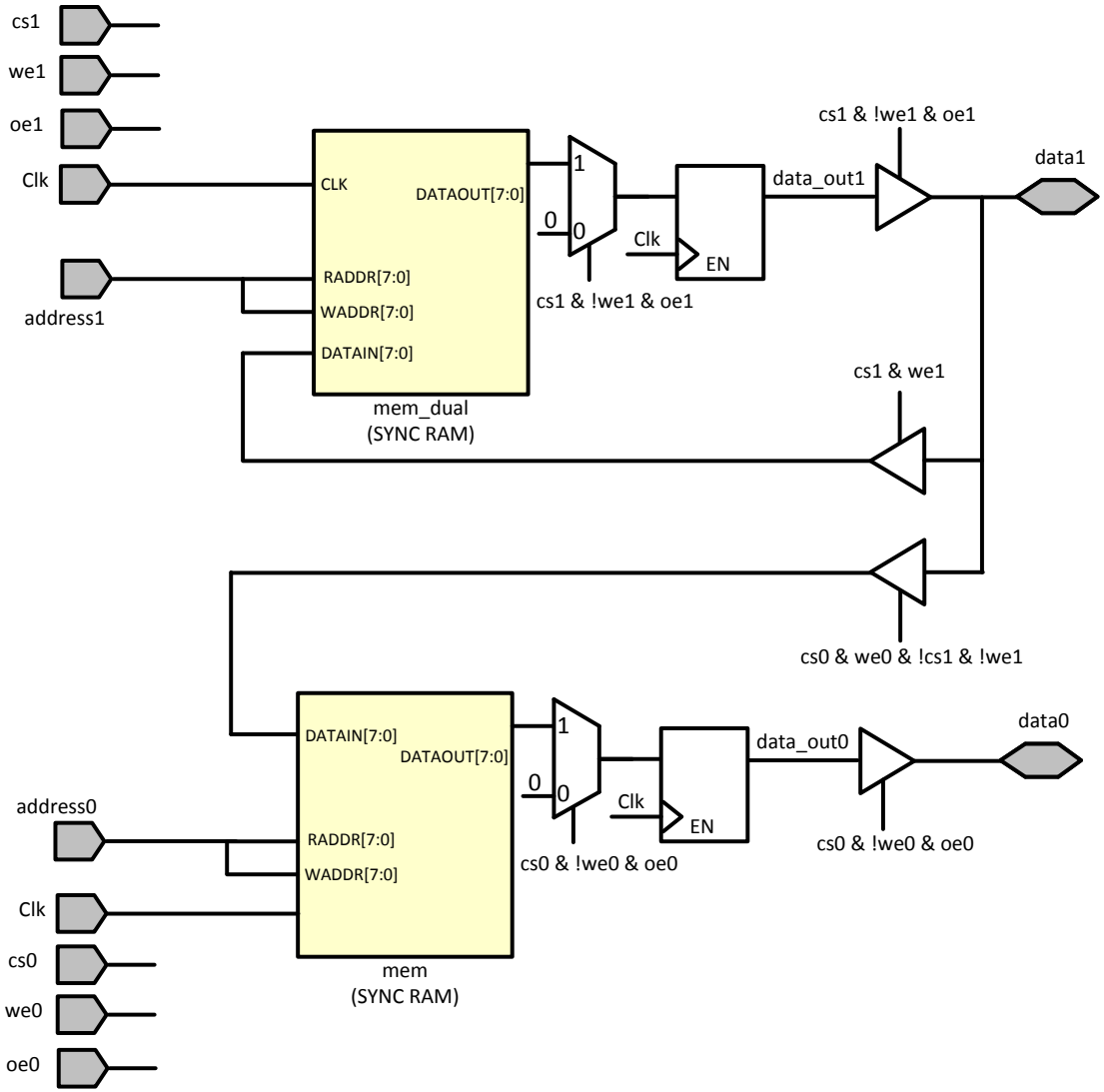
- 256-Byte DPRAM:
  - One write two reads simultaneously

```
module SPRAM ( clk , address_0 , data_0, cs_0, we_0 , oe_0,
 address_1 , data_1, cs_1, we_1 , oe_1);
// cs:chip select, we:Write/Read Enable, oe: Output Enable
parameter DATA_WIDTH = 8 ;
parameter ADDR_WIDTH = 8 ;
parameter RAM_DEPTH = 1 << ADDR_WIDTH;
input [ADDR_WIDTH-1:0] address_0, address_1 ;
input clk, cs_0, we_0 , oe_0, cs_1, we_1 , oe_1 ;
inout [DATA_WIDTH-1:0] data_1, data_2 ;
reg [DATA_WIDTH-1:0] data_out _0, data_out _1;
reg [DATA_WIDTH-1:0] mem [0:RAM_DEPTH-1];
//Tri-state buffer, Output: When we = 0, oe = 1, cs = 1
assign data _0= (cs_0 && oe_0 && !we_0) ? data_out _0: 8'bz;
assign data _1= (cs_1 && oe_1&& !we_1) ? data_out _1: 8'bz;
```

```
// CONTINUED
// Memory Write : when we = 1, cs = 1
always @ (posedge clk)
  begin  : MEM_WRITE_READ
   if ( cs_0 && we_0 )
       mem[address_0] <= data_0;
  else if ( cs_1 && we_1 )
       mem[address_1] <= data_1;
   end
end
always @ (posedge clk)
 if (cs_0 && !we_0 && oe_0)
       data_out_0 <= mem[address_0];
   else
        data_out_0 <= 0;
always @ (posedge clk)
 if (cs_1 && !we_1 && oe_1)
       data_out_1 <= mem[address_1];
   else
        data_out_1 <= 0;
endmodule
```

Verilog Memories: Dual-Port RAM

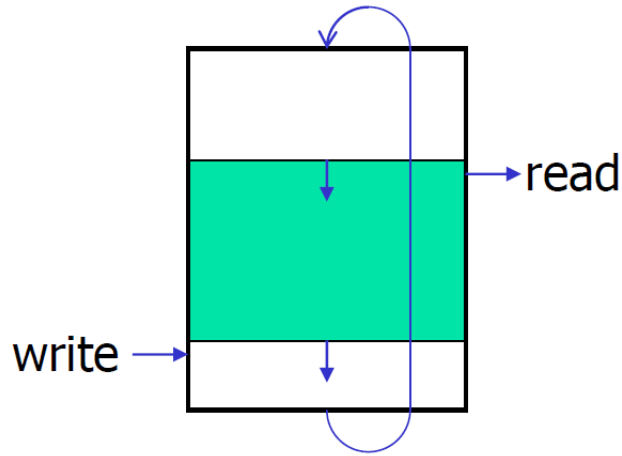
- 256-Byte DPRAM:



FIFO

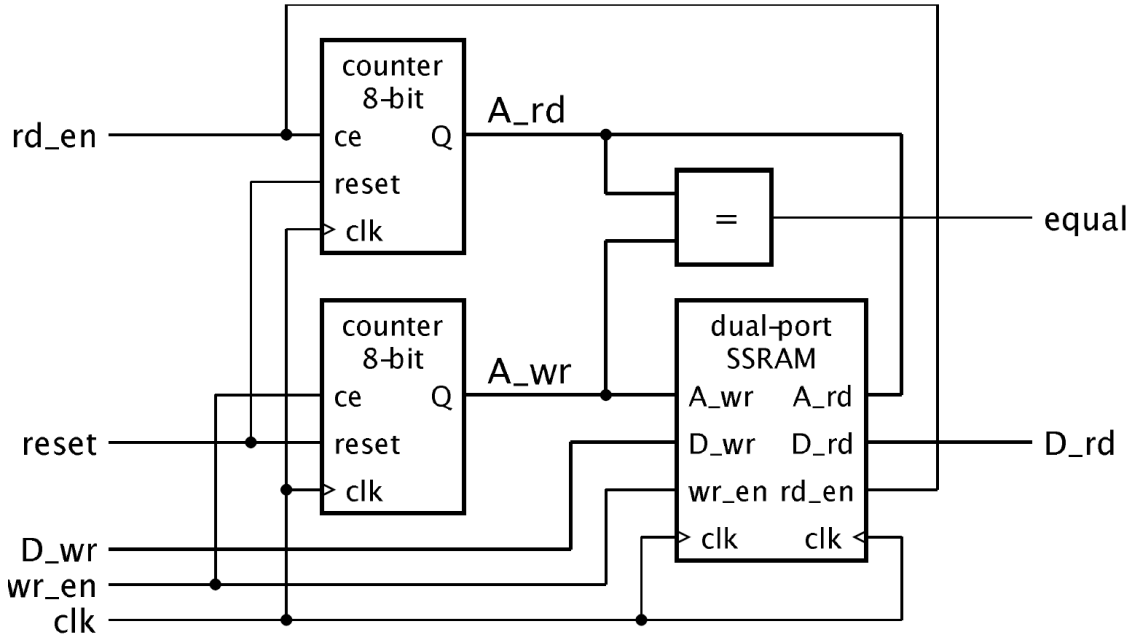
- First-In/First-Out buffer
- Connecting producer and consumer
- Decouples rates of production/consumption

- Implementation using dual-port RAM
- Circular buffer
- Full: write-addr = read-addr
- Empty: write-addr = read-addr



FIFO Example

- Design a FIFO to store up to 256 data items of 16-bits each, using 256x 16-bit dual-port SSRAM for the data storage. Assume the FIFO will not be read when it is empty, not to be written when it is full, and that the write and read ports share a common clock.



Verilog Memories: ROM

- For constant data, or CPU programs
- Masked ROM
  - Data manufactured into the ROM
- Programmable ROM (PROM)
  - Use a PROM programmer
- Erasable PROM (EPROM)
  - UV erasable
  - Electrically erasable (EEPROM)
  - Flash RAM

Verilog Memories: ROM

- ROM can be realized using two methods:
  - Initialized using a file
  - Initialized explicitly using case statement

```
module ROMFile( address , data , read_en , ce );
 input [7:0] address;
output [7:0] data;
input read_en,  ce;
reg [7:0] mem [0:255] ;
assign data = (ce && read_en) ? mem[address] : 8'b0;
initial
   begin
       $readmemb("memory.list", mem);
       // memory.list is the memory file
    end
endmodule
```

```
module ROMCase( address , data , read_en , ce );
 input [2:0] address;
output reg [7:0] data;
input read_en,  ce;

always @ (ce or read_en or address) begin
        case (address)
          0 : data = 10;
          1 : data = 55;
          2 : data = 244;
          3 : data = 0;
          4 : data = 1;
          5 : data = 8'hff;
          6 : data = 8'h11;
          7 : data = 8'h1;
         endcase
 end
endmodule
```

Function Implementation using look-up tables

- Complex or arbitrary functions are not uncommon
- Example:

$$S(x,y) = \int\int F(\mathcal{X},\mathcal{Y},\mathcal{Z}) e^{-j2k(r_i+r_r)}\, d\mathcal{X}\, d\mathcal{Y}$$

- One way is to implement it using Look-up Tables
- Often best to write a Matlab program to write the Verilog table as plain text
- Easy to adapt to other specifications
- Not efficient for very large tables
- Tables with data that is less random will have smaller synthesized area

```
always @ (input) begin
case (input)
  4'b0000: begin real=3'b100; imag=3'b001; end;
  4'b0001: begin real=3'b000; imag=3'b101; end;
  4'b0010: begin real=3'b110; imag=3'b011; end;
                ……
   default: begin real=3'bxxx; imag=3'bxxx; end;
endcase
 end
```

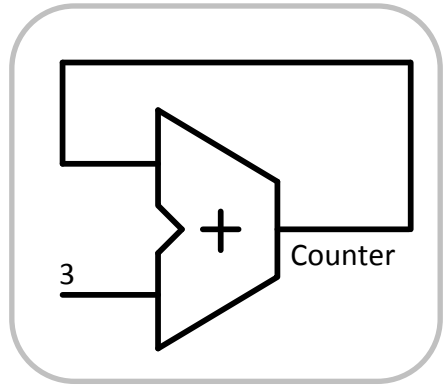
Reviews and Notes

- Every Verilog statement must end with a semicolon ";"
- For comparison "==" has to be used not "="

- When there are multiple assignments to the same variable in an always block, the last statement is evaluated
  - Example:

```
module DUT(Count );
output reg [2:0] Count;
integer k;

always @ (*)
      begin
            Count <= 0;
            for (k=0; k<4; k=k+1)
                Count <= Count + k;
      end
endmodule
```

```
module DUT(Count );
output reg [2:0] Count;
integer k;

always @ (*)
      Count <= Count + 3;
endmodule
```



Reviews and Notes

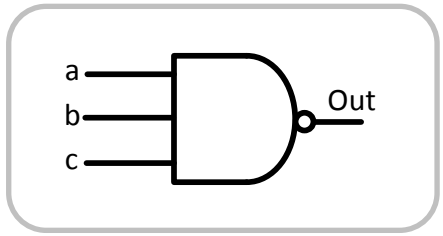
- Two codes with different simulation results might have the same synthesized circuit

```
always @ (a, b, c)
    if (a & b & c)
        Out =0;
    else
        Out = 1;
```

I

```
always @ (a, b)
    if (a & b & c)
        Out =0;
    else
        Out = 1;
```

II



Same Synthesized Circuit

a	b	c	I	II
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	1

Different Simulation

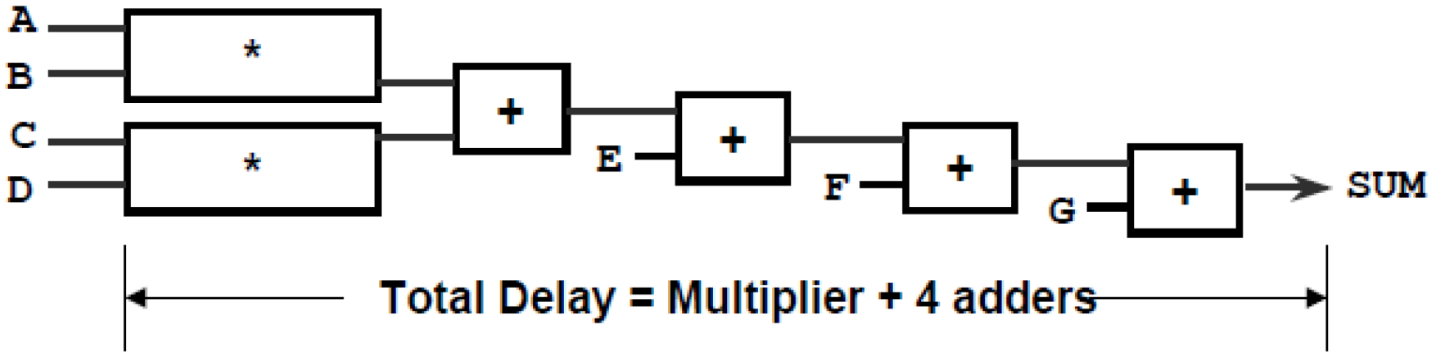
- Therefore, to avoid mismatch b/w simulation and synthesized version, the sensivity list of always block should include all the signals on the RHS

Coding Styles

- Do not mix blocking and non-blocking assignments in an always block
- Use parentheses to optimize logic structure
- Use meaningful names for signals, variables, and modules
- Define if-else and case statements explicitly to avoid latch inference
- Multiple procedural assignments (inside an always block) to a single variable is allowed.
  - The last assignment is evaluated.
- Multiple continuous assignments (assign) to a single net in **NOT** allowed.
- Do not mix edge and level sensitive elements together
- Use assign statements for simple comb. logic and always block for complex comb. logic
- Avoid mixing positive-edge and negative-edge triggered flip-flops in one design
  - Confuses the timing closure

Coding Styles : Parentheses

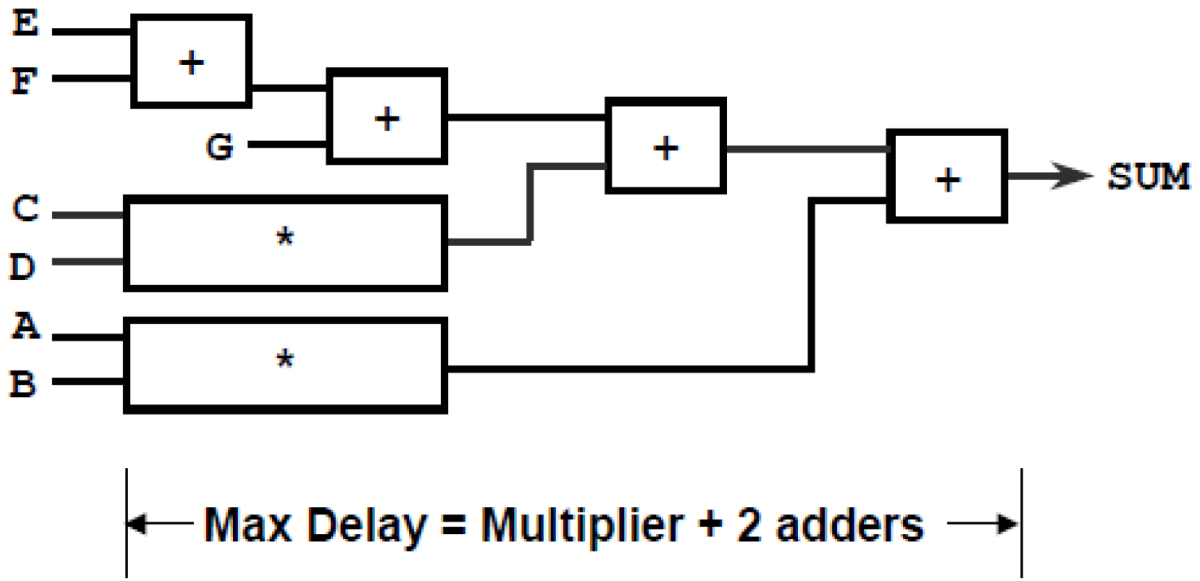
```
SUM <= A*B + C*D + E + F + G
```



Coding Styles : Parentheses

```
SUM <= E + F + G + C*D + A*B
```

```
SUM <= (A*B) + ( (C*D) + ((E+F) + G) )
```



Difference b/w HDL and HLL (1)

- In HLL (high-level language) assignment order is important
- In HDL for "assign" and "non-blocking" assignments, order is NOT important
- Example:

HLL:

```
a = 1; b=0; s=0; na=0; nb=0;
y = na|nb;
nb = b&s;
na = a&~s;
k = a&b;
```

Result: y=0;

```
a = 1; b=0; s=0; na=0; nb=0;
nb = b&s;
na = a&~s;
k = a&b;
y = na|nb;
```

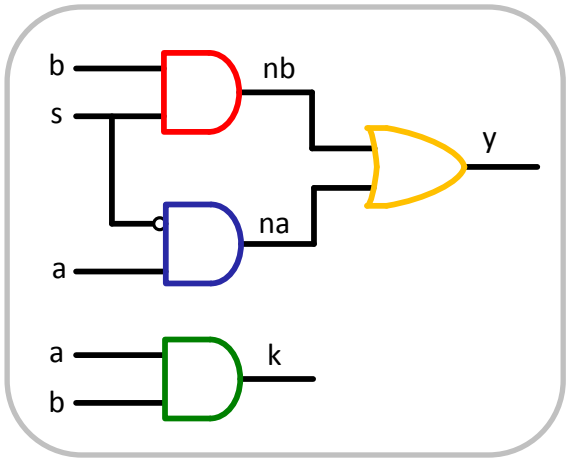
Result: y=1;

HDL:

```
wire na, nb;

assign y = na|nb;
assign nb = b&s;
assign na = a&~s;
assign k = a&b;
```

```
wire na, nb;

assign na = a&~s;
assign k = a&b;
assign nb = b&s;
assign y = na|nb;
```

The same!



Difference b/w HDL and HLL (2)

- In HLL, multiple assignments to a single signal is allowed
- In HDL , multiple continuous assignments to a signal is NOT allowed.

❖ Example:

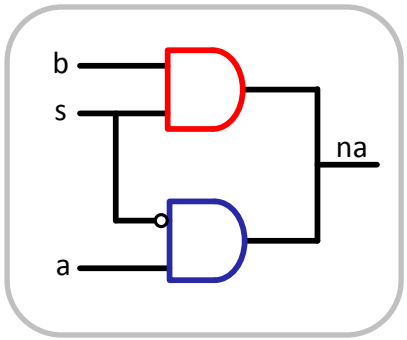
HLL: ⇨

```
a = 1; b=0; s=0; na=0; nb=0;
y = na|nb;
na = b&s;
na = a&~s;
```

Result: na = a&~s;

HDL: ⇨

```
wire na;

assign y = na|nb;
assign na = b&s;
assign na = a&~s;
```



Illegal
(only used for tri-state implementation)

ASIC/FPGA Chip Design

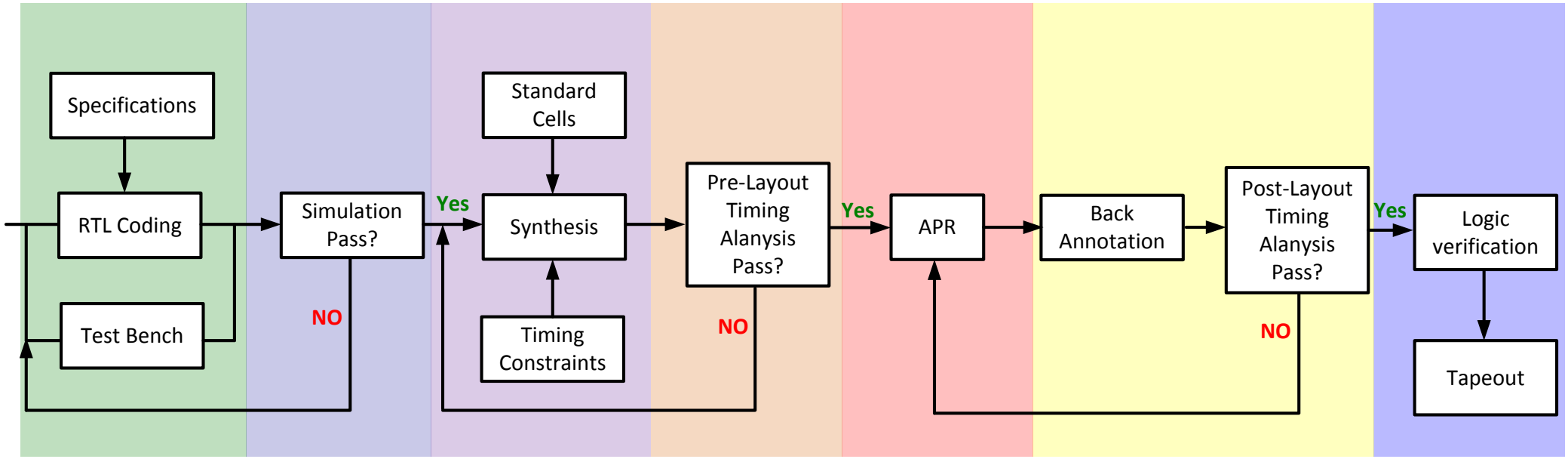
Verilog for Synthesis

Mahdi Shabany

Department of Electrical Engineering

Sharif University of technology

ASIC/FPGA Design Flow

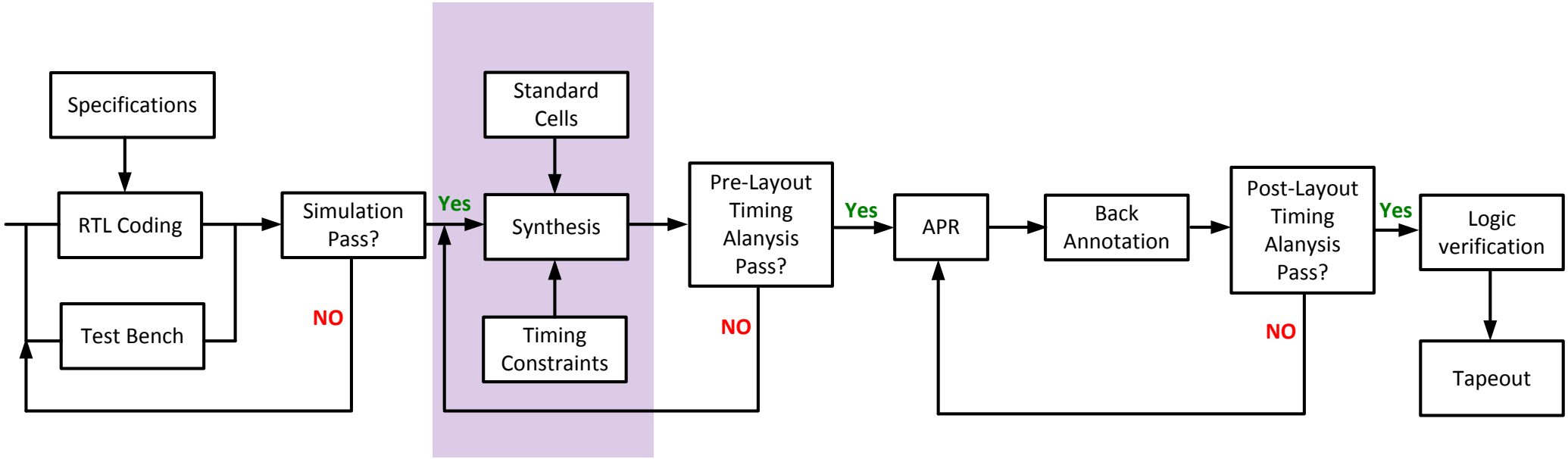


1. HDL Coding **2. Simulation** **3. Synthesis** — **Front-End**

4. Placement & routing **5. Timing Analysis & Verification** — **Back-End**

- In this course we learn all the above steps in detail for
  - ASIC Platform
  - FPGA Platform

Synthesis



- Synthesis tool:
  - Analyzes a piece of Verilog code and converts it into optimized logic gates
  - This conversion is done according to the **"language semantics"**
  - ➡ We have to learn these language semantics, i.e., Verilog code.

Synthesis

- Why using synthesis tools?

  - It is an important tool to improve designers' productivity to meet today's design complexity.

  - If a designer can design 150 gates a day, it will take 6666 man's day to design a 1-million gate design, or almost 2 years for 10 designers! This is assuming a linear grow of complexity when design get bigger.

Synthesis in Different Levels

- Synthesis can be done in different levels:
  - High-level Synthesis
    - To convert an algorithm-level description to an RTL code
  - RTL Synthesis
    - To convert an RTL code to a gate-level netlist
  - Logic Synthesis
    - To convert the gate-level description to a specific logic library

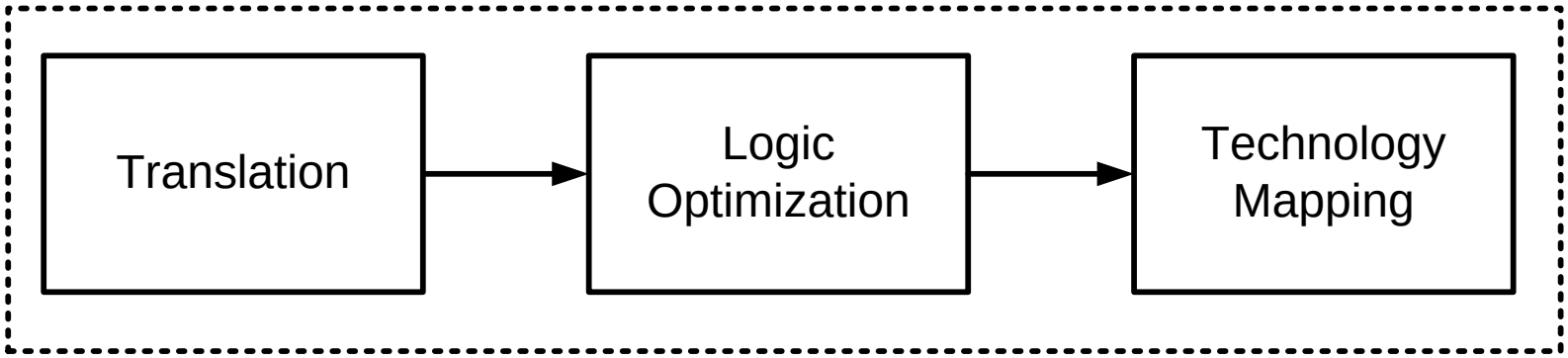
Synthesis

- Synthesis tool: (RTL & Logic Synthesis)
  - **Input:**
    - HDL Code
    - "Technology library" file → Standard cells (known by transistor size, 90nm)
      - Basic gates (AND, OR, NOR, …)
      - Macro cells (Adders, Muxes, Memory, Flip-flops, …)
    - Constraint file (Timing, area, power, loading requirement, optimization Alg.)
  - **Output:**
    - A gate-level "Netlist" of the design
    - Timing files (.sdf)

This process is done using various optimization algorithms

Synthesis

- **Synthesis** = Translation + Logic Optimization + Technology Mapping
  - **Translation:** going from RTL to Boolean function
  - **Logic Optimization :** Optimizing and minimizing Boolean function
  - **Technology Mapping (TM):** Map the Boolean function to the target library



Synthesis

Synthesis

❑ **Synthesis** = Translation + Logic Optimization + Technology Mapping

```
always @ (a, b)
      case ({a,b})
          2'b00: out = 1;
          2'b01: out = 1;
          2'b11: out = 1;
          default: out = 0;
      endcase
```

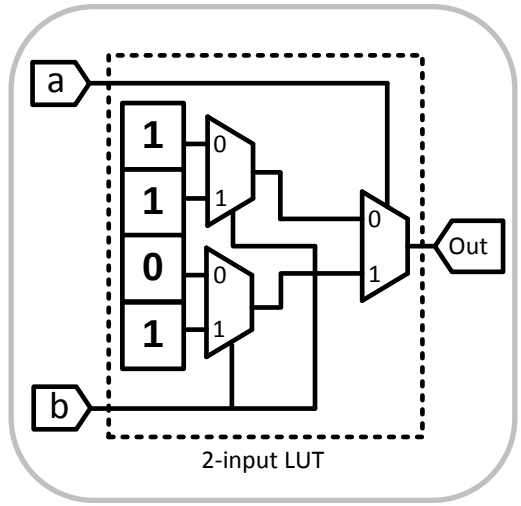
Translation

$$out = \bar{a}b + ab + \bar{a}\bar{b}$$

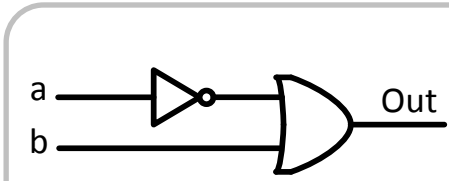
Logic Optimization

$$out = \bar{a} + b$$

Technology Mapping



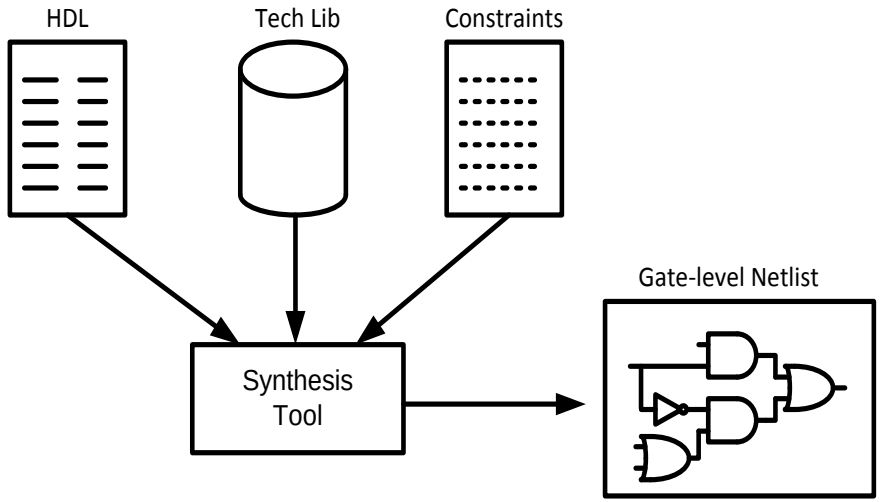
FPGA



ASIC

Synthesis Tools

❑ Synthesis tool:

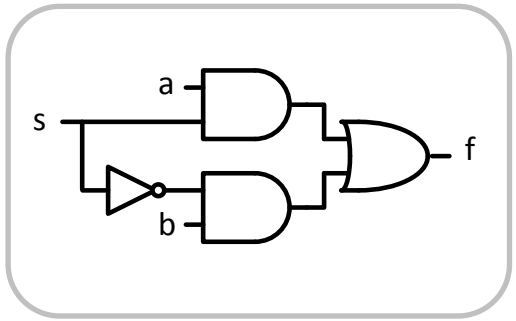


❖ **Example:** A 2-to-1 Multiplexer (2x1-MUX)

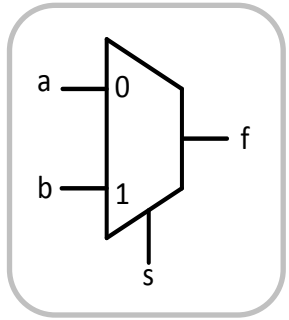
```
If (s==0)
   f = a;
else
   f = b;
```

Verilog code
(has to comply with certain structures)

Synthesis Tool



Synthesized gate-level



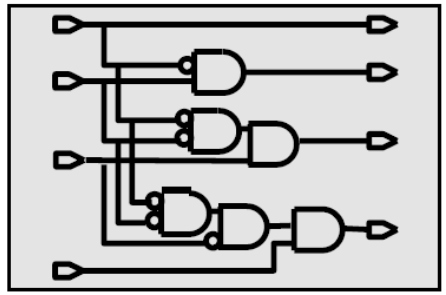
Schematic

Synthesis

```
residue = 16'h0000;
if (high_bits == 2'b10)
   residue = state_table[index];
else
   state_table[index] = 16'h0000;
```

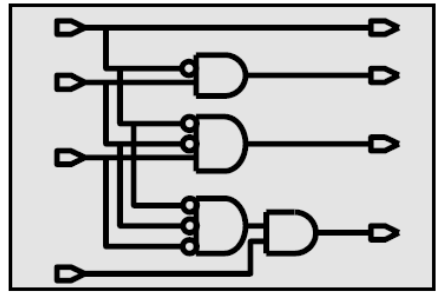
HDL Source

Translate (read)



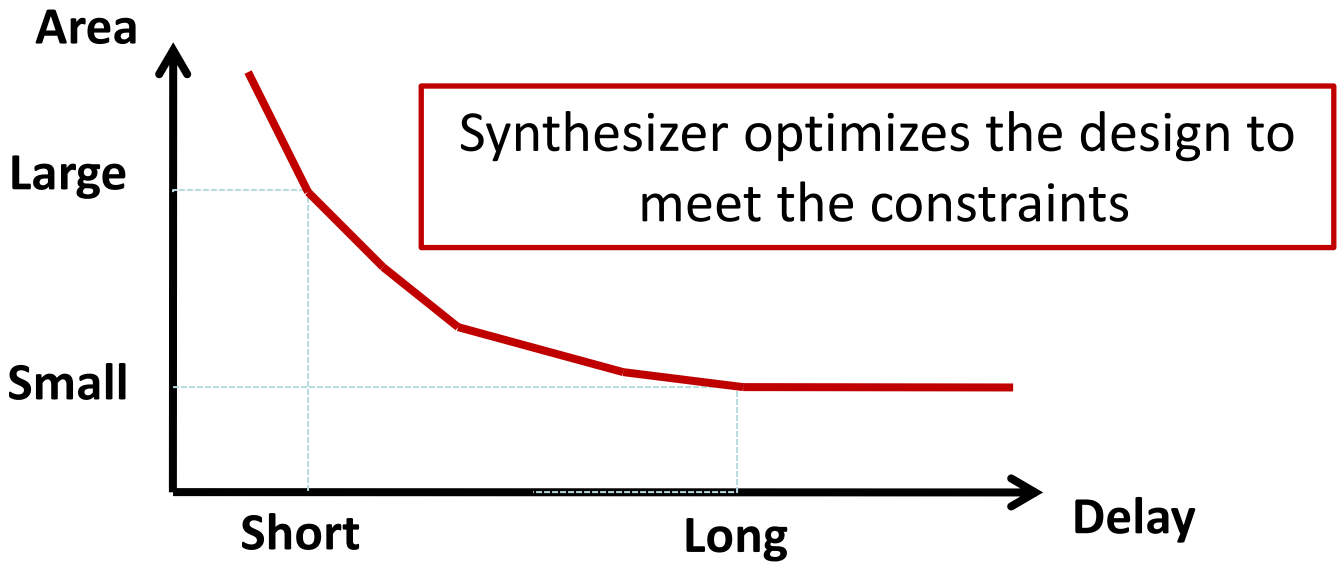
Generic Boolean
(GTECH)

Optimize + Map
(compile)



Target Technology

Synthesis is Constraint-Driven



Synthesis is Constraint-Driven

- Synthesis process takes some time

- Synthesis time is a function of the target critical path
  - Clock period: 10nsec ⟹ synthesis time: 10 minutes
  - Clock period: 5 nsec ⟹ synthesis time: 40 minutes

Synthesis is Constraint-Driven

- The designer guides the synthesis tool by providing **design constraints:**
  - Timing requirements (max. expected clock frequency)
  - Area requirements
  - Maximum power consumption

- The synthesis tool uses this information and tries to generate the smallest possible design that will satisfy the timing requirements

- Without any constraints specified, the synthesis tool will generate a non-optimal netlist, which might not satisfy the designer's requirements

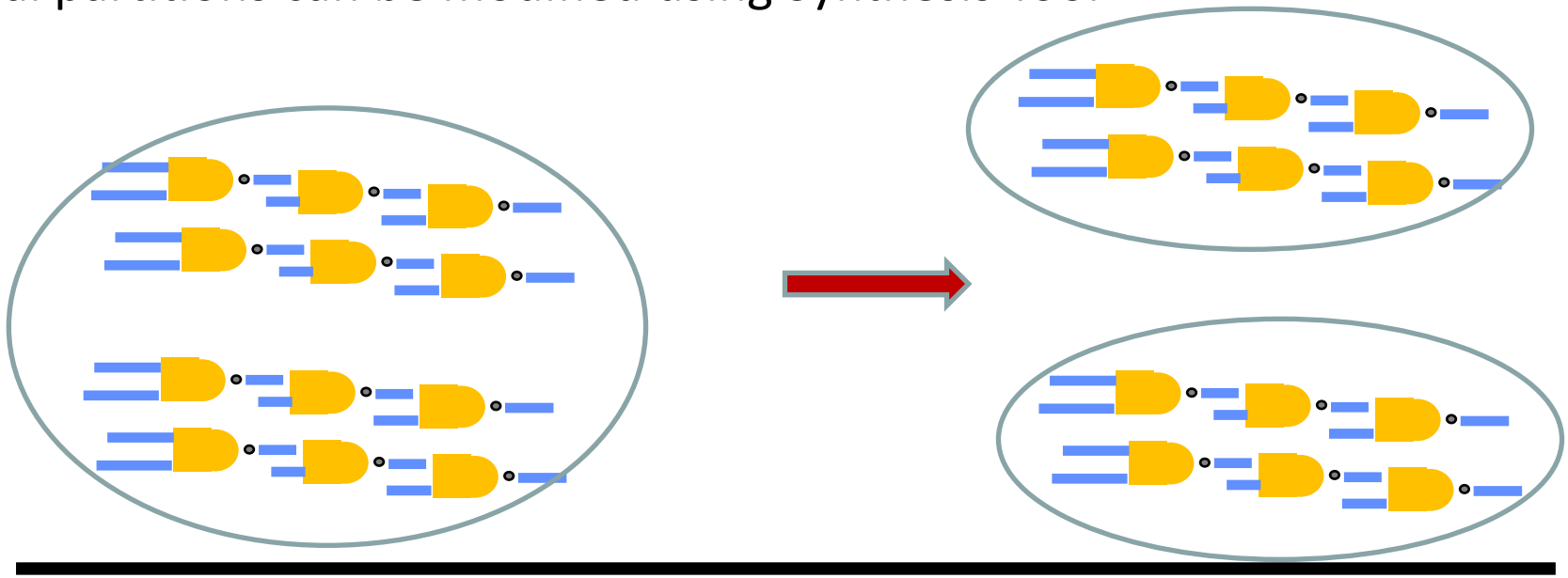
Synthesis Tools

- Commercial Synthesis Tools:

Vendor Name	Product Name	Platform
Altera	Quartus II	FPGA
Xilinx	ISE	FPGA
Mentor Graphics	Modelsim, Precision	FPGA/ASIC
Synopsys	Design Compiler	ASIC
Synplicity	Synplify	ASIC
Cadence	Ambit	ASIC

Divide and Conquer for Optimal Synthesis

- To achieve the best synthesis result, the design is better to be partitioned into smaller parts.
- **Partitioning:** the process of dividing complex designs into smaller parts
- Ideally, all partitions would be planned prior to writing any HDL:
  - Initial partitions are defined by the HDL
  - Initial partitions can be modified using Synthesis Tool



Partitioning

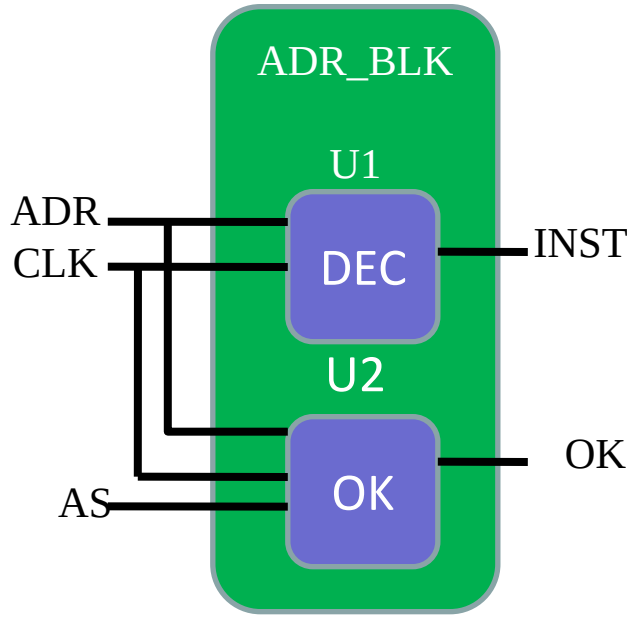
- Partition a design into different modules based on the functionality
  - Pros:
    - Separation of the cores that have different functionality
    - More manageability of smaller modules
    - Easier managements of a design implementation by a team
    - Focus to write optimized HDL code for each module
    - Possibility of reusing smaller IPs/Block in other designs
  - Cons:
    - Routing congestion or increased die size due to more signaling b/w modules

- Not too small not too large modules (~10Kgates)

Partitioning in Verilog

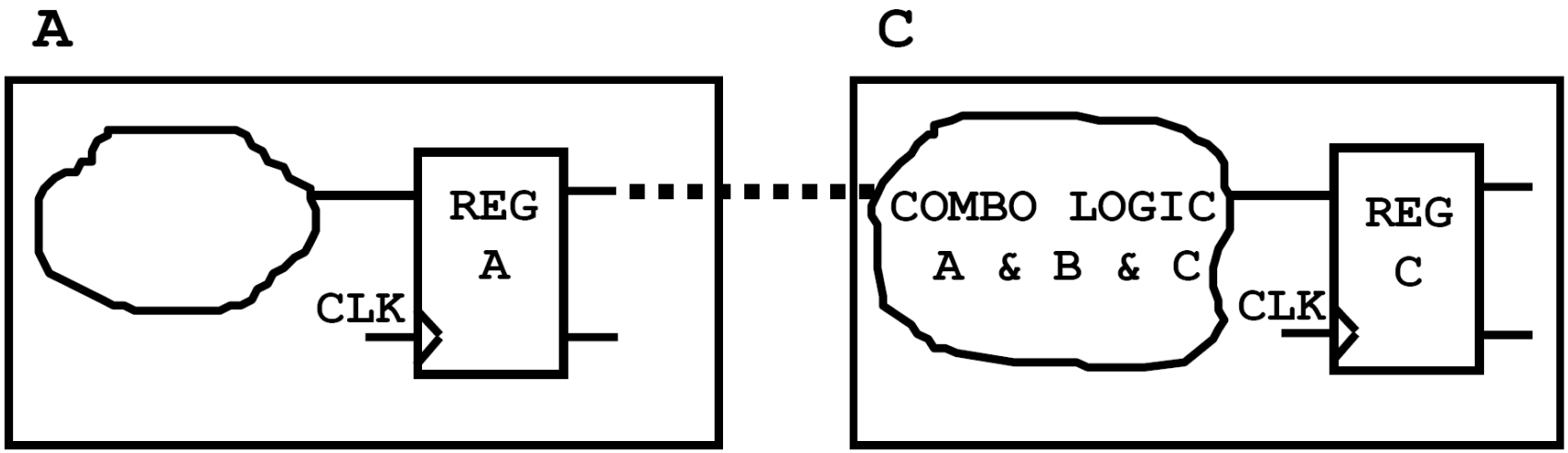
- module statement defines hierarchical blocks (partitions):
  - Instantiation of an entity or module creates a new level of hierarchy
- Inference of Arithmetic Circuits (+, -, *) can create a new level of hierarchy
- Always statements do not create hierarchy

```
module ADR_BLK (….
DEC U1 (ADR, CLK, INST);
OK  U2 (ADR, CLK, AS, OK);
endmodule;
```



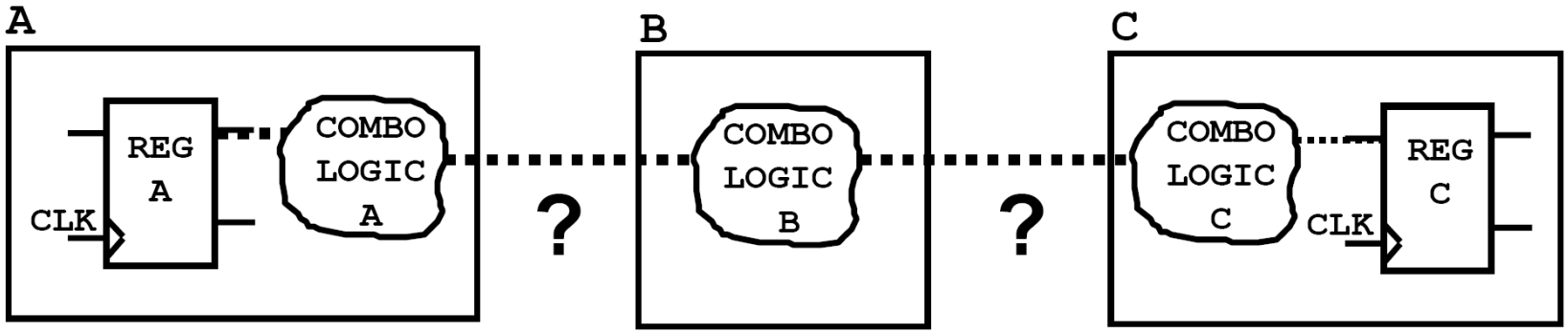
Good Partitioning (Partition at Register Boundaries)

- Try to design so the hierarchy boundaries follow register outputs.
- Related combinational logics in the middle are merged into the same block
  - Combinational optimization techniques can still be fully exploited

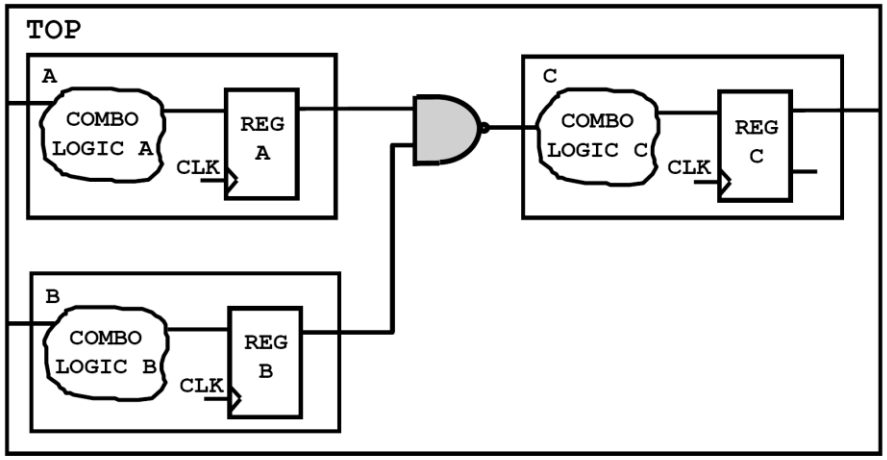


Poor Partitioning (Partition at Combinational Logic)

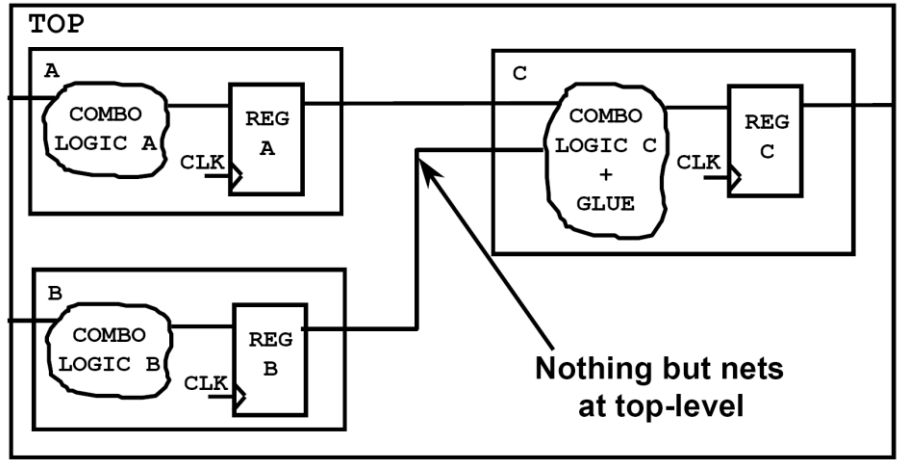
- Try not to break the Comb. Logics into several hierarchies
- Synthesis tool must preserve port definitions
- Logic optimization does not cross block boundaries
  - Adjacent blocks of combinational logic cannot be merged
- Path from REG A to REG C may be larger and slower than necessary!



Good Partitioning (Avoid Glue Logic)



Poor Partitioning



Good Partitioning

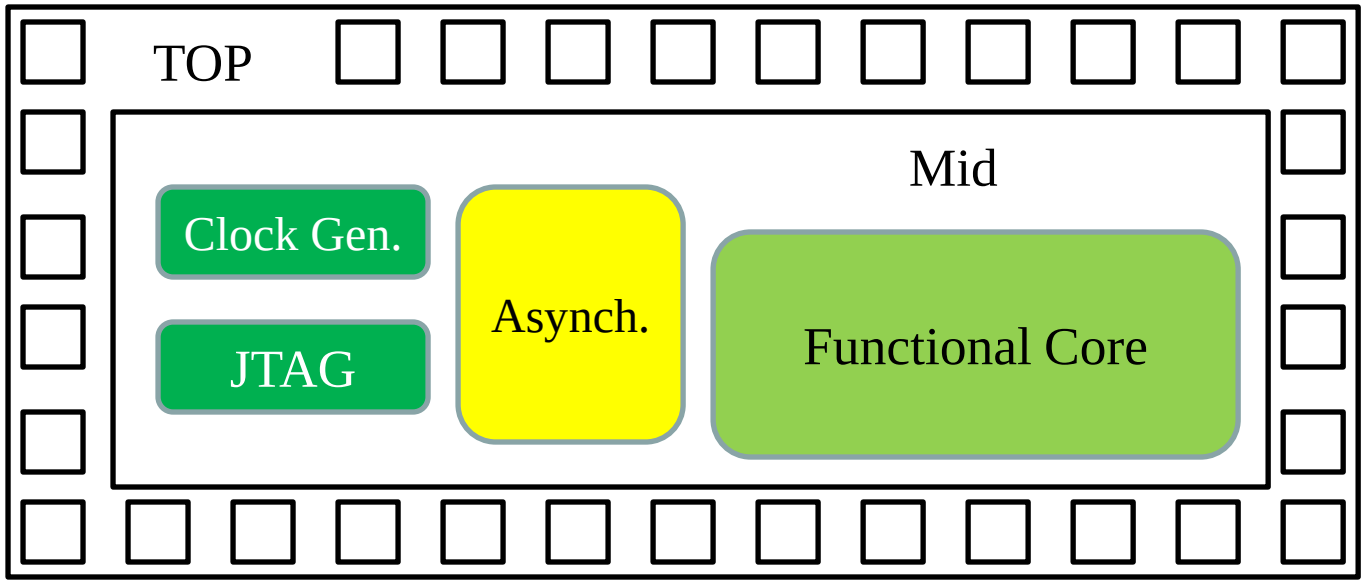
- The NAND gate at the top-level serves only to “glue” the instantiated cells.
- Optimization is limited because the glue logic cannot be “absorbed”
- Additional compile needed at top-level

- The glue logic can now be optimized with other logic
- Top-level design is only a structural netlist, it does not need to be compiled

Good Partitioning

در جزءبندی، Core Logic، کلاکها، پدها، JTAG را از هم جدا کنید.

سلسله مراتب طرح
- Top_level
- Mid_level
- Functional Core

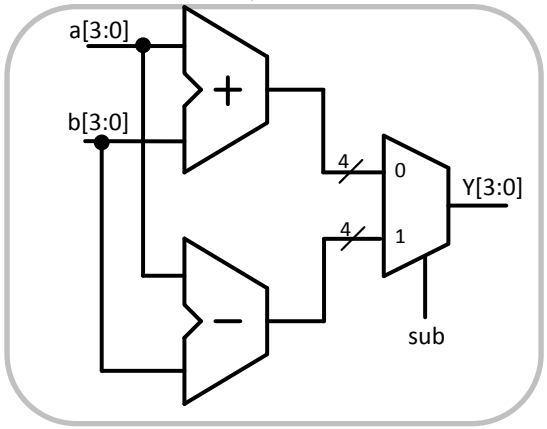


HDL for Synthesis

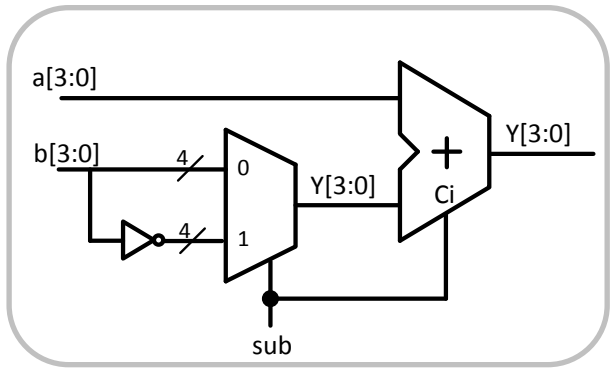
- "Bad" HDL code does not allow efficient optimization during synthesis
  - Garbage in, garbage out!
- Logic synthesizer doesn't do magic! designer has to take some responsibility in coding.

❖ Example:

```
input sub;
input [3:0] a,b;
output [3:0] y;
assign y = sub ? (a-b) : (a+b)
```




```
input sub;
input [3:0] a,b;
output [3:0] y;
wire [3:0] tmp;
assign tmp = sub ? ~b : b;
assign y = a + tmp + {3'b0,sub}
```

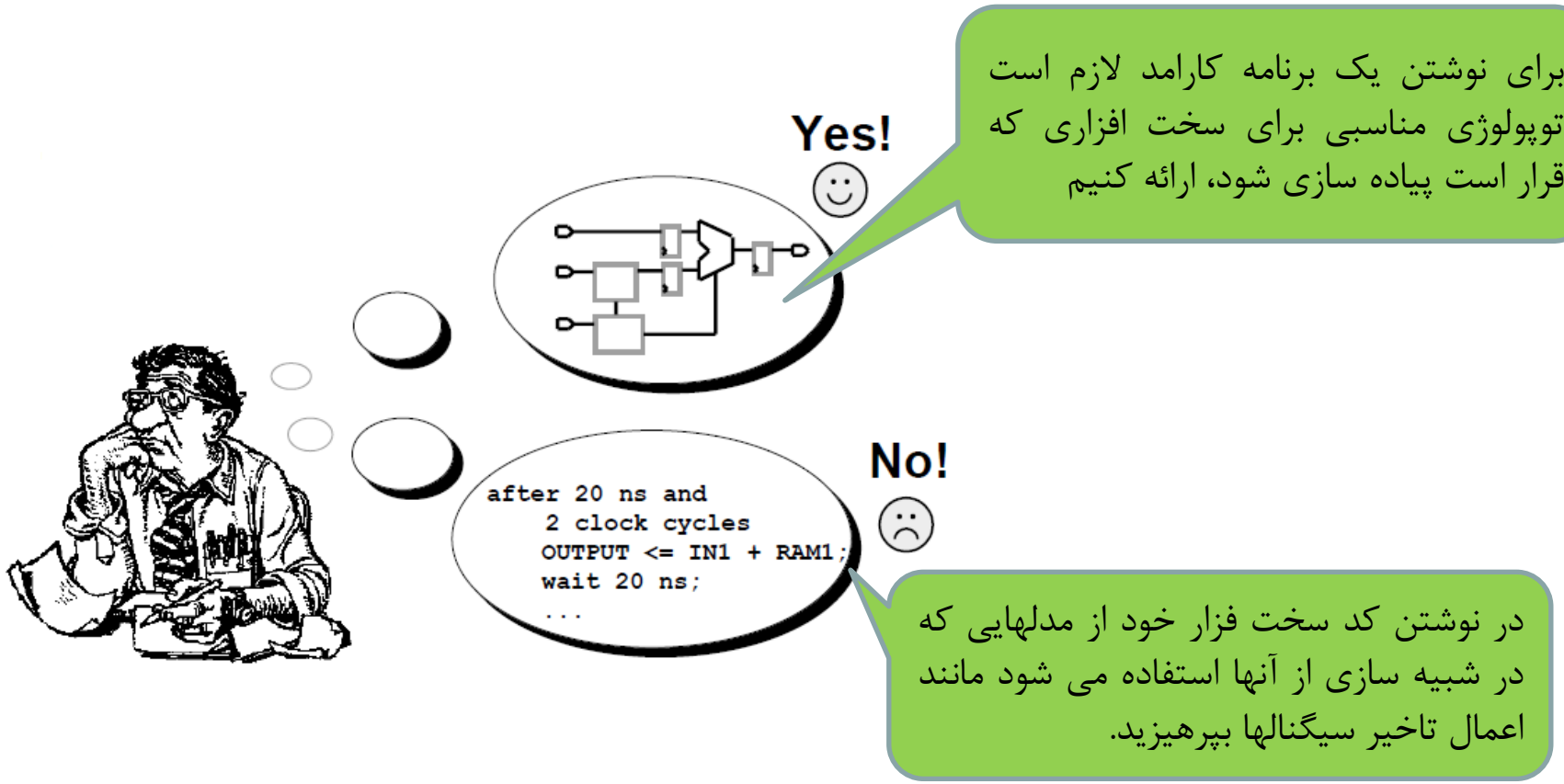


More efficient

HDL for Synthesis (General Guidelines)

- **Think Hardware:**
  - Write HDL hardware descriptions
    - Think of the *topology* implied by the code
  - Do not write HDL simulation models
    - No explicit delays
    - No file I/O
- **Think RTL:**
  - Writing in an RTL coding style means describing:
    - Register architecture
    - Circuit topology
    - Functionality between registers
  - Synthesis tool optimizes logic between registers:
    - It does not optimize the register placement

HDL for Synthesis (General Guidelines)



برای نوشتن یک برنامه کارامد لازم است توپولوژی مناسبی برای سخت افزاری که قرار است پیاده سازی شود، ارائه کنیم

در نوشتن کد سخت فزار خود از مدلهایی که در شبیه سازی از آنها استفاده می شود مانند اعمال تاخیر سیگنالها بپرهیزید.

Synthesizable Constructs

- Not all Verilog constructs are synthesizable because:
  - Does not make sense in hardware (e.g. $display, initial block)
  - Not possible to achieve (e.g. delay control like #10)
  - Not support by design flow (e.g. use of tran in P&R)
  - Too difficult or too abstract for the synthesis software (e.g. A / B)

Synthesizable Constructs

- Ports (input, output, inout)
- Parameter
- module
- wire, reg, tri
- function, task
- always, if, else, case
- assign
- for, while

Non-Synthesizable Constructs

Non-Synthesizable Constructs

- Initial (used only in testbenches)
- real (real type data type)
- time (time data type)
- assign for reg data types
- comparison to X and Z are ignored (e.g., a == 1'bz)
- delays "#" are ignored by synthesis tools as if it is not there

HDL for Synthesis (Priority logic)

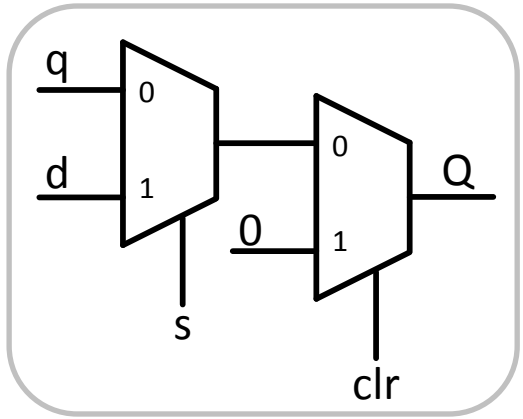
- The order in which assignments are written in an always block may affect the logic that is synthesized.

❖ Example:

```
always @ (s, clr, d, q)
  begin
    Q = q;
    if (s)   Q = d;
    if (clr) Q = 0;
  end
```

Priority

s	clr	Q
0	0	q
0	1	0
1	0	d
1	1	0

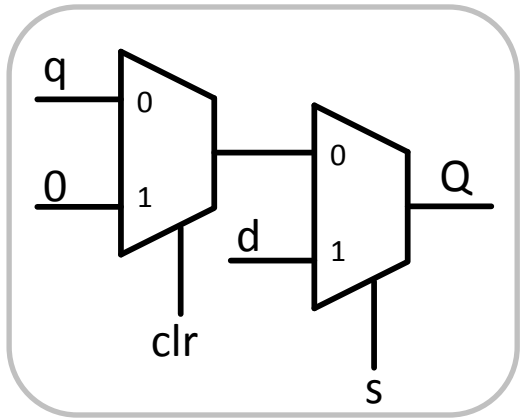


Different

```
always @ (s, clr, d, q)
  begin
    Q = q;
    if (clr) Q = 0;
    if (s)   Q = d;
  end
```

Priority

s	clr	Q
0	0	q
0	1	0
1	0	d
1	1	d



Non of the above infer latch, why?

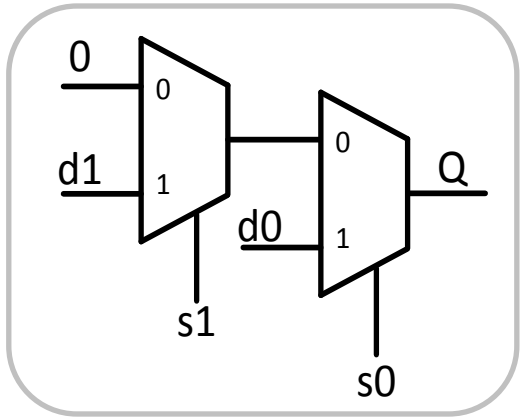
HDL for Synthesis (Priority logic)

- The order in which assignments are written in an always block may affect the logic that is synthesized.

❖ Example:

Priority

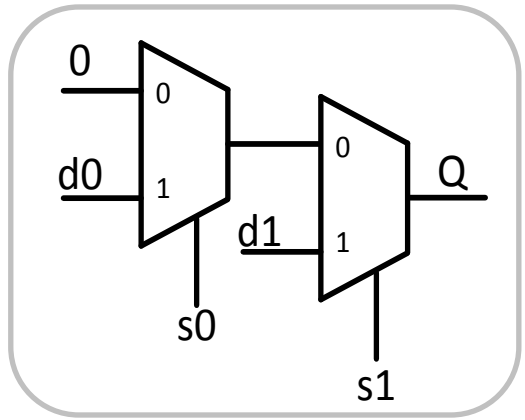
```
always @ (s0,s1, d0, d1)
  begin
    Q = 0;
    if (s0)   Q = d0;
    else if (s1) Q = d1;
  end
```



Different

Priority

```
always @ (s0,s1, d0, d1)
  begin
    Q = 0;
    if (s1)   Q = d1;
    else if (s0) Q = d0;
  end
```



Non of the above infer latch, why?

HDL for Synthesis (Priority logic) : Poor Coding

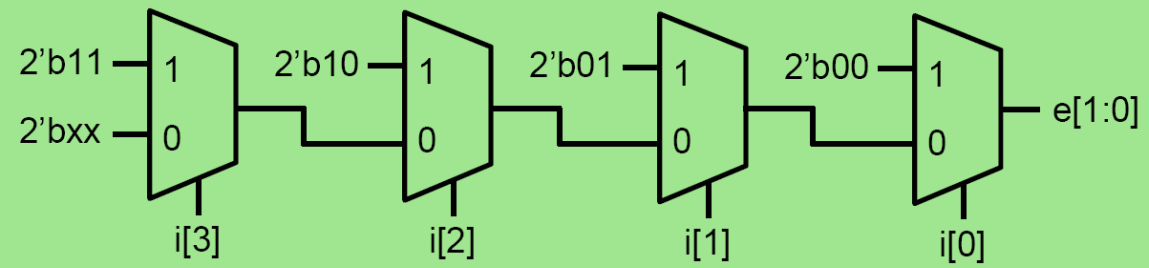
Intent: **if more than one input is 1, the result is a don't-care.**

I_3 I_2 I_1 I_0	E_1 E_0
0 0 0 1	0 0
0 0 1 0	0 1
0 1 0 0	1 0
1 0 0 0	1 1
all others	X X

Code: **if i[0] is 1, the result is 00 regardless of the other inputs.**
i[0] takes the highest priority.

1st Priority → `if (i[0]) e = 2'b00;`
2nd Priority → `else if (i[1]) e = 2'b01;`
3rd Priority → `else if (i[2]) e = 2'b10;`
4th Priority → `else if (i[3]) e = 2'b11;`
`else e = 2'bxx;`
`end`

Inferred Result:



- **`if-else` and `case` statements are interpreted very literally! Beware of unintended priority logic.**

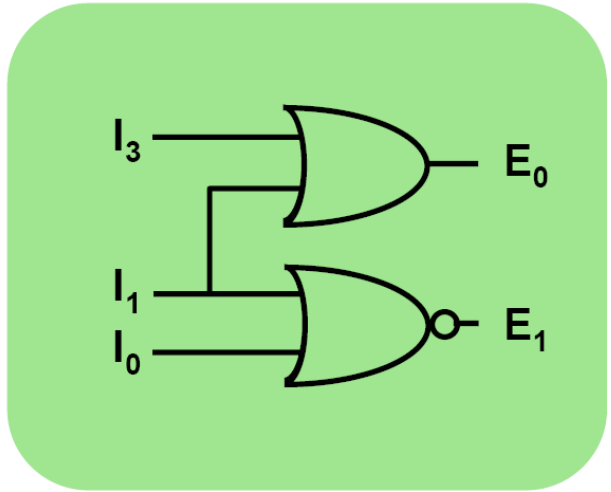
HDL for Synthesis (Priority logic) : Good Coding

- Make sure that `if-else` and `case` statements are *parallel*
  - If mutually exclusive conditions are chosen for each branch...
  - ...then synthesis tool can generate a simpler circuit that evaluates the branches in parallel

Parallel Code:

```
module binary_encoder(i, e);
  input [3:0] i;
  output [1:0] e;
  reg e;

  always @(i)
  begin
    if (i == 4'b0001) e = 2'b00;
    else if (i == 4'b0010) e = 2'b01;
    else if (i == 4'b0100) e = 2'b10;
    else if (i == 4'b1000) e = 2'b11;
    else e = 2'bxx;
  end
endmodule
```

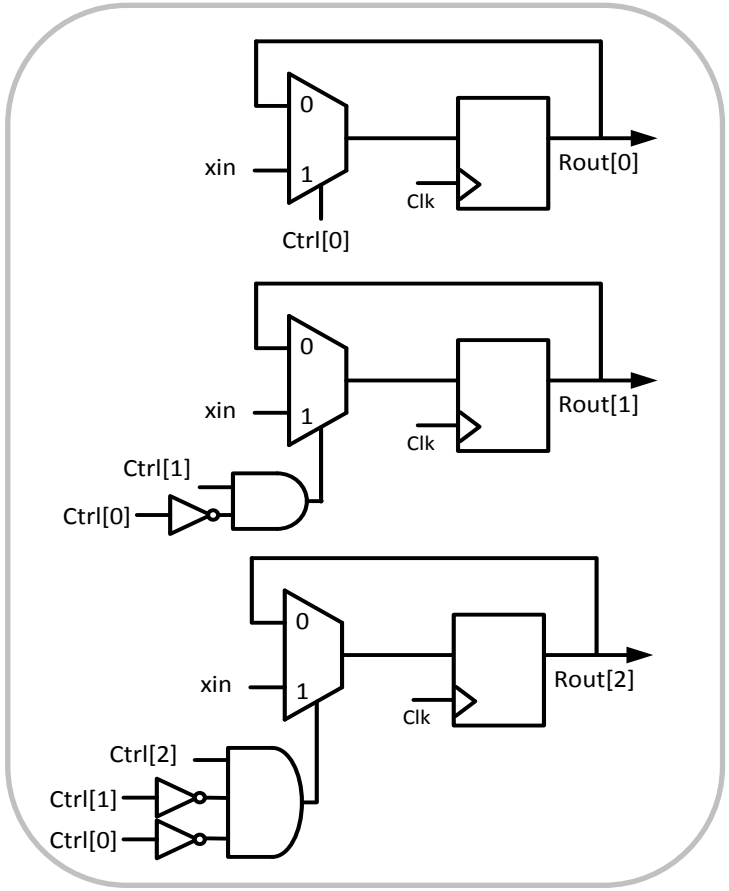
Minimized Result:



Flatten Logic Structure

- Applies to the logic that is chained due to the priority encoding
- Synthesis and layout tools are smart enough to duplicate logic to reduce fan-out, but they are not smart enough to break up logic structures that are coded in a serial Fashion

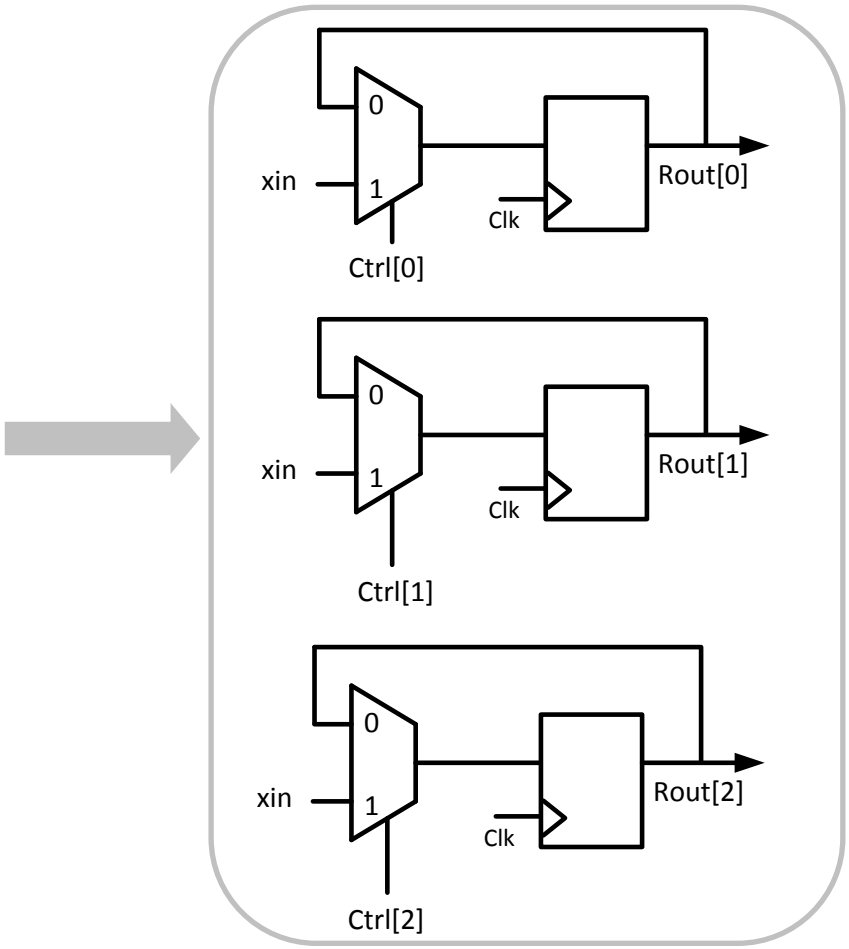
```
module regwrite(
output reg [2:0] Rout,
input clk, in,
input [2:0] Ctrl);
always @ (posedge clk)
if       (Ctrl[0]) Rout[0] <= in;
else if  (Ctrl[1]) Rout[1] <= in;
else if  (Ctrl[2]) Rout[2] <= in;
endmodule
```



Flatten Logic Structure

- Flatten the logic (when conditions are mutually exclusive)
  - No priority logic (each register is controlled independently)
  - Less logic delay

```
module regwrite(
output reg [2:0] rout,
input clk, in,
input [2:0] ctrl);
always @(posedge clk)
 if(ctrl[0]) rout[0] <= in;
 if (ctrl[1]) rout[1] <= in;
 if (ctrl[2]) rout[2] <= in;
endmodule
```



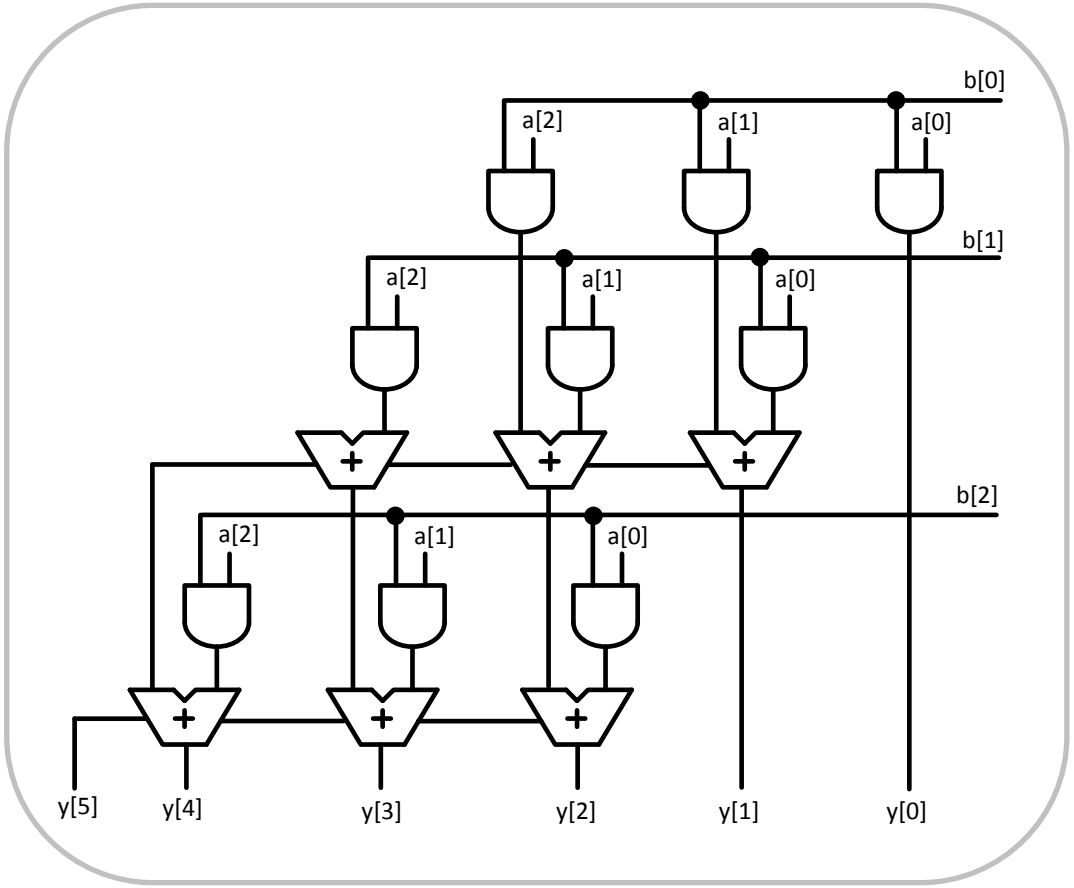
HDL for Synthesis

- It is possible to derive gate-level implementation of "+", "-", "*" operations and write them in Verilog. However, it is better to use just "+, "-", "*" and let the synthesis tool to decide which block to use to meet the performance

❖ Example:

```
module multi3x3 (a, b, y)
input [2:0] a, b;
output [5:0] y;
assign y = a * b;
```

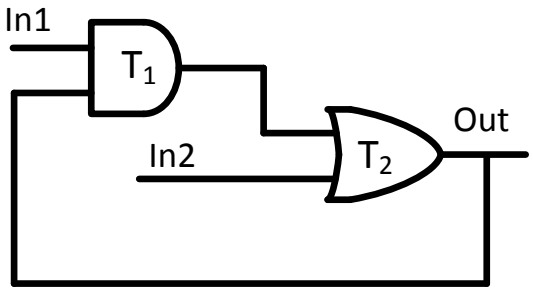
More efficient



- Unsigned operations
- Performance:
  - Power
  - Area
  - Speed

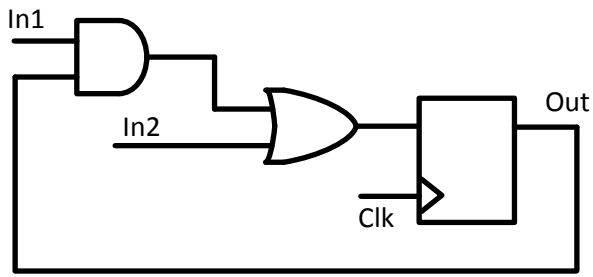
Design for Synthesis (No Timing Loop)

- Do not use timing loops in the circuit
- **Timing loop:** when an output of a combinational logic loops back to its input
  - Results in oscillation
  - Complicated timing analysis
  - Timing glitches
- **Solution:** Add a flip-flop on the feedback path




```
always @ (*)
    Out = (In1 & Out) | In2;
```

Oscillates with $f=1/(T_1+T_2)$ (Not desired)




```
always @ (posedge gated)
    Out <= (In1 & Out) | In2;
```

No Oscillation (Desired)

Latch Inference in Combinational Logic

- When realizing combinational logic with always block using if-else or case constructs care has to be taken to avoid latch inference after synthesis
- The latch is inferred when "incomplete" if-else or case statements are declared
- This latch is "unwanted" as the logic is combinational not sequential
- To avoid latch inference make sure to specify all possible cases "explicitly"
- Two practical approaches to avoid latch inference:
  - **For if-else construct:**
    1. Initialize the variable before the if-else construct
    2. Use else to explicitly list all possible cases
  - **For case constructs:**
    1. Use default to make sure no case is missed!
- If there is some logic path through the always block that does not assign a value to the output a latch is inferred
- Do NOT let it up to the synthesis tool to act in unspecified cases and do specify all cases explicitly.

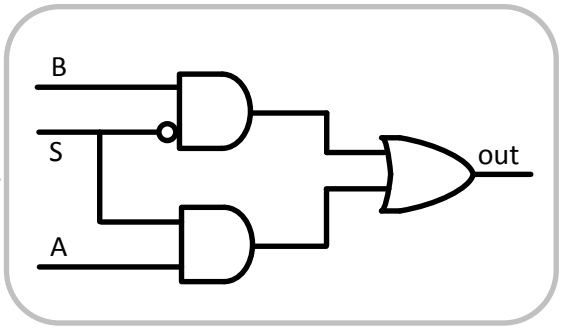
Avoid Latch Inference in If-else Statements

❖Example:

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
end
endmodule
```

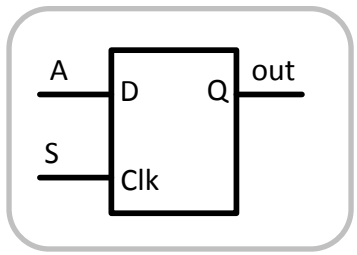
1

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
Begin
   out = B;
   if (S==1)
       out = A;
end
endmodule
```



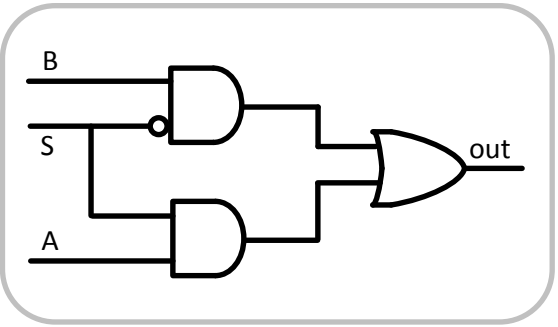
No Latch

2



Latch Inference

```
module DUT (A, B, S, out);
input A, B, S;
output reg out;
always @(*)
begin
   if (S==1)
       out = A;
   else
       out =B;
end
endmodule
```



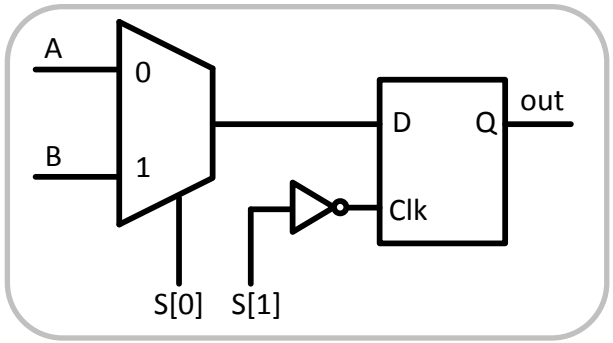
No Latch

Avoid Latch Inference in Case Statements

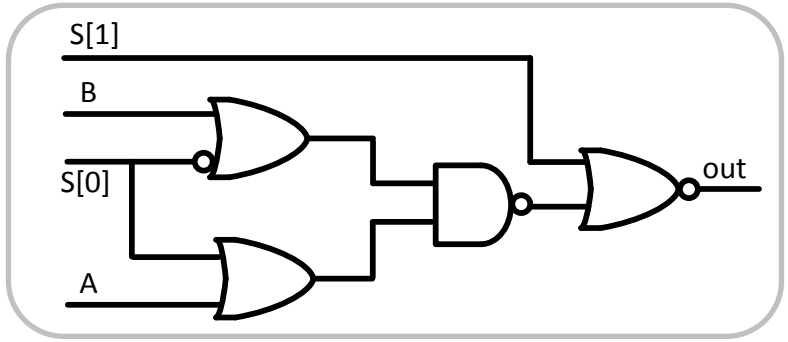
❖Example:

```
module DUT (A, B, S, out);
input A, B;
Input [1:0] S;
output reg out;

always @(A, B, S)
begin
   case (S)
      2'b00: out = A;
      2'b01: out = B;
   endcase
end
endmodule
```

```
module DUT (A, B, S, out);
input A, B;
Input [1:0] S;
output reg out;
always @(A, B, S)
begin
   case (S)
      2'b00: out = A;
      2'b01: out = B;
      default: out = 1'b0;
   endcase
end
endmodule
```



Latch Inference



No Latch

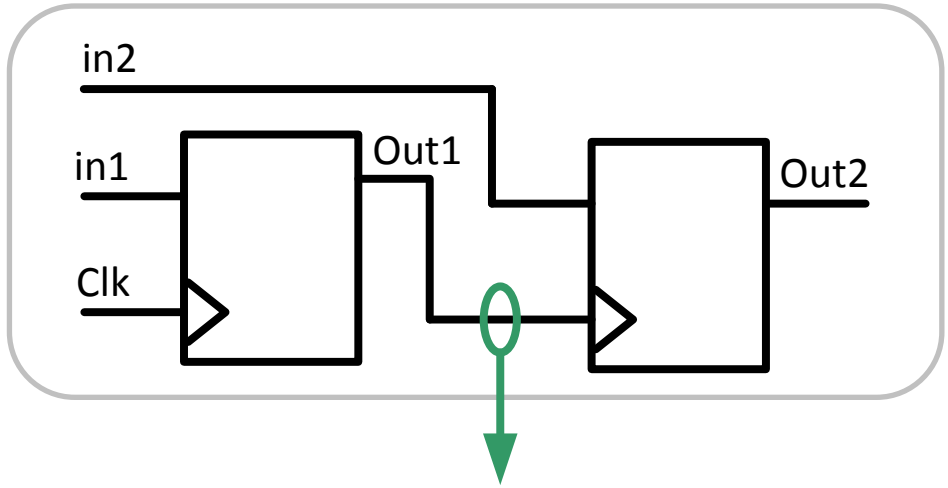
Clock

- Clock is the most important signal in the design (golden)
- Why is it different from other signals?
  - It is a global signal, i.e., it is routed across all modules in the design
- Treat clock as a golden signal
  - No buffering should be done on clock during coding and synthesis
    - Clock buffering to fix the clock skew is done during clock tree synthesis (part of APR in ASIC flow, which is done automatically)
  - No “clock gating”:
    - Clock should be directly connected to flip-flops without any logic gating
    - Otherwise, it results in clock skew in the design (undesired!)

Clock (No Internally Generated Clock)

- Do not use internally generated clocks

```
always @ (posedge Clk)
        Out1 <= In1;

always @ (posedge Out1)
        Out2 <= In2;
```

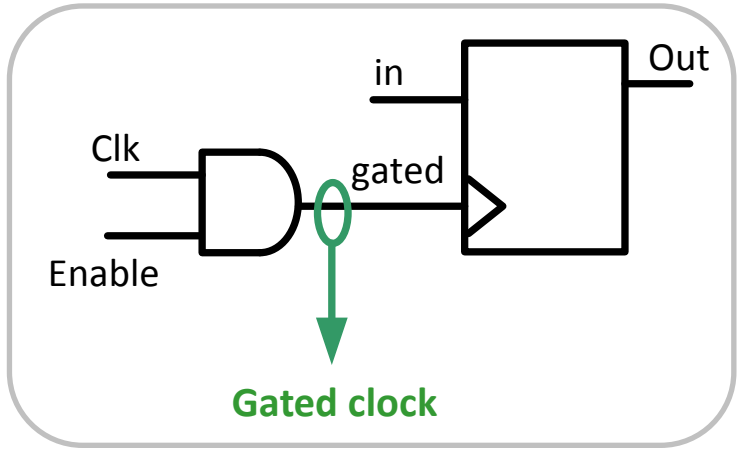


- Complicates the timing analysis
  - Setup time
  - Hold time
- Difficult to deal with during synthesis

Clock (No Gating)

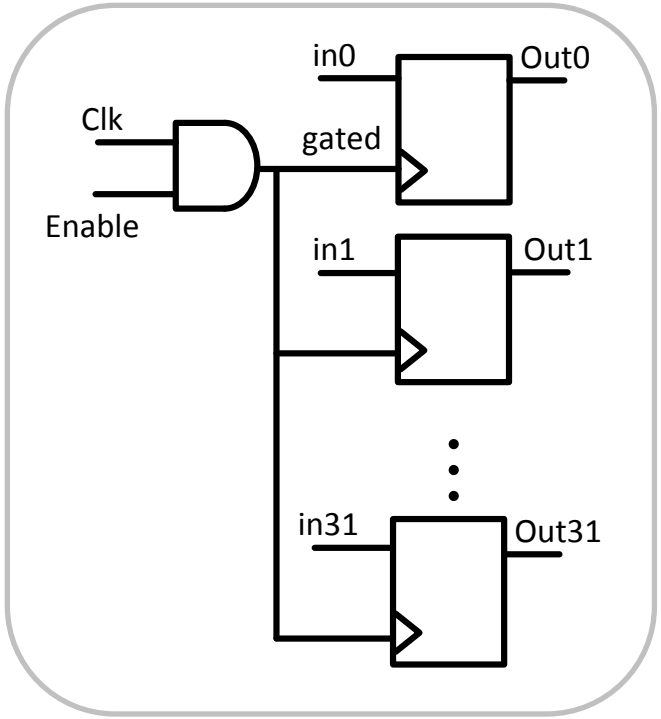
- **Gated clock**: clock that is enabled by an enable signal
- Applications:
  - Used for power saving to switch off part of the chip in fraction of time
- Avoid clock gating as much as possible
  - Because results in clock skew
  - Not a golden signal anymore!

```
assign  gated = Clk & Enable

always @ (posedge gated)
            Out <= In;
```



Clock (No Gating)

- If clock gating is used avoid large fan-outs

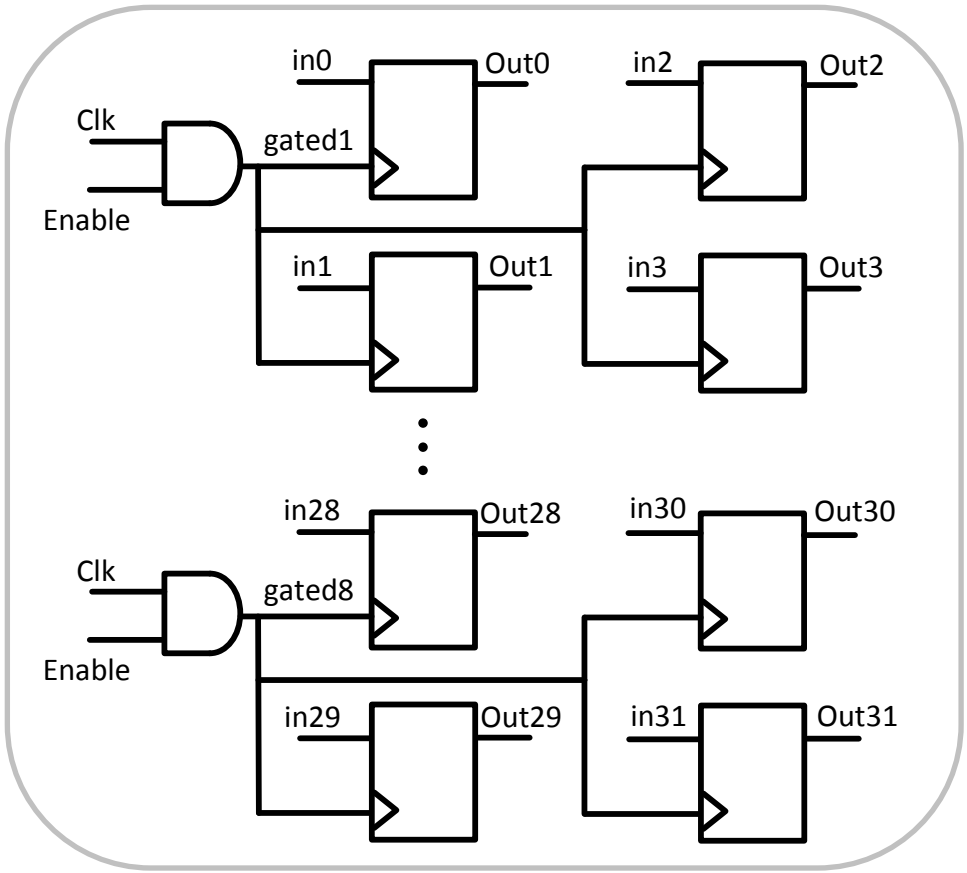



```
assign gated = Clk & Enable;

always @ (posedge gated)
        Out[31:0] <= In[31:0];
```

- Large fan-out (deriving 32 flip flops)
- Large delay ⇨ high clock skew

Clock (No Gating)

- Low Fan-out Alternative:

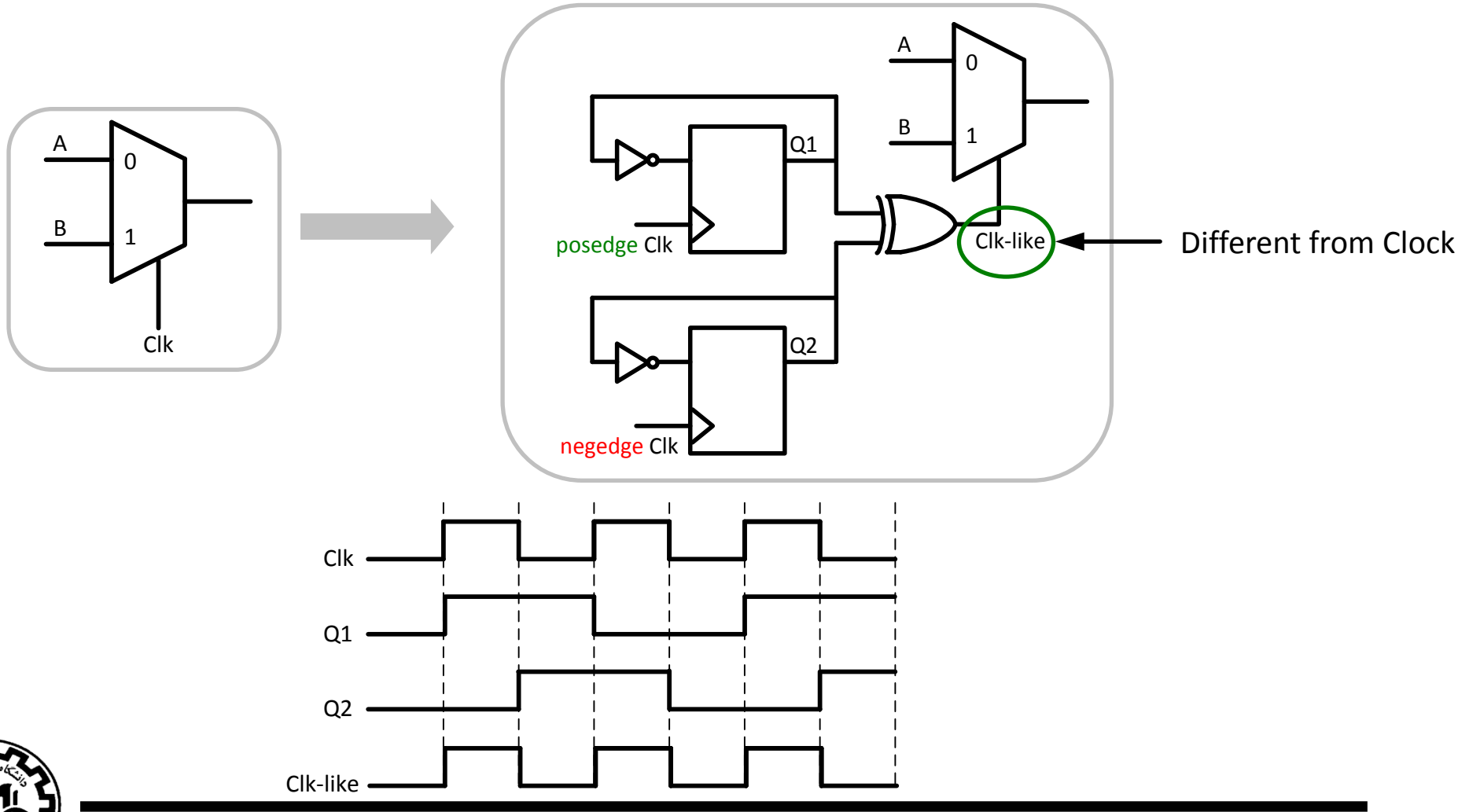


Explicit "and" instantiation

```
and U1 (gated1, Clk, Enable);
and U2 (gated2, Clk, Enable);
   ⋮
and U8 (gated8, Clk, Enable);


always @ (posedge gated1)
        Out[3:0] <= In[3:0];
always @ (posedge gated2)
        Out[7:4] <= In[7:4];
   ⋮
always @ (posedge gated8)
        Out[31:28] <= In[31:28];
```

Clock

- Do not use clock as an input or selector (results in an inefficient clock tree)

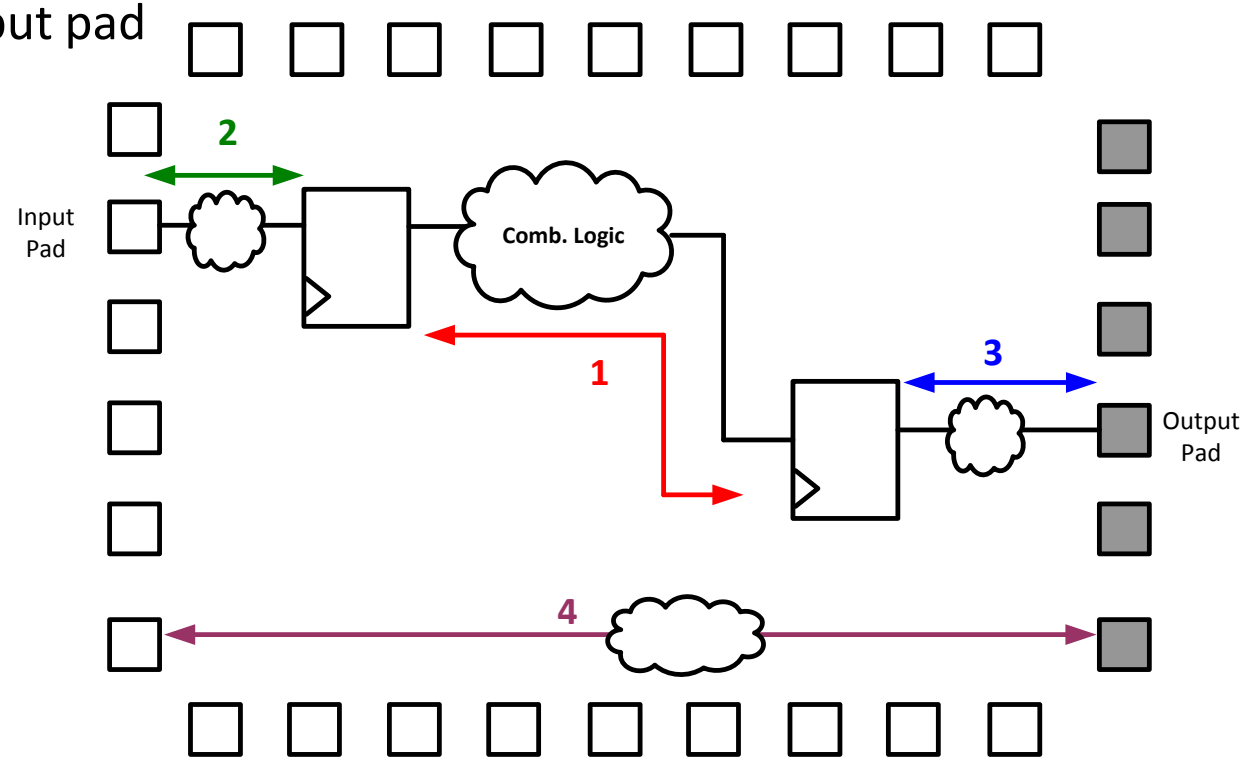


Architectural Techniques : Critical Path

- Critical path in any design is the longest path between
  1. Any two internal latches/flip-flops
  2. An input pad and an internal latch
  3. An internal latch and an output pad
  4. An input pad and an output pad

Use FFs right after/before input/out pads to avoid the last three cases (off-chip and packaging delay)

The maximum delay between any two sequential elements in a design will determine the max clock speed



Digital Design Metrics

- Three primary physical characteristics of a digital design:
  - **Speed**
    - Throughput
    - Latency
    - Timing
  - **Area**
  - **Power**

Digital Design Metrics

❑ Speed

- **Throughput** :
  - The amount of data that is processed per clock cycle (bits per second)

- **Latency**
  - The time between data input and processed data output (clock cycle)

- **Timing**
  - The logic delays between sequential elements (clock period)
  - When a design does not meet the timing it means the delay of the critical path is greater than the target clock period

Maximum Clock Frequency: Critical Path

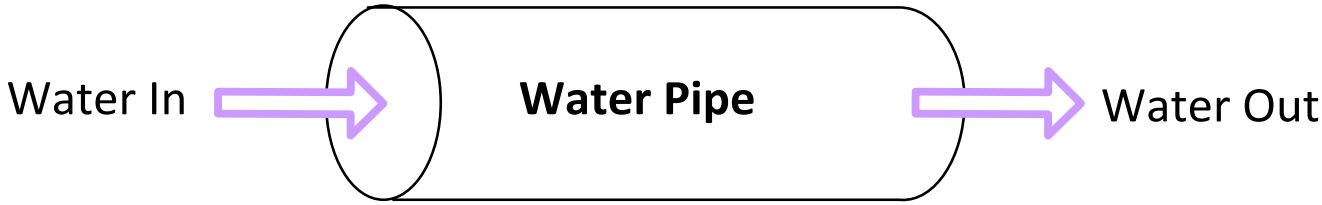
- Maximum Clock Frequency:

$$F_{max} = \frac{1}{T_{clk-q} + T_{logic} + T_{setup} + T_{routing} - T_{skew}}$$

- $T_{clk\text{-}q}$: time from clock arrival until data arrives at Q
- T_{logic} : propagation delay through logic between flip-flops
- $T_{routing}$: routing delay between flip-flops
- T_{setup} : minimum time data must arrive at D before the next rising edge of clock
- T_{skew} : propagation delay of clock between the launch flip-flop and the capture flip-flop.

Pipelining (to Improve Throughput)

- **Pipelining:**
  - Comes from the idea of a water pipe: continue sending water without waiting the water in the pipe to be out
  - Used to reduce the critical path of the design



- **Advantageous**:
  - Reduction in the critical path
  - Higher throughput (number of computed results in a give time)
  - Increases the clock speed (or sampling speed)
  - Reduces the power consumption at same speed

Architectural Techniques :Pipelining

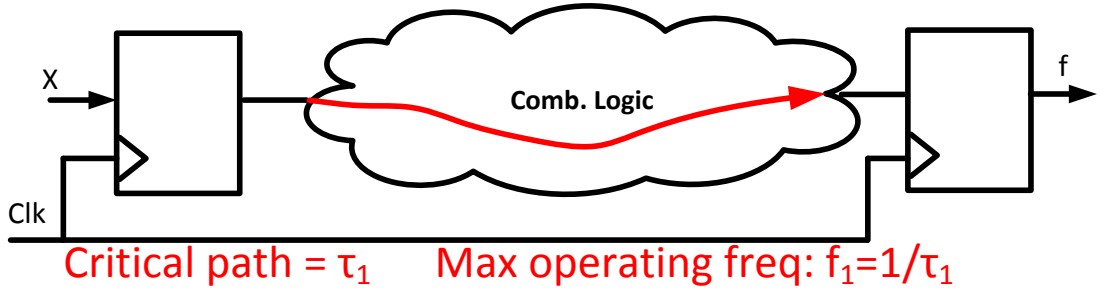
❑ Pipelining:

- ➢ Very similar to the assembly line in the auto industry

- ➢ The beauty of a pipelined design is that new data can begin processing before the prior data has finished, much like cars are processed on an assembly line.

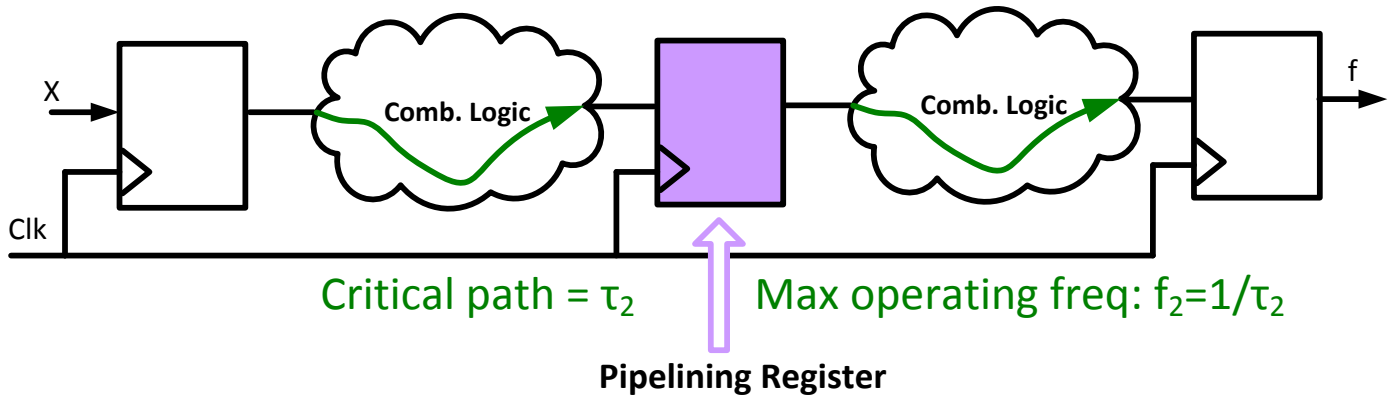
Architectural Techniques :Pipelining

- **Original System: (**Critical path = τ_1 Max operating freq: $f_1=1/\tau_1$**)**



2 cycles later

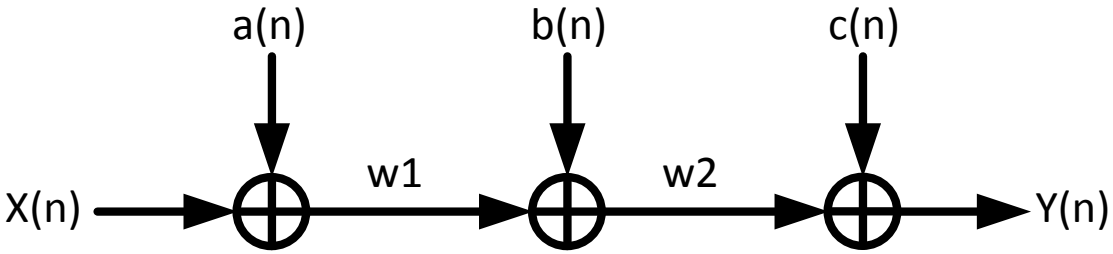
- **Pipelined version: (**Critical path = τ_2 Max operating freq: $f_2=1/\tau_2$**)**
  - Smaller Critical Path ⟹ higher throughput ($\tau_2<\tau_1$ ⟹ $f_2>f_1$)
  - Longer latency



3 cycles later

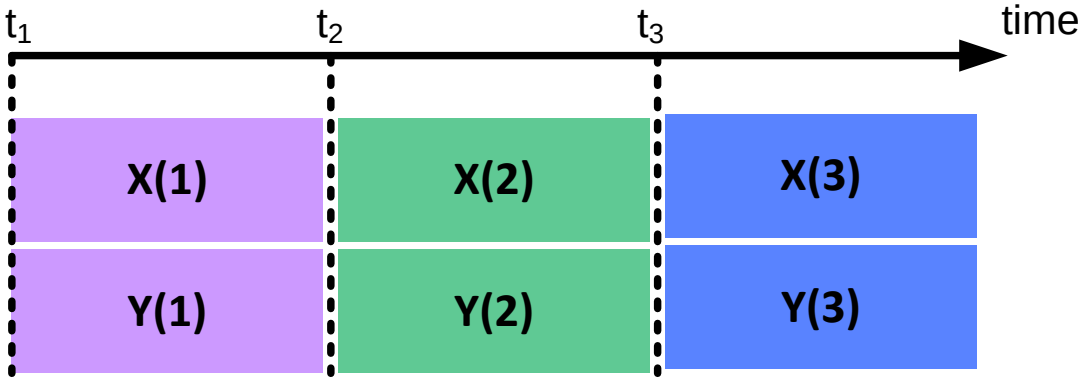
Architectural Techniques : Pipeline depth

- Pipeline depth: 0 (No Pipeline)
  - Critical path: 3 Adders



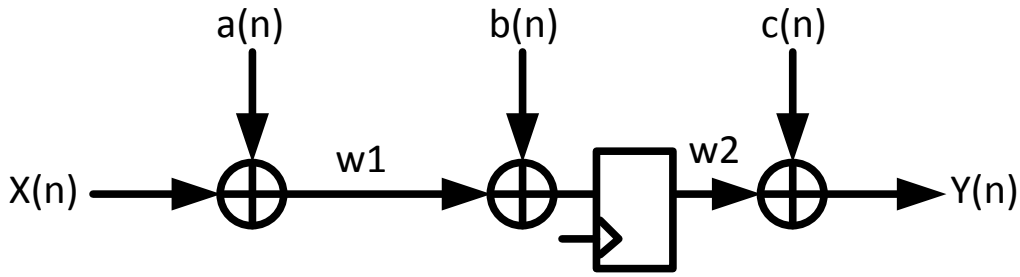

```
wire  w1, w2;
assign  w1 = X + a;
assign  w2 = w1 + b;
assign  Y = w2 + c;
```

- Latency : 0



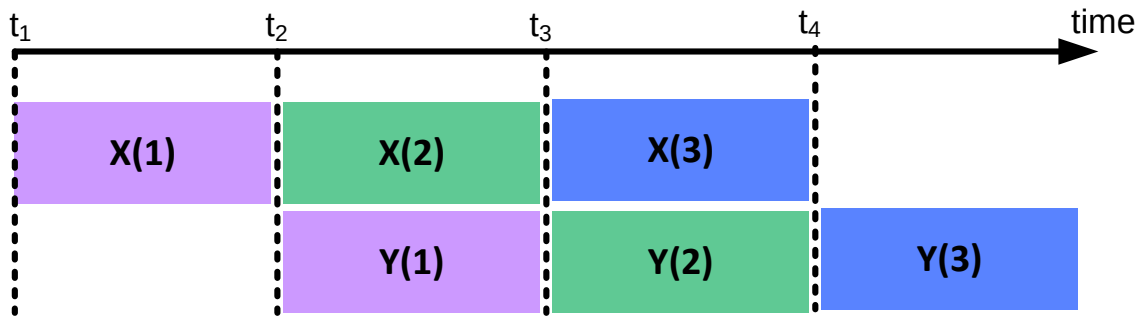
Architectural Techniques : Pipeline depth

- Pipeline depth: 1 (One Pipeline register Added)
  - Critical path: 2 Adders



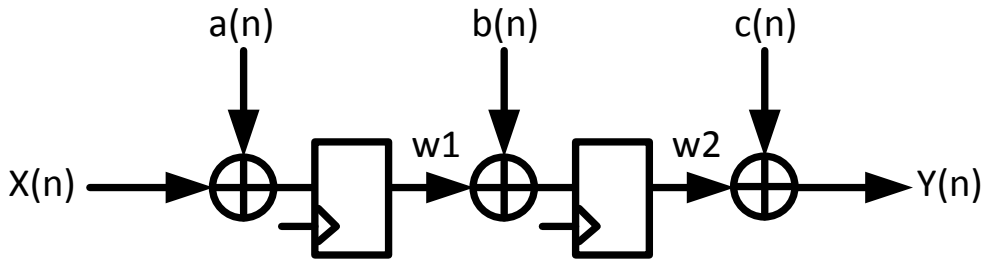

```
wire  w1;
reg   w2;
assign  w1 = X + a;
assign  Y = w2 + c;

always @(posedge Clk)
    w2 <= w1 + b;
```

- Latency : 1



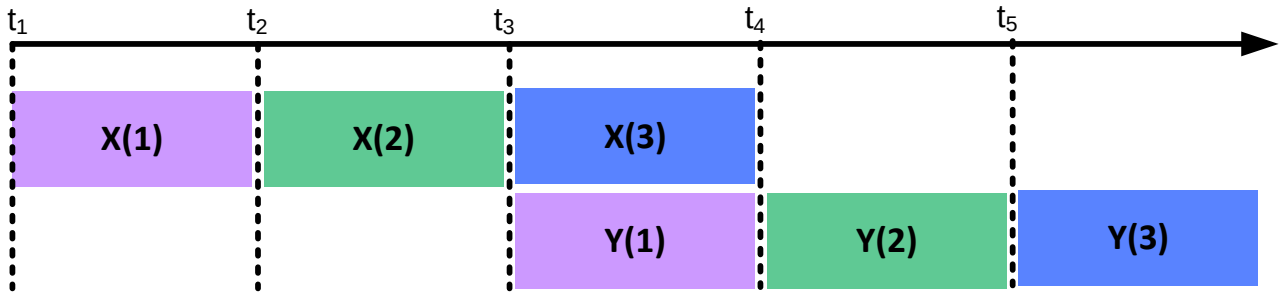
Architectural Techniques : Pipeline depth

- Pipeline depth: 2 (One Pipeline register Added)
  - Critical path: 1 Adder




```
reg   w1, w2;
assign  Y = w2 + c;

always @(posedge Clk)
 begin
    w1 <= X + a;
    w2 <= w1 + b;
 end
```

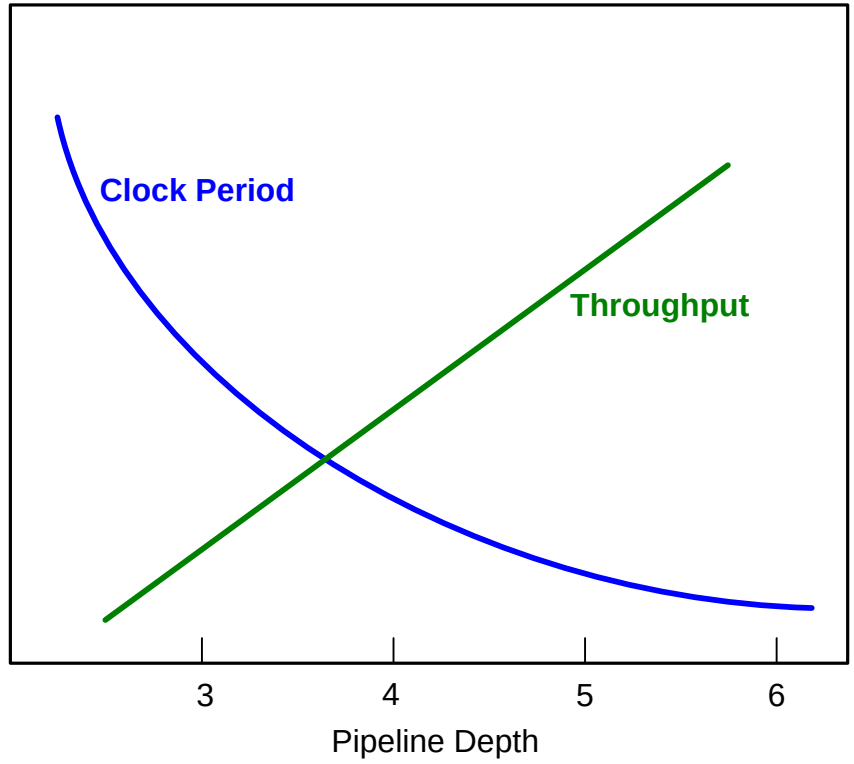
- Latency : 2



Architectural Techniques :Pipelining

- Clock period and throughput as a function of pipeline depth:
  - Clock period : $\tau_{\text{Clk}} \propto \frac{1}{n}$
  - Throughput: $\mathrm{T} \propto n$

Adding register layers improves timing by dividing the critical path into two paths of smaller delay



Architectural Techniques : Pipelining

- **General Rule:**
  - Pipelining latches can only be placed across **feed-forward cutsets** of the circuit.

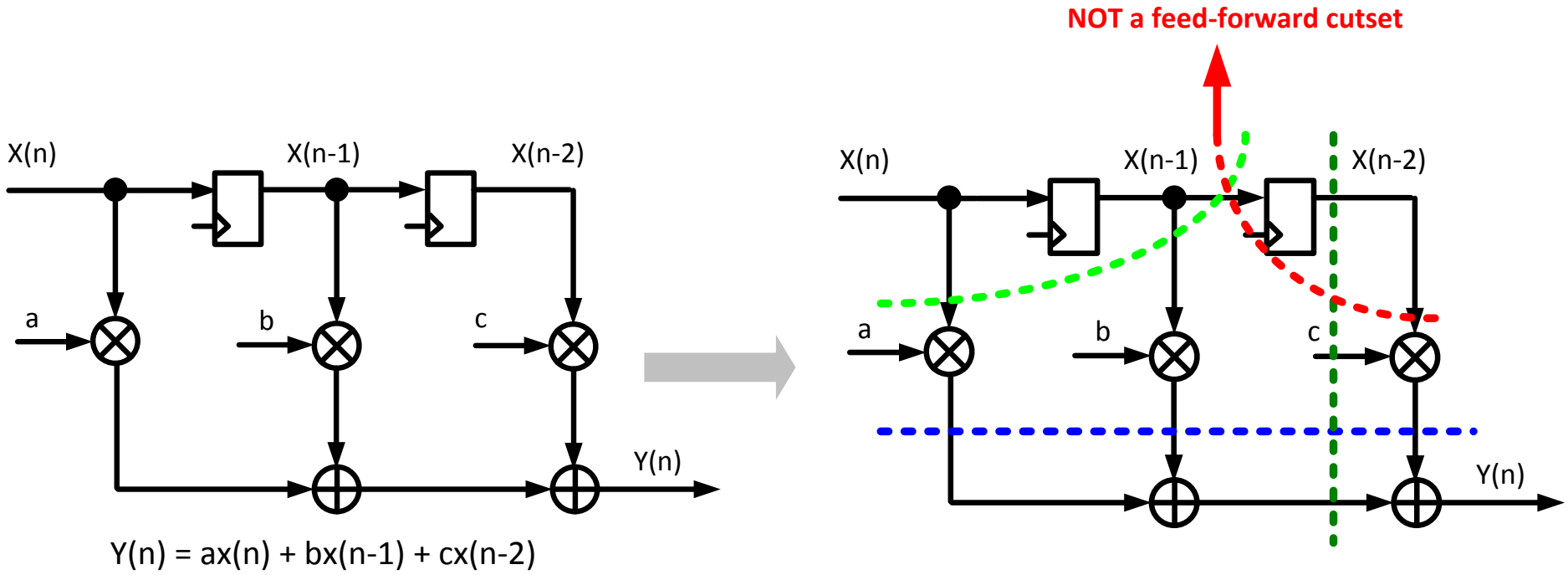
- **Cutset:**
  - A set of paths of a circuit such that if these paths are removed, the circuit becomes disjoint (i.e., two separate pieces)

- **Feed-Forward Cutset:**
  - A cutset is called feed-forward cutset if the data moves in the forward direction on all the paths of the cutset

Architectural Techniques : Pipelining

❖ **Example:**

- FIR Filter
- Three feed-forward cutsets are shown

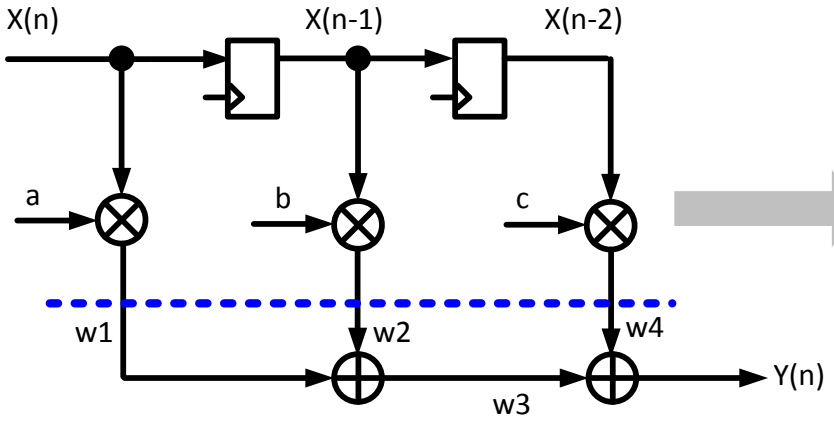


$Y(n) = ax(n) + bx(n-1) + cx(n-2)$

© M. Shabany, ASIC/FPGA Chip Design

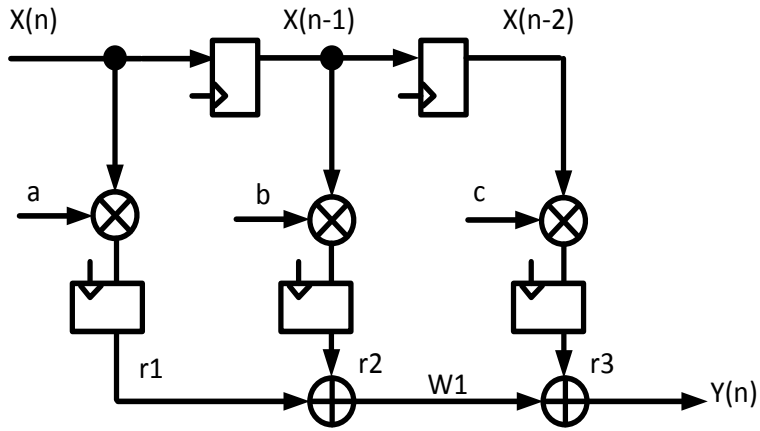
Architectural Techniques : Pipelining

Critical Path: 1M+2A



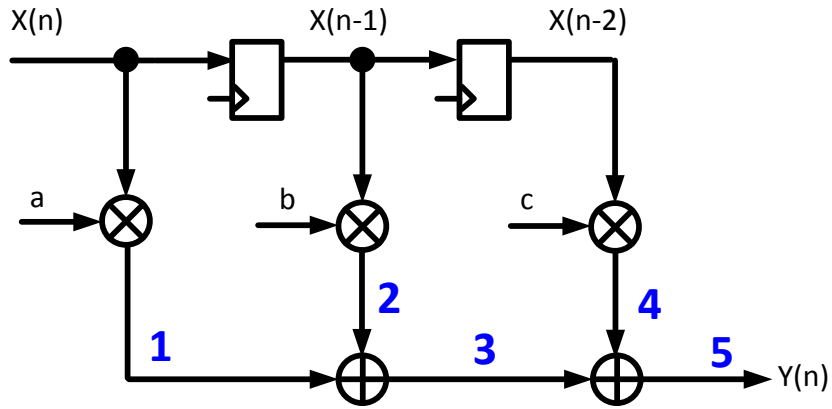

```
assign  w1 = a*Xn;
assign  w2 = b*Xn_1 ;
assign  w3 = w1 + w2;
assign  w4 = c*Xn_2;
assign  Y = w3 + w4;
always @(posedge Clk)
 begin
    Xn_1 <= Xn;
    Xn_2 <= Xn_1;
 end
```

Critical Path: 2A



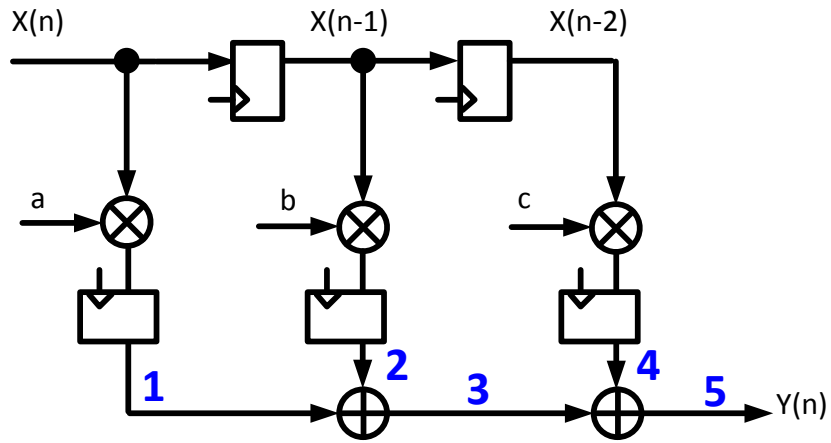

```
assign  Y = r3 + w1;
assign  w1 = r1 + r2;
always @(posedge Clk)
 begin
    Xn_1 <= Xn;
    Xn_2 <= Xn_1;
    r1 <= a*Xn;
    r2 <= b*Xn_1;
    r3 <= c*Xn_2;
 end
```

Architectural Techniques : Pipelining



Clock	Input	1	2	3	4	5	Output
0	X(0)	aX(0)	-	aX(0)	-	aX(0)	Y(0)
1	X(1)	aX(1)	bX(0)	aX(1)+bX(0)	-	aX(1)+bX(0)	Y(1)
2	X(2)	aX(2)	bX(1)	aX(2)+bX(1)	cX(0)	aX(2)+bX(1)+cX(0)	Y(2)
3	X(3)	aX(3)	bX(2)	aX(3)+bX(2)	cX(1)	aX(3)+bX(2)+cX(1)	Y(3)

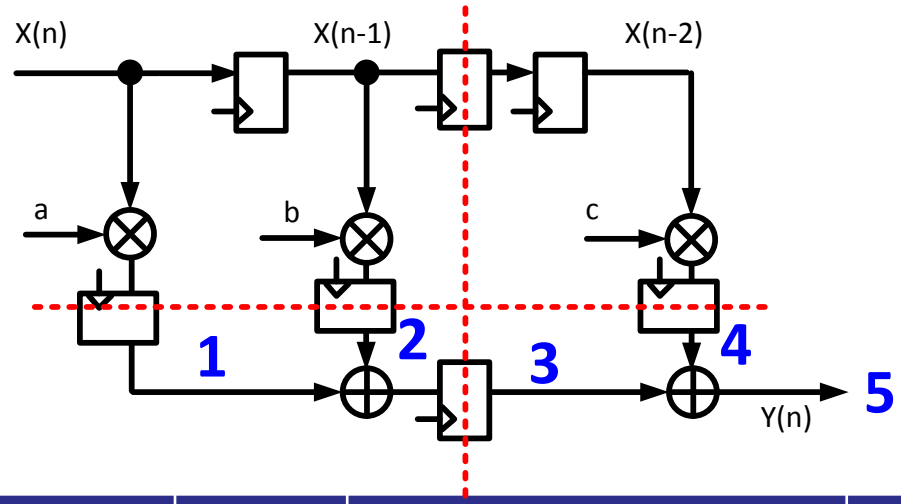
Architectural Techniques : Pipelining



Clock	Input	1	2	3	4	5	Output
0	X(0)	-	-	-	-	-	-
1	X(1)	aX(0)	-	aX(0)	-	aX(0)	Y(0)
2	X(2)	aX(1)	bX(0)	aX(1)+bX(0)	-	aX(1)+bX(0)	Y(1)
3	X(3)	aX(2)	bX(1)	aX(2)+bX(1)	cX(0)	aX(2)+bX(1)+cX(0)	Y(2)

Architectural Techniques : Pipelining

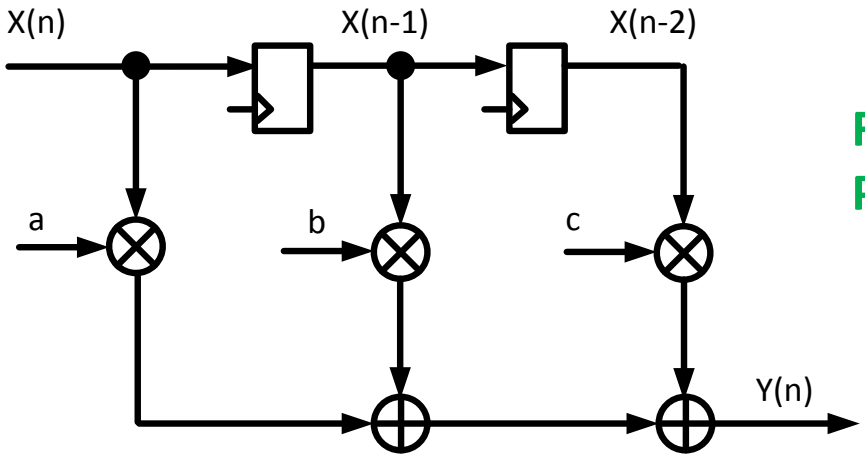
- Even more pipelining



Clock	Input	1	2	3	4	5	Output
0	X(0)	-	-	-	-	-	-
1	X(1)	aX(0)	-	-	-	-	-
2	X(2)	aX(1)	bX(0)	aX(0)	-	aX(0)	Y(0)
3	X(3)	aX(2)	bX(1)	aX(1)+bX(0)	-	aX(1)+bX(0)	Y(1)
4	X(3)	aX(2)	bX(1)	aX(2)+bX(1)	cX(0)	aX(2)+bX(1)+cX(0)	Y(2)

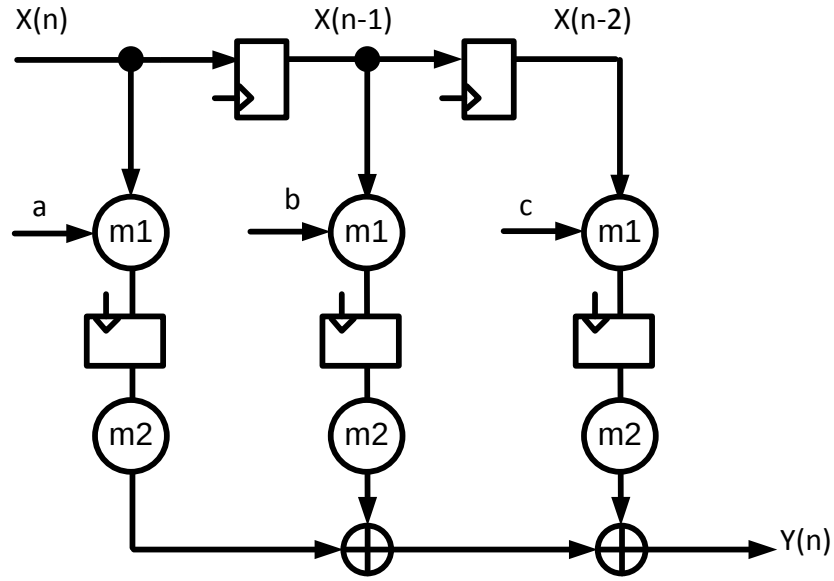
Architectural Techniques : Fine-Grain Pipelining

- Pipelining at the operation level
  - Break the multiplier into two parts



Y(n) = ax(n) + bx(n-1) + cx(n-2)

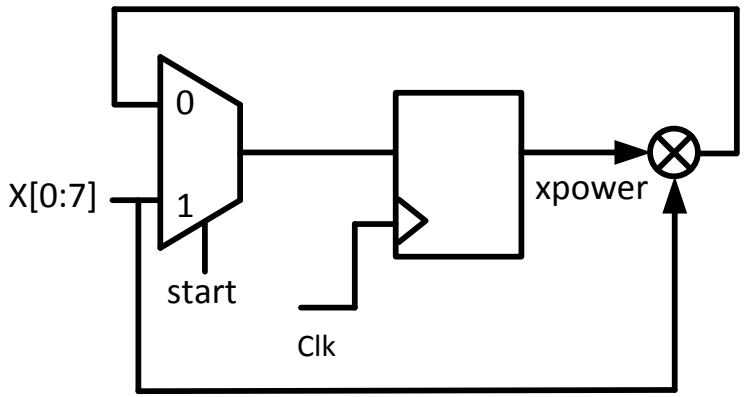
Fine-Grain Pipelining



Unrolling the Loop Using Pipelining

- Calculation of X^3
  - Throughput = 8/3, or 2.7 bits/clock
  - Latency = 3 clocks
  - Timing = One multiplier in the critical path

- Iterative implementation:
- No new computations can begin until the previous computation has completed

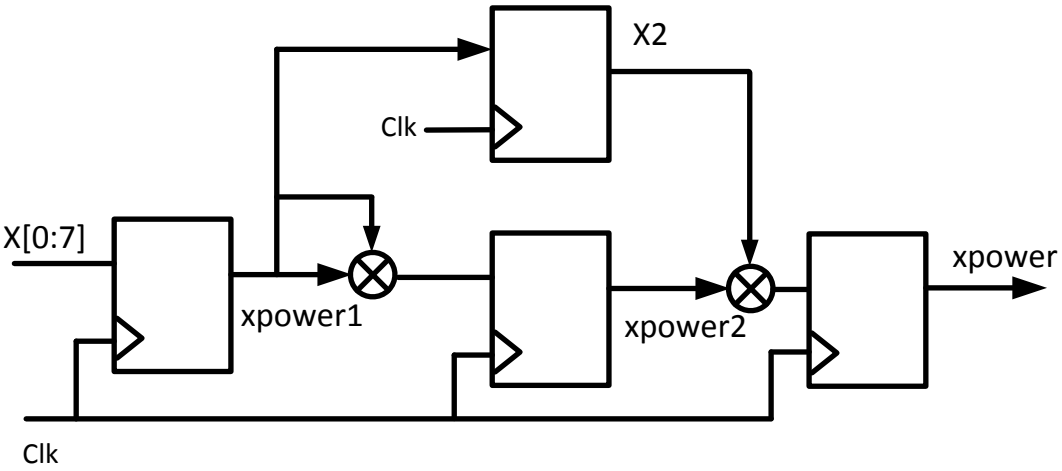



```
module power3(
output reg [7:0] X3,
output finished,
input [7:0] X,
input clk, start);
reg [7:0] ncount;
reg [7:0] Xpower, Xin;
assign finished = (ncount == 0);
        always@(posedge clk)
        if (start) begin
        XPower <= X; Xin<=X;
        ncount <= 2;
        X3 <= XPower;
        end
        else if(!finished) begin
        ncount <= ncount - 1;
        XPower <= XPower * Xin;
        End
endmodule
```

Unrolling the Loop Using Pipelining

- Calculation of X^3
  - Throughput = 8/1, or 8 bits/clock (3X improvement)
  - Latency = 3 clocks
  - Timing = One multiplier in the critical path
- **Penalty**: More Area

Unrolling an algorithm with n iterative loops increases throughput by a factor of n

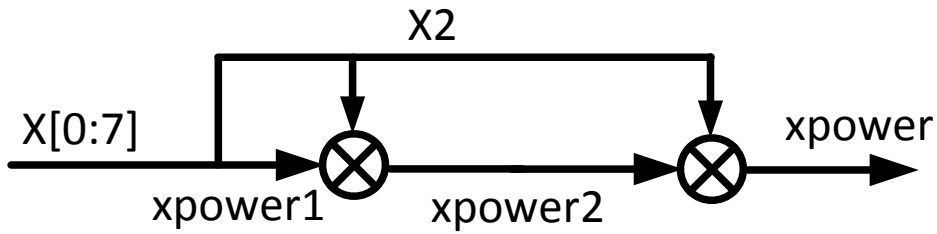



```
module power3(
output reg [7:0] XPower,
input clk,
input [7:0] X);
reg [7:0] XPower1, XPower2;
reg [7:0] X2;
always @(posedge clk) begin
// Pipeline stage 1
XPower1 <= X;
// Pipeline stage 2
XPower2 <= XPower1 * XPower1 ;
X2 <= XPower1 ;
// Pipeline stage 3
XPower <= XPower2 * X2;
end
endmodule
```

Removing Pipeline Registers (to Improve Latency)

- Calculation of X^3
  - Throughput = 8 bits/clock (3X improvement)
  - Latency = 0 clocks
  - Timing = Two multipliers in the critical path

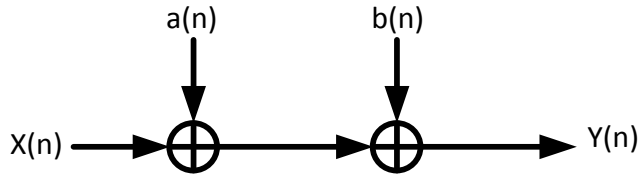
Latency can be reduced by removing pipeline registers




```
module power3(
Output [7:0] XPower,
input [7:0] X);
reg [7:0] XPower1, XPower2;
reg [7:0] X1, X2;
always @(*)
   XPower1 = X;
always @(*)
begin
   X2 = XPower1;
   XPower2 = XPower1*XPower1;
end

assign XPower = XPower2 * X2;

endmodule
```

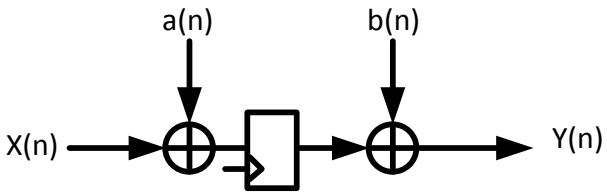
Architectural Techniques : Parallel Processing

- In parallel processing the same hardware is duplicated to
  - Increases the throughput without changing the critical path
  - Increases the silicon area



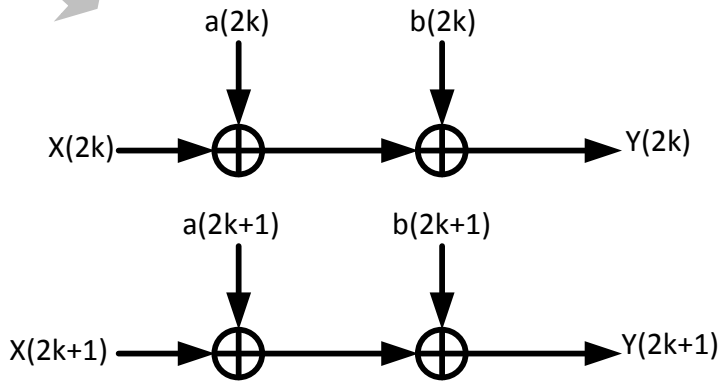
Clock Freq: f
Throughput: M samples

Pipelining



Clock Freq: 2f
Throughput: 2M samples

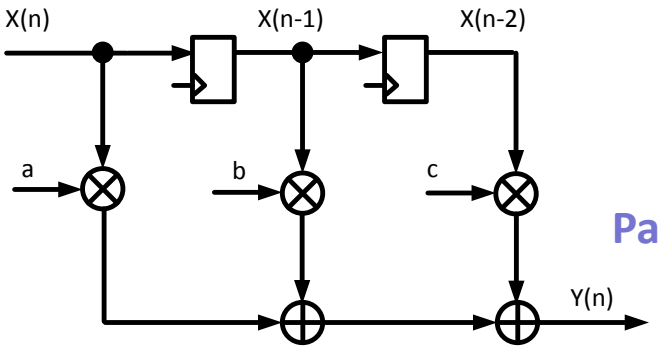
Parallel Processing



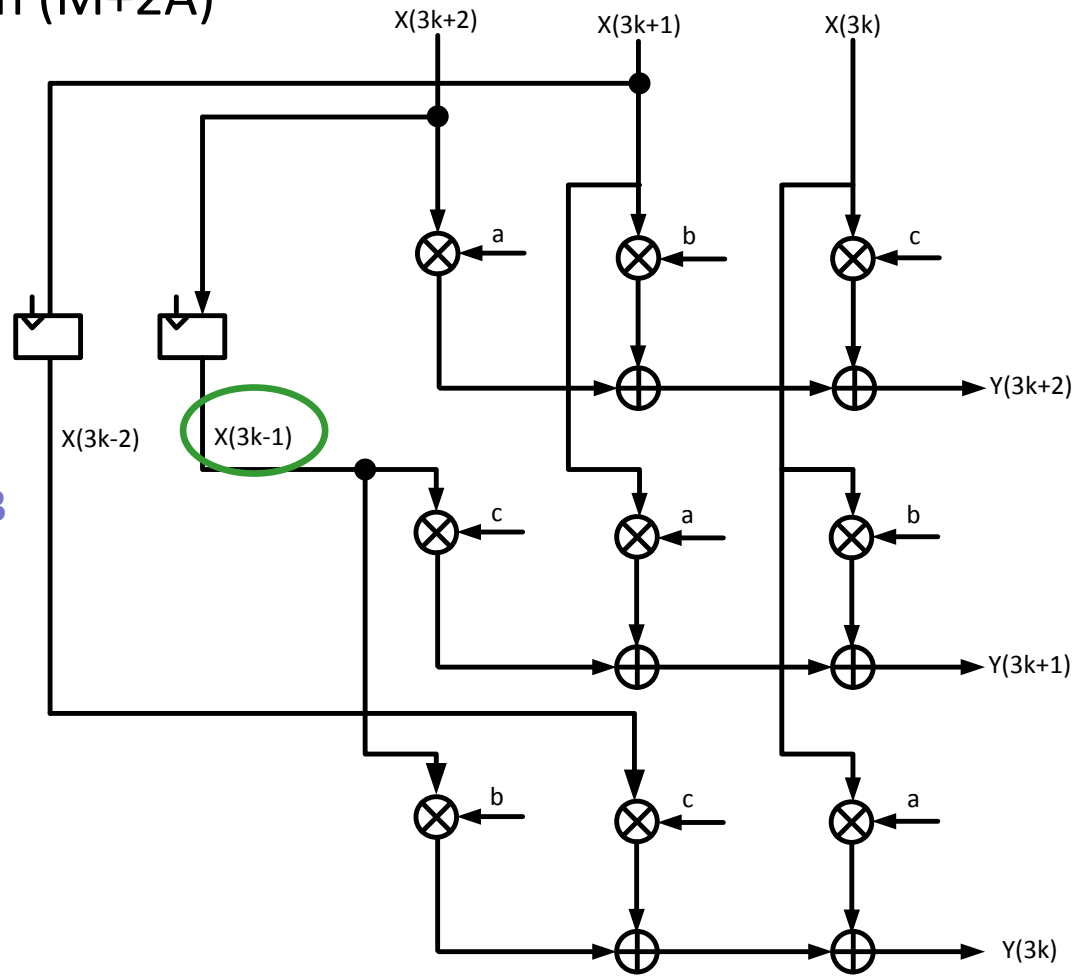
Clock Freq: f
Throughput: 2M samples

Architectural Techniques : Parallel Processing

- Parallel processing for a 3-tap FIR filter
  - Both have the same critical path (M+2A)



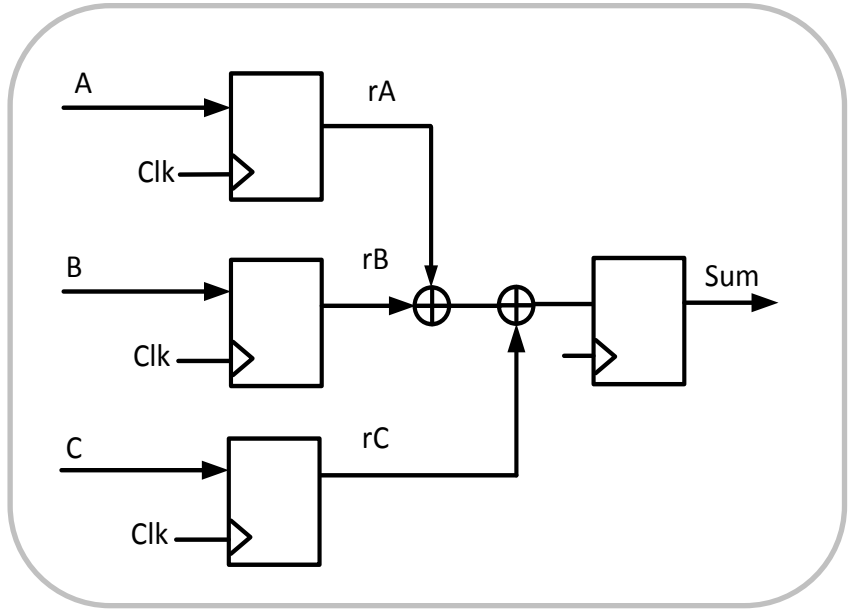
Parallel Factor:3



Register Balancing (to Improve Timing)

- Redistribute logic evenly between registers to minimize the worst-case delay between any two registers
  - b/c clock is limited by only the worst-case delay

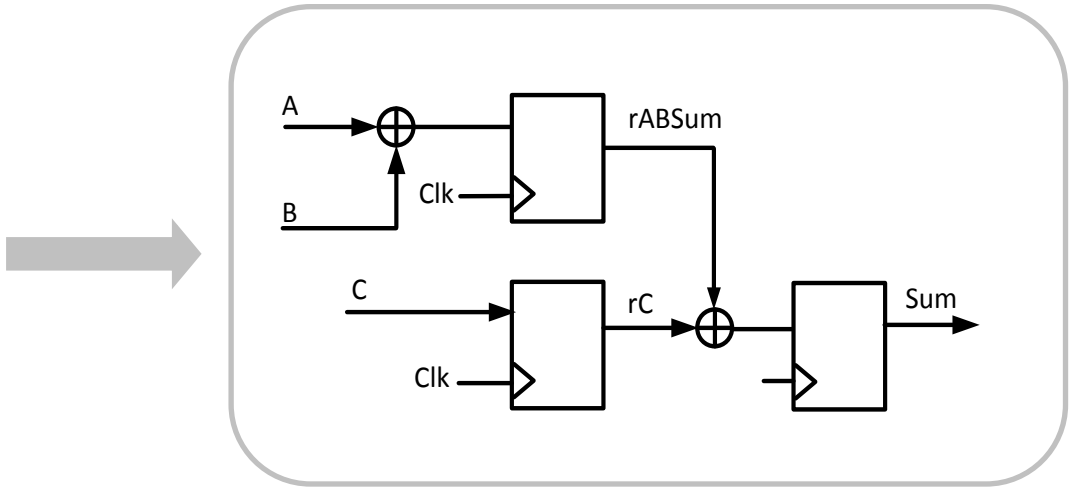
```
module adder(
output reg [7:0] Sum,
input [7:0] A, B, C,
input clk);
reg [7:0] rA, rB, rC;
always @(posedge clk) begin
rA <= A;
rB <= B;
rC <= C;
Sum <= rA + rB + rC;
end
endmodule
```



Register Balancing (to Improve Timing)

- Redistribute logic evenly between registers to minimize the worst-case delay between any two registers
  - b/c clock is limited by only the worst-case delay

```
module adder(
output reg [7:0] Sum,
input [7:0] A, B, C,
input clk);
reg [7:0] rABSum, rC;
always @(posedge clk) begin
rABSum <= A + B;
rC <= C;
Sum <= rABSum + rC;
end
endmodule
```



Speed-related Techniques: Summary

- **Throughput-related:**
  - A high-throughput architecture is one that maximizes the number of bits per second that can be processed by a design.
  - Unrolling an iterative loop increases throughput.
  - The penalty for unrolling an iterative loop is an increase in area.

- **Latency-related:**
  - A low-latency architecture is one that minimizes the delay from the input of a module to the output.
  - Latency can be reduced by removing pipeline registers.
  - The penalty for removing pipeline registers is an increase in combinatorial delay between registers.

Speed-related Techniques: Summary

- **Timing-related:**
  - Timing refers to the clock speed of a design. A design meets timing when the maximum delay between any two sequential elements is smaller than the minimum clock period.
  - Adding register layers improves timing by dividing the critical path into two paths of smaller delay.
  - Separating a logic function into a number of smaller functions that can be evaluated in parallel reduces the delay to the longest of the substructures.
  - By removing priority encodings where they are not needed, the logic structure is flattened, and the path delay is reduced.
  - Register balancing improves timing by moving combinatorial logic from the critical path to an adjacent path.

Area-related Techniques:

- Area is the second primary factors of a digital design

- A topology that targets area is one that reuses the logic resources to the greatest extent possible, often at the expense of throughput (speed).

- This requires a recursive data flow, where the output of one stage is fed back to the input for similar processing.

Area-related Techniques: Rolling Up the Pipeline

- Opposite to the unrolling the loop to increase throughput
- Unrolling the loop achieved by adding more registers to hold intermediate values, i.e., more area
- Thus to reduce the area the reversed action should be done (i.e., Sharing)
- Resource Sharing is used where there are functional blocks that can be used in other areas of the design or even in different modules
- Sharing logic resources requires special control circuitry to determine which elements are input to the particular structure

Rolling Up the Pipeline

- Calculation of P = A*B
  - A: a normal integer with the fixed point just to the right of the LSB (8 bits)
  - B: a fractional number with a fixed point just to the left of the MSB (8 bits)
  - P: the product, which requires only 8-bits

- One implementation alternative:
  - Critical path: one multiplier (complex itself)
  - One product every clock cycle (high throughput)

```
module mult8(
output [7:0] P,
input [7:0] A,
input [7:0] B,
input clk);
reg [15:0] prod16;
assign P= prod16[15:8];
always @(posedge clk)
prod16 <= A * B;
endmodule
```

Rolling Up the Pipeline : Resource Sharing

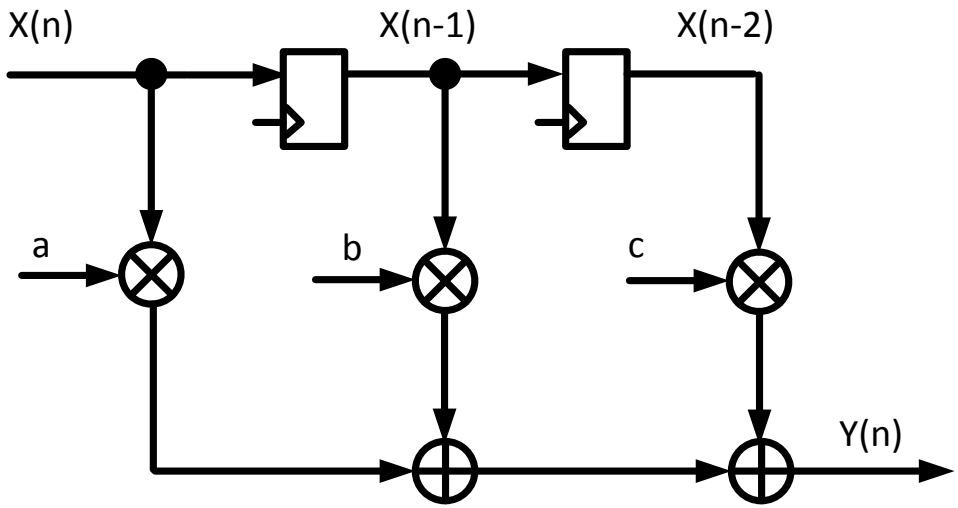
- Rolling Up the Pipeline:
  - Using series of shift-and-add operations
  - Smaller critical path
  - Less area due to the simple operations and sharing
  - One product every 8 clock cycles! (low throughput)

```
module mult8(output done,output reg [7:0] product,
input [7:0] A, input [7:0] B, input clk, input start);
reg [4:0] multcounter; // number of shift/adds
reg [7:0] shiftB; // shift register for B
reg [7:0] shiftA; // shift register for A
wire adden; // enable addition
assign adden = shiftB[7] & !done;
assign done = multcounter[3];
always @(posedge clk) begin
if(start) multcounter <= 0;
else if(!done) multcounter <= multcounter + 1;
// shift register for B
if(start) shiftB <= B;
else shiftB[7:0] <= {shiftB[6:0], 1'b0};
```

```
// shift register for A
if(start) shiftA <= A;
else shiftA[7:0] <= {shiftA[7], shiftA[7:1]};
// calculate multiplication
if(start) product <= 0;
else if(adden) product <= product + shiftA;
end
endmodule
```

Resource Sharing: Area Reduction Technique

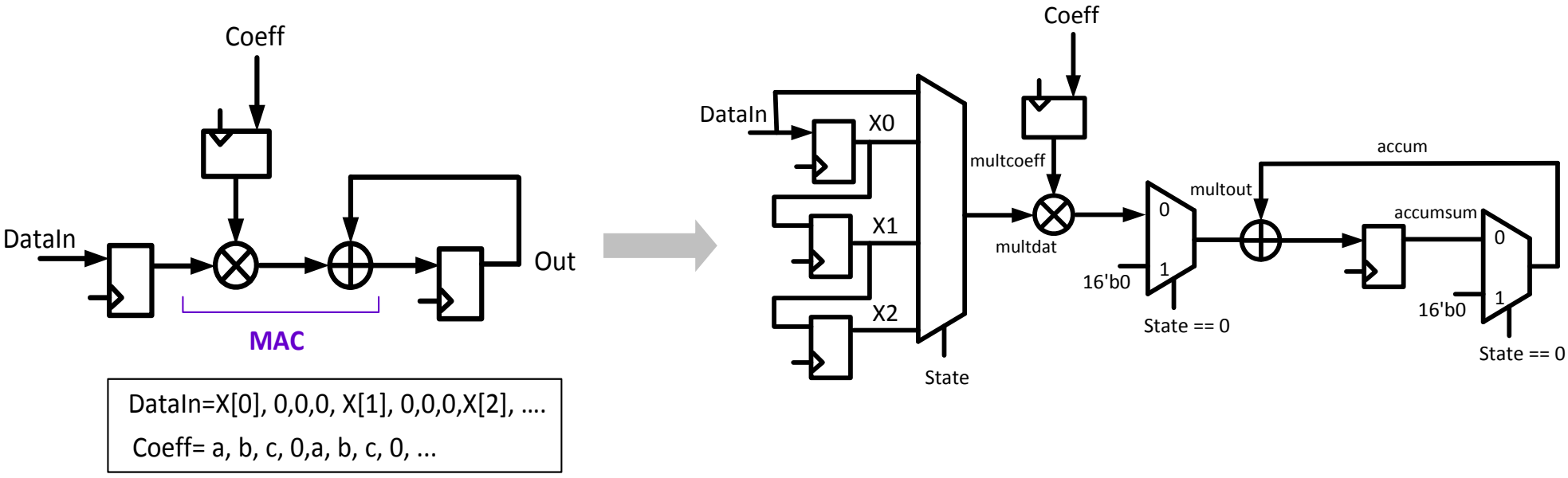
- Back to FIR Filter:
  - Three multiplications, two adders, two registers



$$Y(n) = ax(n) + bx(n-1) + cx(n-2)$$

Resource Sharing: Area Reduction Technique

- Sharing the Multiply-Accumulate (MAC) to reduce area:
  - One multiplication, one adder, one register
  - Requires some control logic to determine which input is inserted (FSM)

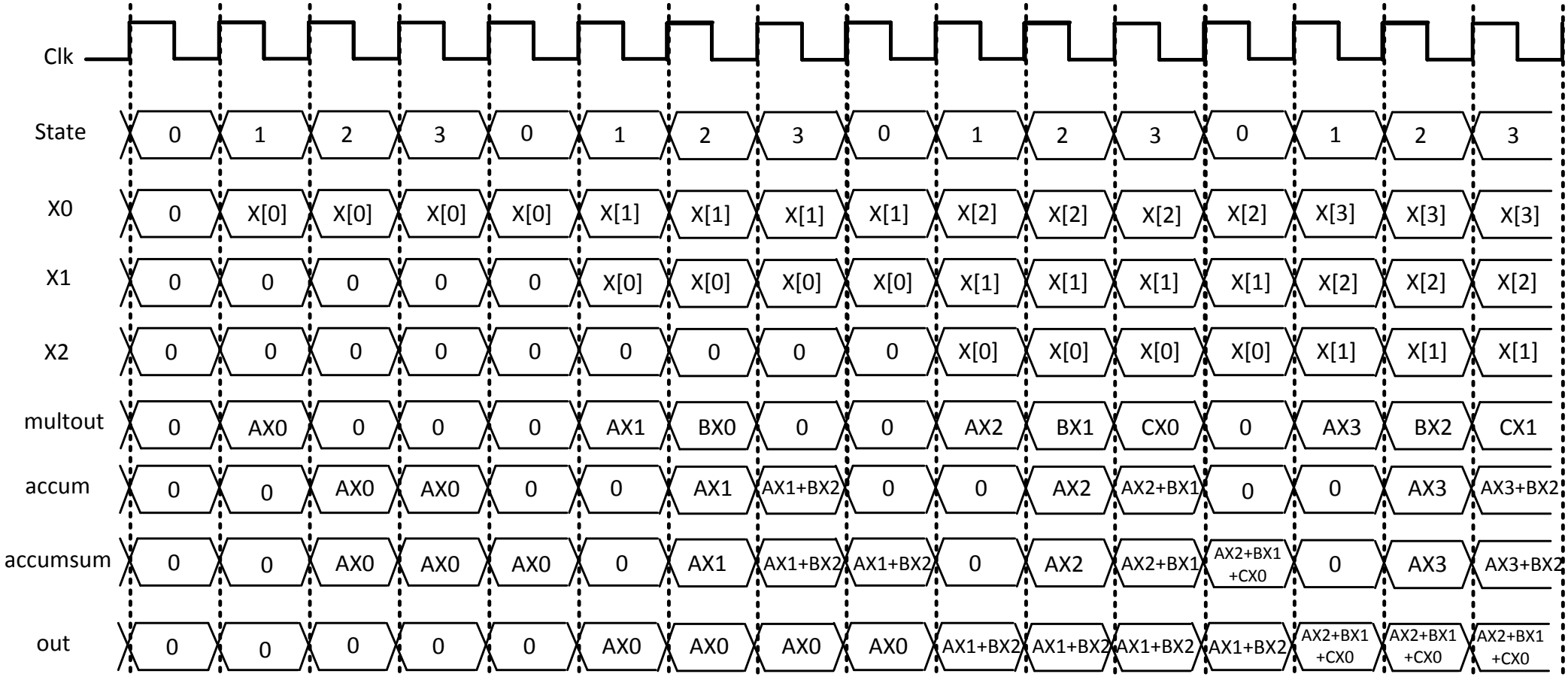


Resource Sharing: Area Reduction Technique

```
module sharing(
output reg [15:0] Out,
input clk,
input [7:0] datain, // X[0]
input [7:0] coeffA, coeffB, coeffC); // coeffs for low pass filter
// define input/output samples
reg [7:0] X0, X1, X2;
reg [2:0] state; // holds state for sequencing through mults
wire [15:0] accum; // accumulates multiplier products
reg [15:0] accumsum;
wire [15:0] multout; // multiplier product
reg [7:0] multdat;
reg [7:0] multcoeff;

assign multout =(state==0)?16'b0:multcoeff * multdat;
// clearing and loading accumulator
assign accum = (state==0)?16'b0:accumsum;

always @(posedge clk)
accumsum <= accum + multout;
```

```
always @ (posedge clk) begin
case(state)
      0: begin // load new data
                X0 <= datain; X1 <= X0; X2 <= X1;
                multdat <= datain; multcoeff <= coeffA;
                state <= 1;
                 Out <= accumsum;
      end
      1: begin // A*X[0] is done, load B*X[1]
                multdat <= X1; multcoeff <= coeffB;
                state <= 2;
       end
       2: begin // B*X[1] is done, load C*X[2]
                multdat <= X2; multcoeff <= coeffC;
                state <= 3;
        end
        3: begin // C*X[2] is done, load output
                state <= 0;
         end
         default
                state <= 0;
         endcase
end
endmodule
```

Resource Sharing: Area Reduction Technique

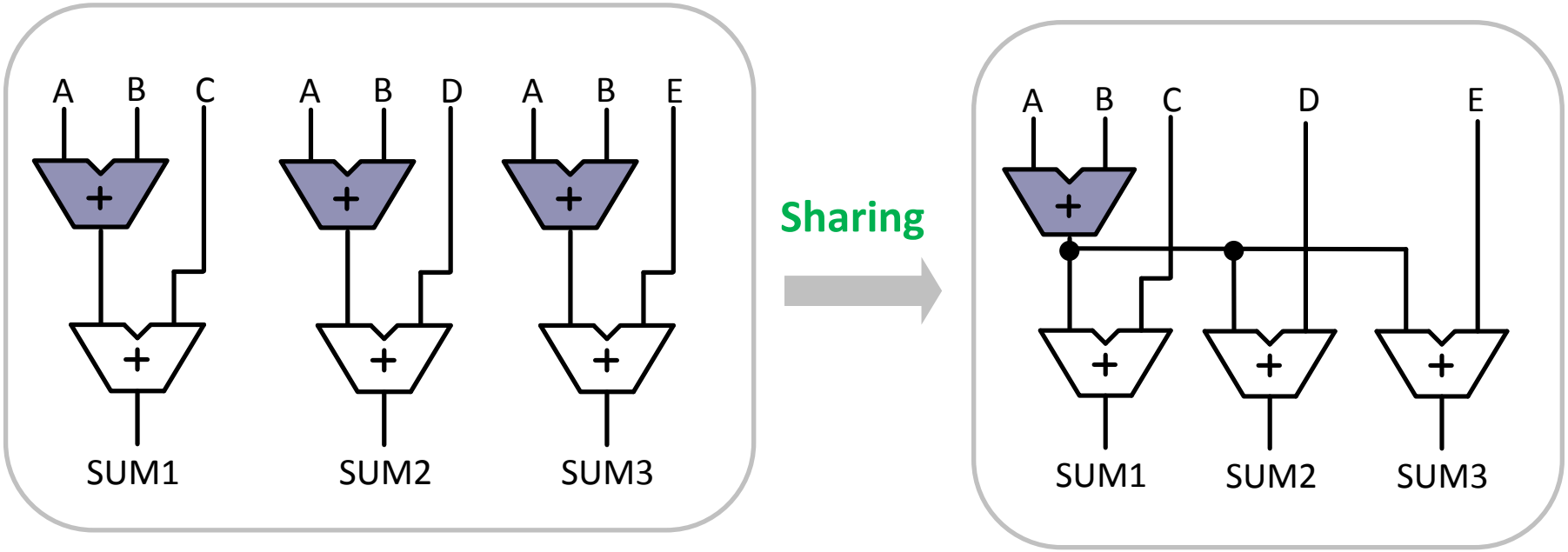


Resource Sharing: Area Reduction Technique

- HDL coding style can force a specific topology to be synthesized

```
SUM1 <= A+B+C;
SUM2 <= A+B+D;
SUM3 <= A+B+E;
```

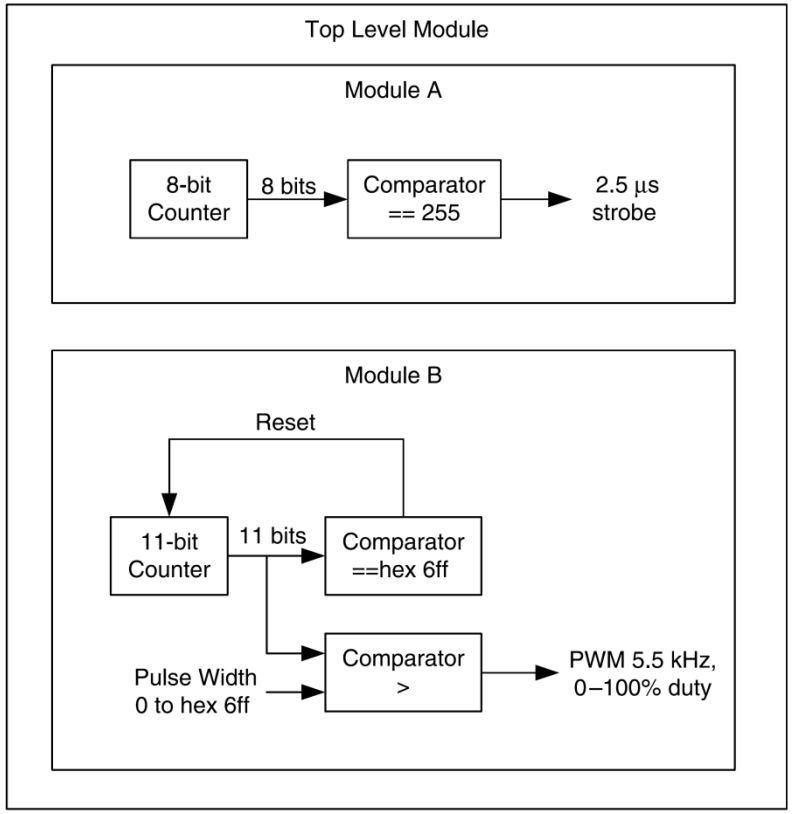
```
assign  tmp = A+B;
SUM1 <= tmp +C;
SUM2 <= tmp +D;
SUM3 <= tmp +E;
```



© M. Shabany, ASIC/FPGA Chip Design

Resource Sharing: Area Reduction Technique

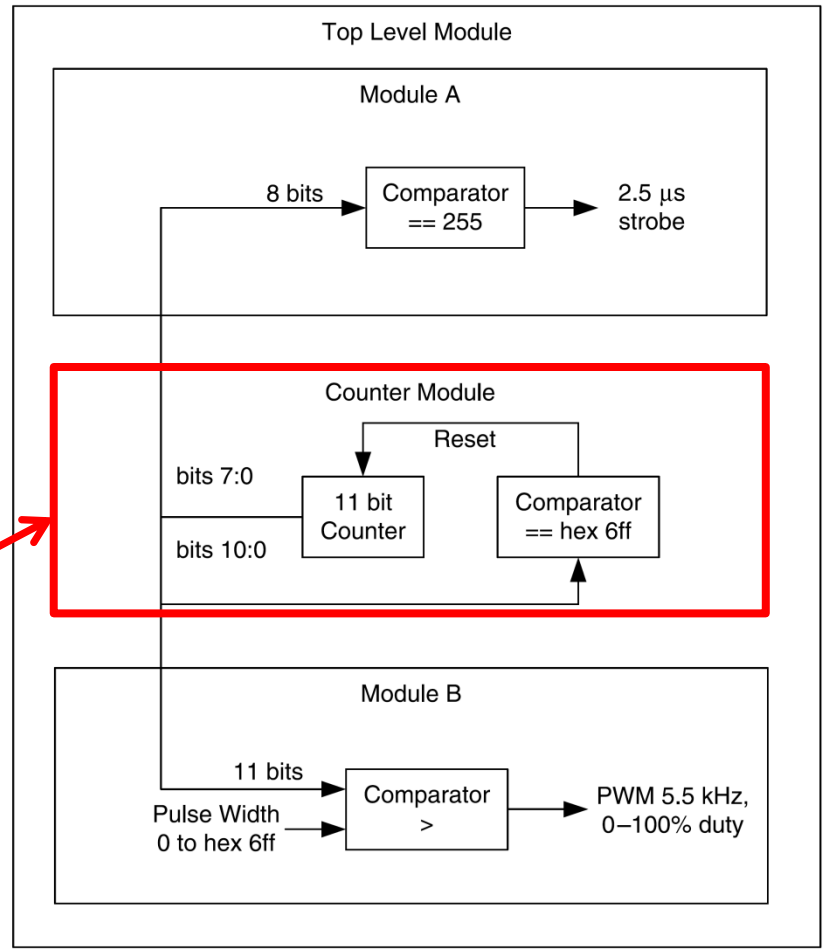
- Let's assume we have two counters controlling different design sections
  - CounterA, a free running 8-bit counter
  - CounterB, an 11-bit counter, counting from 0 to 1666 and resets to zero



Resource Sharing: Area Reduction Technique

- Resource-shared version of these two counters

For compact designs where area is the primary requirement, search for resources that have similar counterparts in other modules that can be brought to a global point in the hierarchy and shared between multiple functional areas.



Shared Part

Area Techniques: Reset Issues

- Some predefined modules are optimized area-wise in all vendors
- These modules have a specific constraint, e.g., only support synchronous reset, or only support reset to zero
- If the same functionality is used but violating these constraints, the function can not be realizable using predefined optimized modules
- In this case, the function is realized using general LUTs, thus a lot more area

Area Techniques: Reset Issues

- Multiply-Accumulate (MAC) module can be realized using built-in DSP blocks

```
module dsp(
output reg [15:0] oDat,
Input Reset, Clk,
input [7:0] Dat1, Dat2);
reg [15:0] multfactor;
always @(posedge Clk or negedge Reset)
if(!Reset) begin
multfactor <= 0;
oDat <= 0;
end
else begin
multfactor <= (Dat1 * Dat2);
oDat <= multfactor + oDat;
end
endmodule
```

Resources	Amount
Total Pins	34
Combination LUTs	0
Dedicated Logic Registers	0
Total Register	0
9-bit DSP Block	4

Area Techniques: Reset Issues

- Multiply-Accumulate (MAC) module can **NOT** be realized using built-in DSP blocks
- Because the DSP block is customized only for asynchronous reset.

```
module dsp(
output reg [15:0] oDat,
Input Reset, Clk,
input [7:0] Dat1, Dat2);
reg [15:0] multfactor;
always @(posedge Clk)
if(!Reset) begin
multfactor <= 0;
oDat <= 0;
end
else begin
multfactor <= (Dat1 * Dat2);
oDat <= multfactor + oDat;
end
endmodule
```

Resources	Amount
Total Pins	34
Combination LUTs	16
Dedicated Logic Registers	32
Total Register	32
9-bit DSP Block	1

Area Techniques: Reset Issues

- Multiply-Accumulate (MAC) module can **NOT** be realized using built-in DSP blocks
- Because asynchronous reset can be done only to zero not a non-zero value

```
module dsp(
output reg [15:0] oDat,
Input Reset, Clk,
input [7:0] Dat1, Dat2);
reg [15:0] multfactor;
always @(posedge Clk or negedge Reset)
if(!Reset) begin
multfactor <= 16'hffff;
oDat <= 16'hffff;
end
else begin
multfactor <= (Dat1 * Dat2);
oDat <= multfactor + oDat;
end
endmodule
```

Resources	Amount
Total Pins	34
Combination LUTs	48
Dedicated Logic Registers	32
Total Register	32
9-bit DSP Block	1

RAM Reset Issues

- Some vendors have either synchronous or asynchronous Block RAMs (BRAMs)

```
module resetckt(
output reg [15:0] oDat,
input iReset, iClk, iWrEn,
input [7:0] iAddr, oAddr,
input [15:0] iDat);
reg [15:0] memdat [0:255];
always @(posedge iClk)
if(!iReset)
oDat <= 0;
else begin
if(iWrEn)
memdat[iAddr] <= iDat;
oDat <= memdat[oAddr];
end
endmodule
```

Resources	Amount
Total Pins	51
Combination LUTs	1652
Dedicated Logic Registers	4152
Total Register	4152
Total Block Memory Bits	0

RAM Reset Issues

- Some vendors have either synchronous or asynchronous Block RAMs (BRAMs)

```
module resetckt(
output reg [15:0] oDat,
input iReset, iClk, iWrEn,
input [7:0] iAddr, oAddr,
input [15:0] iDat);
reg [15:0] memdat [0:255];
always @(posedge iClk or negedge iReset)
if(!iReset)
oDat <= 0;
else begin
if(iWrEn)
memdat[iAddr] <= iDat;
oDat <= memdat[oAddr];
end
endmodule
```

Resources	Amount
Total Pins	51
Combination LUTs	17
Dedicated Logic Registers	1
Total Register	1
Total Block Memory Bits	4096

Summary: An optimized FPGA resource will not be used if an incompatible reset is assigned to it. The function will be implemented with generic elements and will occupy more area.

Power Reduction Techniques:

- In CMOS technology, dynamic power consumption is related to charging and discharging parasitic capacitances on gates and metal traces.

$$P = C.f.V^2$$

  - **C**: Capacitance
    - Related to the number of gates that are toggling at any given time and the lengths of the routes connecting the gates
  - **f**: frequency (related to the clock frequency)
  - **V**: voltage (usually fixed in FPGAs)

All of the power-reduction techniques ultimately aim at reducing one of these three components.

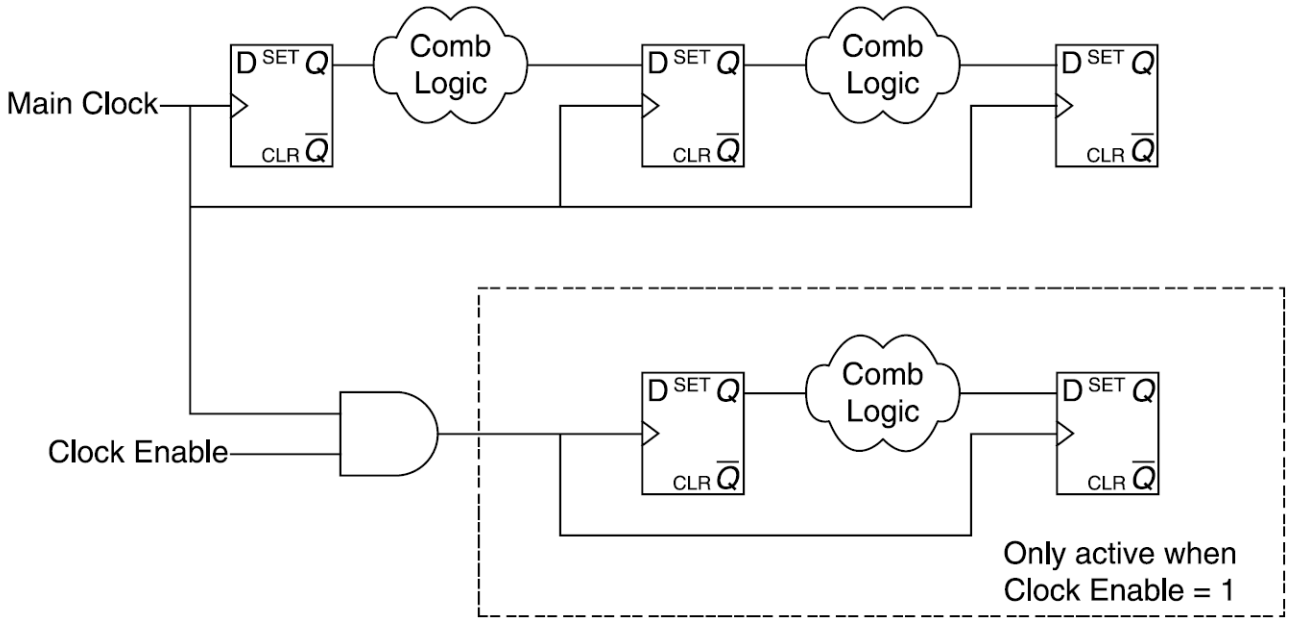
Power Reduction Techniques: Clock Issues

- The most effective power reduction technique related to the clock is to dynamically disable the clock in specific regions that do not need to be active in specific times

- To do this use
  - Clock enable pin on Flip-flops (available in most modern devices)
  - Global clock mux
  - Clock gating (not preferred)

By gating portions of circuitry, the designer reduces the dynamic power dissipation proportional to the amount of logic (capacitance C) and the average toggling frequency of the corresponding gates (frequency f). Note that the clock tree dissipates a lot of power!

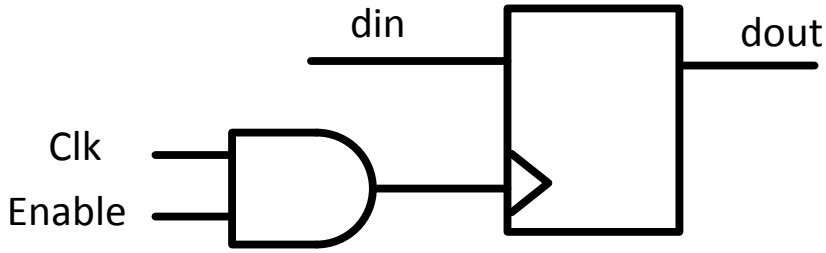
Power Reduction Techniques: Clock Issues



- When a clock is gated, the new net on the clock generates a new clock domain. This new clock net requires a low-skew path to all FFs in its domain.
- For ASIC, these low-skew lines can be built in the custom clock tree, but it is problematic for FPGA b/c of the limited number/fixed layout of low-skew lines.

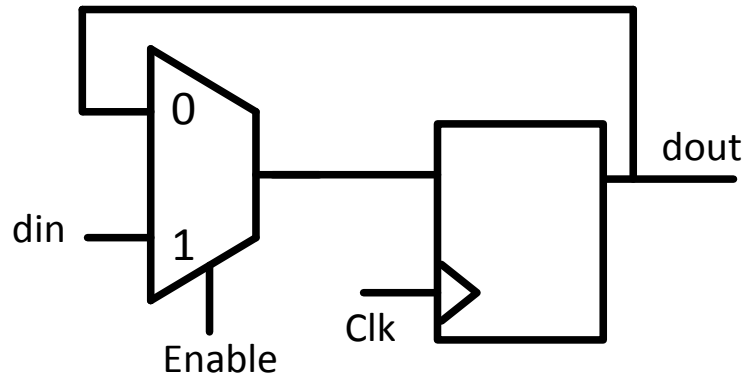
Power Reduction Techniques: Gated Clock

- Some synthesis tools have an option called “fix gated clocks”.
- This feature will automatically move the gating operation off of the clock line and into the data path.




```
module clockstest(
output reg dout,
input Clk, Enable,
input Din);
wire gated_clock = Clk & Enable;
always @(posedge gated_clock)
dout <= din;
endmodule
```

Gated clock removed

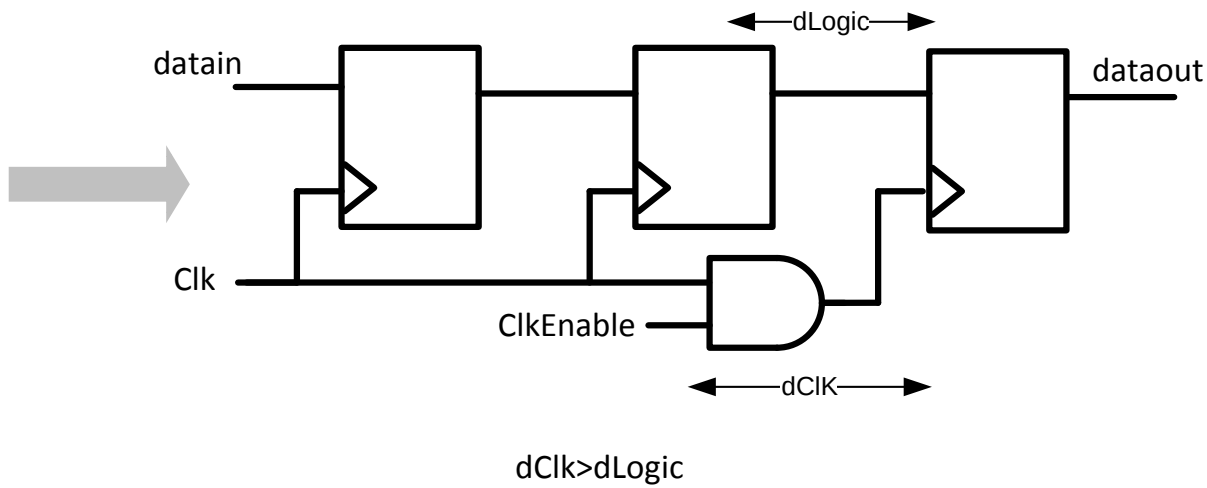


MUX adds delay to the data path

Power Reduction Techniques: Clock Issues (skew)

- Hold time violation in FPGAs is mainly because of the excessive delay on the clock
- Hold time violation b/c of low logic delay is rare due to the built-in delays of the logic blocks and routing resources

```
module clockgating(
output dataout,
input clk, datain,
input ClockEnable);
reg ff0, ff1, ff2;
wire clk1;
assign clk1 = clk & ClockEnable;
assign dataout = ff2;
always @(posedge clk)
ff0 <= datain;
always @(posedge clk)
ff1 <= ff0;
always @(posedge clk1)
ff2 <= ff1;
endmodule
```



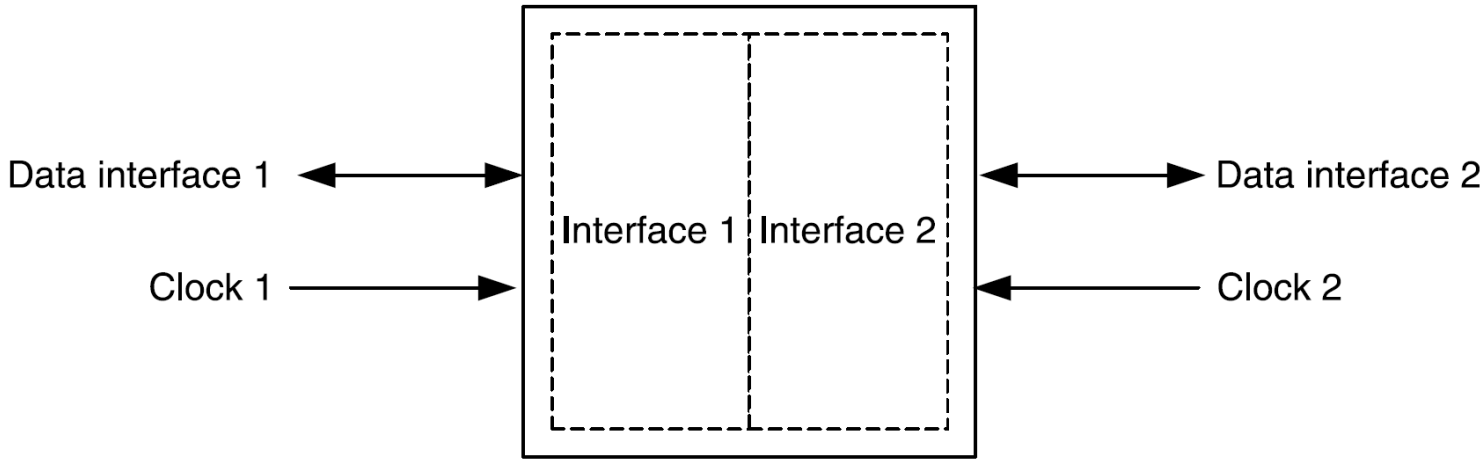
dClk>dLogic

Clock Domain

- A clock domain is a section of logic where all synchronous elements (flip-flops, synchronous RAMs, pipelined multipliers) are clocked by the same net.

- If all flip-flops are clocked by a single global clock input to the FPGA, then there is one clock domain.

- Two clock domains for different interfaces:



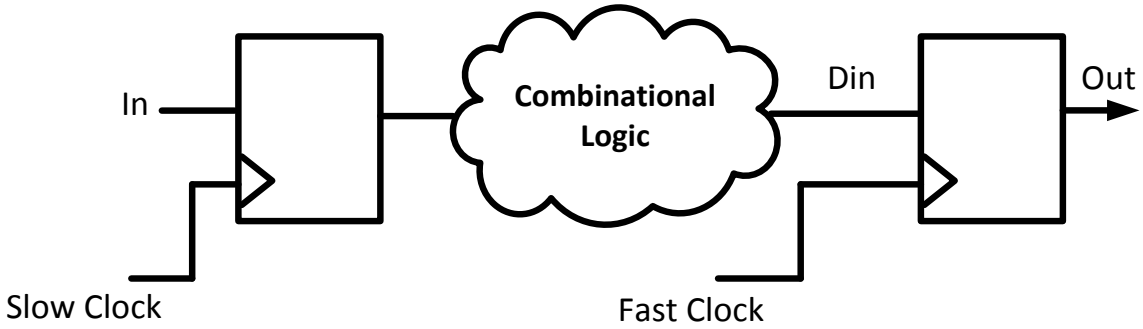
Crossing Clock Domains

- **Crossing clock domains**: passing signals between multiple clock domains
- Clock domain crossing can be a major source of problem b/c:

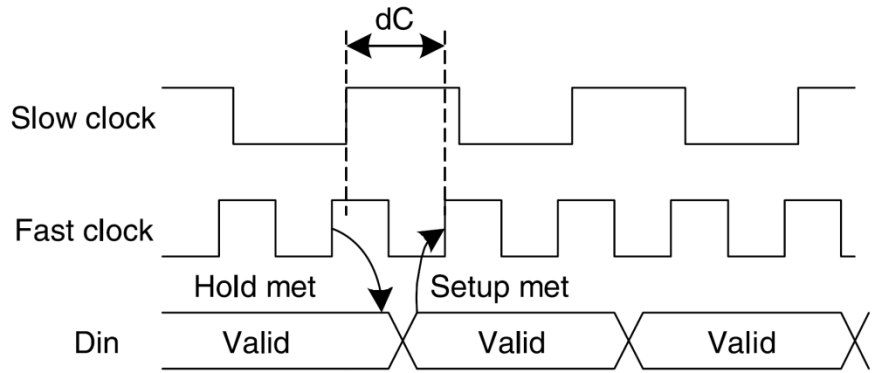
1. If there are two clock domains that are asynchronous, then failures are often related to the relative timing between the clock edges.
2. Problems will vary from technology to technology. Higher speed technologies with smaller setup and hold constraints will have statistically fewer problems
3. EDA tools typically do not detect these problems. Static timing analysis tools analyze timing based on individual clock zones
4. Cross-clock domain failures are difficult to detect and debug if they are not understood. It is very important that all inter-clock interfaces are well defined and handled before any implementation takes place.

Crossing Clock Domains : Issues

- Two clock domains
- Consider a **fine** case, which works properly

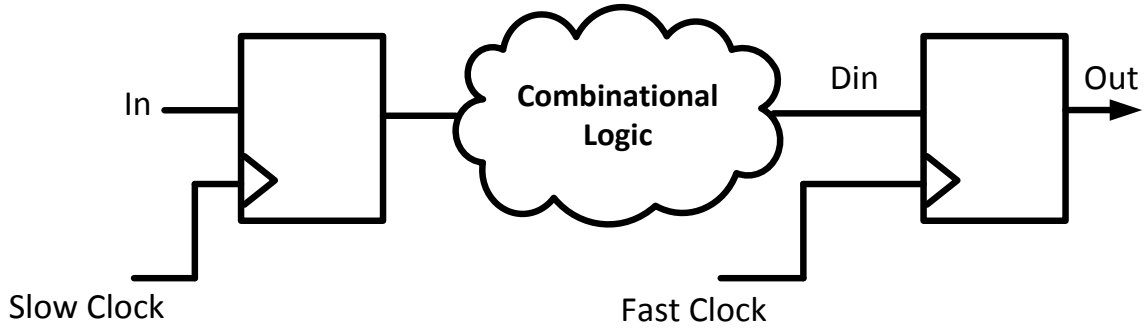


- Phase difference is such that both hold time and setup time constraints are met

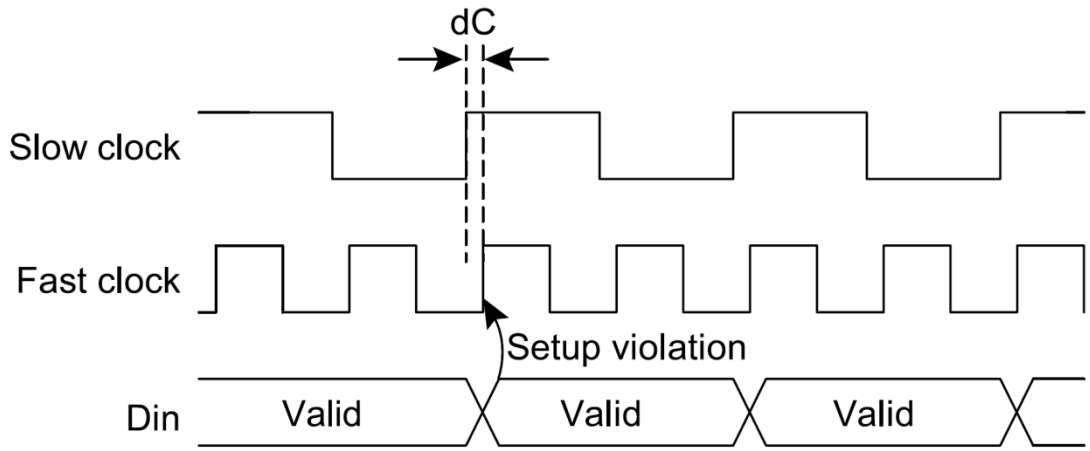


Crossing Clock Domains : Issues

- Consider a **problematic** case



- Phase difference is such that the setup time for the second FF is violated



Crossing Clock Domains: What happens with violation?

- A timing violation occurs when the data input to a flip-flop transitions within a window around the active clock edge as defined by the setup and hold times

- This timing violation exists b/c if the setup and hold times are violated, a node within the flip-flop can become suspended at a voltage that is not valid for either a logic-0 or logic-1.

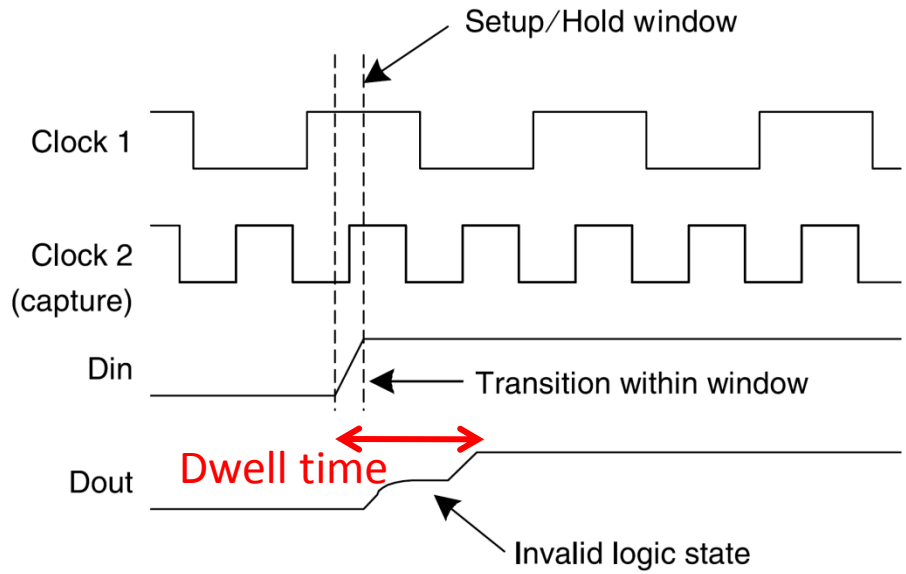
- Rather than saturating at a high or low voltage, the transistors may dwell at an intermediate voltage before settling on a valid level (which may or may not be the correct level).

This is called Metastability

Crossing Clock Domains: Metastability

- Data transition is not allowed within the setup/hold condition window
- The output may remain metastable for the entire clock period (functional failure)
- In an RTL simulation, no setup and hold violations occur and thus no signal will ever go metastable.
- Even harder to verify in the gate-level simulations.

- Undefined dwell time before the signal settles
- This dwell time adds to the path's propagation delay



How to Avoid Metastability?

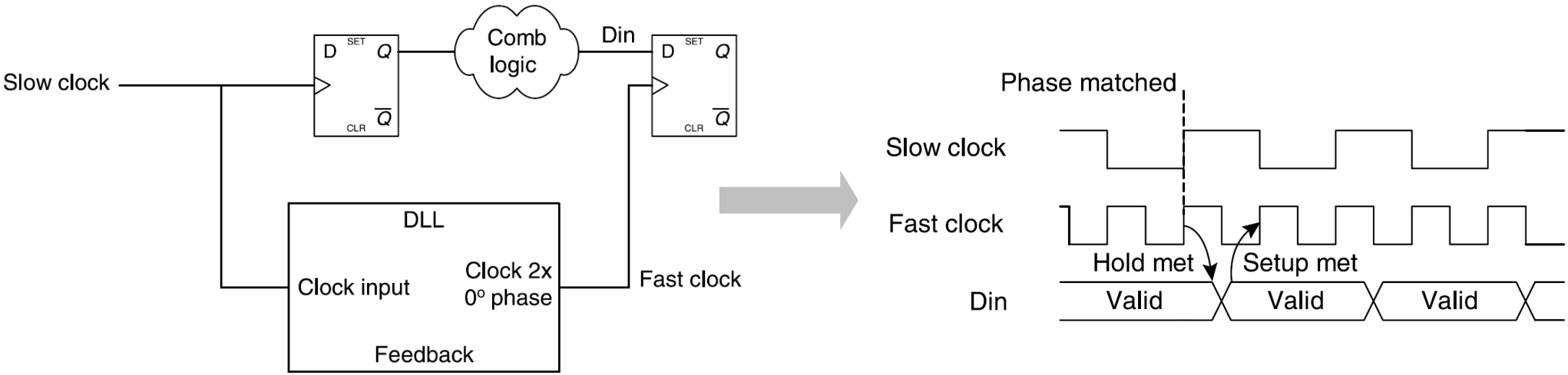
- To resolve the metastability problem, three main solutions may be used:

  - **Case I: the period of one clock is a multiple of the other**
    - Using Phase Locked Loop (PLL) or Delay Locked Loop (DLL)
  - **Case II: Otherwise**
    - Double Flopping (pass a single-bit signal b/w asynchronous clock domains)
    - Using a FIFO structure (pass multi-bit signals b/w asynchronous clocks)

Avoid Metastability Using PLL or DLL

- DLL adjusts the phase of the faster (capture) clock domain to match that of the slower (transmitting) clock domain.
- The total amount of time available for data to pass between the two domains is always at its maximum possible value.

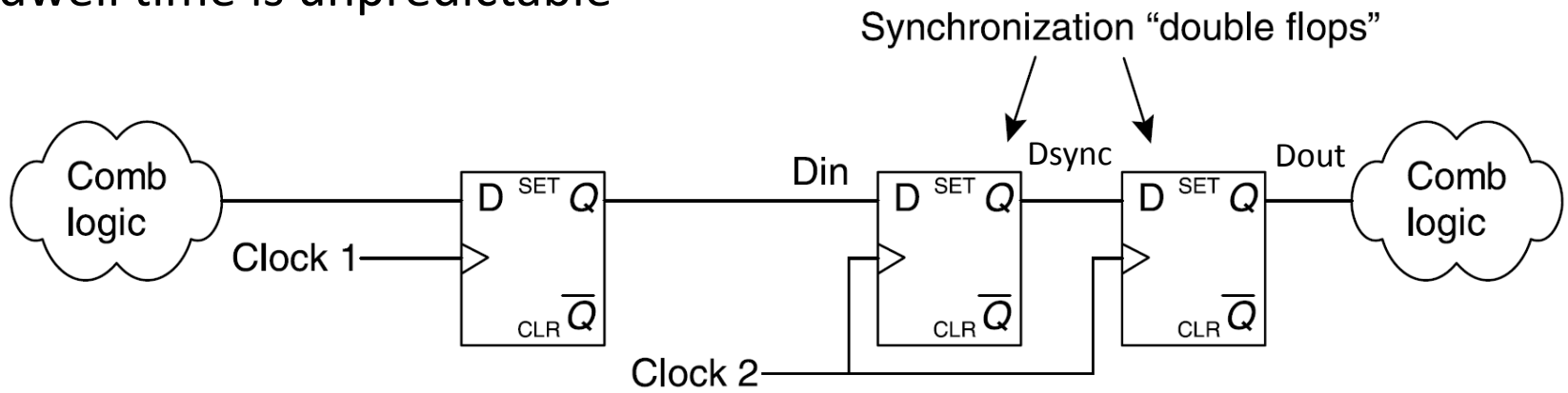
Condition: Propagation delay b/w FFs less than the period of the fast clock

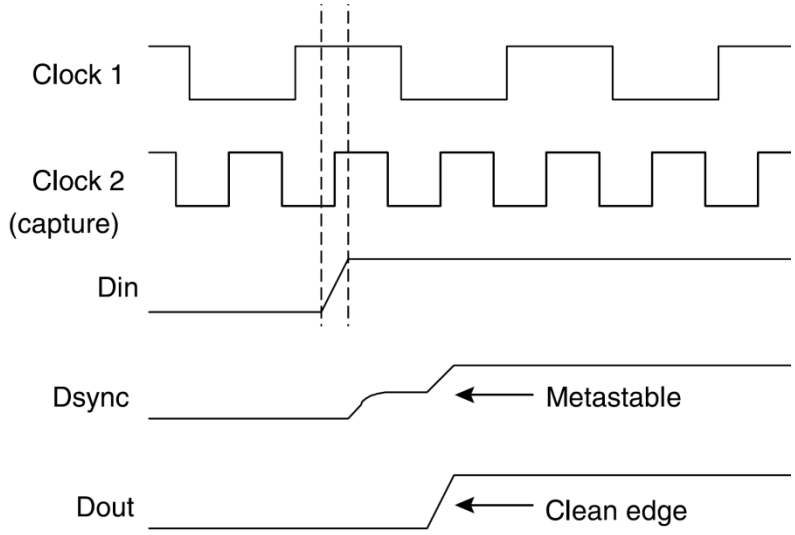


Phase adjustment in general not possible

Avoid Metastability by Double Flopping

- The dwell time before the signal settles adds to the path's propagation delay
- This dwell time is unpredictable



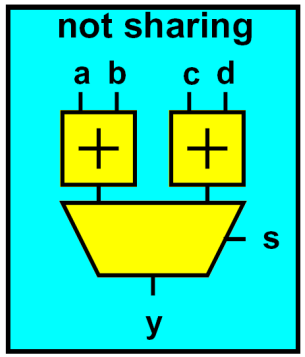


By adding no logic between the two FFs, we maximize the amount of time provided to the signal to settle.

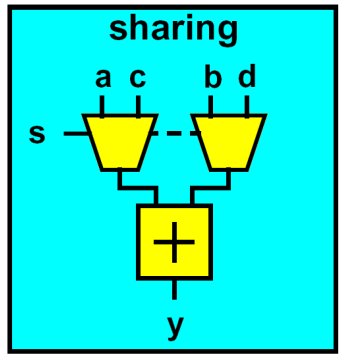
Used to pass a single bit

Architectural Techniques : Resource Sharing

More than one RTL statement can utilize an expensive computational resource.




```
always @(a or b or c
         or d or s)
  begin
    if (s)
      y = a + b;
    else
      y = c + d;
  end
```




```
always @(a or b or c
         or d or s)
  begin: newscope
    reg tmp1, tmp2;
    tmp1 = s ? a : c;
    tmp2 = s ? b : d;
    y = tmp1 + tmp2;
  end
```

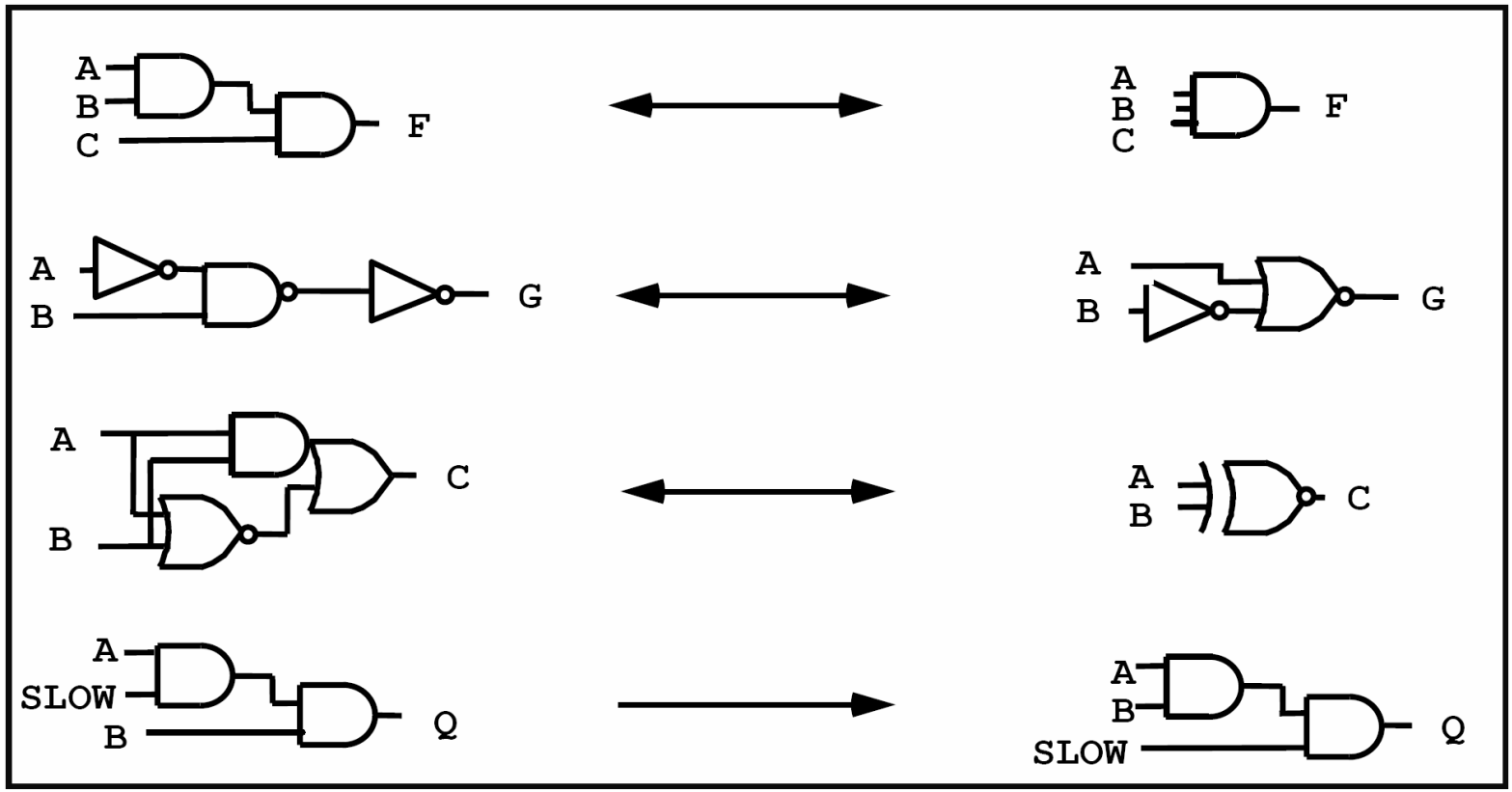
Some synthesis tools automatically share resources.

What if your synthesis tool does not automatically share resources?

You can write your RTL code to do its own resource sharing!

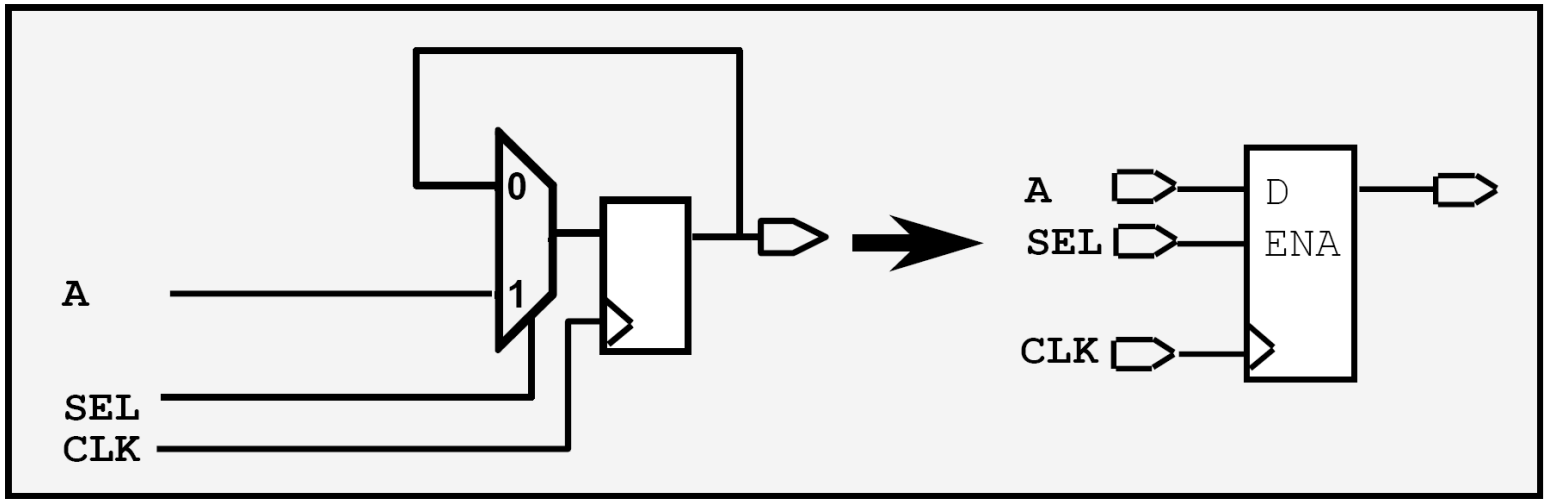
Synthesis Tool Optimization (Target Library)

The process of using gates from the target library to generate a design that meets timing and area goals.



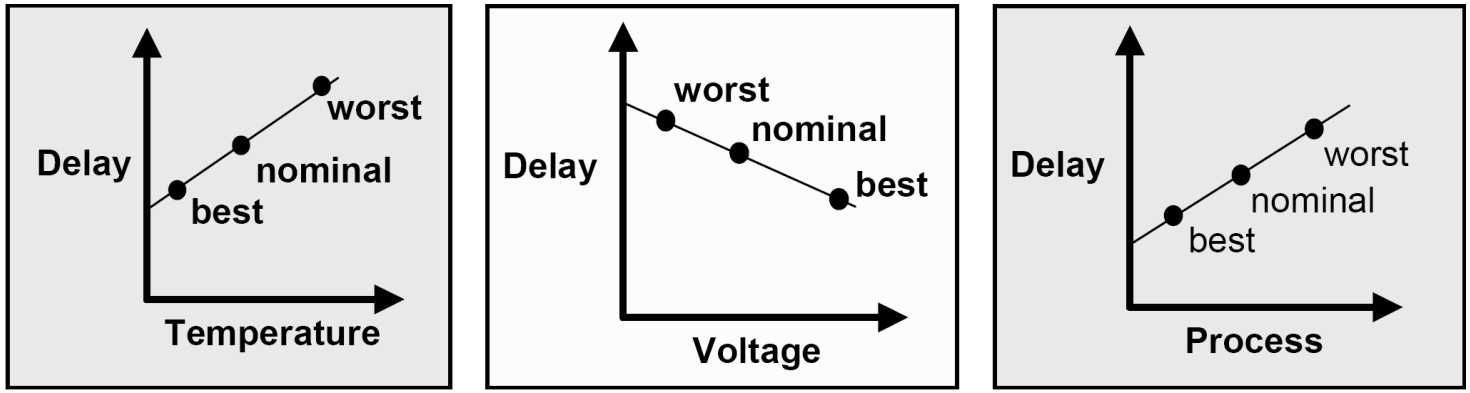
Synthesis Tool Optimization (Target Library)

- The process by which Synthesis tool maps to sequential cells from the technology library:
  - Tries to save speed and area by using a more complex sequential cell
  - May be slower but will reduce area



Synthesis in Design Corners

- Library standard cells are typically characterized under “normal” temperature and voltage
- Vendors allow for synthesis of circuits, which will not operate under “nominal” conditions by embedding operating condition models in the technology libraries



ASIC/FPGA Chip Design

Verification

Mahdi Shabany

Department of Electrical Engineering

Sharif University of technology

Verification

❑ Exponential increase in the complexity of ASIC implies need for sophisticated verification methods to be incorporated in the ASIC design process.

❑ Catch the bug as early as possible

- ➢ So catch it in simulation ➡ saves time and money.
- ➢ Imagine respin of a chip if the same bug is caught in Silicon

❑ **Verification:**

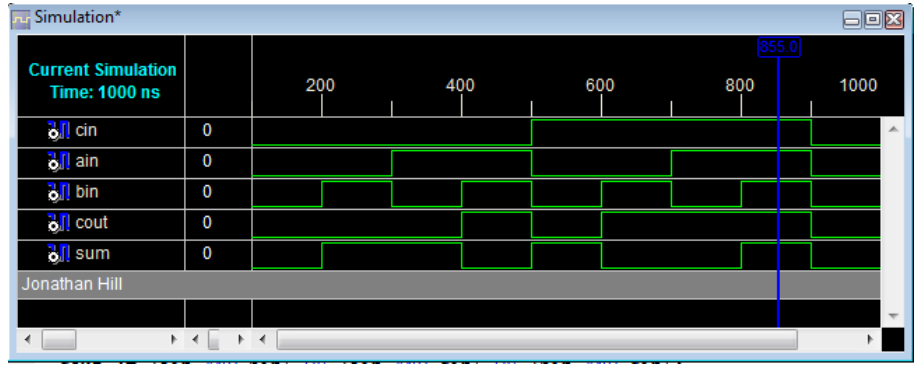
A process that ensures conformance of a design to predefined expectations

Verification

- **Types of Verification:**
  - **Functional:**
    - Functional verification of RTL
  - **Gate-level simulation:**
    - To verify that the synthesized netlist matches the expected functionality
  - **Formal Verification (equivalence checking):**
    - To make sure that the gate level netlist is equivalent to the RTL
  - **Timing Verification:**
    - To verify that the design can run at speed

Functional Verification

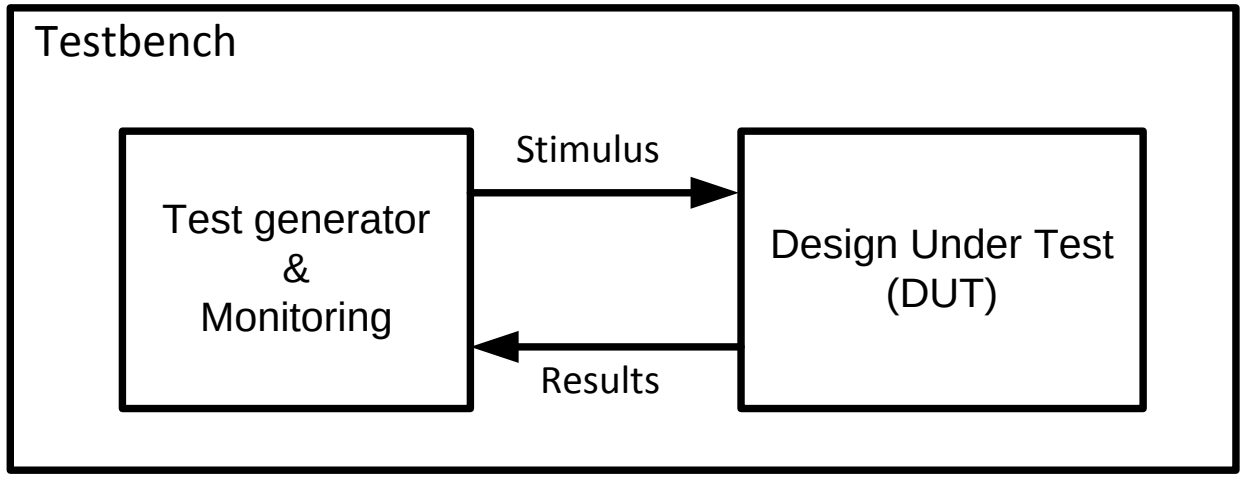
- Two general methods:
  - **Graphical and manually:**
    - Inserting inputs and clock manually and run graphical simulations using CAD tools such as Quartus or ISE
      - Good for simple designs
      - Suitable for early evaluations



  - **Automated:**
    - Writing **testbench** to run the simulation/verification automatically
      - Suitable for large designs
      - Suitable for stress testing

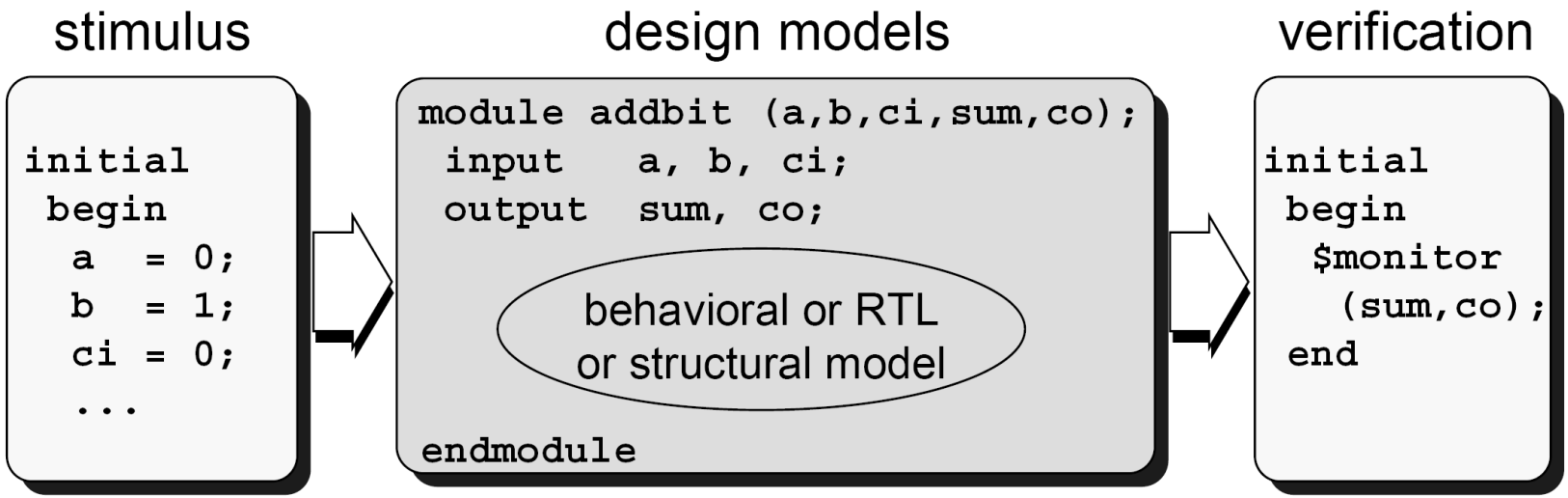
Testbench (Functional Verification)

- **Testbench** is a Verilog module to simulate a Design Under Test (DUT)
- Testbench:
  - A Verilog module
  - Instantiates the DUT
  - Apply stimulus (data) to the DUT
  - Monitors the results to verify that the test was successful (i.e., the output of the DUT conformed to expectations)



Testbench

- The Verilog HDL is used to model a simulation test bench
  - The test bench is a ***module***, which contains:
    - An instance of the top level of the design
    - Procedures to describe the input stimulus
    - Procedures to describe output verification



Testbench (Functional Verification)

- Simulation of Verilog designs takes the following steps:

  - **Compilation & Elaboration:**
    - The simulator reads the design description, processes compiler directives, and builds a data structure that defines the design hierarchy.

  - **Initialization:**
    - The simulator initializes module parameters, storage elements and nets.
    - When simulation starts at time zero, the simulator propagates these changes and executes the statements in each **initial** and **always** block up to a timing control.

  - **Simulation:**
    - The simulator processes events and applies them to the DUT
    - The results are displayed and/or saved for investigation

Verilog Simulation Commands:

- The Verilog includes *compiler directives* and *system tasks* to control the simulation of a Verilog mode
  - **`<directive> compiler directives:**
    - Executed prior to simulation time zero
    - Instructions to simulators on how to compile models
    - Always start with a ` accent grave (the “back tic”)

```
`include    `timescale    `define
```

  - **$<task> system tasks:**
    - Executed during simulation (i.e.: to display values)
    - Used as programming statements
    - Always start with a $ dollar sign

```
$monitor    $display    $time
```

Compiler Directives (Include Files)

Use `**include** files to ensure project-wide consistency of common source.

```
`include "defines.inc"
module clkgen (clk);
output clk; reg clk;
  always begin
      #(`PERIOD/2) clk = 0;
      #(`PERIOD/2) clk = 1;
    end
  initial begin
      #(`TIMEOUT)
      $display("TIMEOUT
ERROR");
      $finish;
    end
endmodule
```

```
// defines.inc
`timescale 1 ns / 10 ps
`define PERIOD 20
`define TIMEOUT 10000000
```

Defines a constant to be used hereafter

Terminates the simulation

Compiler Directives (Time Scale)

- `` `timescale `` is used to define time delay **unit/precision** :

```
`timescale  10ns/1ns
```

Unit/Precision	Delay Specification	Time Delayed	Comments
1 ns/ 1ns	#3	3ns	
10 ns/ 1ns	#3	30ns	Delay is 3*10ns (time unit)
10 ns/ 1ns	#3.2	32ns	3.2 is multiplied by 10
10 ns/ 1ns	#3.36	34ns	
10 ns/ 10ns	#3.2	30ns	
10 ns/ 100ps	#3.748	37.5ns	3.748 is multiplied by 10 and rounded to 2 decimal place
1 ns/ 1ns	#3.22	3ns	

System Tasks ($display)

- **$time:** Returns the current simulation time as a 64-bit unsigned integer
  - The returned value is scaled to the time unit of the module that invoked it
- **$display:** Immediately displays the values of its arguments

```
$display ("format string", argument list)
```

❖Example:

```
$display ("Reset is = %b and Clock is = %b \n", Reset, Clk)
```

```
Reset is = 1 and Clock is = 0
Reset is = 1 and Clock is = 1
Reset is = 0 and Clock is = 0
Reset is = 0 and Clock is = 1
```

System Tasks ($display)

- **$display:** supports multiple default radixes
  - $displayb (binary)
  - $displayd (decimal)
  - $displayh (hexadecimal)
  - $displayo (octal)
- Formatters and Escaped literals can be used inside the format string.
- Formatters can be used to overrides the default radix
  - Formatters:

```
%b  %c  %h  %d  %t  %o  %s
```

  - Escaped literals:

```
\" (double quote)  \n (new line) \\(backslash)  \t(tab)
```

System Tasks ($monitor)

- **$monitor:** to monitor changes in the signal values in its argument list
  - Continuously monitors the variables in its argument list and displays the formatted arguments at the end of each simulation time in which any of signals changes value

```
$monitor ("format string", argument list)
```

- Only one **$monitor** system task can be active. Any subsequent invocation of **$monitor** starts monitoring the new signal arguments and discontinues monitoring the previous signal arguments.
- **$monitoroff** and **$monitoron** system tasks can be used to restrict monitoring to intervals of time, rather than monitoring entire remainder of simulation session.
- The **$monitor** system task accepts the same formatters/argument list as **$display**.

File Open/Close

- The stimulus can be read from a file and then applied to the DUT
- A file can be opened for reading or writing, and the syntax is as below:

```
file = $fopen("filename",r); // For reading
file = $fopen("filename",w); // For writing
```

- All fopen options:

“r” or “rb”	Open for reading
“w” or “wb”	Truncate to zero length or create for writing
“a” or “ab”	Append (open for writing at end of file)
“r+”, “r+b”, or “rb+”	Open for update (reading and writing)
“a+”, “a+b”, or “ab+”	Append; Open or create for update at end-of-file

- A file can be closed as follows:

```
file = $fclose("filename”);
```

Initial Construct

- **Initial:** Same as always except that whatever that is inside it executes only once
  - Used for initialization at the beginning of the testbench
  - Any signal assigned a value inside it should be of type variable (e.g., reg)

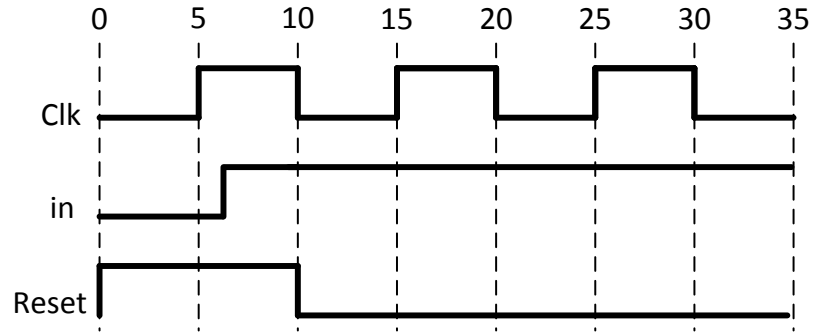
```
module  test ();
 `timescale 1ns/1ns;
  reg Clk, Reset,  in;
  wire Out;

  Moore   DUT(in, Clk, Reset, Out);
  initial
   begin
        Clk = 1'b0; in = 1'b0; Reset = 1'b1;
     end
  initial   #10 Reset = 1'b0;
  always #5 Clk = ~Clk;
  initial  #7 in = 1'b1;
endmodule
```

Requires begin-end for multiple statements

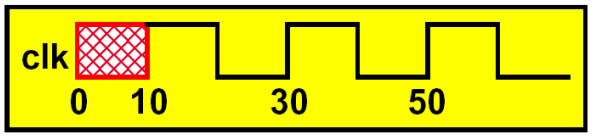
Top module instantiation

A clock with 10ns period

Executes only once (as opposed to always)

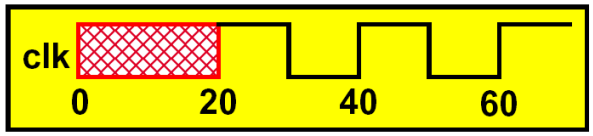


Creating Clock

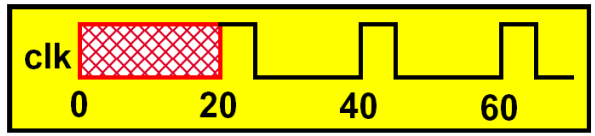



```
`timescale 1ns/1ns;
  reg Clk;
  always
     begin
        # 10 Clk = 1;
        # 10 Clk = 0;
     end
```

```
`timescale 1ns/1ns;
  reg Clk;
  initial   # 10 Clk = 1;
  always  #10 Clk = ~Clk
```




```
`timescale 1ns/1ns;
  reg Clk;
  initial
     begin
        # 20 Clk = 1;
        forever begin
           # 10 Clk = 0;
           # 10 Clk = 1;
        end
     end
```




```
`timescale 1ns/1ns;
  reg Clk;
  initial
     begin
        # 20 Clk = 1;
        forever begin
           # 5 Clk = 0;
           # 15 Clk = 1;
        end
     end
```

Applying Stimulus

- Common stimulus application techniques are:
  - In-line stimulus, applied from an **initial** block
  - Stimulus applied from a loop or **always** block
  - Stimulus applied from an array of vectors or integers

In-Line Stimulus

In-line stimulus has the following characteristics:

- You list the variables only when their values change
- You can easily define complex timing relationships between signals
- The testbench can become very long for tests of real designs

```
module inline_tb;
  wire [7:0] results;
  reg [7:0] data_bus, addr;
  DUT u1 (results, data_bus, addr);
  initial fork
      #10 addr = 8'h01;
      #10 data_bus = 8'h23;
      #20 data_bus = 8'h45;
      #30 addr = 8'h67;
      #30 data_bus = 8'h89;
      #40 data_bus = 8'hAB;
      #45 $finish;
    join
endmodule
```

fork-join construct is used to execute them all in parallel

In-Loop Stimulus

Stimulus applied from a loop has the following characteristics:

- For each iteration you assign a new stimulus vector
- The timing relationships between signals are regular in nature
- The testbench is compact

```
module loop_tb;
  wire [7:0] response;
  reg [7:0] stimulus;
  reg clk;
  integer i;
  DUT u1 (response, stimulus);
  initial clk = 0;
  always begin
      #10 clk = 1;
      #10 clk = 0;
    end
  initial begin
      for (i = 0; i <= 255; i = i + 1)
        @(negedge clk) stimulus = i;
      #20 $finish;
    end
endmodule
```

In-Loop stimulus

Random Stimulus

- **$random:** Returns a 32-bit signed integer random value

```
Initial  repeat (13)   #5   Clk = ~Clk;
Initial  repeat (10)   #6   In = $random;
Initial  forever @(posedge Clk) #3   In2 = $random
```

"In" receives random data 10 times every 6 time units

3 time units after the posedge Clk, In2 gets a new data (No overlapping b/w data and Clk)

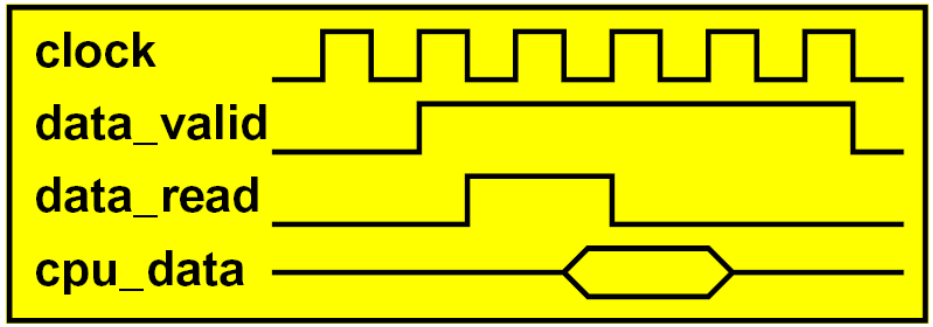
- There are two ways to terminate simulations
  - **$stop:**
    - A stopped simulation can be resumed after termination
  - **$finish:**
    - A finished simulation can NOT be resumed after termination

```
Initial  #180  $stop;
Initial  #180  $finish;
```

Both terminate simulation after 180 time units

Task Construct in Testbench

Use Verilog tasks in your testbench to encapsulate repeated operations.



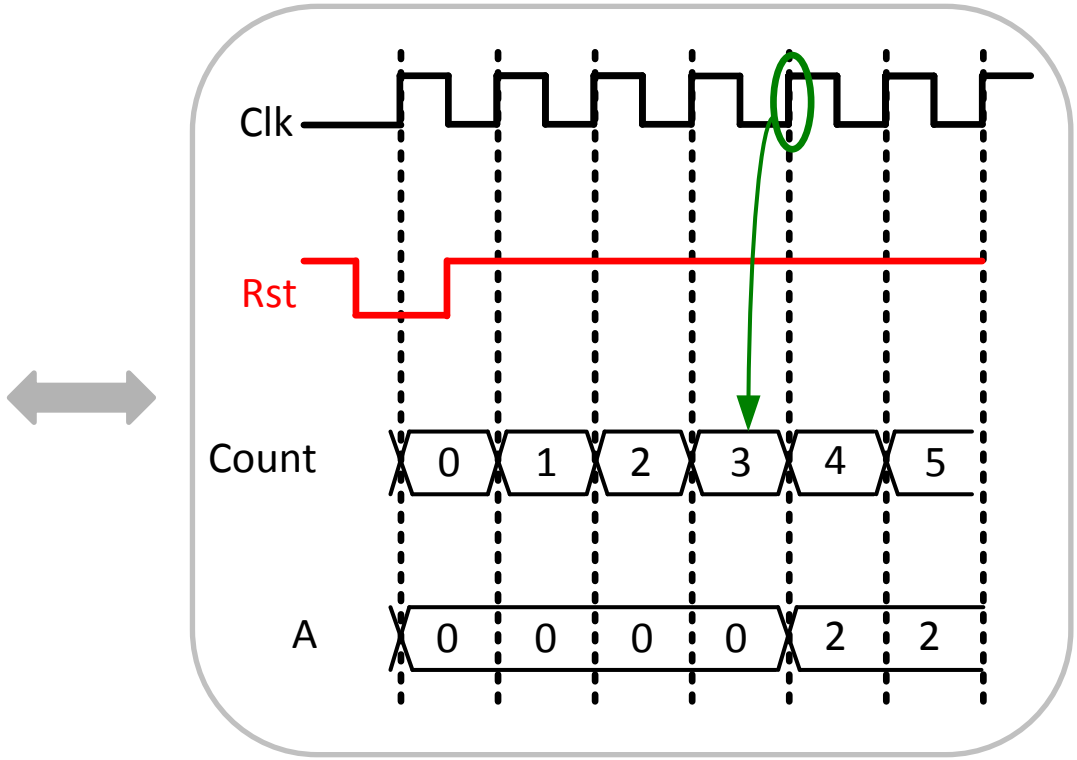

```
task cpu_read;
  begin
    #30 data_valid = 1;
    wait (data_read == 1);
    #20 cpu_data = data_in;
    wait (data_read == 0);
    #20 cpu_data = 8'hzz;
    #30 data_valid = 0;
  end
endtask
```

Clock period: 20 time units

Applying Stimulus

- Make sure you understand the timing transitions:

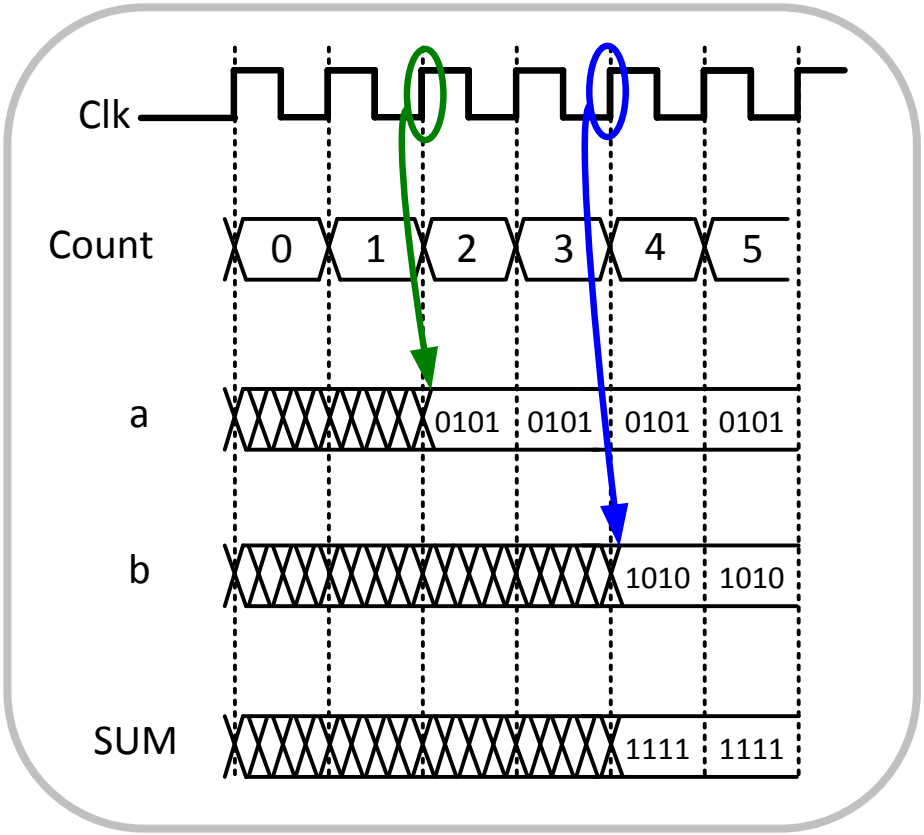
```
reg  [10:0] Count;
reg  [3:0] A;
always @ (posedge Clk)
  begin
   if ( !Rst )
      A <= 4'b0;
   else if ( Count == 3)
      A <= 4'b2;
  end
always @ (posedge Clk)
  begin
   if ( !Rst )
      Count <= 11'b0;
   else
      Count <= Count + 1;
  end
```



Applying Stimulus

- If another block wants to sample “SUM” value, it has to sample it when or after Count is 4.

```
reg  [3:0] a, b;
wire [3:0] SUM;
always @ (posedge Clk)
  begin
    if ( Count == 1 )
       a <= 4’b0101;
    else if ( Count == 3)
       b <= 4’b1010;
  end
assign  SUM = a+b;
```



Applying Stimulus

❖ Example:

```
module Counter(clk, reset, out);
input clk, reset;
output [2:0] out;
reg [2:0] count;

always @ (posedge clk)
begin
if (reset)
 count <= 0;
 else if (count == 3'b111)
  count <= 3'b000;
  else
  count <= count +3'b1;
end
assign out = count;
endmodule
```

Module

```
`timescale 1 ps/ 1 ps
module Counter_tb();
reg clk, reset;
wire [2:0] out;

Counter i1 (.clk(clk),.out(out),.reset(reset));
initial
        clk=0;
always
begin
        #5 clk = 1;
        #5 clk = 0;
        $monitor("Reset is = %b and Clock is =
        %b \n", Reset, Clk);
end
endmodule
```

Testbench

File Reading for Verification

- **$readmemb:** Reads a file in binary and stores it in a reg variable
  - **$readmemh** :reads in in hex

```
$readmemb ("file name", reg_name, [start_add,[finish_add]]);
```

```
module  Count_tb ();
 `timescale 1ns/1ns;
  reg Clk, Reset;
  wire Out;
  integer indx;
  parameter N = 100;
  reg [1:0] InReg [N-1:0];
  reg [1:0] DummyReg;

  test  DUT(Clk, Reset, Out);
```

```
initial
      begin
          indx = 0;
          $readmemb("TB.txt", InReg);
          $display("Running Testbench");
          repeat   (N)
             begin
                    DummyReg = InReg[indx];
                     Clk = DummyReg[0];
                     Reset = DummyReg[1];
                     #5 index = indx + 1;
             end
          $monitor("Output is %b", Out);
       end
endmodule
```

Applying Stimulus

❖ Example:

Module

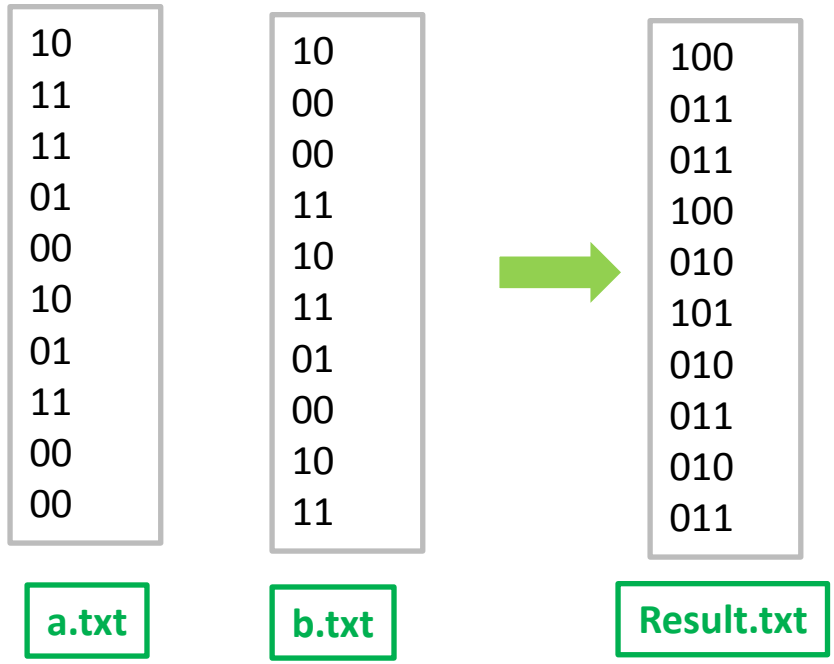
```
`timescale 1ns / 1ps
module adder(a,b,c);
input [1:0] a;
input [1:0] b;
output [2:0] c;

assign c=a+b;

endmodule
```

Testbench

```
`timescale 1ns / 1ps
module TestBench();
reg Clk;
reg [1:0] aa;// aa and bb can be n-bit variables
reg [1:0] bb;
wire [2:0] out;
// Pointers should be defined as integer
integer op1, op2, op_out, k, j;
initial // Opening files for read and write
begin
Clk=0;
op1=$fopen ("a.txt","r");
op2=$fopen ("b.txt","r");
op_out=$fopen ("Result.txt","w");
 end
always     #10 Clk<=~Clk;
 always @(posedge Clk)
begin
k <= $fscanf (op1, "%b \n", aa);
j <= $fscanf (op2, "%b \n",bb);
end
adder add1(.a(aa),.b(bb),.c(out));
always @(posedge Clk) begin
 $fwrite (op_out,"%b \n",out);
end
endmodule
```

Applying Stimulus

- Example:



Example: Code (4-bit Ripple-Carry Counter)

```
module ripple_carry_counter (q, clk,
reset);
output [3:0] q;
input clk, reset;
T_FF tff0 (q[0], clk, reset);
T_FF tff1 (q[1], q[0], reset);
T_FF tff2 (q[2], q[1], reset);
T_FF tff3 (q[3], q[2], reset);
endmodule

module T_FF (q, clk, reset);
                output q;
input clk, reset;
wire d;

D_FF dff0 (q, d, clk, reset);
not n1 (d, q);  //not is Verilog provided
primitive
endmodule

// module D_FF with synchronous
reset
```

Module

```
module D_FF (q, d, clk, reset);
output q;
input d, clk, reset;
req q;
always @(posedge reset or negedge clk)
                if (reset)
                                q = 1,b0;
//module D_FF with synchronous reset
                else
                                q = d;
endmodule;
```

Example: Testbench

```
module stimulus;
req clk;
req reset;
wire [3:0] q;
 //instantiate the design block;
Ripple_carry_counter r1(q, clk, reset);

 //control the clk signal that drives the design block  //cycle time = 10
initial
                clk = 1'b0; //set clk to 0
always
                #5 clk = ~clk; //toggle clk every 5 time units

//control the clk signal that drives the design block
//reset the arrested from 0 to 20 and from 200 to 220.
initial
begin
reset = 1'b1;
#15 reset = 1'b0;
#180 reset = 1'b1;
#10 reset = 1'b0;
#20 $finish; //terminate the simulation
end
```

```
 //monitor the outputs
initial
Smonitor($time, "output q = %d", q);
endmodule
```

Testbench

Example : Results

```
0 output q = 0
20 output q = 1
30 output q = 2
40 output q = 3
50 output q = 4
60 output q = 5
70 output q = 6
80 output q = 7
90 output q = 8
100 output q = 9
110 output q = 10
120 output q = 11
130 output q = 12
140 output q = 13
150 output q = 14
160 output q = 15
170 output q = 0
180 output q = 1
190 output q = 2
195 output q = 0
210 output q = 1
220 output q = 2
```

Result

Simulation Tool: Modelsim

- We use Modelsim as the simulation tool for our testbenches
- Modelsim can also compile the design
- The tutorial for Modelsim should be read and done with the files provided on the course website.

ASIC & FPGA Chip Design:

FPGA Architectures

Mahdi Shabany

Department of Electrical Engineering
Sharif University of technology

Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

Outline

- **Introduction**
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

Introduction: Digital System Design

- To design digital systems there are three options:
  - **Microprocessors and DSP** [software-based]
    - Fetch & execute software instructions (e.g., running a word processing program)
    - Very efficient for complex sequential math-intensive tasks
    - Slow & Power hungry

Course Focus

  - **Programmable Logic devices (PLDs)** [Hardware-based]
    - Directly implements logic functions on hardware
    - Faster
    - Less power consumption
  - **Application Specific Integrated Circuit (ASIC)** [Hardware-Based]
    - Fastest
    - Lowest power consumption

Introduction: Digital System Design

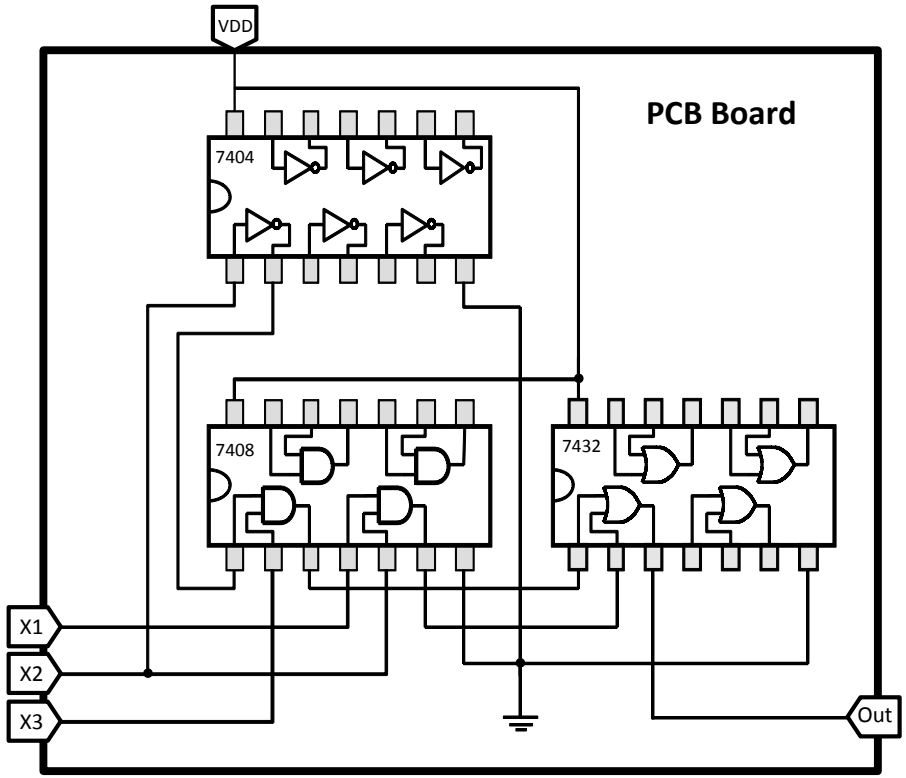
- **DSP** [software-based]
  - Easy to program (usually standard C)
  - Very efficient for complex sequential math-intensive tasks
  - Fixed data path-width. Ex: 24-bit adder, is not efficient for 5-bit addition
  - Limited resources

- **FPGA & ASIC** [Hardware-based]
  - Requires HDL language programming
  - Efficient for highly parallel applications
  - Efficient for bit-level operations
  - Large number of gates and resources
  - Does not support floating point, must construct your own

Introduction: Digital System Design

I. Conventional Approach:

- Board-based designs
  - Large # of chips (containing basic logic gates) on a single Printed Circuit Board (PCB)

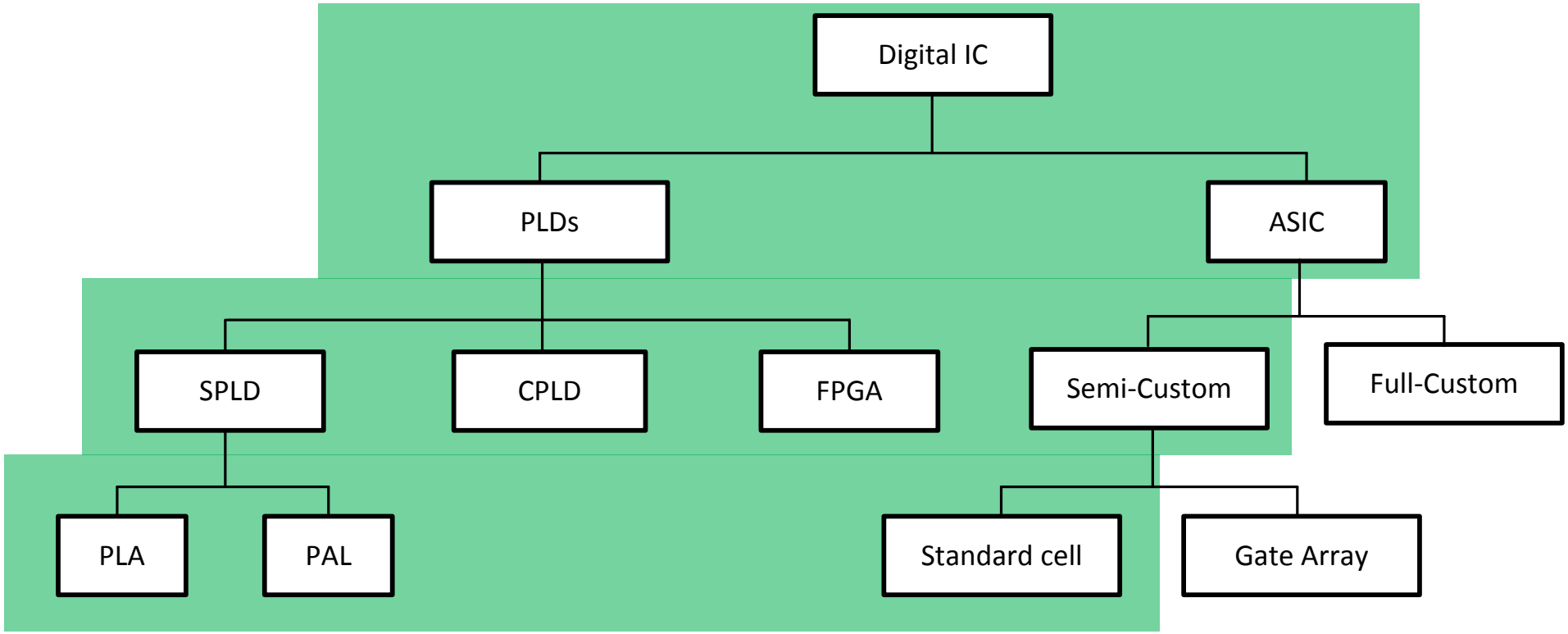


Introduction: Digital System Design

II. High-density Single Chip

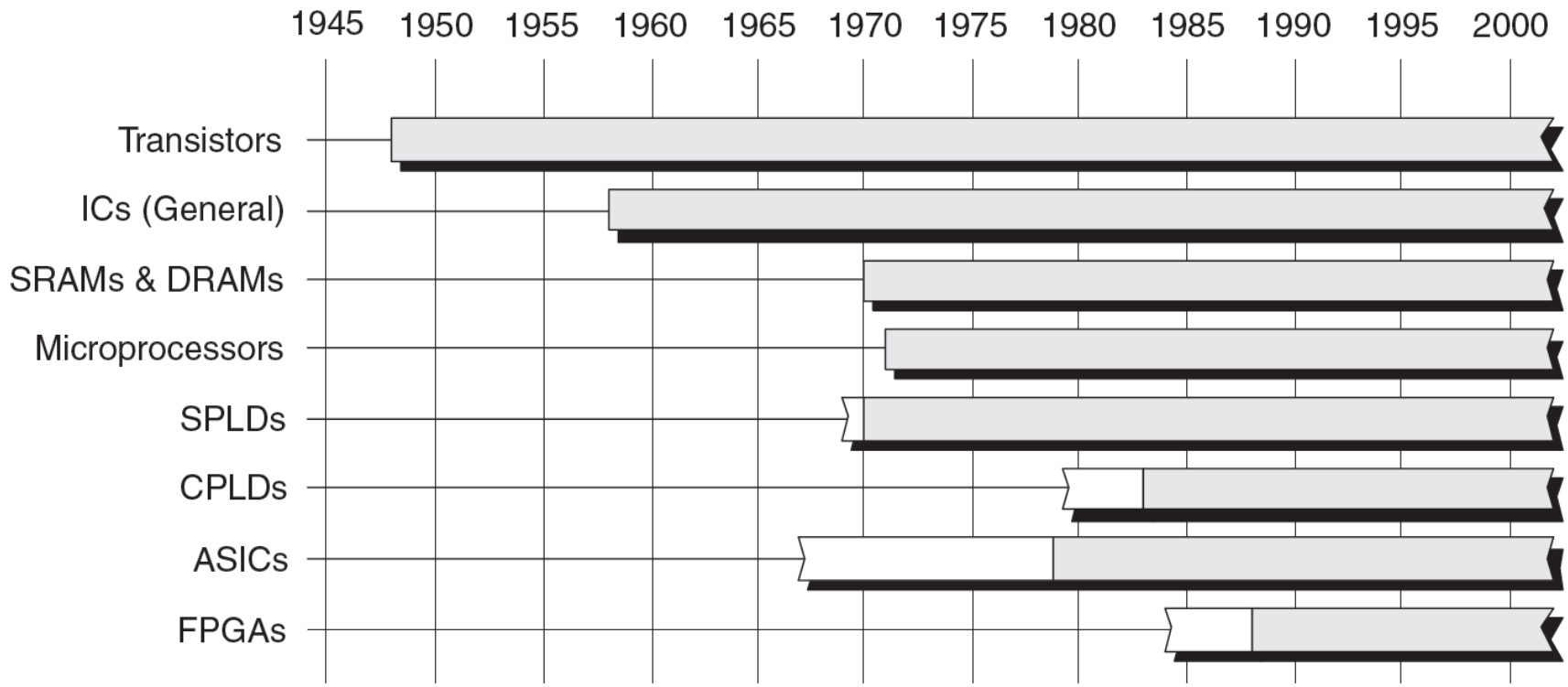
- A single chip replaces the whole multi-chip design on PCB
- Programmable Logic Designs (PLDs) or
- Application Specific Integrated Circuits (ASICs)
  - Lower overall cost
  - On-chip interconnects are many times faster than off-chip wires
  - Lower area with the same functionality
  - Lower power consumption
  - Lower noise

Programmable Logic Designs (PLDs)



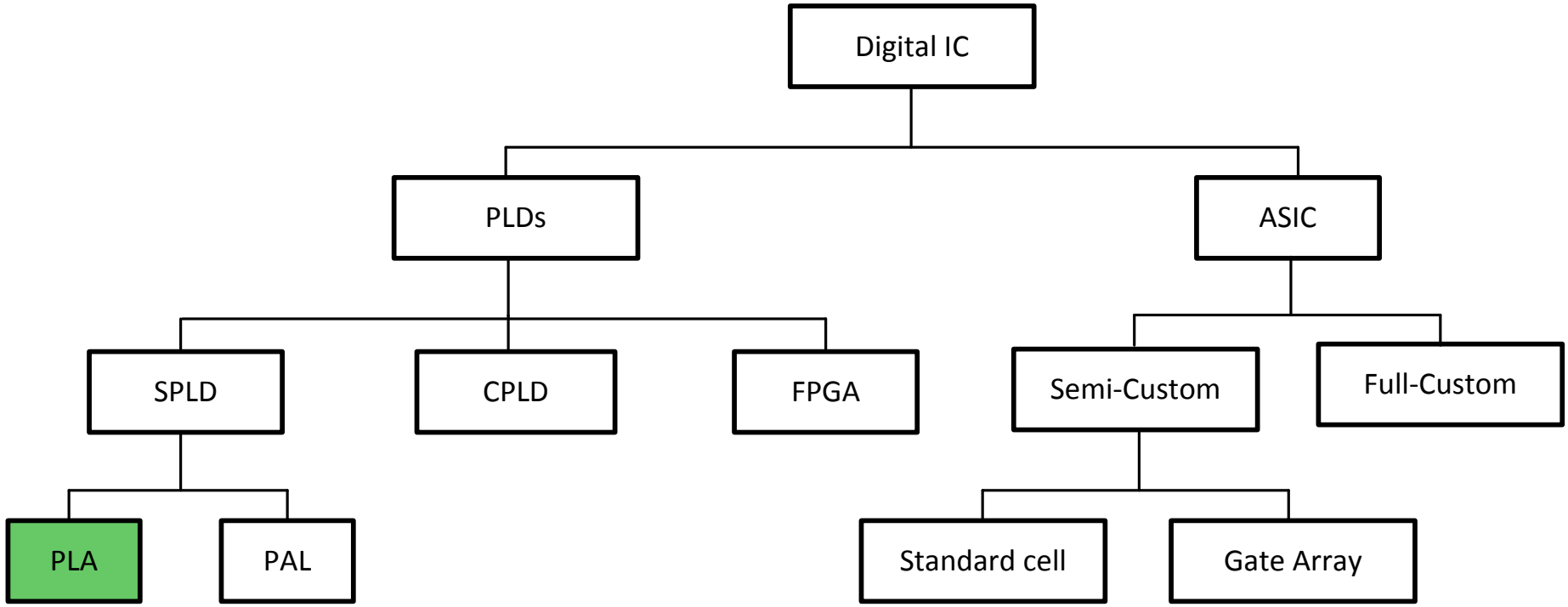
This course

Technology Timeline



The white portions of the timeline bars indicate that although early incarnations of these technologies may have been available, they weren't enthusiastically received by the engineers working in the trenches during this period. For example, although Xilinx introduced the world's first FPGA as early as 1984, design engineers didn't really start using it until the early 1990s.

Programmable Logic Designs (PLDs)



Outline

- Introduction
- **Simple Programmable Logic Designs (SPLDs)**
  - **PLA**
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

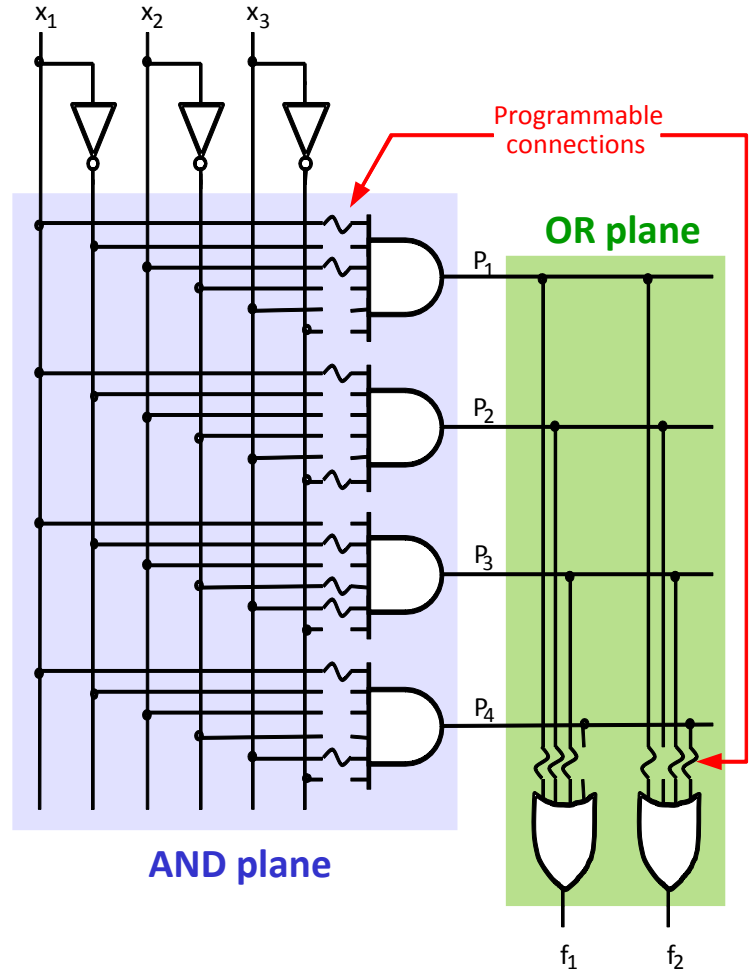
Simple Programmable Logic Designs (SPLDs)

❑ Field Programmable Logic Arrays (FPLA or PLA)

- ➢ Introduced in early 1970s by Philips
- ➢ Consists of two levels of logic gates
  - ▪ Programmable "wired" **AND**-plane
  - ▪ Programmable "wired" **OR**-plane
- ➢ Two levels of programmability
- ➢ Well-suited for implementing functions in sum-of-product (SOP) form.

$$f_1 = x_1x_2 + x_1\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$$

$$f_2 = x_1x_2 + x_1x_3 + \bar{x}_1\bar{x}_2x_3$$

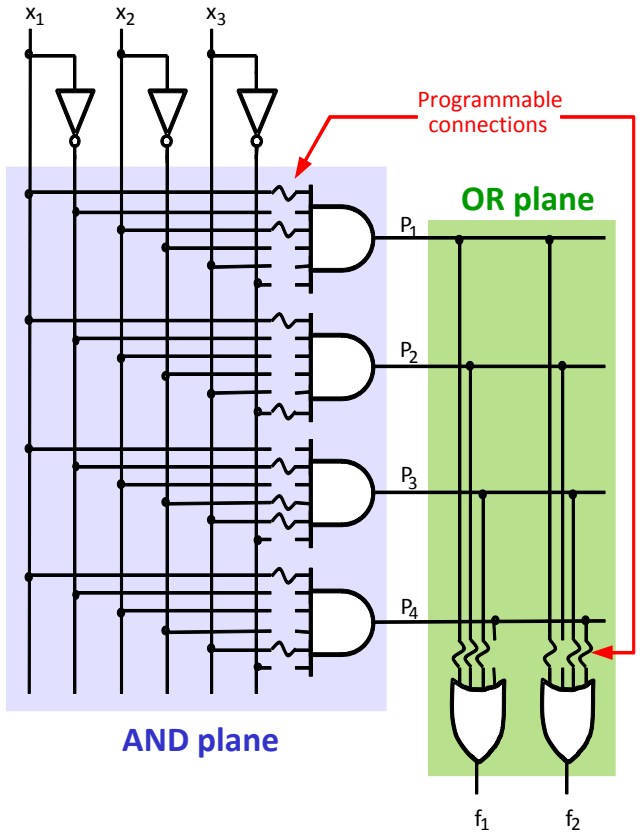


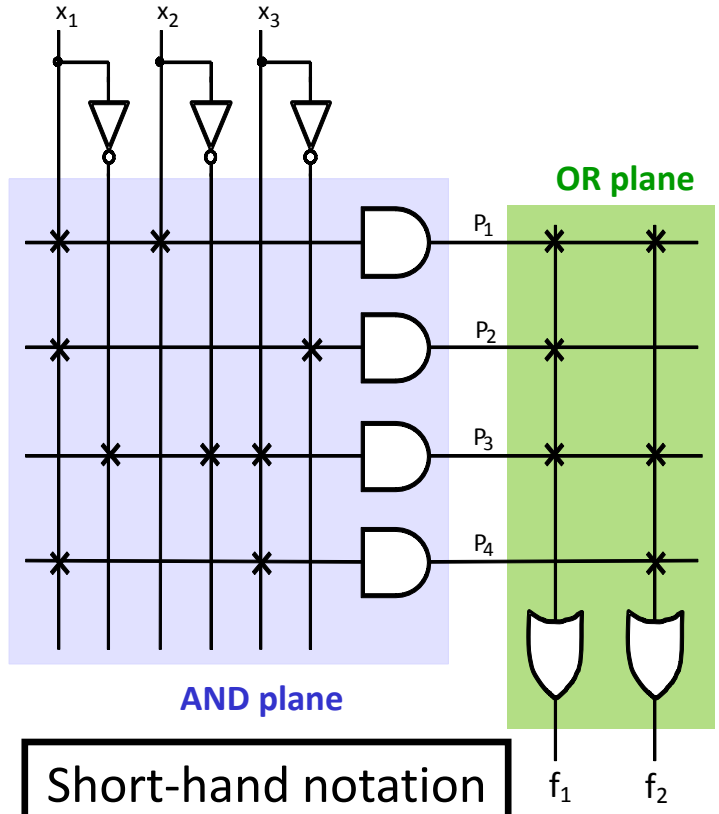
SPLD: Programmable Logic Arrays (PLA)

- Each "AND" gate or "OR" gate can have many inputs
  - **Wide** AND/OR gates

$$f_1 = x_1x_2 + x_1\bar{x}_3 + \bar{x}_1\bar{x}_2x_3$$
$$f_2 = x_1x_2 + x_1x_3 + \bar{x}_1\bar{x}_2x_3$$

Unwanted connections are "blown"





Short-hand notation

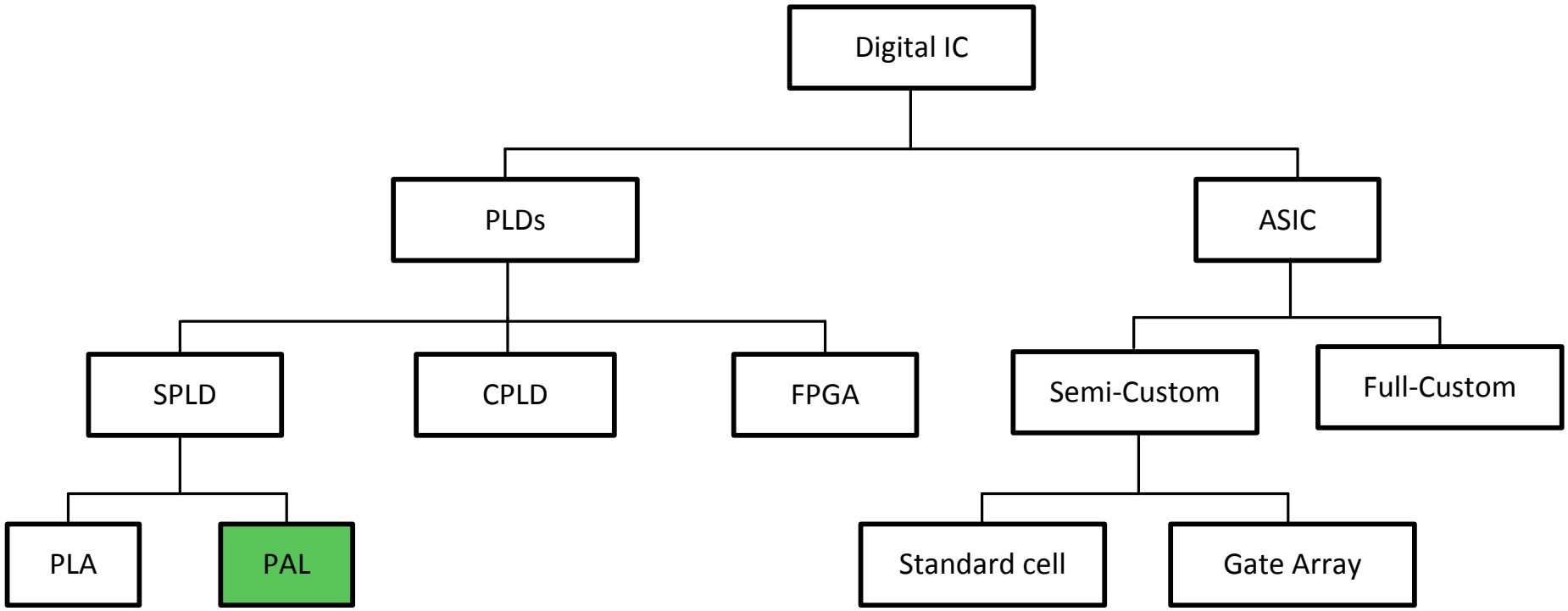
SPLD: PLAs

- **Advantages:**
  - PLA is efficient in terms of its required area for its implementation on IC
  - Often used as part of larger chips, e.g., microprocessors

- **Drawbacks:**
  - Two-level programmable logic planes are difficult to fabricate
  - Two-level programmable structure introduces significant propagation delay
  - Normally many pins, large package thus, high fabrication cost

To overcome these drawbacks, PAL was introduced

Programmable Logic Designs (PLDs)



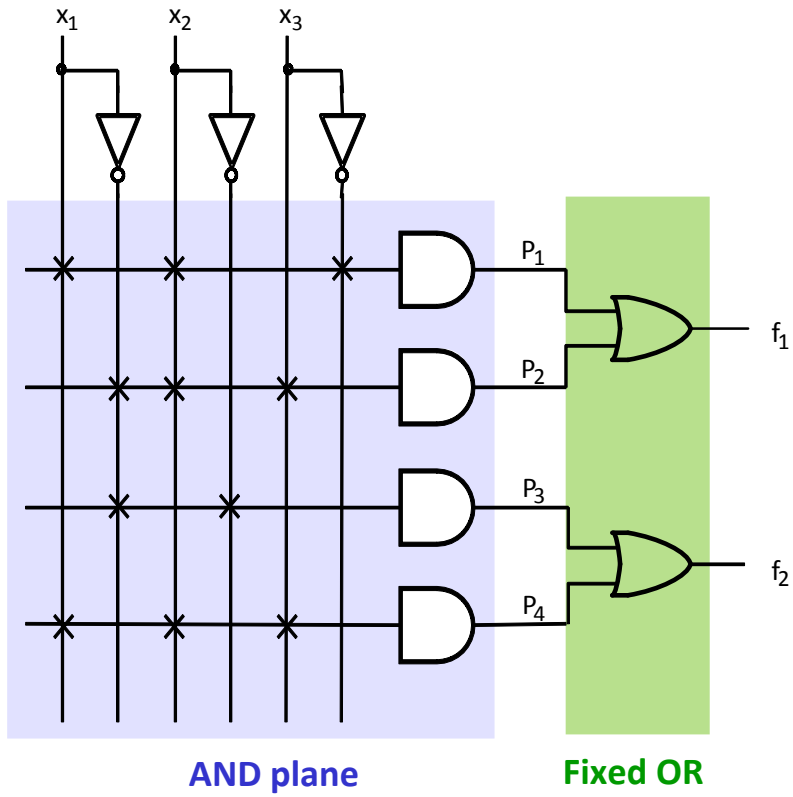
Outline

- Introduction
- **Simple Programmable Logic Designs (SPLDs)**
  - PLA
  - **PAL**
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

SPLD: Programmable Array Logic (PAL)

- **PAL:**
  - Consists of two levels of logic gates
    - Programmable "wired" **AND**-plane
    - Fixed **OR**-gates
  - Single level of programmability

- **Advantages**:
  - Simpler to fabricate
  - Better performance
- **Drawbacks**:
  - Less flexibility

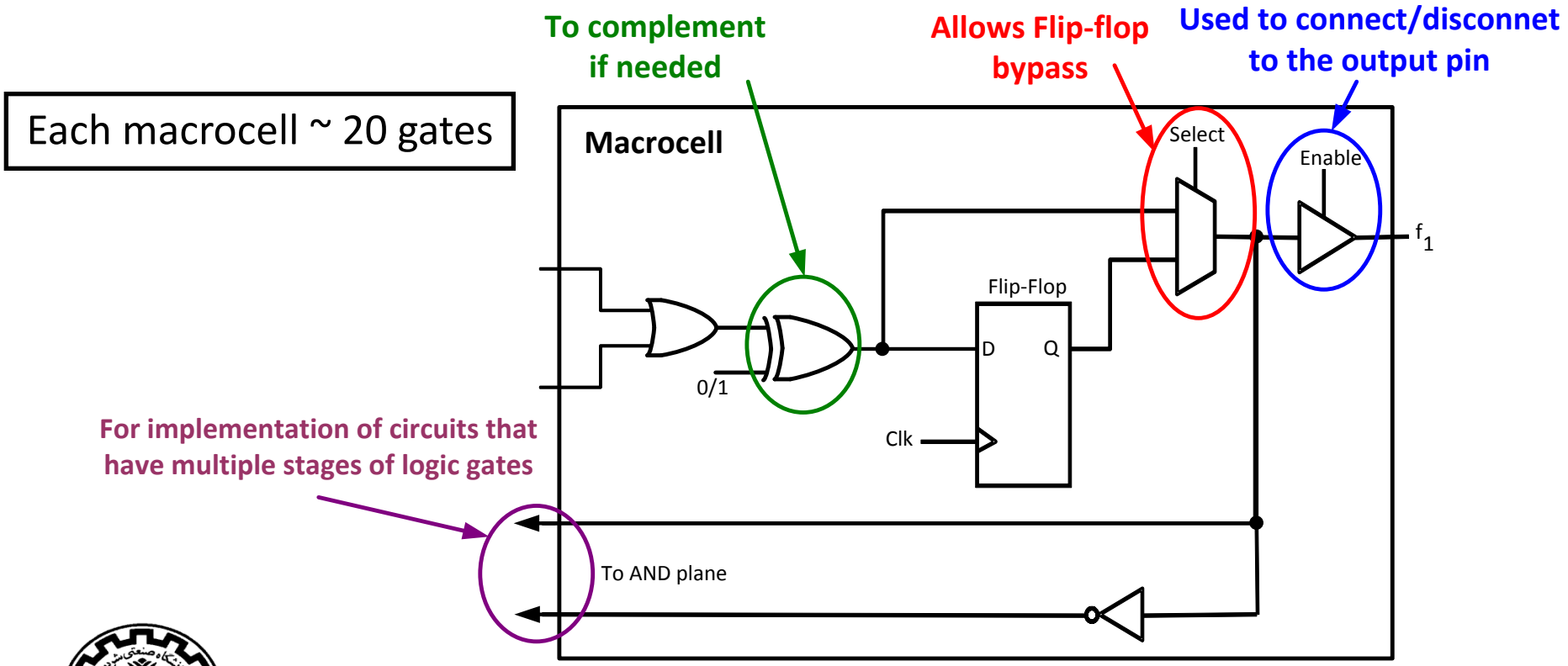


$$f_1 = \bar{x}_1 x_2 x_3 + x_1 x_2 \bar{x}_3$$

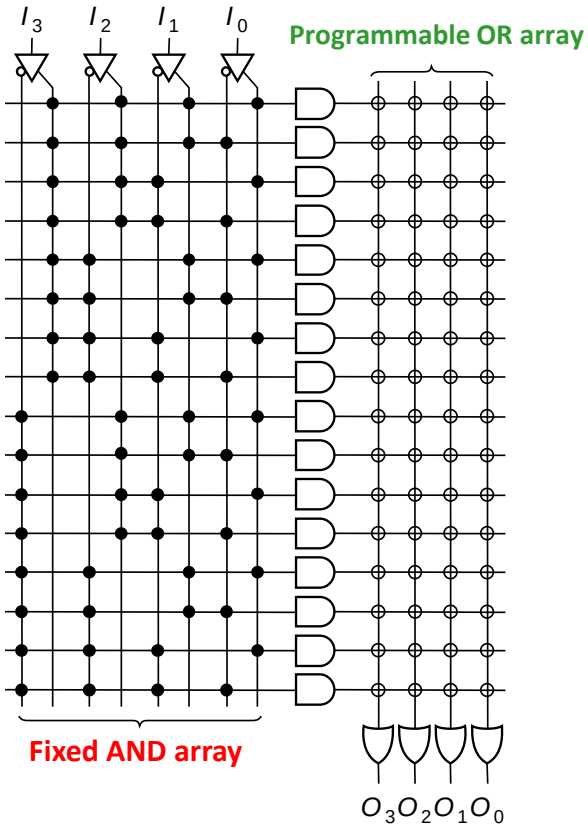
$$f_2 = x_1 x_2 x_3 + \bar{x}_1 \bar{x}_2$$

SPLD: Programmable Array Logic (PAL)

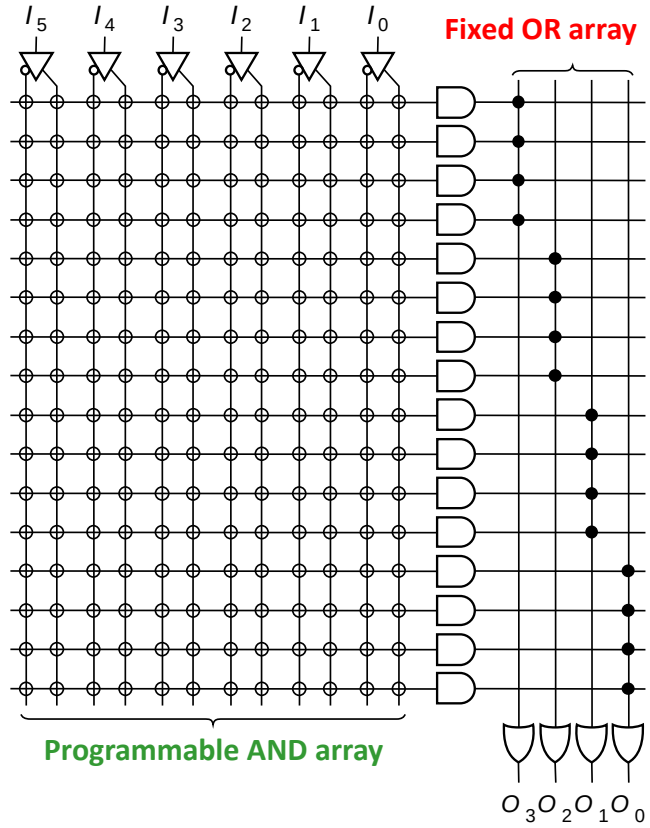
- **To increase flexibility:**
  - PALs with various sizes of OR-gates.
  - Add extra circuitry to the OR-gate output (Called “Macrocell”)



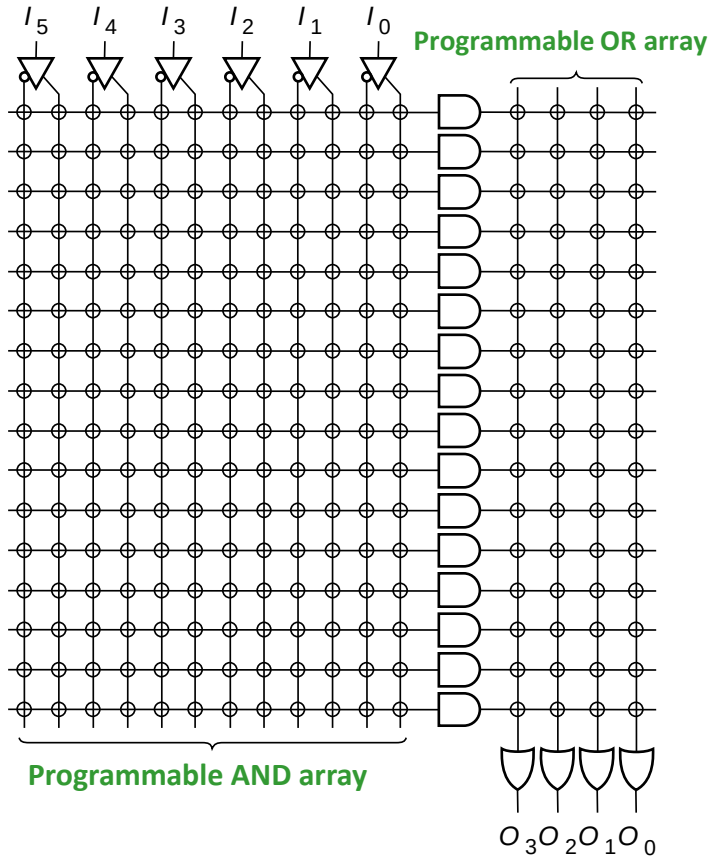
PAL vs. PLA vs. ROM



PROM



PAL



PLA

Indicates programmable connection

Indicates fixed connection

© M. Shabany, ASIC/FPGA Chip Design

Commercial SPLD Products

- Commercial SPLD Products:

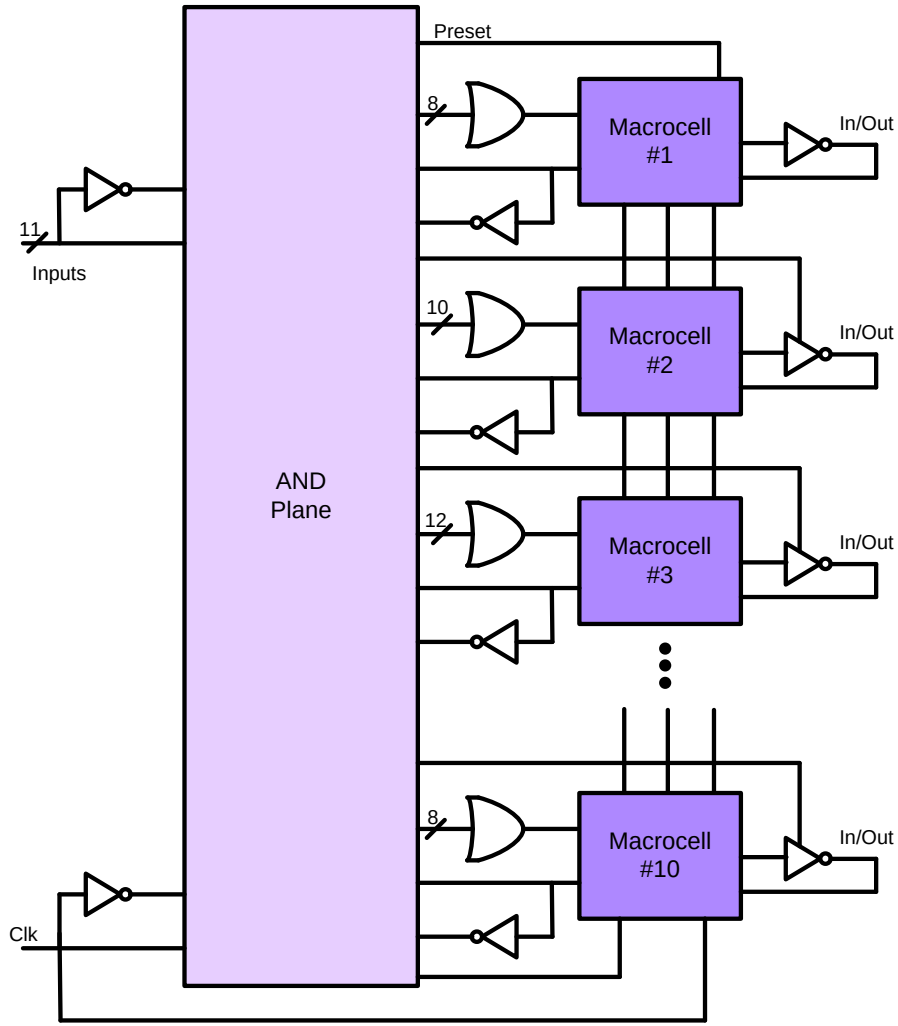
Manufacturer	Product
Altera	Classic
Atmel	PAL
Lattice	ispGAL

- Part number: NN X MM – S
  - NN: Max # of inputs
  - MM: Max # of outputs (some can be used as inputs)
  - X=R (outputs are registered by a D-FF)
  - X=V (Volatile)
  - S: Speed grade

Example:
- 22 V 10-1
- 16 R 8-2

PAL: 22V10 (Lattice Semiconductors)

- Maximum of 22 inputs
  - 11 inputs, one clock, 10 in/outs
- 10 inputs/outputs
- Variable OR gates (8 to 16 inputs)



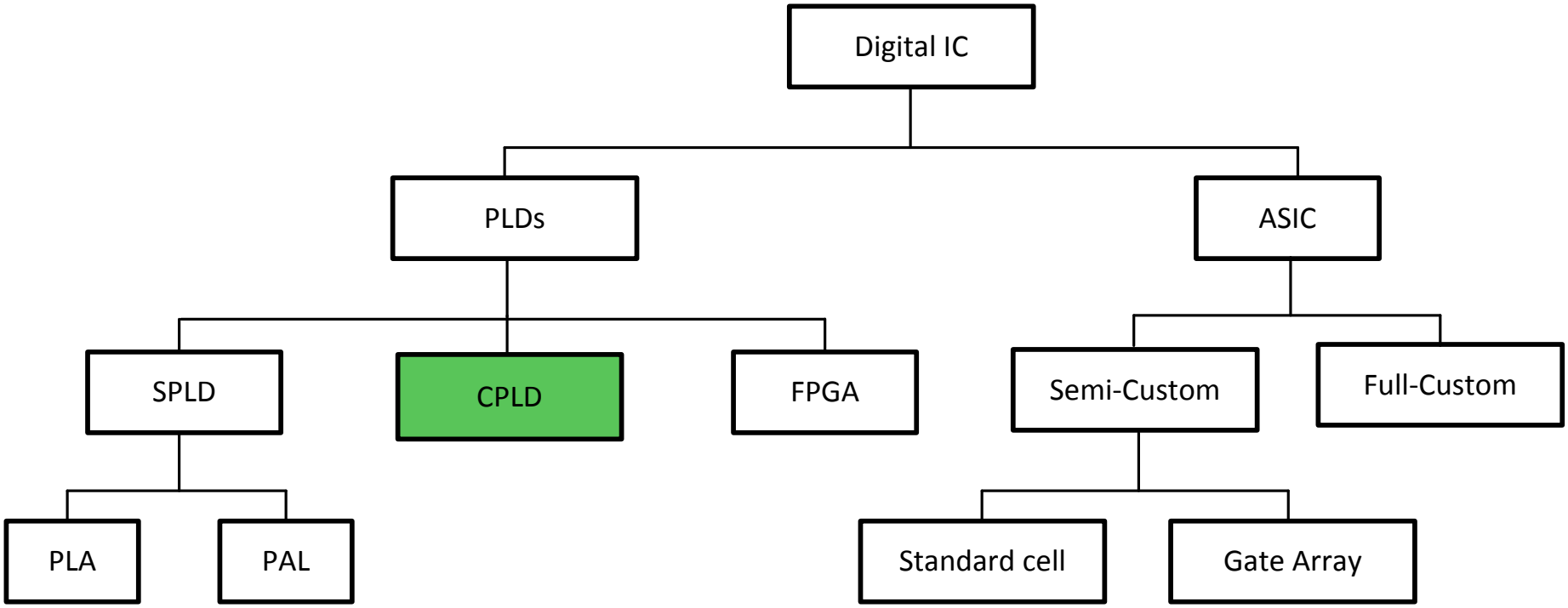
SPLD Scalability

- It is very hard to scale SPLDs for more complex designs
  - b/c the structure of the logic planes grow too quickly in size as the # of inputs increases

- **Solution:**
  - Integrate multiple SPLDs onto a single chip
  - Plus internal programmable interconnect to connect them together

→ **Complex PLDs (CPLD)**

Programmable Logic Designs (PLDs)

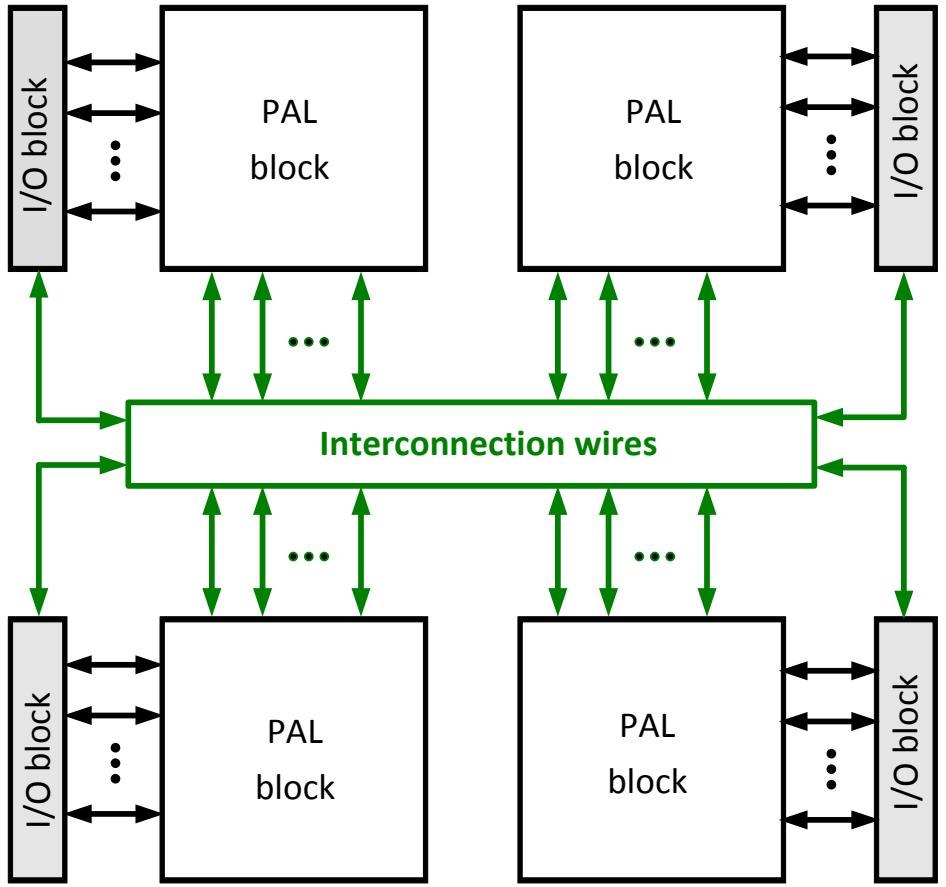


Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- **Complex Programmable Logic Designs (CPLDs)**
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

CPLD

- Consists of 2 to 100 PAL blocks
- Interconnection contains programmable switches
- The number of switches is critical

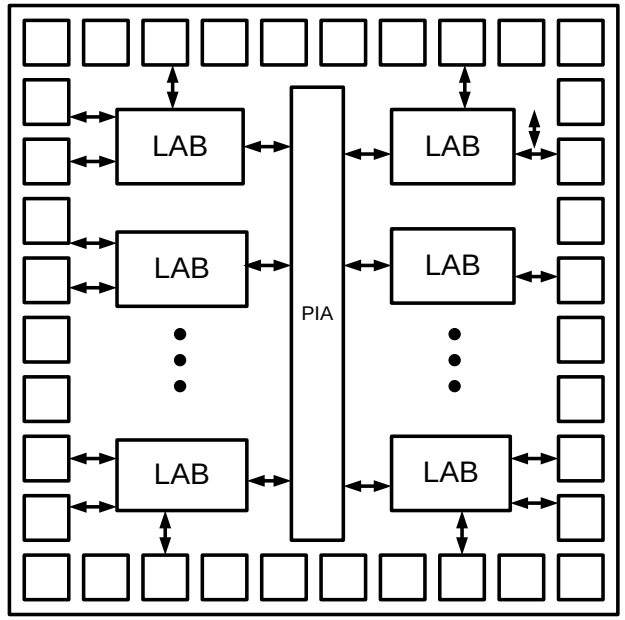


Commercial CPLDs:

Manufacturer	Product
Altera	MAX 7000, MAX 10K
Atmel	ATF
Xilinx	XC9500
AMD	Mach series
ICT	PEELArray
Lattice	ispLSI series

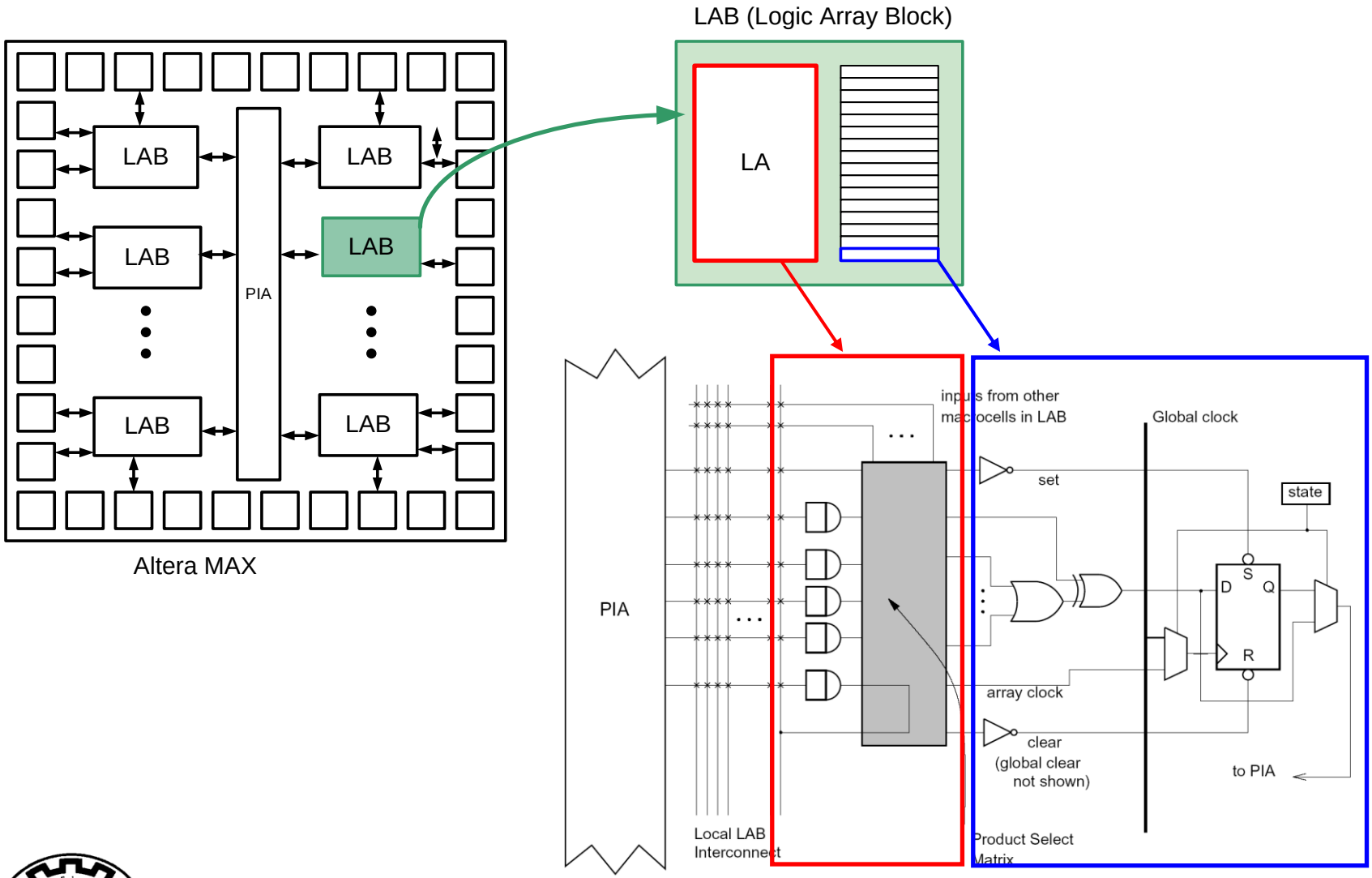
CPLD: Altera MAX7000

- Comprises:
  - Several Logic Array Blocks (LAB), a set of 16 macrocells
  - Programmable Interconnect Array (PIA)
    - Consists of set of wires that span the entire device
    - Makes connections between macrocells and chip's input/output pins
- In total consists of 32 to 512 macrocells
- Four dedicated input pins
  - For global clock or FF resets



Altera MAX 7000

CPLD: Altera MAX7000



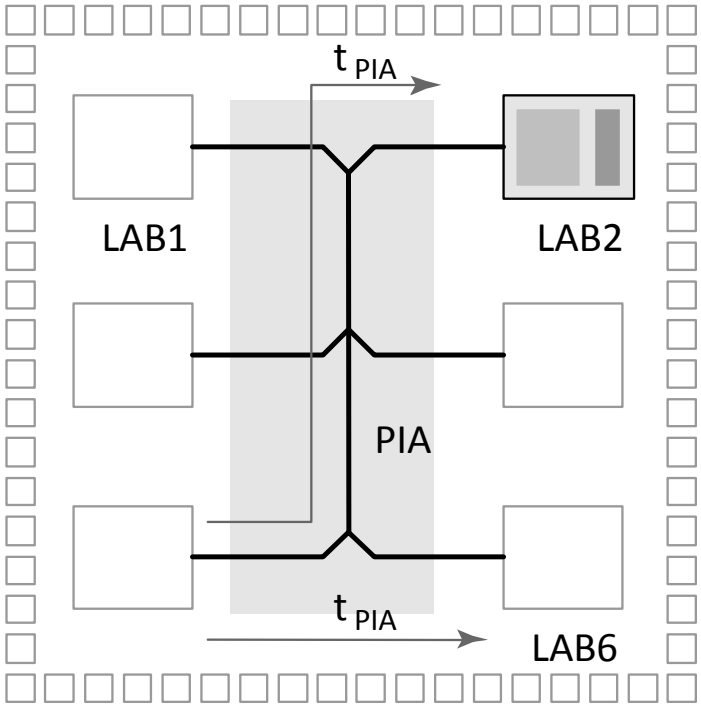
CPLD: Altera MAX7000

- Comprises:
  - Wide programmable AND array followed by
  - A narrow fixed OR array

- OR gate can be fed from:
  - Any of the five product terms within the macrocell
  - or up to 15 extra product terms from other macrocells in the same LAB

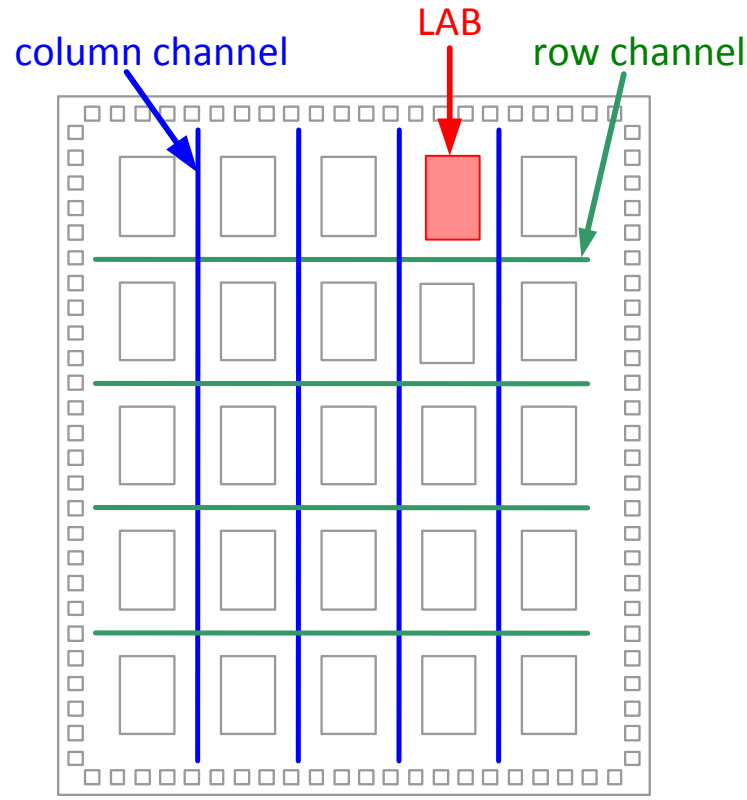
  ⇒ more flexibility

CPLD: Altera MAX7000 Interconnect Architecture



Array-based (MAX 3000, 7000)

- ❑ Fixed routing delay b/w blocks
- ❑ Simple and predictable delay
- ❑ Not scalable to large # of macrocells

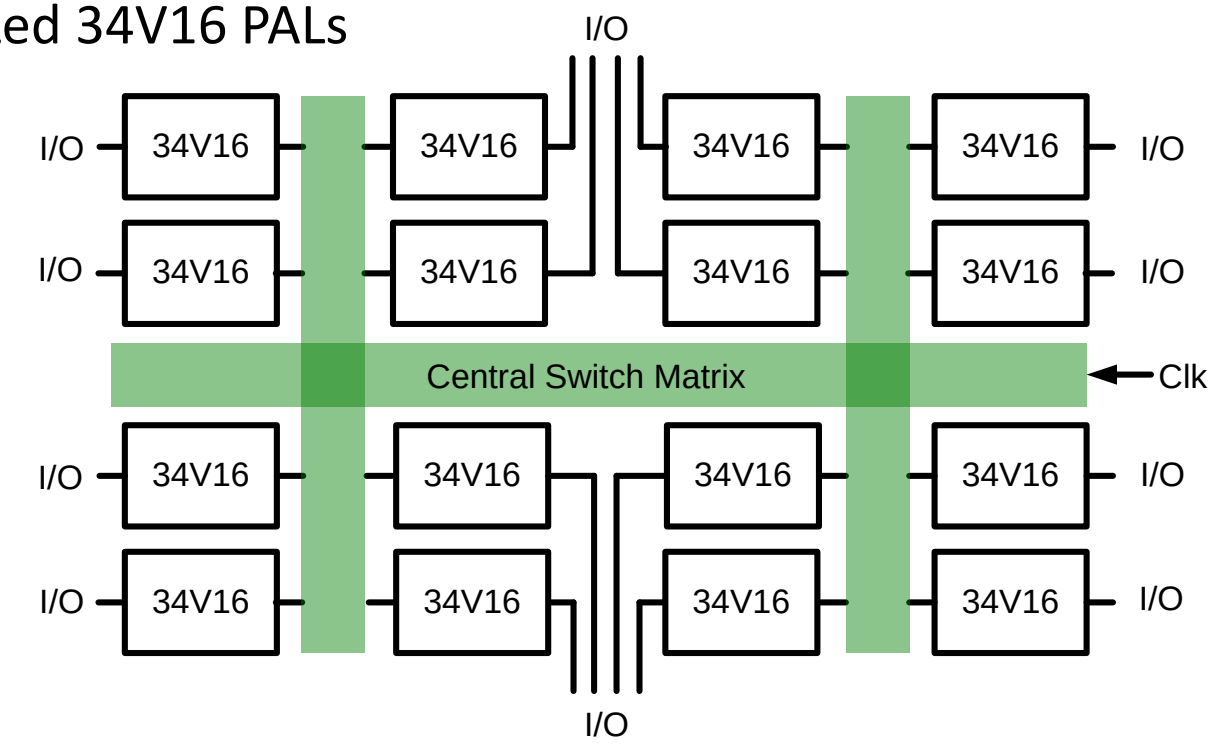


Mesh-based (MAX 9000, 10K)

- ❑ LABs can connect to row and column channels
- ❑ Suitable for large # of macrocells (512)

Advanced Micro Devices (AMD) CPLDs:

- Mach family (Mach 1 to Mach 5) all EEPROM-based technology
  - Mach 1, 2: Multiple 22V16 PALs
  - Mach 3, 4, 5: Several optimized 34V16 PALs
- **Mach 4:**
  - Consists of:
    - 6 to 16 PAL (2K-5K gates)
    - Central switch matrix
  - In-circuit programmable

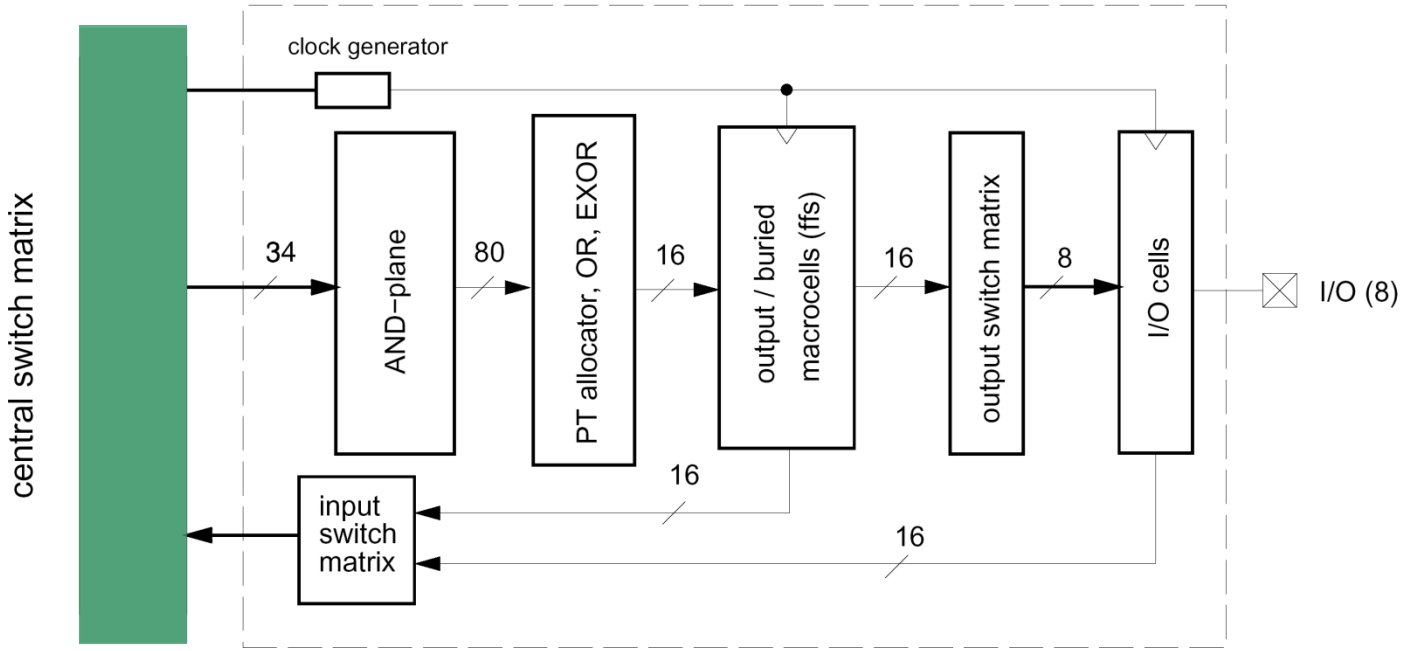


All connections b/w PALs and even inside a PAL routed through the central switch matrix

AMD Mach 4 PAL Block:

- 34V16 (34 maximum inputs, volatile, max 16 outputs)
- In addition to a normal PAL, it consists of:
  - product term allocator b/w AND plane and macrocells, which distributes product terms to whichever OR-gate required
  - Output switch matrix b/w OR gates and I/O

Any macrocell can drive any of the I/O pins (more flexibility)



CPLD Applications:

- Circuits that can exploit wide AND/OR gates and do not need large number of flip-flops
  - Graphic controllers
  - LAN controllers
  - UARTs
  - Cache control

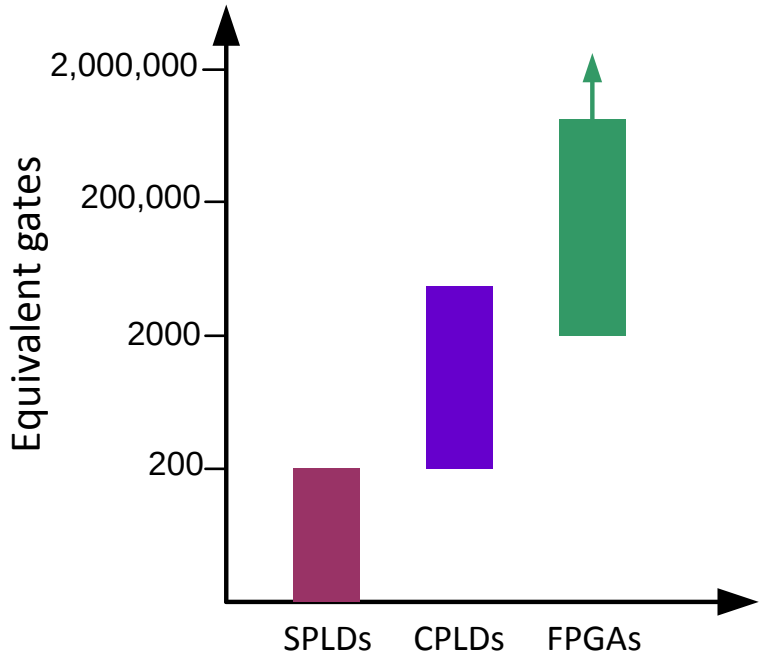
- **Advantages:**
  - Easy to re-program even in-system
  - Predictability of circuit implementation
  - High-speed implementation

Circuit Size Metric

❑ Size Metric:

- ➢ How many basic gates can be built on the circuit
- ➢ Common measure: number of two-input NAND gates

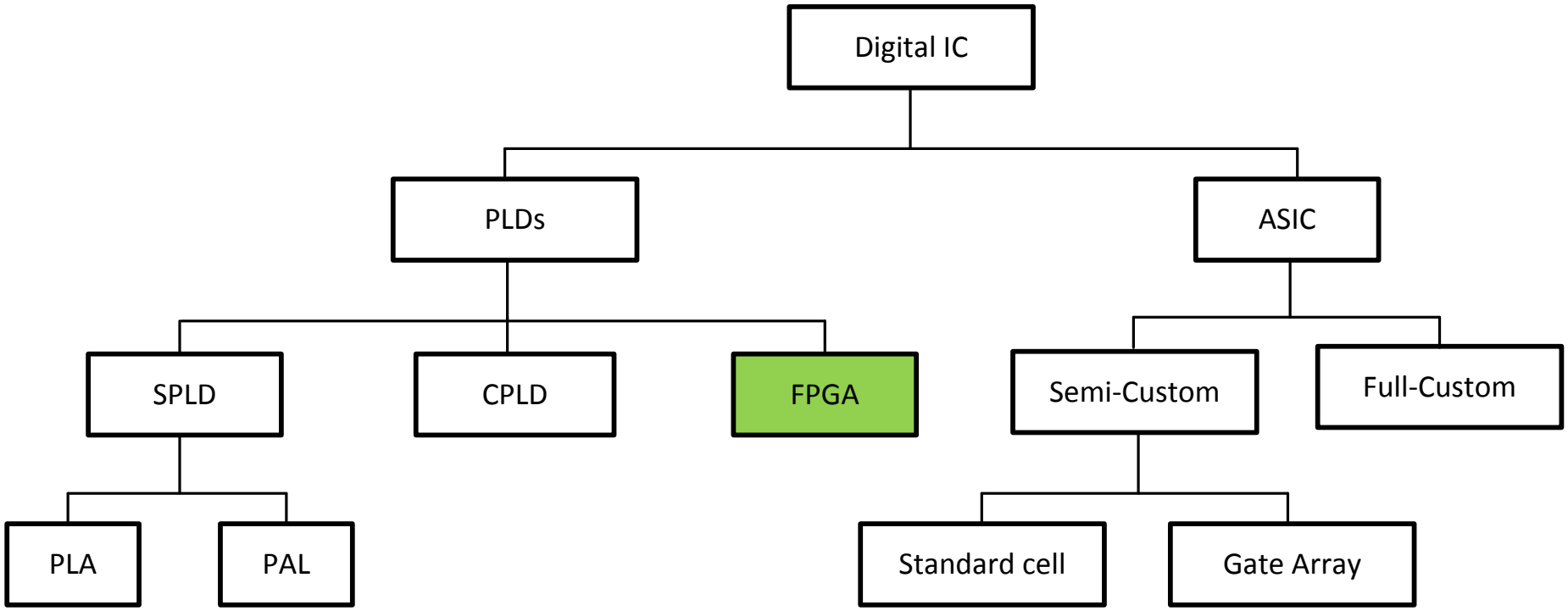
Device	Size	Design Type
SPLD	~ 200 gates	Small
CPLD	~ 10,000 gates	Moderate
FPGA	~ 1,000,000 gates	Large



Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- **Field-Programmable Gate Array (FPGAs)**
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

Programmable Logic Designs (PLDs)



FPGA

❑ FPGA: (Field-Programmable Gate Array)

- ➢ Pre-fabricated silicon devices that can be electrically programmed to become any kind of digital circuit or system
- ➢ A very large array of programmable logic blocks surrounded by programmable interconnects
- ➢ Contains logic blocks instead of AND/OR planes (multi-level logic of arbitrary depth)
- ➢ Can be programmed by the end-user to implement specific applications
- ➢ Capacity up to multi-millions gates
- ➢ Clock frequency up to 500MHz

FPGA

❑ Three ages of FPGAs

Period	Age	Comments
1984 - 1991	Invention	• Technology is limited, FPGAs are much smaller than the application problem size • Design automation is secondary • Architecture efficiency is key
1991 - 1999	Expansion	• FPGA size approaches the problem size • Ease-of-design becomes critical
2000 - 2007	Accumulation	• FPGAs are larger than the typical problem size • Logic capacity limited by I/O bandwidth

FPGA Applications

- **Popular applications:**
  - Prototyping a design before the final fabrication (using single FPGA)
  - Emulation of entire large hardware systems (using multiple FPGAs)
  - Configured as custom computing machines
    - Using programmable parts to “execute” software rather than software compilation on a CPU
  - On-site hardware reconfiguration
  - Low-cost applications
  - DSP, logic emulation, network components, etc…

FPGA History

- First SRAM-based FPGA by Wahlstorm 1967
- First modern-era FPGA by Xilinx 1984
  - 64 logic blocks
  - 58 input/outputs
- Today:
  - Four main manufacturers (Altera, Xilinx, Actel, Lattice)
    - Over 300,000 logic blocks
    - Over 1100 input/outputs

FPGA Structure

- FPGAs consists of 3 main resources:

FPGA Fabrics

1. Logic Blocks

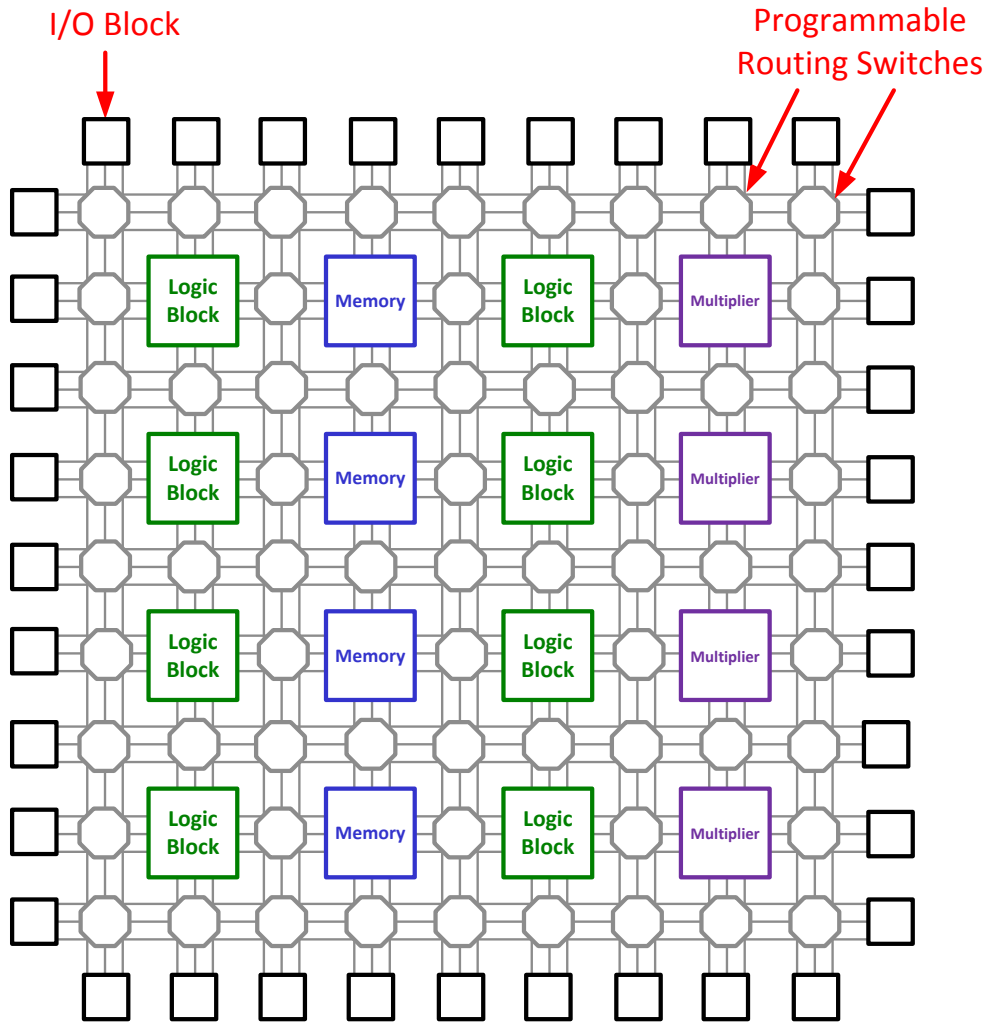
- General logic blocks
- Memory blocks
- Multiplier blocks

2. Program. Routing Switches

- Programmable horizontal/vertical routing channels
- Connecting blocks together and I/O

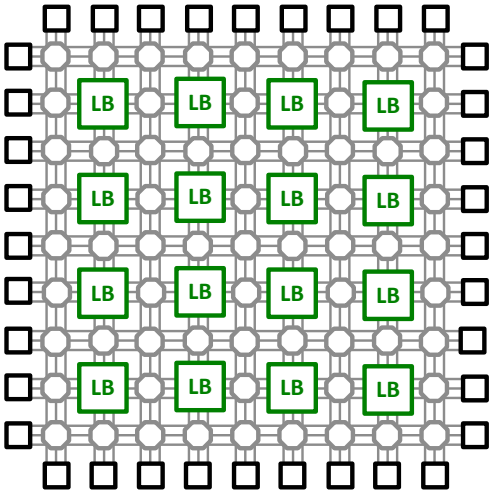
3. I/O Blocks

- Connecting the chip to the outside

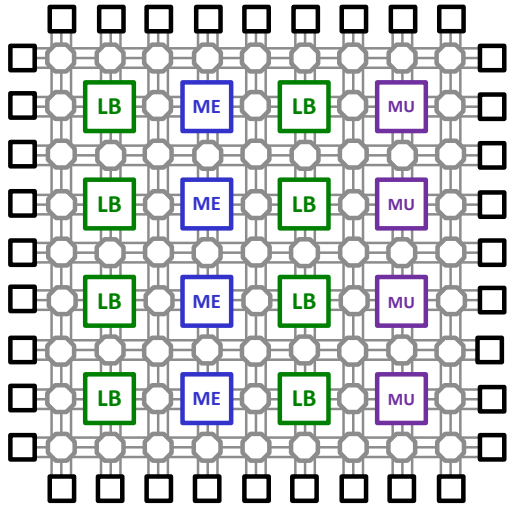


FPGA Categories (Structure)

- There are two main categories of FPGAs in terms of their structure:
  - **Homogeneous:**
    - Employs only one type of logic block
  - **Heterogeneous:**
    - Employs mixture of different blocks such as dedicated memory/multiplier
    - Very efficient for specific functions
    - Might go waste if not used!



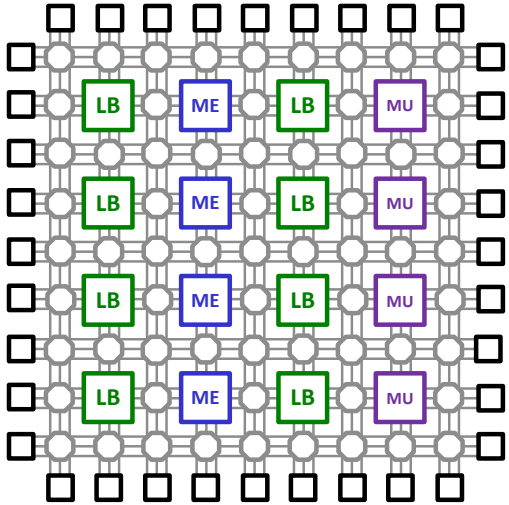
Homogeneous



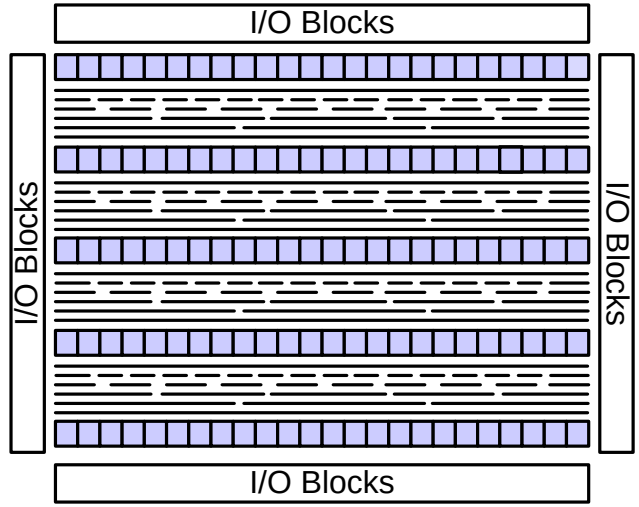
Heterogeneous

FPGA Categories (Floor Plan)

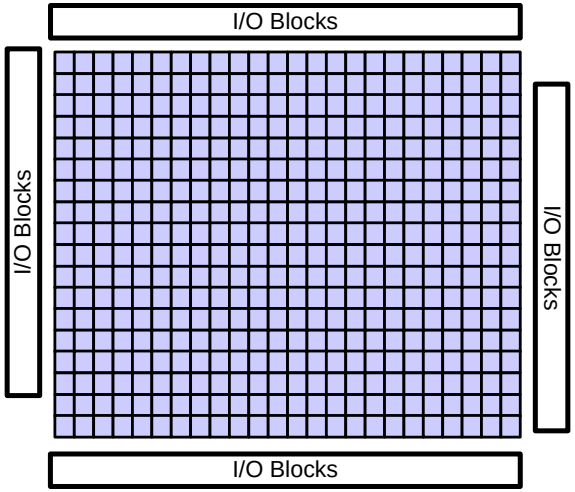
Symmetrical Array



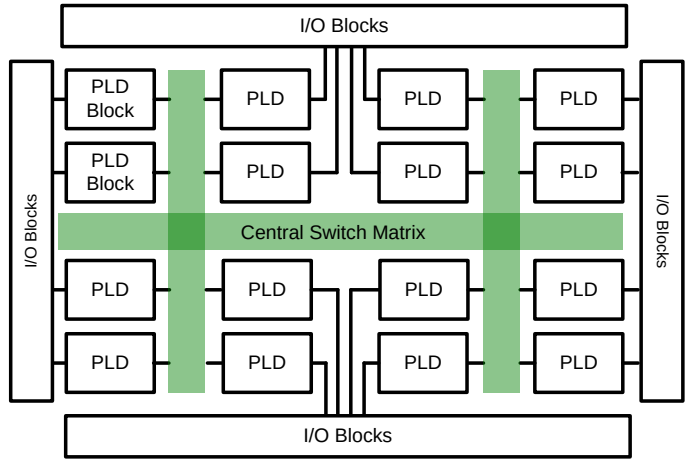
Row-Based



Sea-of-Gates



Hierarchical PLD

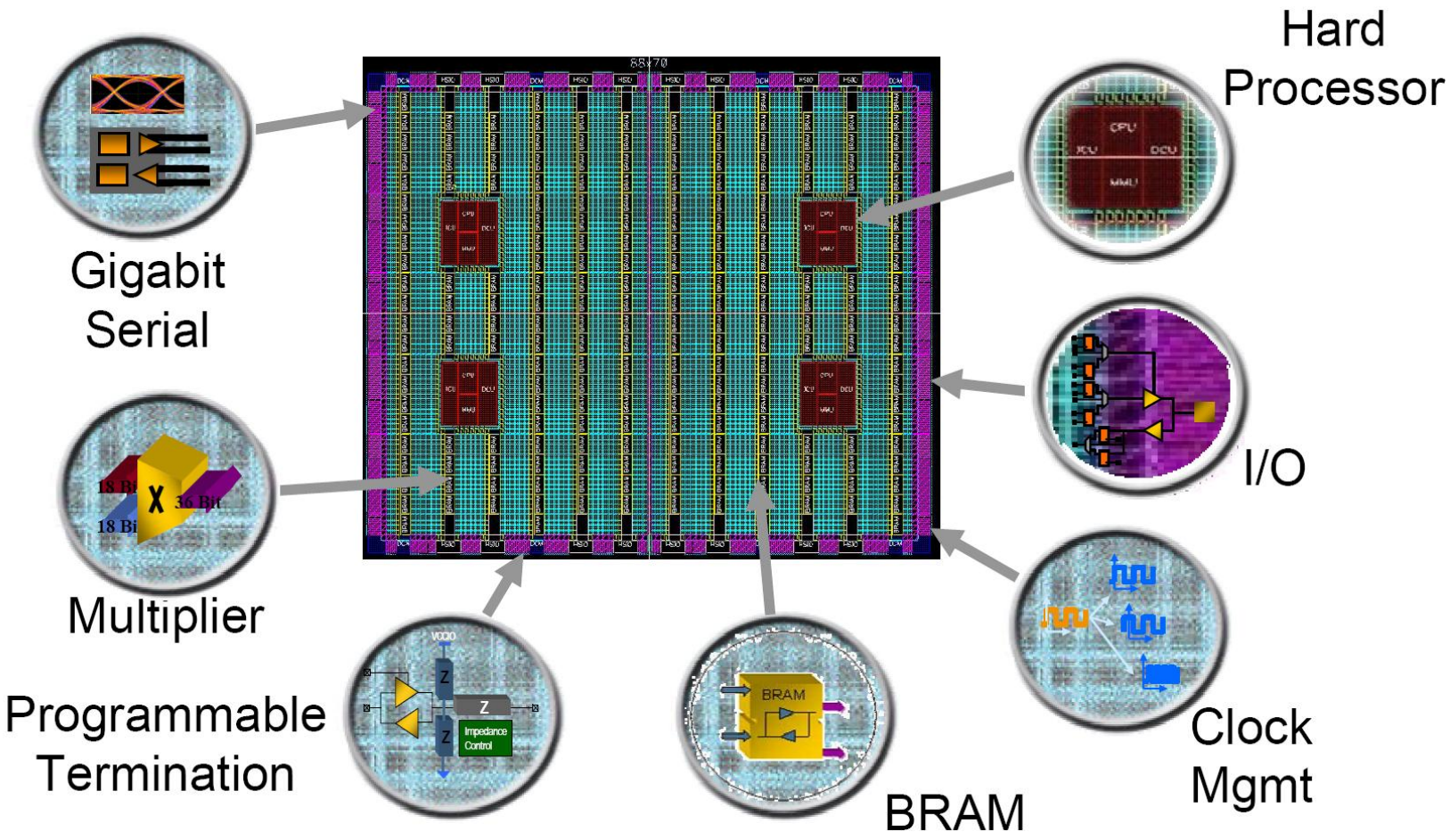


© M. Shabany, ASIC/FPGA Chip Design

FPGA Categories (Architecture)

- There are three main categories of FPGAs in terms of their architecture:
  - **Fine-grained: (early stages)**
    - Logic Block (LB) consists of logic gates plus a register
  - **Coarse-grained: (more efficient)**
    - LB consists of logic gates, MUXs
    - Multi-bit ALU
    - Multi-bit registers
  - **Platform FPGAs:**
    - Sophisticated logic blocks
    - CPU (PowerPC) to run some functions in software
    - PCI bus
    - RAM, PLL
    - Very fast Gbps transceivers for high-speed serial off-chip communication

Modern Commercial FPGAs

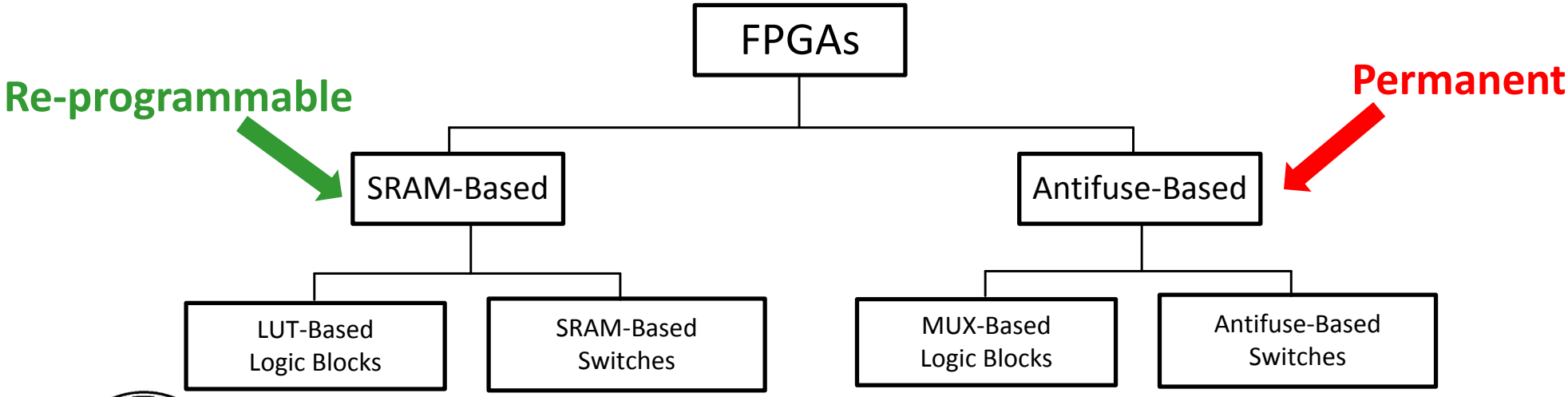


Modern Commercial FPGAs

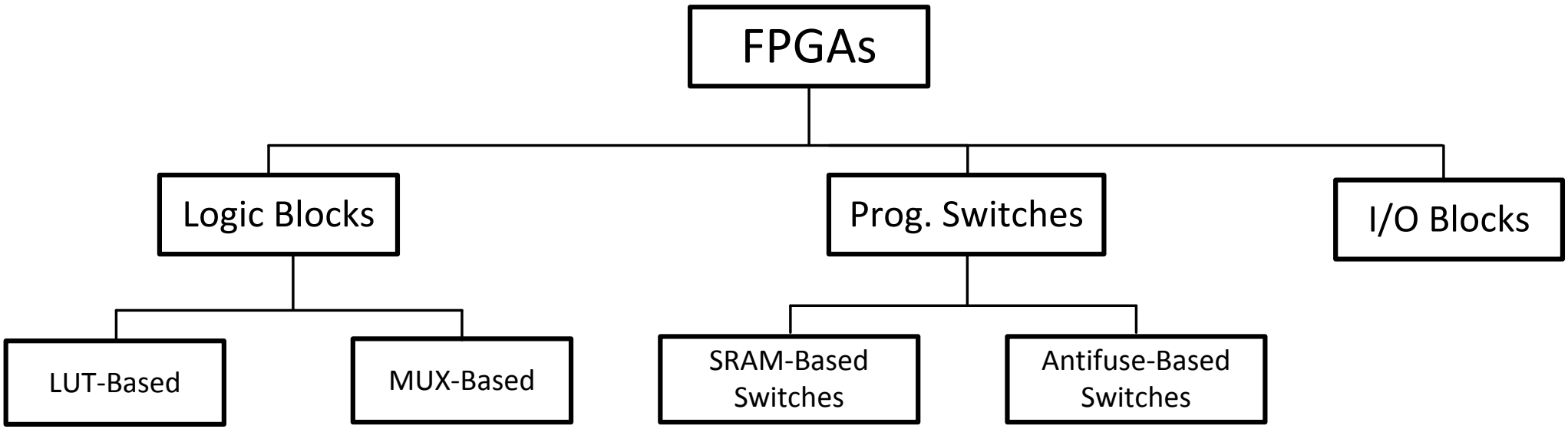
- The concept of coupling microprocessors with FPGAs in heterogeneous platforms was considerably attractive.
- In this programmable platform, microprocessors implement the control-dominated aspects of DSP systems and FPGAs implement the data-dominated aspects.
- With FPGAs, the user is given full freedom to define the architecture which best suits the application.

FPGA Categories (Fabrics)

- There are two main categories of FPGAs in terms of their fabrics:
  - **SRAM-based FPGAs (Xilinx, Altera)** **[Re-programmable, Re-configurable]**
    - Using Lookup Tables (LUTs) to implement logic blocks
    - Using SRAM-cells to implement programmable switches
  - **Antifuse-based FPGAs (Actel, Lattice, Xilinx, QuickLogic, Cypress)** **[Permanent]**
    - Using multiplexers (MUXs) to implement logic blocks
    - Using antifuses to implement programmable switches



FPGA Categories (Another View)



Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- **Field-Programmable Gate Array (FPGAs)**
  - **Logic Blocks**
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

Logic Block

- The logic block is the most important element of an FPGA, which provides the basic computation and storage elements used in digital logic systems

- Logic blocks are used to implement logic functions

- A logic block has a small number of inputs and outputs

- The logic block of an FPGA is considerably more complex than a standard CMOS gate b/c:
  - A CMOS gate implements only one chosen logic function
  - An FPGA logic block must be configurable enough to implement a number of different functions

Logic Block Design

❑ Transistors as the basic logic block (fine-grained)

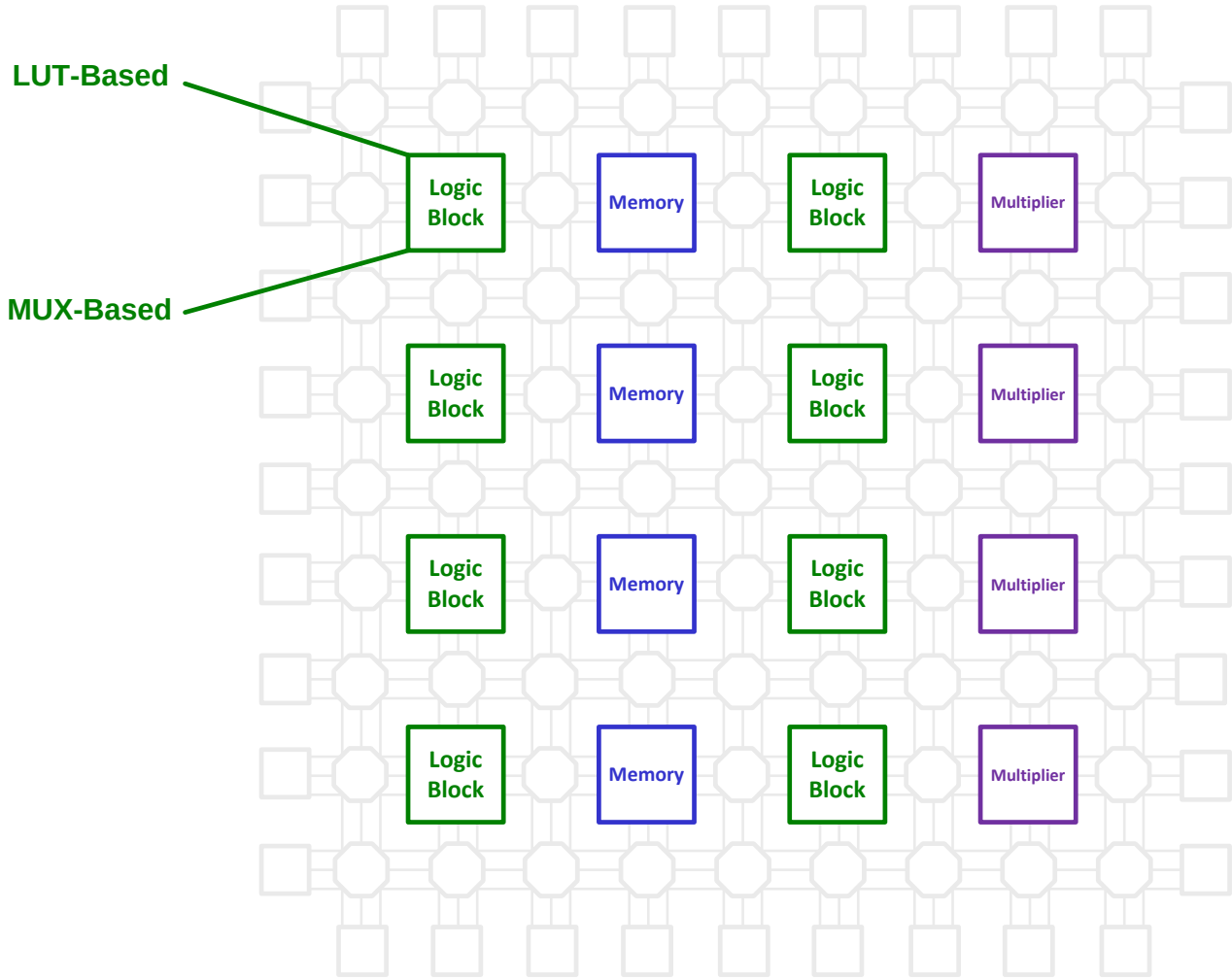
- ➢ Build gates & storage elements from it
- ➢ Tried in Crosspoint
- ❖ **Drawbacks:**
  - ➢ Requires huge amount of Prog. interconnects to create a typical logic function
  - ➢ Low area-efficiency (b/c Prog. switches are area intensive)
  - ➢ Low performance (b/c each routing hop is slow)
  - ➢ high power consumption (higher interconnects capacitance to charge and discharge)

❑ Processors as the basic logic block (coarse-grained)

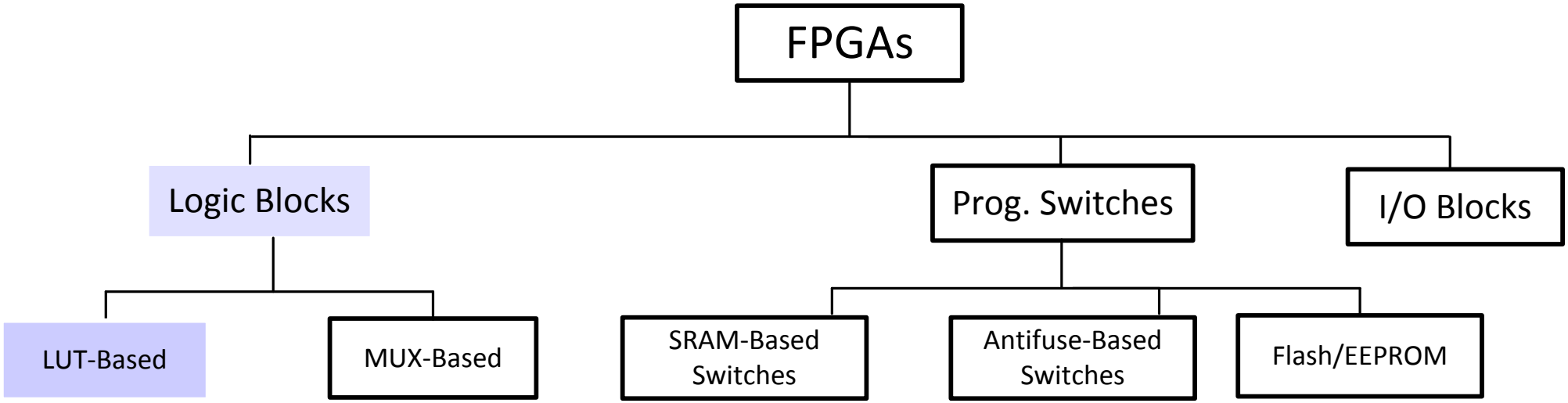
- ❖ **Drawbacks:**
  - ➢ Incredibly inefficient for implementing simple functions
  - ➢ Less performance than customized hardware

→ Logic blocks should be designed as something in between

FPGA Categories



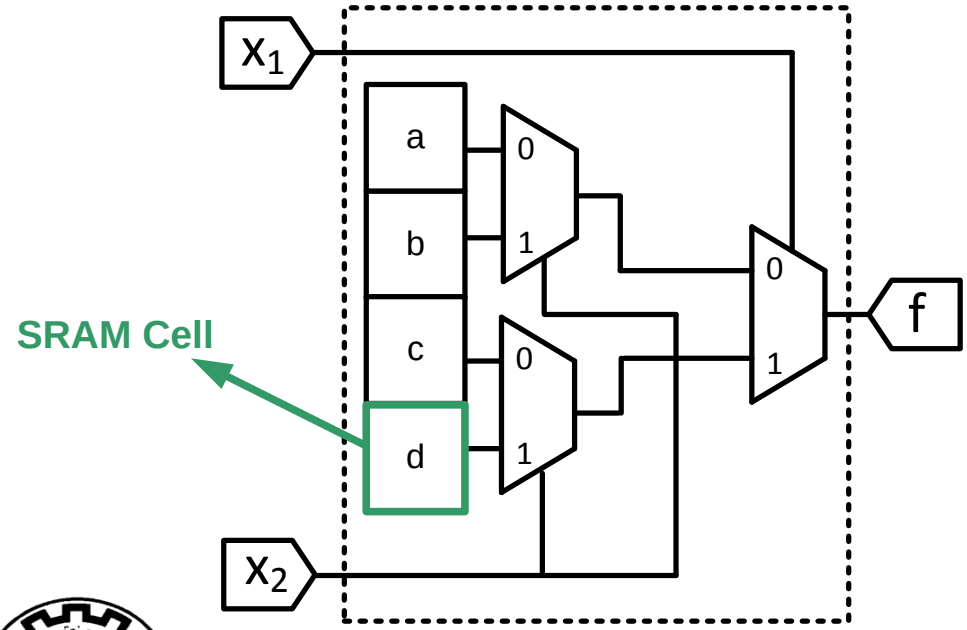
Logic Blocks (LUT-Based)



LUT-Based Logic Block (Used in SRAM-Based FPGAs)

- Lookup Table (LUT)
  - Uses a set of 1-bit storage elements to implement logic functions

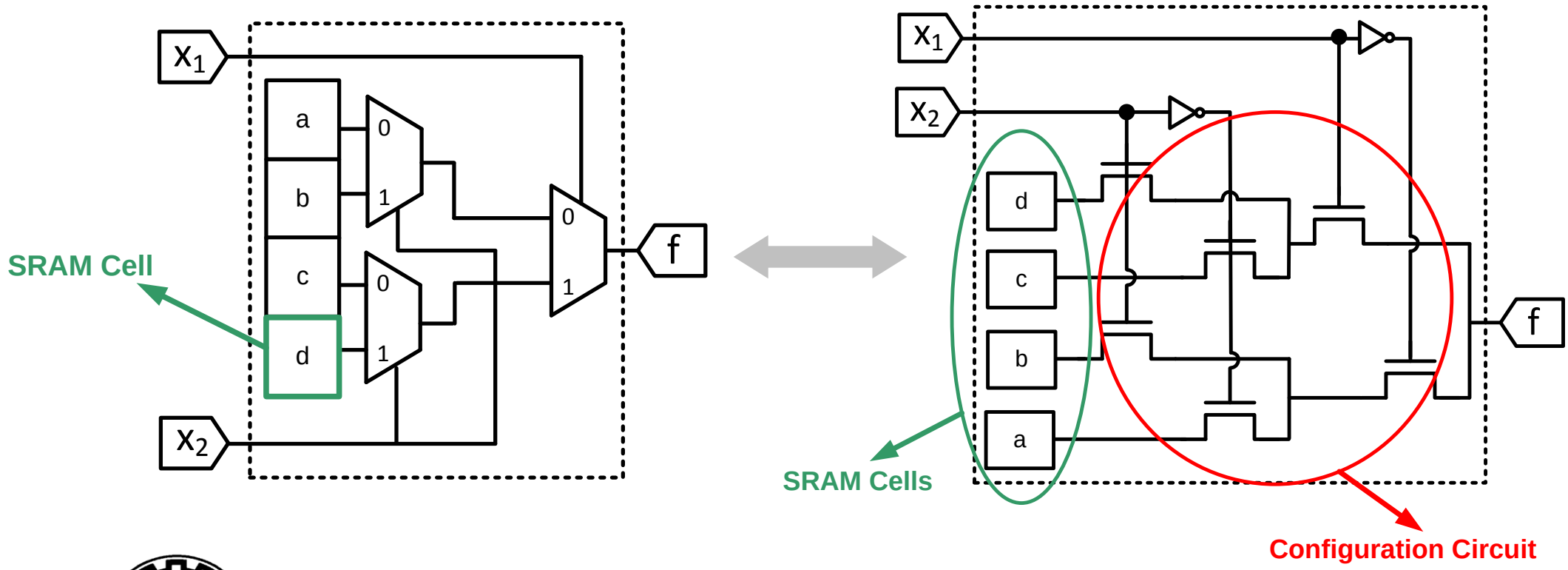
- Example:
  - A 2-input LUT
    - Capable of implementing any logic function of two variables



x_1	x_2	f
0	0	a
0	1	b
1	0	c
1	1	d

LUT-Based Logic Block

- Lookup Table (LUT) consists of:
  - Memory (SRAM Cells)
  - Configuration circuit that selects the proper memory bit

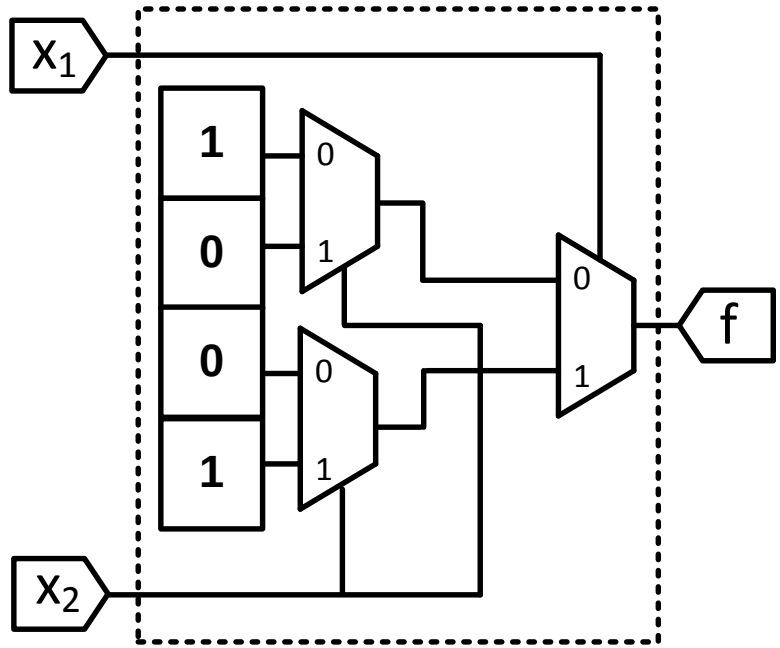


LUT-Based Logic Block

❖ Example:

➢ $f = \bar{x}_1\bar{x}_2 + x_1x_2$

x_1	x_2	f
0	0	**1**
0	1	**0**
1	0	**0**
1	1	**1**

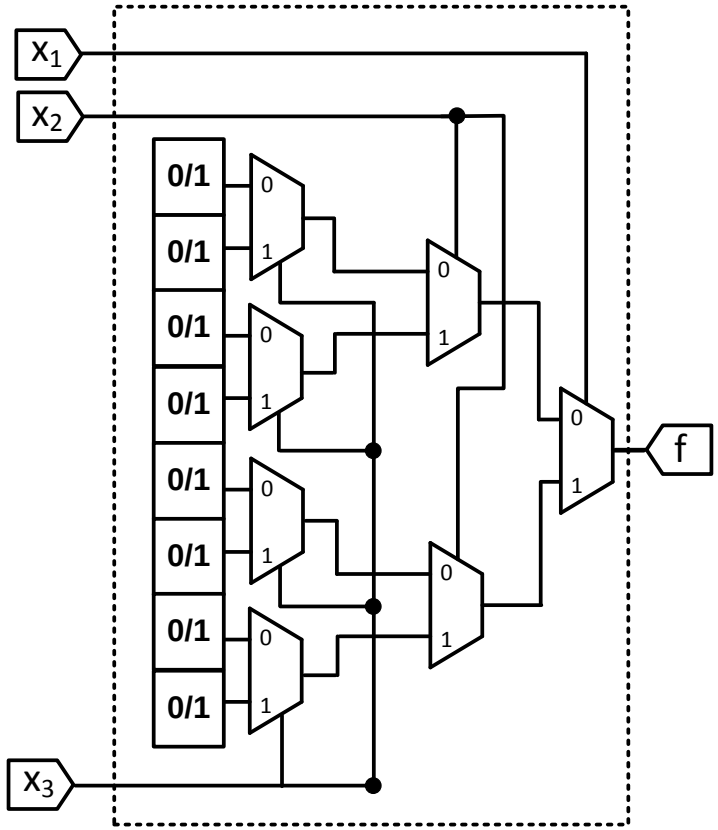


LUT-Based Logic Block

❖ Example:

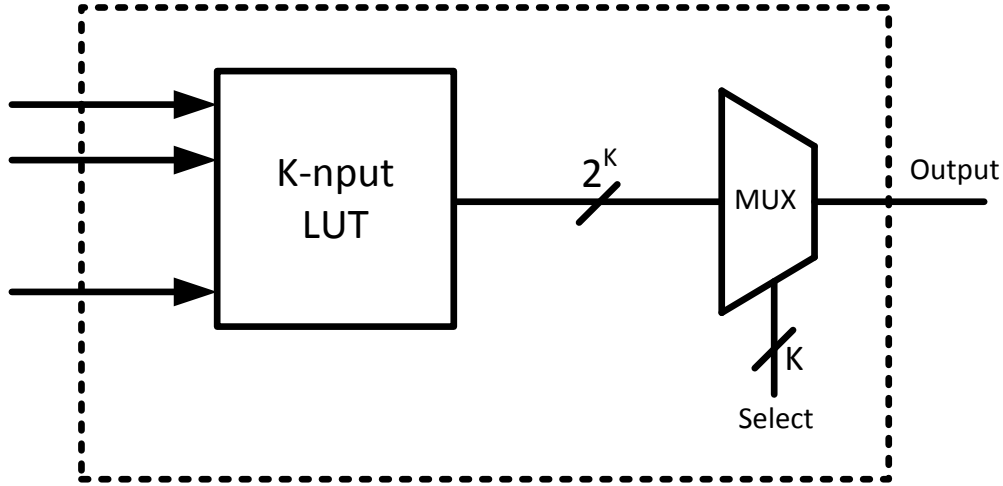
➢ A 3-input LUT

- Capable of implementing any logic function of three variables



LUT-Based Logic Block

- In general: (A K-input LUT)
  - Capable of implementing any logic function of K variables
  - Can implement 2^{2K} different logic functions
  - The logic in LUT can be easily changed by changing the bits stored in the SRAM cells



- A typical logic block in commercial FPGAs has 4-6 inputs (6-input LUTs)

LUT-Based Logic Block

- A typical logic block in commercial FPGAs has 4-6 inputs
  - 4-input LUTs:
    - Xilinx XC4000
    - Xilinx Virtex family up to and including Virtex 4
    - Altera FLEX, Cyclone, Stratix I
  - Fracturable 6-input LUTs: (a.k.a **Adaptive Logic Module (ALM)**)
    - Xilinx Virtex 5
    - Altera Stratix II

LUT-Based Logic Block

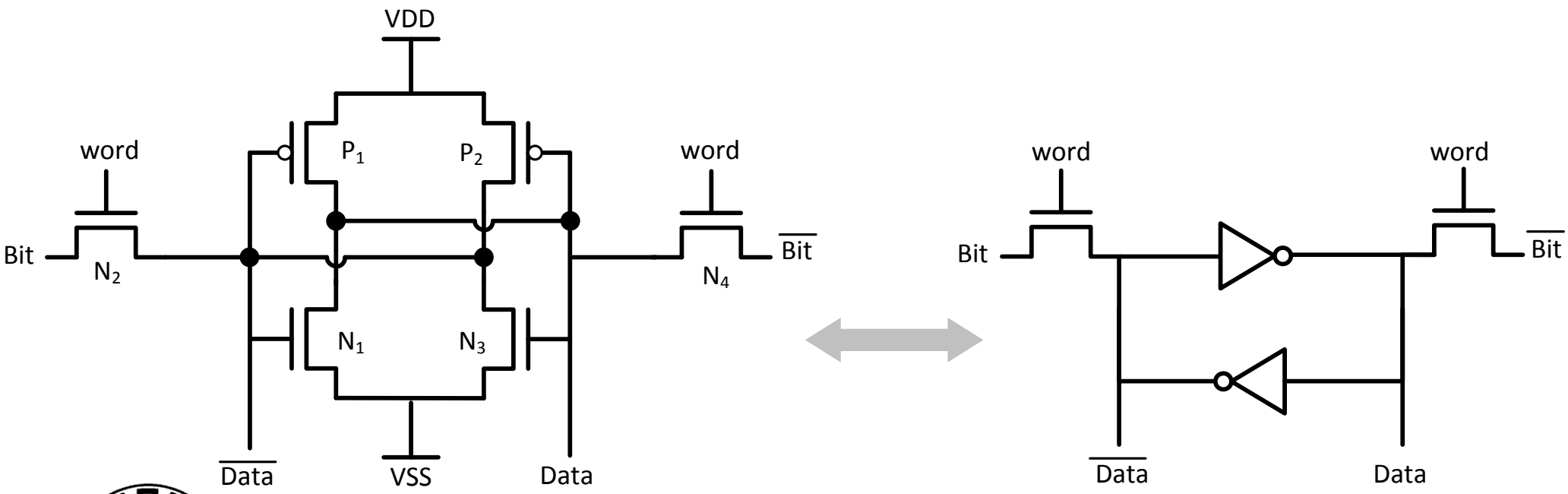
- Storage cells in the LUT are SRAM cells that are "volatile"
  - Lose their values when the power supply turns off
    - ⟹ Therefore, FPGA has to be re-programmed again

- Often a small memory chip, programmable read only memory (PROM) is used to hold their contents permanently

- LUT values are loaded automatically from the PROM when power is applied to the chip.

SRAM Cell used in LUT-based FPGAs

- The value is stored in the middle four transistors
- These four transistors form a pair of inverters connected in a loop
  - "word=0" ⟹ SRAM cell stores the value
  - "word=1" ⟹ Read/Write is performed



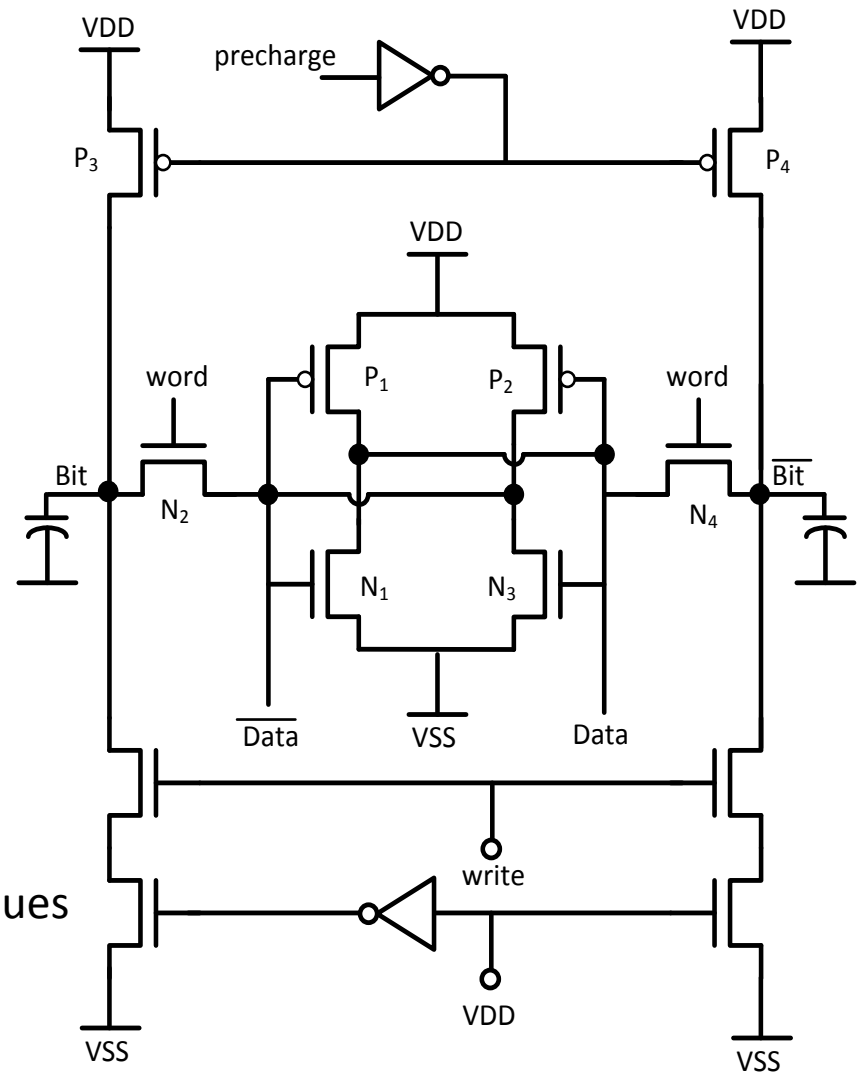
SRAM Cell Read/Write Operation

- **Read Operation:**
  - 1) Bit & $\overline{\text{Bit}}$ are precharged to VDD
    - (Data=0 & $\overline{\text{Data}}$=1 or Data=1 & $\overline{\text{Data}}$=0)
  - 2) Then "word=1"
    - if Data=0 ⟹ Bit discharges through N_2 & N_1
    - if Data=1 ⟹ $\overline{\text{Bit}}$ discharges through N_4 & N_3

Read Stability Condition: $W_{N_1,N_3} > W_{N_2,N_4}$

- **Write Operation**
  - 1) Bit & $\overline{\text{Bit}}$ are set to the desired values
    - (e.g., Bit = 1 and $\overline{\text{Bit}}$=0 if "1" is to be written)
  - 2) Then "word" is set to "1"
    - Charge sharing forces the inverter to switch values

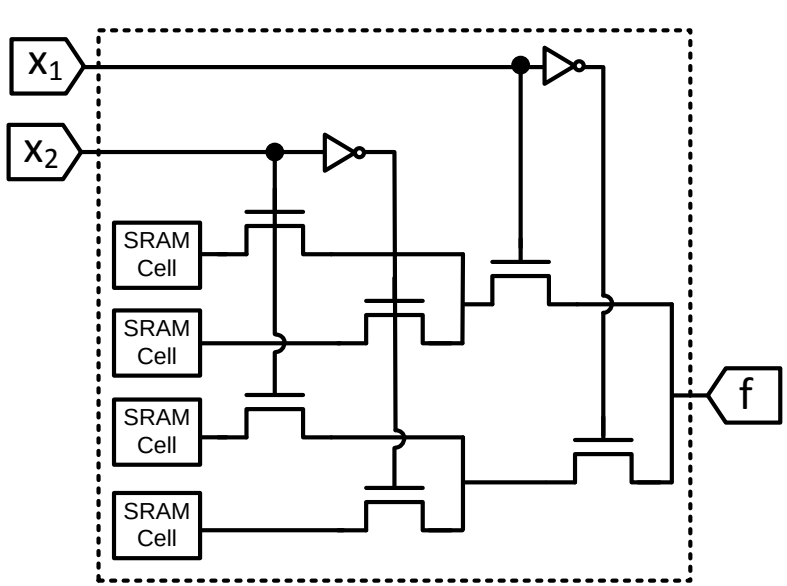
Write Stability Condition: $W_{N_2,N_4} > W_{P_1,P_2}$



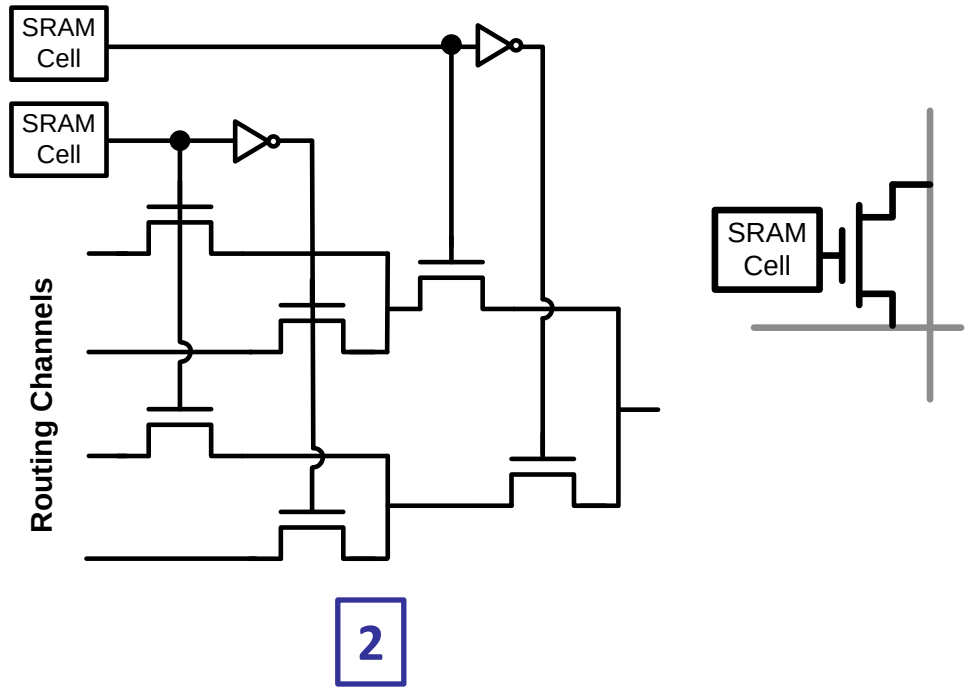
© M. Shabany, ASIC/FPGA Chip Design

SRAM Cell

- **Two primary uses:**
  1. To store data in LUTs to implement logic functions
     - Uses only one side of the cell (e.g., Bit)
  2. To set the select lines in the programmable interconnects



1



2

LUT-Based Logic Block

❑ LUT-based logic blocks in most commercial FPGAs have some additional elements for efficient implementation (better than their LUT-based realizations)

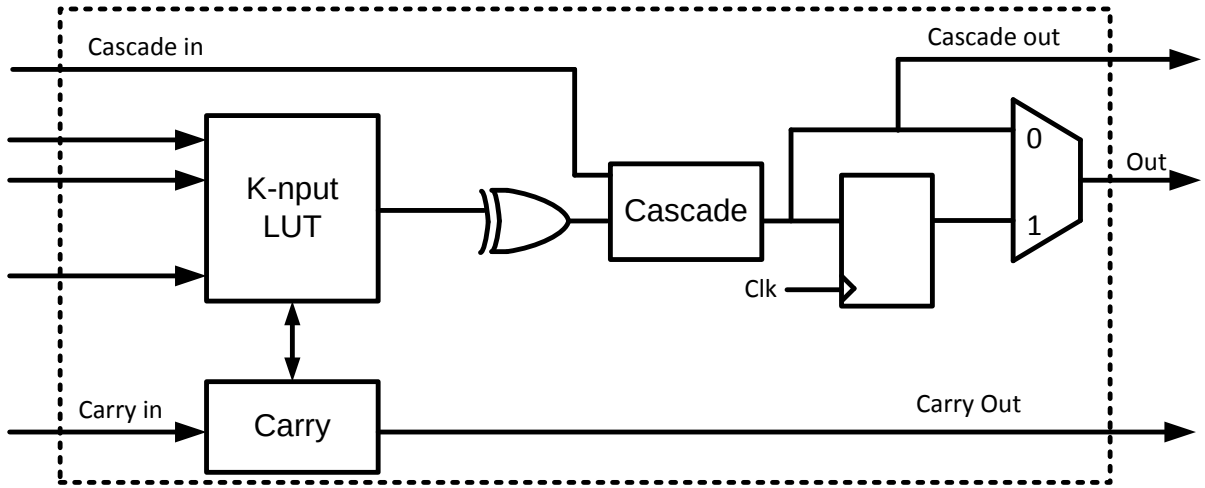
➢ **Extra elements inside LUT-based logic blocks: (Soft Logic)**

Coarse-grained

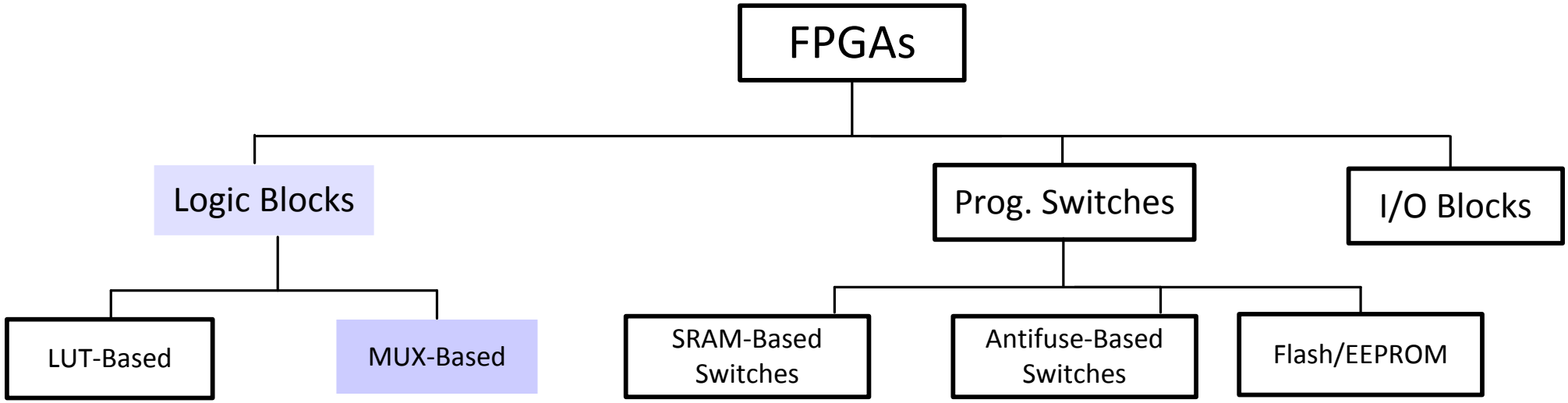
- LUT
- Flip-flops, MUXs, XOR

Fine-grained

- Blocks to support arithmetic carry, sum, and subtraction functions
- Cascade (to implement wide AND and larger functions)



Logic Blocks (MUX-Based)



MUX-Based Logic Block (Used in Antifuse-Based FPGA)

- The logic block in **antifuse-based FPGAs** are generally based on multiplexing
- Functions can be realized using MUXs based on Shannon's expansion
- Shannon's Expansion Theorem:

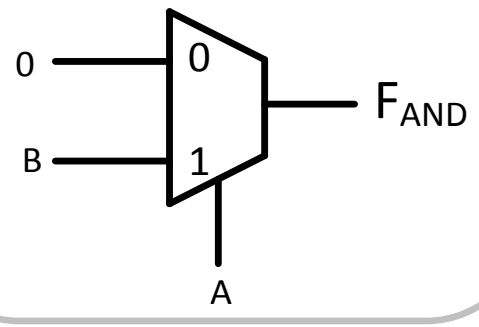
> Any logic function $f(x_1, x_2, \ldots, x_n)$ can be expanded in the form of:
>
> $$x_k . f(x_1, x_2, x_{k-1}, 1, x_{k+1}, \ldots, x_n) + x_k' . f(x_1, x_2, x_{k-1}, 0, x_{k+1}, \ldots, x_n)$$

- Example:
  - $F(A, B, C) = A'B + ABC' + A'B'C$
    $= A.F(1,B,C) + A'.F(0,B,C)$
    $= A(BC') + A'(B+B'C)$

MUX-Based Logic Block

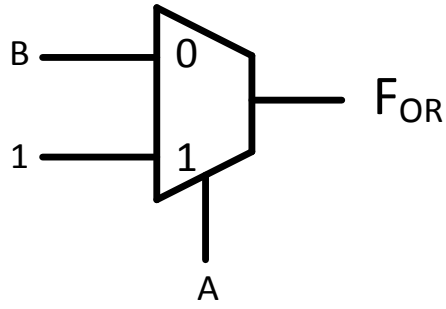
AND Gate:

$F_{AND} = A.B = A.B + A'0$



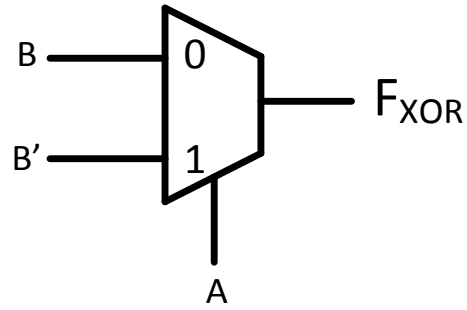
OR Gate:

$F_{OR} = A+B = A.1 + A'B$



XOR Gate:

$F_{AND} = AB'+A'B$



MUX-Based Logic Block

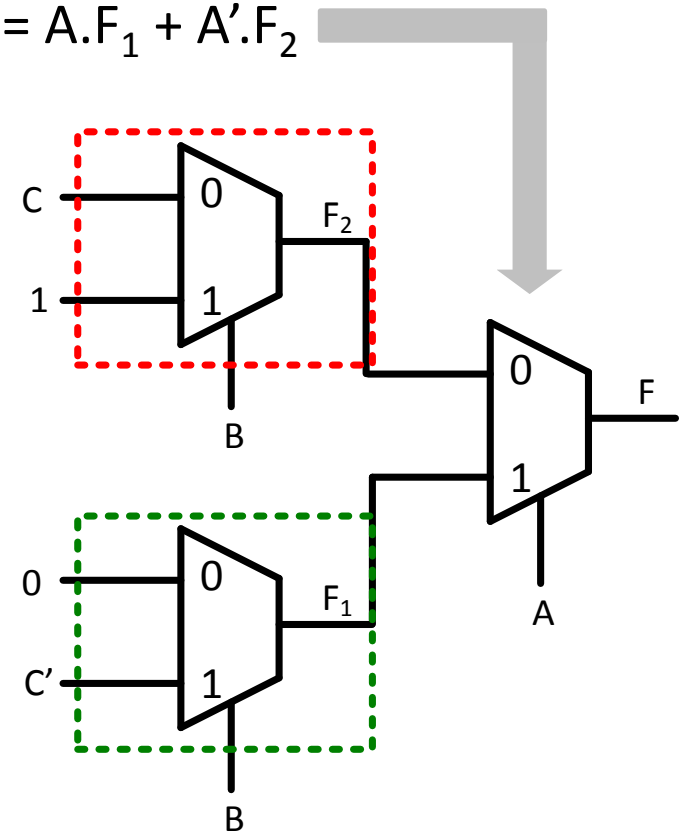
❖ Example:

➢ F(A, B, C) = A'B + ABC' + A'B'C

= A(BC') + A'(B+B'C) = $A.F_1 + A'.F_2$

➢ F_2 = B+B'C = B.1 + B'.C

➢ F_1 = BC' = BC' + B'.0

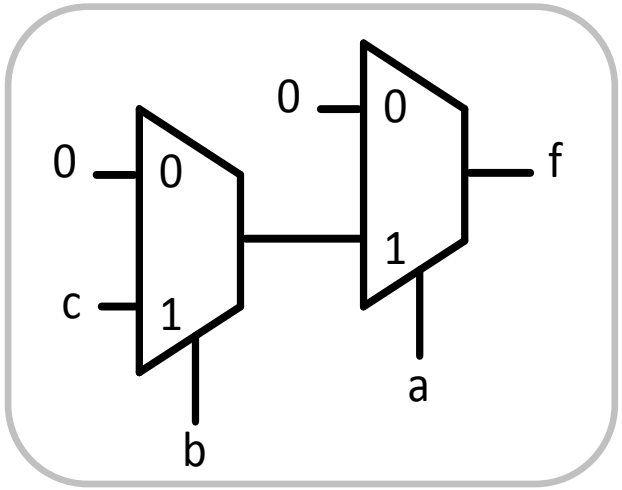


MUX-Based Logic Block

- The logic block in **antifuse-based FPGAs** are generally based on multiplexing

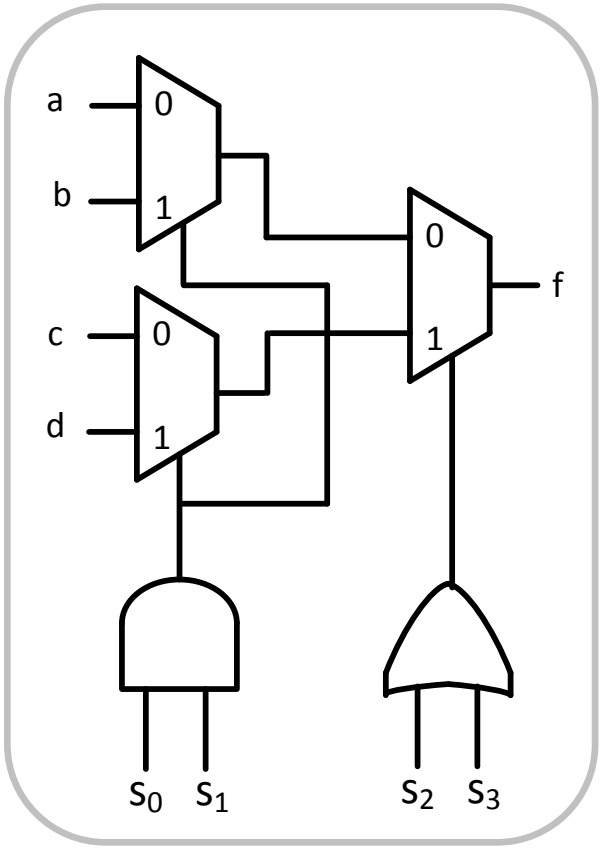
❖ Example:

- Three-input AND function
  - f = a.(b.c+b'.0)+a'(0)



MUX-Based Logic Block

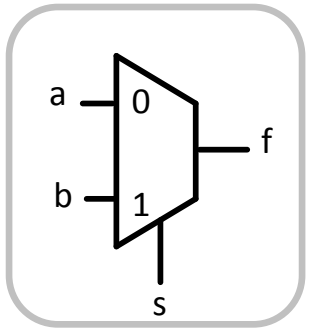
- A more complex logic block



S_0	S_1	S_2	S_3	f
0	0	0	0	a
0	0	0	1	a
0	0	1	0	a
0	0	1	1	b
0	1	0	0	c
0	1	0	1	c
0	1	1	0	c
0	1	1	1	d
1	0	0	0	c
1	0	0	1	c
1	0	1	0	c
1	0	1	1	d
1	1	0	0	c
1	1	0	1	c
1	1	1	0	c
1	1	1	1	d

MUX-Based Logic Block

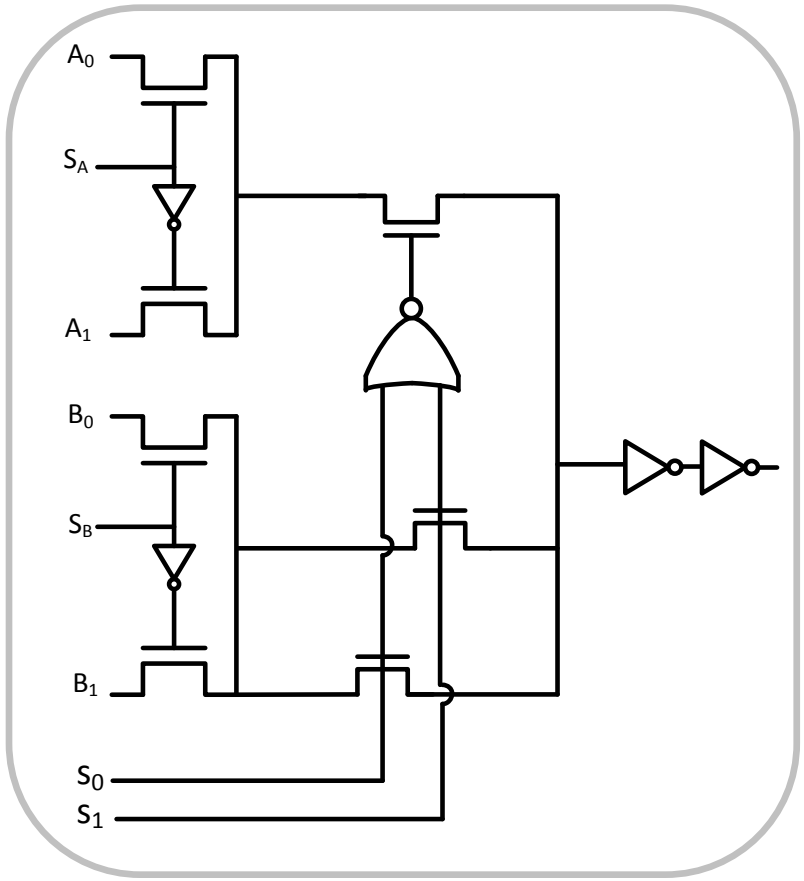
- MUX-Based configurable logic block



a	b	s	f
0	0	0	0
0	X	1	X
0	Y	1	Y
0	Y	X	XY
X	0	Y	XY’
Y	0	X	X’Y
Y	1	X	X+Y
1	0	X	X’
1	0	Y	Y’
1	1	1	1

MUX-Based Logic Block

- MUX-Based configurable logic block (can also be used to build latches/registers)



A_0	A_1	B_0	B_1	S_A	S_1	S_0	S_B	OUT
1	1	0	1	A	0	B	A	(AB)’
0	1	0	1	0	0	B	A	(AB)’
0	1	0	1	0	B	0	A	(AB)’
0	1	0	1	0	0	A	B	(AB)’
1	0	0	1	A	0	B	A	A^B
1	0	0	1	A	B	0	A	A^B
Q	0	D	0	CLR	CLK	0	CLR	Latch
Q	0	CLR	0	CLR	CLK	0	D	Latch

Comparison b/w MUX-based and LUT-based

- **LUT-based Logic Block (LB) using SRAM cells:**
  - An n-input LUT function requires 2^n SRAM cells
  - Each SRAM cell requires 8 transistors
    - e.g., a 4-input function requires 16x8=128 transistors
  - Decoding circuitry is also required
    - e.g., decoder for a 4-input LUT is a MUX with 96 transistors
  - Delay of LUT is independent of the function implemented and is dominated by the delay through the SRAM cell (same for all functions!)
  - SRAM consumes power even when its inputs do not change. The stored charge in the SRAM cell dissipates slowly.
  - LUT-based LB is considerably more expensive than a static CMOS gate.
  - Easier implementation through loading configuration bits

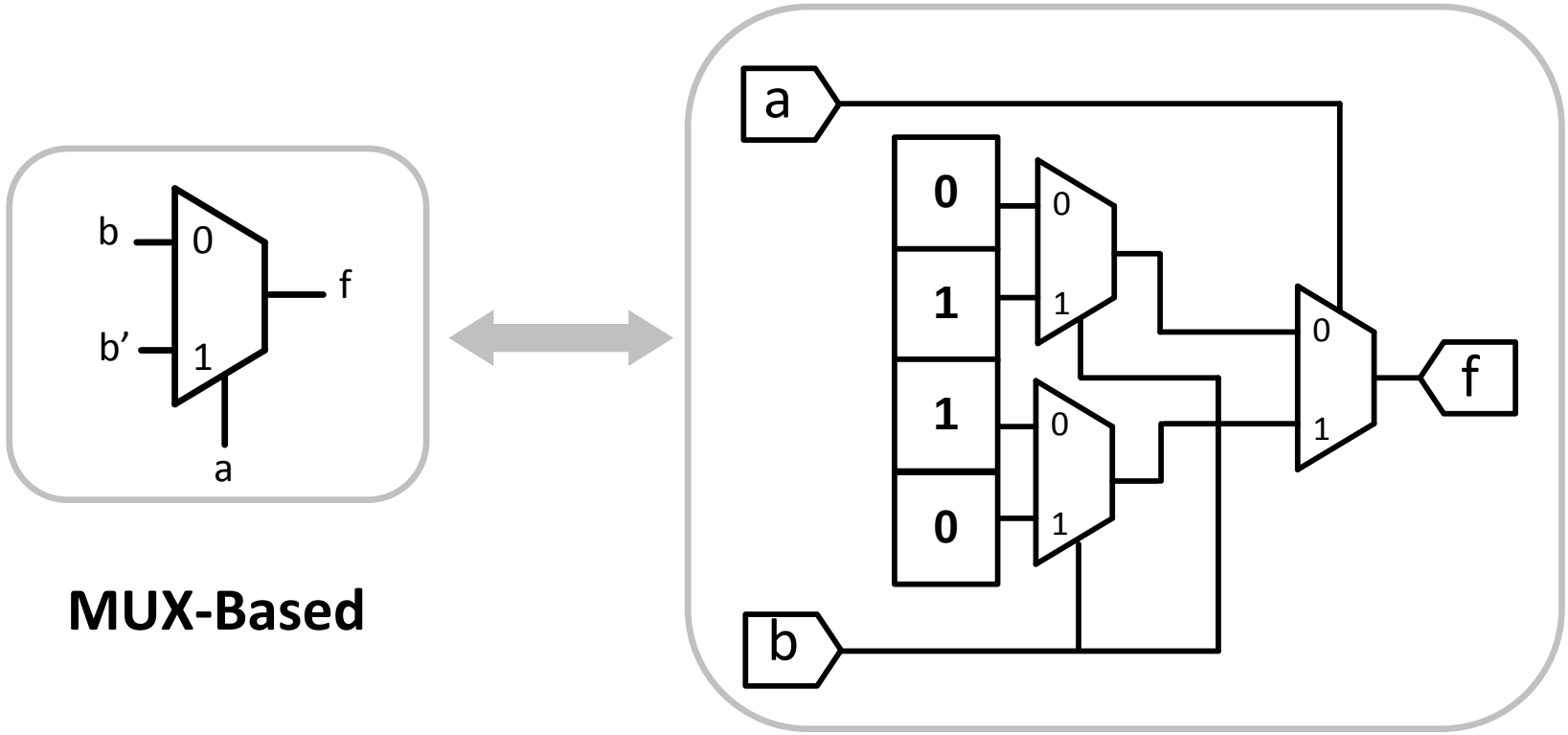
Comparison b/w MUX-based and LUT-based

- **MUX-based LB using Static CMOS:**
  - Number of transistors a function of number of inputs and the function
    - An n-input NAND requires 2n transistors
    - An n-input XOR is more complicated
  - The delay of a static gate depends on the number of inputs, function, and the transistor sizes
  - MUX-based implementation consumes no power while the inputs are stable (ignoring the leakage power)
  - Synthesizer has a hard time figuring out how to implement a certain function into the given MUX structure

Comparison b/w MUX-based and LUT-based

❖ **Example:**

➢ Implementation of an XOR in two cases:

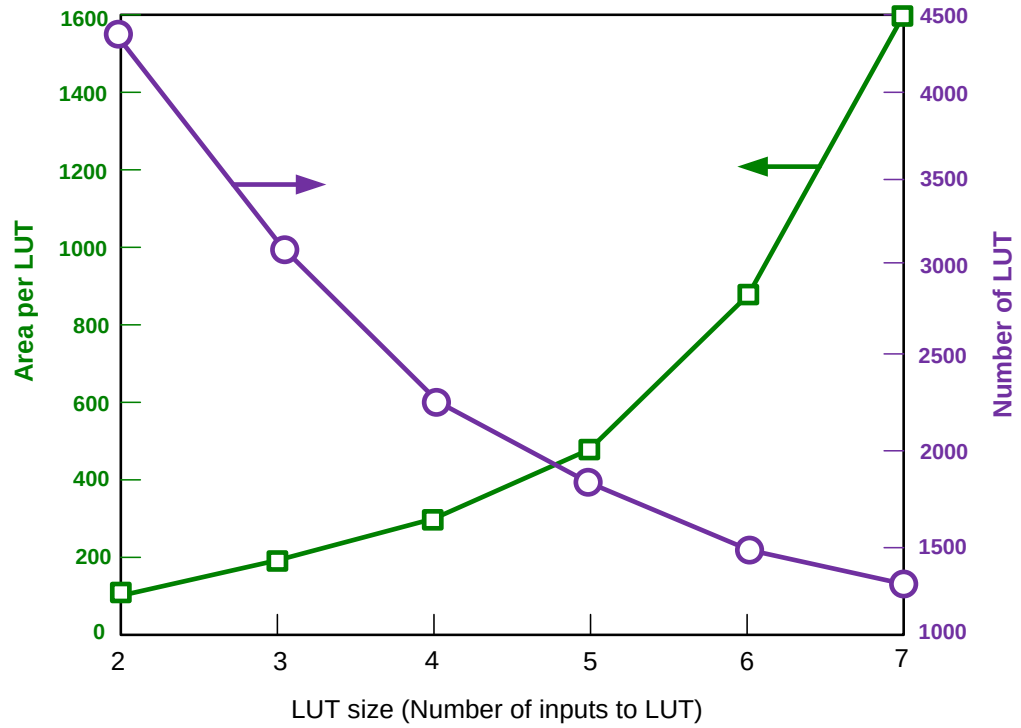


MUX-Based

LUT-Based

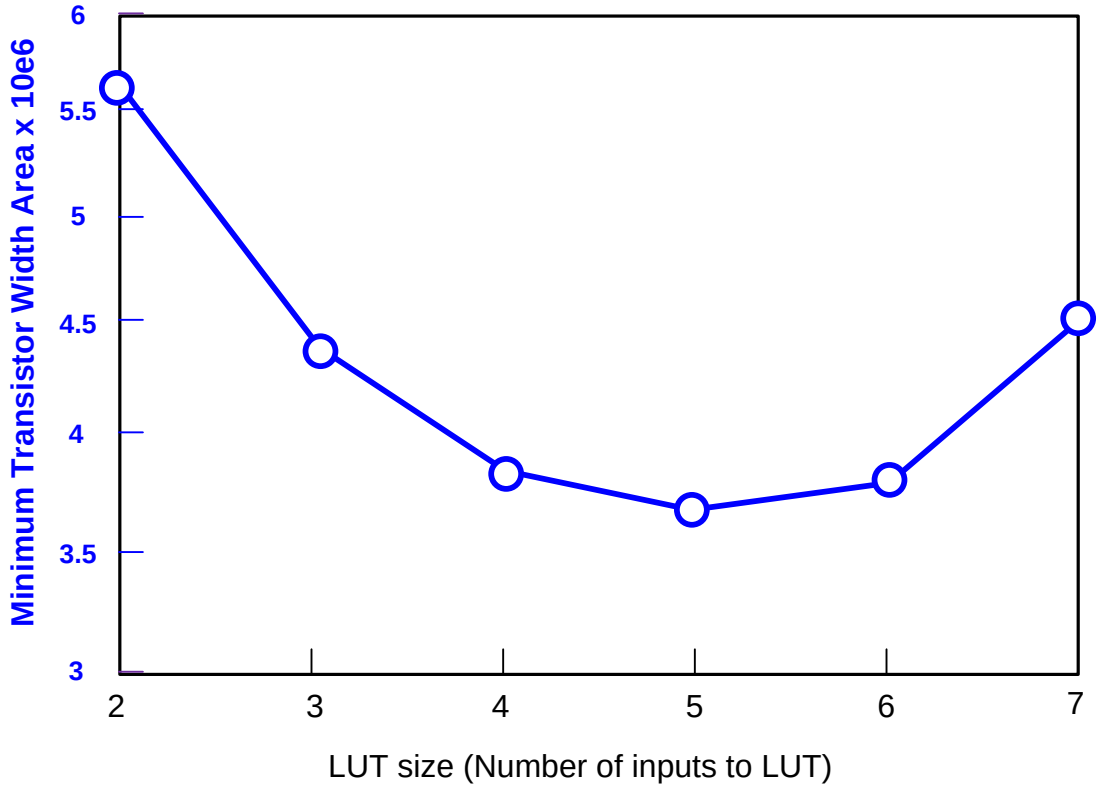
Logic Block Design: Area Trade-off

- As the functionality of a logic block (LB) increases:
  - Fewer LBs are needed to implement a given design (**good**)
  - Its size and the amount of routing increases (**bad**)
    - Number of bits in a K-input LUT is 2^K (exponential area increase with K)



Logic Block Design: Area Trade-off

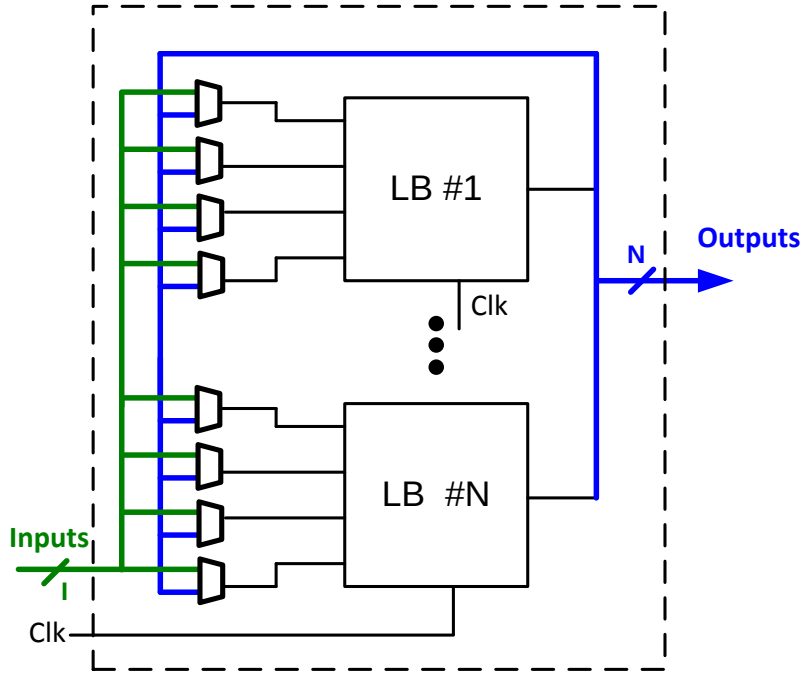
- Total area as a function of LUT size: (product of two previous curves)



4 to 6-input LUT size is optimal in terms of area!

Logic Block Design: Granularity

- An alternative is to change the granularity of each logic block
- It means to integrate a few logic blocks in a cluster (Clusters of LUTs)
- Logic blocks in a cluster are programmably connected together by a local interconnect structure
- This idea is used in most current commercial FPGAs



Logic Block Design: Granularity

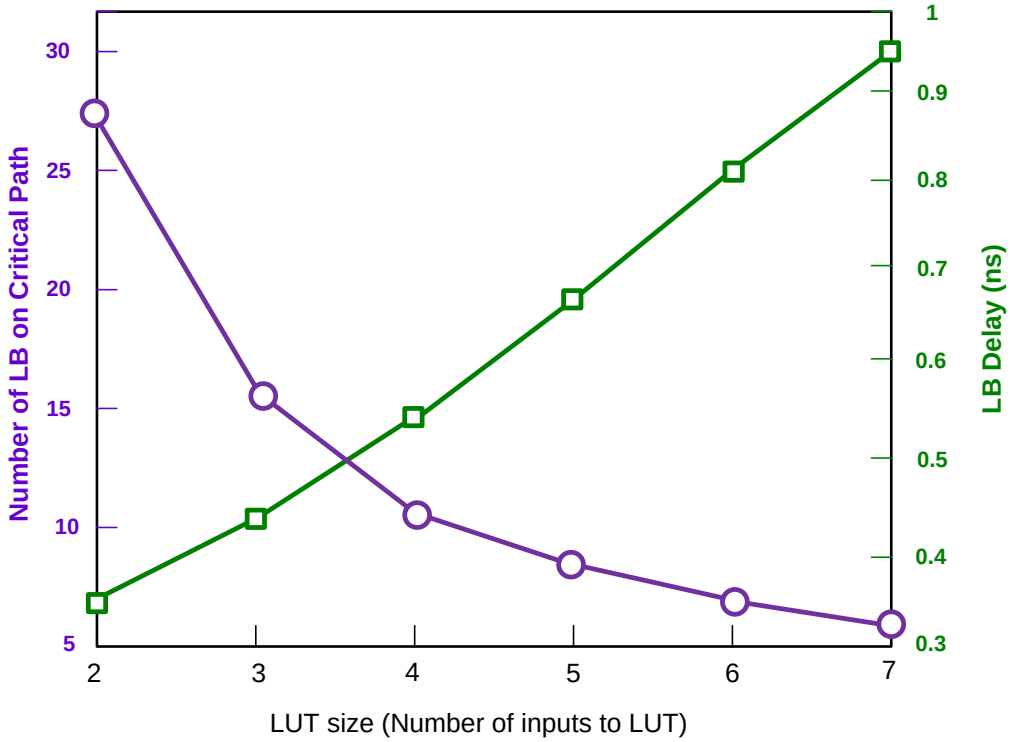
- In this approach the size of the logic and internal routing grows quadratically as opposed to the exponential growth for the LUT size

- More area per logic block with less area increase

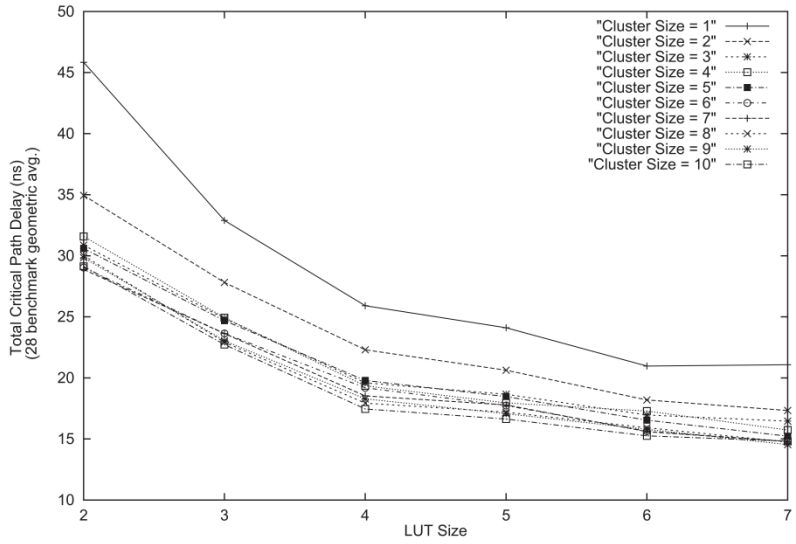
- There is also pin saving as follows:
  - Number of pins needed for N basic logic block with K-input LUT: KN
  - Number of pins needed for a cluster of N K-input LUTs: K(N+1)/2
    - Thus, there are fewer inputs to the cluster from the external inter-cluster routing than the total number of inputs to the basic logic blocks inside the cluster

Logic Block Design: Speed Trade-off

- As the functionality of a logic block (LB) increases:
  - Fewer LBs are used on the critical path (**good**)
  - Less inter-logic routing ⟶ less delay ⟶ higher speed performance
  - The internal delay of each LB increases (**bad**)



Logic Block Design: Speed Trade-off



- **Observations:**
  - Increasing the cluster size, decreases the critical path (up to 3 significant)
  - Higher LUT size results in less delay on the critical path
    - Not too different after LUT size of 4-5
- Optimal point considering both the area and speed optimizations:

LUT size: 4-input or 5-input

Cluster size: 2-4

Logic Block Design in Heterogeneous FPGAs

- If there is a dedicated specific-purpose hard circuit on the FPGA for a function, it has superior area, speed and power consumption over its implementation in general purpose logic blocks.

- For instance, a Flip-Flop (FF) can be built using LUTs and gates but it can also be explicitly designed or customized inside a logic block, much more efficient.

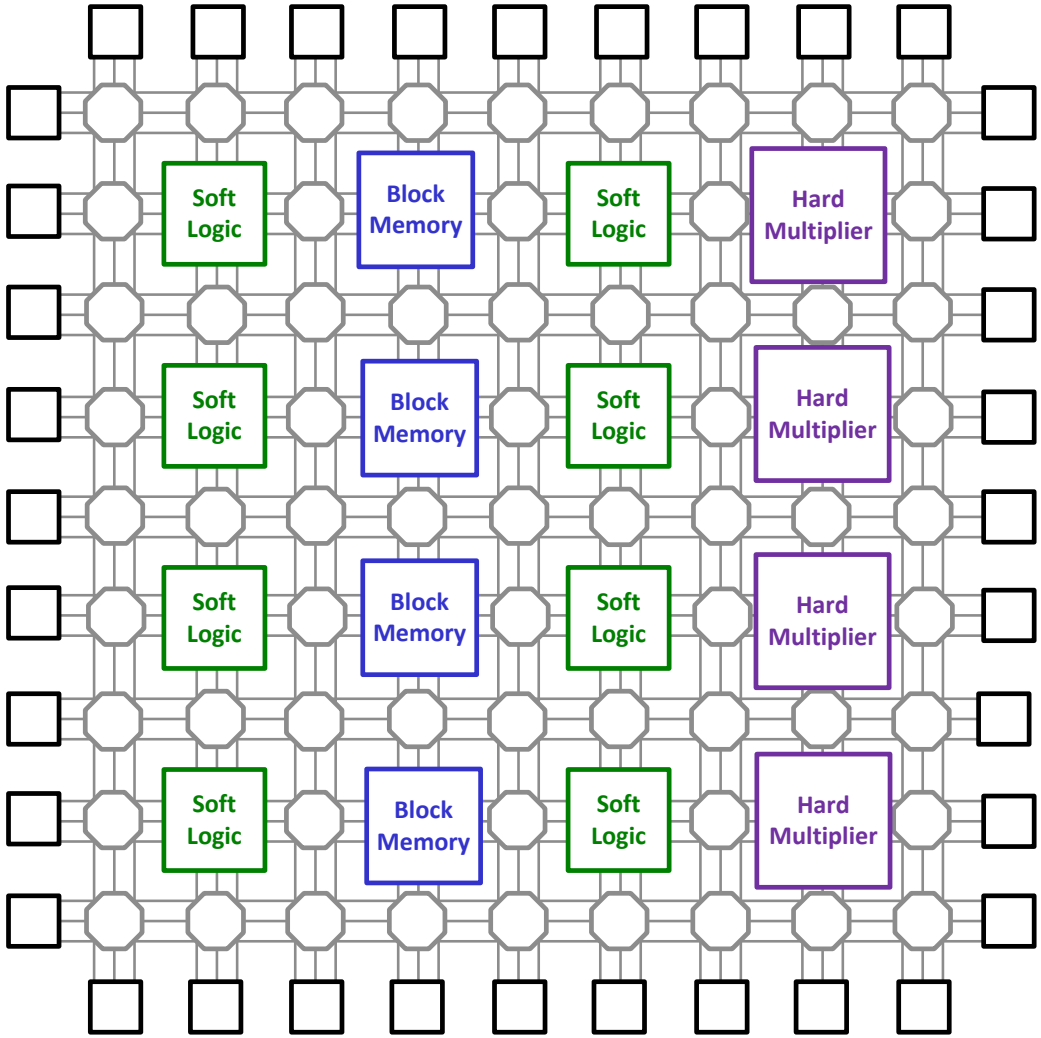
- In all commercial heterogeneous FPGAs, various dedicated blocks are designated to improve area and speed efficiency.

- What kind of specific functions should be included?

Logic Block Design in Heterogeneous FPGAs

- Heterogeneity may exist in two levels:
  - **Extra elements inside general purpose logic blocks: (Soft Logic)**
    - Flip-flops
    - MUXs
    - XOR
    - Blocks to support arithmetic carry, sum, and subtraction functions
  - **Different types of blocks: (Hard Logic)**
    - Multi-bit block RAMs (first used in FLEX 10K)
    - Multiply-accumulation (MAC) blocks (e.g., in Startix I, II, III)
    - Hard multiplier blocks (e.g., in Xilinx Virtex families)

Logic Block Design in Heterogeneous FPGAs



Soft Logic in Heterogeneous FPGAs

- Carry logic modules are dedicated blocks, provided to help implement faster addition operations

- The carry over is passed b/w internal LUTs via dedicated routing
  - General routing is avoided to achieve less signal delay

- Normally an XOR gate is also included in the carry chain to generate the SUM to build an adder

Memory Blocks in Heterogeneous FPGAs

- First appeared in Altera FLEX 10K
- Flexibility of being configured in various aspect ratios is crucial
  - b/c different applications need different block sizes and aspect ratios
  - e.g., in Flex 10K a 2K memory in (1x2048), (2x1024), (256x8)
- Covers a significant fraction of the FPGA die area
- More important in larger systems
- Most complementary FPGAs employ dual-port memory blocks

Computation-Oriented Blocks in Heterogeneous FPGAs

- **Most common: Hard multiplier**
  - e.g., Virtex II contains 18x18 2's complement multipliers
  - Startix I contains a single 36x36 multiplier (can also be broken into eight 9x9 multipliers and an adder to sum the results)
- If multipliers are not used by an application their blocks are wasted
- In order to avoid waste of resources:
  - Multiple sub-families of a device with different ratios of soft logic to hard logic are created (so choose the one that fits the best)
  - For example, Virtex 4/5 have three sub-families:
    - More soft logic and memory
    - Focus on arithmetic unit
    - Focus on high-speed interface

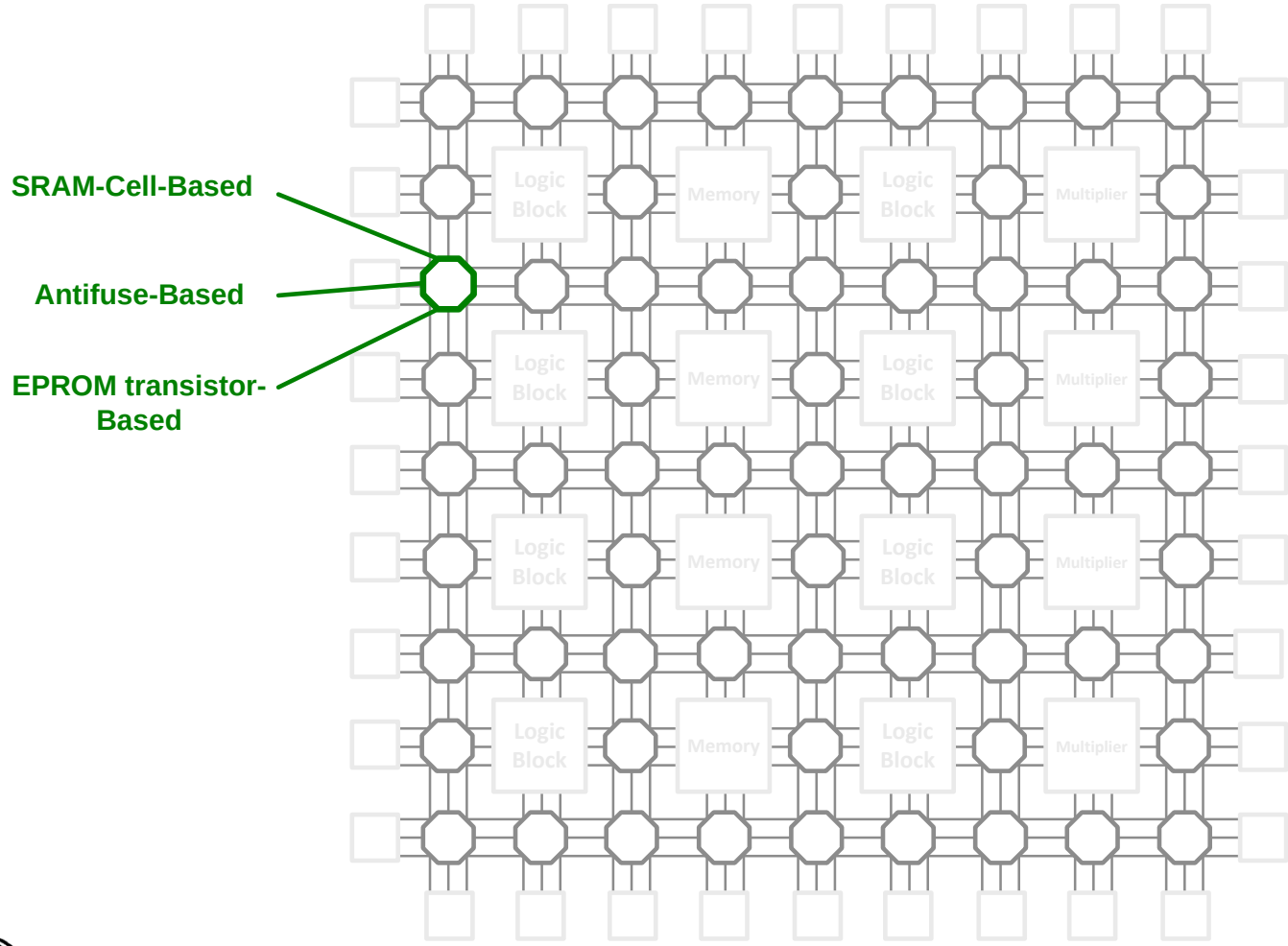
Microprocessors in Heterogeneous FPGAs

- Microprocessors are vital in many digital systems
- Often used in conjunction with FPGA logic
- It is a great idea to integrate it with FPGAs on a single die
- For example:
  - Xilinx Virtex II Pro FPGAs have 1, 2, or 4 IBM power PC cores integrated with Virtex II logic fabric
  - Virtex 4, 5 subfamilies also support power PC cores on the die

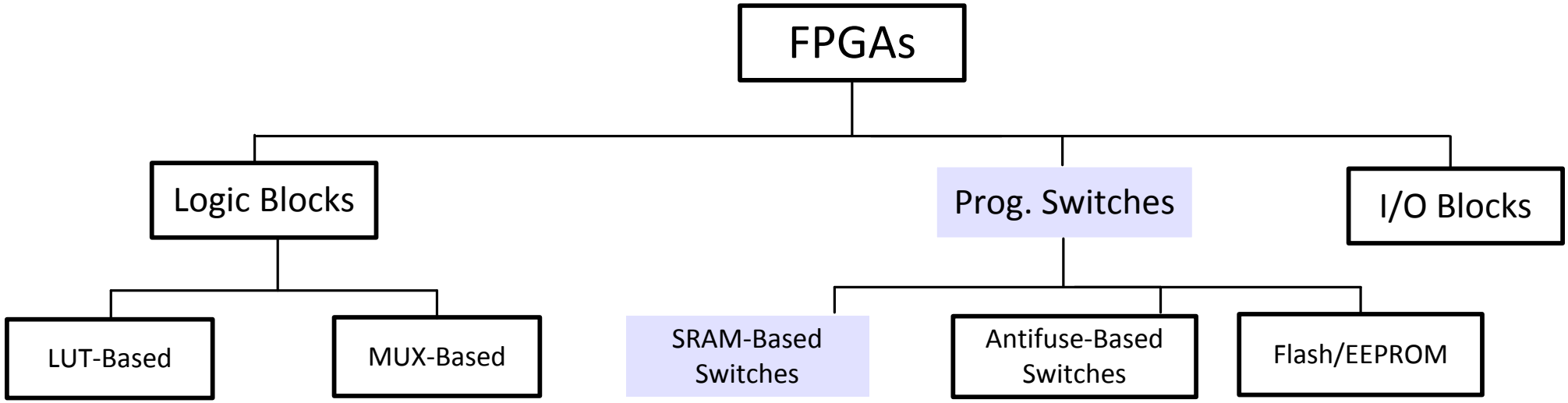
Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- **Field-Programmable Gate Array (FPGAs)**
  - Logic Blocks
  - **Programmable Routing Switches**
  - I/O Pads
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

Programmable Switches

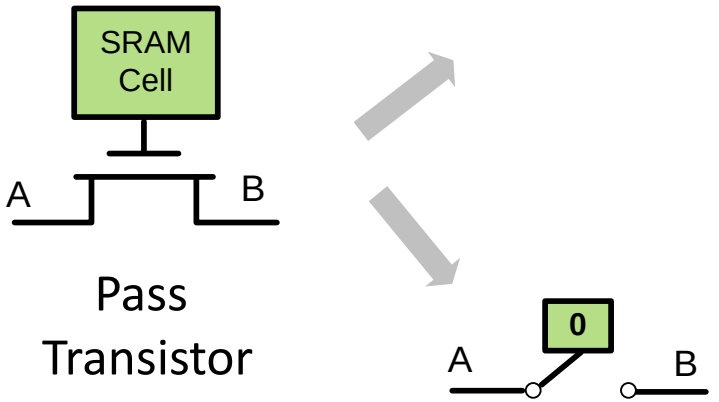


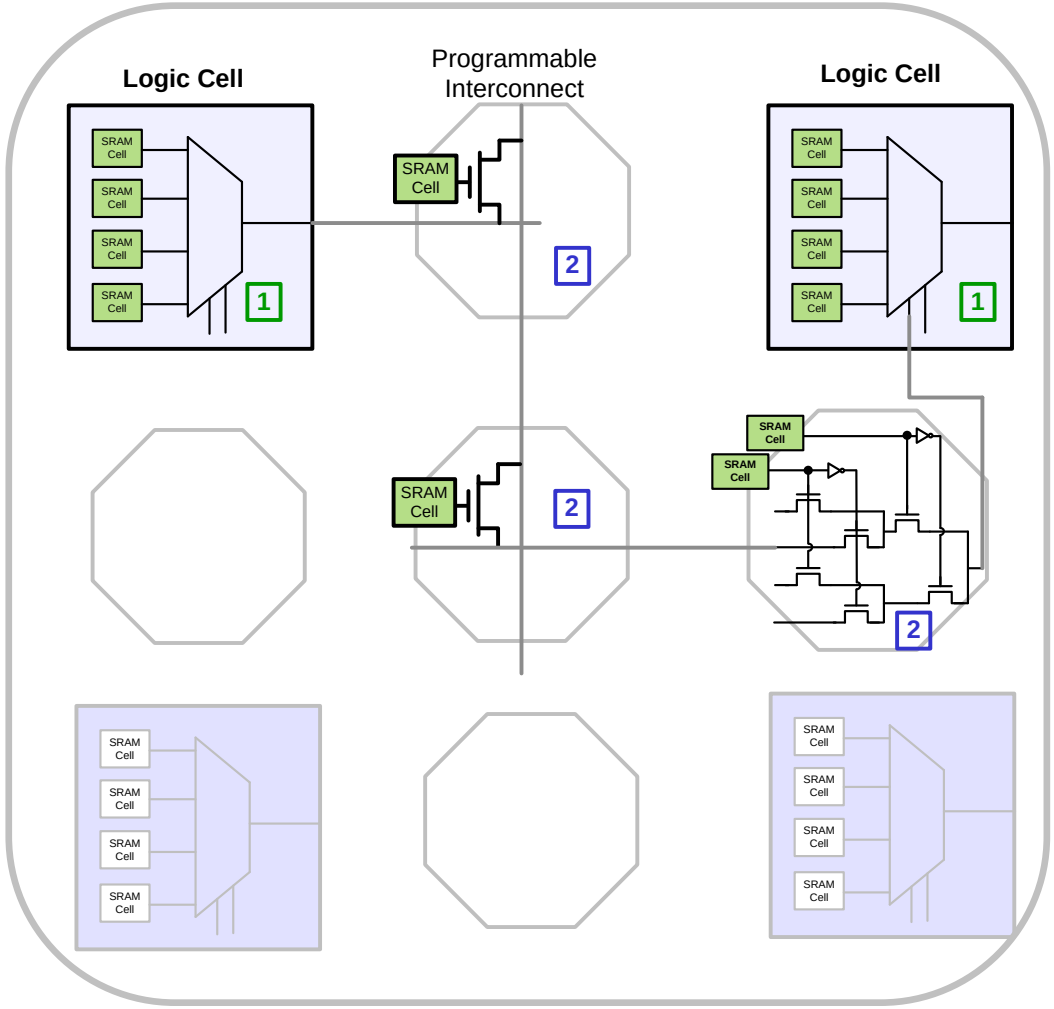
SRAM-Based Programmable Switches



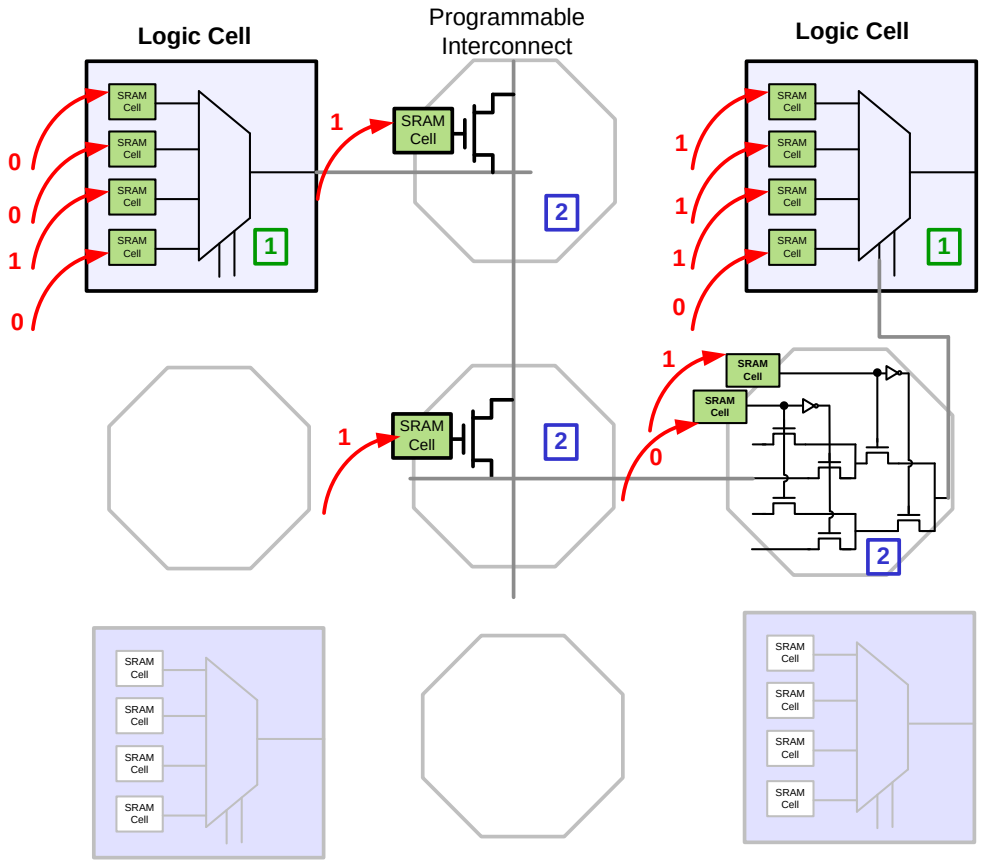
SRAM-Based Programmable Switches

- SRAM Cell is used both in logic blocks and the Prog. Interconnections:





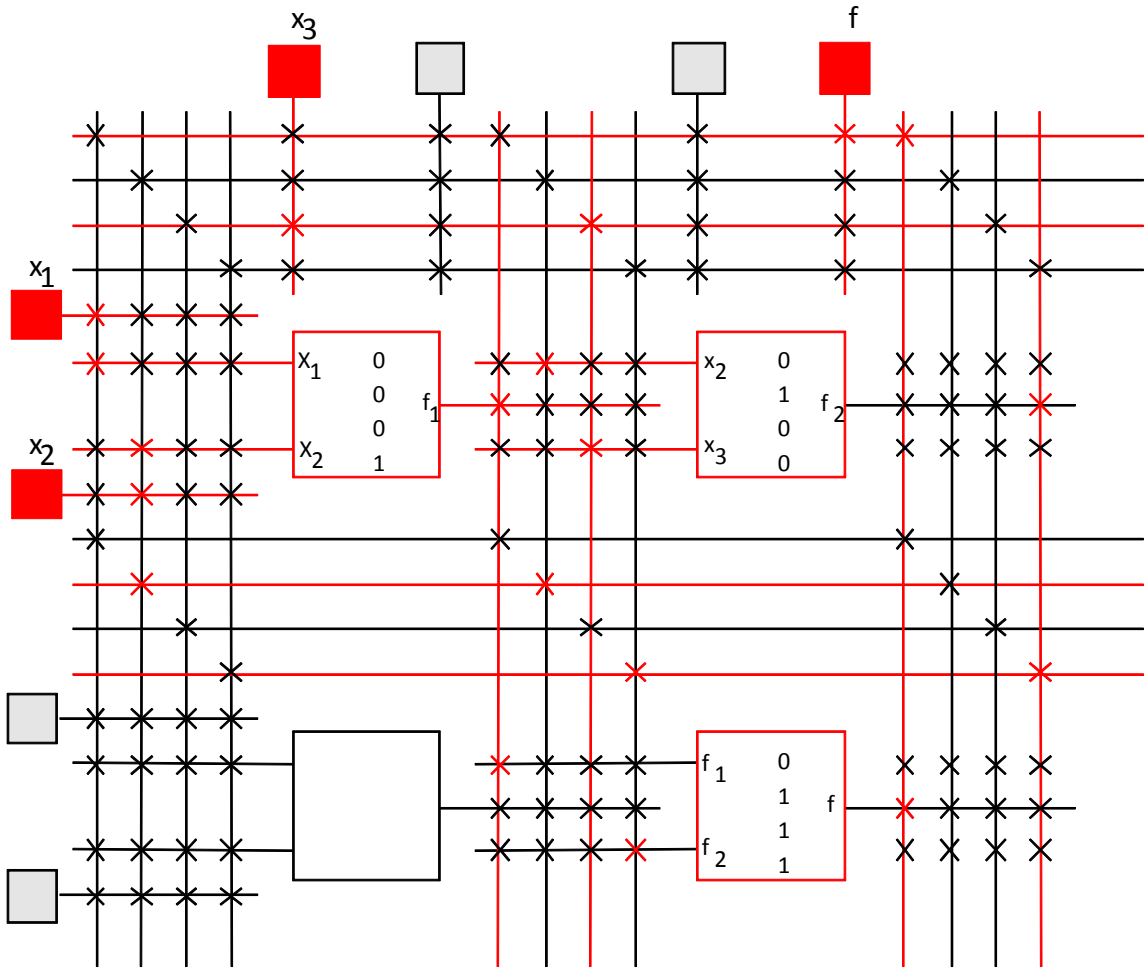
SRAM-Based Programmable Switches



When programming, configuration bits are loaded into SRAM cells both in the LUTs and interconnection switches

Programming FPGAs

- Programming an FPGA by configuring Logic Blocks & Routing



Configuration of SRAM-based FPGA

- SRAM-based FPGAs are reconfigured by changing the content of the SRAM cells in LUTs and programmable interconnect

- A few pins are dedicated for configuration

- Two ways of configuration:
  1. Download the configuration bits directly from PC using a download cable
     - Good for prototyping and debugging mode
     - Not reliable in the production mode
  2. Store configuration bits in PROMs on the PCB with the FPGA
     - Upon power-up they are loaded into the FPGA

FPGA Interconnect Design

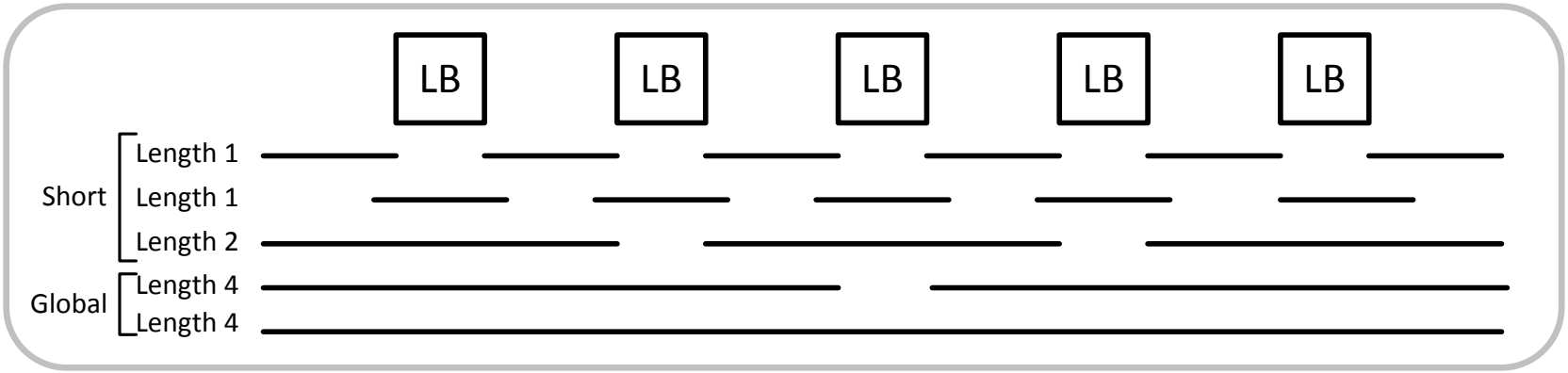
- Interconnect design is really important b/c the most area in an SRAM-based FPGA is consumed by the routing switches.

- Interconnects are organized in wiring channels or “routing channels”

- A typical FPGA has many different kinds of interconnect to be fully customized for different delay/speed requirements:
  - Short wires
  - Global wires
  - General purpose wires
  - Clock distribution network

FPGA Interconnect Design

- In order to make all required connections b/w logic blocks efficiently, FPGA routing channels have wires of a variety of lengths (segmentation)
- **Segmentation:**
  - **Short wires:** Connect only local logic blocks (e.g., the carry chain in LBs)
    - Do not take up much area and have small delay
  - **Global wires:** Designed for long-distance communication
    - May have built-in electrical repeaters to reduce delay



SRAM-Based Programmable Interconnect

- Interconnect design in SRAM-Based FPGAs is tricky b/c the circuitry can introduce significant delay and cost a large silicon area.
- Two options:
  - Pass transistor
  - Three-state buffer (larger but provides amplification)

Pass Transistor

Tri-state Buffer

SRAM-Based Programmable Interconnect

- These elements introduce delay to the interconnect
- **Objective:** reduce the delay, How?
  1. Increase the width of the transistors
     - Less delay (**good**)
     - More silicon area (**bad**)
  2. Increase the wire width
     - Less resistance (**good**)
     - More capacitance (**bad**)
- How much we should increase the width?

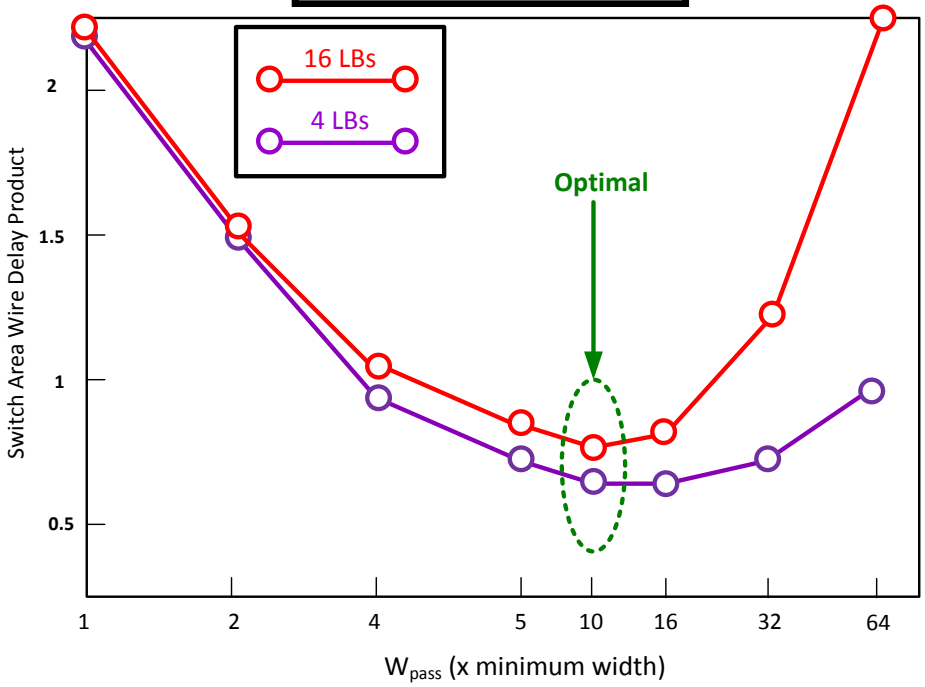
Define a metric: Area-delay product

(To consider both restrictions)

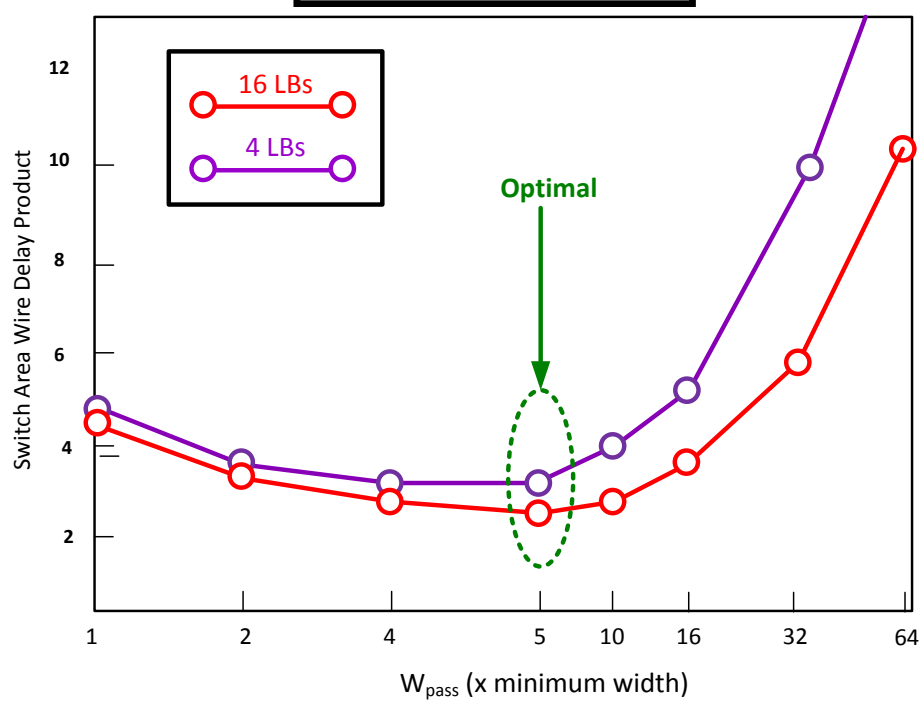
SRAM-Based Programmable Interconnect

- Consider 4 and 16 logic blocks

Pass Transistor



Tri-state Buffer



The tri-state buffer requires smaller transistors (b/c it provides amplification)

SRAM-Based Programmable Switches

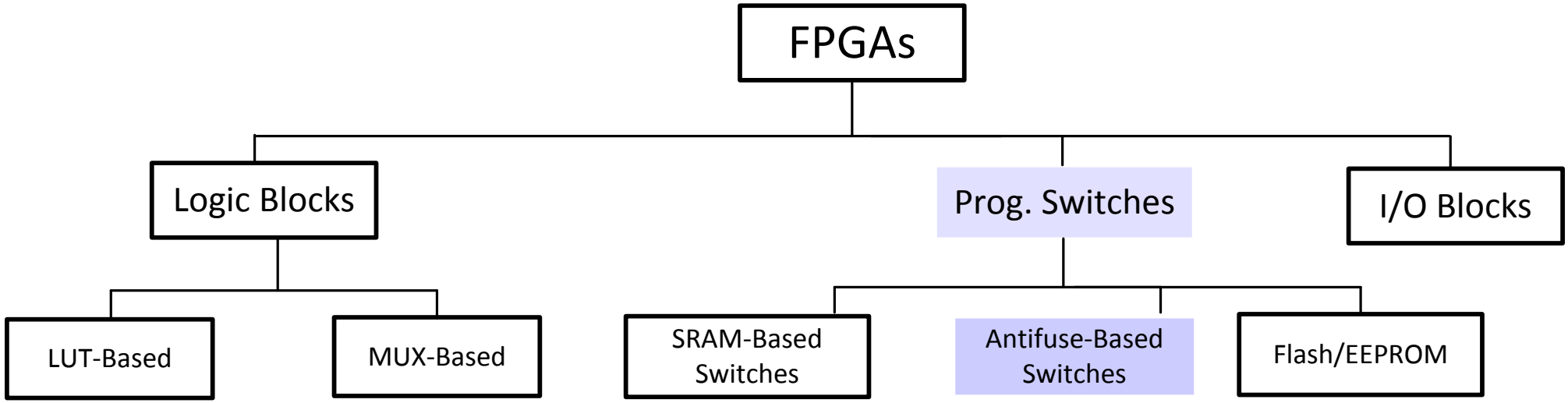
❑ Advantages:

- ➢ Re-programmability (infinite number of times)
- ➢ Use of standard CMOS fabrication process technology
  - ▪ Use of the latest CMOS technology
  - ▪ Benefits from increased integration, higher speed, lower dynamic power

❑ Drawbacks:

- ➢ **Size**: SRAM cell requires 6 transistors
- ➢ **Volatility**: an external device (like an EPROM) is needed to permanently store the configuration bits when the device is powered down (extra cost)
- ➢ **Non-ideal pass transistors**: SRAM cells rely on pass transistors that have large on-resistance and capacitance load
- ➢ **Reliability**: the bits in the SRAM are susceptible to theft

Antifuse-Based Programmable Switches



Antifuse-Based Programmable Switches

- The programmable element is an antifuse
- Programmed by applying a voltage across it
  - Normal condition: high resistance link
  - When programmed (blown): low resistance (20-100 Ohm)
    - Permanently programmed (unlike SRAM)

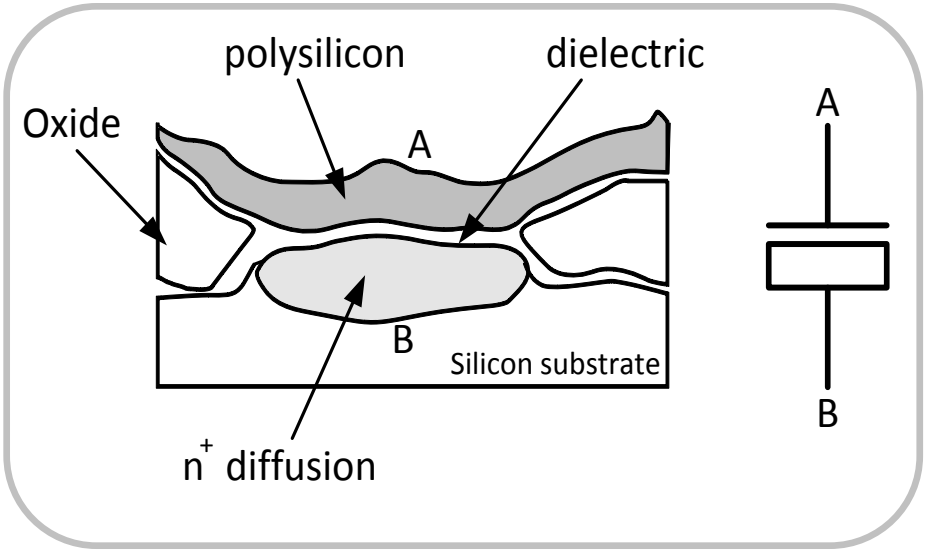
A high voltage blows the antifuse so it conducts

- Why antifuse and not fuse?
  - Well, interconnect networks are sparsely populated, which means that most of them are not connected
  - So antifuse is used, which is an open circuit by default

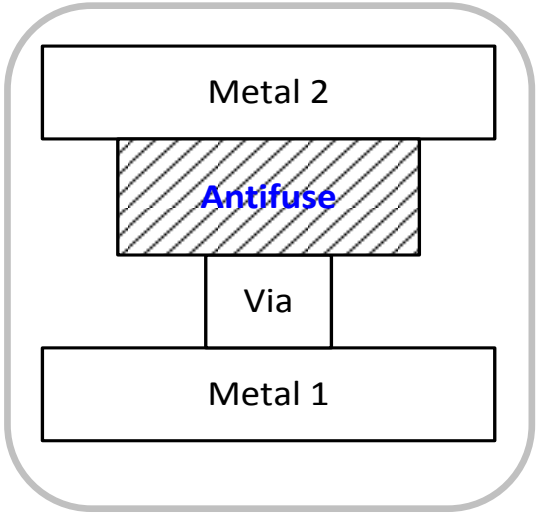
Antifuse-Based Programmable Switches

- **Two general structures:**

Poly to Diffusion (Actel)

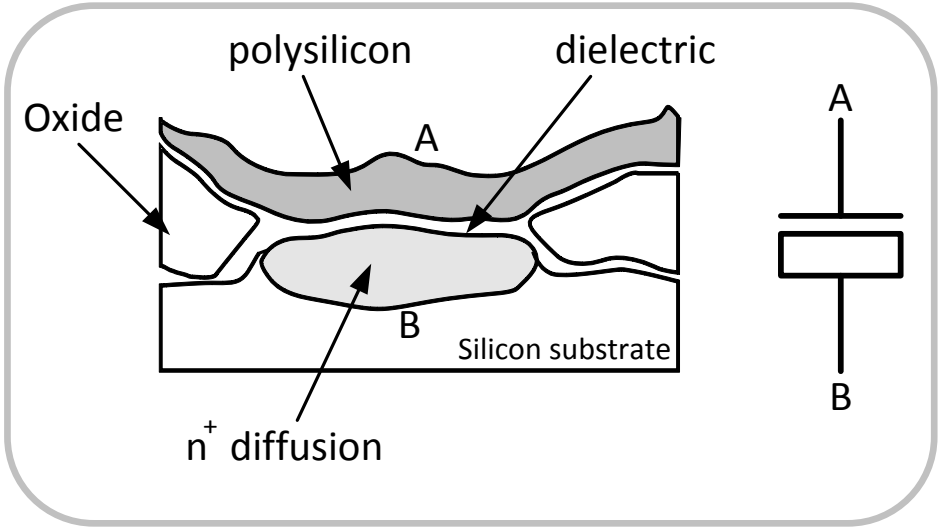


Metal to Metal (Via Link)



Antifuse: Poly-to-Diffusion (Actel)

- Three-layer sandwich structure: (called **PLICE**)
  - **Top layer**: polysilicon (conductor)
  - **Middle layer**: dielectric (insulator)
    - Isolates top and bottom (un-prog.)
    - Low-resistance link (programmed)
    - Amorphous silicon or silicon oxide
  - **Bottom layer**: n^+ diffusion (conductor)



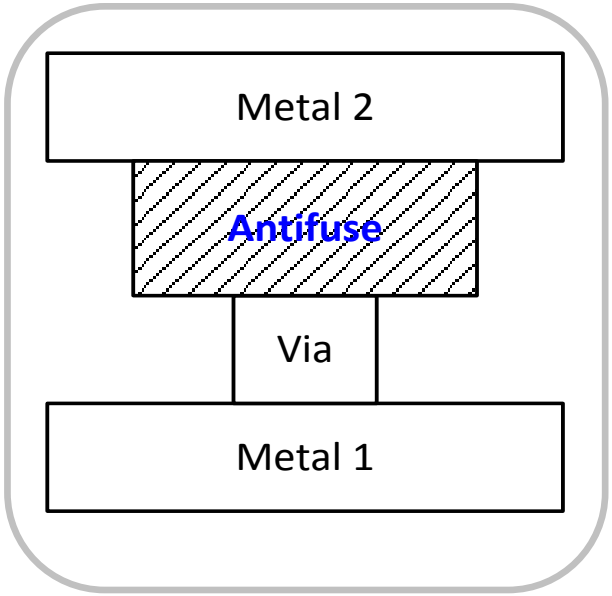
Antifuse

- Each antifuse in the FPGA has to be programmed separately

A high voltage/current breaks down/melts the insulator and it conducts (Permanent Link)

Antifuse: Metal-to-Metal (QuickLogic)

- Three-layer sandwich structure: (called **ViaLink**)
  - **Top layer**: Metal (conductor)
  - **Middle layer**: Thin amorphous Si (insulator)
    - Isolates top and bottom (un-prog.)
  - **Bottom layer**: Metal (conductor)



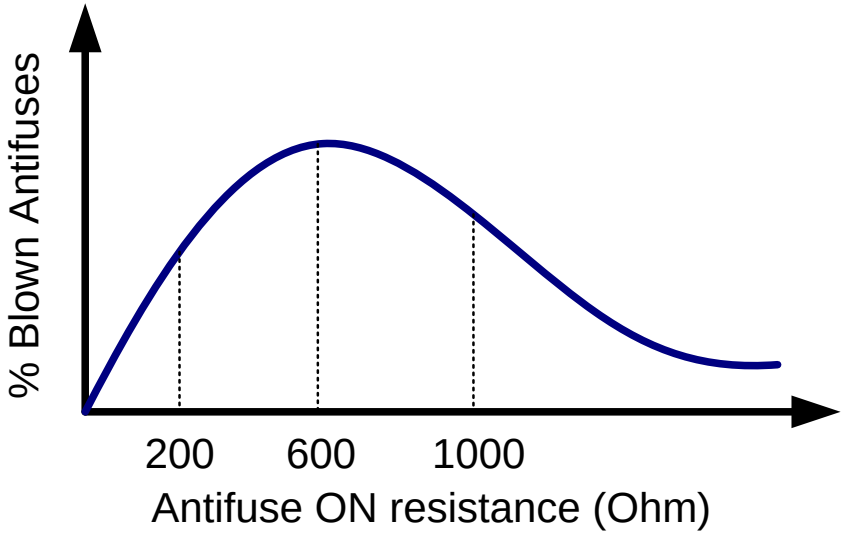
Antifuse

- **Advantages:**
  - Direct metal to metal eliminating connection b/w poly & diffusion thus reducing parasitic capacitance and interconnect space requirement
  - Lower resistance

Antifuse-Based Programmable Switches

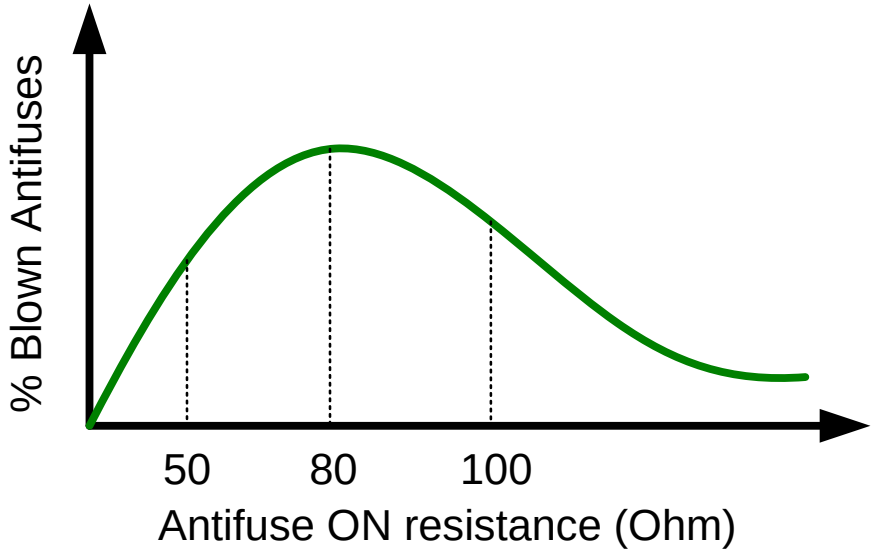
❑ Comparison of the ON resistance

Poly to Diffusion (Actel)



PLICE

Metal to Metal (QuickLogic)



ViaLink

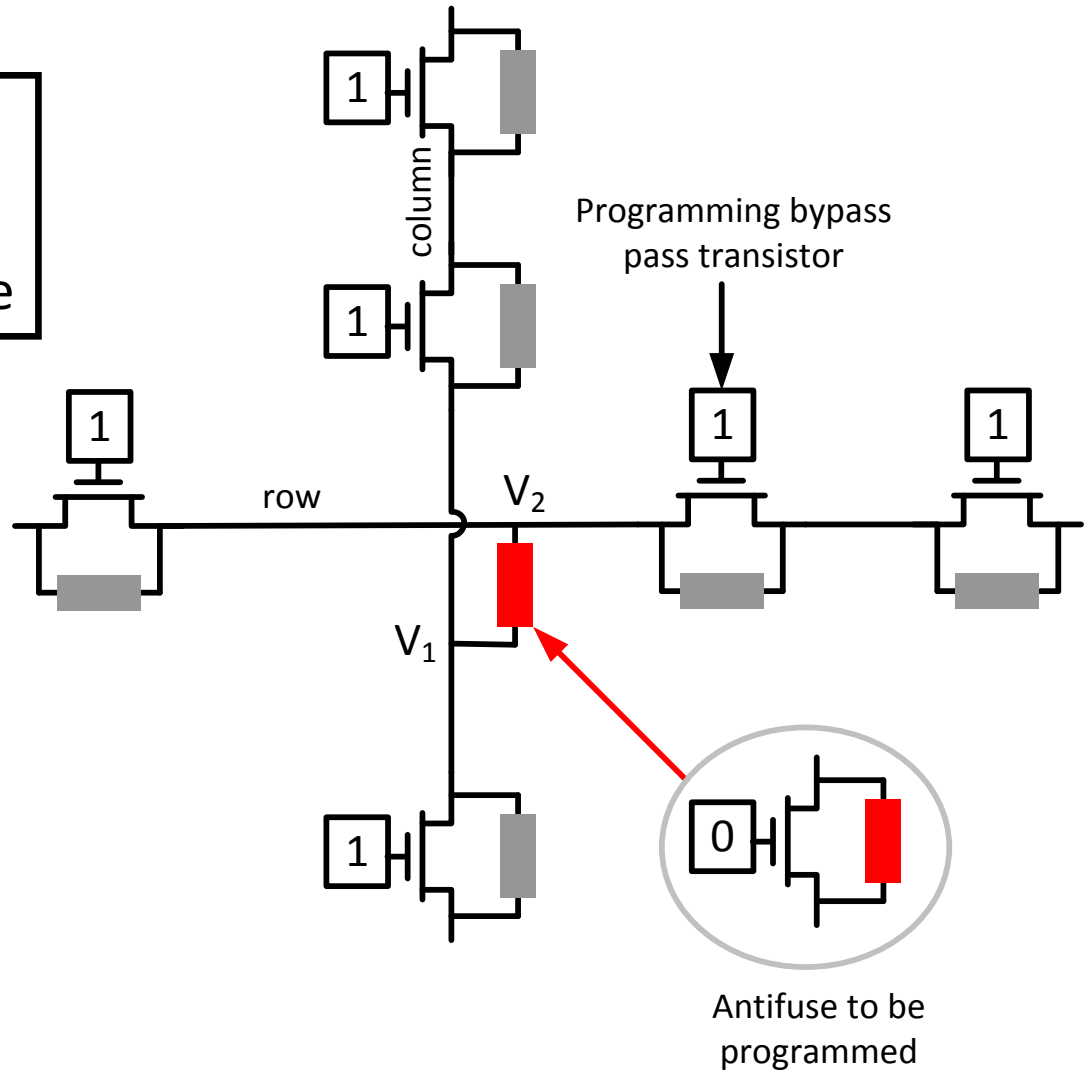
Antifuse-Based Programmable Switches

- An antifuse slows down the interconnect path less than a pass transistor in a SRAM-based FPGA.
- To be able to program every antifuse, each antifuse is connected in parallel with a pass transistor
- The pass transistor allows the antifuse to be bypassed during programming
- Gates of the pass transistors are controlled to select the appropriate row & column for the desired antifuse
- Voltage is applied across row/column so that only the desired antifuse receives the voltage.
- FPGA has circuitry that allows each antifuse to be separately programmed

To program an antifuse-based FPGA, chip is plugged into a socket on a special programming box that generates the programming voltage.

Antifuse-Based Programmable Switches

The voltage is applied across rows/columns so that only the desired antifuse receives the voltage



Antifuse-Based Programmable Switches

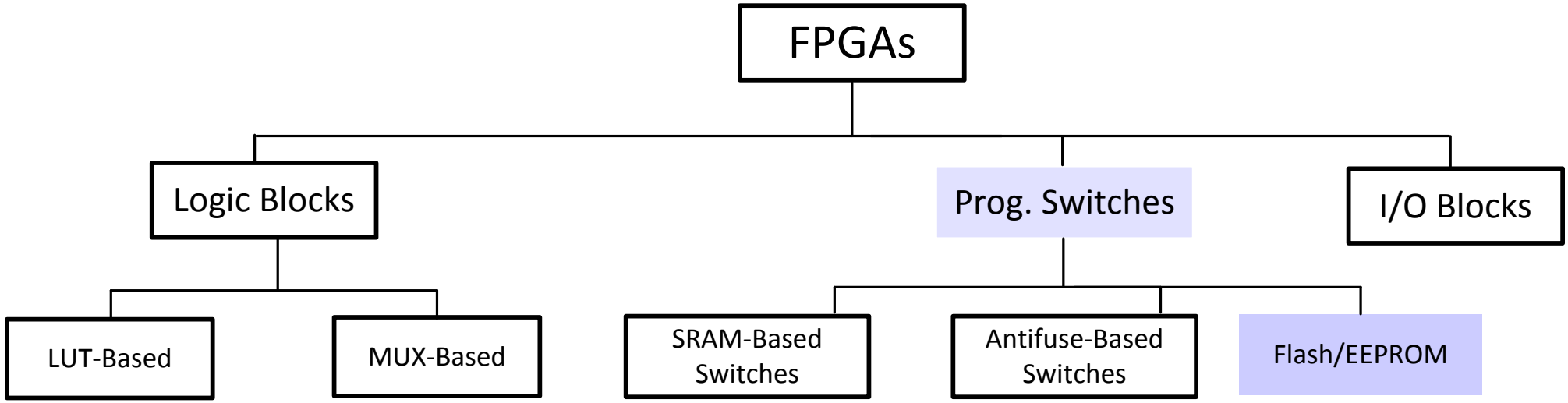
❑ Advantages:

- ➢ Requires no silicon area (low area) ⟹ more switches per device
- ➢ Lower resistance and parasitic capacitance than other technologies
- ➢ Non-volatility means
  - ▪ Instant operation
  - ▪ No need for additional on-chip memory (as opposed to SRAM-based)

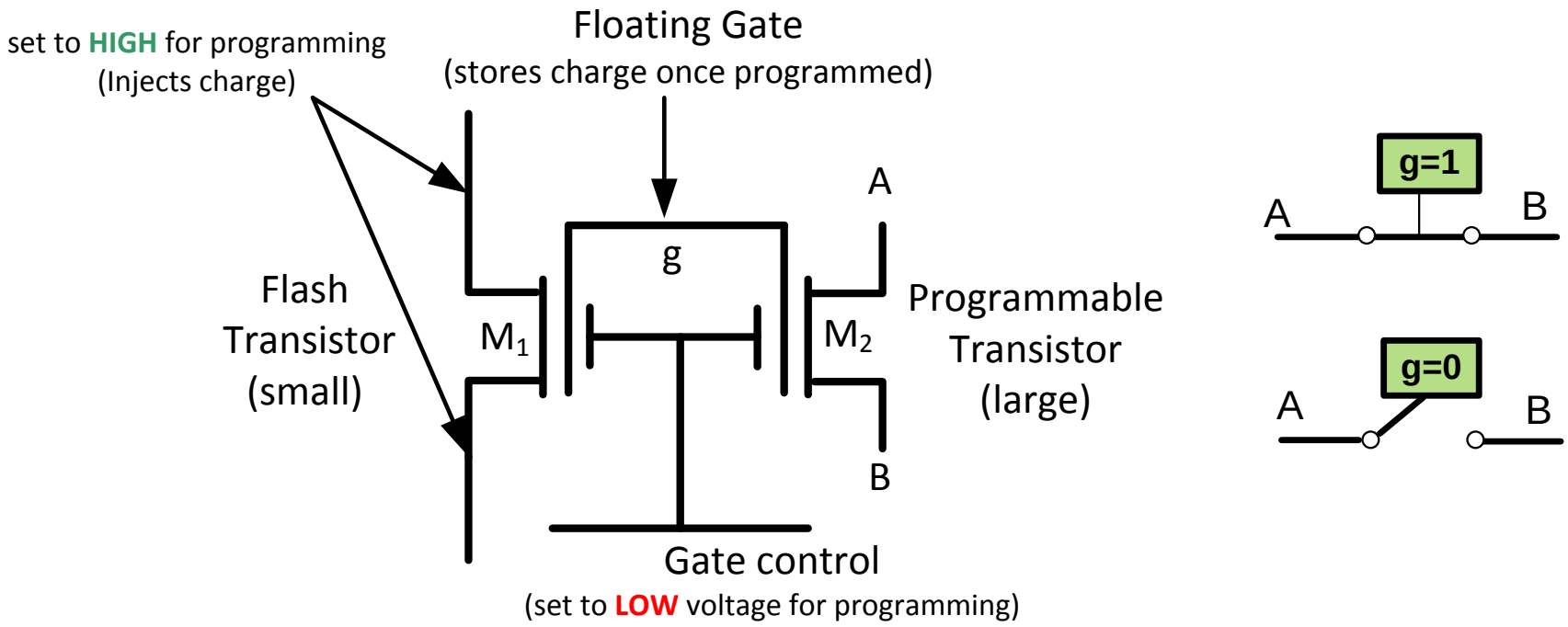
❑ Drawbacks:

- ➢ Requires non-standard CMOS process
  - ▪ Behind SRAM-based tech. in manufacturing
- ➢ Scaling challenges for antifuse
  - ▪ Hard to realize in deep sub-micron
- ➢ Not re-programmable

EEPROM/Flash-Based Programmable Switches



EEPROM/Flash-Based Programmable Switches

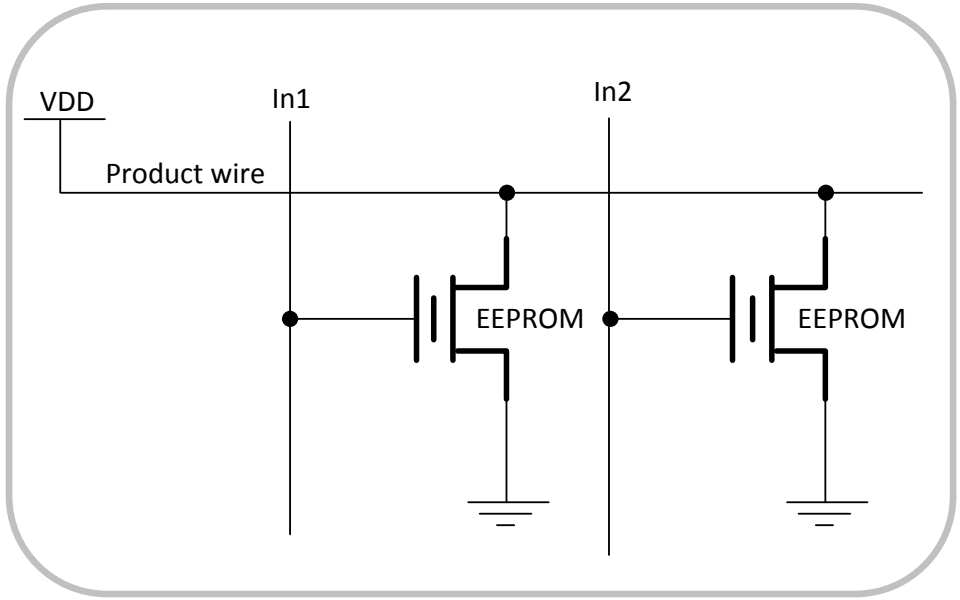


- Flash memory is a high-quality programmable read-only memory
- Has a floating gate structure, where a low-leakage capacitor holds a voltage that controls a transistor gate
- This memory cell can be used to control programming transistors

EEPROM/Flash-Based Programmable Switches

- An EEPROM transistor is also used as a programmable switch for CPLDs by placing the transistor between two wires in a way that facilitates implementation of wired-AND functions.

- An input to the AND plane can drive a product wire to ‘0’



EEPROM/Flash-Based Programmable Switches

❑ Advantages:

- ➢ Non-volatile ⟹ Does not lose information when the device is powered off
  (Thus no extra memory/flash is required)
- ➢ Improved area efficiency (less transistors needed compared to SRAM-cell)
- ➢ Re-programmable

❑ Drawbacks:

- ➢ Tricky floating-gate design
  - ▪ source-drain voltage should be low enough to prevent charge injection into the floating gate
- ➢ Can NOT be reprogrammed infinite number of times!
  - ▪ b/c of charge build-up in the oxide (e.g., Actel ProASIC3 are rated for 500 times)
- ➢ Uses non-standard CMOS process
- ➢ High resistance and capacitance due to the use of transistor-based switches

Programmable Switches

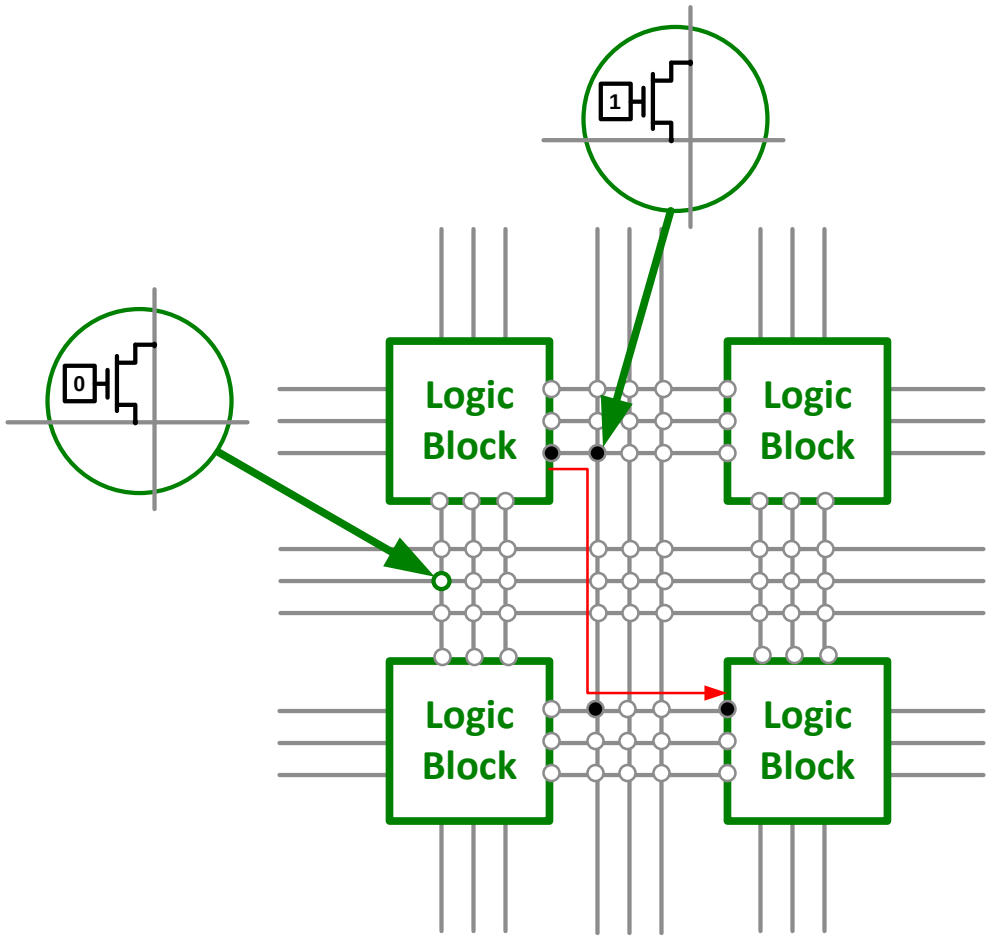
- So there are three technologies for switches:
  - SRAM cell
  - Antifuse
  - Flash-based
- The ideal technology is the one that is:
  - Non-volatile
  - Reprogrammable infinite number of times
  - Based on standard cell CMOS process
  - Offer low on resistance and capacitance
- Recent trend by Xilinx, Altera and Lattice:
  - On-chip flash memory for storage of configuration bits
  - SRAM-based interconnect switches

Comparison Between All Technologies

Manufacturer	SRAM	Flash/EEPROM	Antifuse
Volatile	Yes	No	No
Re-Programmable	Yes	Yes	No
Area	High (6 transistors)	Moderate(1 transistor)	Low(0 transistor)
Manufacturing Process	Standard CMOS	Flash Process(EECMOS)	Antifuse (CMOS+)
In-system Programmable	Yes	Yes	No
Switch Resistance	500-1000 Ohm	500-1000 Ohm	20-100 Ohm
Switch Capacitance	1-2fF	1-2fF	<1 fF
Yield	100%	100%	>90%

Routing Channels

❑ Interconnect wiring is grouped into routing channels, each of which contains a complete grid of horizontal and vertical wires.



Routing Channels

- FPGA wiring with programmable interconnect is slower than typical wiring in a custom chip b/c:
  - Pass transistor on an interconnect is not a perfect on-switch
  - Programmable interconnect is slower than a pair of wires permanently connected by a via
  - FPGA wires are generally longer than would be necessary for a custom chip

Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- **Field-Programmable Gate Array (FPGAs)**
  - Logic Blocks
  - Programmable Routing Switches
  - **I/O Pads**
- Commercial FPGA Products
- Application Specific Integrated Circuits (ASICs)

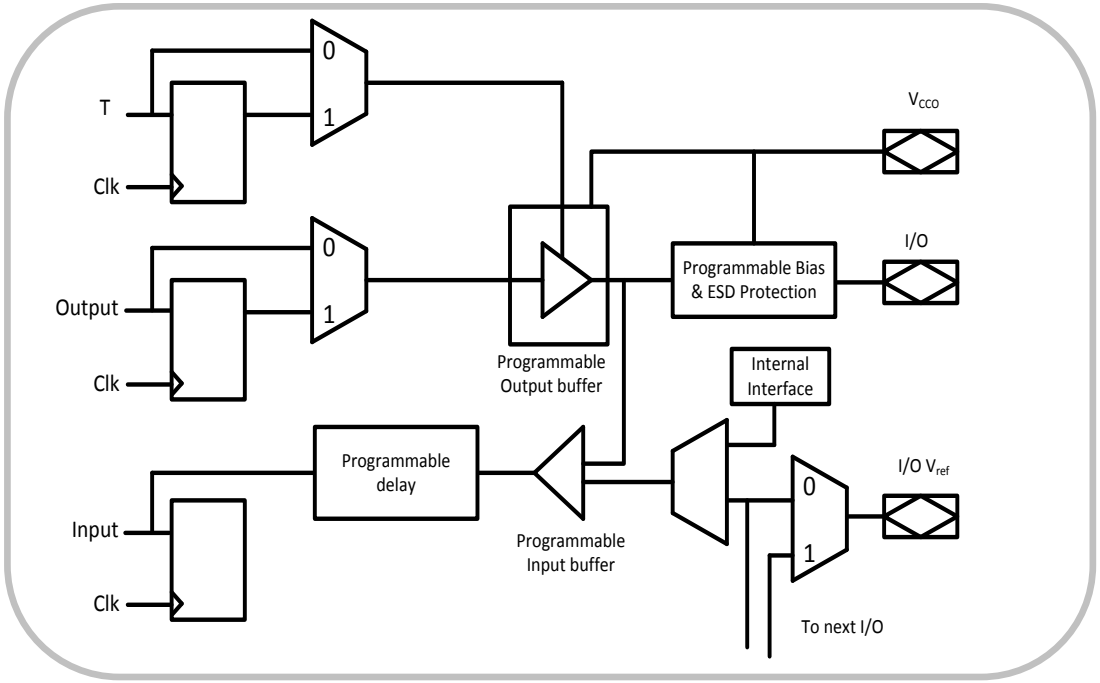
FPGA Chip I/O

- I/O pins on a chip connect it to the outside world and perform some basic functions
  - Input pins provide electrostatic discharge (ESD) protection
  - Output pins provide buffers with sufficient drive to produce adequate signals on the pins
  - Three-state pins include logic to switch b/w input and output modes
- The pins on an FPGA can be configured to act as
  - Input pin
  - Output pin
  - Tri-state pin

Xilinx Spartan II 2.5V Family I/O Pins

- Supports a wide range of I/O standards
- The I/O has three registers, one each for input, output and tri-state operation
  - Each has its own enable signal
  - They all share the same clock connection
  - Can be configured as latch or FF

The Prog. delay on the input path is to eliminate variations in hold times from pin to pin



Xilinx Spartan II 2.5V Family I/O Pins

- Supports a wide range of I/O standards, divided into eight banks
- Pads within each bank share the same reference voltage, threshold voltage and use standards that have the same V_{CCO}

I/O Standard	Input Ref. Voltage (V_{ref})	Output Source Voltage (V_{CCO})
LVTTL	N/A	3.3
LVCMOS2	N/A	2.5
PCI	N/A	3.3
GTL	0.8	N/A
HSTL Class I	0.75	1.5
SSTL3 Class I/II	1.5	3.3
CTT	1.5	3.3
AGP-2X	1.32	3.3

Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- **Commercial FPGA Products**
- Application Specific Integrated Circuits (ASICs)

Commercial FPGA Products

- As of summer 2009

Manufacturer	FPGA Products	LUT/Antifuse based	Floorplan
Actel	MX, SX, eX, Axcelerator	Antifuse-based	Row-Based
QuickLogic	PASIC, QuickRAM, Eclipse (Plus/II)	Antifuse-based	Symmetrical array
Lattice	ECP2/M, SC	Antifuse-based	Symmetrical array
Atmel	AT40K, AT40KAL	LUT-Based	Hierarchical PLD
Altera	Stratix (II/III/IV), Cyclone (II/III), Arria (II), Flex 8000, 10K	LUT-based	Hierarchical PLD
Xilinx	Virtex-II Pro, Virtex-(E,4,5,6) Spartan-(II/3) (A/E), XC4000	LUT-based	Symmetrical array

Most Dominant

Commercial FPGA Products

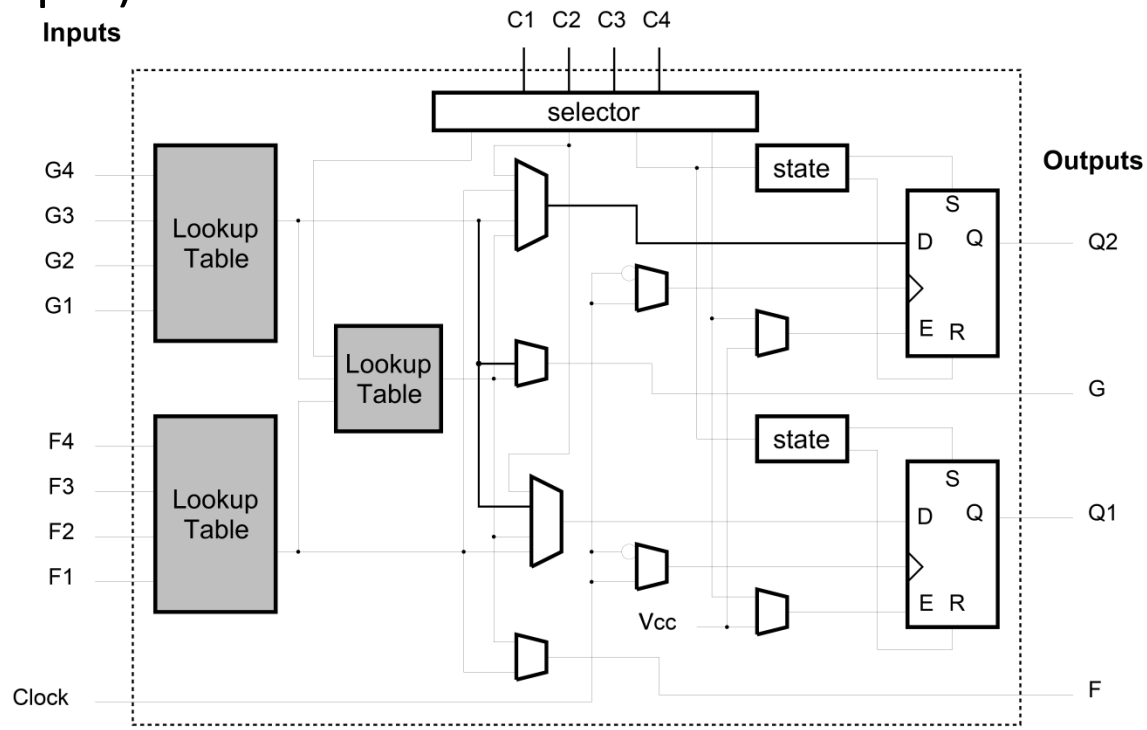
- **Xilinx:**
  - **SRAM-Based:**
    - XC2000
    - XC3000
    - XC4000
    - XC5000
    - Virtex Family (II Pro, 4, 5, 6)
    - Spartan Family
  - **Antifuse-Based:**
    - XC8100

Xilinx (XC4000 Series)

- 2,000 to 15,000 gates (XC4085 supports up to 100,000 gates)
- The building block in Xilinx FPGAs is called **Configurable Logic Block (CLB)**
- XC4000 CLB is LUT-based and consists of
  - 3 LUTs (two 4-input and one 3-input)
  - 2 Flip-Flops (FFs)
- These 3 LUTs allow implementation of:
  - Logic functions of up to 9 inputs
  - Two separate 4-input functions
- Each CLB contains circuitry that allows

Implementation of fast carry operations

(soft logic, coarse-grained)



Xilinx (XC4000 Series): Interconnect

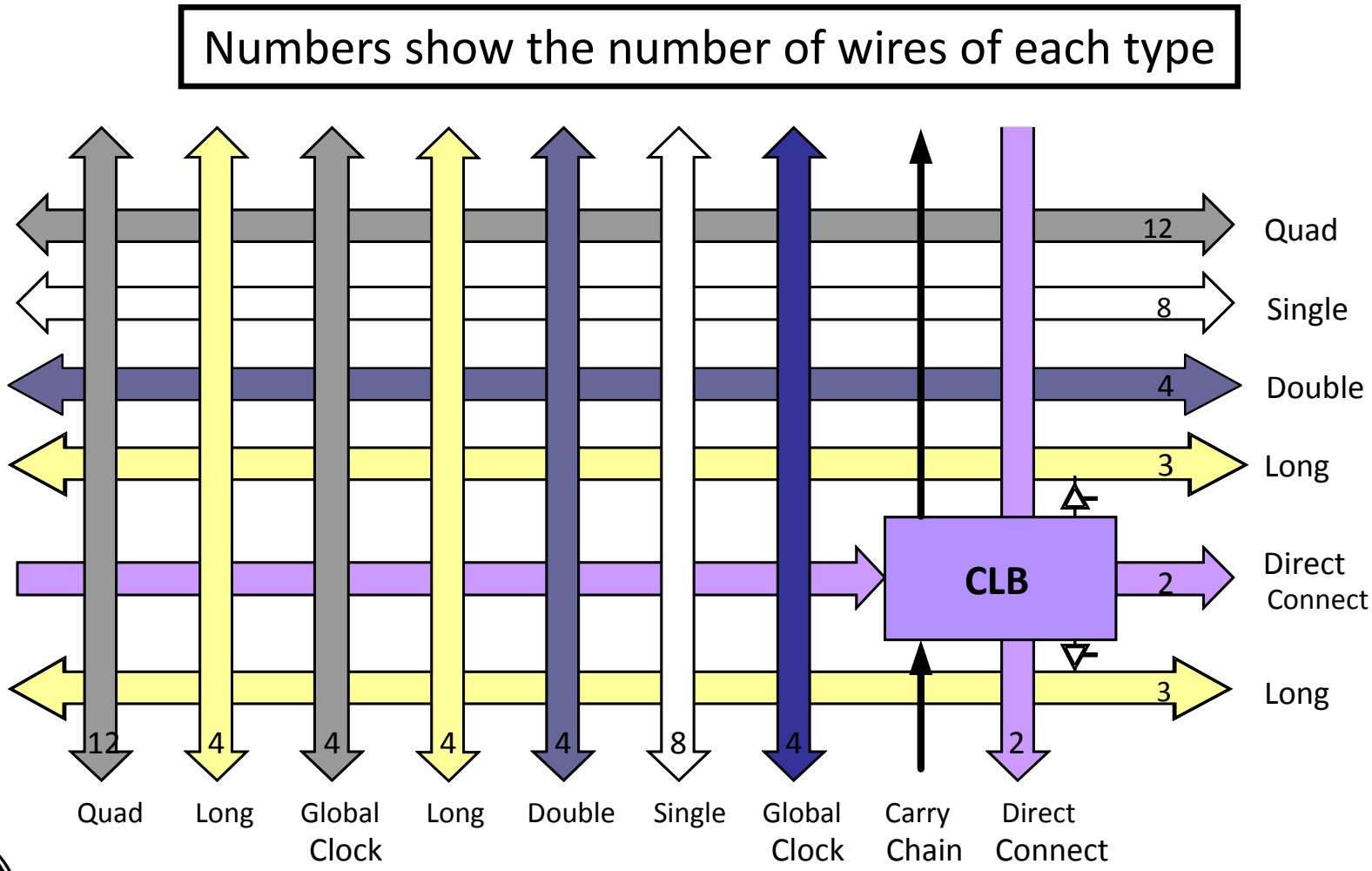
- Consists of horizontal and vertical channels

- Wires in each channel in XC4000 series are of different types
  - **Wire segments:** of length 1, 2, 4 (single, double, quad)
  - **Direct interconnect:** For local connections, with min delay, small fan-out
    - Effective for implementation of fast arithmetic modules
  - **Long Wires:** for global routing, high fan-out,
    - Used for time critical signals or signals distributed over long distances (Bus)
  - **Special wires:** for clock routing

LUTs in a CLB can be configured as read/write RAM cells

Xilinx (XC4000 Series): Interconnect

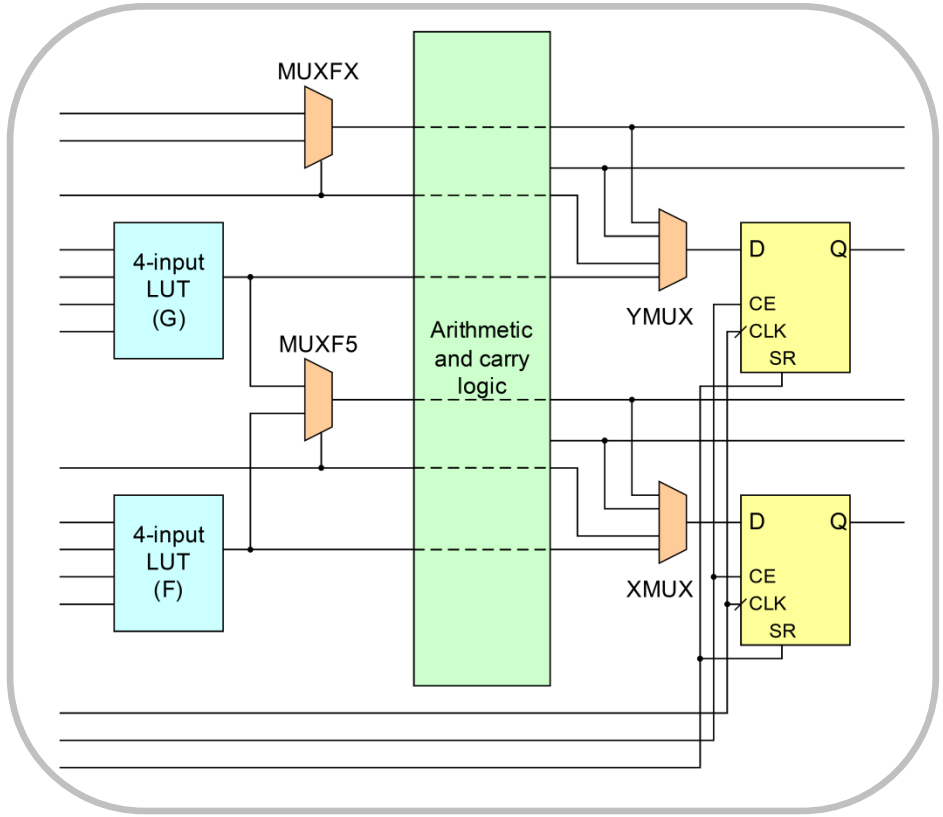
- Interconnect Architecture:



Xilinx (Virtex Series)

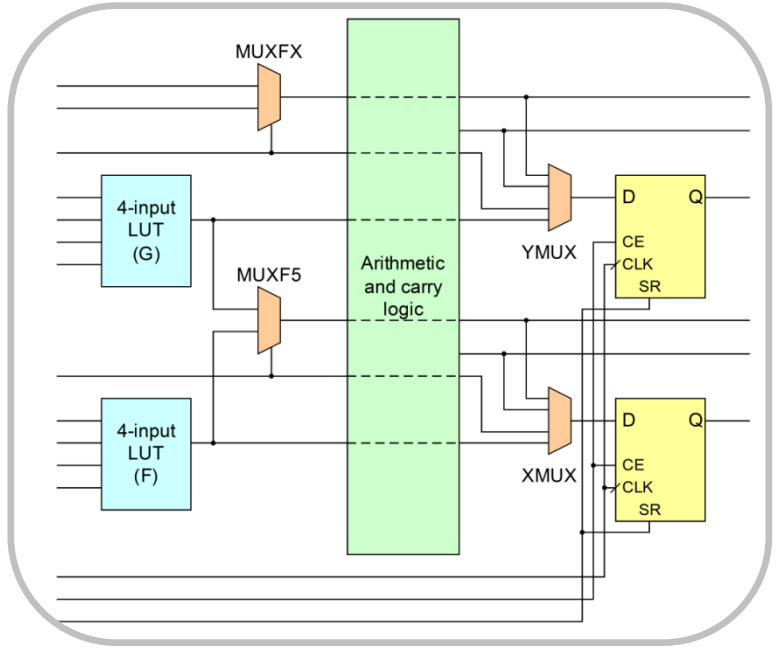
- The elementary Prog. block in Virtex/Spartan FPGAs is called **"Slice"**
- Two slices form a **Configurable Logic Block (CLB)**
- Inside each Virtex 4 slice:

Virtex-4 Slice:



Xilinx Virtex 4 Slice Architecture

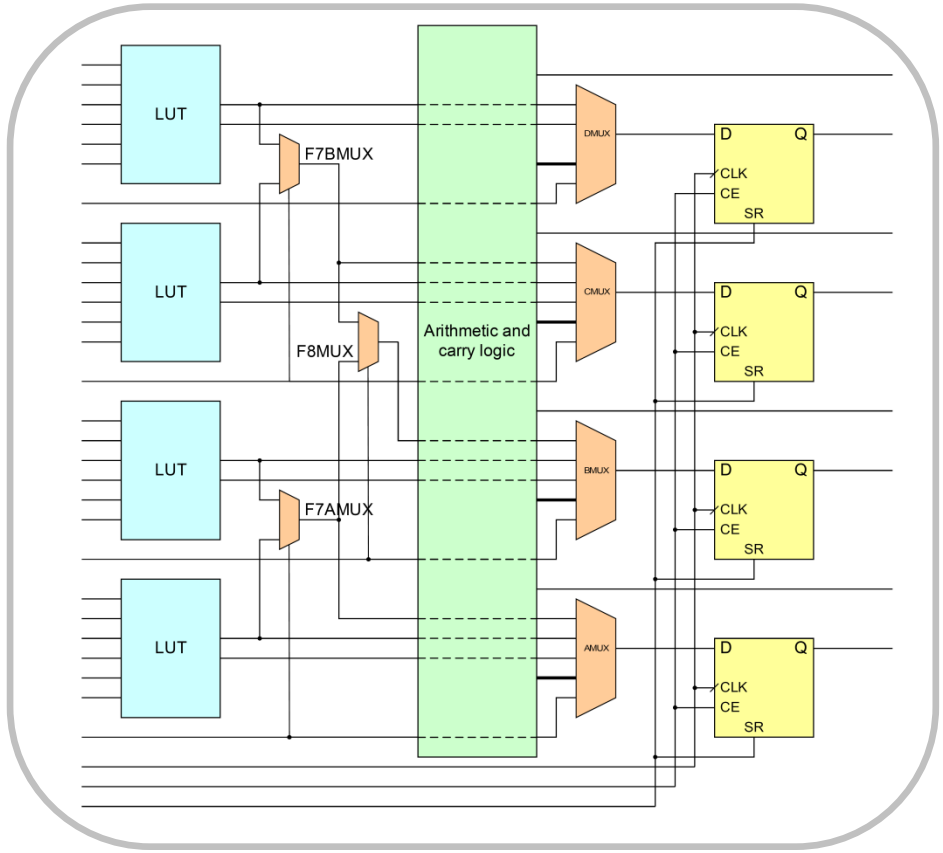
- Two 4-input LUTs (G, F)
- Two dedicated user-controlled MUXs for combinational logic
  - MUXF5 to combine outputs of G, and F to implement 5-input combinational circuit.
  - MUXFX to combine outputs of the other MUXF5 and MUXFX (from the other slices).
- Two 1-bit registers (configured as FF or latches)
- YMUX/XMUX to control the input to the registers
- Dedicated arithmetic logic
  - Two 1-bit adders
  - Carry chain
  - Two AND gates for fast multiplication



Xilinx (Virtex 5 Series)

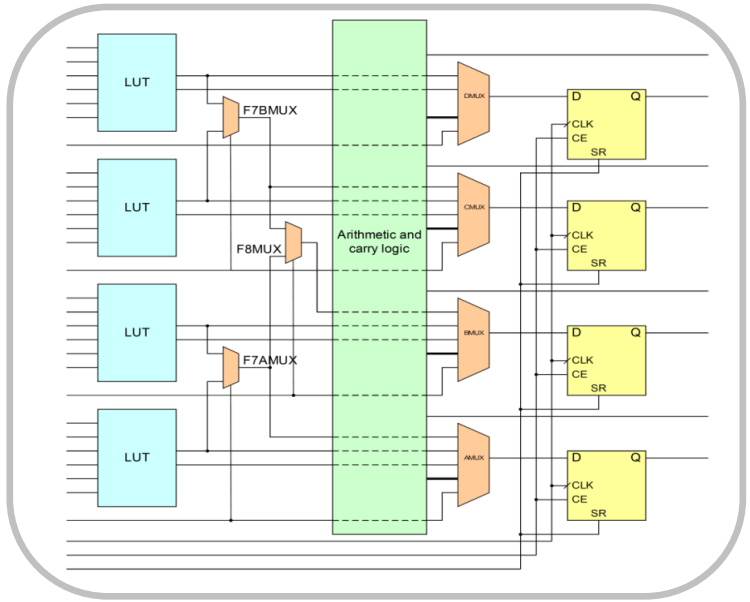
- The Virtex 5 Slice consists of four 6–input LUTs
  - As opposed to two 4-input in Virtex 4

Virtex-5 Slice:



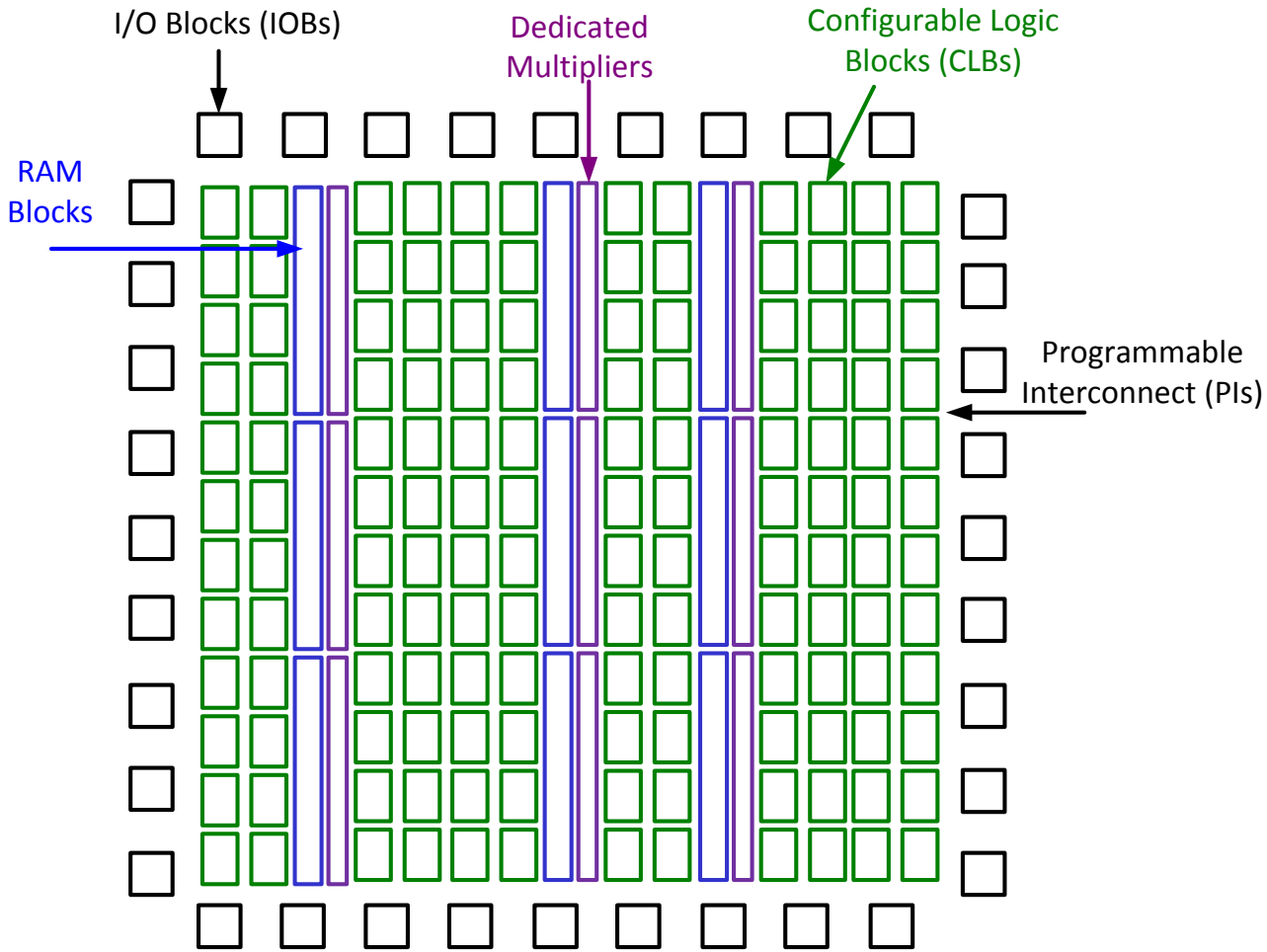
Xilinx Virtex 5 Slice Architecture

- Four LUTs that can be configured as:
  - 6-input LUTs with one output
  - 5-input LUTs with two outputs
- Three dedicated user-controlled MUXs for combinational logic
  - F7AMUX/F7BMUX to combine outputs of the LUTs to implement 7-input combinational circuits.
  - F8MUX to combine outputs of F7AMUX/F7BMUX from the other slices).
- Four 1-bit registers (configured as FF or latches)
- Dedicated arithmetic logic
  - Two 1-bit adders
  - Carry chain
  - Two AND gates for fast multiplication



Xilinx Spartan II

- Heterogeneous blocks:



Commercial FPGA Products

- **Altera:**
  - **SRAM-Based:**
    - FLEX 8000
    - FLEX 6000
    - FLEX 10000
    - Cyclone II/III
    - Stratix II, III, IV

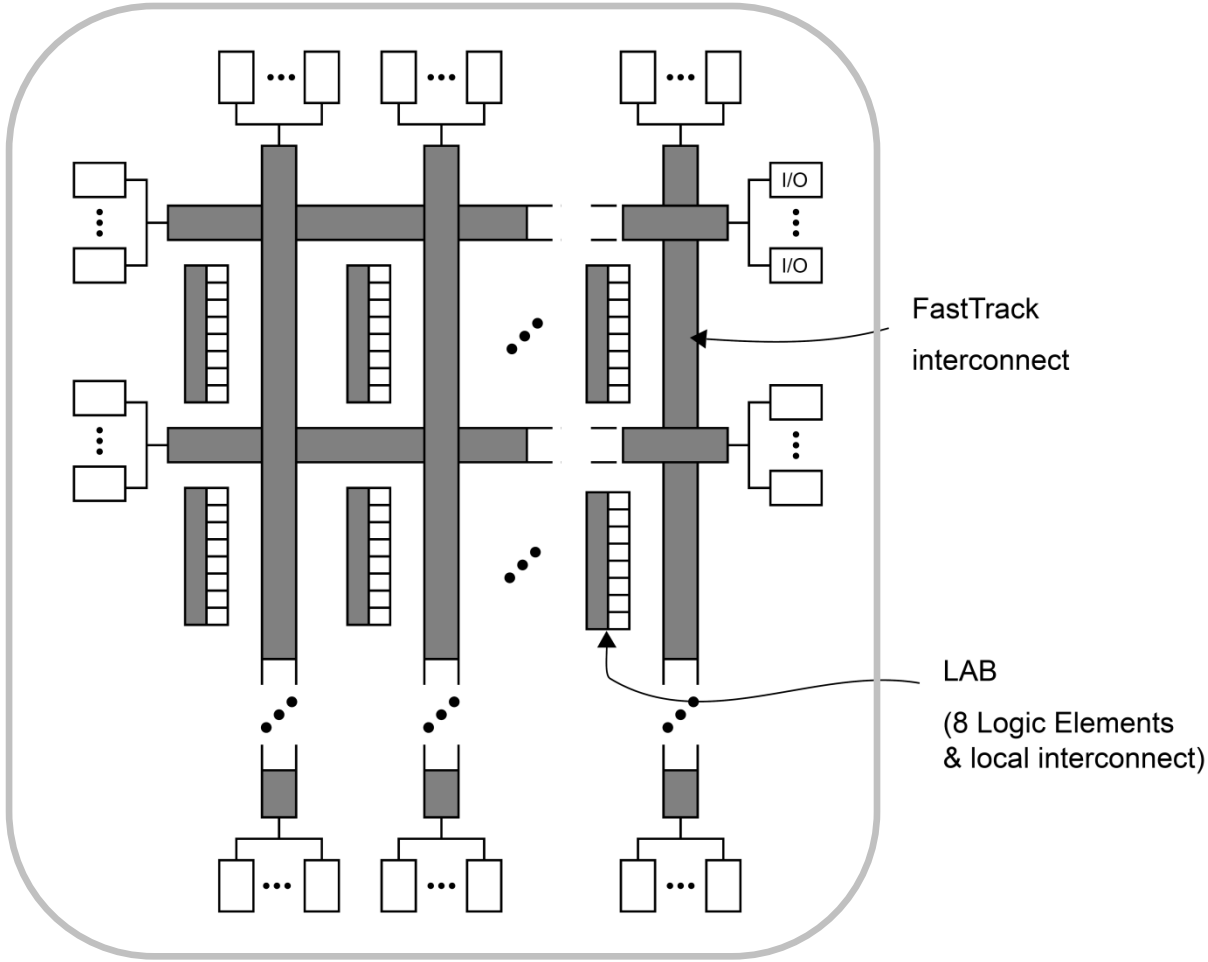
Altera (FLEX 8000 Series)

- The logic block in Altera FPGAs is called **Logic Element (LE)**

- FLEX8000 contains three main components
  1. The main building block is called **Logic Array Block (LAB)**
     - Contains eight LUT-based LEs
  2. FastTrack interconnect
     - Horizontal and vertical to connect LABs
  3. I/O pads

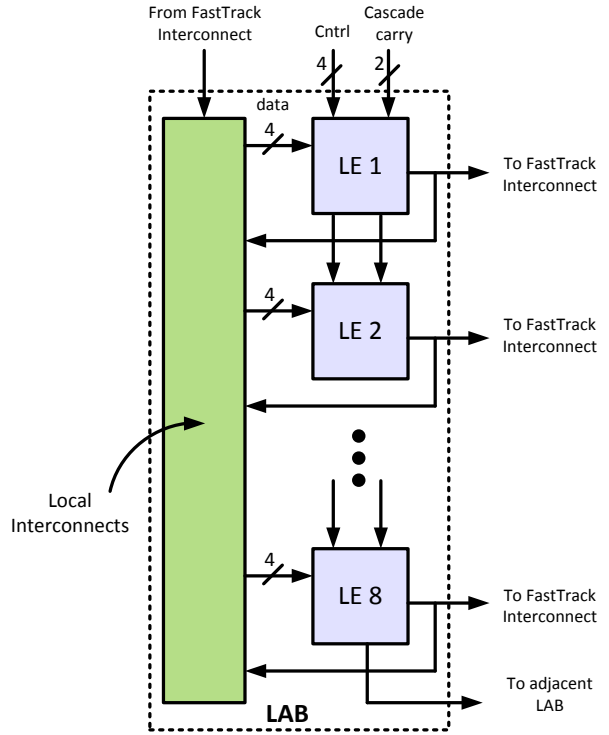
Altera (FLEX 8000 Series)

- Architecture of FLEX8000
  - LAB
  - FastTrack
  - I/O



Altera (FLEX 8000 Series)

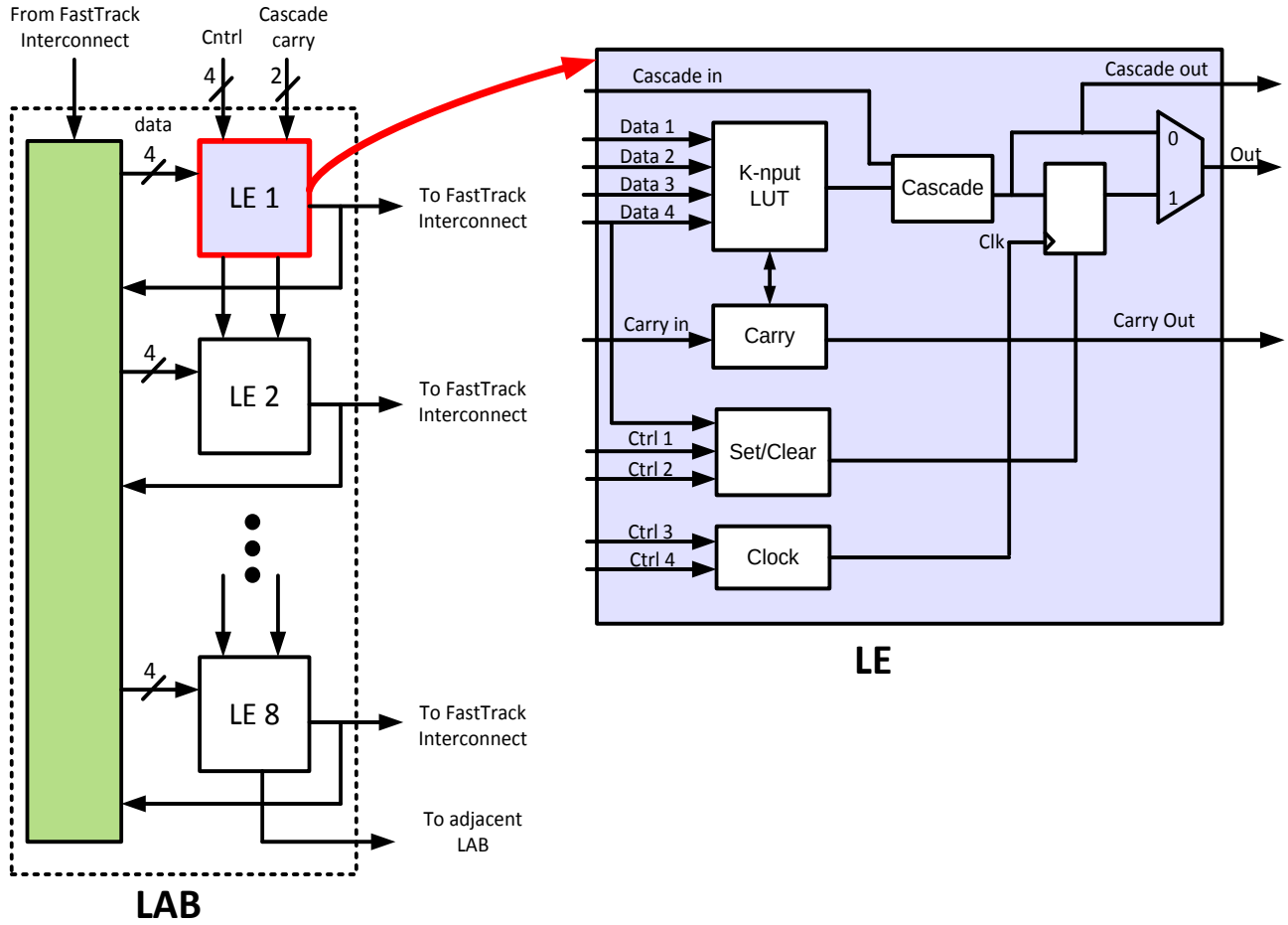
- Each FLEX8000 LAB is a group of eight LEs
- Each LAB:
  - Has a number of inputs provided from the adjacent row interconnect wires
  - Its outputs connect to the adjacent row/column wires
  - Contains local interconnects to connect any LE to another LE inside the same LAB
  - Connected to the global interconnect (fastTrack)
    - Similar to the Xilinx long lines



Altera (FLEX 8000 Series)

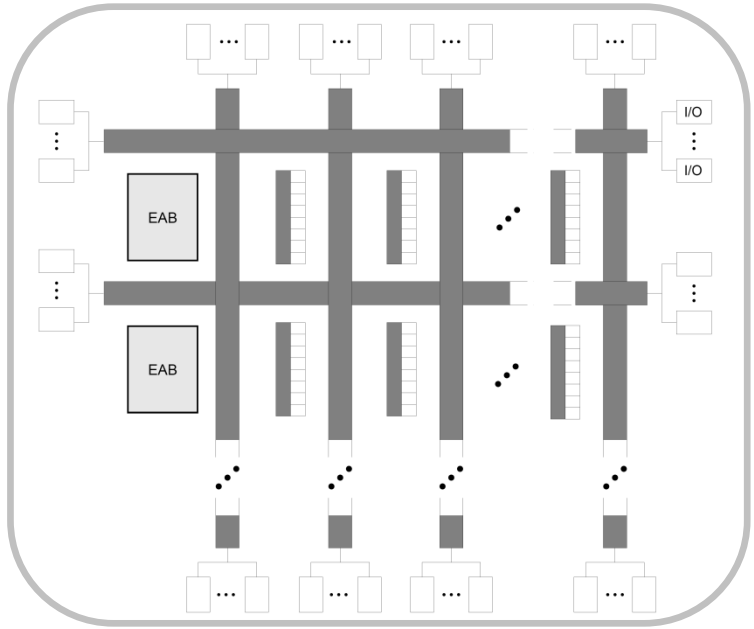
- Each FLEX8000 LE is LUT-based and consists of :
  - A 4-input LUT(to implement two 3-input functions, i.e. sum/carry fcns in a full adder)
  - A Flip-Flops (FF)
  - Carry circuitry
  - Cascade circuitry

(soft logic, coarse-grained)



Altera (FLEX 10K Series)

- Offers all the features of FLEX8000
- It also has variable-sized blocks of SRAM in each row
  - Called **Embedded Array Block (EAB)**
- Each LAB can serve as
  - An SRAM block with aspect ratios of
    - (256X8) (512X4) (1KX2) (2KX1)
  - A large multi-output LUT
    - To implement a complex circuit
    - For example a multiplier



- FLEX 10K chips are in sizes from 10K10 (10,000 gates) to 10K250 (250,000 gates)
- Chips are in various speeds, indicated by a speed grade
  - For example: 10K10-1 (faster) or 10K10-2 (slower)

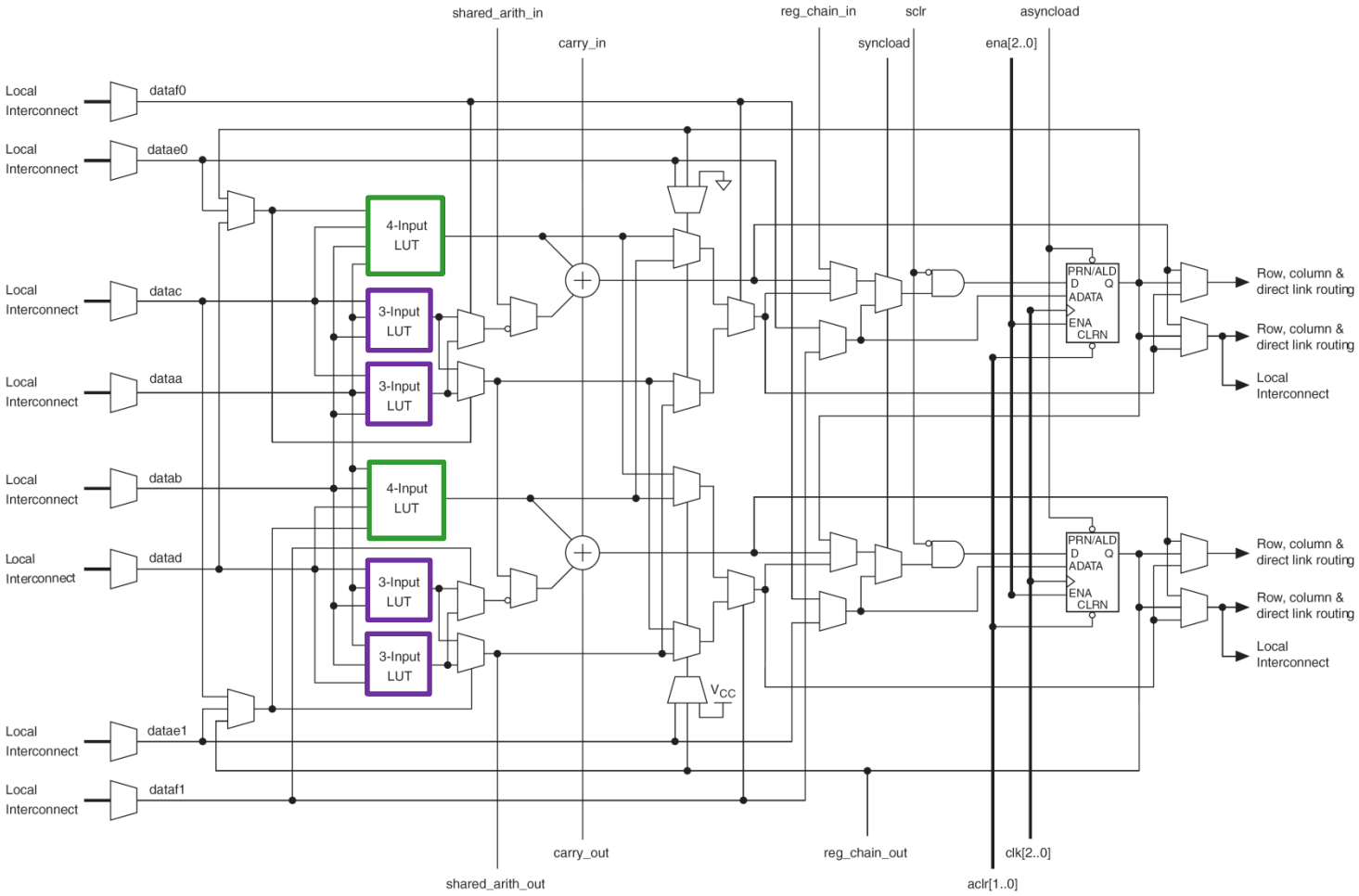
Altera (Stratix II)

- Using an Adaptive Logic Module (ALM) as its logic element.

- Stratix II ALM is an 8-input structure that can implement many combinations of functions including:
  - One 6-input logic function
  - Two 4-input logic functions
  - One 5-input and one 3-input logic functions
  - Two 6-input logic functions that share the same function and four inputs

Altera (Stratix II)

- Stratix II Adaptive Logic Module (ALM) structure:

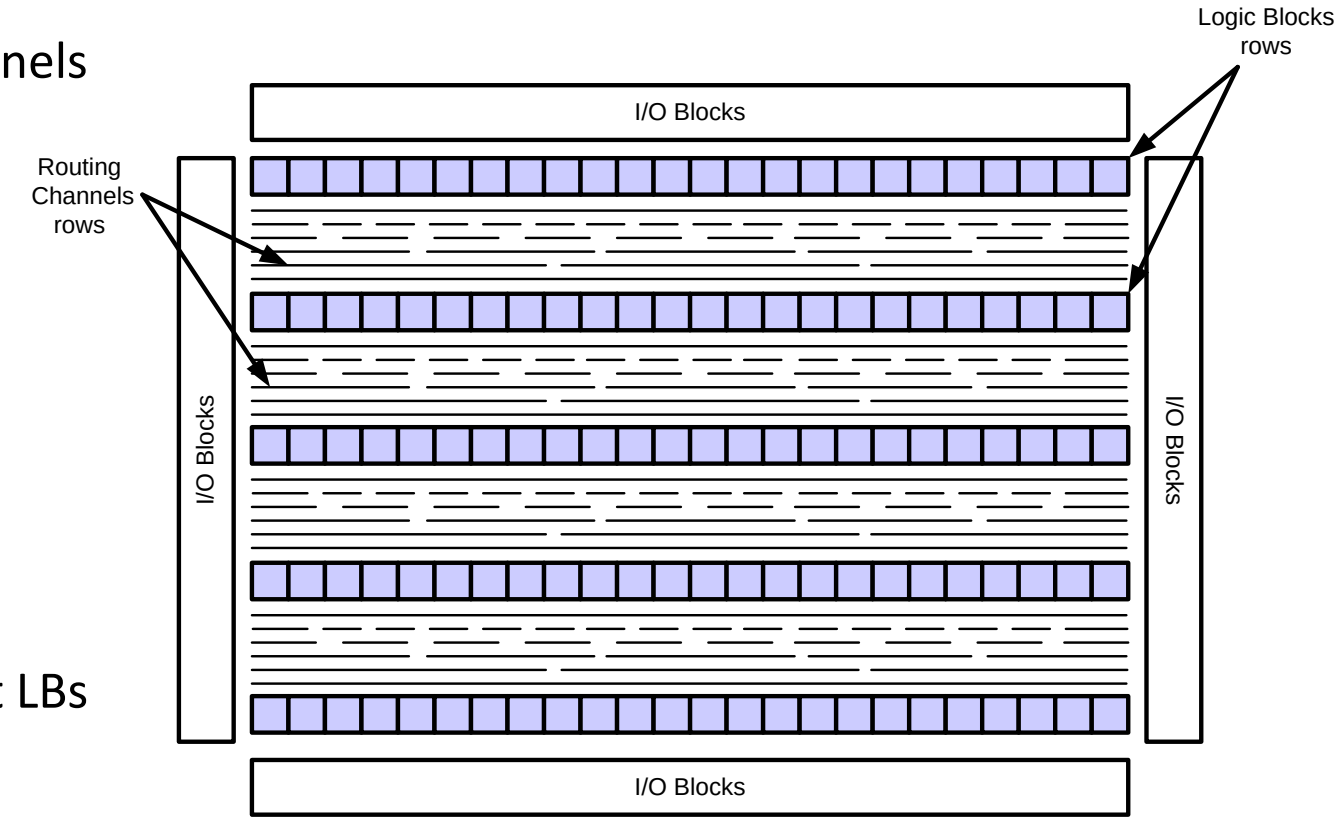


Commercial FPGA Products

- **Actel:**
  - **Antifuse-Based:**
    - Act 1
    - Act 2
    - Act 3
    - SX-A
    - Axcelerator

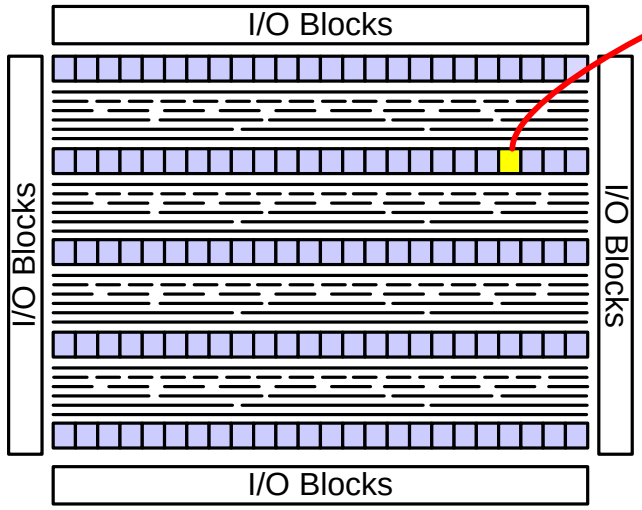
Actel FPGAs (Act 3 Series)

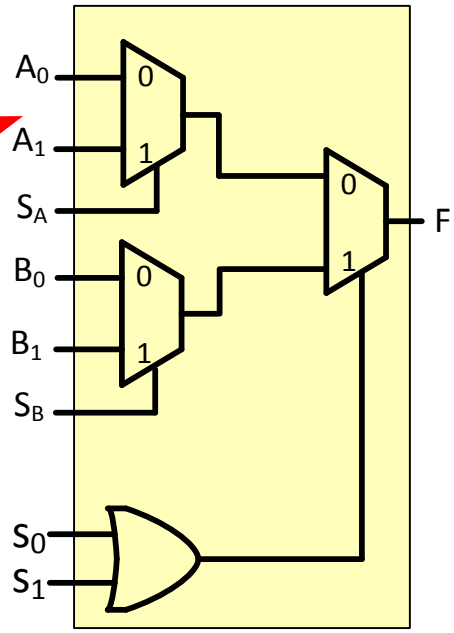
- A structure similar to the traditional gate arrays
- Consists of
  - Horizontal logic blocks
  - Horizontal routing channels
  - I/O blocks
- **MUX-based logic blocks**
  - MUX
  - AND/OR
- **Interconnects:**
  - Only horizontal
  - Segmented wires
  - Use antifuse to connect LBs to routing channels



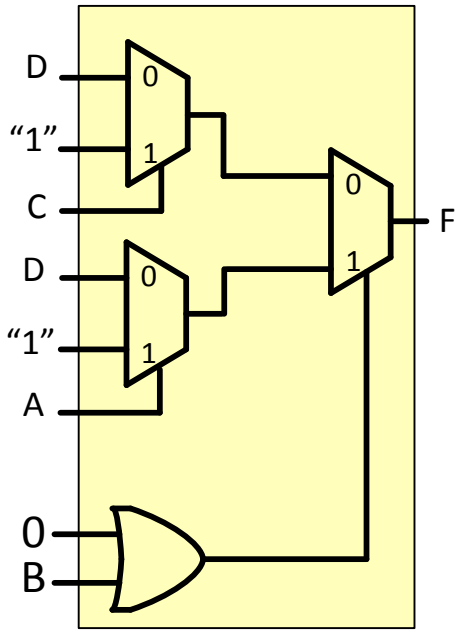
Actel FPGAs (Act 3 Series)

- Detailed architecture of Act 3:





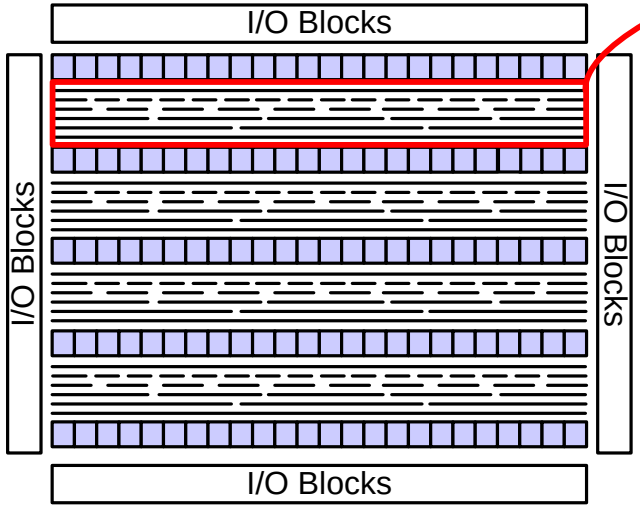
Example

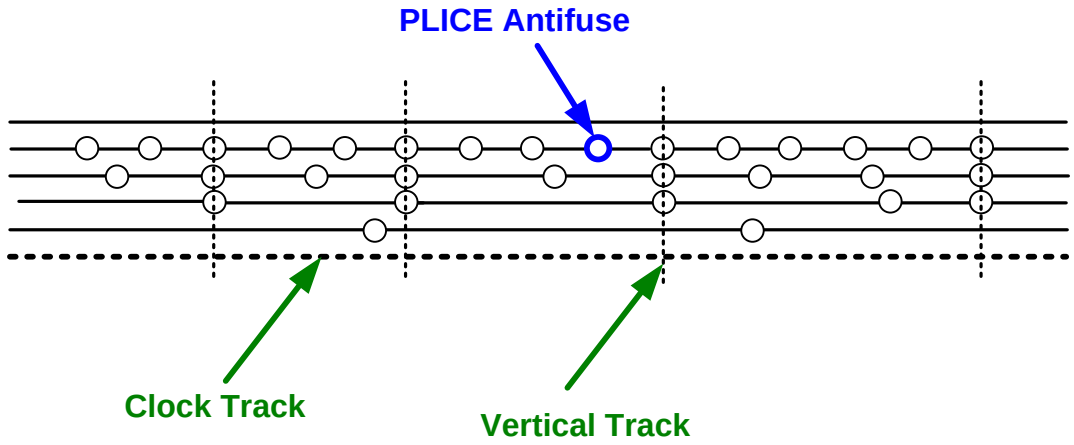


F=(A.B)+(B'.C)+D

Actel FPGAs (Act 3 Series)

- Detailed architecture of Act 3:

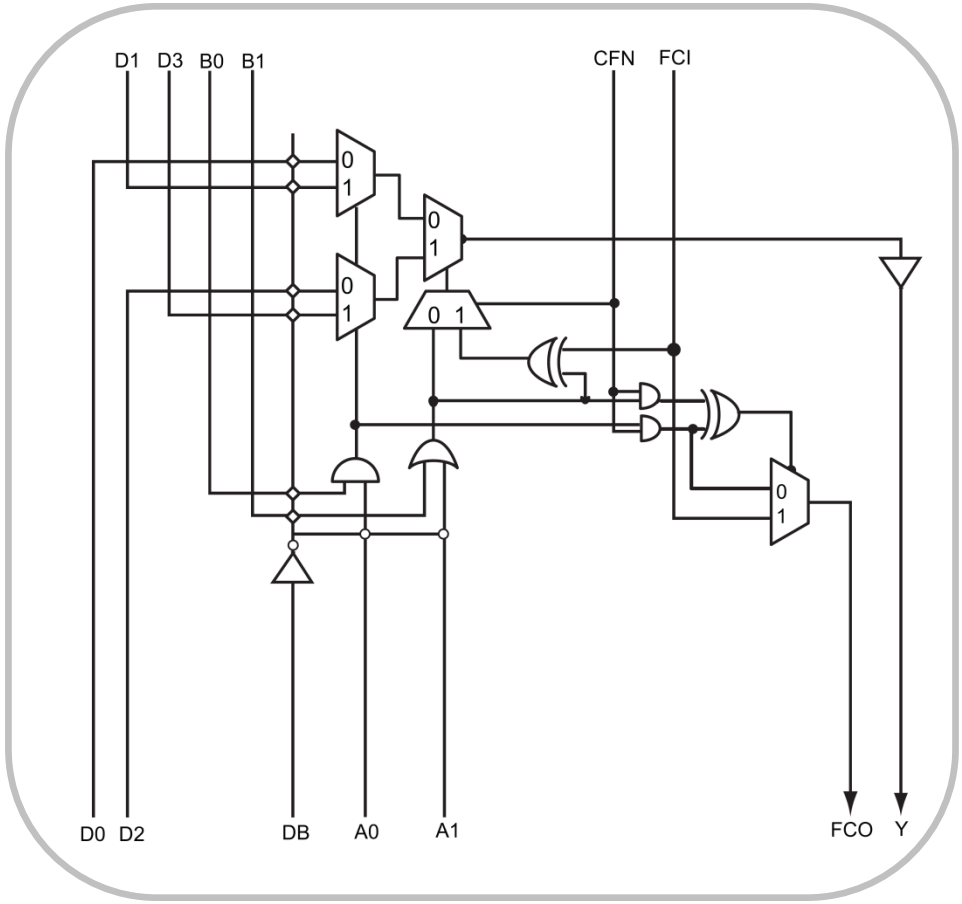




Actel Device	Number of Antifuses
A1010	112,000
A1225	250,000
A1280	750,000

Actel FPGAs (Axcelerator Series)

- Advance recent antifuse-based FPGA with 2 million equivalent gates.
- It comes with
  - Embedded SRAM Blocks
  - Chip-wide highway routing
  - Carry logic
  - PLL
- AX125 Logic Block:



Actel FPGAs (ProASIC 500K Series)

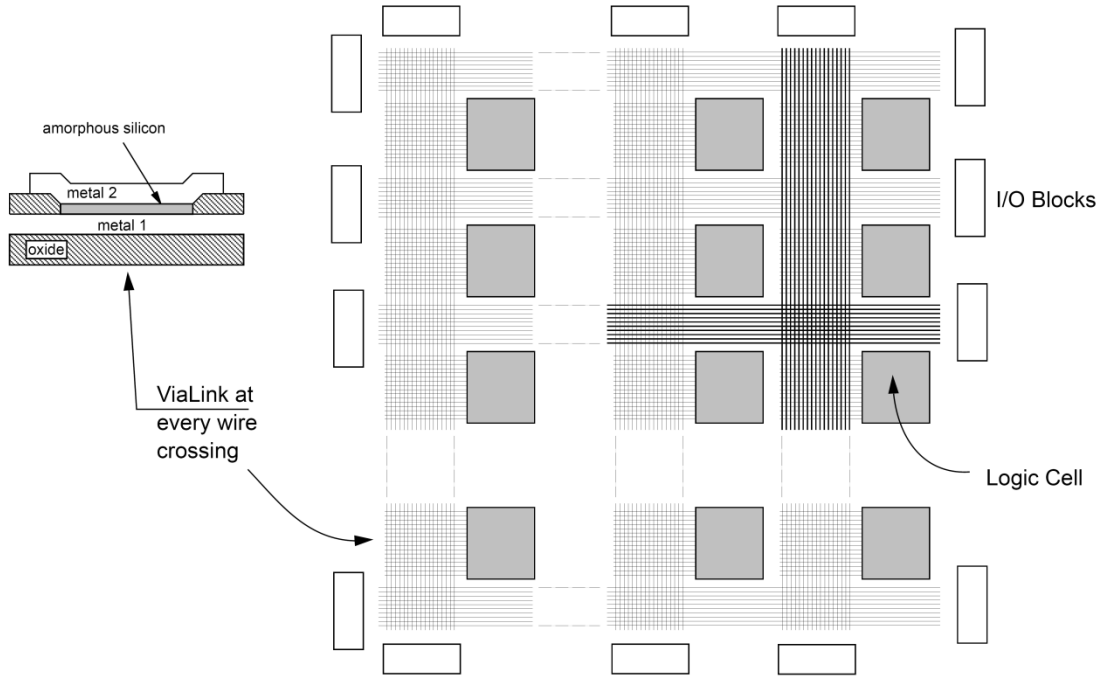
- Flash-based FPGA, using switches and MUXs for programmability to implement logic functions
- The programmed switches are used to select alternate inputs to the core logic
- It can implement any function of 3 inputs except the 3-input XOR
- The feedback paths allow the logic block to be configured as a latch
  - **in2:** clock
  - **in3:** reset

Commercial FPGA Products

- **QuickLogic:**
  - **Antifuse-Based:**
    - pASIC
    - pASIC-2

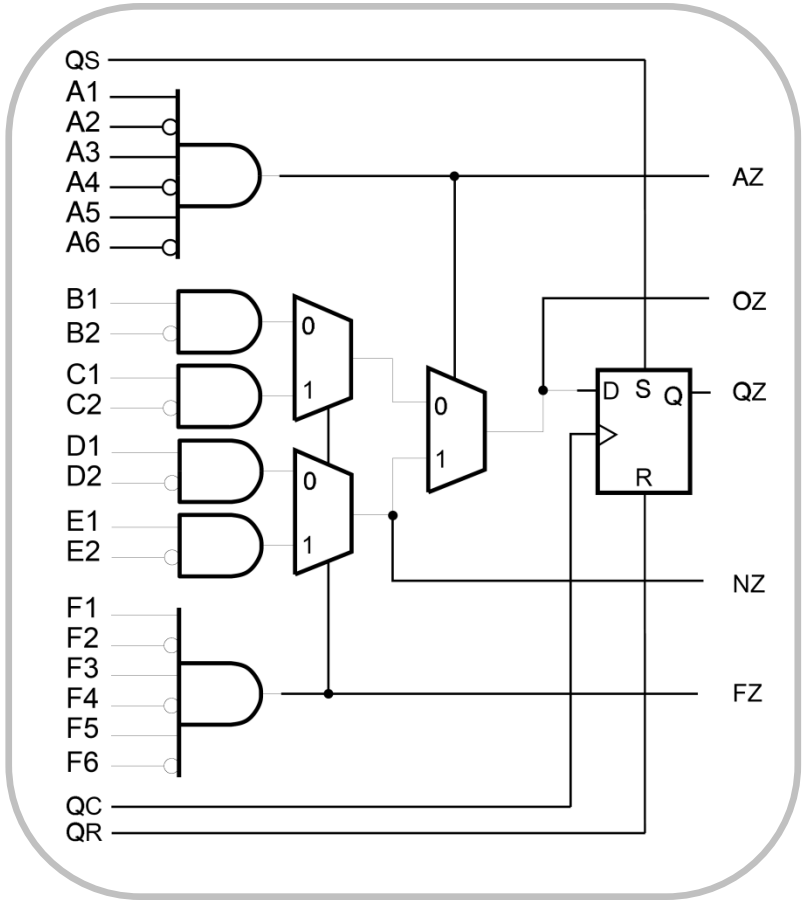
QuickLogic pASIC FPGAs

- The main competitor for Actel antifuse-based FPGAs
- Array based structure like Xilinx FPGAs
- MUX-Based logic blocks
- Interconnect consists of only long lines
  - Present at every crossing of LB pins & interconnect wires
  - Generous connectivity
- Metal-to-Metal antifuse structure
  - Called ViaLink
  - Less resistance than Actel PLICE



QuickLogic pASIC FPGAs

- Inside a pASIC Logic Block:



Commercial FPGA Products : Applications

- **Communication:**
  - Virtex 4/5/6 & Virtex II Pro **(Xilinx)**
  - Stratix II/III/IV & Stratix GX **(Altera)**

- **Consumer Electronics, Automotive &Micro Controllers:**
  - Spartan 3 **(Xilinx)**
  - Cyclone 2 **(Altera)**
  - ProASIC3/E **(Actel)**

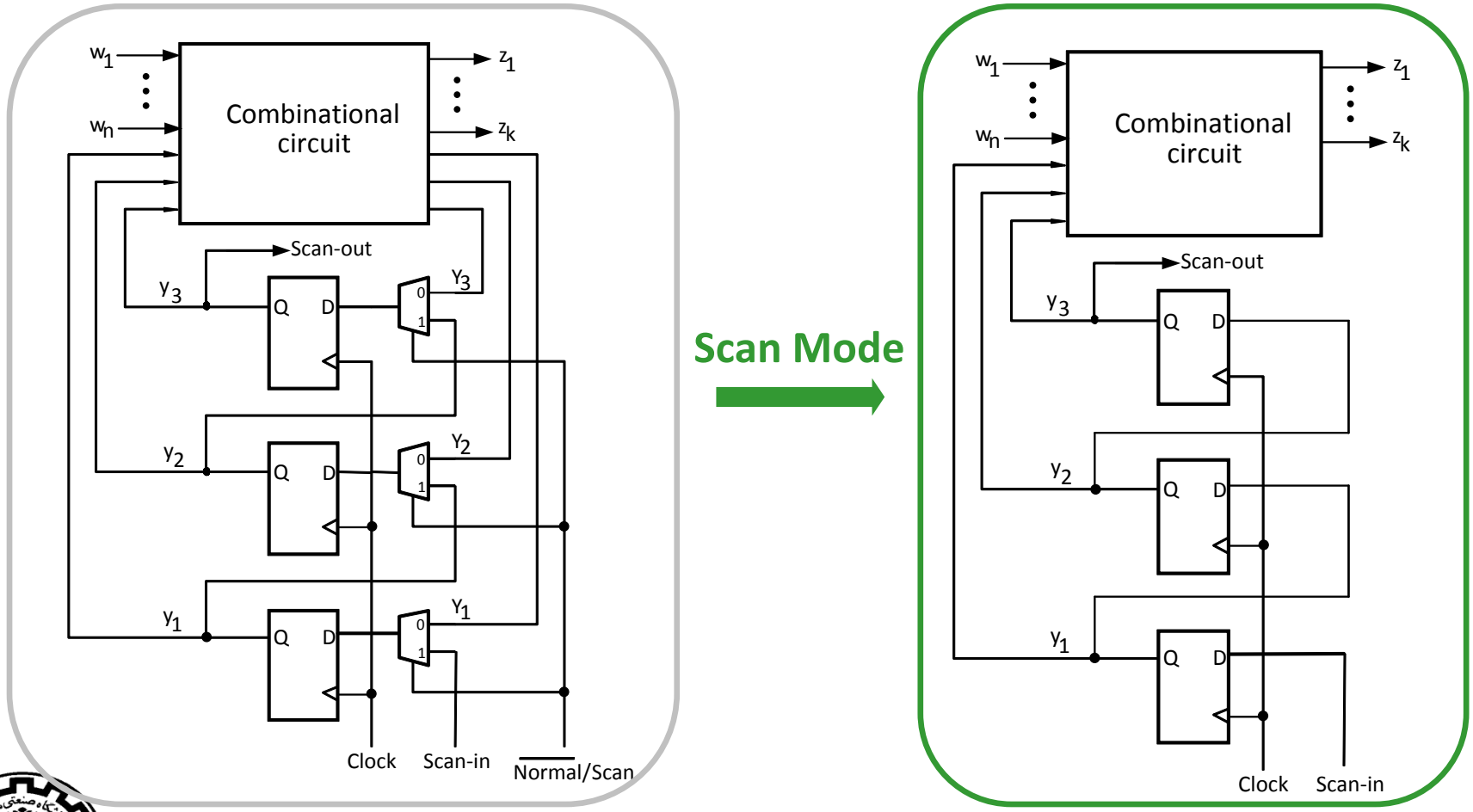
- **Aerospace & Military Applications:**
  - Axcelerator (**Actel**)

FPGA Specifications

- Number of I/O Pads
- Maximum clock frequency
- Number of equivalent gates that can be filled
- Amount of on-chip memory blocks
- Interfaces (such as PCI Express)
- On-chip CPU

FPGA Testing (Scan Chain)

- Many modern FPGAs have some **scan chains** into their testing circuitry
- Test circuitry is used to ensure that the chip/board was properly manufactured



FPGA Testing (JTAG)

- The **JTAG Standard** (Joint Test Action Group) was created to allow chips on boards to be easily tested

- It is also called “boundary scan” b/c it is designed to scan the pins at the boundary b/w the chip and the board

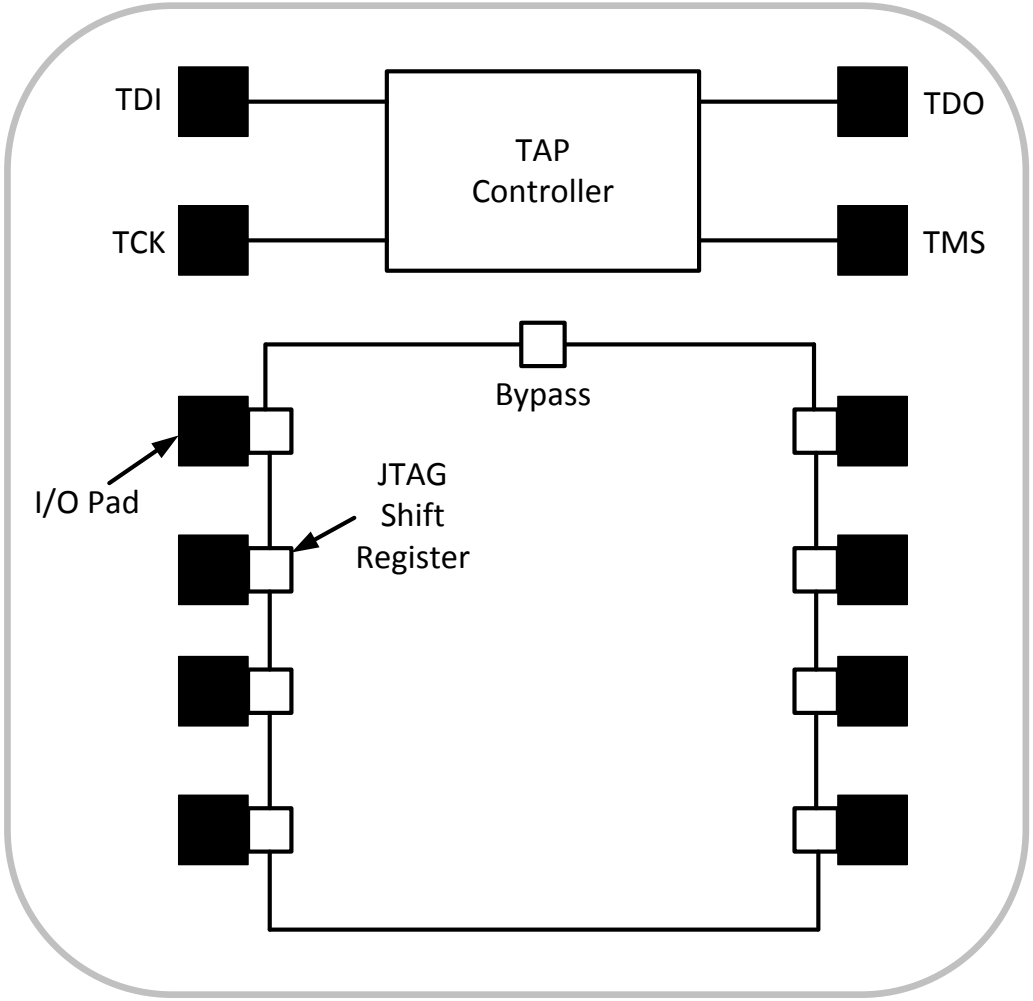
- JTAG is built into the pins of the chip

- During testing they are decoupled from the chip and used as a shift register

- Using the shift register, input values are placed on the chip’s pins and output values are read from the pins (controlled by test access port (TAP) block)

FPGA Testing

- JTAG has four pins:
  - TDI : Shift Register Input
  - TDO : Shift Register Output
  - TCK : Test Clock
  - TMS : Test Mode Select

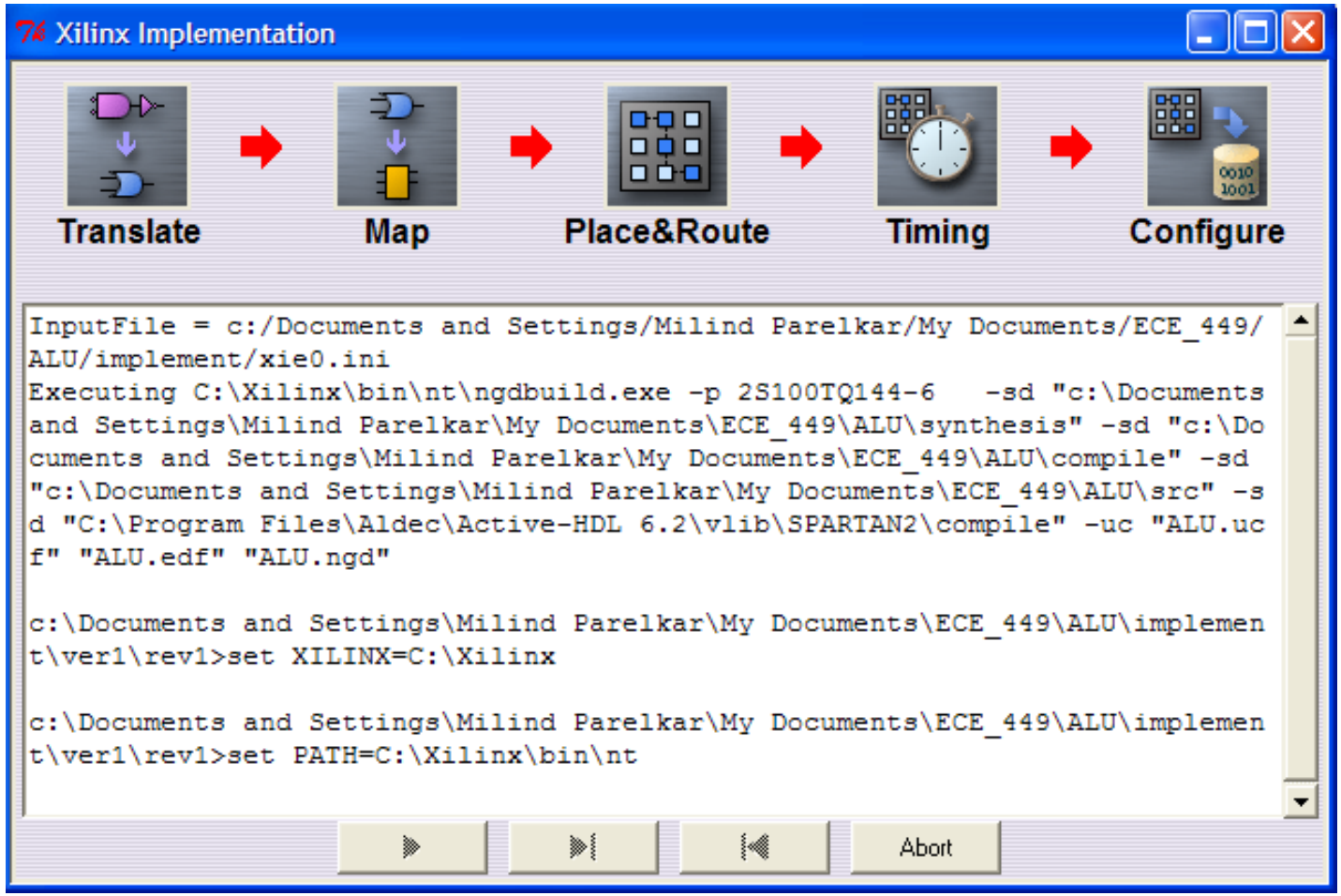


Two Important Factors in Testing:

- Controllability
- Observability

FPGA Design Flow

FPGA Design Flow

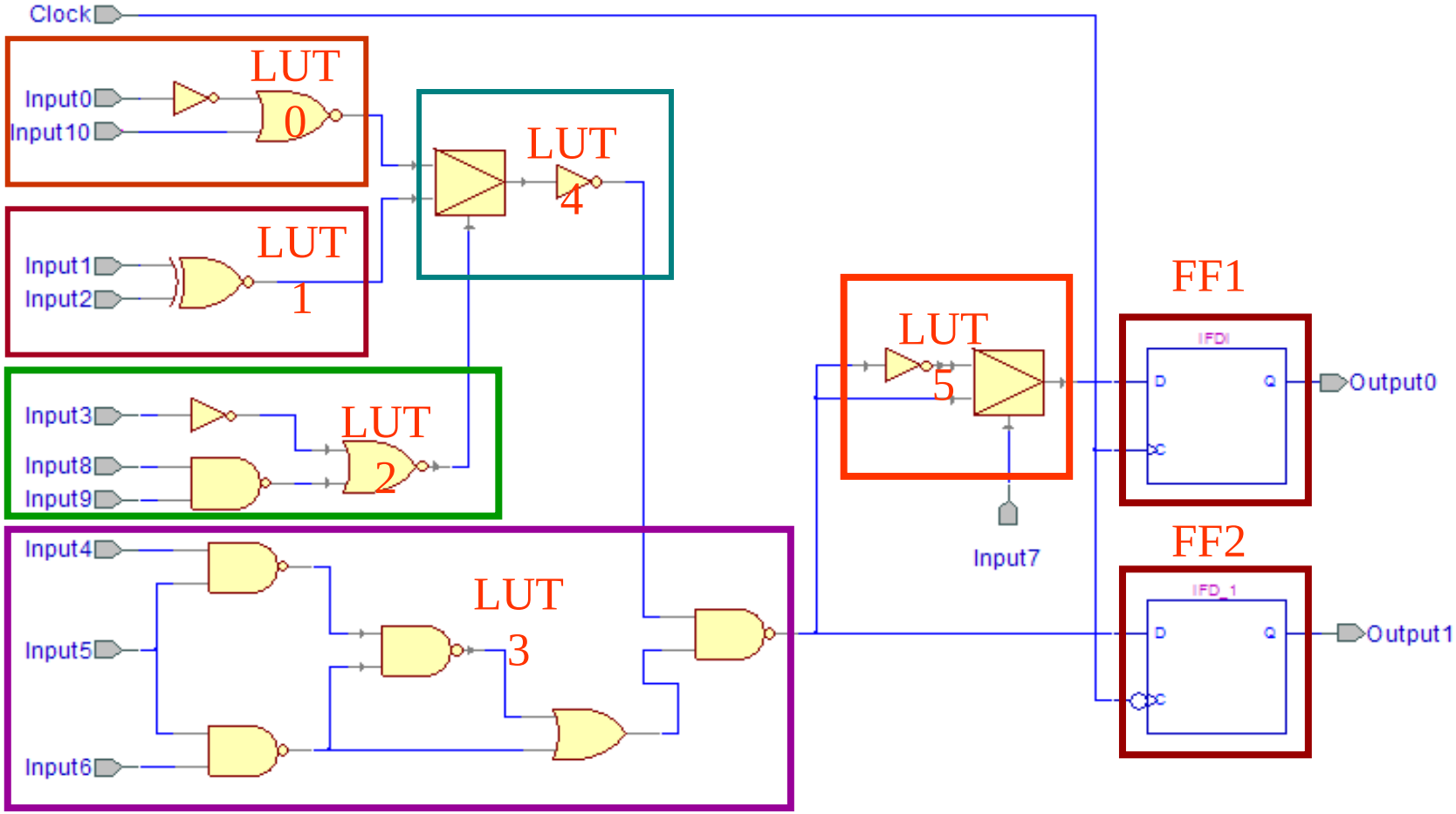


FPGA Design Flow: Mapping

- **Technology Mapping: Schematic/HDL to Physical Logic units**
- **Compile functions into basic LUT-based groups (function of target architecture)**

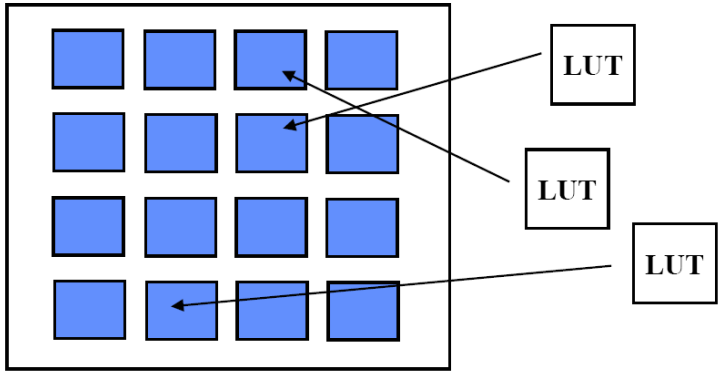
```
always @(posedge Clock or negedge Reset)
 begin
  if (! Reset)
    q <= 0;
  else
    q <= (a & b & c) | (b & d);
 end
```

FPGA Design Flow: Mapping

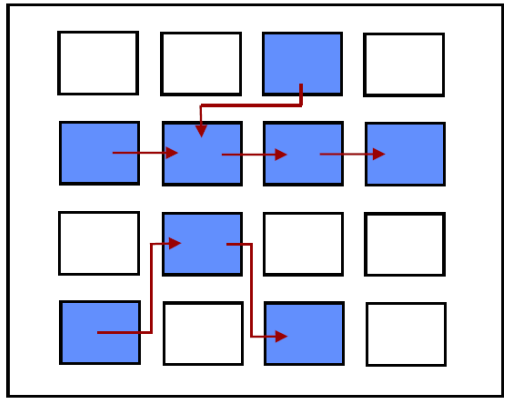


FPGA Design Flow: Placement & Routing

- **Placement** – assign logic location on a particular device



- **Routing** – iterative process to connect CLB inputs/outputs and IOBs. Optimizes critical path delay – can take hours or days for large, dense designs



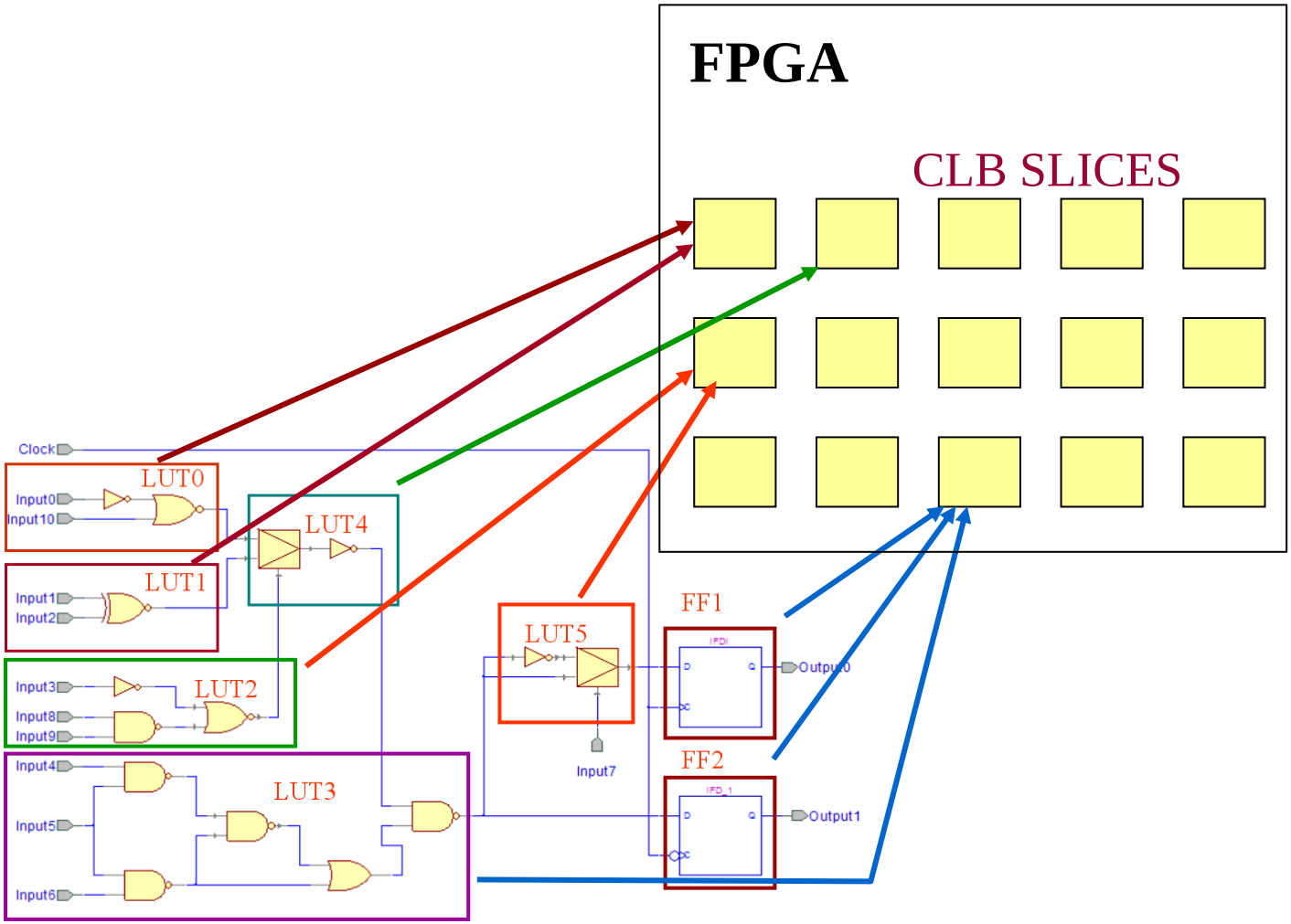
Iterate placement if timing not met

Satisfy timing? → Generate Bitstream to config device

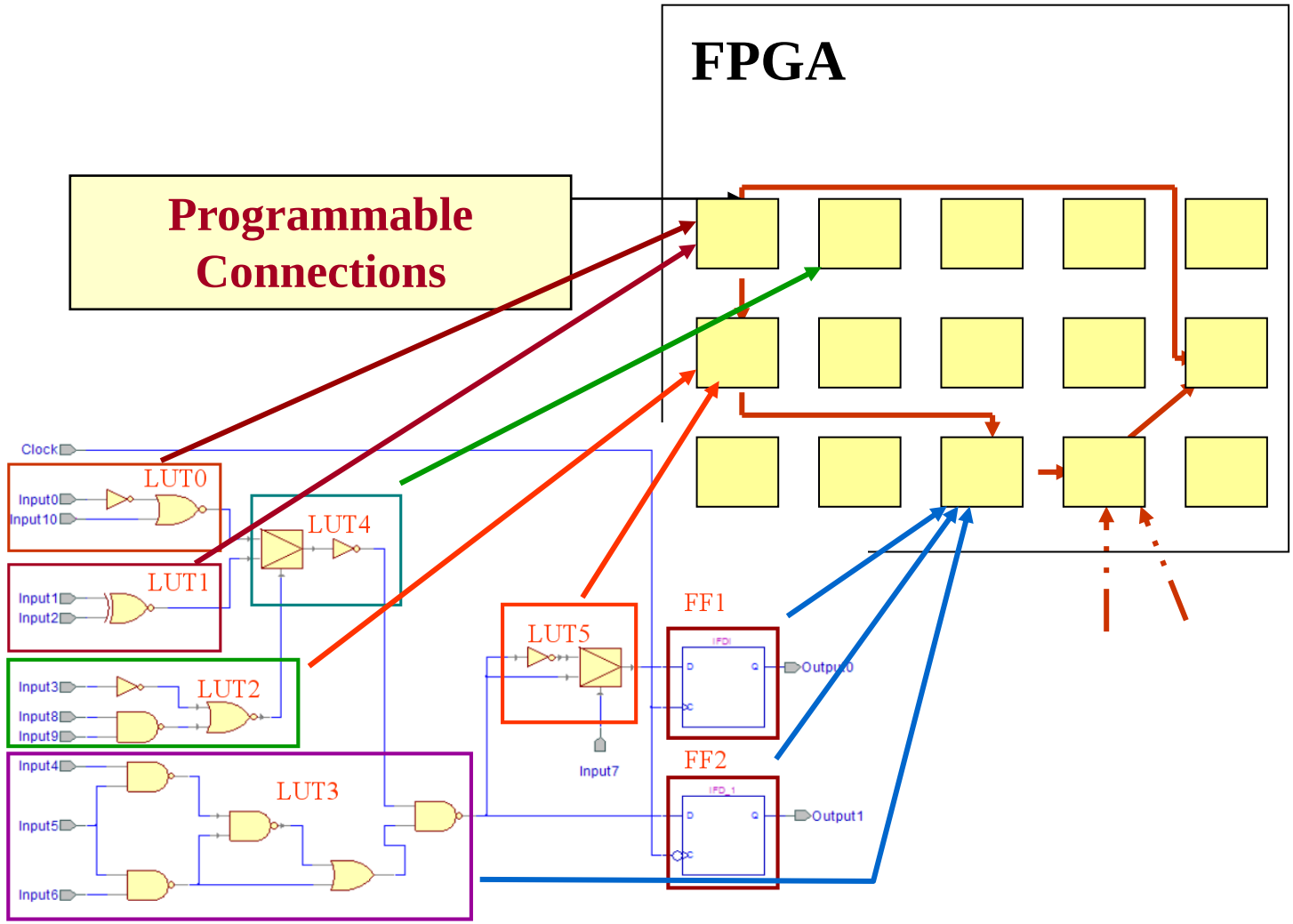
Challenge! Cannot use full chip for reasonable speeds (wires are not ideal).

Typically no more than 50% utilization.

FPGA Design Flow: Placement

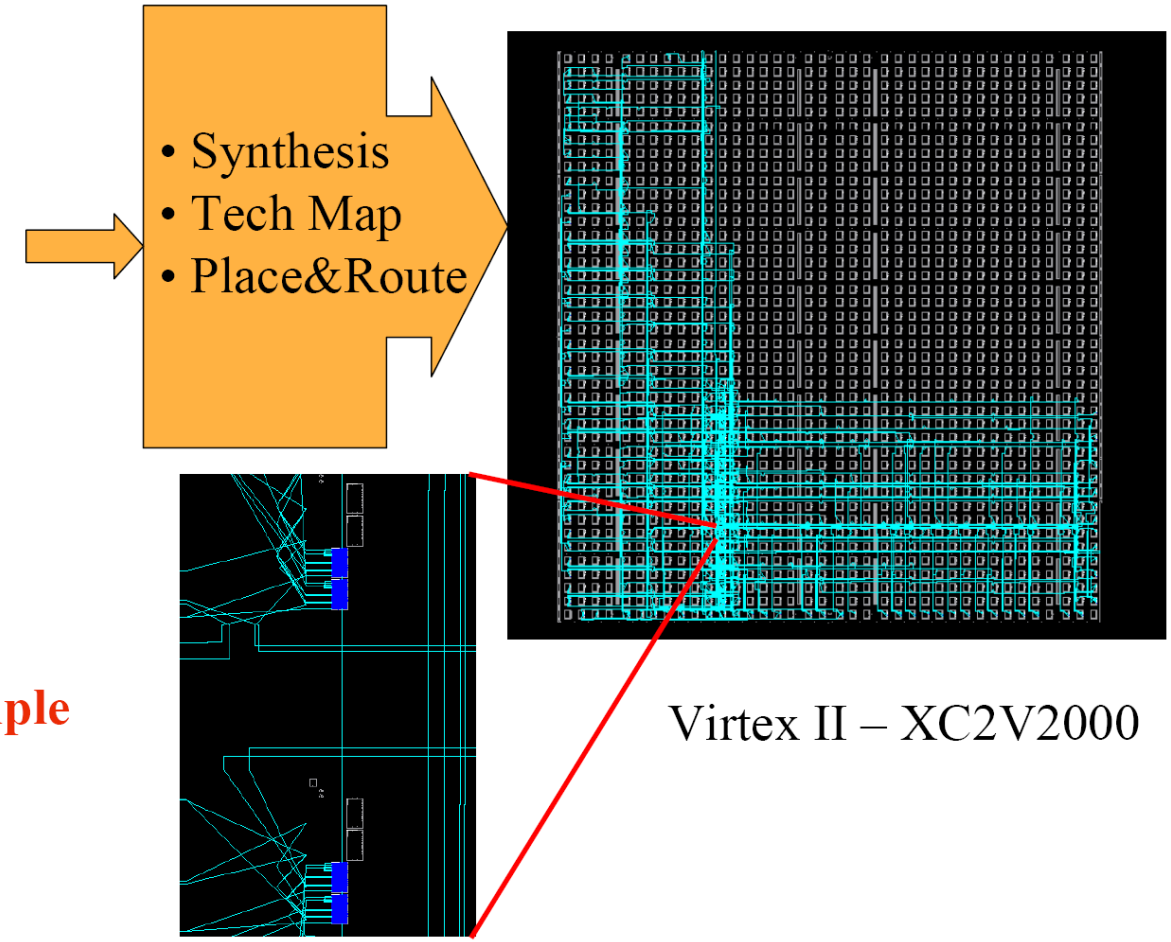


FPGA Design Flow: Routing



FPGA Design Flow: In a Glance

```
module adder64 (a, b, sum);
  input  [63:0] a, b;
  output [63:0] sum;

  assign sum = a + b;

endmodule
```

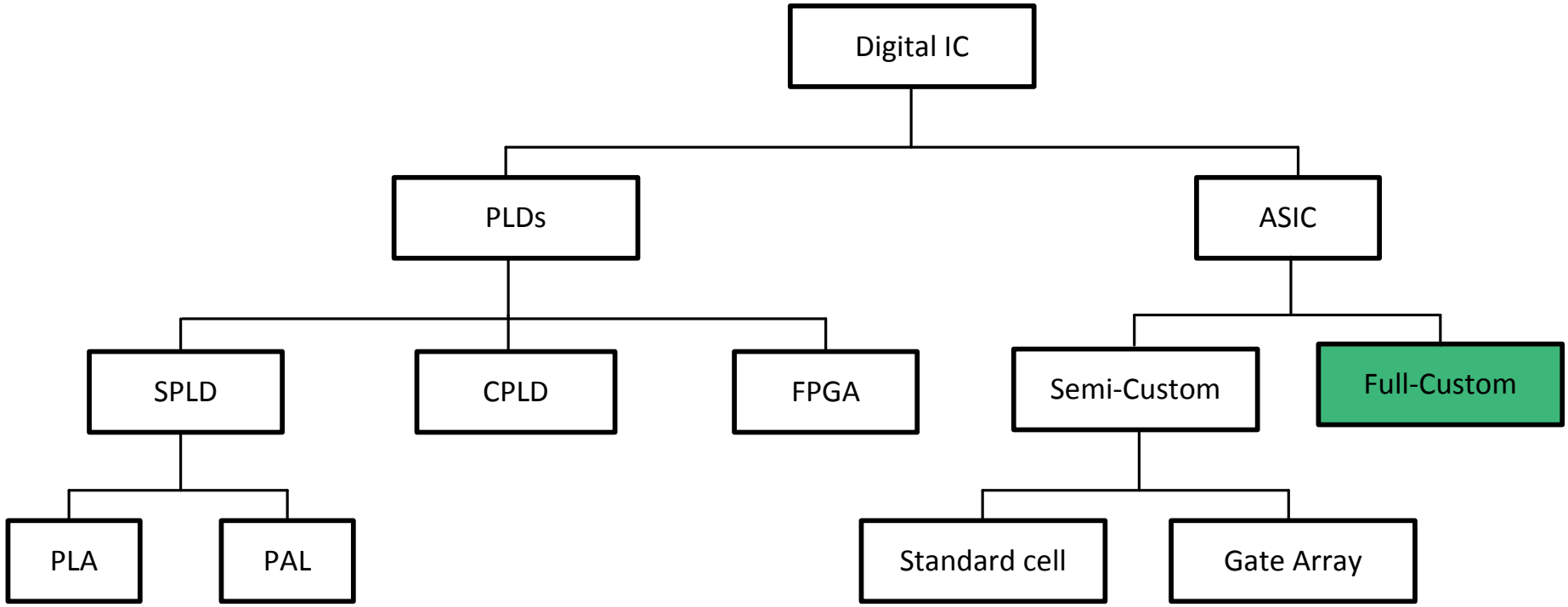


64-bit Adder Example

Outline

- Introduction
- Simple Programmable Logic Designs (SPLDs)
  - PLA
  - PAL
- Complex Programmable Logic Designs (CPLDs)
- Field-Programmable Gate Array (FPGAs)
  - Logic Blocks
  - Programmable Routing Switches
  - I/O Pads
- Commercial FPGA Products
- **Application Specific Integrated Circuits (ASICs)**

Full Custom VLSI Technology

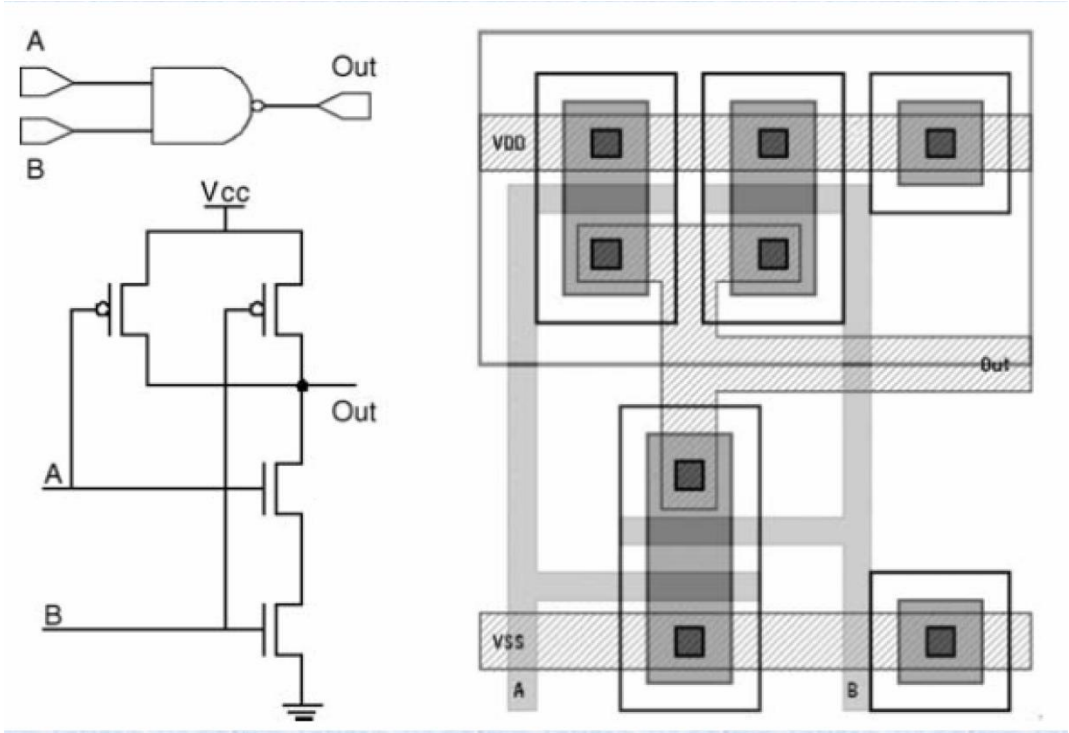


Full Custom VLSI Technology

- All layers are optimized/customized for the particular implementation:
  - Placing transistors
  - Sizing transistors
  - Routing wires

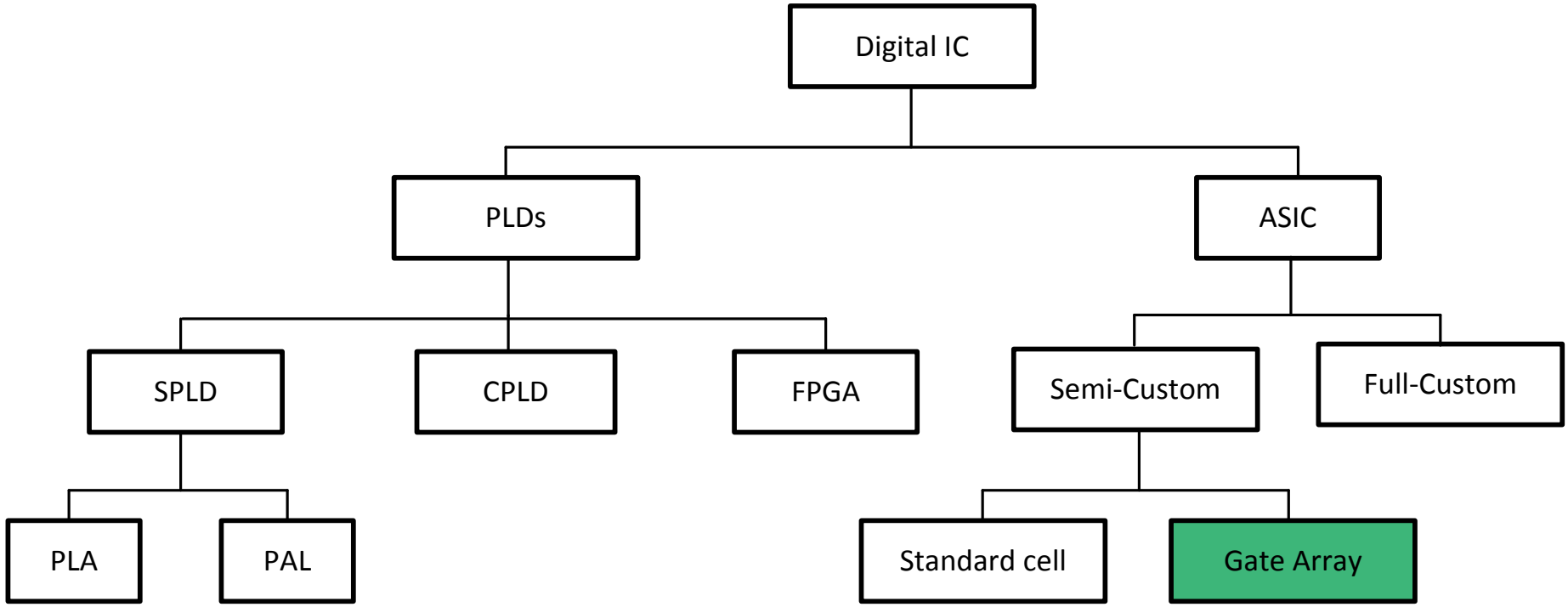
- **Benefits:**
  - Excellent performance
  - Small size
  - Low power

- **Drawbacks:**
  - High NRE cost
  - Long time-to-market



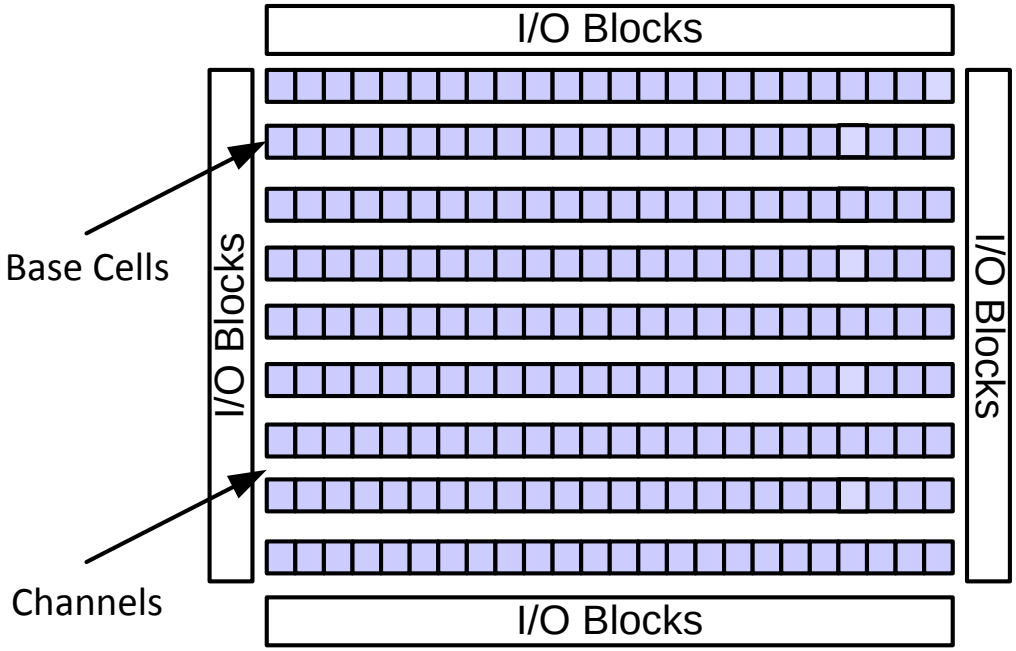
Not too common today!

Semi-Custom VLSI Technology (Gate Array)

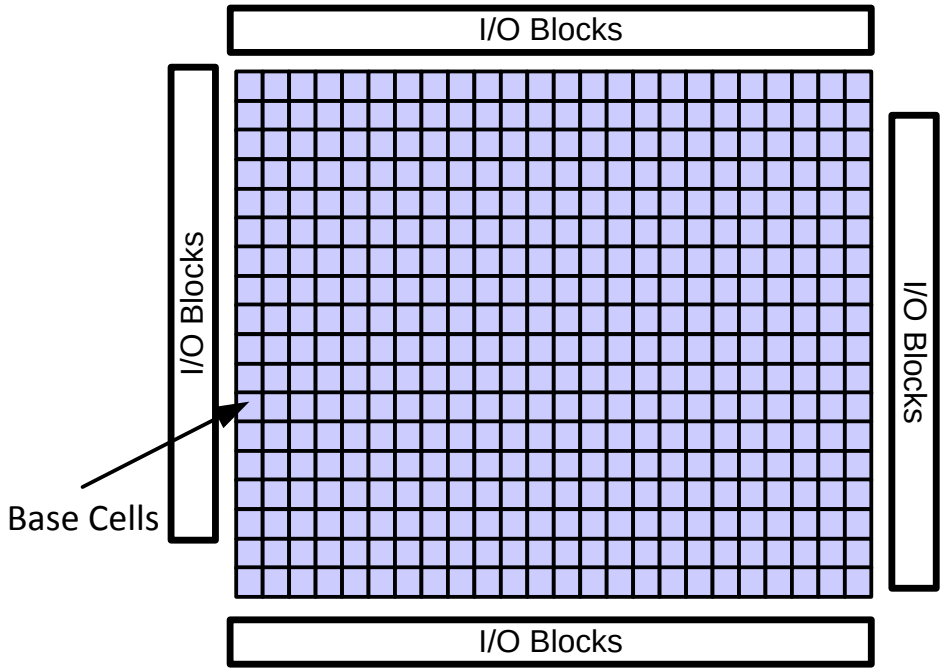


Semi-Custom VLSI Technology (Gate Array)

- Gate arrays (GAs) composed of arrays of p- and n-type transistors.
- The mapping, from transistors to gates, performed through CAD tools.

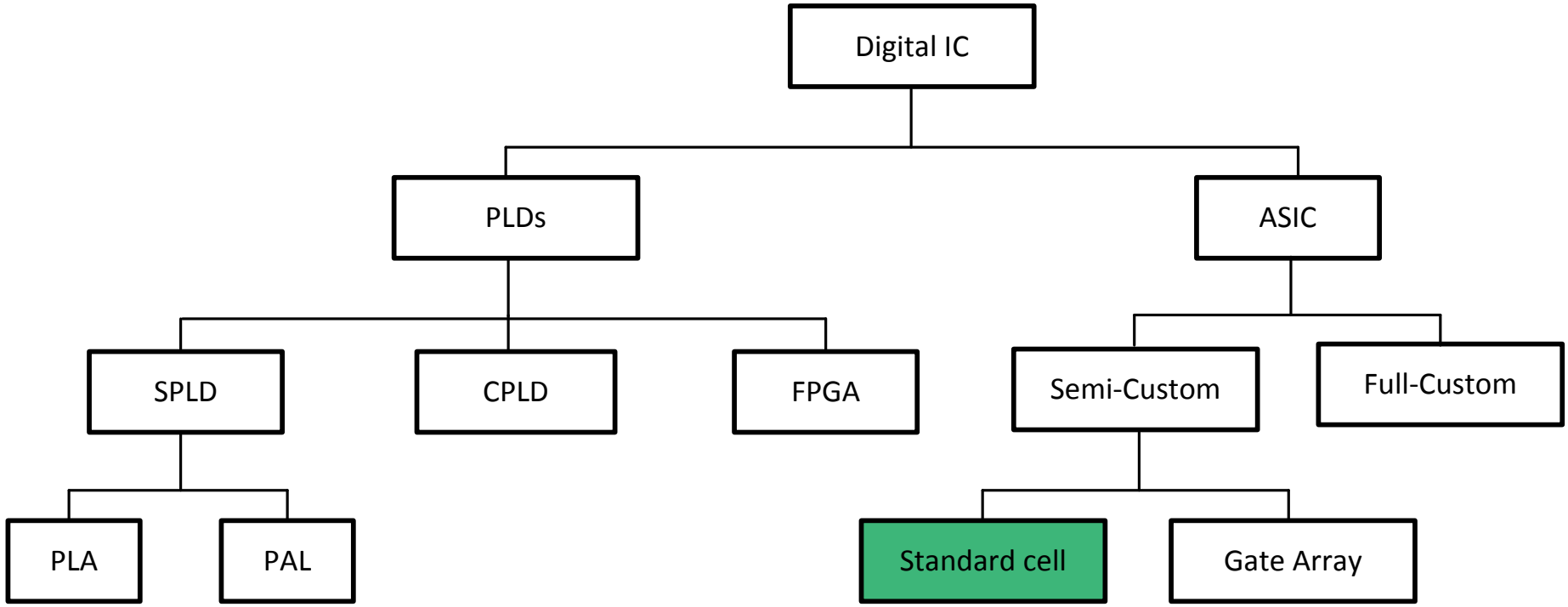


Channeled Gate Array



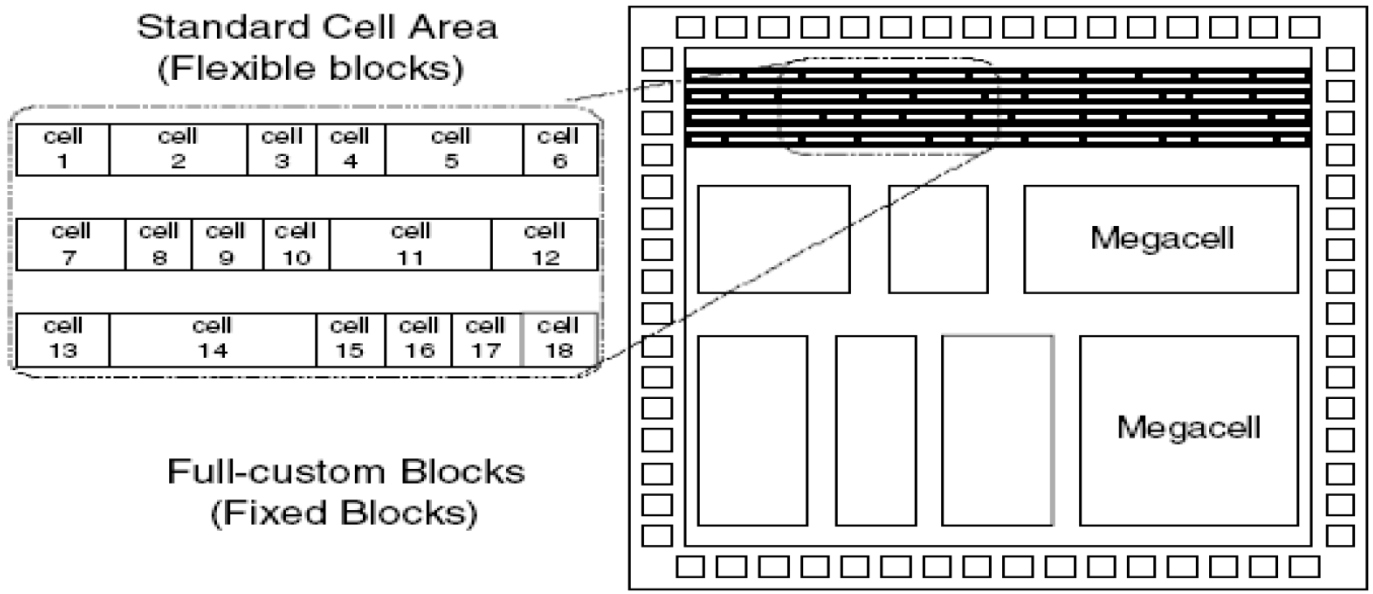
Channel-less Gate Array

Semi-Custom VLSI Technology (Standard Cell)



Standard Cell-Based ASICs

- Common logic components (e.g., gates, multiplexers, adders, …) previously designed and stored in a library for different area, speed, power requirements
- Logic components get converted to chip layouts.
- Standard-cell designs are organized, as rows of constant height cells.



FPGA vs. ASIC

❑ FPGA Advantages:

- ➢ Fast programming and testing time by the end user (instant turn-around)
- ➢ Excellent for prototyping
  - ▪ Easy to migrate from prototype to the final design
- ➢ Can be re-used for other designs
- ➢ Cheaper (in small volumes) ⟹ lower start-up costs
- ➢ Re-programmable
- ➢ Lower financial risk
- ➢ Ease of design changes/modifications
- ➢ Cheaper design tools

FPGA vs. ASIC

❑ FPGA Drawbacks:

- ➢ Slower than ASIC (2-3 times slower)
- ➢ Power hungry (up to 10 times more dynamic power)
- ➢ Use more transistors per logic function
- ➢ More area (20 to 35 times more area than a standard cell ASIC)

FPGA vs. ASIC

❑ ASIC Advantages:

- ➢ Faster
- ➢ Lower power
- ➢ Cheaper (if manufactured in large volumes)
- ➢ Use less transistors per logic function

❑ ASIC Drawbacks:

- ➢ Implements a particular design (not programmable)
- ➢ Takes several months to fabricate (long turn-around)
- ➢ More expensive design tools
- ➢ Very expensive engineering/mask cost for the first successful design

Implementation Approaches (ASIC vs. FPGA)

ASIC

Application **S**pecific
Integrated **C**ircuit

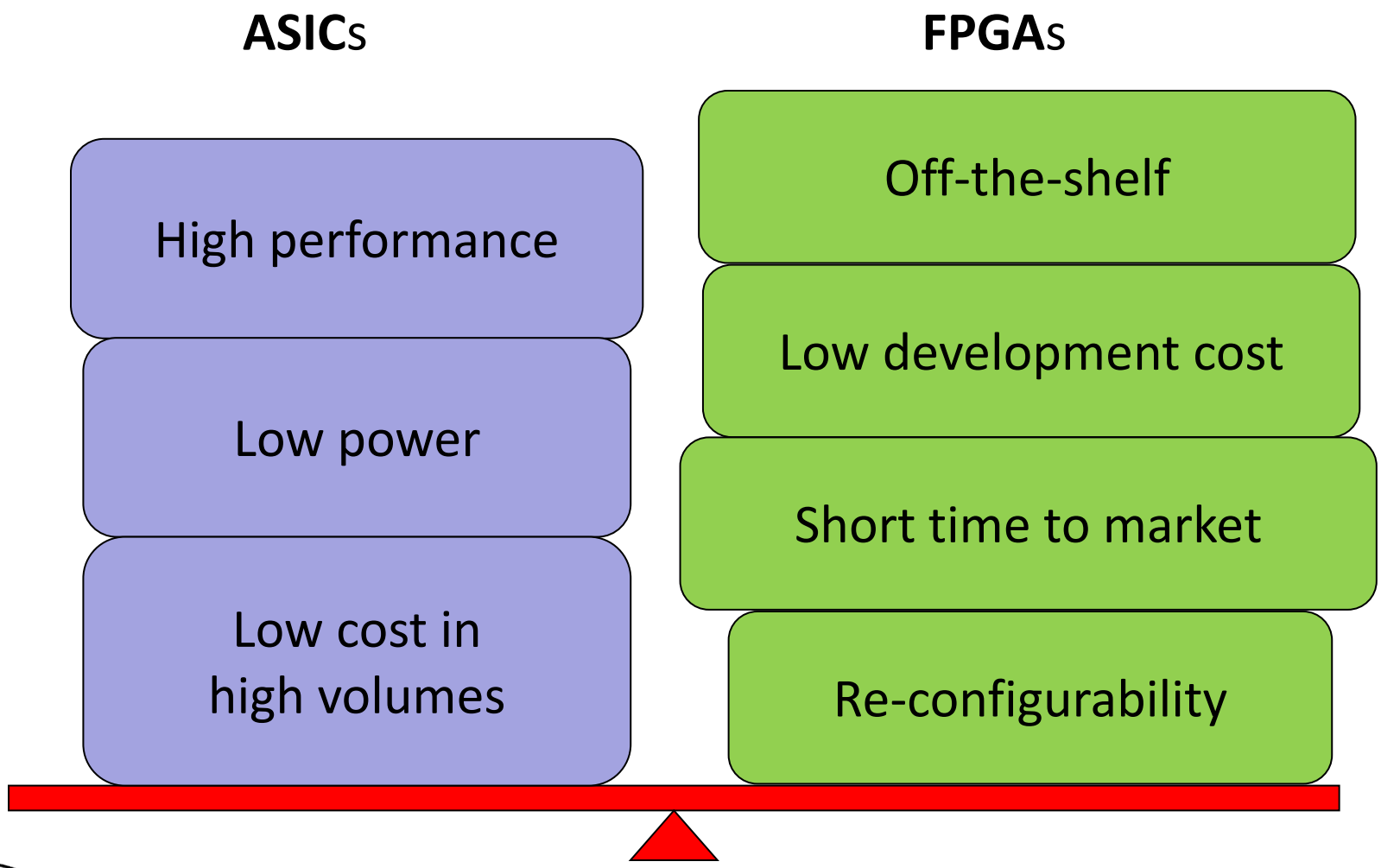
- ❑ Expensive & time consuming fabrication in semiconductor foundry
- ❑ Designed all the way from behavioral description to physical layout

FPGA

Field **P**rogrammable
Gate **A**rray

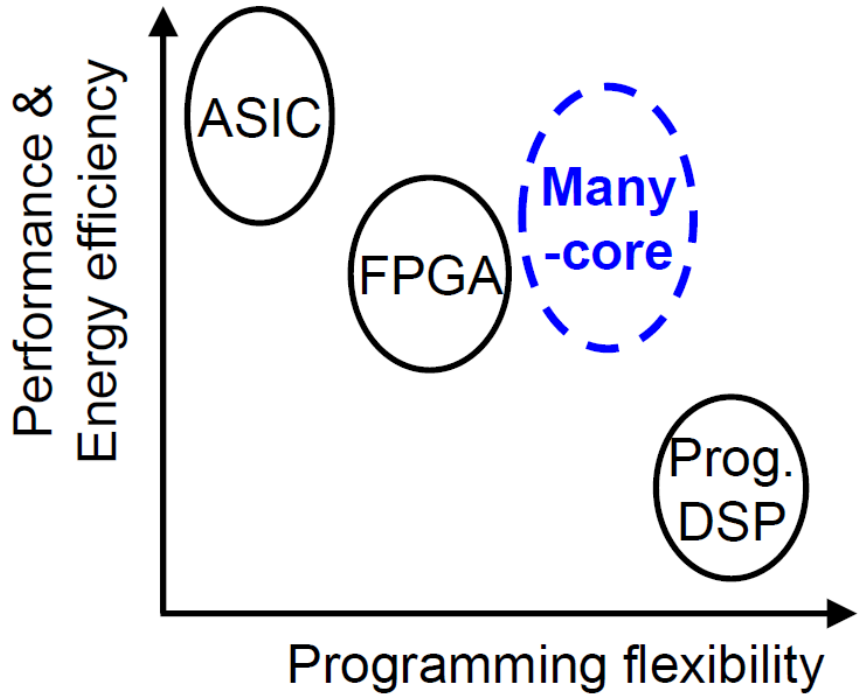
- ❑ Bought off the shelf & reconfigured by the end designers
- ❑ No physical layout design
- ❑ Design ends with a bitstream used to configure a device

Implementation Approaches (ASIC vs. FPGA)



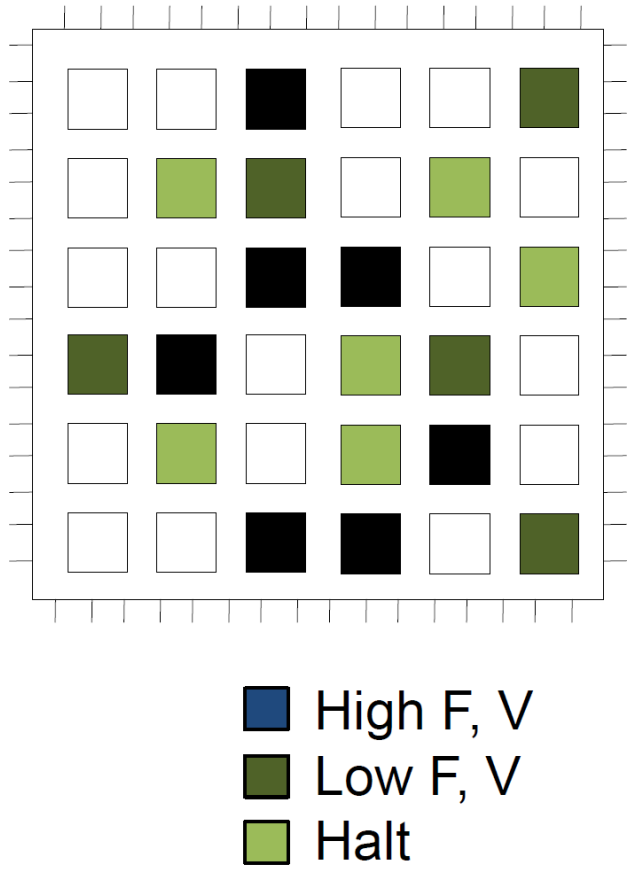
Current Trend

- Programming flexibility
- High performance
  - Throughput
  - Latency
- High energy efficiency
- Suitable for future fabrication technologies



Target Many-core Architecture

- High performance
  - Exploit task-level parallelism in digital signal processing and multimedia
  - Large number of processors per chip to support multiple applications
- High energy efficiency
  - Voltage and frequency scaling capability per processor



ASIC & FPGA Chip Design:

Synthesis

Mahdi Shabany

Department of Electrical Engineering

Sharif University of technology

Outline

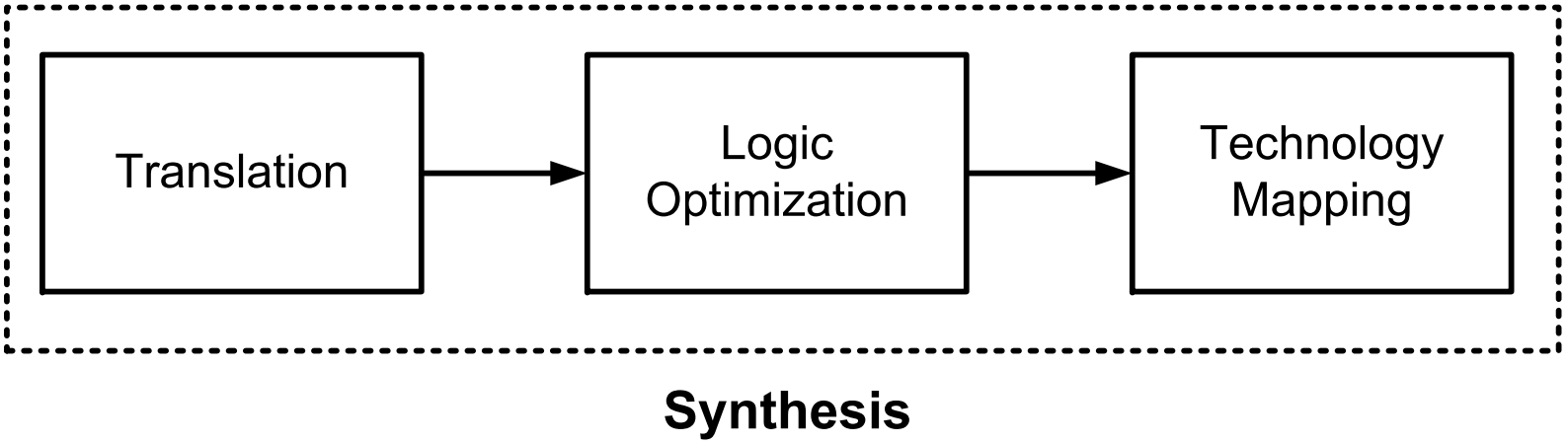
- **Introduction to Synthesis**
- **Digital Logic Basics**
- **Logic Optimization**
  - Two-level logic synthesis
  - Multi-level logic synthesis
- **Technology Mapping**
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Outline

- **Introduction to Synthesis**
- **Digital Logic Basics**
- **Logic Optimization**
  - Two-level logic synthesis
  - Multi-level logic synthesis
- **Technology Mapping**
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Synthesis

- **Synthesis** = Translation + Logic Optimization + Technology Mapping
  - **Translation:** going from RTL to Boolean function
  - **Logic Optimization :** Optimizing and minimizing Boolean function
  - **Technology Mapping (TM):** Map the Boolean function to the target library



Synthesis

❑ **Synthesis** = Translation + Logic Optimization + Technology Mapping

```
always @ (a, b)
        case ({a,b})
            2'b00: out = 1;
            2'b01: out = 1;
            2'b11: out = 1;
            default: out = 0;
        endcase
```

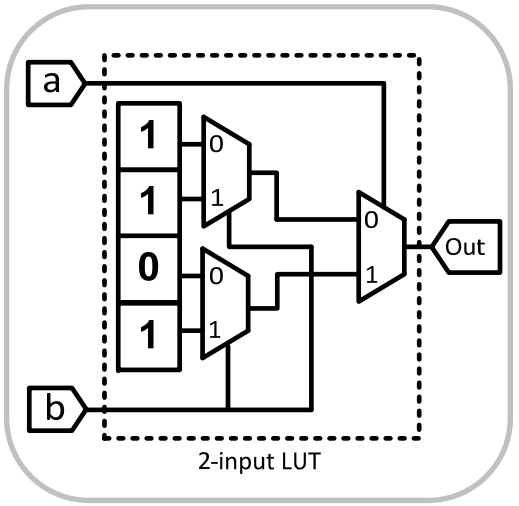
Translation

$$out = \bar{a}b + ab + \bar{a}\,\bar{b}$$

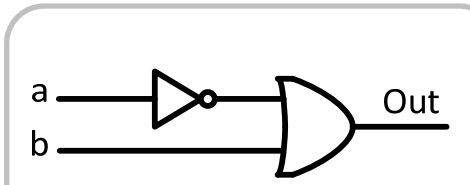
Logic Optimization

$$out = \bar{a} + b$$

Technology Mapping



FPGA



ASIC

Outline

- Introduction to Synthesis
- **Digital Logic Basics**
- Logic Optimization
  - Two-level logic synthesis
  - Multi-level logic synthesis
- Technology Mapping
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Digital Logic Basics : Boolean Function

$f(x) : B^n \longrightarrow B$

$B = \{0, 1\}$, $x = (x_1, x_2, \ldots, x_n)$

- $x_1, x_2, \ldots$ are variables
- $x_1, \overline{x_1}, x_2, \overline{x_2}, \ldots$ are literals
- each vertex of B^n is mapped to 0 or 1
- the onset of *f* is a set of input values for which f(x) = 1
- the offset of *f* is a set of input values for which f(x) = 0

Digital Logic Basics

- A Boolean function can be represented by:
  - **A truth table**

x_1	x_2	f
0	0	0
0	1	1
1	0	1
1	1	0

  - **A logic expression**
    - We use logic variables and operators (AND, OR, NOT, XOR, XNOR, NAND, NOR) to express a logical relation b/w input variables and the output function

$$f = x_1 \oplus x_2$$

Digital Logic Basics : SOP

- Sum of Products (SOP):
  - A logic function that is represented as an OR of product (AND) terms :

$$f(x_1, x_2, x_3) = x_1 + x_2x_3 + \bar{x}_1x_2x_3$$

cube: x_2x_3; minterm: $\bar{x}_1x_2x_3$; Literal: x_3

- A literal is a function input (e.g., $\bar{x}_1, x_2$).
- A product term is formed using an AND operation and literals in either true or complemented form.
- A cube is the AND of set of literals
- A minterm is a cube that contains all literals of a logic function
  - A function with K variables has 2^K possible K-literal minterms
- A cover of "**f**" is a set of cubes that represent the logic function "**f**"
  - (e.g., $C = \{x_1, x_2x_3, \bar{x}_1x_2x_3\}$)

Digital Logic Basics : POS

- Product of Sums (POS):
  - A logic function that is represented as an AND of sum (OR) terms :

$$f = (x_1 + x_2)(x_3 + x_2)$$

  - A sum term is formed using an OR operation and literals in either true or complemented form.
  - A maxterm is a sum term that contains all literals of a logic function

- POS can be derived from SOP by DeMorgan's Theorem and double complementation

$$f = x_1 + x_2 x_3$$

$$\overline{f} = \overline{x_1 + x_2 x_3} = \overline{(x_1)} \bullet \overline{(x_2 x_3)} = \overline{x}_1 \bullet (\overline{x}_2 + \overline{x}_3) = \overline{x}_1 \overline{x}_2 + \overline{x}_1 \overline{x}_3$$

$$f = \overline{\overline{f}} = \overline{\overline{x}_1 \overline{x}_2 + \overline{x}_1 \overline{x}_3} = \overline{(\overline{x}_1 \overline{x}_2)} \bullet \overline{(\overline{x}_1 \overline{x}_3)}$$

$$f = (x_1 + x_2)(x_1 + x_3)$$

Digital Logic Basics : Karnaugh Map

❑ Karnaugh Map:

- ➢ Variables assigned to rows and columns.
- ➢ Adjacent valuations differ by 1 position.
- ➢ Form product terms by creating groups with 2^k 1s that are adjacent to one another

cd \ ab	00	01	11	10
00	0	0	0	1
01	0	1	1	1
11	0	1	1	0
10	1	1	0	1

$$f = bd + a\bar{b}\bar{c} + \bar{a}c\bar{d} + \bar{b}c\bar{d}$$

Digital Logic Basics : Implementation Cost

❑ Cost of an Implementation:

Cost of implementation = # of inputs + # of gates (except NOTs)

❖ Example:

$$f = \underbrace{bd}_{3} + \underbrace{a\bar{b}\bar{c}}_{4} + \underbrace{\bar{a}c\bar{d}}_{4} + \underbrace{\bar{b}c\bar{d}}_{4}$$

5

➢ Cost = 3 + 4 + 4 + 4 + 5 = 20

Digital Logic Basics : Canonical Form

❑ Canonical Form:

- ➢ A form of Boolean logic function representation is said to be canonical if and only if for each logic function there exists a unique representation in the given form
- ➢ Function is described using its equivalent minterms

❖ Example:

- ➢ **Canonical** $f(a,b,c) = a\bar{b}c + abc$
- ➢ **Non-canonical** $f(a,b,c) = a\bar{b}c + abc + bc$
  - ▪ Because it has a cube that does not include all the literals

❑ Any Boolean logic can be represented in a canonical form

Outline

- Introduction to Synthesis
- Digital Logic Basics
- **Logic Optimization**
  - Two-level logic synthesis
  - Multi-level logic synthesis
- Technology Mapping
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Two-Level Logic Synthesis : Cubical Notation

❑ Cubical Notation:

- ➢ A different way to represent a product term
  - "0" : to represent inverted variable
  - "1" : to represent a variable in true form
  - "X" : to represent a variable not used in the product term

abcd	p-term
X1X1	⟶
100X	⟶
0X10	⟶
X010	⟶

f = {X1X1, 100X, 0X10, X010}

(Easier to implement in a computer program)

Two-Level Logic Synthesis : Cubical Notation

❑ Cubical Notation:

- ➢ A different way to represent a product term
  - "0" : to represent inverted variable
  - "1" : to represent a variable in true form
  - "X" : to represent a variable not used in the product term

abcd	p-term
X1X1	bd
100X	$a\bar{b}\bar{c}$
0X10	$\bar{a}c\bar{d}$
X010	$\bar{b}c\bar{d}$

f = {X1X1, 100X, 0X10, X010}

(Easier to implement in a computer program)

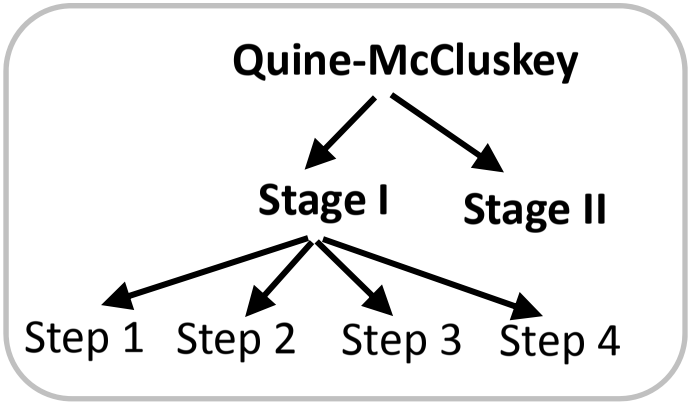
Two-Level Logic Synthesis: Quine-McCluskey Method

- **How do we use cubical notation to synthesize logic functions?**
  - Quine-McCluskey method
    - Willard Quine (1908-2000)) and Edward McCluskey (1928-present)

- **Quine-McCluskey method: (2 Stages)**

  Stage I : Take minterms for a function (Canonical) and form Prime Implicants (PIs).
  Stage II : Pick minimum Prime Implicants to generate a cover.

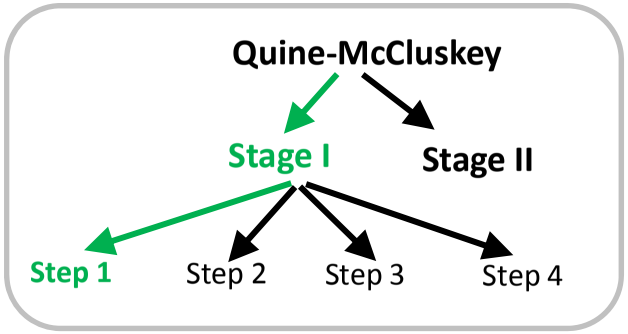
  A cover of a function is a set of cubes that represent that logic function



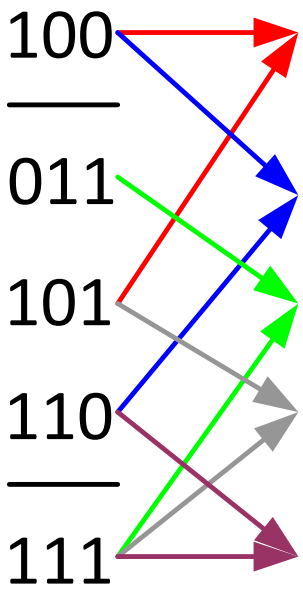
Quine-McCluskey Method : Stage I – Step 1

❑ **Stage I** of Quine-McCluskey method has four steps:

$$f(a,b,c)=\sum m(3,4,5,6,7)$$



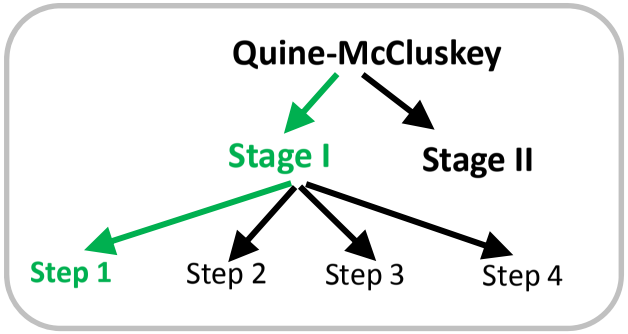
Step 1: List all minterms grouped in increasing order of the number of 1s they contain. Take cubes from adjacent groups to form larger product terms.



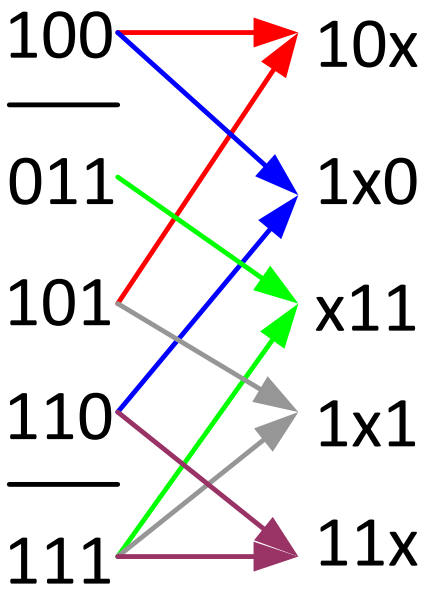
Quine-McCluskey Method : Stage I – Step 1

❑ **Stage I** of Quine-McCluskey method has four steps:

$$f(a,b,c)=\sum m(3,4,5,6,7)$$



Step 1: List all minterms grouped in increasing order of the number of 1s they contain. Take cubes from adjacent groups to form larger product terms.



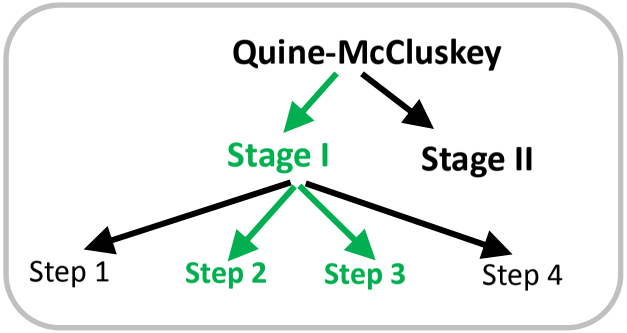
Quine-McCluskey Method : Stage I – Step 2/3

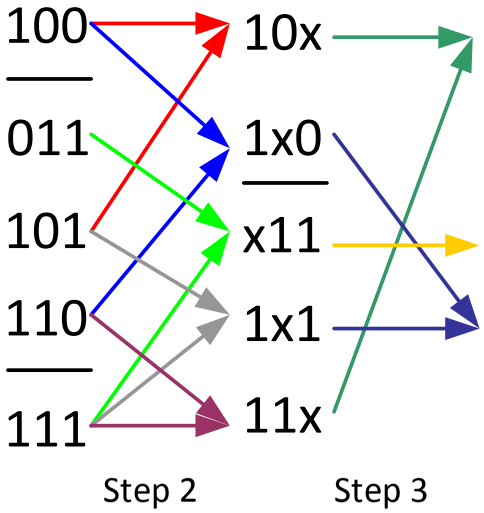
Step 2: Take original set of cubes and the newly created ones.
Remove any cube from the set that is completely covered by any other cube.

- In this case we remove all minterms and use only the newly created cubes.

Step 3: Group cubes in the increasing order of **non-zero positions.** By taking terms from adjacent groups form larger product terms.

- **Rules:** X's must match and the other position must differ by exactly one position (one cube has a 1 where the other has a 0).





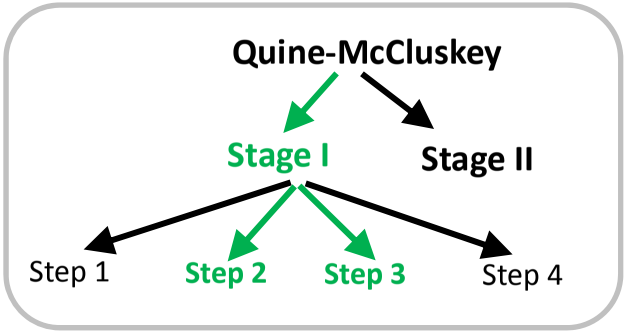
Quine-McCluskey Method : Stage I – Step 2/3

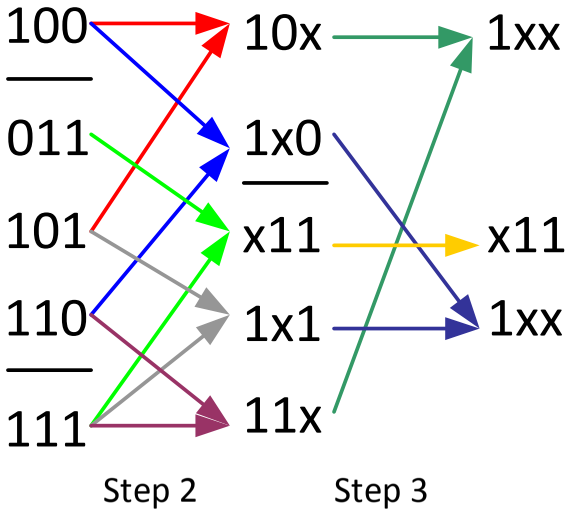
Step 2: Take original set of cubes and the newly created ones.
Remove any cube from the set that is completely covered by any other cube.

- In this case we remove all minterms and use only the newly created cubes.

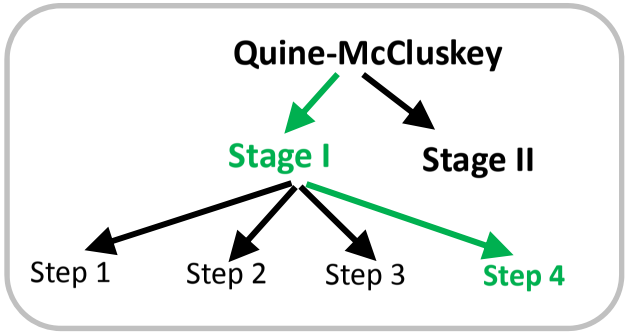
Step 3: Group cubes in the increasing order of **non-zero positions.** By taking terms from adjacent groups form larger product terms.

- **Rules:** X's must match and the other position must differ by exactly one position (one cube has a 1 where the other has a 0).



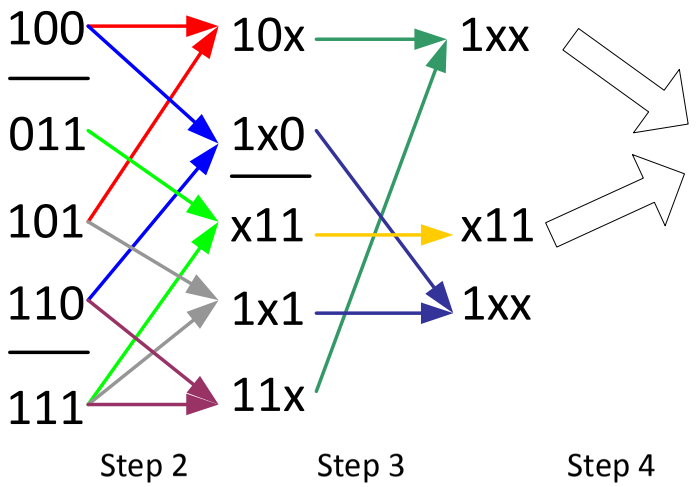


Quine-McCluskey Method : Stage I – Step 4

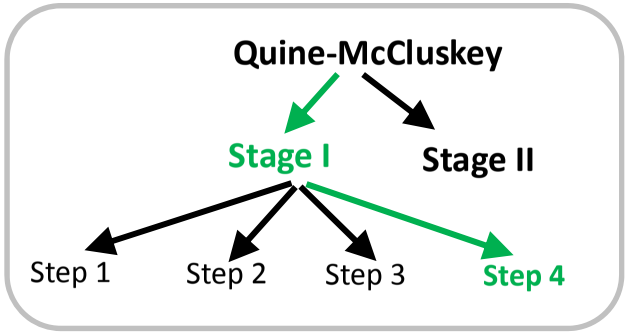


Step 4: Repeat steps 2-4 until all PIs are generated.

- In our case we are done, because 1xx and x11 are the PIs for this function.
- PI set = { 1xx, x11 }

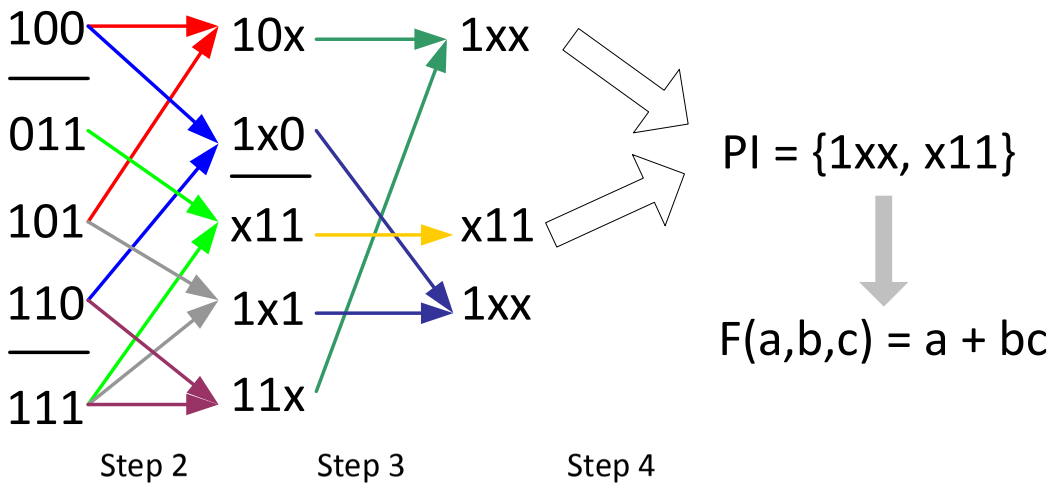


Quine-McCluskey Method : Stage I – Step 4

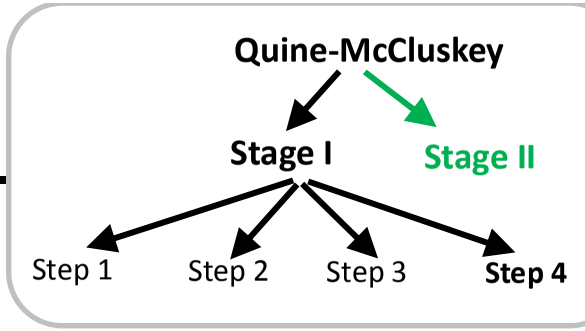


Step 4: Repeat steps 2-4 until all PIs are generated.

- In our case we are done, because 1xx and x11 are the PIs for this function.
- PI set = { 1xx, x11 }



Quine-McCluskey Method : Stage II



- **Stage II:** Build a table list the minterms that are covered by each PI
  - (Prime Implicant Cover Table)
- Pick the essential prime implicants (columns with only one check mark)
  - 1XX (the only one to cover 4, 5, 6, 7), X11 (the only one to cover 3)

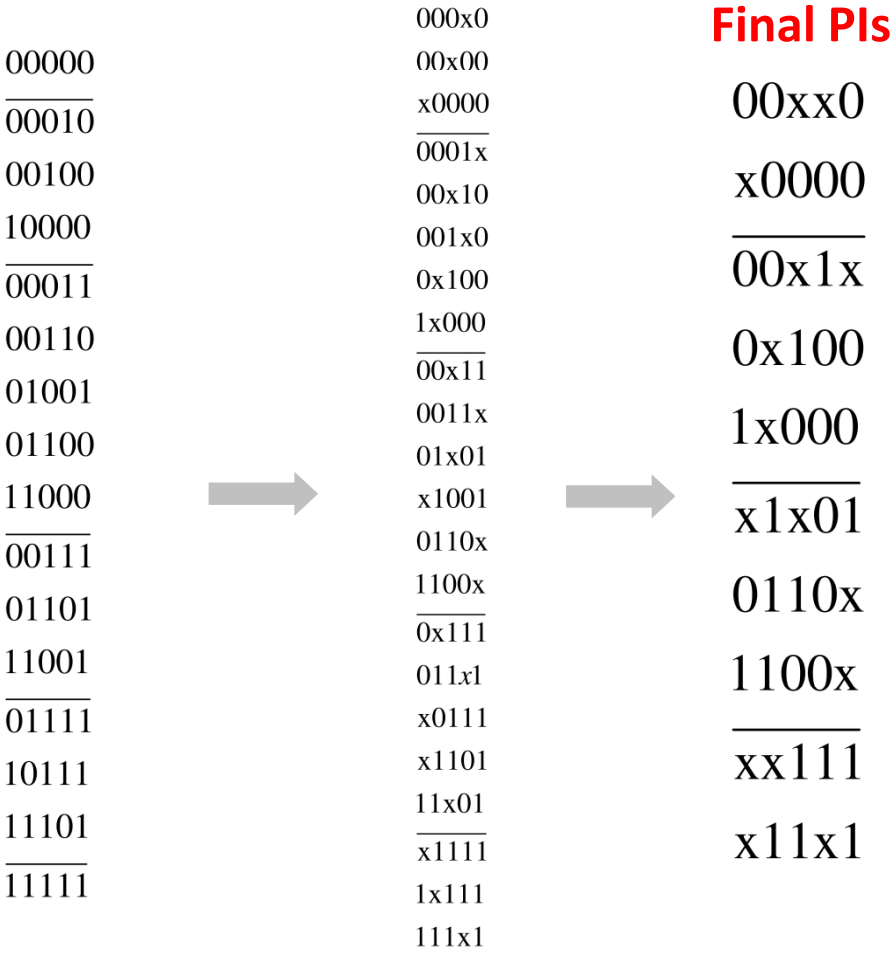
Cube	3	4	5	6	7
1xx		X	X	X	X
x11	X				

PI = {1xx, x11}

Done!

Quine-McCluskey Method : Stage I – (Step 1-4)

❖ **Example:** $f(a,b,c,d,e) = \sum m(0,2,3,4,6,7,9,12,13,15,16,23,24,25,29,31)$



Final PIs

00xx0

x0000

00x1x

0x100

1x000

x1x01

0110x

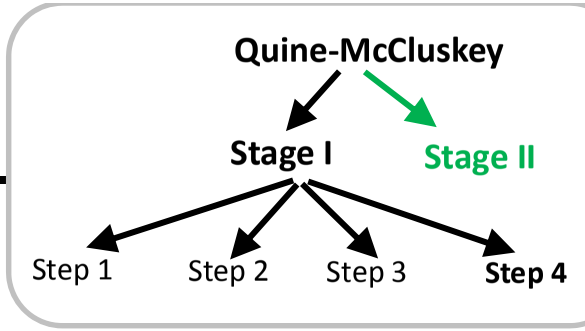
1100x

xx111

x11x1

When starting with minterms, we need to only consider the combination of cubes that create a larger cube (more x's).

Quine-McCluskey Method : Stage II

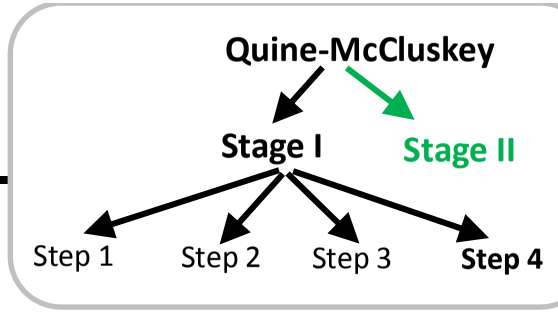


- **Stage II:** Build a table list the minterms that are covered by each PI
  - (Prime Implicant Cover Table)
- Pick the essential prime implicants (columns with only one check mark)
  - 00x1x, x1x01, xx111 (the only one to cover 23)

Cube	0	2	3	4	6	7	9	12	13	15	16	23	24	25	29	31
00xx0	X	X		X	X											
x0000	X										X					
00x1x		X	X		X	X										
0x100				X				X								
1x000											X		X			
x1x01							X		X					X	X	
0110x								X	X							
1100x													X	X		
xx111						X				X		X				X
x11x1									X	X					X	X

PI = {00x1x, x1x01, xx111}

Quine-McCluskey Method : Stage II



- Remove the columns covered by the essential prime implicants
  - Columns: 2, 3, 6, 7 covered by 00x1x
  - Columns: 9, 13, 25, 29 by x1x01
  - Columns: 15, 23, 31 by xx111

Remove used PIs and covered minterms from the table

Cube	0	2	3	4	6	7	9	12	13	15	16	23	24	25	29	31
00xx0	X	X		X	X											
x0000	X										X					
00x1x		X	X		X	X										
0x100				X				X								
1x000											X		X			
x1x01							X		X					X	X	
0110x								X	X							
1100x													X	X		
xx111						X				X		X				X
x11x1									X	X					X	X

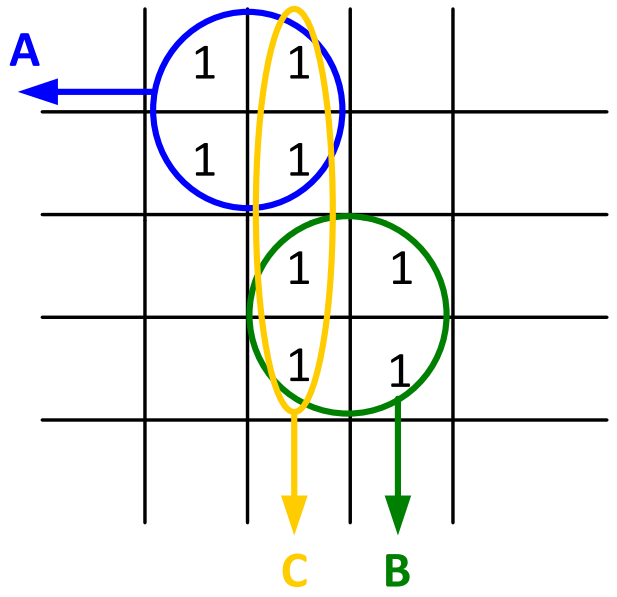
PI = {00x1x, x1x01, xx111}

© M. Shabany, ASIC/FPGA Chip Design

Quine-McCluskey Method : Stage II

- Notice that cube x11x1 does not cover any of the remaining minterms. So remove it.

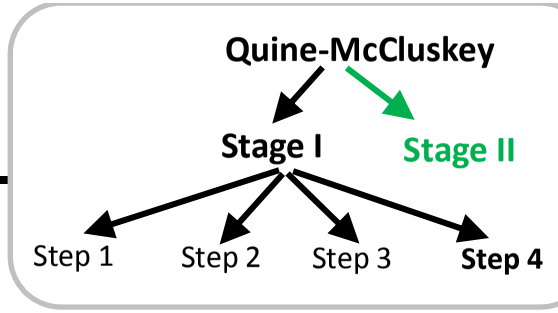
	0	4	12	16	24
00xx0	X	X			
X0000	X			X	
0x100		X	X		
1x000				X	X
0110x			X		
1100x					X
x11x1					



If we select A and B, then C does not do anything!

PI = {00x1x, x1x01, xx111}

Quine-McCluskey Method : Stage II



❑ Definition 1: Row dominance

- ➢ Row A in the covering table dominates another row B if and only if row A covers at least the same set of columns as row B, and the PI in row B has equal or greater cost than the PI in row A.
- ➢ Notice that 1x000 dominates 1100x, 0x100 dominates 0110x., so:

	0	4	12	16	24
00xx0	X	X			
X0000	X			X	
0x100		X	X		
1x000				X	X
0110x			X		
1100x					X

Remove

	0	4	12	16	24
00xx0	X	X			
X0000	X			X	
0x100		X	X		
1x000				X	X

PI = {00x1x, x1x01, xx111}

Quine-McCluskey Method : Stage II

❑ Definition 1: Column dominance

- ➢ Column i dominates column j if and only if column i is covered by at least the same set of cubes as column j. In such cases we remove the column i from the table.
- ➢ Notice that column 4 dominates column 12, and column 16 dominates 24, so:

	0	4	12	16	24
00xx0	X	X			
X0000	X			X	
0x100		X	X		
1x000				X	X

	0	12	24
00xx0	X		
X0000	X		
0x100		X	
1x000			X

PI = {00x1x, x1x01, xx111, 0x100, 1x00}

Essential PIs

Quine-McCluskey Method : Stage II

- By row dominance, x0000 is removed, so:
  - It covers the same number of columns as 00xx0, but it has a larger cost

	0	12	24
00xx0	X		
X0000	X		

	0	12	24
00xx0	X		

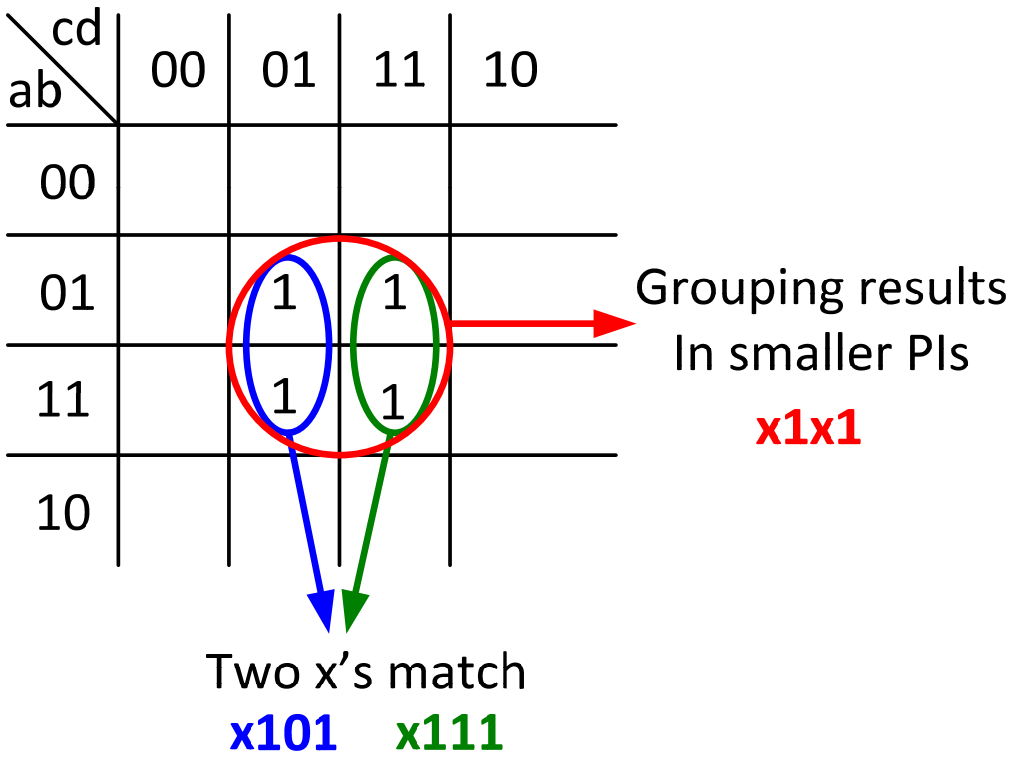
PI = {00x1x, x1x01, xx111, 0x100, 1x00, 00xx0}

Quine-McCluskey Method : Summary

1. Generate all PIs starting with minterms
2. Create a prime implicant covering table. List PIs in the rows and function minterms in the columns.
3. For each PI indicate which minterms it covers by putting a checkmark in the corresponding column.
4. For each column covered by exactly one PI add the corresponding PI to your cover, removing the PI from the table, as well as any columns it covers.
5. Apply concepts of row and column dominance to reduce the table.
6. Repeat steps 4-6 until the table cannot be further reduced.

Quine-McCluskey Method : Note 1

- Why should x's match?
  - Matching x's is equivalent to grouping PIs in the Karnaugh map.



Quine-McCluskey Method : Note 2

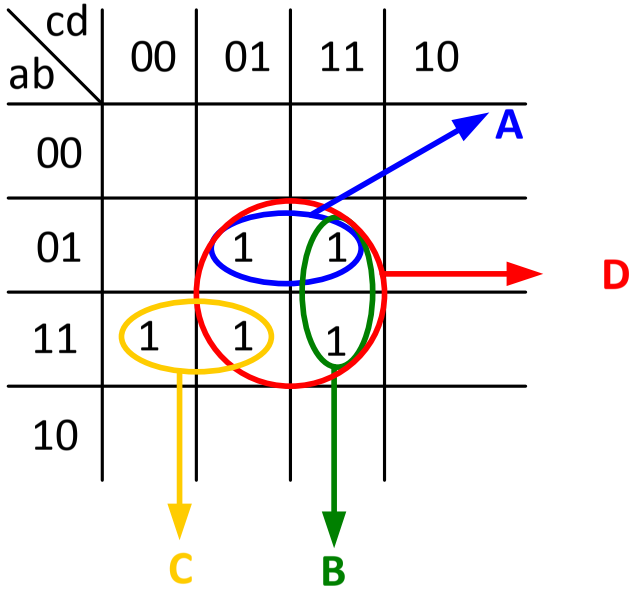
- When starting with minterms (Canonical form), the above procedure works fine.
- Otherwise, it is necessary to represent the function as a sum of minterms first

❖ Example:

$$f = \underbrace{\bar{a}bd}_{A} + \underbrace{bcd}_{B} + \underbrace{ab\bar{c}}_{C} \longrightarrow \text{PI set} = \{ 01x1, x111, 110x\}$$

- Not possible to combine/reduce the above PI set (no x's match)
- Represent "f" as a sum of minterms (canonical form)

$$f = ab\bar{c}\bar{d} + ab\bar{c}d + abcd + \bar{a}b\bar{c}d + \bar{a}bcd$$



Outline

- Introduction to Synthesis
- Digital Logic Basics
- **Logic Optimization**
  - Two-level logic synthesis
  - Multi-level logic synthesis
- Technology Mapping
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Multi-Level Logic Synthesis

- Two-level logic synthesis is effective and mature
- Two-level logic synthesis is directly applicable to PLAs and PLDs

However, …

- There are many functions that are too expensive to implement in two-level forms (too many product terms!)
- Two-level implementation constrains layout (AND-plane, OR-plane)

Multi-level logic synthesis may be employed

- Rule of thumb:
  - Two-level logic is good for control logic
  - Multi-level logic is good for data path or random logic

Multi-Level Logic Synthesis

- Multi-level logic synthesis:
  - Decompose a logic function into smaller functions
  - A simple tool to do this is called Shannon's Decomposition
    - Claude Shannon 1916-2001
- Shannon's Expansion Theorem:

Any logic function $f(x_1, x_2, \ldots, x_n)$ can be expanded in the form of:

$$x_k . f(x_1, x_2, x_{k-1}, 1, x_{k+1}, \ldots, x_n) + \bar{x}_k . f(x_1, x_2, x_{k-1}, 0, x_{k+1}, \ldots, x_n)$$
$$= x_k . f_{x_k} + \bar{x}_k . f_{\bar{x}_k}$$

- $f_{x_k} = f(x_k = 1, \ldots)$: 1- cofactor
- $f_{\bar{x}_k} = f(x_k = 0, \ldots)$: 0- cofactor

Multi-Level Logic Synthesis

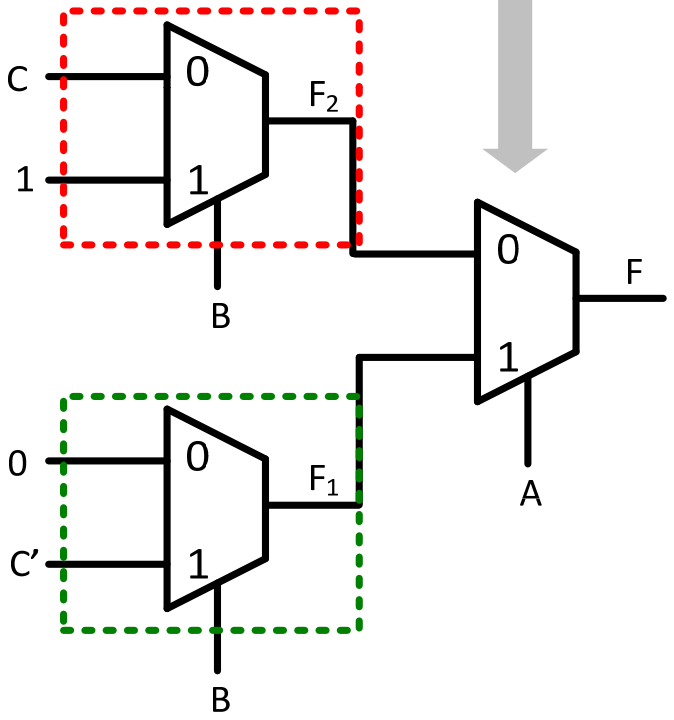
❖ Example:

- F(A, B, C) = A'B + ABC' + A'B'C

  = A(BC') + A'(B+B'C) = $A.F_1 + A'.F_2$

- F_2 = B+B'C = B.1 + B'.C

- F_1 = BC' = BC' + B'.0



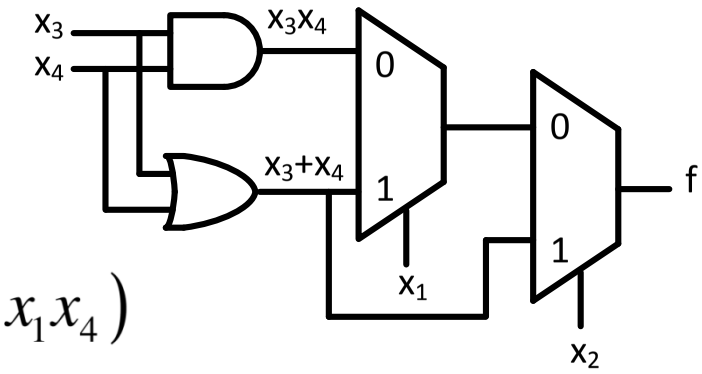
Multi-Level Logic Synthesis

❖ Example:

$$f = \overline{x}_1\overline{x}_2x_3x_4 + x_1x_3 + x_1x_4 + x_2x_3 + x_2x_4$$

$$f = x_2\left(x_1x_3 + x_1x_4 + x_3 + x_4\right) + \overline{x}_2\left(\overline{x}_1x_3x_4 + x_1x_3 + x_1x_4\right)$$

$$f = x_2\left(x_3 + x_4\right) + \overline{x}_2\left(\overline{x}_1x_3x_4 + x_1\left(x_3 + x_4\right)\right)$$

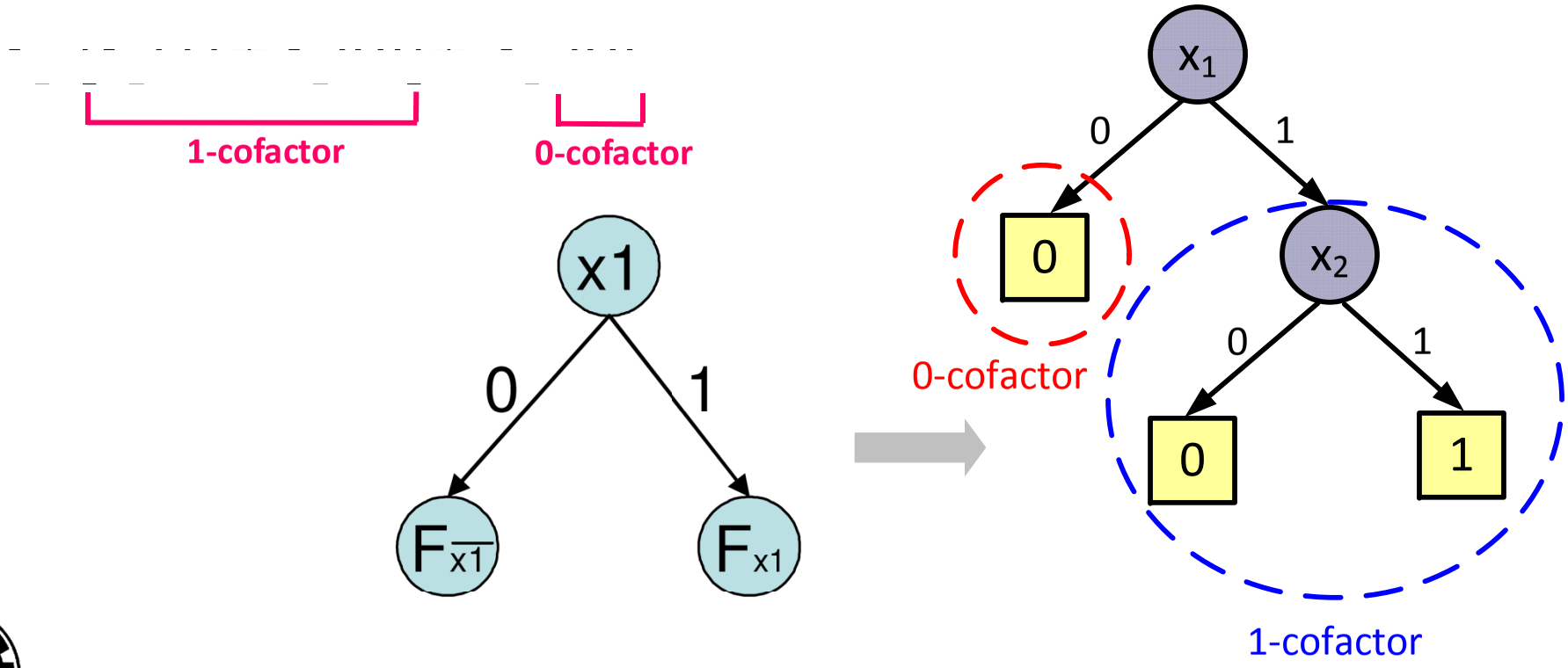


- We found out that logic expression (x_3+x_4) appears in multiple places. Hence, this knowledge could be used to simplify the implementation of this circuit.
- Keeping track of this type of relationships in a logic function can be tedious in a large expression.
- Can we represent this information in a better way?
  - Yes, we can use **Binary Decision Diagrams (BDDs)**

Binary Decision Decomposition (BDD)

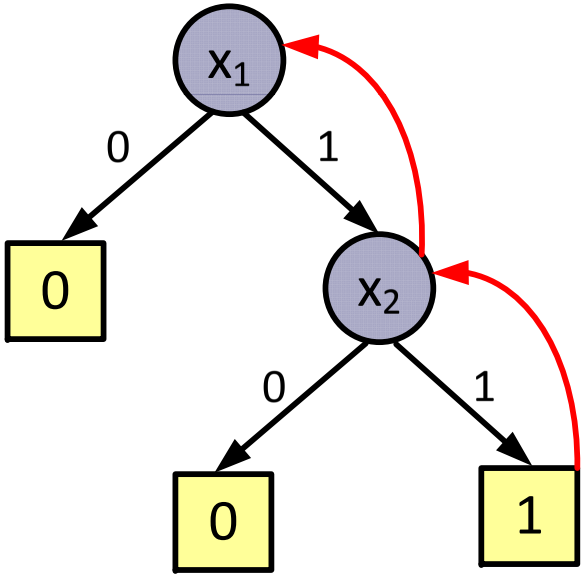
- Let us decompose a function "$f=x_1x_2$" with respect to x_1:

$$f = x_1 . f_{x_1} + \overline{x}_1 . f_{\overline{x}_1}$$

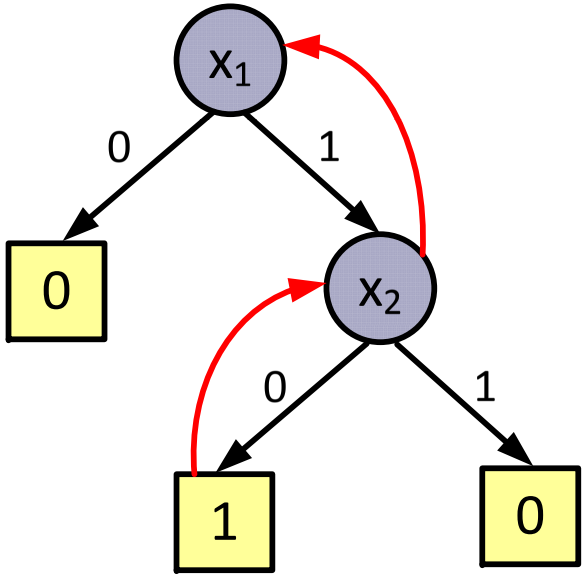


Binary Decision Decomposition (BDD)

- To derive the function from BDD, start from the bottom to top
- Start only with terminal nodes 1
- If see edge “1” use the variable otherwise its complement



$f = x_1 x_2$



$f = x_1 \bar{x}_2$

Binary Decision Decomposition (BDD)

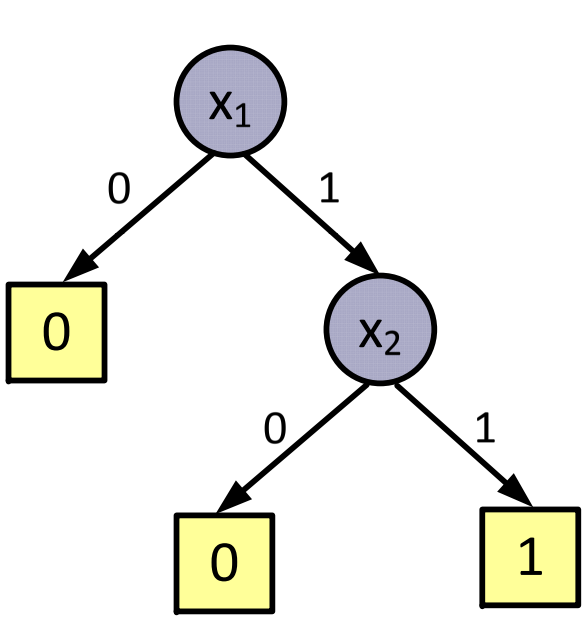
- A binary decision diagram represents a logic function by using Shannon's decomposition to decompose a function into cofactors, one variable at a time:

$$f = x_k . f_{x_k} + \overline{x}_k . f_{\overline{x}_k}$$

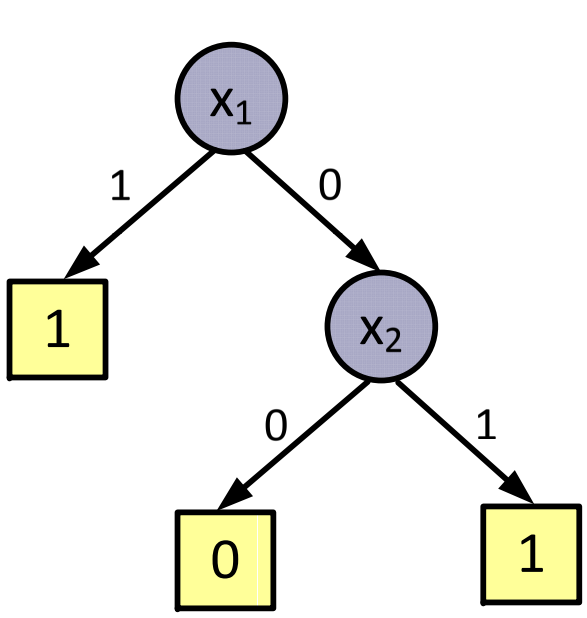
- The decomposition steps are then represented as a directed graph G = (V,E), where:
  - **V** is a set of vertices. Each vertex is associated with a variable or a constant 0/1.
  - **E** is a set of directed edges. Each edge is assigned a label of 0 or 1. A 0 edge always points to a 0-cofactor and a 1 edge points to a 1-cofactor.

Binary Decision Decomposition (BDD)

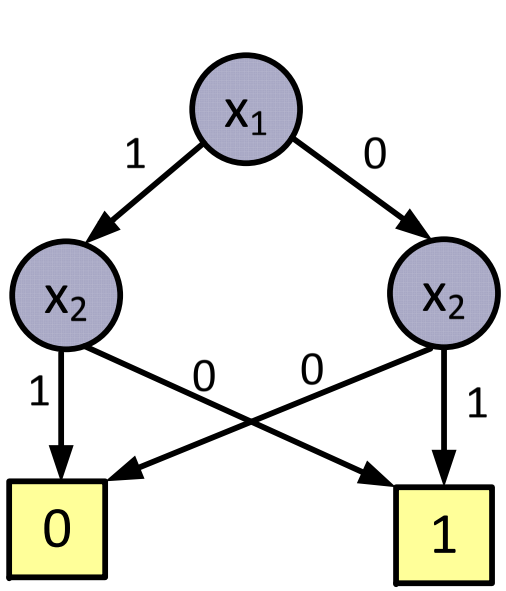
- BDD of some basic logic functions:



$f = x_1 x_2$



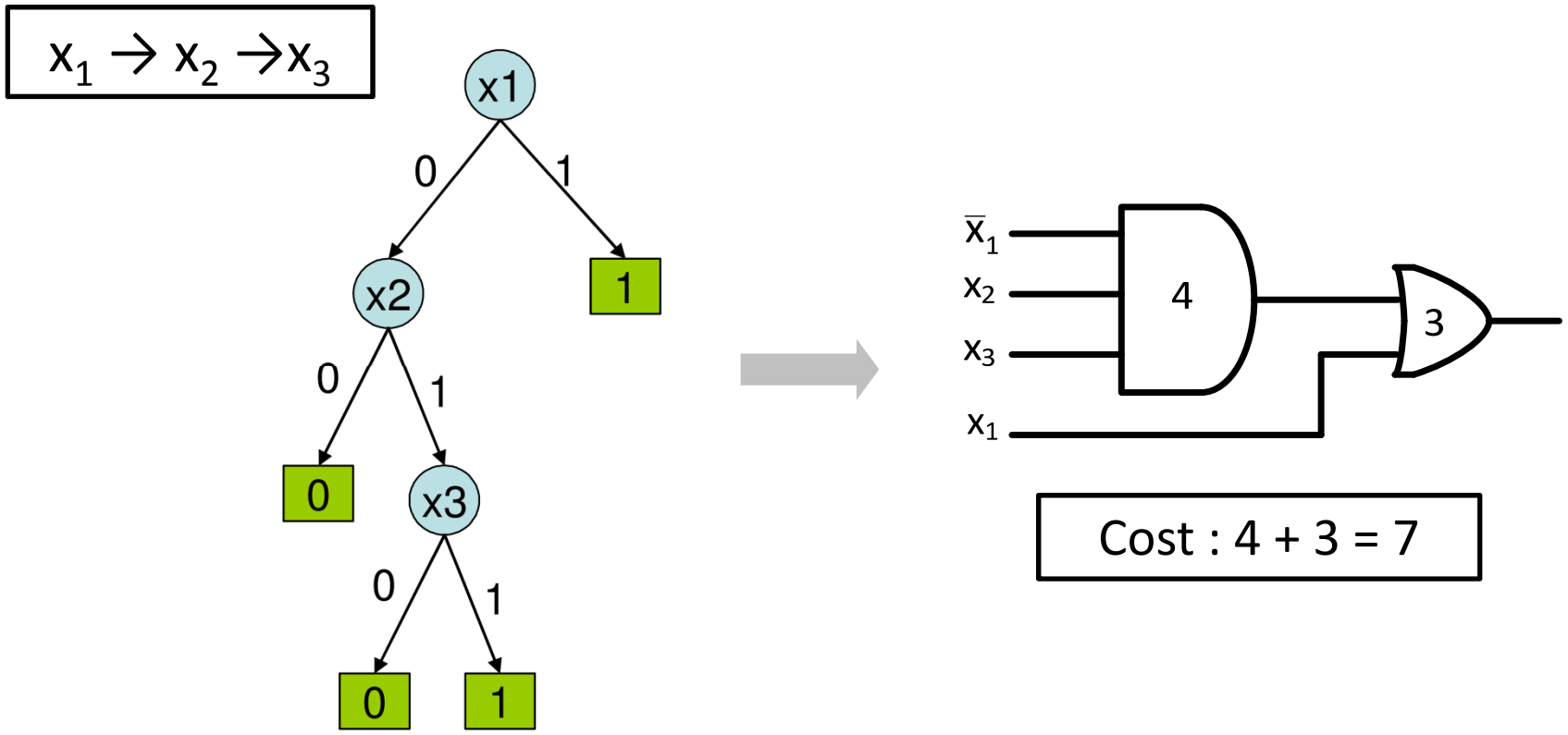
$f = x_1 + x_2$



$f = x_1 \oplus x_2$

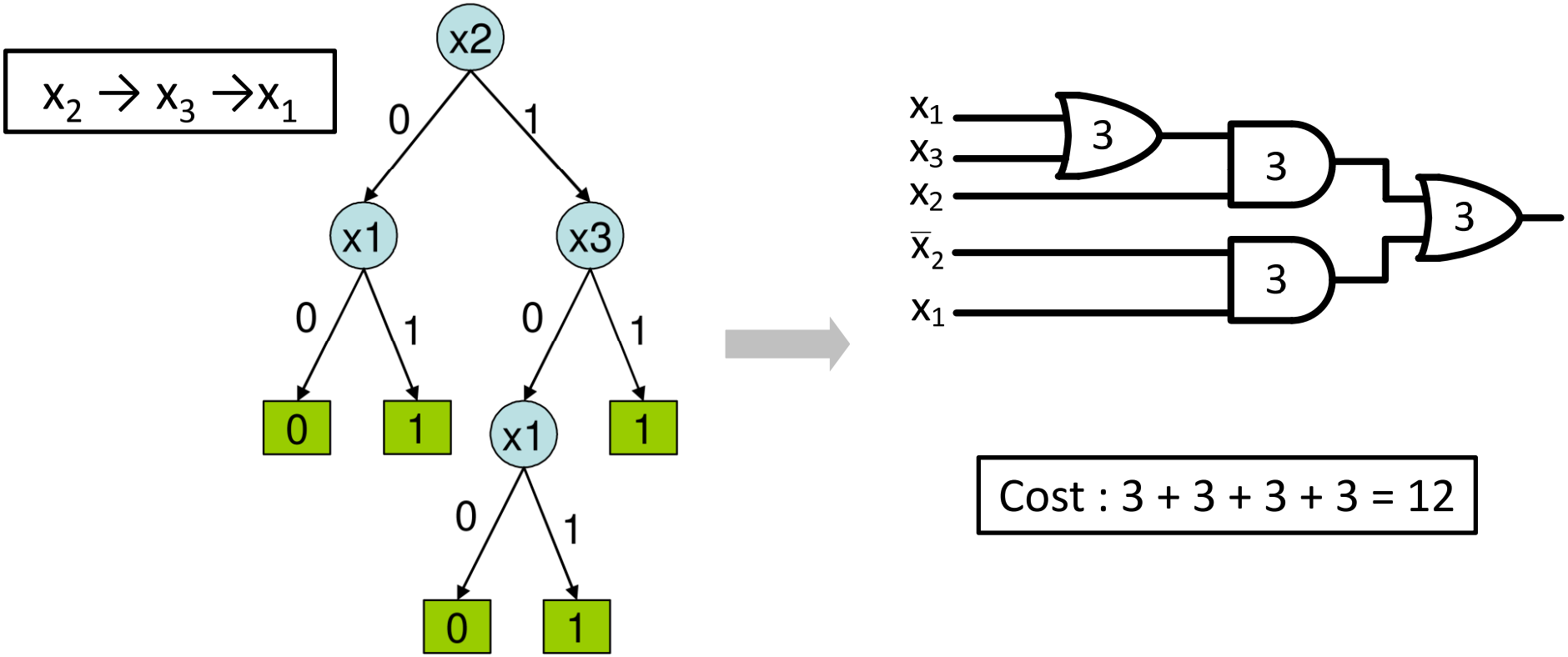
BDD: Decomposition Order Matters

- Example: $f = x_1 + x_2x_3 = x_1(1) + \bar{x}_1x_2x_3$



BDD: Decomposition Order Matters

- Example: $f = x_1 + x_2x_3$

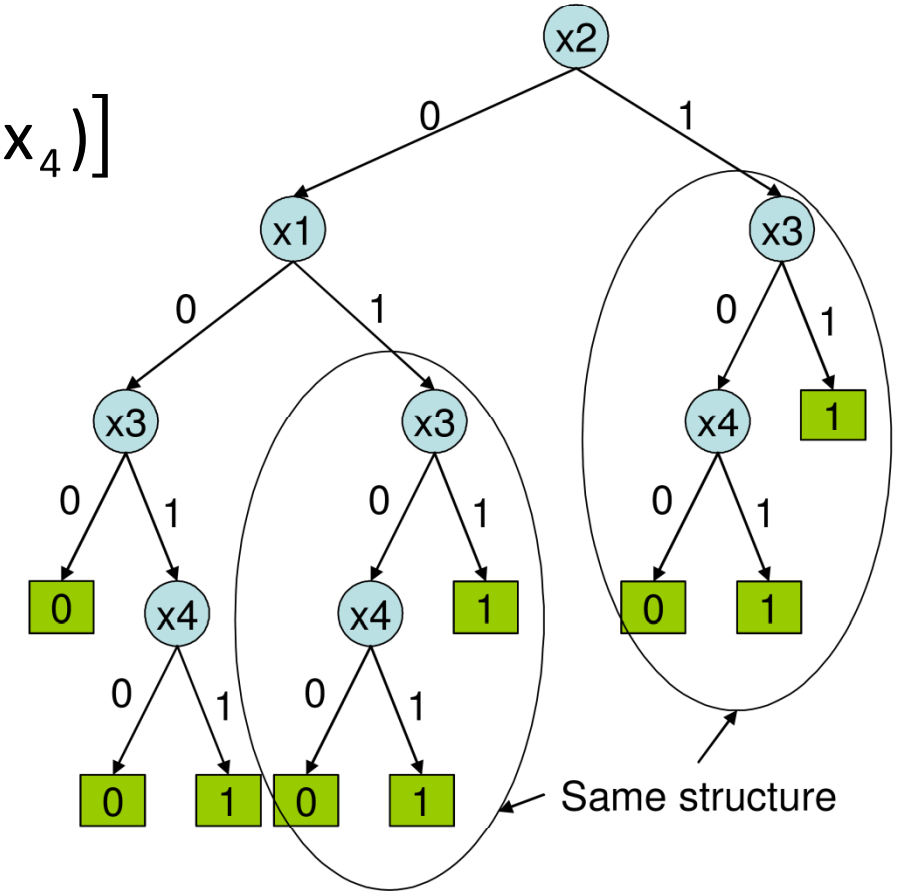


Different order of decomposition => different size/cost of the diagram

Ordered Binary Decision Decomposition (OBDD)

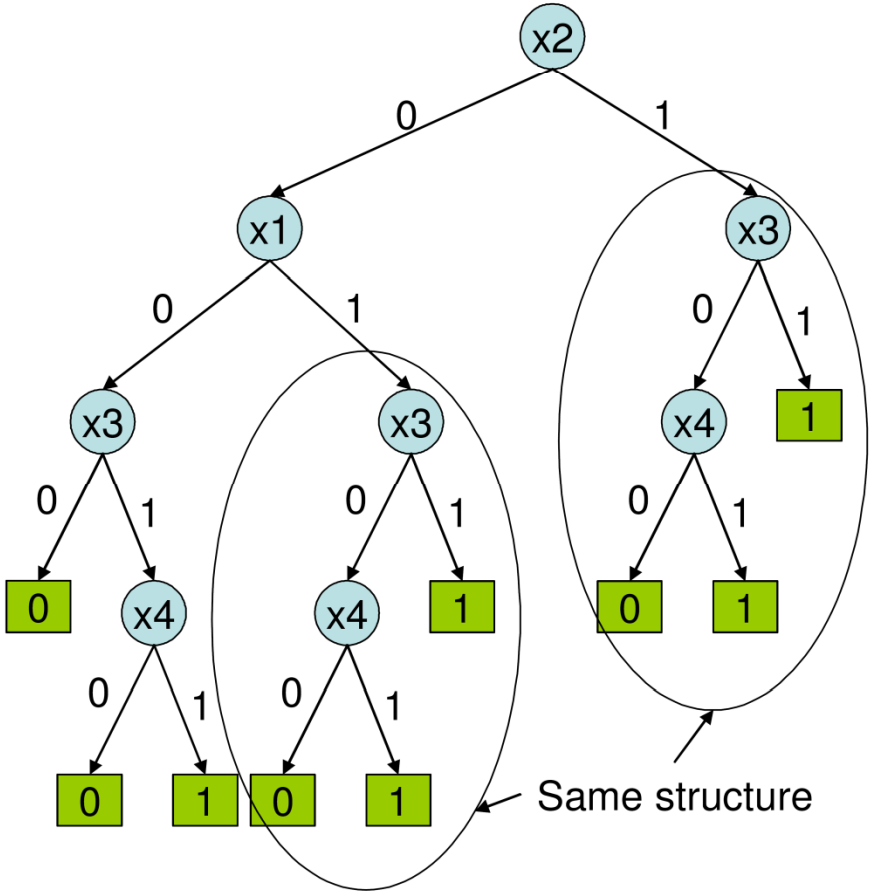
- We could impose a decomposition order by fixing the sequence of variable with respect to which we decompose a function **(Ordered BDD = OBDD)**
- Let us consider the following function:

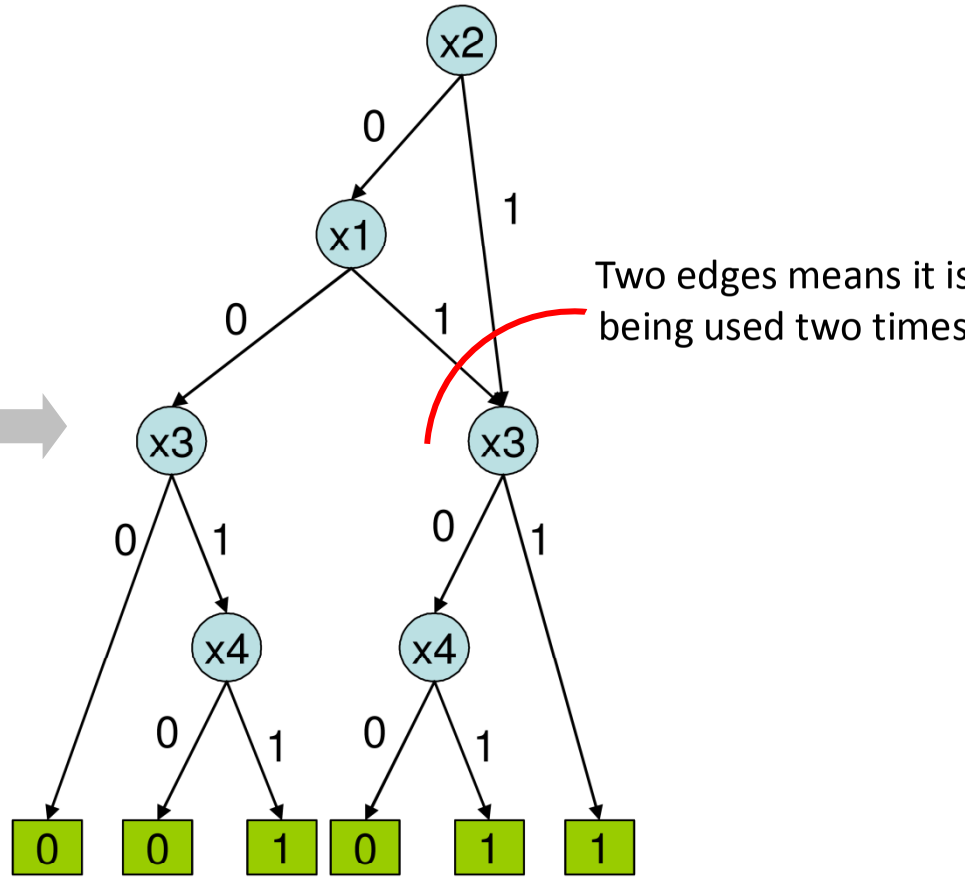
$$f = x_2(x_3 + x_4) + \overline{x}_2\left[\overline{x}_1(x_3 x_4) + x_1(x_3 + x_4)\right]$$



Reducing OBDDs

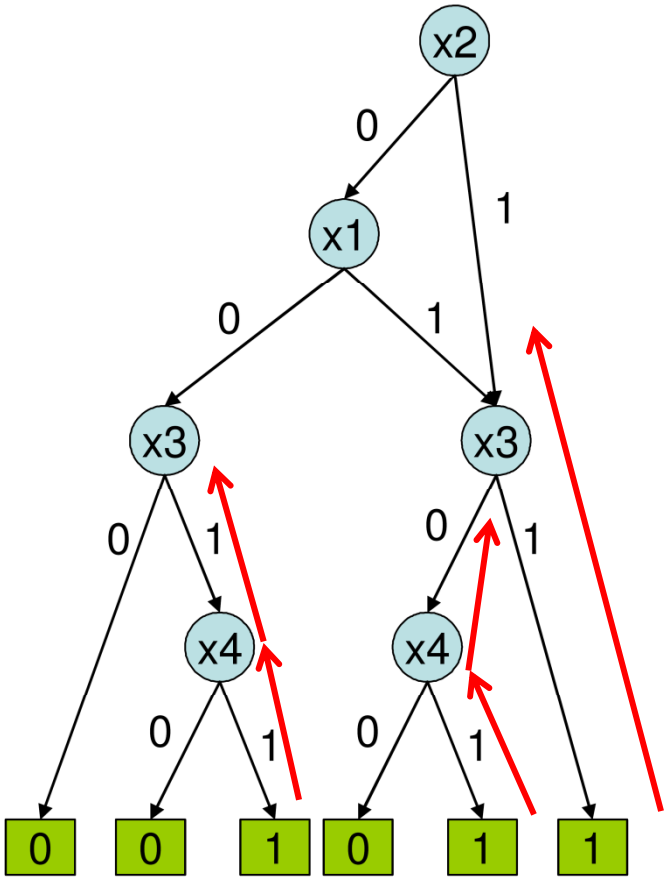
- Similar structure can be used to reduce the OBDD





Reducing OBDDs

- Cost of this OBDD:

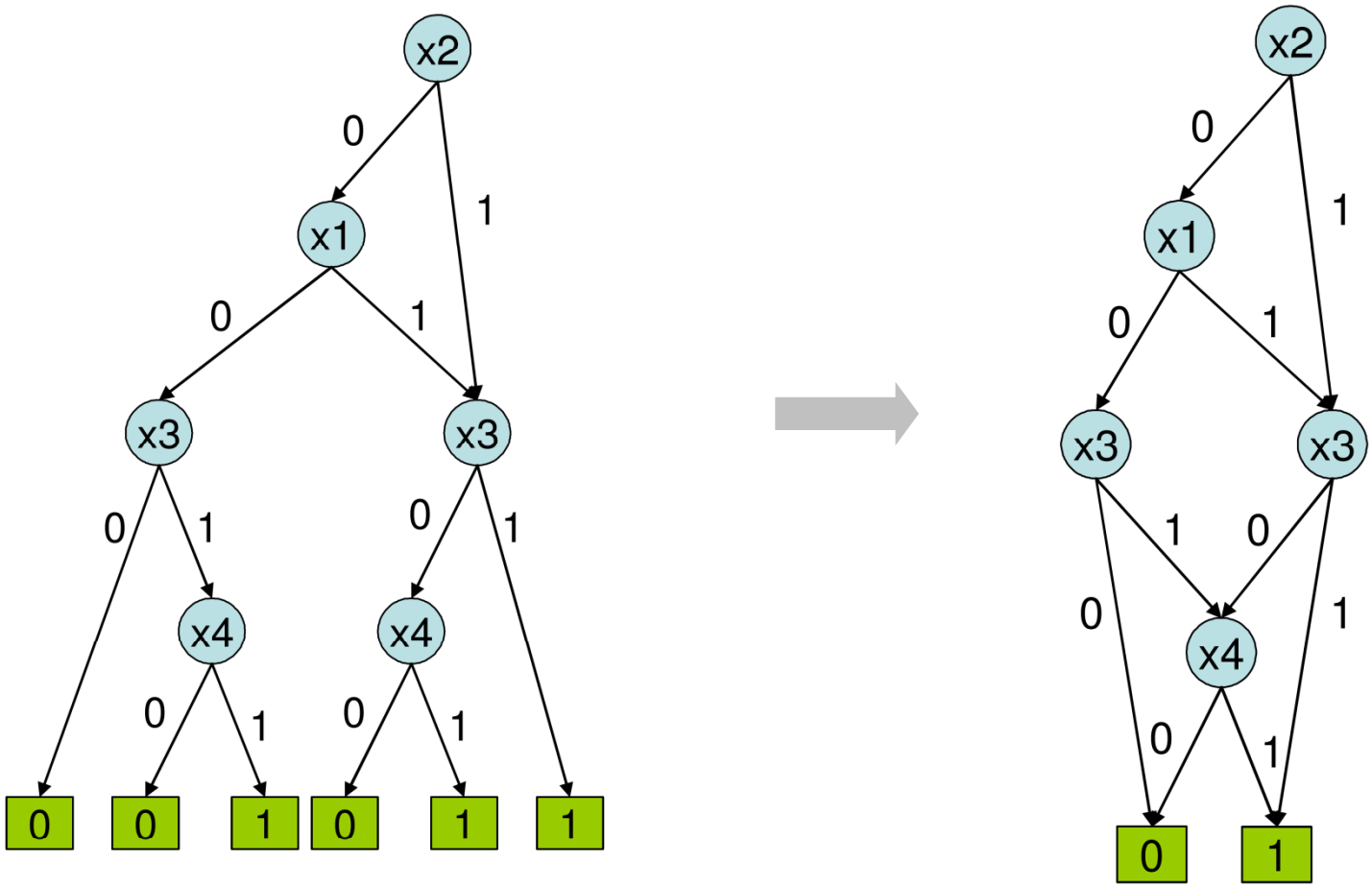


$$f = x_4 x_3 \bar{x}_1 \bar{x}_2 + (x_4 + x_3)(x_1 + x_2)$$

Cost : 5 + 3 + 3 + 3 + 3 = 17

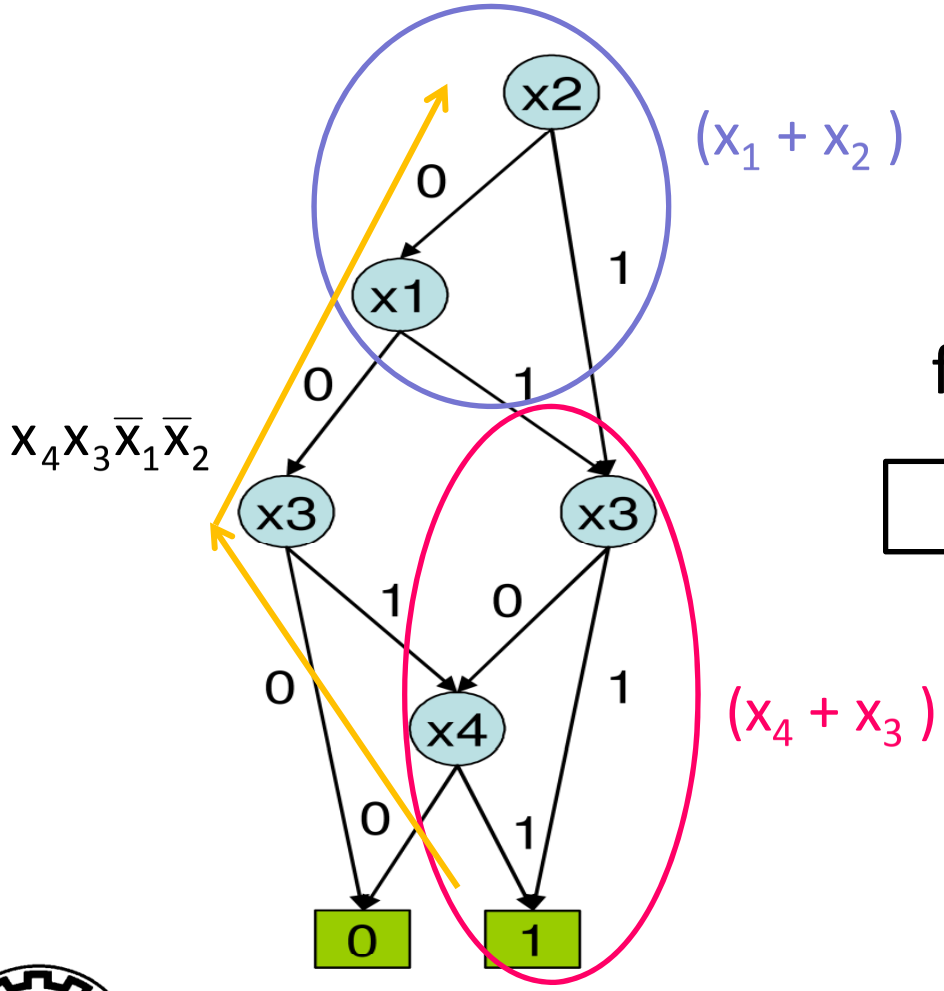
Reduced Ordered BDD (ROBDD)

- Bottom-up merging of the isomorphic graphs together to simplify the OBDD into a **Reduced Ordered BDD** (ROBDD)



Reduced Ordered BDD (ROBDD)

- Thus, bottom-up merging the graphs together will reduce the cost

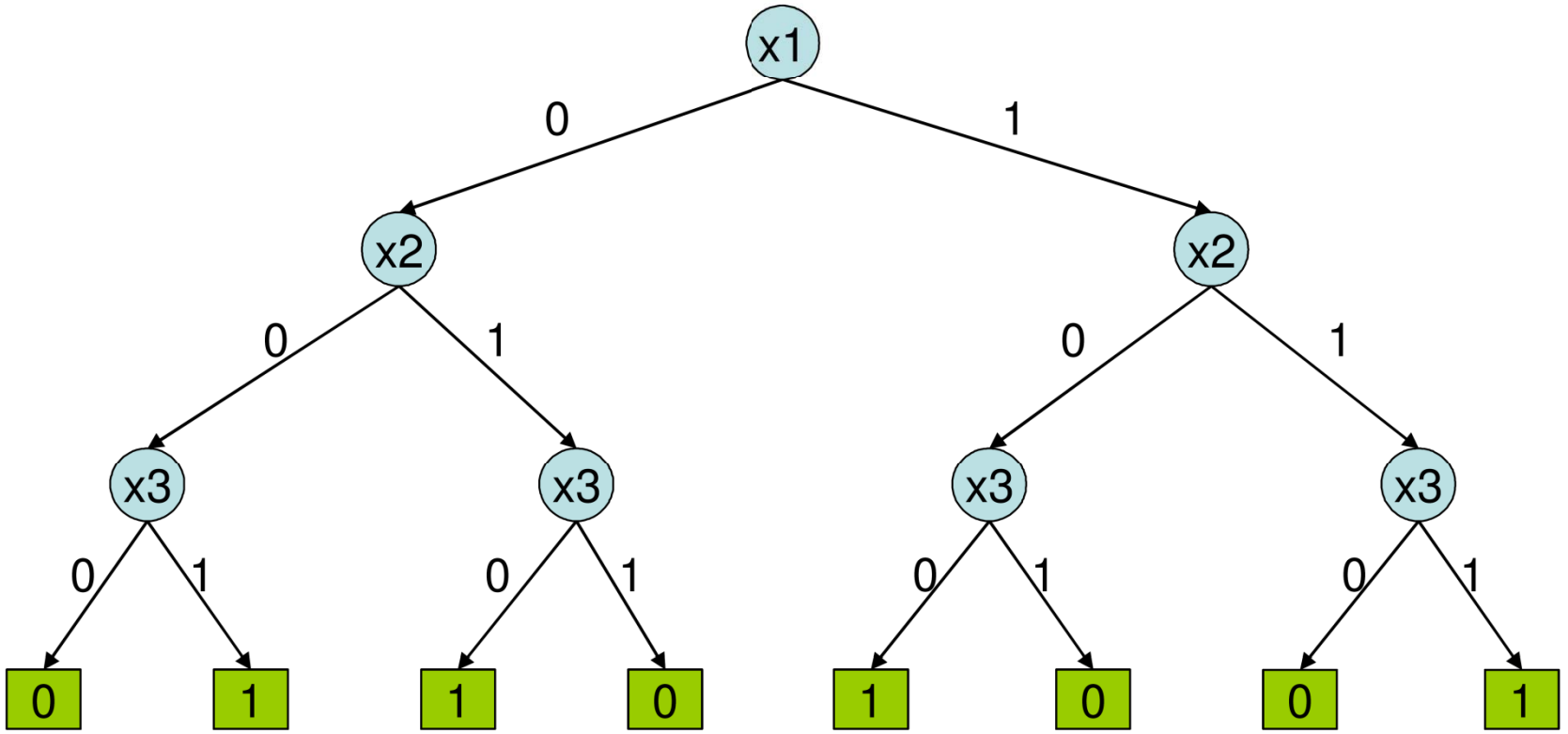


$$f = x_4x_3\bar{x}_1\bar{x}_2 + (x_3 + x_4)(x_1 + x_2)$$

Cost : 5 + 3 + 3 + 3 + 3 = 17

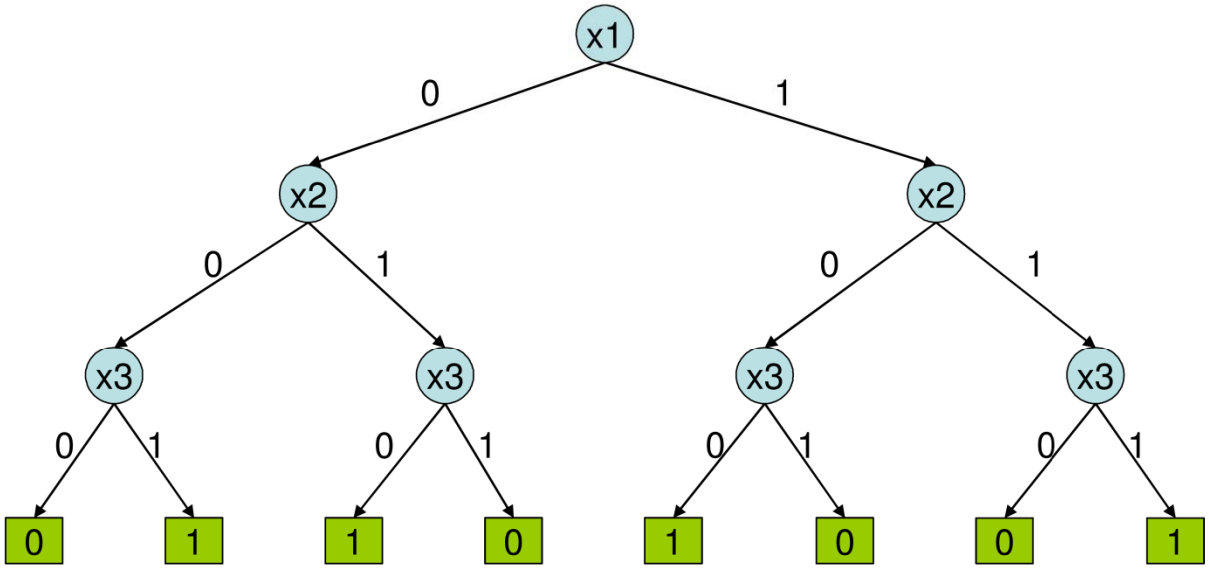
Reduced Ordered BDD (ROBDD)

❖ **Example:** $f = x_1 \oplus x_2 \oplus x_3$

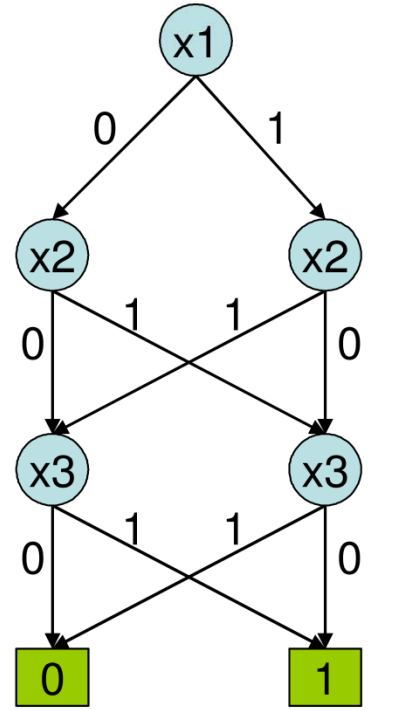


Reduced Ordered BDD (ROBDD)

❖ **Example:** $f = x_1 \oplus x_2 \oplus x_3$



15 nodes



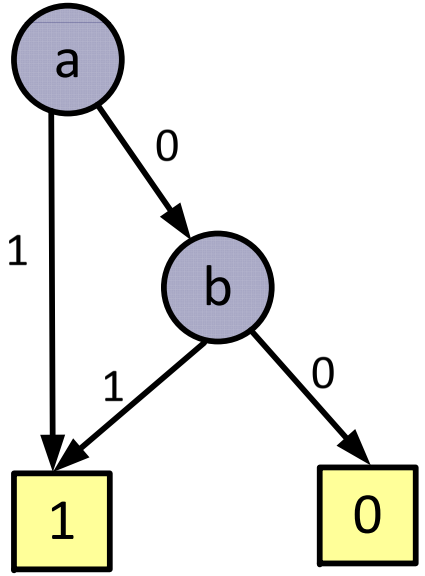
7 nodes

BDD of f <op> g

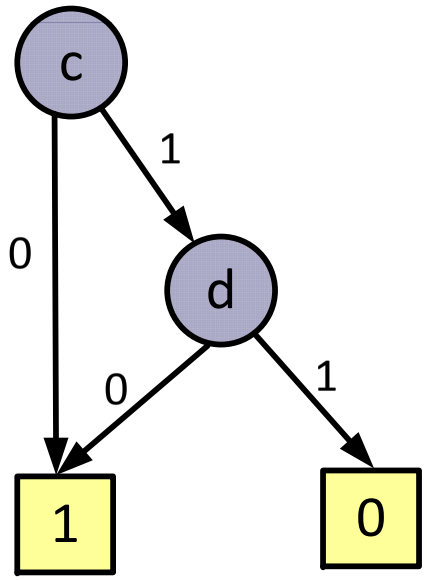
- Goal: take BDDs for functions “f” and “g” and produce a BDD for **f <op> g.**
- **Case 1:** functions f and g have distinct support (i.e., $\sup(f) \cap \sup(g) = \phi$)

❖ **Example:**

$f = a + b$

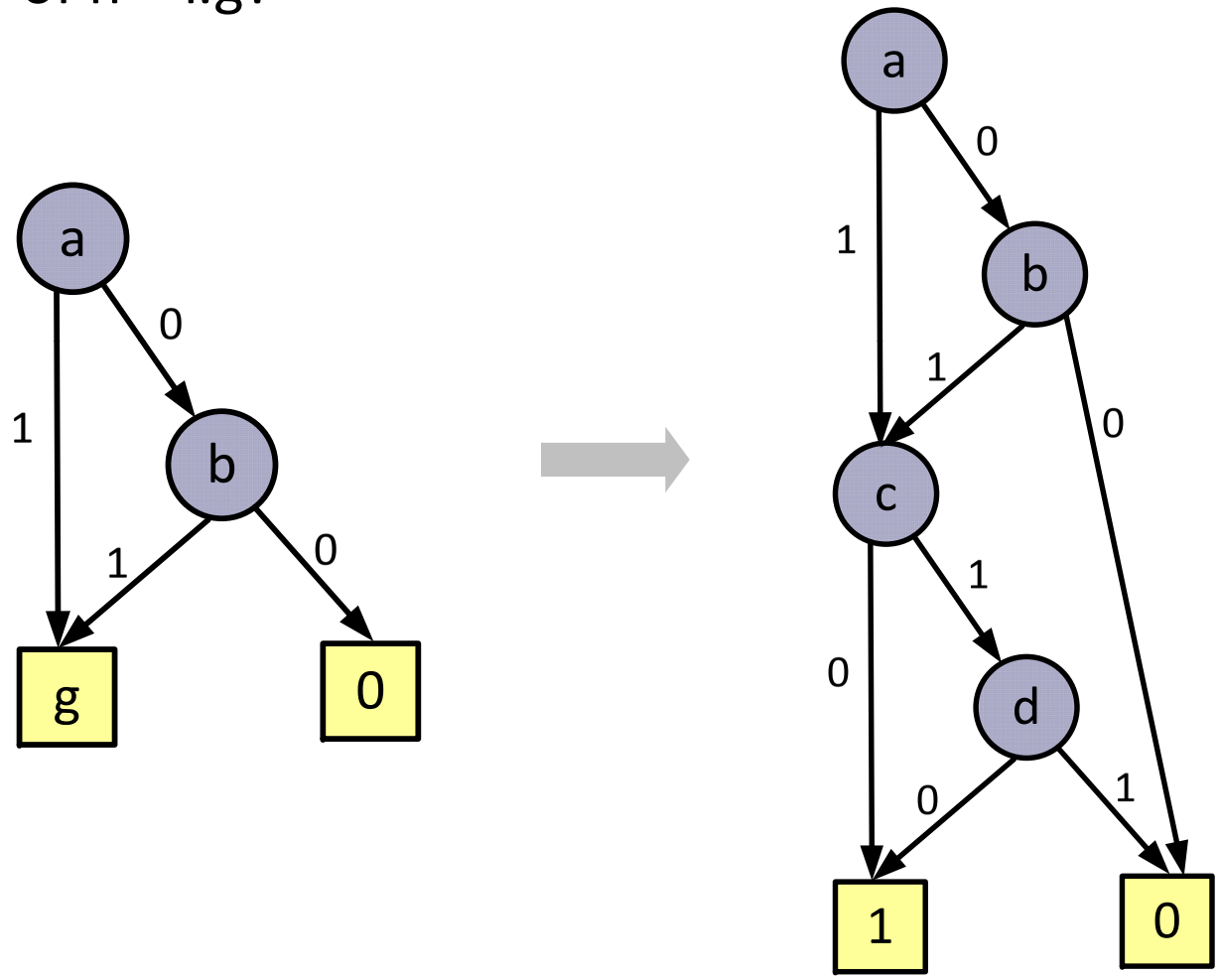


$g = \overline{cd}$



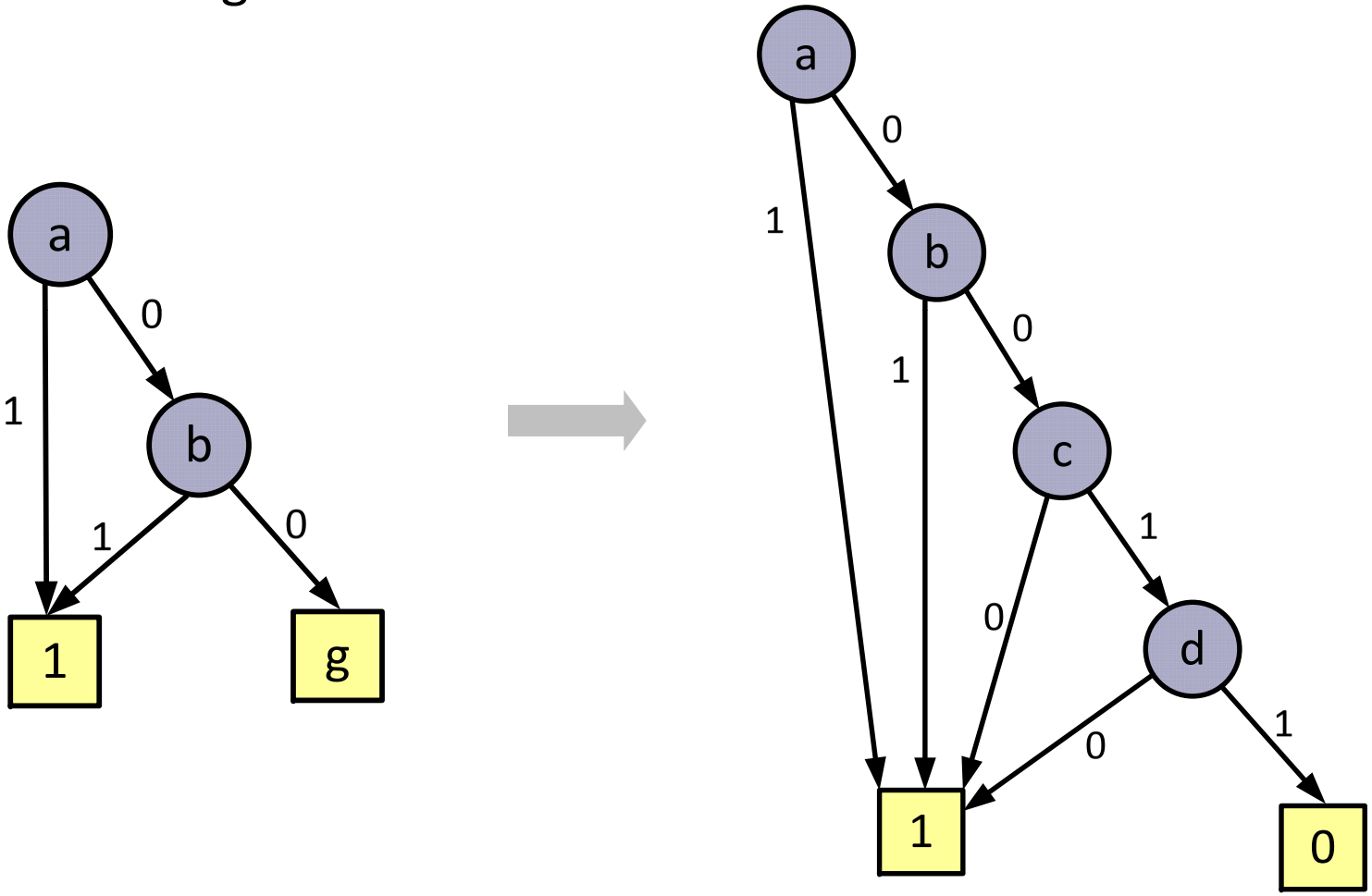
BDD of (f . g)

- BDD of h = f.g?



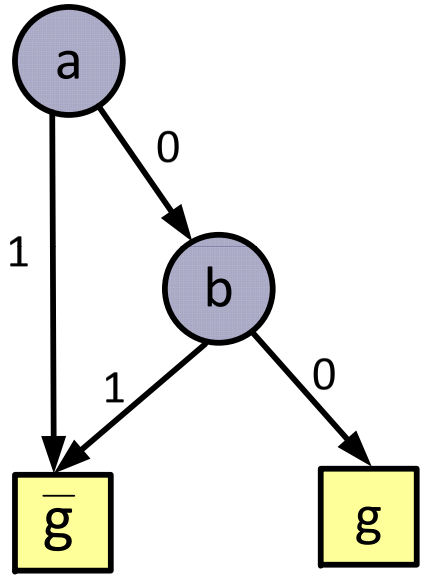
BDD of (f + g)

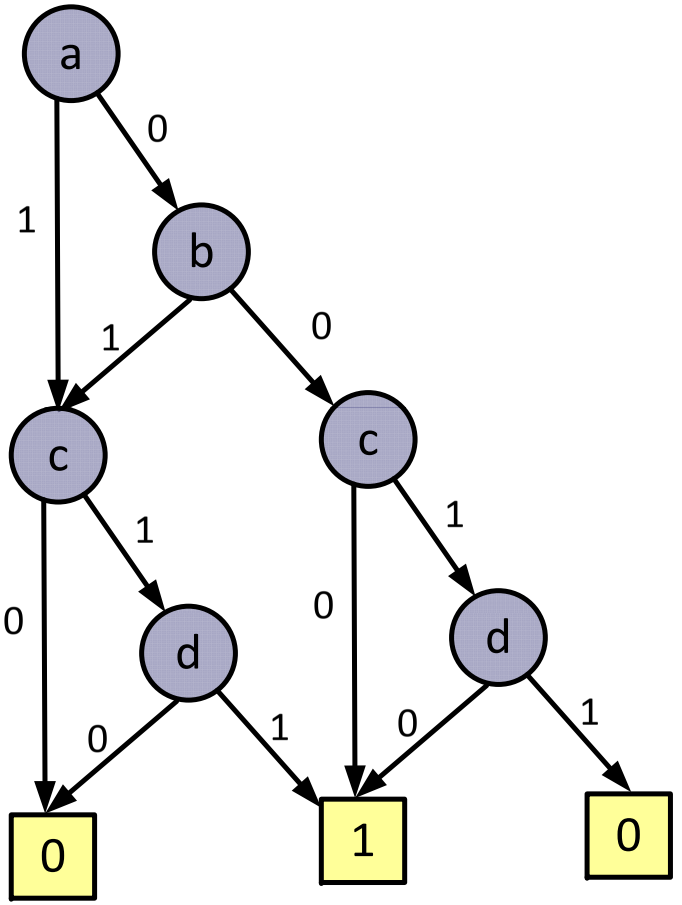
❑ BDD of h = f+g?



BDD of ($h = f \oplus g$)

- BDD of $h = f \oplus g = \bar{f}g + f\bar{g}$?



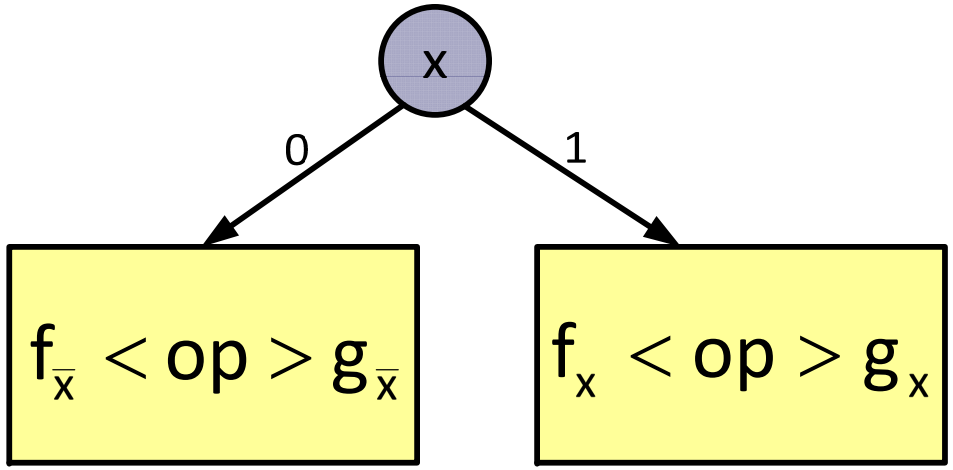


When the support of "f" and the support of "g" are disjoint, then replace terminals of f with 0, 1, g, or $\bar{g}$

BDD of ($h = f \oplus g$)

- **Case 2:** both functions have the same support (i.e., $\sup(f) = \sup(g)$)

$$h = f(x,y) < op > g(x,y) = \left[xf(1,y) + \bar{x}f(0,y)\right] < op > \left[xg(1,y) + \bar{x}g(0,y)\right]$$
$$= x\left[f(1,y) < op > g(1,y)\right] + \bar{x}\left[f(0,y) < op > g(0,y)\right]$$

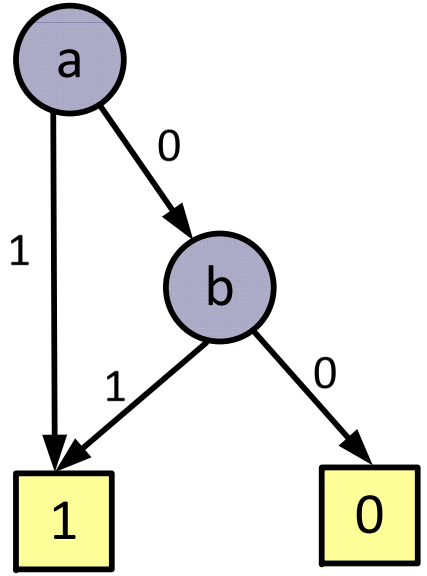


Pick a variable x and decompose both f and g with respect to it.
Then the 0-cofactor is $f_{\bar{x}} < op > g_{\bar{x}}$ and the 1-cofactor is $f_x < op > g_x$

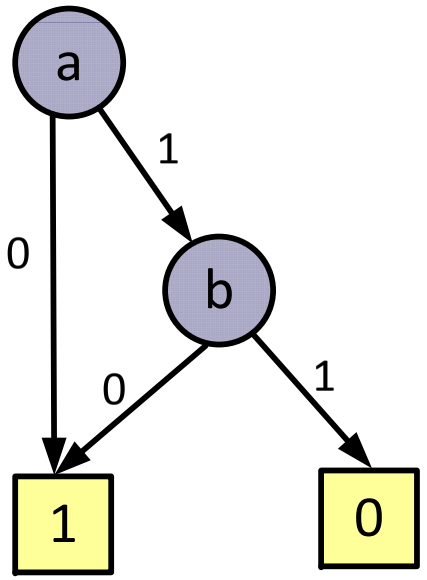
BDD of f <op> g

❖ **Example:** find f.g?

$f = a + b$

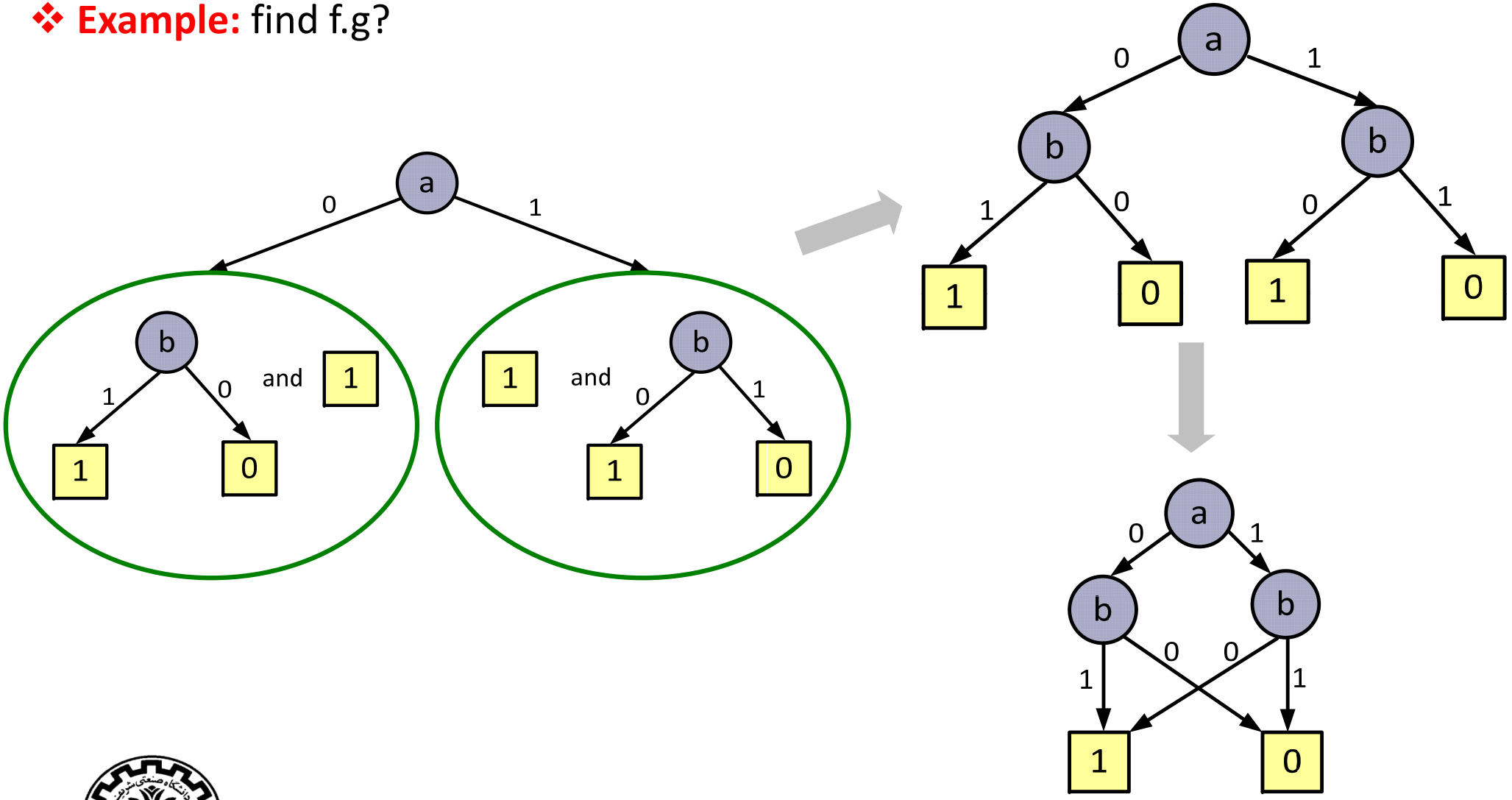


$g = \overline{ab}$



BDD of f . g

❖ **Example:** find f.g?



© M. Shabany, ASIC/FPGA Chip Design

Boolean Satisfiability: Theory

❑ Boolean Satisfiability: (a.k.a SAT Problem)

- A Boolean expression in a conjunctive normal form (CNF):

$$f(x_1, x_2, \ldots, x_n) = (x_1 + x_2 + x_3)(x_4 + \bar{x}_5 + x_6)(\bar{x}_7 + \bar{x}_1 + x_8)\cdots(\cdots)$$

is satisfiable if and only if there exists a valuation for variables (x_1, x_2, …, x_n) such that f=1.

❖ **Example:** Is the following function satisfiable?

$$f = (x_1 + x_2)\ (x_1 + x_3)\ (\bar{x}_1 + x_3)(\bar{x}_1 + \bar{x}_3)$$

Try: x_1=1 ⇨ 1 1 x_3=1 x_3=0 ✗ Conflict!

Try: x_1=0 ⇨ x_2=1 x_3=1 1 1 ✓ Satisfiable!

"f" is satisfiable.

Boolean Satisfiability: Theory

- **2-SAT:**
  - Boolean satisfiability problem where each sum term consists of no more than 2 literals
- **3-SAT:**
  - Boolean satisfiability problem where each sum term consists of no more than 3 literals
- If there are n variables in "f", (i.e., $(x_1, x_2, \ldots, x_n)$), we have to search over 2^n possible cases to verify satisfiability!

  NP-complete problem, i.e., non-deterministic polynomial time complete

- Is there any better way?
  - **YES** using dynamic programming algorithm based on Implication graphs.

Boolean Satisfiability: Implication Graph

❑ Implication Graph:

- ➢ An implication graph consists of
  - **Nodes:** which represent a variable and its assignments
  - **Edges:** which indicate an implication

❖ Example:

$$f = (\bar{a} + b + c)(a + c + d)(a + c + \bar{d})(a + \bar{c} + d)(a + \bar{c} + \bar{d})(\bar{b} + \bar{c} + d)(\bar{a} + b + \bar{c})(\bar{a} + \bar{b} + c)$$

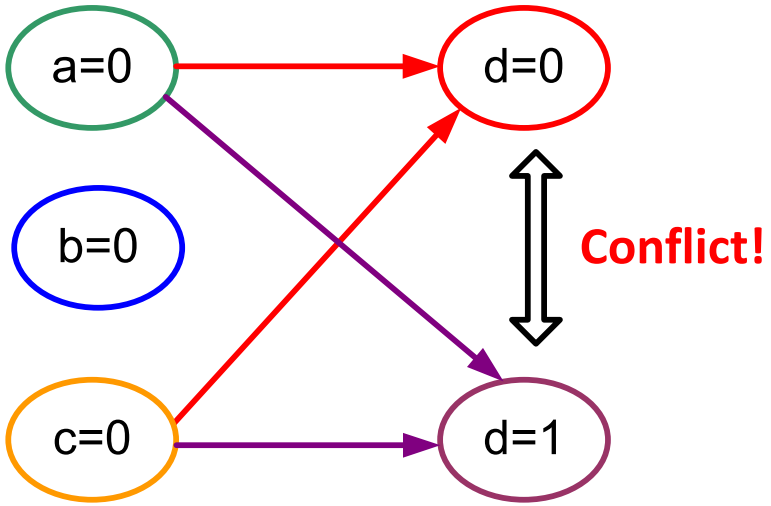
✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓

Satisfiable!
(SAT)

Boolean Satisfiability: Implication Graph

❑ **Implication Graph:**

$$f = (\bar{a} + b + c)(a + c + d)(a + c + \bar{d})(a + \bar{c} + d)(a + \bar{c} + \bar{d})(\bar{b} + \bar{c} + d)(\bar{a} + b + \bar{c})(\bar{a} + \bar{b} + c)$$

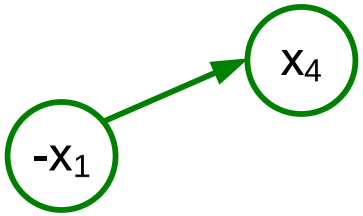


Boolean Satisfiability: Implication Graph

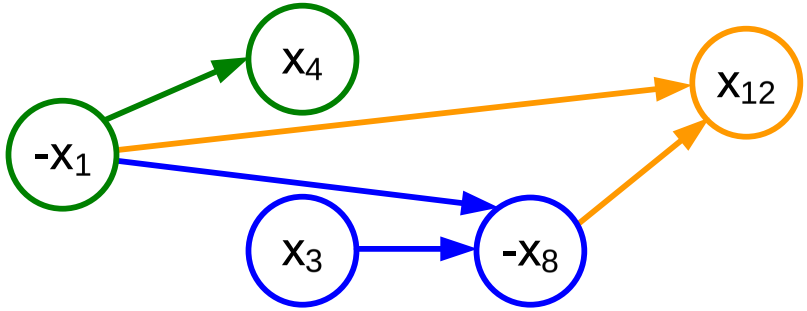
$$f = (x_1 + x_4)(x_1 + \bar{x}_3 + \bar{x}_8)(x_1 + x_8 + x_{12})(x_2 + x_{11})(\bar{x}_7 + \bar{x}_3 + x_9)(\bar{x}_7 + x_8 + \bar{x}_9)(x_7 + x_8 + \bar{x}_{10})(x_7 + x_{10} + \bar{x}_{12})$$

- **Idea:**
  - Assign a variable such that it simplifies the largest number of clauses (removes literal from a clause)
  - Draw an implication graph

- **Step 1**: $x_1=0 \Rightarrow x_4=1$



- **Step 2**: $x_3=1 \Rightarrow x_8=0$

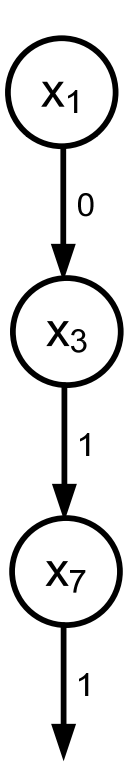


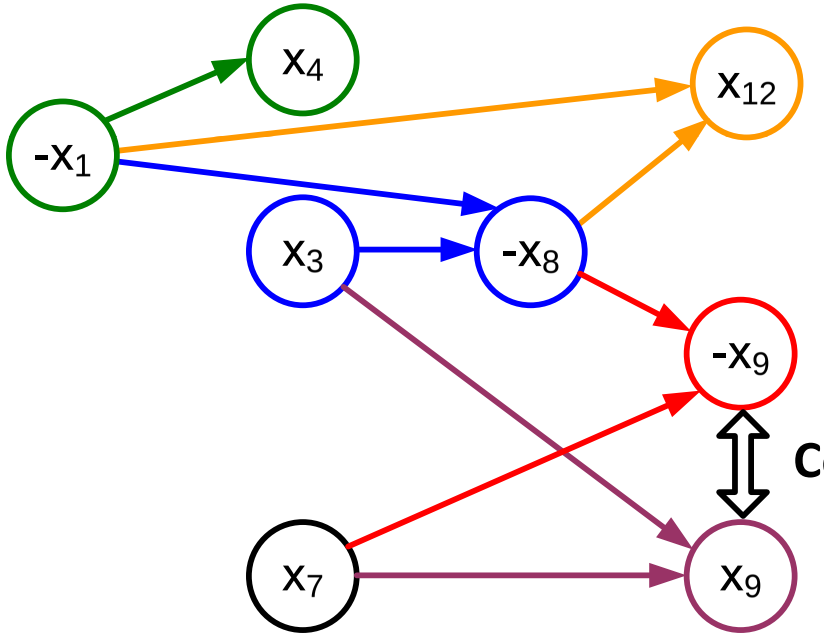
Boolean Satisfiability: Implication Graph

$$f = (x_1 + x_4)(x_1 + \bar{x}_3 + \bar{x}_8)(x_1 + x_8 + x_{12})(x_2 + x_{11})(\bar{x}_7 + \bar{x}_3 + x_9)(\bar{x}_7 + x_8 + \bar{x}_9)(x_7 + x_8 + \bar{x}_{10})(x_7 + x_{10} + \bar{x}_{12})$$

✓ ✓ ✓ ✓ ✓

❑ **Step 3**: $x_7 = 1$





Conflict : $x_7 x_3 \bar{x}_8$

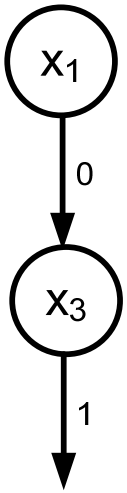
✓ $(x_1 + x_4)$
✓ $(x_1 + \bar{x}_3 + \bar{x}_8)$
✓ $(x_1 + x_8 + x_{12})$
$(x_2 + x_{11})$
✓ $(\bar{x}_7 + \bar{x}_3 + x_9)$
✓ $(\bar{x}_7 + x_8 + \bar{x}_9)$
$(x_7 + x_8 + \bar{x}_{10})$
$(x_7 + x_{10} + \bar{x}_{12})$

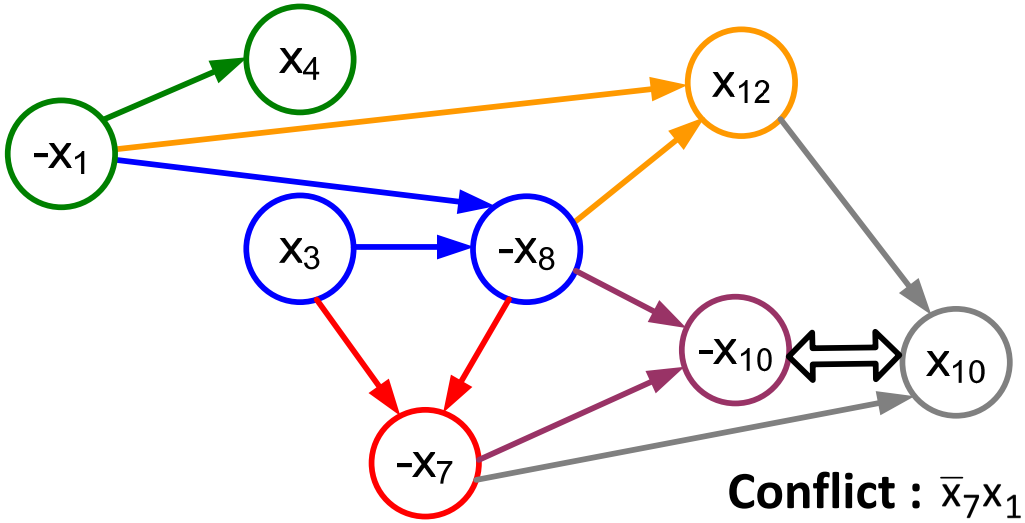
Conflict Clause: $\overline{(x_7 x_3 \bar{x}_8)} = (\bar{x}_7 + \bar{x}_3 + x_8)$

This will be 1 when
$x_7 \neq 1$ or $x_3 \neq 1$ or $x_8 \neq 0$
So does not change the function

Boolean Satisfiability: Implication Graph

- When a conflict clause is found:
  - Add the conflict clause to f
  - Does not the conflict to happen again
  - Backtrack to the point where we first assigned one of the variables in the conflict clause
- So we should go back to step 2 where x_3=1:





Conflict : $\bar{x}_7 x_{12} \bar{x}_8$

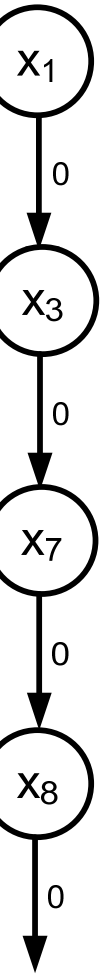
Conflict Clause: $x_7 + \bar{x}_{12} + x_8$

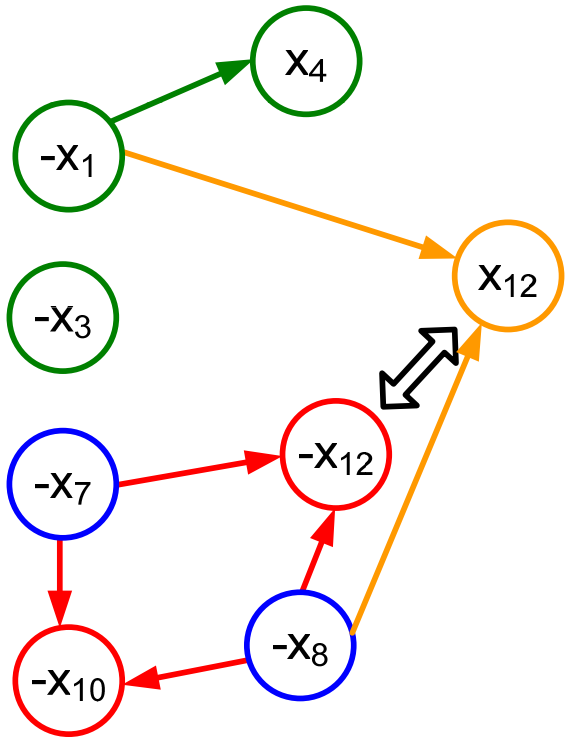
- ✓ $(x_1 + x_4)$
- ✓ $(x_1 + \bar{x}_3 + \bar{x}_8)$
- ✓ $(x_1 + x_8 + x_{12})$
- $(x_2 + x_{11})$
- $(\bar{x}_7 + \bar{x}_3 + x_9)$
- $(\bar{x}_7 + x_8 + \bar{x}_9)$
- ✓ $(x_7 + x_8 + \bar{x}_{10})$
- ✓ $(x_7 + x_{10} + \bar{x}_{12})$
- ----------
- ✓ $(\bar{x}_7 + x_8 + \bar{x}_3)$

© M. Shabany, ASIC/FPGA Chip Design

Boolean Satisfiability: Implication Graph

- Note: no variable that was previously set is present in the conflict clause. Therefore, the current branch is unsatisfiable, so explore the other branch, i.e., x_3=0.
- Also set x_7=0, which does not force anything
- So set x_8=0 then conflict!





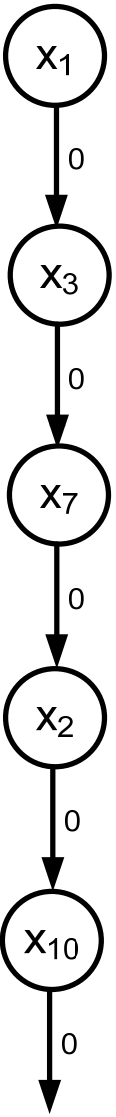
Conflict : $\bar{x}_7\bar{x}_1\bar{x}_8$

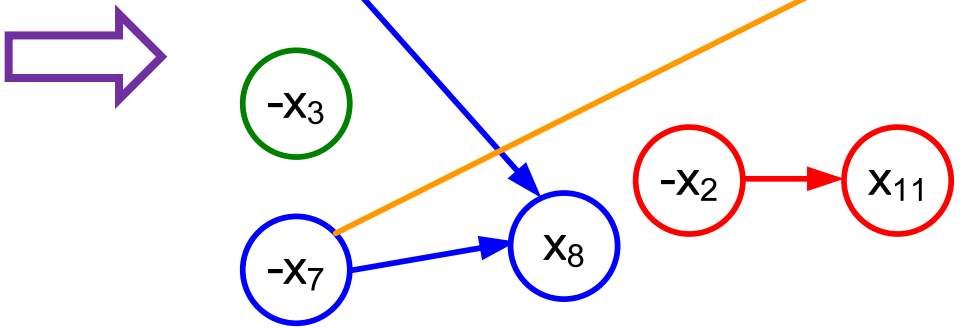
Conflict Clause: $x_7 + x_1 + x_8$

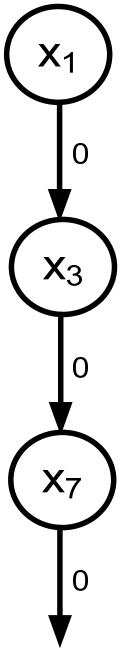
- ✓ $(x_1 + x_4)$
- ✓ $(x_1 + \bar{x}_3 + \bar{x}_8)$
- ✓ $(x_1 + x_8 + x_{12})$
- $(x_2 + x_{11})$
- ✓ $(\bar{x}_7 + \bar{x}_3 + x_9)$
- ✓ $(\bar{x}_7 + x_8 + \bar{x}_9)$
- ✓ $(x_7 + x_8 + \bar{x}_{10})$
- $(x_7 + x_{10} + \bar{x}_{12})$
- ---
- ✓ $(\bar{x}_7 + x_8 + \bar{x}_3)$
- ✓ $(x_7 + x_8 + \bar{x}_{12})$

Boolean Satisfiability: Implication Graph

- Go back to x_1=0, x_3=0, x_7=0.
- Set x_2=0
- Set x_{10}=0







- ✓ $(x_1 + x_4)$
- ✓ $(x_1 + \bar{x}_3 + \bar{x}_8)$
- ✓ $(x_1 + x_8 + x_{12})$
- ✓ $(x_2 + x_{11})$
- ✓ $(\bar{x}_7 + \bar{x}_3 + x_9)$
- ✓ $(\bar{x}_7 + x_8 + \bar{x}_9)$
- ✓ $(x_7 + x_8 + \bar{x}_{10})$
- ✓ $(x_7 + x_{10} + \bar{x}_{12})$

- ✓ $(\bar{x}_7 + x_8 + \bar{x}_3)$
- ✓ $(x_7 + x_8 + \bar{x}_{12})$
- ✓ $(x_7 + x_8 + x_1)$

Satisfiable!
(SAT)

We explored 12 cases of 2^{12}=4096 possible cases!

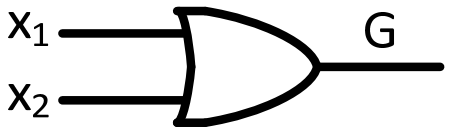
© M. Shabany, ASIC/FPGA Chip Design

Boolean Satisfiability: SAT Solvers

- Go back to our question.
- Given a logic structure can we implement a given logic expression within it?

- **Characteristic Equation (CE):**
  - A logical expression that takes as inputs all literals of a function as well as its output, and it produces a "1" iff for a given set of inputs the output is correct

❖ Example: Dose G = f ?



x_1	x_2	f
0	0	0
0	1	1
1	0	1
1	1	1

x_1	x_2	G	F_{OR}
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

G follows the OR-gate behavior

Means G should not be "0" when x_1=1 & x_2=0

Characteristic Equation

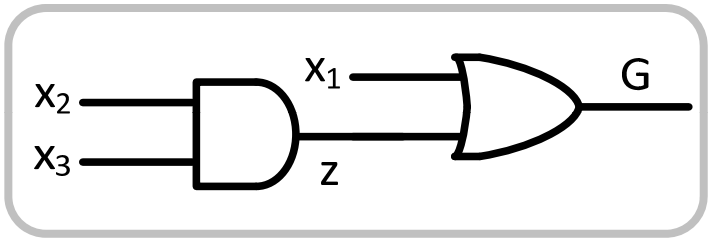
$$F = (\bar{x}_2 + G)(\bar{x}_1 + G)(x_1 + x_2 + \bar{G})$$

If we substitute (x_1+x_2) for G then F=1
Thus, F is SAT so G implements an OR function

Boolean Satisfiability: SAT Solvers

- **Characteristic Equation (CE) for a Logic Network:**
- **Divide-and-Conquer**
  - Create a CE for each gate
  - Combine CEs for gates to form a CE for the network

x_2	x_3	z	F_{AND}
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



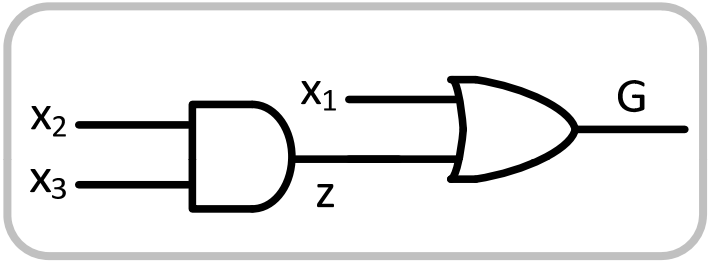
$$F_{OR} = (\bar{z} + G)(\bar{x}_1 + G)(x_1 + z + \bar{G})$$

$$F_{AND} = (x_3 + \bar{z})(x_2 + \bar{z})(\bar{x}_3 + \bar{x}_2 + z)$$

$$F = F_{OR} \cdot F_{AND} = (x_3 + \bar{z})(x_2 + \bar{z})(\bar{x}_3 + \bar{x}_2 + z)(\bar{z} + G)(\bar{x}_1 + G)(x_1 + z + \bar{G})$$

Boolean Satisfiability: SAT Solvers

- **SAT Theorem:**
  - In order for a given logic network, represented by a characteristic equation F, to be equivalent to a logic function H, then for all x_is, F(G=H) must be SAT.



❖ **Example:** Assume H = x_1+x_2, find out if G = H ?

- This question is equivalent to see if F(G= x_1+x_2) is SAT for all x_1,x_2 ,x_3 values?

$$F\left(G\Big|_{x_1+x_2}\right)=(x_3+\bar{z})(x_2+\bar{z})(\bar{x}_3+\bar{x}_2+z)(\bar{z}+x_1+x_2)(\bar{x}_1+(x_1+x_2))(x_1+z+\overline{(x_1+x_2)})$$

$$x_3=0 \Longrightarrow F=x_1+\bar{x}_1\bar{x}_2$$ (Not SAT for all x_1, x_2 values)

So we can NOT implement an OR function in F

SAT Theorem

- **SAT Theorem:**

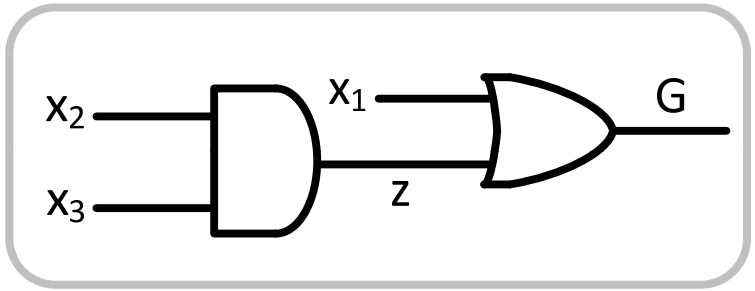
Characteristic Equation **F** represents **H** if and only if:

$$\forall i, \forall j, \forall x_i \; \exists z_j \; F(G = H) : SAT$$

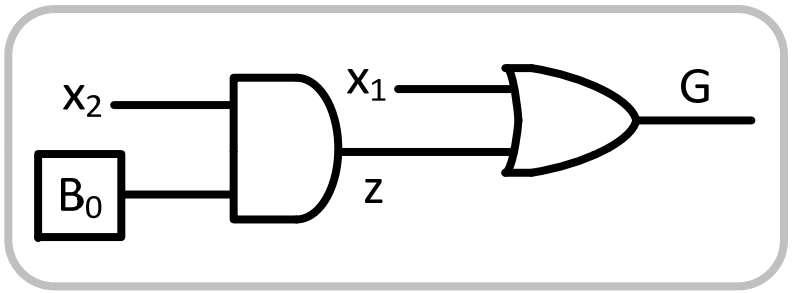
- x_i : Literals
- z_j : Internal wires

Quantified SAT (QSAT) Problem:

SAT Problem



QSAT Problem



Question:

Can **F(G)** implement **H** for all x_1,x_2 ,x_3 values?

Question:

Can **F(G)** implement **$H=x_1+x_2$** for all x_1,x_2 values by adjusting B_0?

Quantified SAT (QSAT) Problem:

- **Step 1**: Generate CE for F(G):

$$F = (B_0 + \bar{z})(x_2 + \bar{z})(\bar{B}_0 + \bar{x}_2 + z)(\bar{z} + G)(\bar{x}_1 + G)(x_1 + z + \bar{G})$$

- **Step 2**: For each valuation of x_1, x_2, determine what B_0 should be in order for G to implement H

  - i) $F_{\bar{x}_1\bar{x}_2}(G) = (B_0 + \bar{z})(\bar{z})(1)(\bar{z} + G)(1)(z + \bar{G})$

    $H(x_1 = 0, x_2 = 0) = 0 \Rightarrow G = 0 \Rightarrow z = 0 \Rightarrow$ B_0 : don't care

  - ii) $F_{\bar{x}_1 x_2} = (B_0 + \bar{z})(1)(\bar{B}_0 + z)(\bar{z} + G)(1)(z + \bar{G})$

    $H(x_1 = 0, x_2 = 1) = 1 \Rightarrow G = 1 \Rightarrow z = 1 \Rightarrow B_0 = 1$

Quantified SAT (QSAT) Problem:

- **Step 2, Cont'd:**
  - iii) $F_{x_1\bar{x}_2} = (B_0 + \bar{z})(\bar{z})(1)(\bar{z} + G)(G)(1)$

    $H(x_1 = 1, x_2 = 0) = 1 \rightarrow G = 1 \rightarrow z = 0 \rightarrow$ B_0 : don't care

  - iv) $F_{x_1x_2}(G) = (B_0 + \bar{z})(\overline{B}_0 + z)(1)(\bar{z} + G)(G)(1)$

    $H(x_1 = 1, x_2 = 1) = 1 \rightarrow G = 1 \rightarrow z = 1 \rightarrow B_0 = 1 \rightarrow$ **SAT**

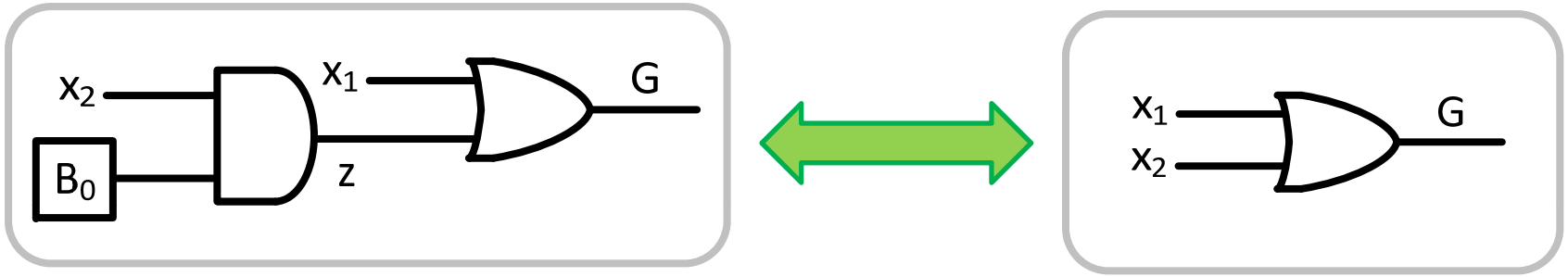
    $z = 0 \rightarrow B_0 = 0$

    Contradicts the results from $F_{\bar{x}_1x_2}$

All Fs are SAT and B_0=1 is the consistent solution

Quantified SAT (QSAT) Problem:

QSAT Problem



Question:

Can **F(G)** implement **$H=x_1+x_2$** for all x_1,x_2 values by adjusting B_0?

Answer:

YES for $B_0=1$

Boolean Satisfiability: SAT Solvers

- Can we solve this problem with a single equation?
  - Yes, solve SAT for

$$F = F_{\bar{x}_1\bar{x}_2} \cdot F_{\bar{x}_1 x_2} \cdot F_{x_1\bar{x}_2} \cdot F_{x_1 x_2}$$

  - G is replaced with $H(x_1, x_2)$
  - Note that signal z should be replicated as
    - z_1 for $F_{\bar{x}_1\bar{x}_2}$
    - z_2 for $F_{\bar{x}_1 x_2}$
    - z_3 for $F_{x_1\bar{x}_2}$
    - z_4 for $F_{x_1 x_2}$
  - This is b/c z is an intermediate variable and based on the above calculations z results in a conflict
  - The choice of B_0=1 will be resolved due to conflict clauses

QSAT Theorem

- **QSAT Theorem:**

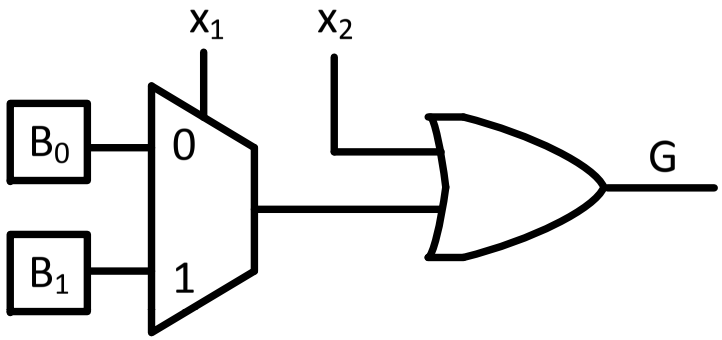
Characteristic Equation **F** represents **H** if and only if:

$$\forall i, \forall j, \forall k, \exists B_k \forall x_i \exists z_j \, F(G = H) : SAT$$

- x_i : Literals
- z_j : Internal wires

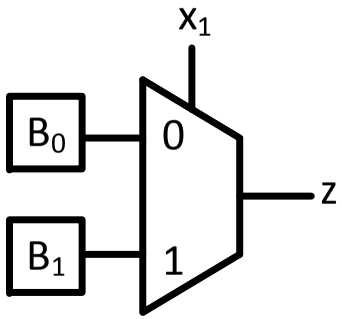
QSAT Theorem

- Can we implement function $H = x_1 + x_2$ into:



- We are looking for a value for B_0 and a value for B_1

- **Step 1**: Represent G as a logic expression. This expression is the characteristic equation



QSAT Theorem

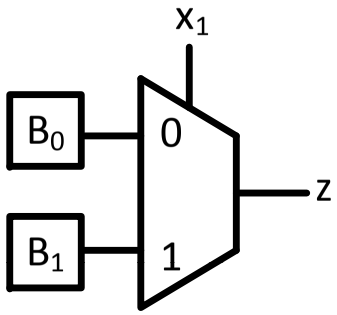
❑ **Step 1**: Represent MUX as a logic expression. This expression is the characteristic equation of the MUX

I) $B_0 + x_1 + \overline{z}$

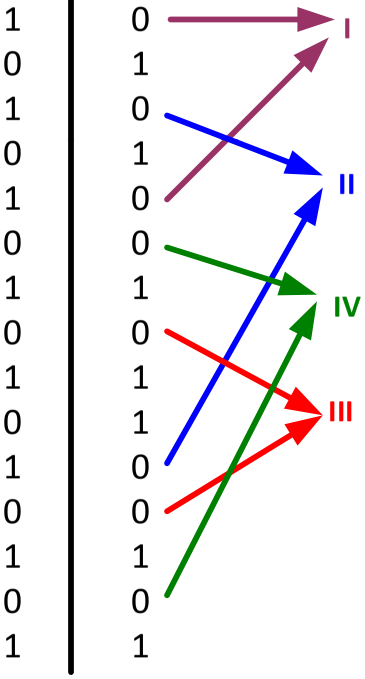
II) $B_1 + \overline{x}_1 + \overline{z}$

III) $\overline{B}_0 + x_1 + z$

IV) $\overline{B}_1 + \overline{x}_1 + z$



B_0	B_1	x_1	z	F_{MUX}
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

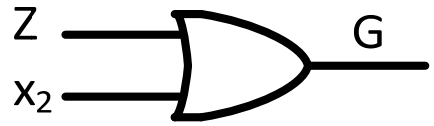


$$F_{MUX} = (\overline{B}_0 + x_1 + \overline{z})(B_1 + \overline{x}_1 + z)(\overline{B}_0 + x_1 + z)(\overline{B}_1 + \overline{x}_1 + z)$$

Boolean Satisfiability: SAT Solvers

$$F_{OR} = (\overline{x}_2 + G)(\overline{z} + G)(z + x_2 + \overline{G})$$

x_2	Z	G	F_{OR}
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



- Thus the complete characteristic equation for the logic structure is:

$$F = (\overline{B}_0 + x_1 + \overline{z})(B_1 + \overline{x}_1 + z)(\overline{B}_0 + x_1 + z)(\overline{B}_1 + \overline{x}_1 + z)(x_2 + z + \overline{G})(\overline{x}_2 + G)(\overline{z} + G)$$

- **Step2:** Take F and see if a given logic expression can be implemented in it. This means that we need to find a value for B_0 and B_1 such that for all x_1, x_2 values, B_0 and B_1 are consistent.

Quantified SAT (QSAT) Problem:

❑ **Step 2**: For each valuation of x_1, x_2, determine what B_0 and B_1 should be in order for G to implement $H = x_1 + x_2$

➢ i) $F_{\bar{x}_1\bar{x}_2}(G) = (B_0 + \bar{z}_1)(1)(\bar{B}_0 + z_1)(1)(z_1 + \bar{G})(1)(\bar{z}_1 + G)$

G has to be equal to H, i.e., 0

$G = 0 \Rightarrow z_1 = 0 \Rightarrow B_0 = 0 \quad H = 0$ ✓

➢ ii) $F_{x_1\bar{x}_2} = (1)(B_1 + \bar{z}_2)(1)(\bar{B}_1 + z_2)(z_2 + \bar{G})(1)(\bar{z}_2 + G)$

G has to be equal to H, i.e., 1

$G = 1 \Rightarrow z_2 = 1 \Rightarrow B_1 = 1 \quad H = 1$ ✓

Quantified SAT (QSAT) Problem:

- iii) $F_{\bar{x}_1 x_2}(G) = (B_0 + \bar{z}_3)(1)(\bar{B}_0 + z_3)(1)(1)(G)(\bar{z}_3 + G)$

$H = 1 \rightarrow G = 1 \rightarrow z_3 = 0 \rightarrow B_0 = 0$ ✓

- iv) $F_{x_1 x_2} = (1)(B_1 + \bar{z}_4)(1)(\bar{B}_1 + z_4)(1)(G)(\bar{z}_4 + G)$

$H = 1 \rightarrow G = 1 \rightarrow$

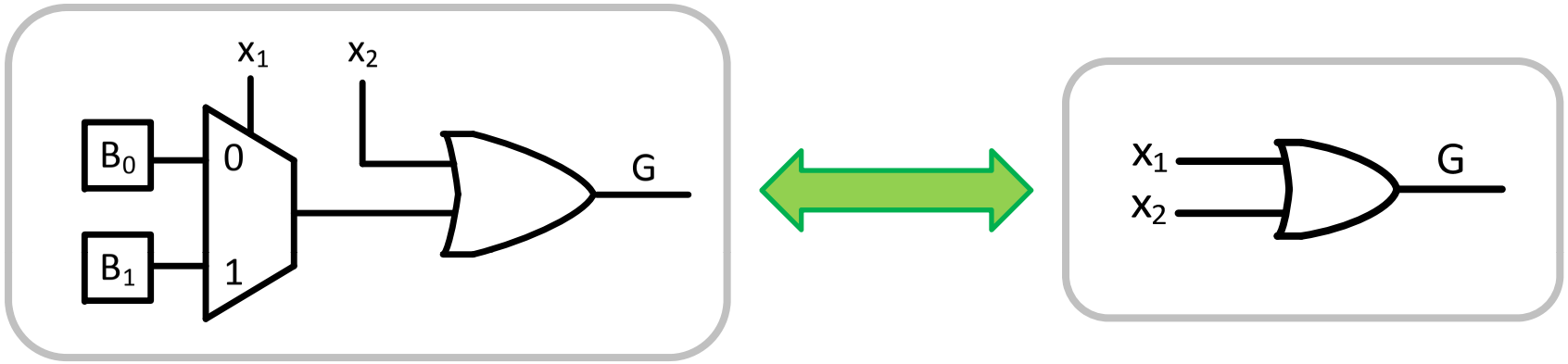
$B_1 = 1 \rightarrow z_4 = 1$ ✓

$B_1 = 0 \rightarrow z_4 = 0$ ✗

No surprise as when x_1=1 then z_4 has to be equal to B_1

Find a consistent value: $B_1 = 1$ and $B_0 = 0$

Quantified SAT (QSAT) Problem:



Question:

Can **F(G)** implement **$H=x_1+x_2$** for all x_1,x_2 values by adjusting B_0?

Answer:

YES for $B_0=0$ and $B_1=1$

Outline

- Introduction to Synthesis
- Digital Logic Basics
- Logic Optimization
  - Two-level logic synthesis
  - Multi-level logic synthesis
- **Technology Mapping**
  - Boolean Satisfiability
  - ASIC/FPGA-oriented Technology Mapping

Logic Synthesis

- Logic synthesis normally in two steps:
  - **Technology independent**
    - Manipulate equations
    - Optimize the logic equations
    - Independent of target IC media
  - **Technology dependent (Technology Mapping (TM))**
    - Equations are turned into netlist of the available gates

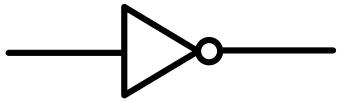
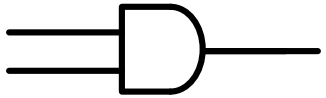
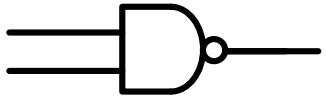
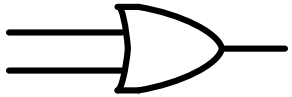
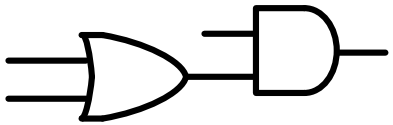
Technology Mapping

- **Problem Definition:**
  - **Given:**
    1. Boolean network G(v,e), where
       $v \in V$: represent logic functions
       $e \in E$: represent dependencies between logic functions
    2. Library of available gates

  - **Find:**
    - Netlist of gates from library that implements logic function G so as to minimize some of the following metrics:
      - Area
      - Delay
      - Power
      - Defect

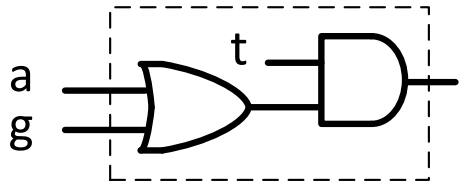
Technology Mapping (TM)

❖ Example: Implement function y=t(a+g) given the following library

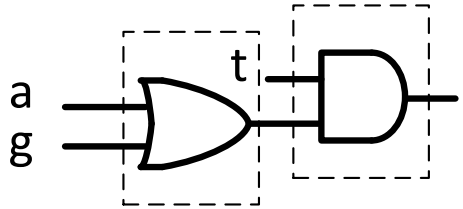
Library Element	Area (sq units)	Delay (nsec)
INV 	3	1 ns
AND 	8	2.5 ns
NAND 	6	2 ns
OR 	8	2.5 ns
AND-OR 	25	4 ns

Technology Mapping (TM)

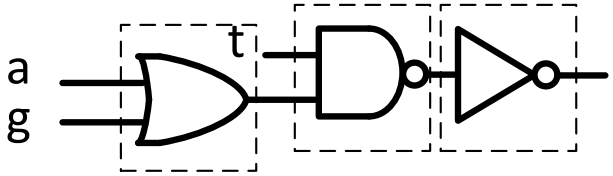
- Possible Implementations:



Area	Delay	
25	4	**Best Delay**



Area	Delay	
16	5	**Best Area**



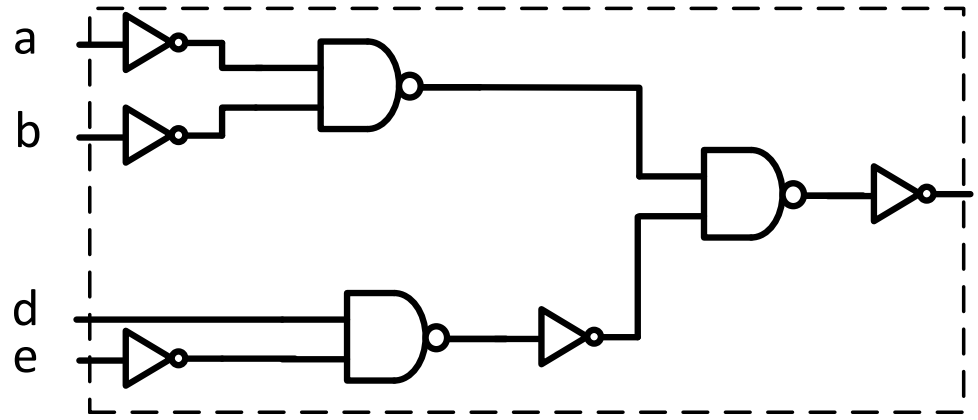
Area	Delay
17	5.5

- This is a small function with small library ➡ 3 choices
- Larger designs + libraries ➡ many more choices

Technology Mapping (TM)

- Algorithmic approach to TM (DAGON Keutzer 80s)
  1. Take input Boolean network and do a simple mapping into a network of "**base functions**" to create a "**subject graph**"
     e.g., 2-input NANDs + inverters

$$F = d\bar{e}(a+b)$$
$$= d\bar{e}\cdot\overline{(\bar{a}\,\bar{b})}$$

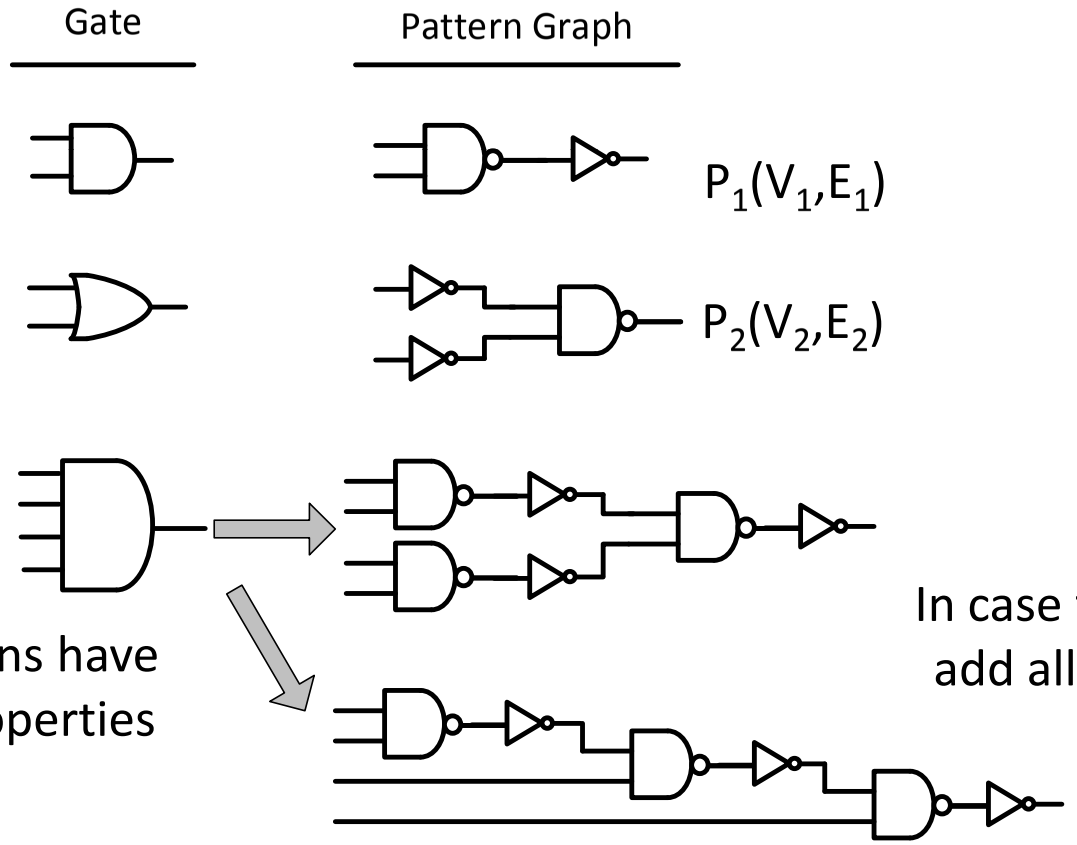


"Subject Network"

G(V,E)

Technology Mapping (TM)

- Algorithmic approach to TM (DAGON Keutzer 80s)
  2. Take gates in library and create a "**pattern graph**" for each that is functionally equivalent and expressed in the same base functions



Different implementations have different area/delay properties

In case there are alternatives, add all of the pattern graphs

Technology Mapping (TM)

- Now we have subject graph G(V,E) and set of pattern graphs $P_1(V_1,E_1)$, $P_2(V_2,E_2)$,…, $P_L(V_L,E_L)$ with their area and delay properties

- We have to cover G(V,E) with the set of pattern graphs that results in a minimum cost

- Cost: area (sum of area of all gates), delay (max delay along any path),…

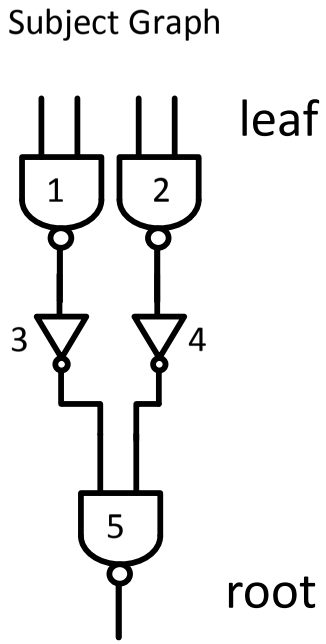
- Find the minimum-cost cover is equivalent to dynamic programming (DP)
- **DP Idea:**
  - Break the overall problem into sub-problems
  - Solve sub-problems optimally (store results in a table)
  - Use solutions to sub-problems, construct solution to the overall problem

Technology Mapping (TM)

- **DP for TM: (Work Backward)**
  - Begin at leafs of tree and find optimal mapping of each leaf node
  - Move up tree find optimal mappings for sub-trees using already-computed mappings

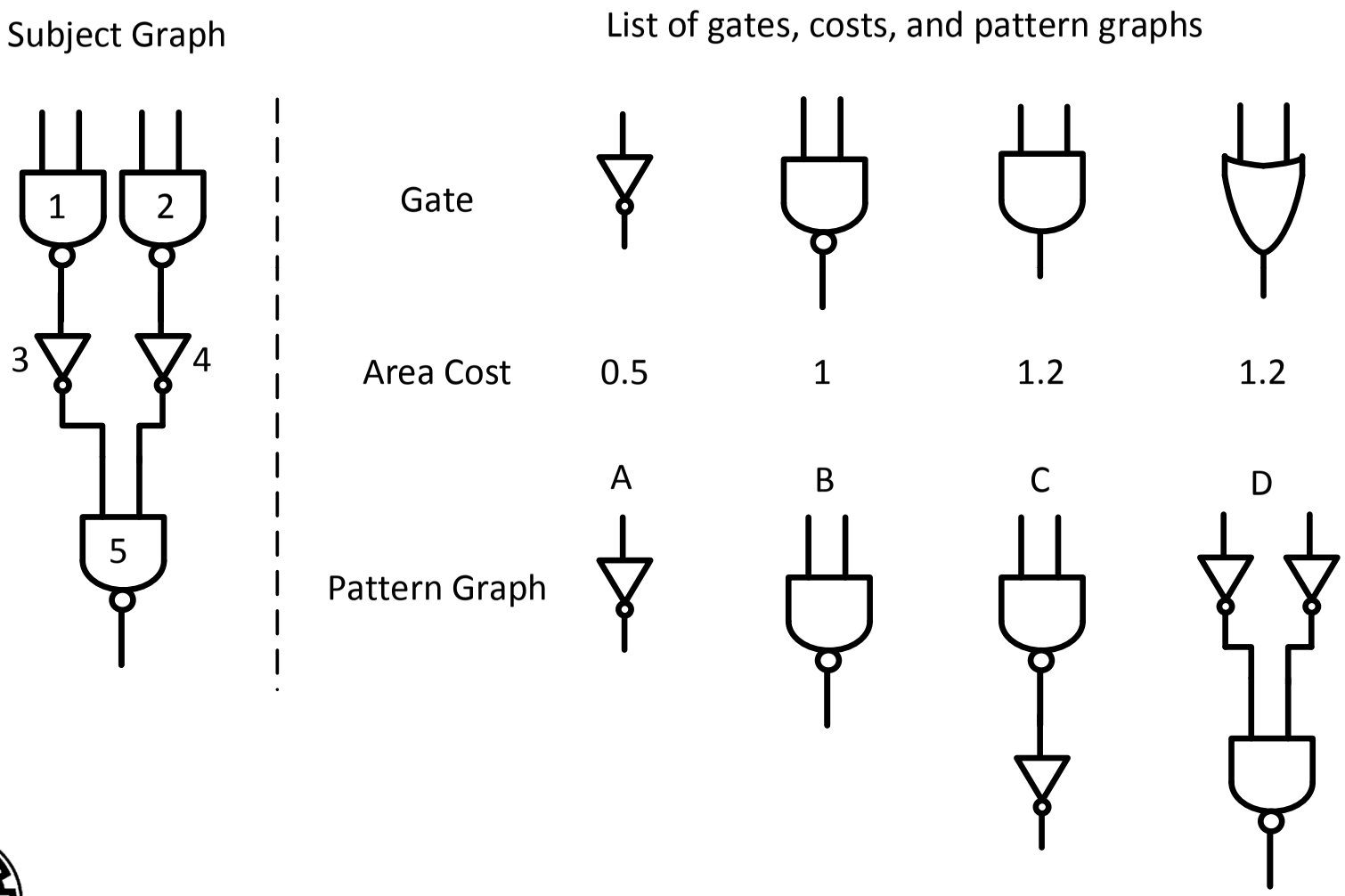
$$F = \bar{a} + \bar{b} + \bar{c} + \bar{d}$$
$$= \overline{ab} + \overline{cd}$$
$$= \overline{\overline{ab} \cdot \overline{cd}}$$

Create subject graph using base functions



Technology Mapping (TM)

❑ DP for TM: (Work Backward)



© M. Shabany, ASIC/FPGA Chip Design

Technology Mapping (TM)

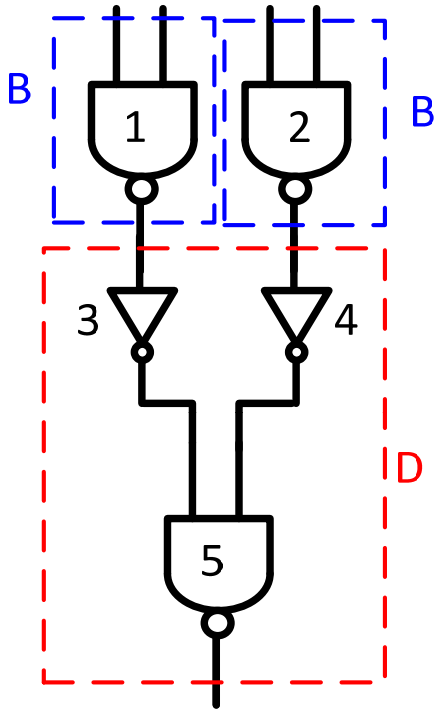
- Begin at leafs, traverse all nodes
  - Find all pattern graphs that match at each node (match the entire pattern graph all the way back (with trace back))
  - Record the best total cost to implement sub-tree rooted at node

1) B matches, cost = 1 → record this match + cost
2) B matches, cost = 1 → record this match + cost
3) A matches, cost = 0.5 +1 (cost shared for node 1)
   C matches, cost = 1.2 → record this match + cost
4) Same as node 3
5) B matches, cost = 1 + 1.2 + 1.2 =3.4
   D matches, cost = 1.2 + 1 + 1 = 3.2 → Best for node 5

Solution: D for node 5, B for nodes 1 and 2 (Optimal Mapping)

Technology Mapping (TM)

❑ **Optimal Solution:**



❑ **Key DP point:**

- If we later need to implement the output of 3 explicitly (as the input to another gate), we need only consider using C for node 3 (DP result).

Technology Mapping (TM)

- **Summary:**
  - Post-order tree traversal (from leaves to the root)
  - For each node, find all the pattern graphs that match with trace back
  - Choose the one with the best cumulative cost
  - At root of tree, have optimal cost, trace back to construct mapping
  - Can also be used for delay (carry forward the longest path delay)

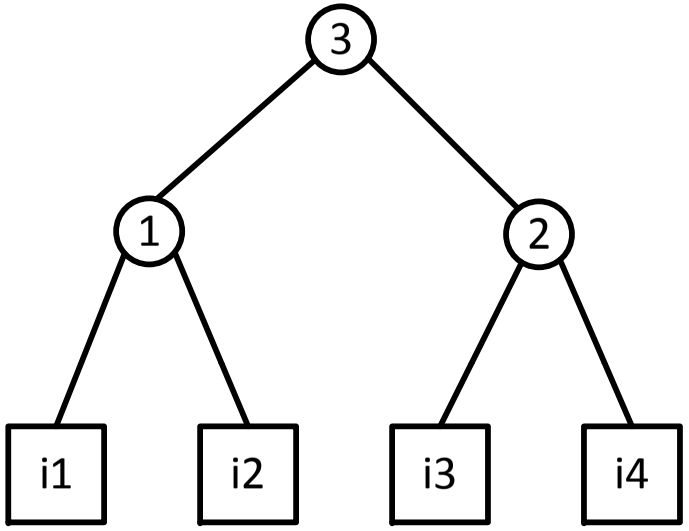
FPGA Technology Mapping (TM)

- **Target gates:** LUTs with K-inputs that can implement any function with K inputs
- Subject graph need not be in base functions
- Nodes must have less than K inputs (K-bounded network)
- Same bottom-up DP TM algorithm
  - Matching is different
  - Only need to care about # of inputs not functionality

FPGA Technology Mapping (TM)

❖ **Example:**

- ➢ Mapping to 3-LUTs (K=3)



❑ What are matches at node 3?

- ➢ Each match is a cut
- ➢ Cuts of nodes 3:
  - ▪ {i1, i2, 2}
  - ▪ {i3, i4, 1}
  - ▪ {1,2}

❑ Finding all matches at a node = finding set of K-feasible cuts for that node

A cut is K-feasible if its size is less than K

ASIC/FPGA Chip Design

Power Dissipation

Mahdi Shabany

Department of Electrical Engineering

Sharif University of technology

Outline

- Introduction
- Dynamic Power Dissipation
- Static Power Dissipation

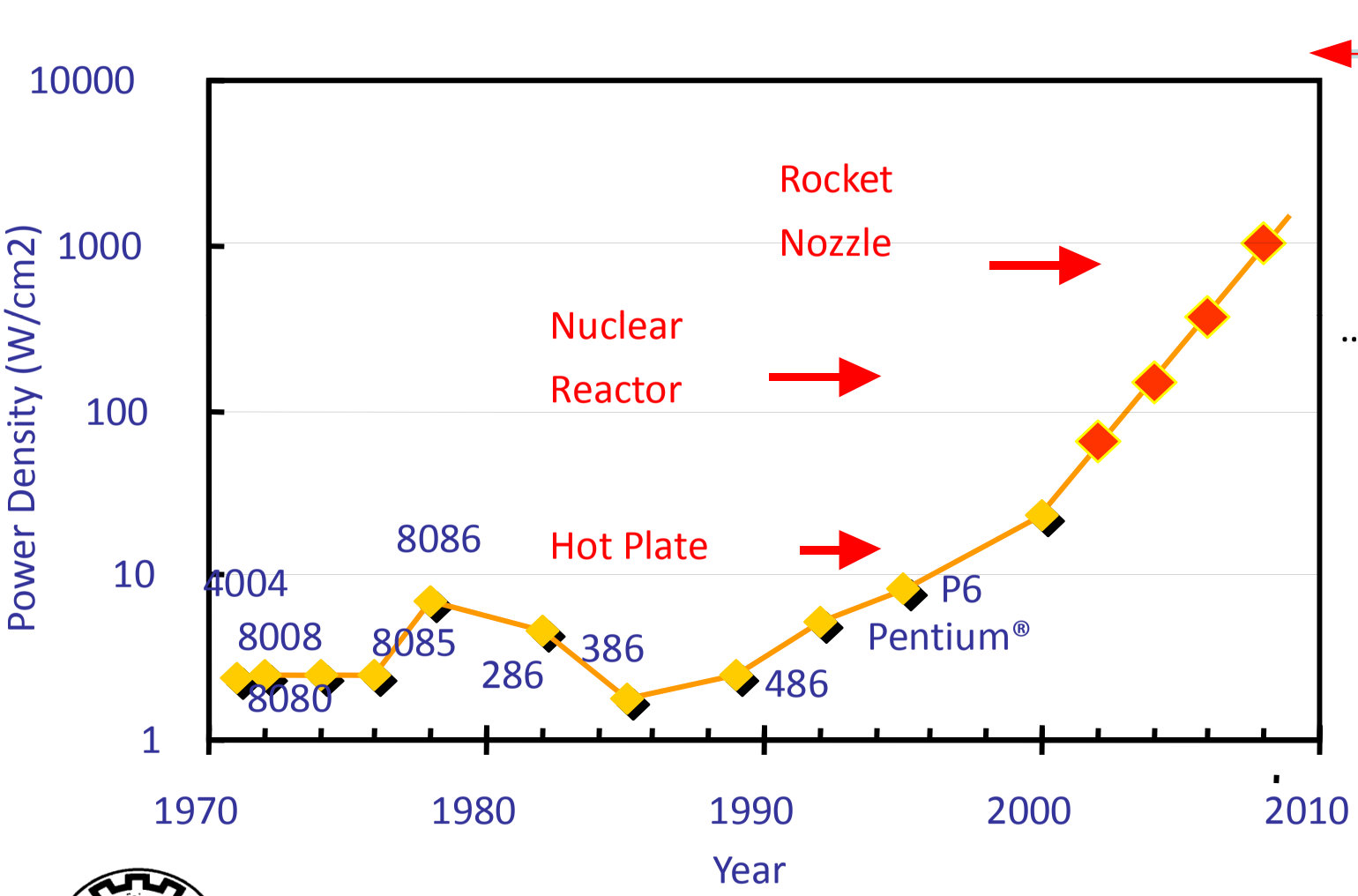
Outline

- Introduction
- Dynamic Power Dissipation
- Static Power Dissipation

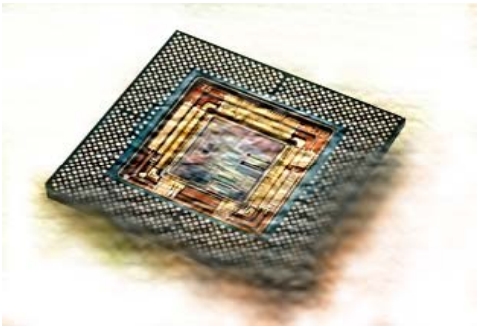
Why Power Matters?

- Packaging costs
- Power supply rail design
- Chip and system cooling costs
- Noise immunity and system reliability
- Battery life (in portable systems)
- Environmental concerns

Why worry about power? Chip Power Density



…chips might become hot…

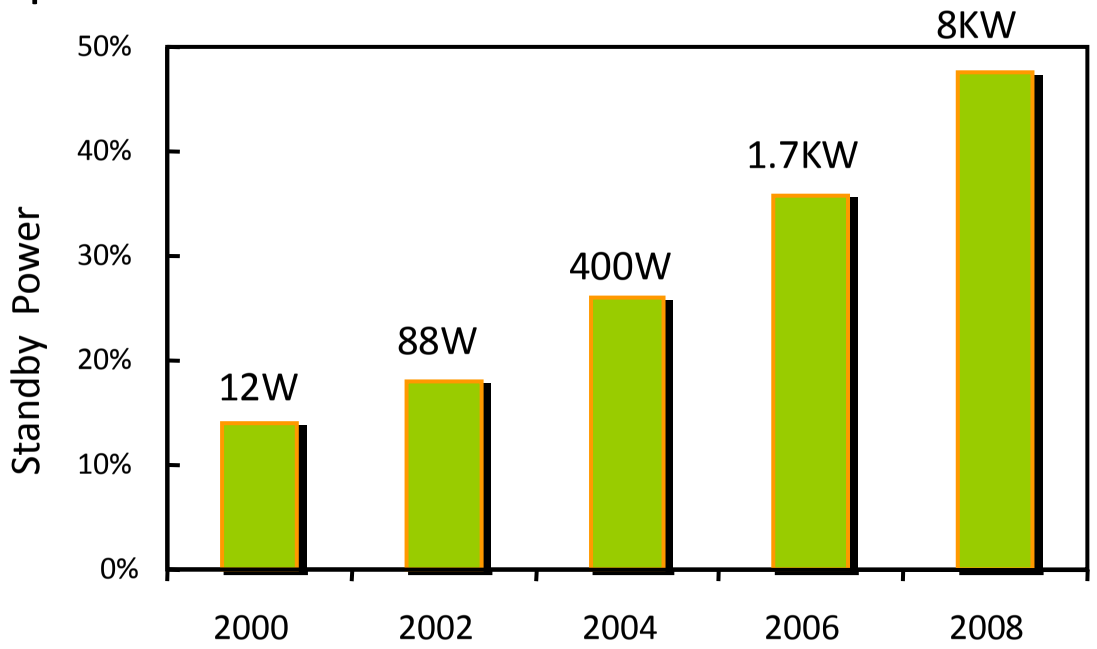


Source: Borkar, De Intel®

Why worry about power? Standby Power

Year	2002	2005	2008	2011	2014
Power supply V_{dd} (V)	1.5	1.2	0.9	0.7	0.6
Threshold V_T (V)	0.4	0.4	0.35	0.3	0.25

- Drain leakage will increase as V_T decreases to maintain noise margins and meet frequency demands, leading to excessive battery draining standby power consumption.



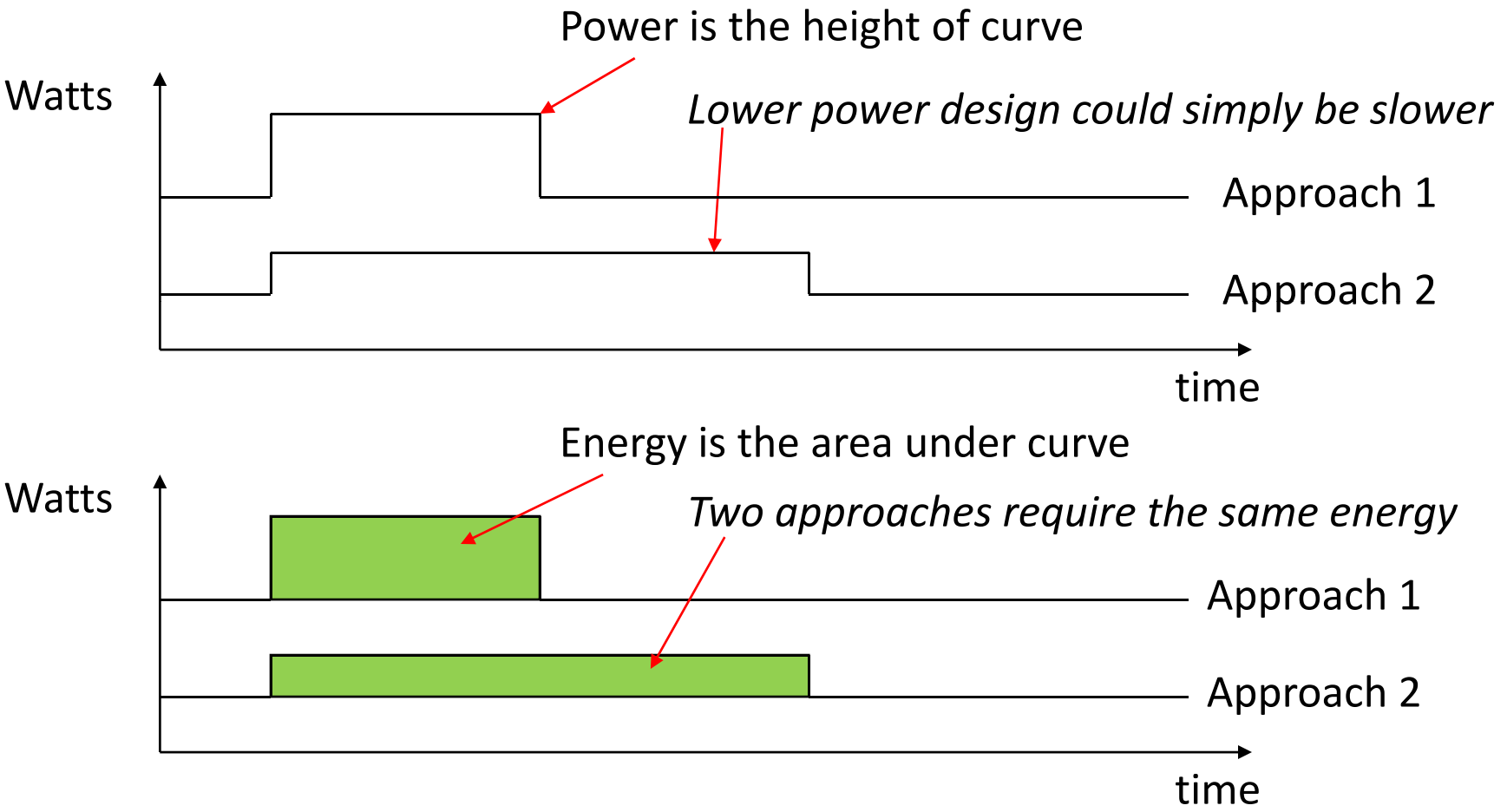
© M. Shabany, ASIC/FPGA Chip Design

Power and Energy Figures of Merit

- Power consumption in Watts
  - Determines battery life in hours
- Peak power
  - Determines power ground wiring designs
  - Sets packaging limits
  - Impacts signal noise margin and reliability analysis
- Energy efficiency in Joules
  - Rate at which power is consumed over time
- Energy = power * delay
  - Joules = Watts * seconds
  - Lower energy number means less power to perform a computation at the same frequency

Power is the rate at which energy is delivered or exchanged; Power dissipation is the rate at which energy is taken from the source and converted into heat

Power vs. Energy



Power and Energy

- Power is drawn from a voltage source attached to the V_{DD} pin(s) of a chip.

- Instantaneous Power: $$P(t) = I(t)V(t)$$

- Energy: $$E = \int_0^T P(t)dt$$

- Average Power: $$P_{\text{avg}} = \frac{E}{T} = \frac{1}{T}\int_0^T P(t)dt$$

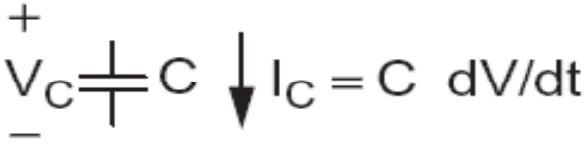
Power in Circuit Elements

$$P_{VDD}(t) = I_{DD}(t) V_{DD}$$

$$P_R(t) = \frac{V_R^2(t)}{R} = I_R^2(t) R$$

$$E_C = \int_0^{\infty} I(t) V(t)\, dt = \int_0^{\infty} C \frac{dV}{dt} V(t)\, dt$$

$$= C \int_0^{V_C} V(t)\, dV = \tfrac{1}{2} C V_C^2$$



© M. Shabany, ASIC/FPGA Chip Design

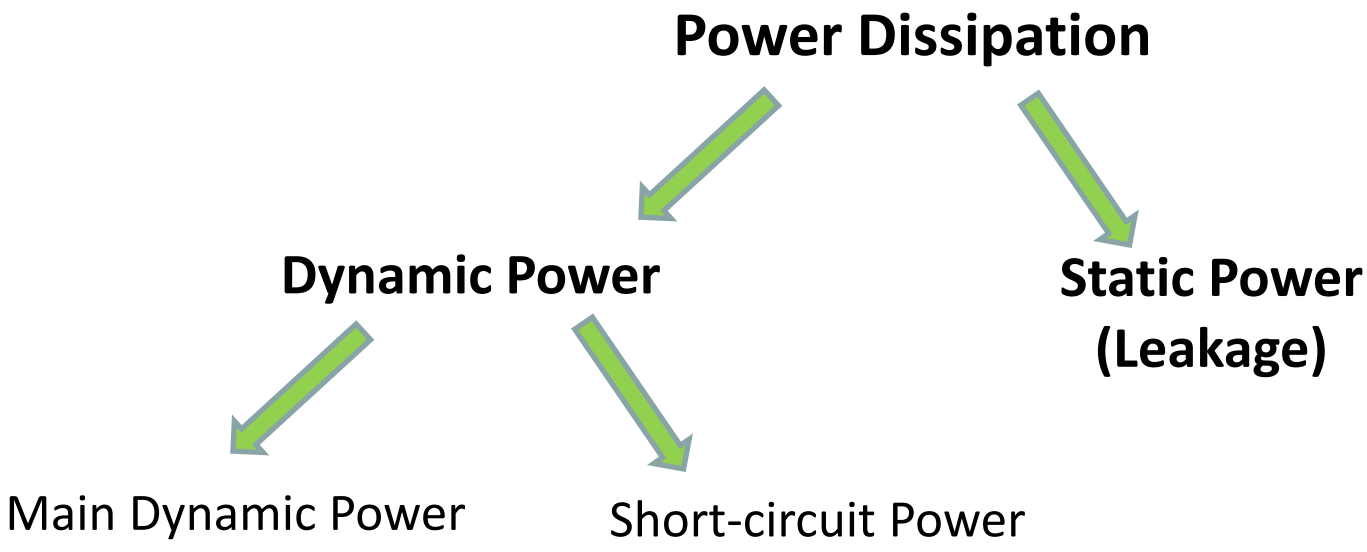
Power Dissipation

- **Power:** Due to the current flowing from supply to ground

$$P = I_D V_{DD}$$

- **Power Dissipation:**
  - **Dynamic Power :** Occurs only when the gate switches
    - Charging/discharging of load capacitances
    - Short-circuit power during switching (when both NMOS and PMOS are ON)
  - **Static Power**: Due to the presence of a path in the gate b/w the power supply & GND
    - In CMOS, when circuit is quiescent (no switching) one of the transistors is OFF thus ideally no current flows through an OFF transistor so no current b/w VDD and GND thus zero static power

Power Dissipation



CMOS Total Energy & Power Equations

$$E = C_L V_{DD}^2 \alpha_{0\to1} + \alpha_{sc} V_{DD}^2 C_L + V_{DD} I_{static}$$

$$f_{0\to1} = \alpha_{0\to1} * f_{Clk}$$

$$P = C_L V_{DD}^2 f_{0\to1} + \alpha_{sc} V_{DD}^2 C_L f_{Clk} + V_{DD} I_{static}$$

Dynamic Power (~90% today and decreasing relatively)

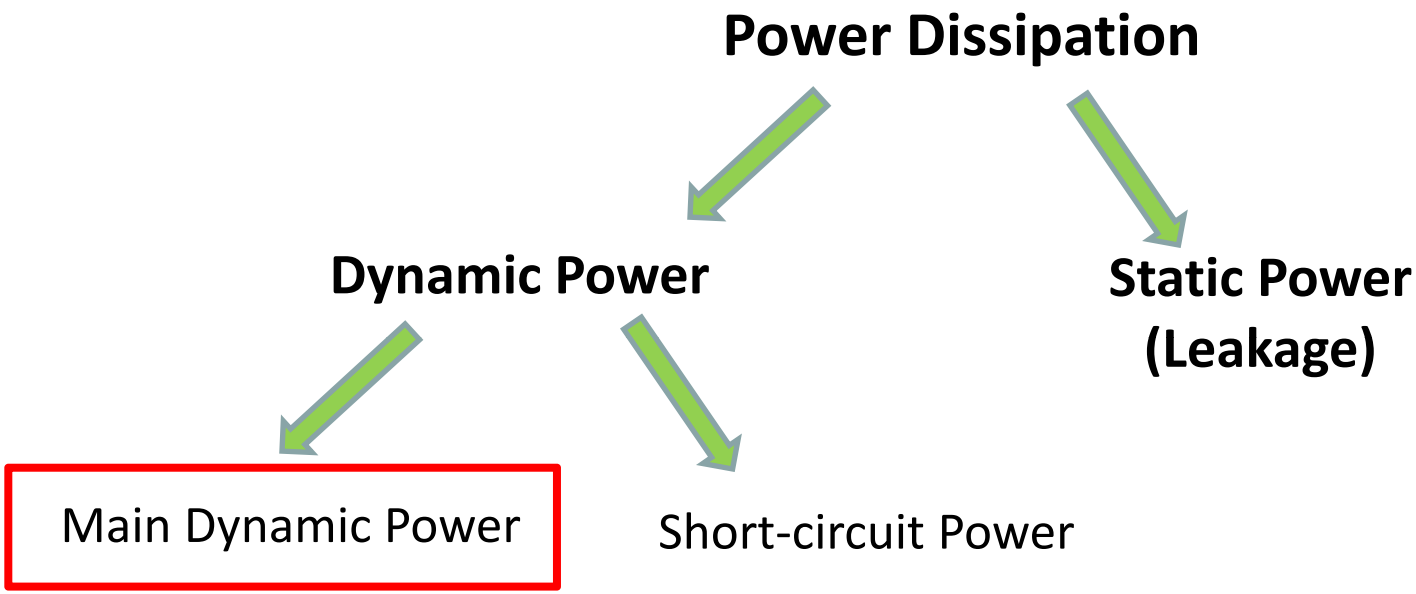
Short-circuit Power (~8% today and decreasing absolutely)

Static Power (~2% today and increasing)

Outline

- Introduction
- Dynamic Power Dissipation
- Static Power Dissipation

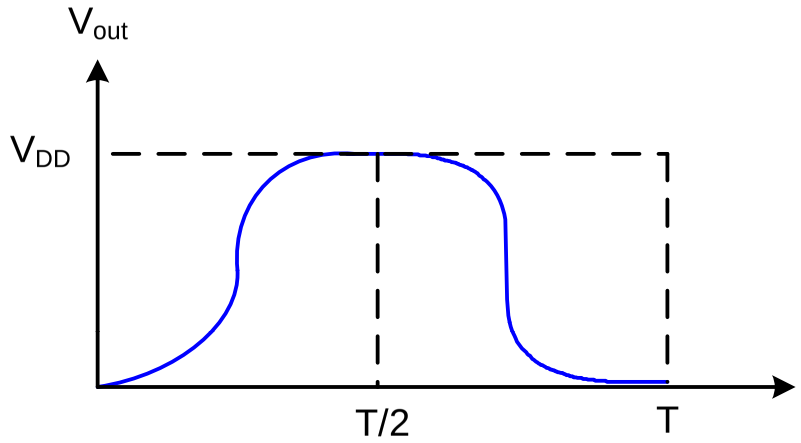
Power Dissipation



Power Dissipation: Main Dynamic Power

- Due to the charging/discharging the load capacitances

$$P_{avg} = \frac{1}{T}\int_0^T v(t)i(t)dt$$



$$P_{avg} = \frac{1}{T}\left[\int_0^{T/2}(V_{out})C_L\frac{dV_{out}}{dt}dt + \int_{T/2}^{T}V_{out}(-C_L\frac{dV_{out}}{dt})dt\right]$$

$$\Rightarrow P_{avg} = \frac{1}{T}\left[\left(\frac{V_{out}^2}{2}C_L\right)_0^{T/2} - \left(\frac{V_{out}^2}{2}C_L\right)_{T/2}^{T}\right] = \frac{1}{T}V_{DD}^2C_L = C_LV_{DD}^2f_{Clk}$$

Power Dissipation: Dynamic Power

- Average Dynamic Power:
  - Linearly dependent to f_{Clk} (Clock frequency)
  - Independent of the transistor sizing

- Considering the utilization factor:

$$P_{avg} = \alpha C_L V_{DD}^2 f_{Clk} \qquad \alpha : \text{Activity Factor}$$

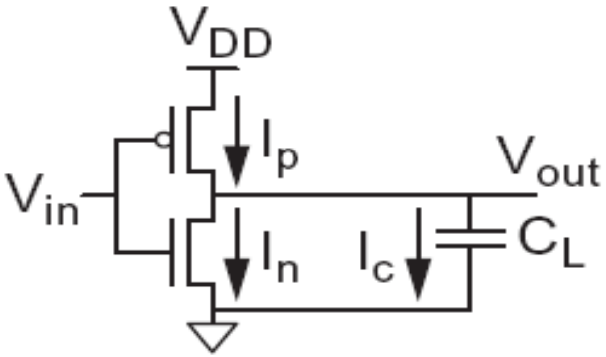
- In general, a chip with higher area burns more power unless its utilization factor is lower

- Power Delay Product: (dissipated as heat in transistors)

$$PDP = C_L V_{DD}^2$$

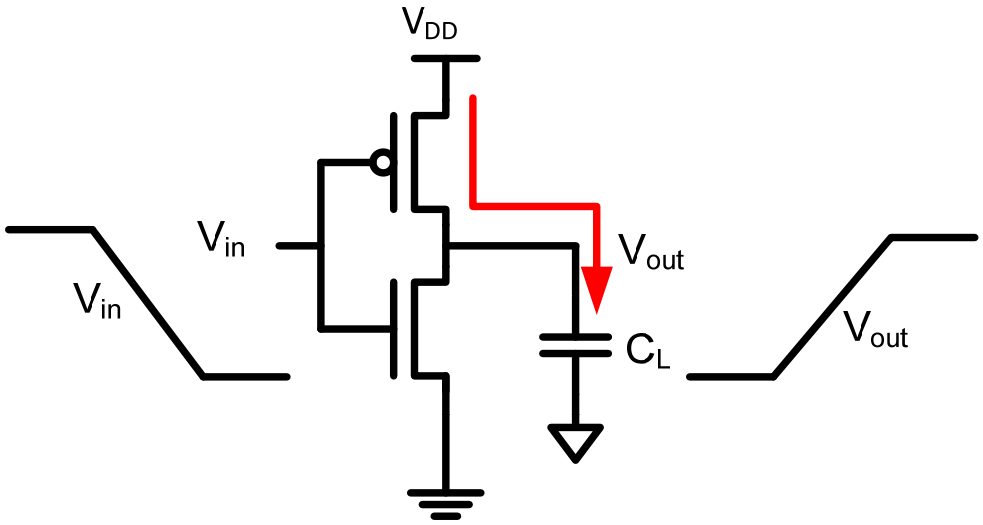
Charging a Capacitor

- When the gate output rises
  - Energy stored in capacitor is
    $$E_C = \tfrac{1}{2} C_L V_{DD}^2$$
  - But energy drawn from the supply is
    $$E_{VDD} = \int_0^\infty I(t) V_{DD}\, dt = \int_0^\infty C_L \frac{dV}{dt} V_{DD}\, dt$$
    $$= C_L V_{DD} \int_0^{V_{DD}} dV = C_L V_{DD}^2$$
  - Half the energy from V_{DD} is dissipated in the PMOS transistor as heat, other half stored in the capacitor



- When the gate output falls
  - Energy in capacitor is dumped to GND
  - Dissipated as heat in the NMOS transistor

Dynamic Power Consumption



$$\text{Energy/transition} = C_L * V_{DD}^2 * \alpha_{0\to1}$$

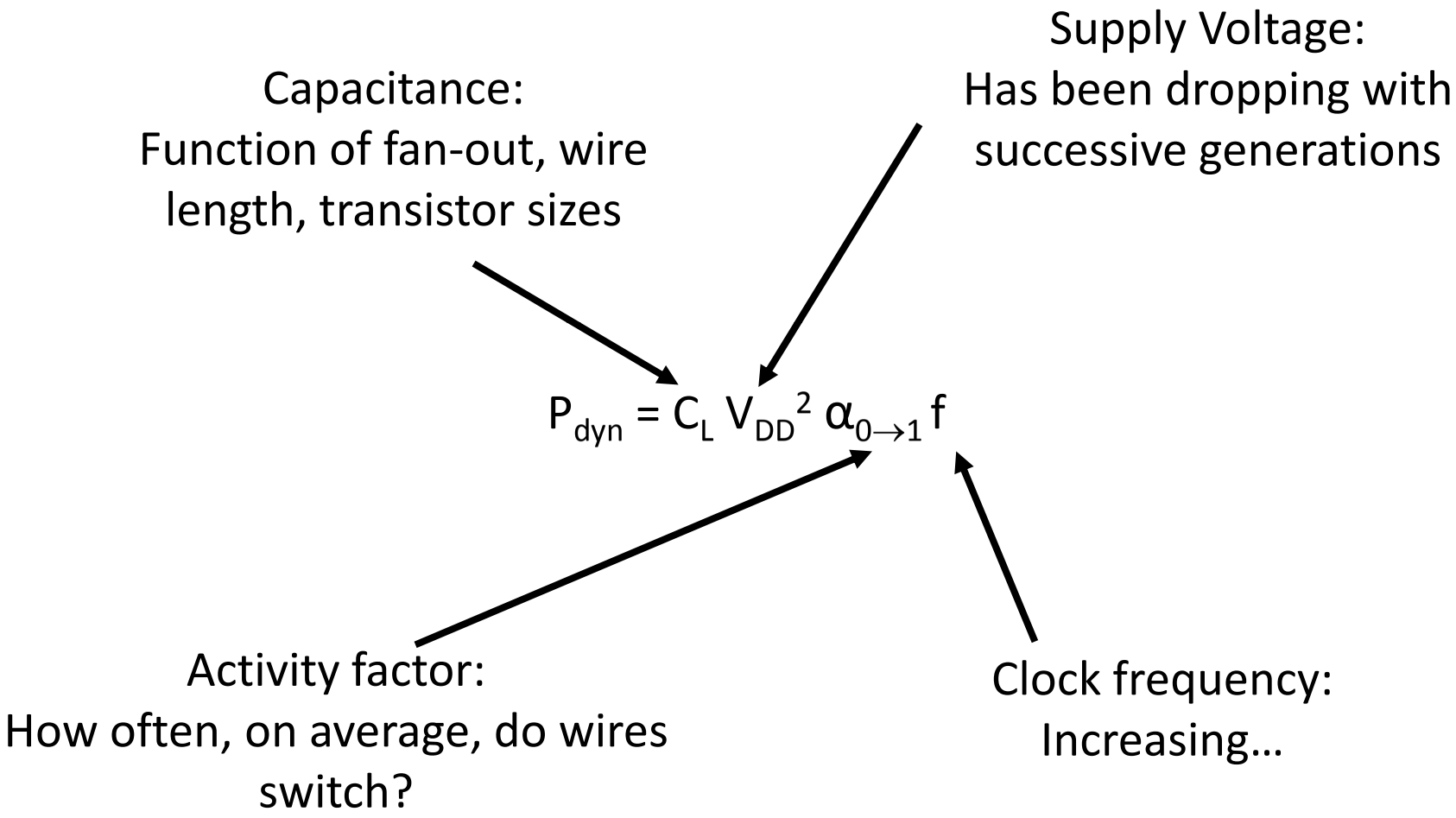
$f_{0\to1}$

$$P_{dyn} = \text{Energy/transition} * f = C_L * V_{DD}^2 * \alpha_{0\to1} * f$$

$$P_{dyn} = C_{eff} * V_{DD}^2 * f \quad \text{where } C_{eff} = \alpha_{0\to1} C_L$$

- Not a function of transistor sizes!
- Data dependent - a function of switching activity!

Lowering Dynamic Power



Lowering Dynamic Power

- Try to minimize:
  - Activity factor
  - Capacitance
  - Supply voltage
  - Frequency

$$P_{dyn} = C_L V_{DD}^2 \alpha_{0\rightarrow1} f$$

Lowering Dynamic Power – Activity Factor

❑ Probability that output is "zero" in one cycle and will be "one" in the next cycle

$$\alpha_{0\to1} = P_0 P_1 = \frac{N_0}{2^N}\frac{N_1}{2^N} = \frac{N_0(2^N - N_0)}{2^{2N}}$$

➢ where

N_0: Number of zero entries in the output column of the function truth table

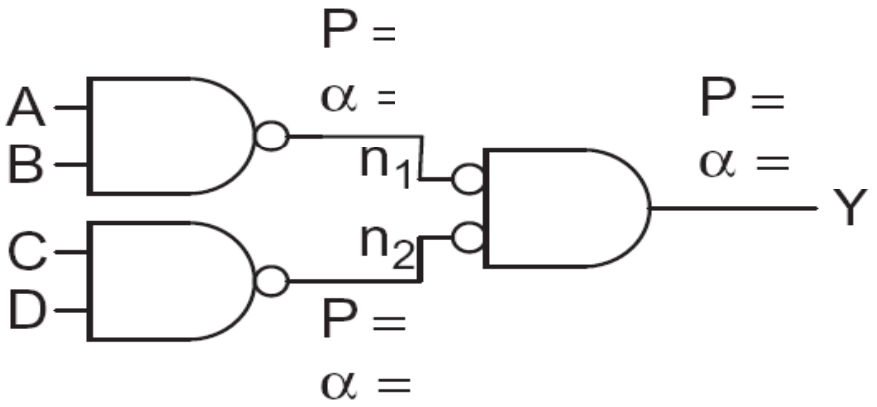
N_1: Number of one entries in the output column of the function truth table

❖ **Example**: A 2-input NOR

$$\alpha_{0\to1} = \frac{3(4-3)}{2^4} = \frac{3}{16} \Rightarrow P_{avg} = \frac{3}{16} C_L V_{DD}^2 f_{Clk}$$

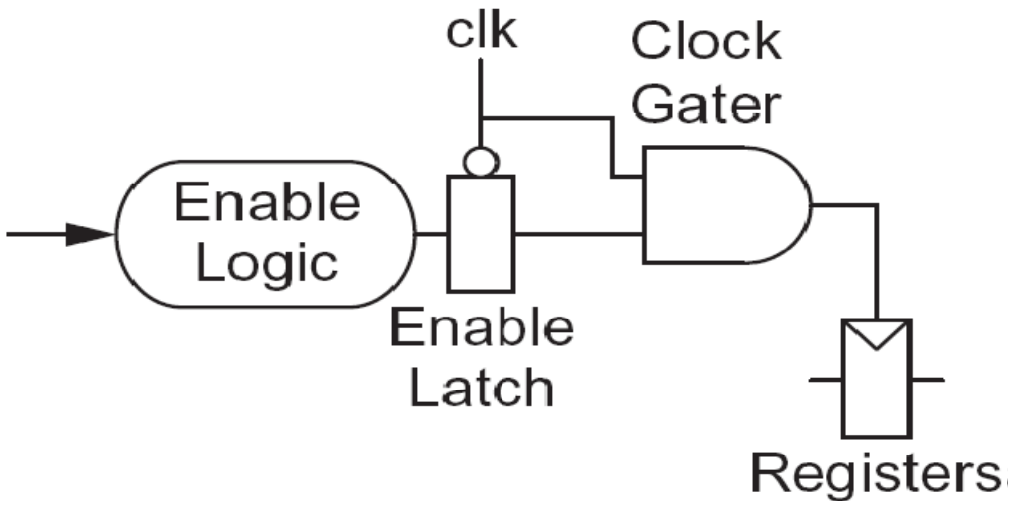
Example

- A 4-input AND is built out of two levels of gates
- Estimate the activity factor at each node if the inputs have P = 0.5



Lowering Dynamic Power: Clock Gating

- The best way to reduce the activity is to turn off the clock to registers in unused blocks
  - Saves clock activity ($\alpha = 1$)
  - Eliminates all switching activity in the block
  - Requires determining if block will be used

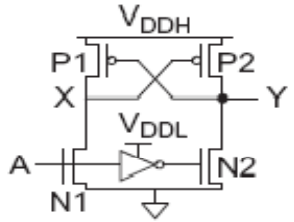


Lowering Dynamic Power: Capacitance

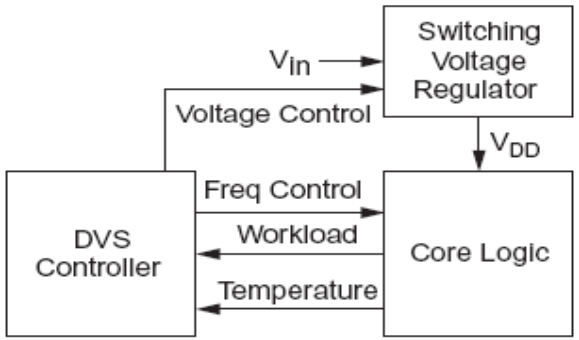
- Gate capacitance
  - Fewer stages of logic
  - Small gate sizes
- Wire capacitance
  - Good floorplanning to keep communicating blocks close to each other
  - Drive long wires with inverters or buffers rather than complex gates

Lowering Dynamic Power: Voltage / Frequency

- Run each block at the lowest possible voltage and frequency that meets performance requirements
- Voltage Domains
  - Provide separate supplies to different blocks
  - Level converters required when crossing from low to high V_{DD} domains



- Dynamic Voltage Scaling
  - Adjust V_{DD} and f according to workload



Power Dissipation: Dynamic Power

- In a digital CMOS circuit:

$$t_p \cdot I_{sat} = C\Delta V$$

$$t_p \cdot K(V_{DD} - V_t)^2 = C\Delta V \Rightarrow t_p \propto \frac{CV_{DD}}{(V_{DD} - V_t)^2} \Rightarrow f_{max} \propto \frac{(V_{DD} - V_t)^2}{V_{DD}} \propto V_{DD}$$

- Therefore, it can be shown that

$$P_{avg} \propto CV_{DD}^2 f \propto V_{DD}^3$$

$$\Rightarrow V_{DD} \uparrow \Rightarrow \text{delay} \downarrow \Rightarrow \text{Power} \uparrow$$

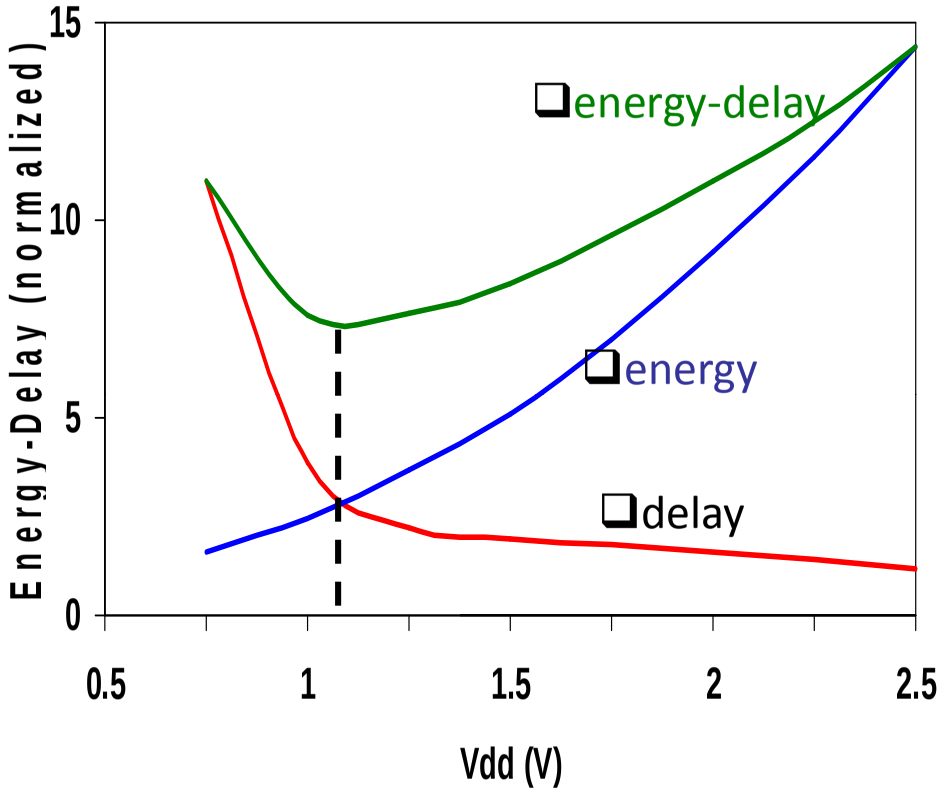
Throughput can be compromised for power

PDP and EDP

- **Power-delay product (PDP)** = $P_{av} * t_p = C_L V_{DD}^2$
  - PDP is the average energy consumed per switching event (Watts * sec = Joule)
  - Lower power design could simply be a slower design
  - For a given structure the PDP may be made arbitrarily low by reducing the supply voltage that comes at the expense of performance.

- **Energy-delay product (EDP)** = $PDP * t_p = P_{av} * t_p^2$
  - EDP is the average energy consumed multiplied by the computation time required
  - Takes into account that one can trade increased delay for lower energy/operation (e.g., via supply voltage scaling that increases delay, but decreases energy consumption)

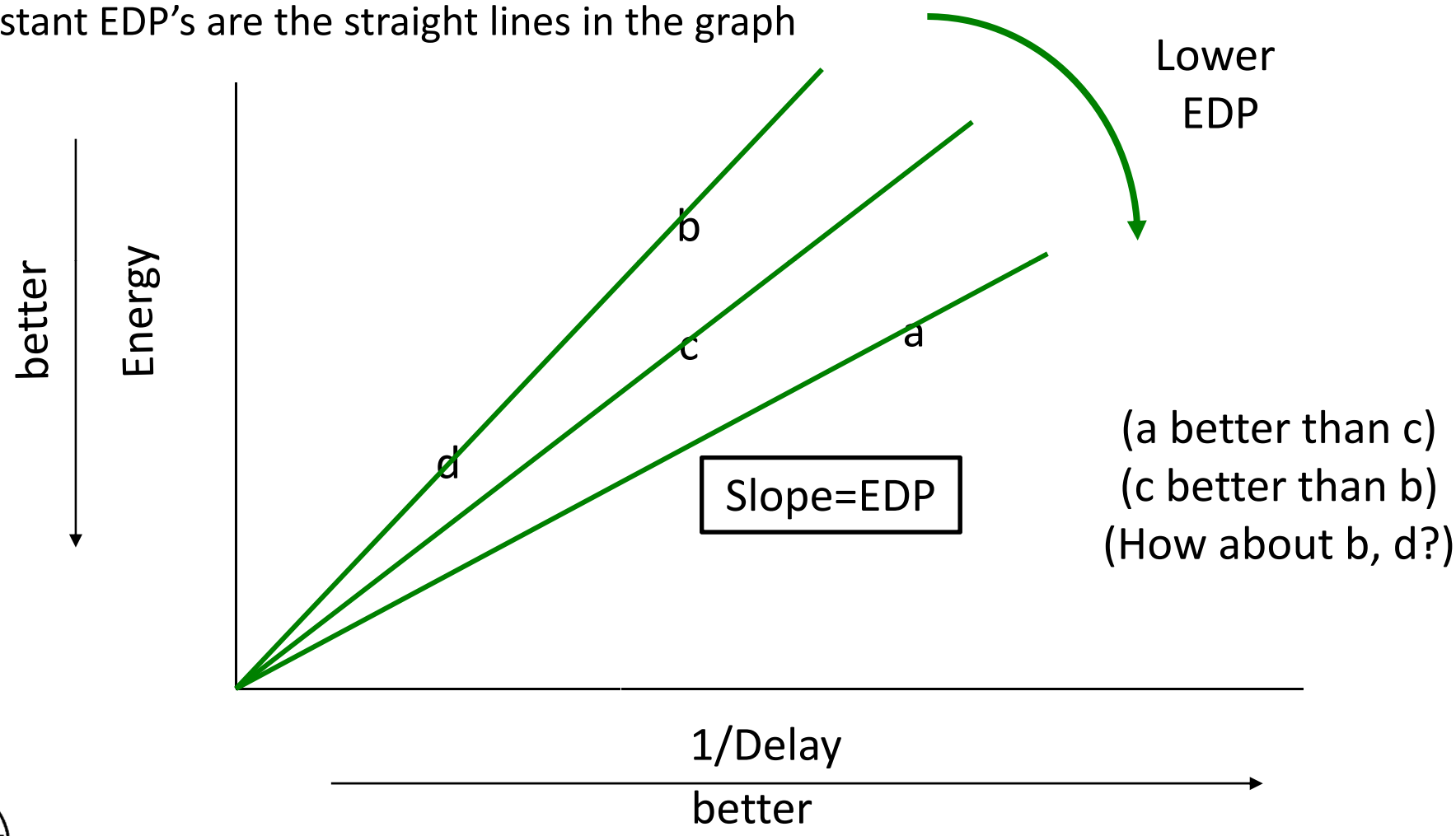
PDP and EDP



Rule-of-thumb: $V_{DD}^{Opt} = \frac{3}{2}\left(V_t + \frac{V_{DS}^{Sat}}{2}\right)$

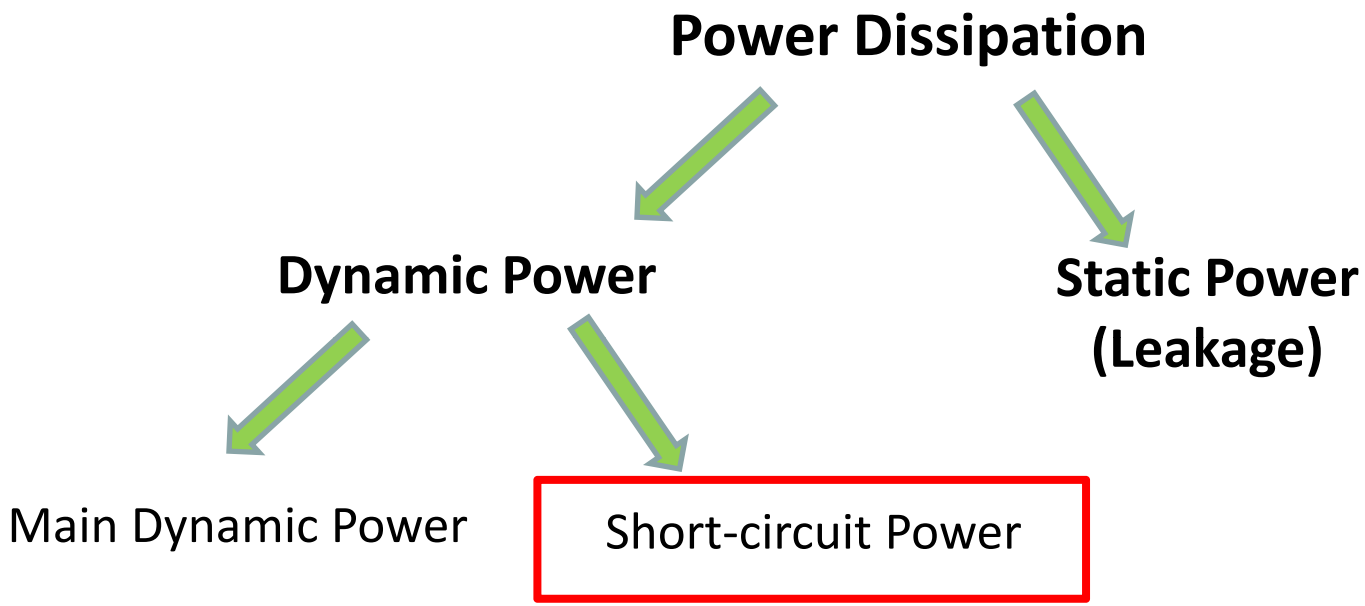
Understanding Tradeoffs

- Which design is the "best" (fastest, coolest, both) ?
  - Constant EDP's are the straight lines in the graph



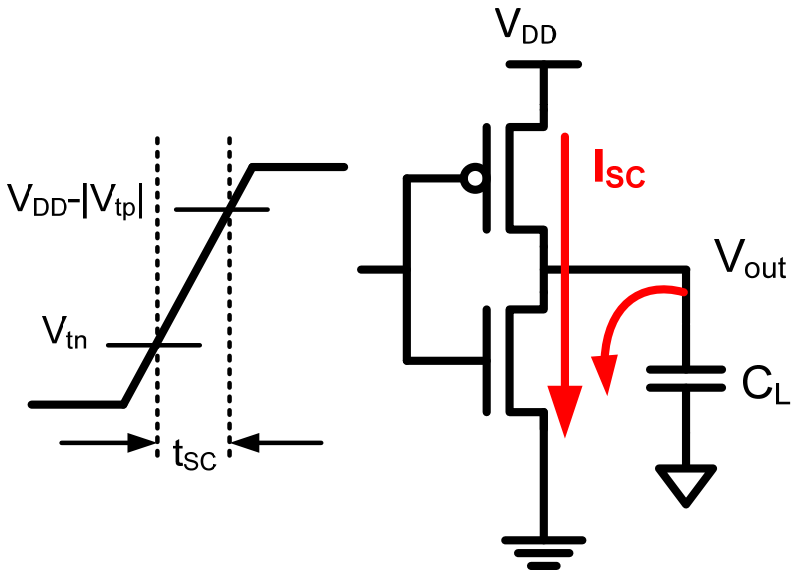
© M. Shabany, ASIC/FPGA Chip Design

Power Dissipation

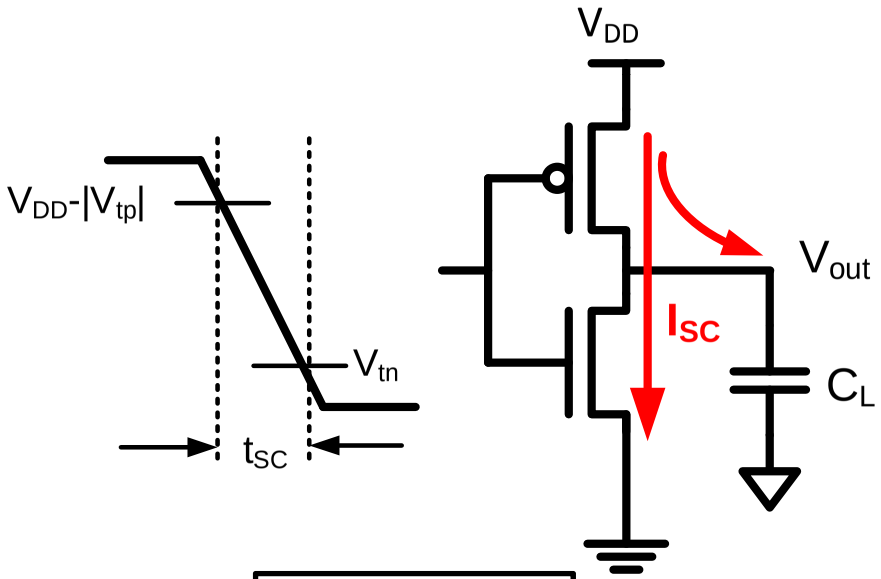


Short Circuit Power Consumption

❑ Finite slope of the input signal causes a direct current path between V_{DD} and GND for a short period of time during switching when both the NMOS and PMOS transistors are conducting.



Rise Time



Fall Time

Short Circuit Currents Determinates

$$t_{sc} = t_{sc}^{r} + t_{sc}^{f}$$

$$I = C\frac{dv}{dt} \Rightarrow t_{sc} I_{sc,avg} = C_{sc} V_{DD}$$

$$I_{sc} = \frac{t_{sc} \cdot I_{sc,avg}}{T}$$

Peak and duration of I_{sc} both increase as the input slope decreases

$$P_{sc} = I_{sc} V_{DD} = t_{sc} I_{sc,avg} V_{DD} f$$

$$\Rightarrow P_{sc} = C_{sc} V_{DD}^2 f = \alpha_{sc} C_L V_{DD}^2 f$$

$$\Rightarrow E_{sc} = C_{sc} V_{DD}^2 = \alpha_{sc} C_L V_{DD}^2$$

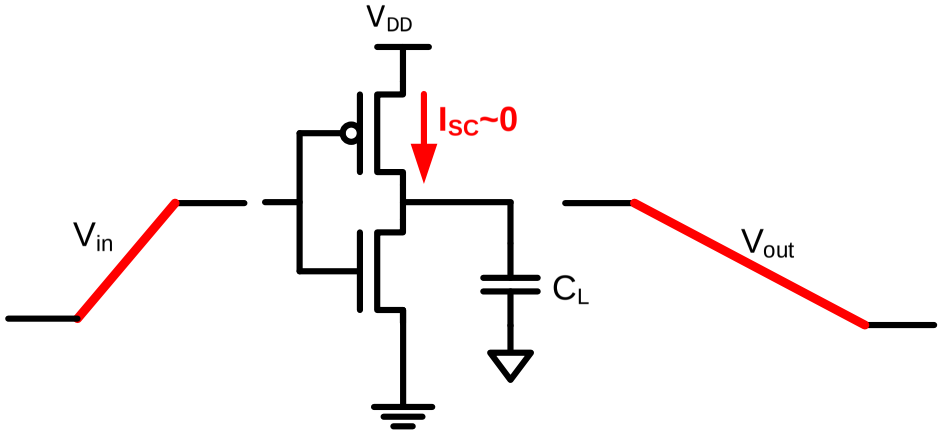
Short Circuit Currents Determinates

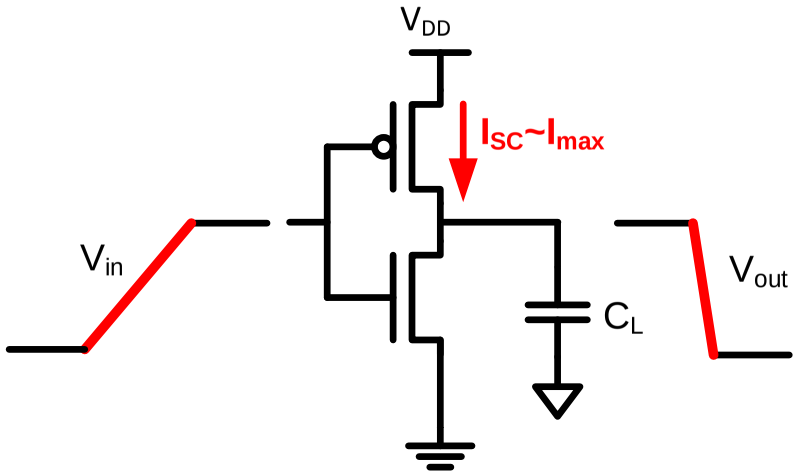
$$P_{sc} = \alpha_{sc} C_L V_{DD}^2 f$$

$$E_{sc} = \alpha_{sc} C_L V_{DD}^2$$

- I_{peak} determined by
  - Saturation current of the P and N transistors, which depend on their sizes, process technology, temperature, etc.
  - Strong function of the ratio between input and output slopes
    - Function of C_L

Impact of C_L on I_{sc}





Large capacitive load

⇩

Output fall time significantly larger than input rise time.

Small capacitive load

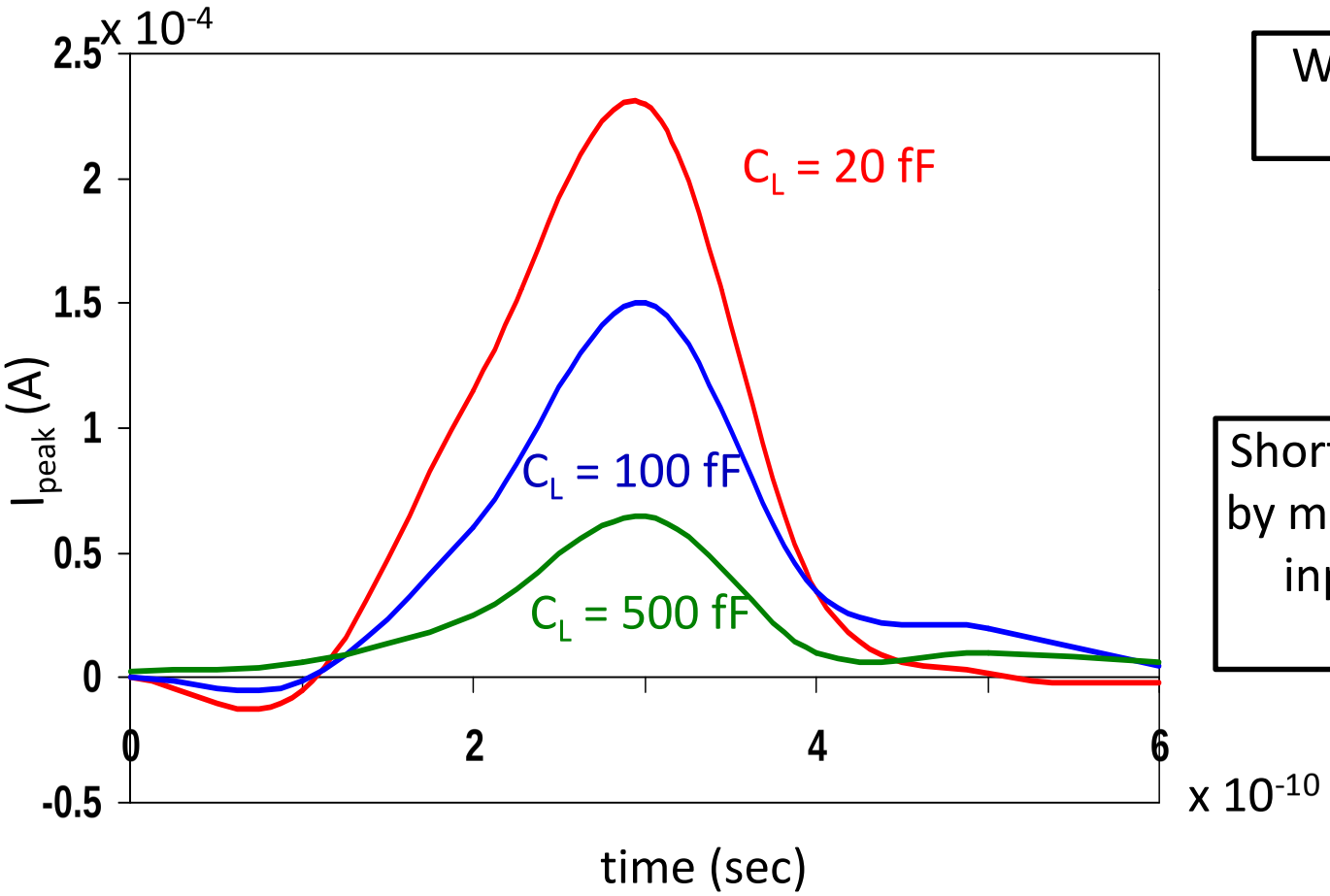
⇩

Output fall time substantially smaller than the input rise time.

❑ As the source-drain voltage of the PMOS is approximately 0 during transition, the device shuts off without ever delivering any current, so I_{sc} is close to zero.

❑ Drain-source voltage of PMOS equals VDD for most of the transition period, giving maximum I_{sc}

I_{peak} as a Function of C_L

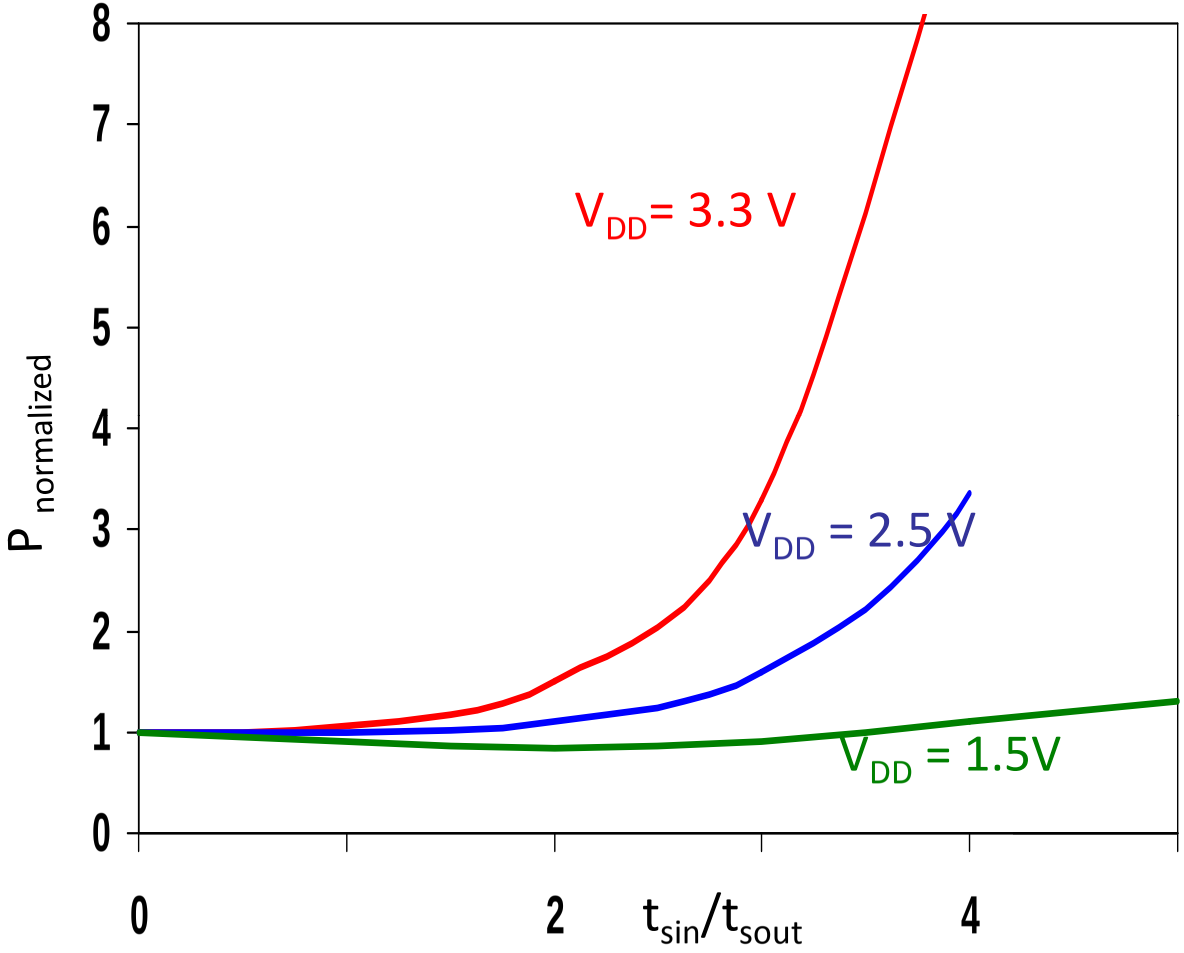


When load capacitance is small, I_{peak} is large.

Short circuit dissipation is minimized by matching the rise/fall times of the input and output signals - slope engineering.

© M. Shabany, ASIC/FPGA Chip Design

P_{sc} as a Function of Rise/Fall Times



When load capacitance is small ($t_{sin}/t_{sout} > 2$ for $V_{DD} > 2V$) the power is dominated by P_{sc}

If $V_{DD} < V_{Tn} + |V_{Tp}|$ then P_{sc} is eliminated since both devices are never ON at the same time.

- For large capacitance values, all the power dissipation is devoted to charging and discharging the load capacitance.
- When the rise/fall times of inputs and outputs are equalized, most power dissipation is associated with dynamic power and only a minor fraction (<10%) is devoted to P_{sc}.

Dynamic Power Example

- 1 billion transistor chip
  - 50M logic transistors
    - Average width: 12 L_{min}
    - Activity factor = 0.1
  - 950M memory transistors
    - Average width: 4 L_{min}
    - Activity factor = 0.02
  - 1.0 V 25 nm process
  - C = 1 fF/mm (gate) + 0.8 fF/mm (diffusion)
- Estimate dynamic power consumption @ 1 GHz. Neglect wire capacitance and short-circuit current.

Dynamic Power Example

$$C_{\text{logic}} = \left(50 \times 10^6\right)\left(12\lambda\right)\left(0.025\mu m / \lambda\right)\left(1.8\, fF / \mu m\right) = 27 \text{ nF}$$

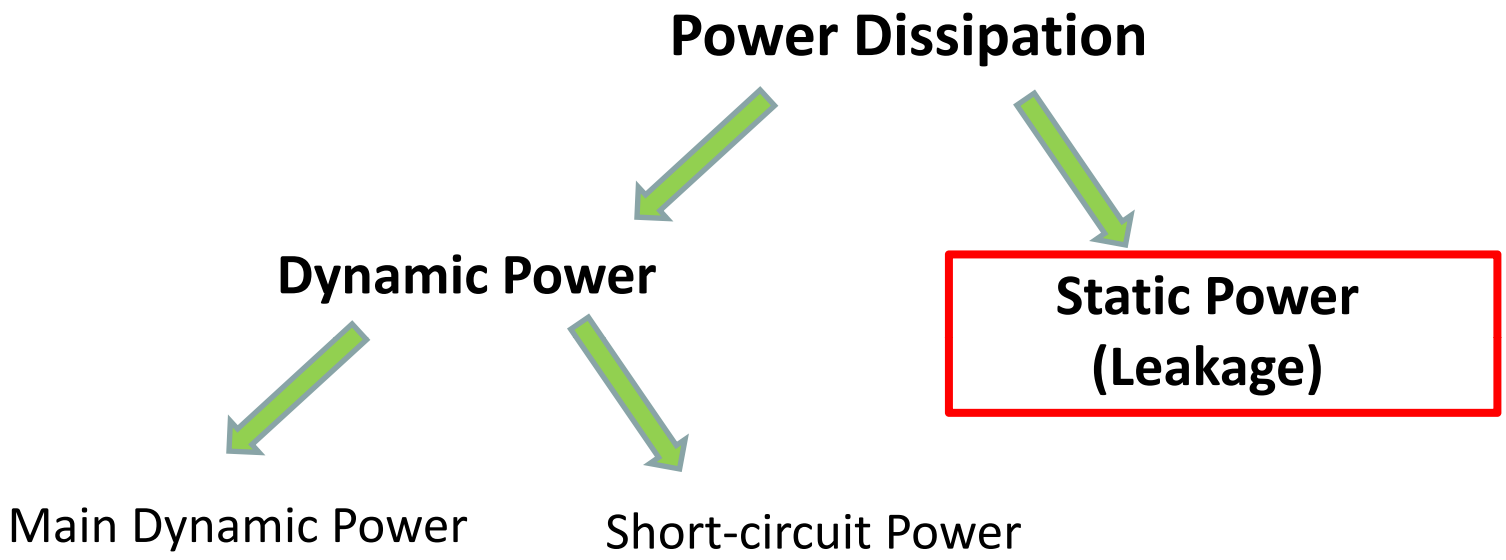
$$C_{\text{mem}} = \left(950 \times 10^6\right)\left(4\lambda\right)\left(0.025\mu m / \lambda\right)\left(1.8\, fF / \mu m\right) = 171 \text{ nF}$$

$$P_{\text{dynamic}} = \left[0.1C_{\text{logic}} + 0.02C_{\text{mem}}\right]\left(1.0\right)^2\left(1.0 \text{ GHz}\right) = 6.1 \text{ W}$$

Outline

- Introduction
- Dynamic Power Dissipation
- Static Power Dissipation

Power Dissipation



Power Dissipation: Static Power

- Non-ideal Effects: small leakage current flows through the OFF transistor (I_{static})

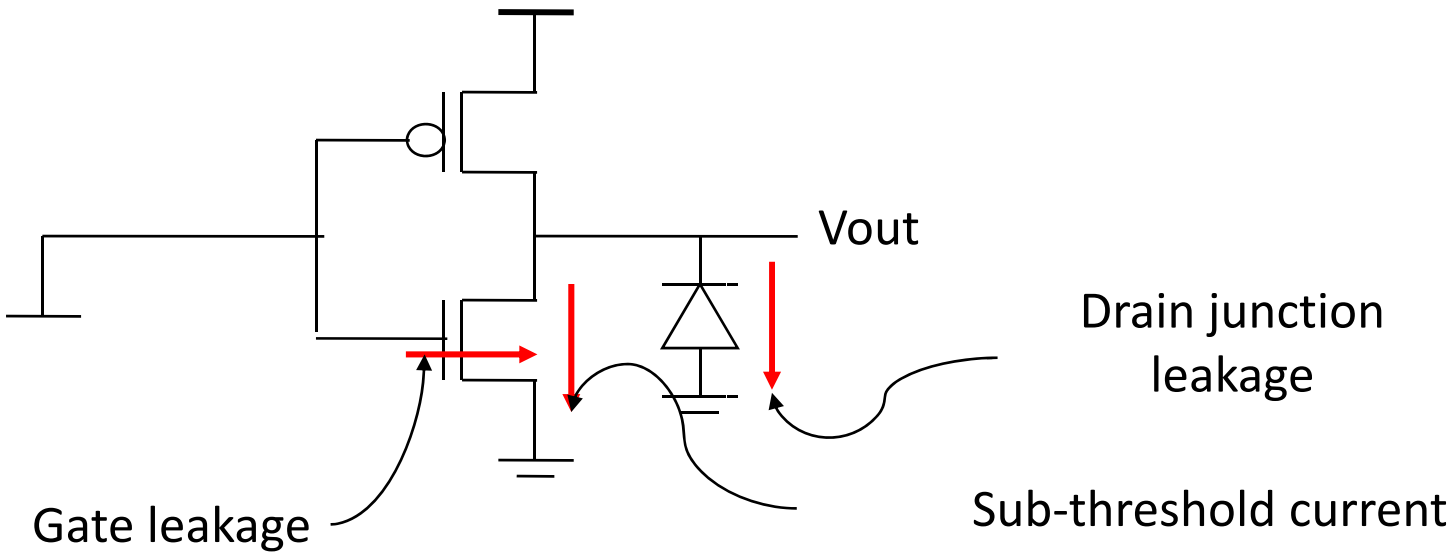
$$P_{static} = \frac{1}{T}\int_0^T i_{static} V_{DD} dt = I_{static} V_{DD}$$

- Sources of Leakage:
  - **Sub-threshold Conduction**: Exponentially increases as V_T scales down
  - **Tunneling through the gate oxide**: Exponentially increases as oxide thickness decreases
    - (Important for 130nm and smaller technologies)
  - **Leakage through reverse-biased diodes**

Static power dissipation an issue in deep sub-micron processes

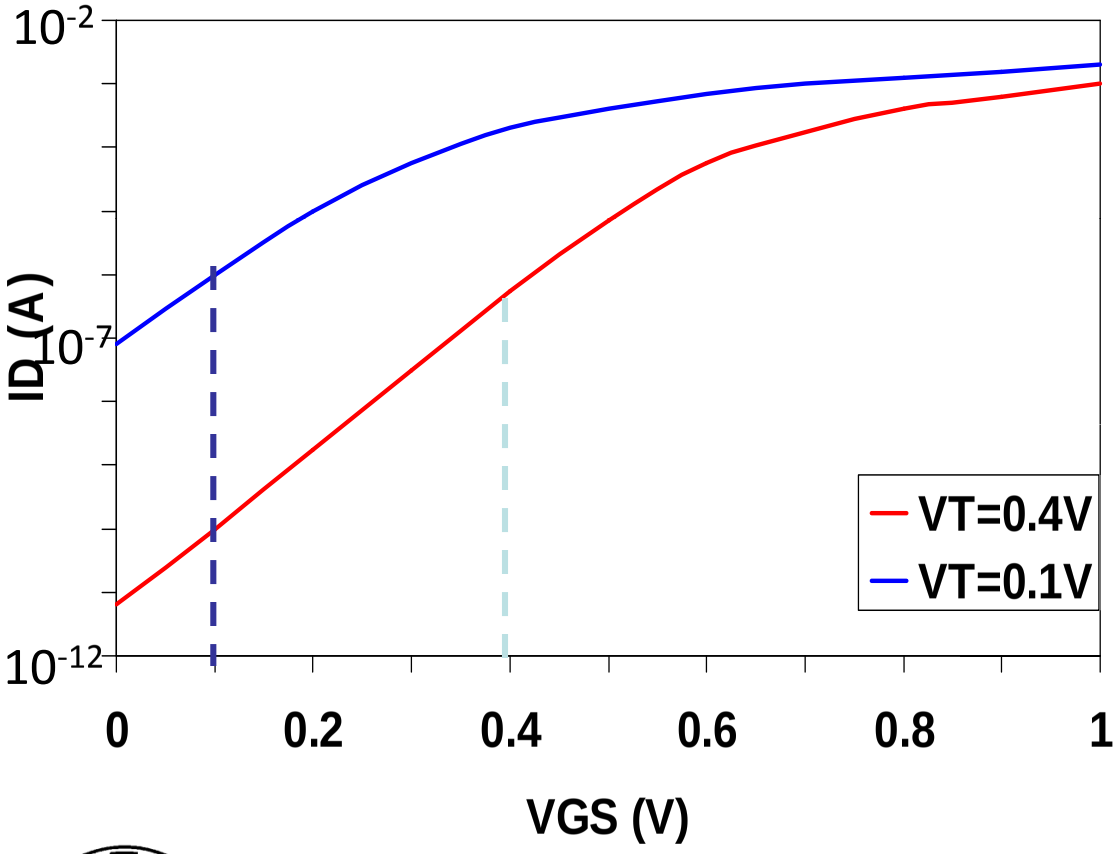
Power Dissipation: Static (Leakage) Power

- Sub-threshold current is the dominant factor.
- All increase exponentially with temperature!



Leakage as a Function of V_T

- Continued scaling of supply voltage and the subsequent scaling of threshold voltage will make sub-threshold conduction a dominate component of power dissipation.



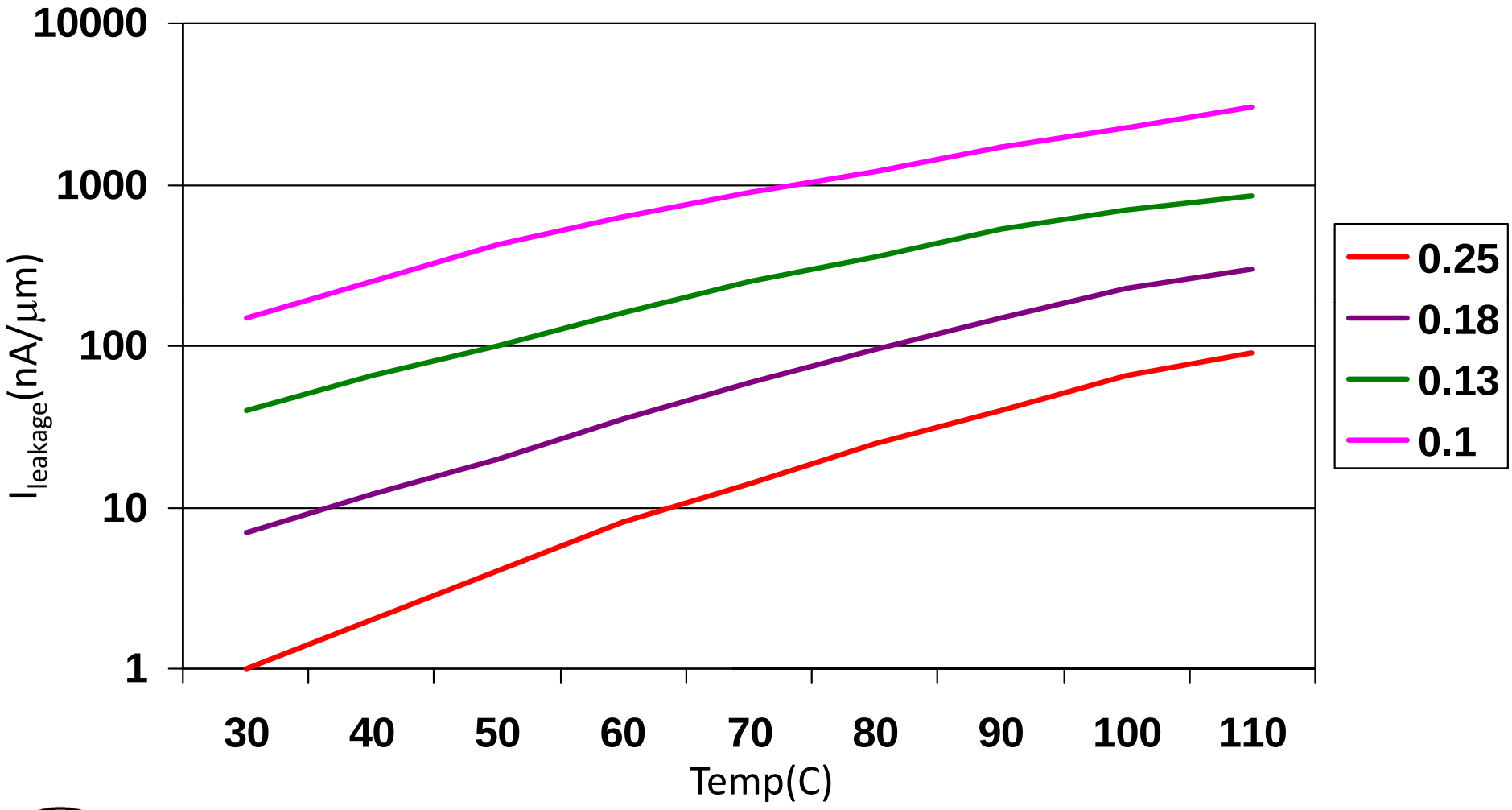
An 90mV/decade V_T roll-off - so each 255mV increase in V_T gives 3 orders of magnitude reduction in leakage (but adversely affects performance)

TSMC Processes Leakage and V_T

	CL018 G	CL018 LP	CL018 ULP	CL018 HS	CL015 HS	CL013 HS
V_{dd}	1.8 V	1.8 V	1.8 V	2 V	1.5 V	1.2 V
T_{ox} (effective)	42 Å	42 Å	42 Å	42 Å	29 Å	24 Å
L_{gate}	0.16 μm	0.16 μm	0.18 μm	0.13 μm	0.11 μm	0.08 μm
I_{DSat} (n/p) (μA/μm)	600/260	500/180	320/130	780/360	860/370	920/400
I_{off} (leakage) (ρA/μm)	20	1.60	0.15	300	1,800	13,000
V_{Tn}	0.42 V	0.63 V	0.73 V	0.40 V	0.29 V	0.25 V
FET Perf. (GHz)	30	22	14	43	52	80

(G: generic, LP: low power, ULP: ultra low power, HS: high speed)

Exponential Increase in Leakage Currents



© M. Shabany, ASIC/FPGA Chip Design

Leakage Control

- Leakage and delay trade off
  - Aim for low leakage in sleep and low delay in active mode
- To reduce leakage:
  - Increase V_t: *multiple V_t*
    - Use low V_t only in critical circuits
  - Increase V_s: *stack effect*
    - *Input vector control* in sleep
  - Decrease V_b
    - *Reverse body bias* in sleep
    - Or forward body bias in active mode

Gate Leakage

- Extremely strong function of t_{ox} and V_{gs}
  - Negligible for older processes
  - Approaches sub-threshold leakage at 65 nm and below in some processes
- An order of magnitude less for PMOS than NMOS
- Control leakage in the process using $t_{ox} > 10.5$ Å
  - High-k gate dielectrics help
  - Some processes provide multiple t_{ox}
    - e.g. thicker oxide for 3.3 V I/O transistors
- Control leakage in circuits by limiting V_{DD}

Static Power Example

- Revisit power estimation for 1 billion transistor chip
- Estimate static power consumption
  - Subthreshold leakage
    - Normal V_t: 100 nA/mm
    - High V_t: 10 nA/mm
    - High Vt used in all memories and in 95% of logic gates
  - Gate leakage 5 nA/mm
  - Junction leakage negligible

Solution

$$W_{\text{normal-V}_t} = \left(50\times10^6\right)\left(12\lambda\right)\left(0.025\mu\text{m}/\lambda\right)\left(0.05\right) = 0.75\times10^6\ \mu\text{m}$$

$$W_{\text{high-V}_t} = \left[\left(50\times10^6\right)\left(12\lambda\right)\left(0.95\right)+\left(950\times10^6\right)\left(4\lambda\right)\right]\left(0.025\mu\text{m}/\lambda\right) = 109.25\times10^6\ \mu\text{m}$$

$$I_{sub} = \left[W_{\text{normal-V}_t}\times100\ \text{nA}/\mu\text{m}+W_{\text{high-V}_t}\times10\ \text{nA}/\mu\text{m}\right]/2 = 584\ \text{mA}$$

$$I_{gate} = \left[\left(W_{\text{normal-V}_t}+W_{\text{high-V}_t}\right)\times5\ \text{nA}/\mu\text{m}\right]/2 = 275\ \text{mA}$$

$$\text{P}_{static} = \left(584\ \text{mA}+275\ \text{mA}\right)\left(1.0\ \text{V}\right) = 859\ \text{mW}$$

Review: Designing Fast CMOS Gates

- Transistor sizing
- Progressive transistor sizing
  - MOS closest to the output is smallest of series MOS transistors
- Transistor ordering
  - put latest arriving signal closest to the output
- Logic structure reordering
  - replace large fan-in gates with smaller fan-in gate network
- Apply “logical effort”
- Buffer (inverter) insertion
  - separate large fan-in from large C_L with buffers
  - uses buffers so there are no more than four TGs in series